Java WebDeveloper 2017

Treinamento Sábado - Manhã

Início: 17/06/2017 – Término: 20/01/2018

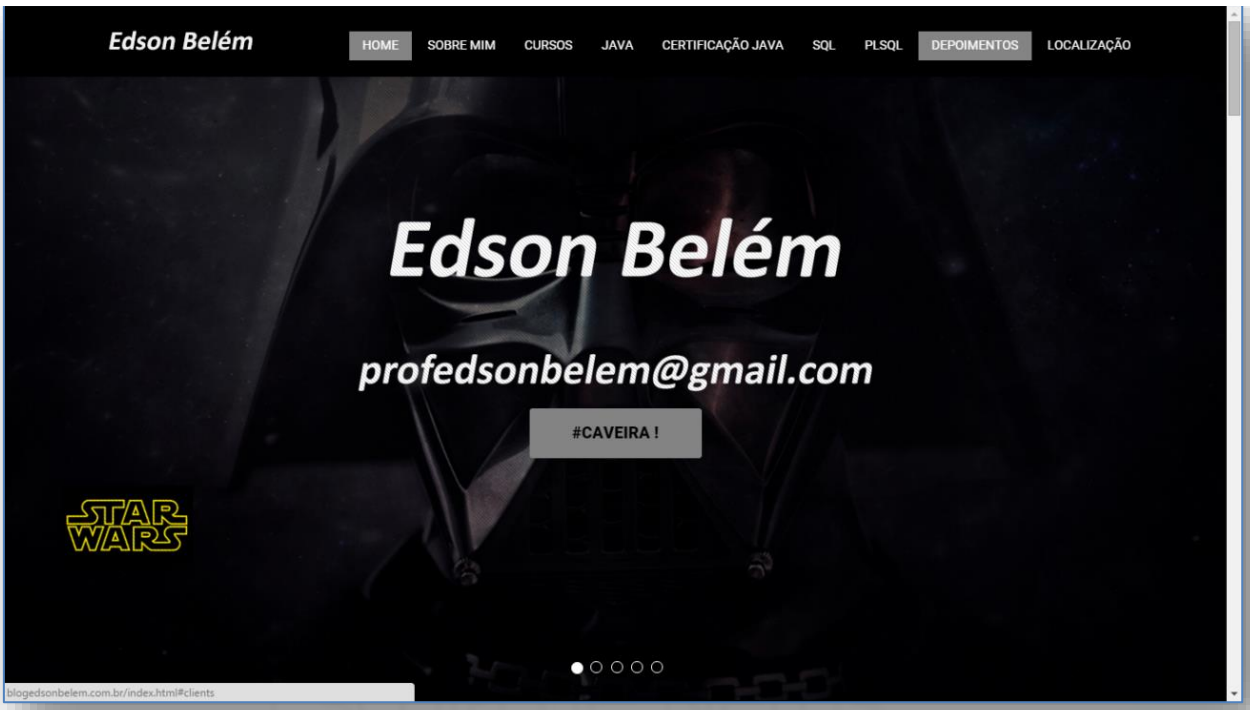
Prof. Edson Belém

profedsonbelem@gmail.com
Celular: 98199-0108

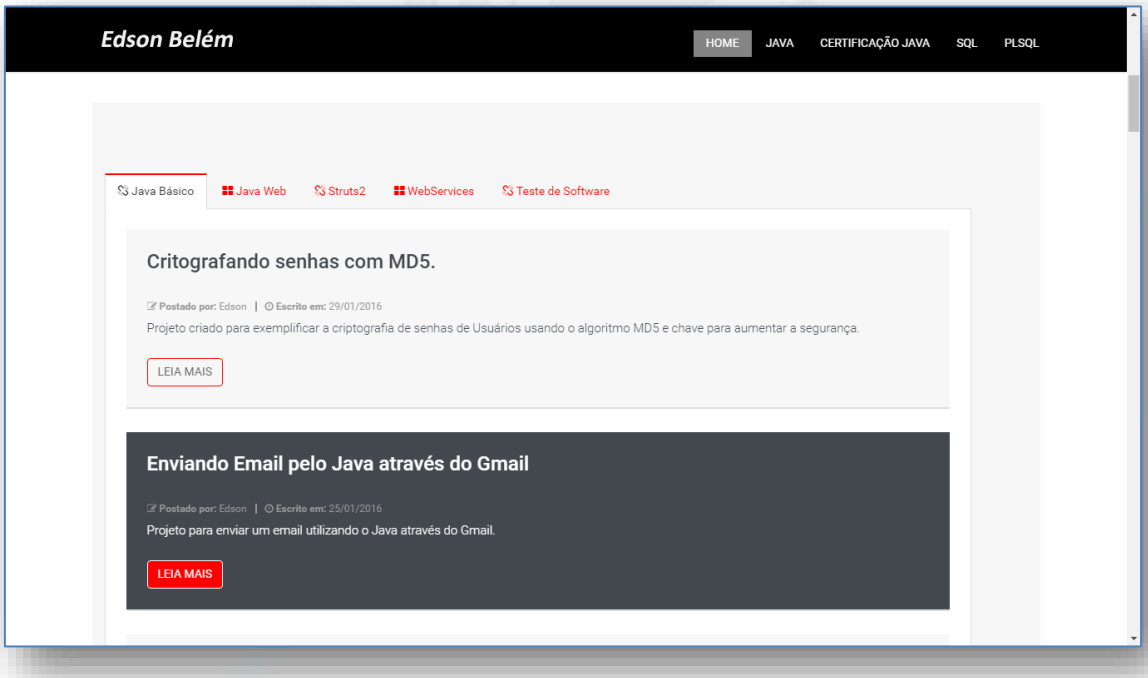
profedsonbelem@gmail.com / face
98199-0108 / zap

lucianamedeiros.coti@gmail.com
98201-2525

www.blogedsonbelem.com.br

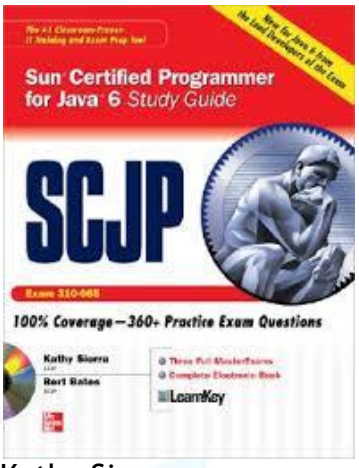


http://www.blogedsonbelem.com.br/artigos/java/java.html



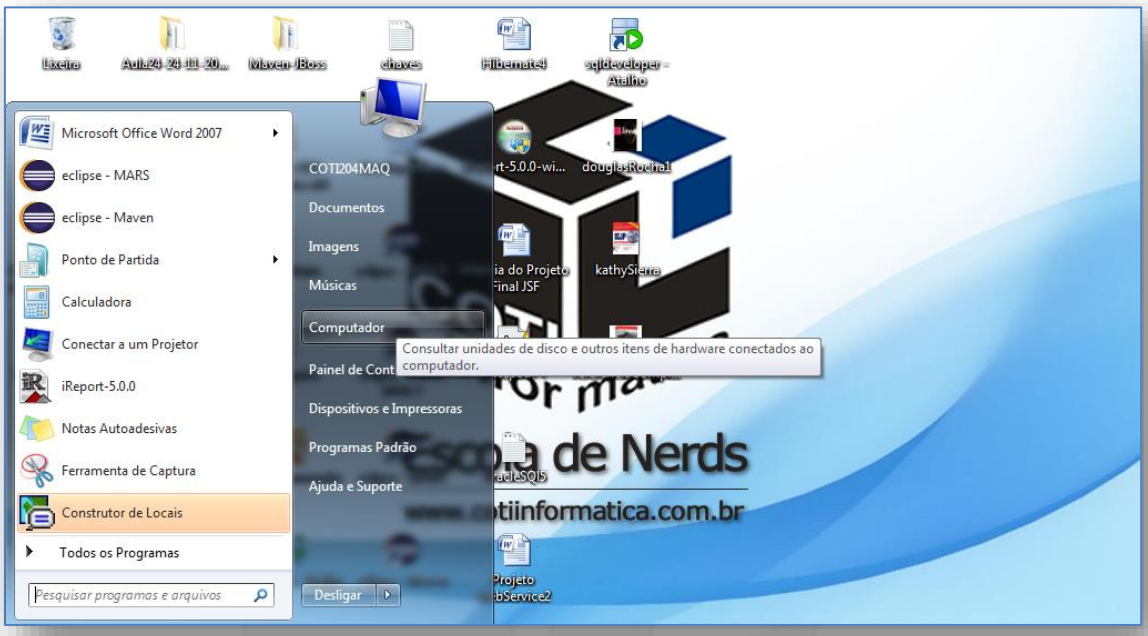
Indicação de livros:

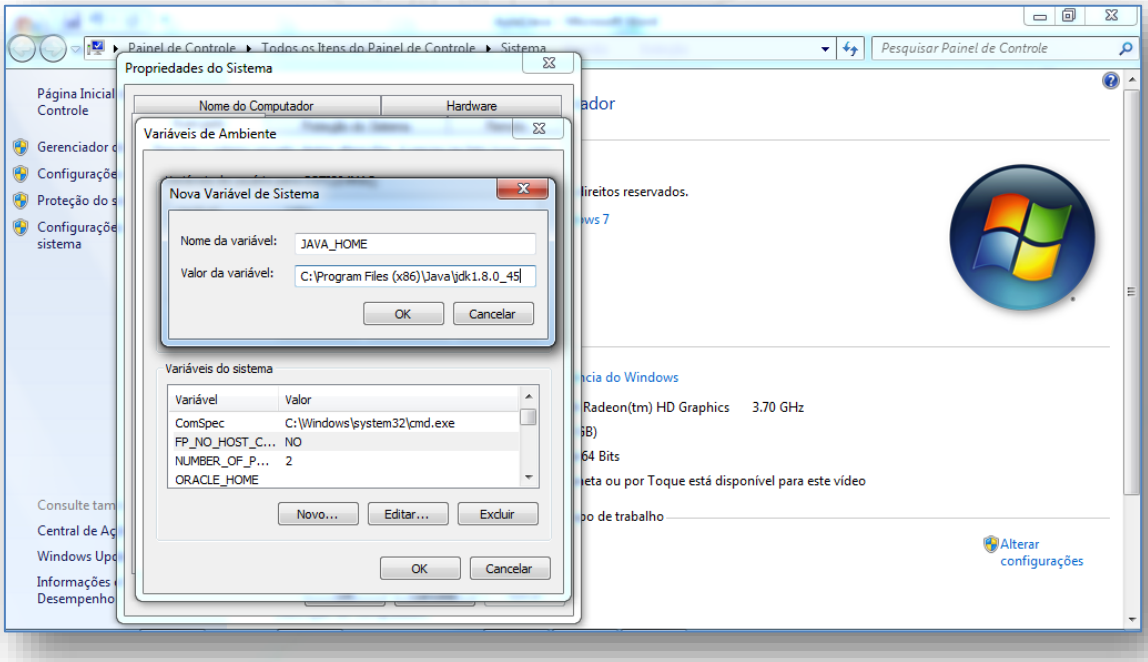
Kathy Sierra

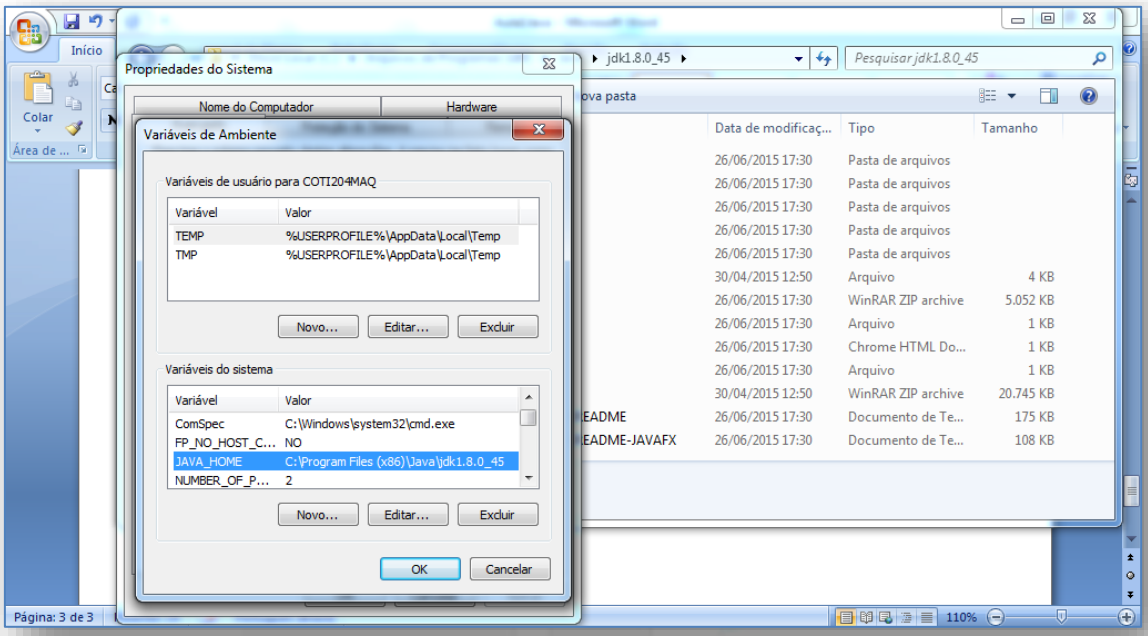


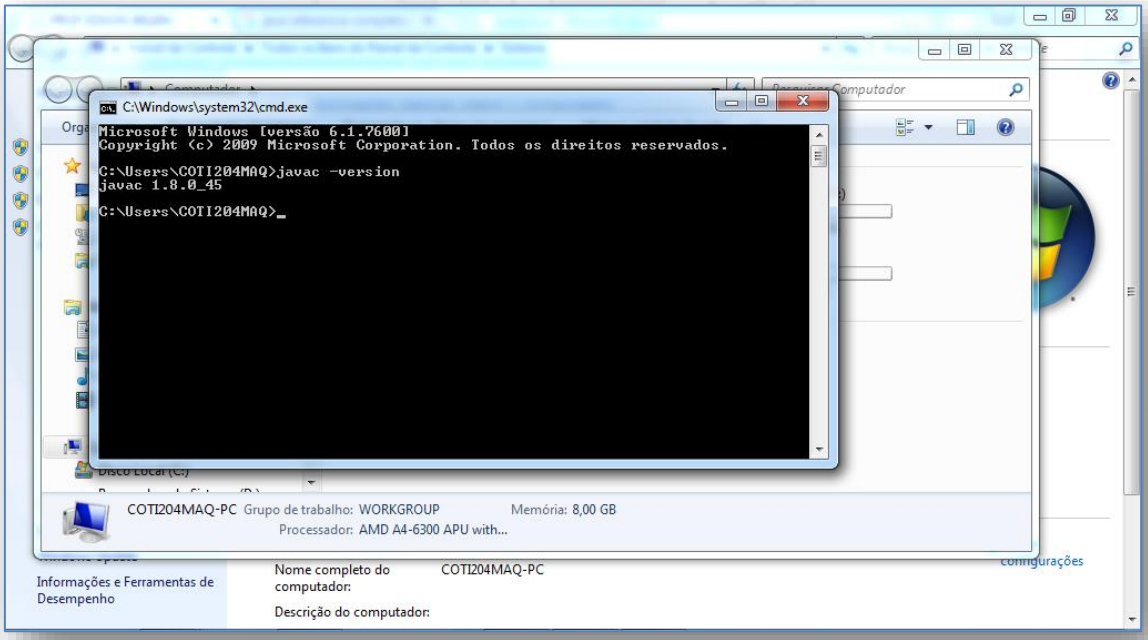
Kathy Sierra

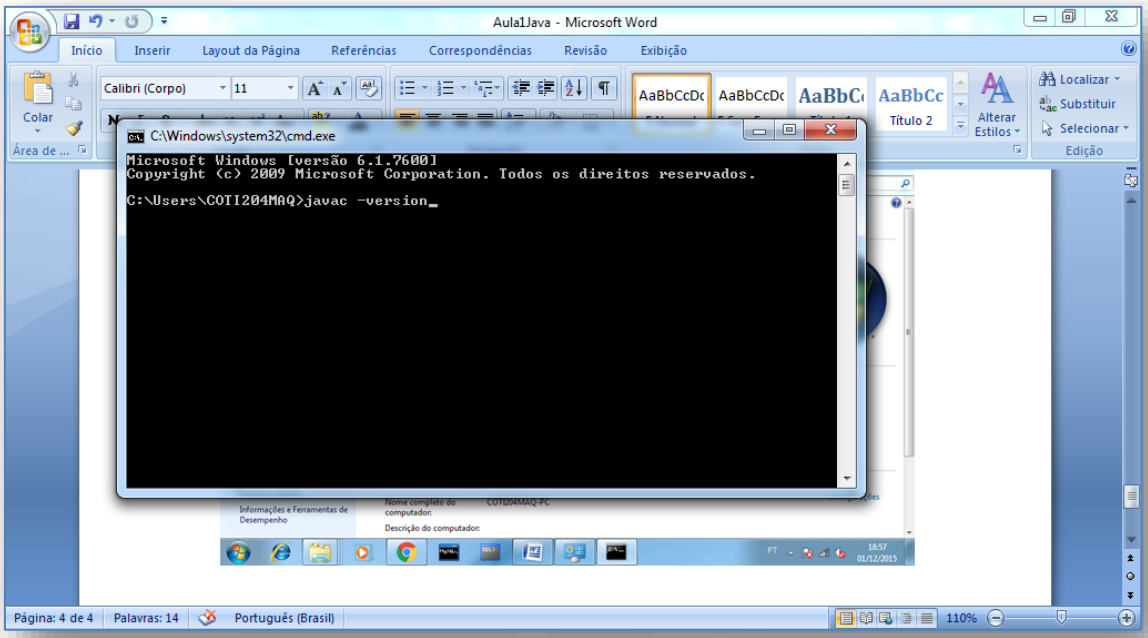
Configuração do JDK8:



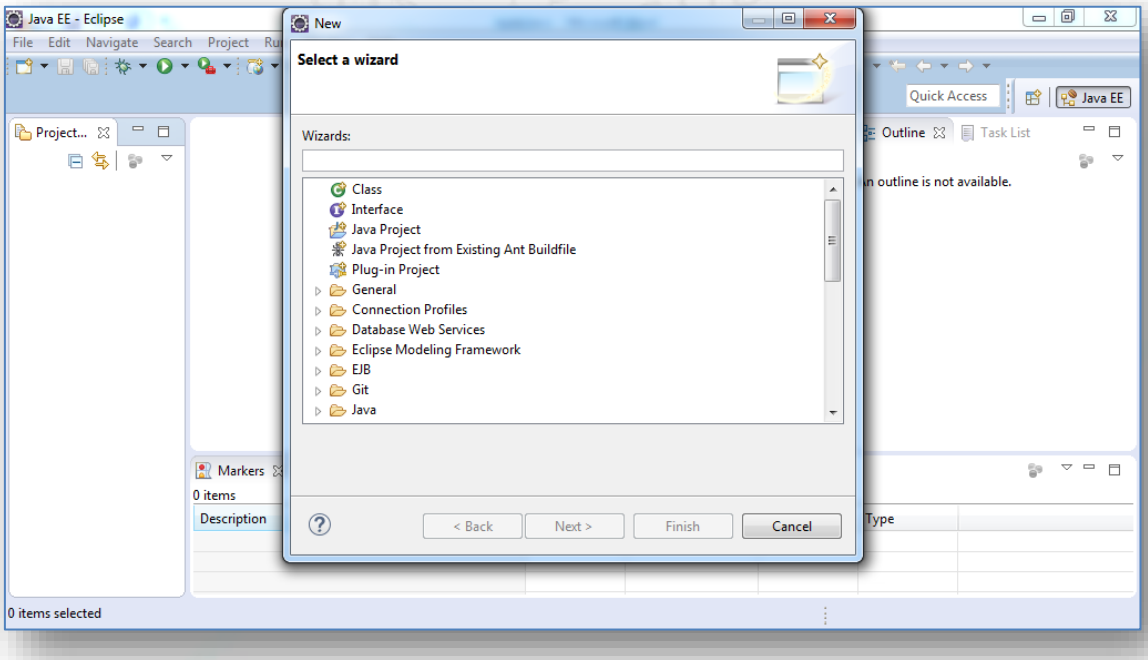


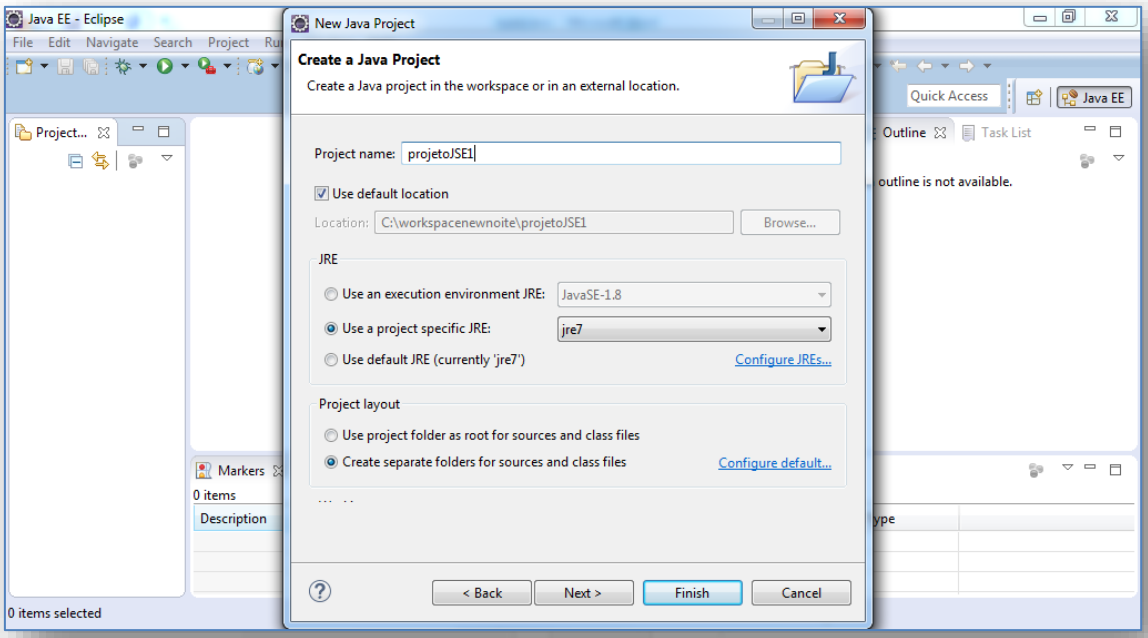

Propriedades do Sistema
Nome do Computador
Hardware
Variáveis de Ambiente
Variáveis de usuário para COTI204MAQ
Variável
Valor
TEMP
%USERPROFILE%\AppData\Local\Temp
TMP
%USERPROFILE%\AppData\Local\Temp
Novo...
Editar...
Excluir
Variáveis do sistema
ComSpec
C:\Windows\system32\cmd.exe
FP_NO_HOST_C...
NO
JAVA_HOME
C:\Program Files (x86)\Java\jdk1.8.0_45
NUMBER_OF_P...
2
OK
Cancelar
jdk1.8.0_45
Pesquisar jdk1.8.0_45
Data de modificaç...
Tipo
Tamanho
Página: 3 de 3
110%

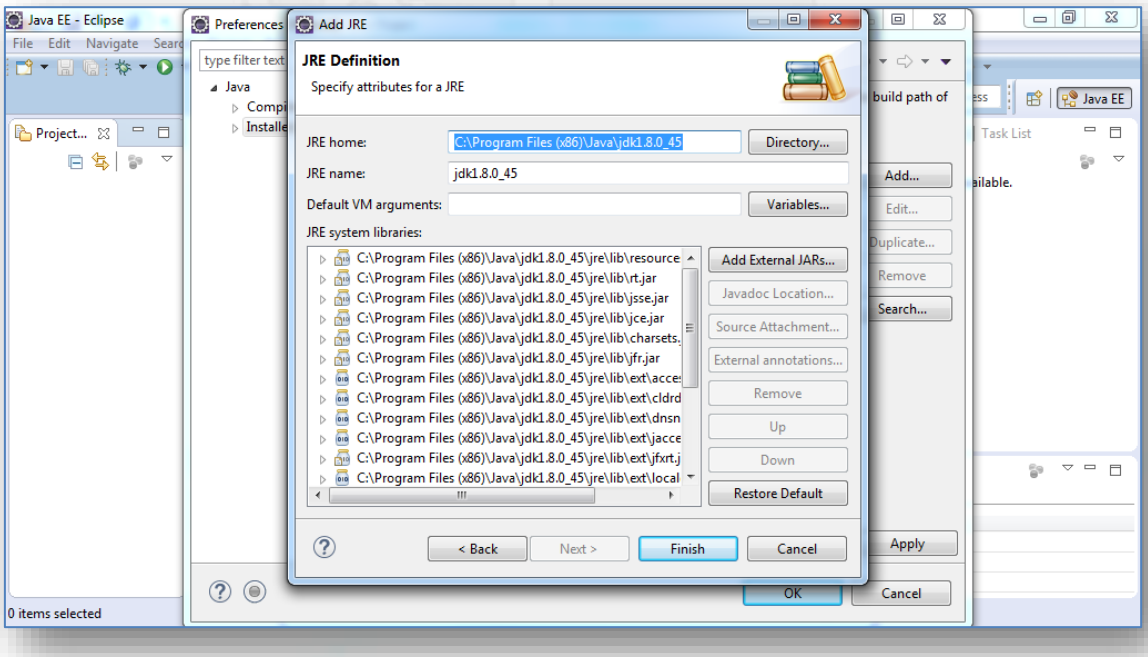

C:\Windows\system32\cmd.exe
Microsoft Windows [versão 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. Todos os direitos reservados.
C:\Users\COTI204MAQ>javac -version
javac 1.8.0_45
C:\Users\COTI204MAQ>
COTI204MAQ-PC Grupo de trabalho: WORKGROUP
Memória: 8,00 GB
Processador: AMD A4-6300 APU with...
Informações e Ferramentas de Desempenho
Nome completo do computador:
COTI204MAQ-PC
Descrição do computador:

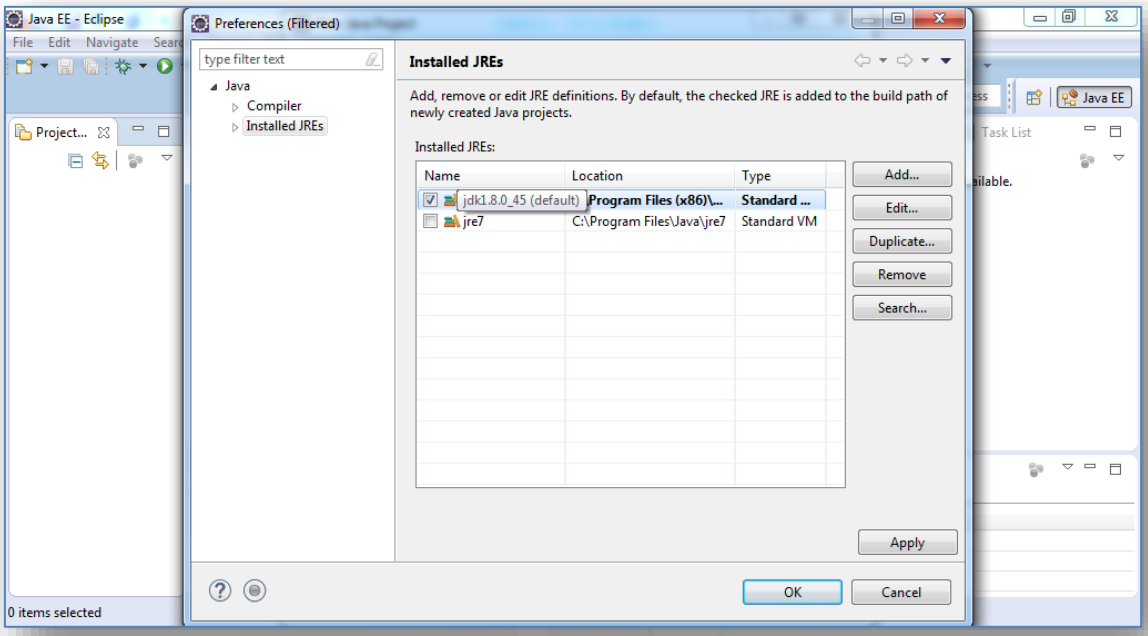


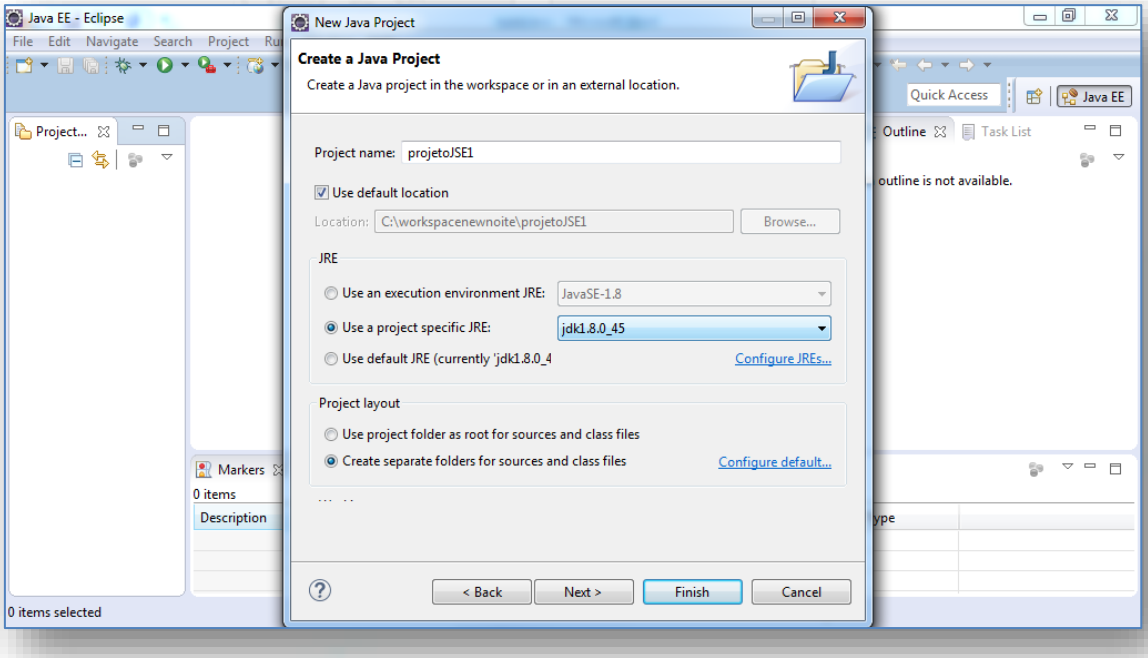
Eclipse
file /new / other

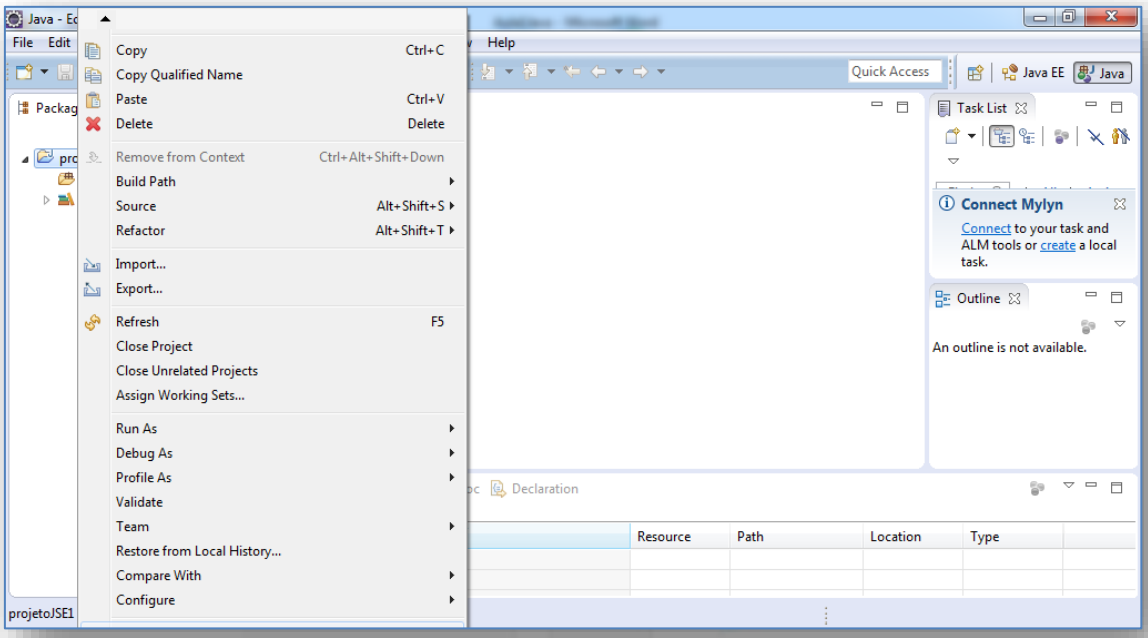


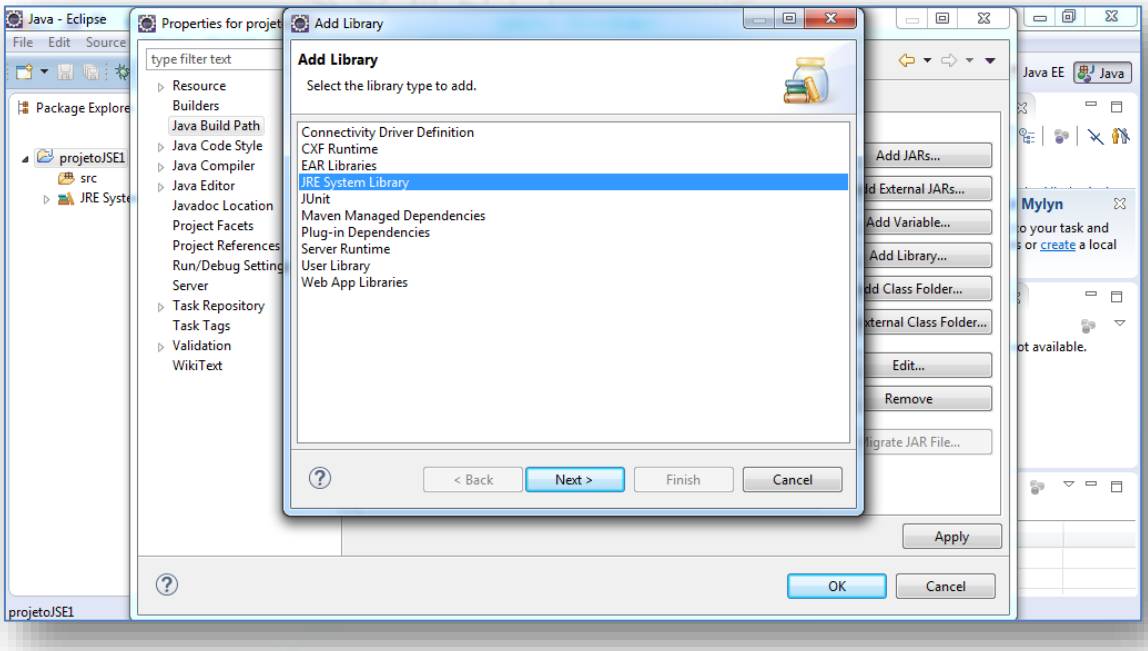

Java EE - Eclipse
New Java Project
Create a Java Project
Create a Java project in the workspace or in an external location.
Project name: projetoJSE1
Use default location
Location: C:\workspacenewnoite\projetoJSE1
Browse...
JRE
Use an execution environment JRE: JavaSE-1.8
Use a project specific JRE: jre7
Use default JRE (currently 'jre7')
Configure JREs...
Project layout
Use project folder as root for sources and class files
Create separate folders for sources and class files
Configure default...
< Back
Next >
Finish
Cancel


Java EE - Eclipse
Preferences
Add JRE
JRE Definition
Specify attributes for a JRE
JRE home: C:\Program Files (x86)\Java\jdk1.8.0_45
Directory...
JRE name: jdk1.8.0_45
Default VM arguments:
Variables...
JRE system libraries:
Add External JARs...
Javadoc Location...
Source Attachment...
External annotations...
Remove
Up
Down
Restore Default
< Back
Next >
Finish
Cancel
OK
Cancel
Apply

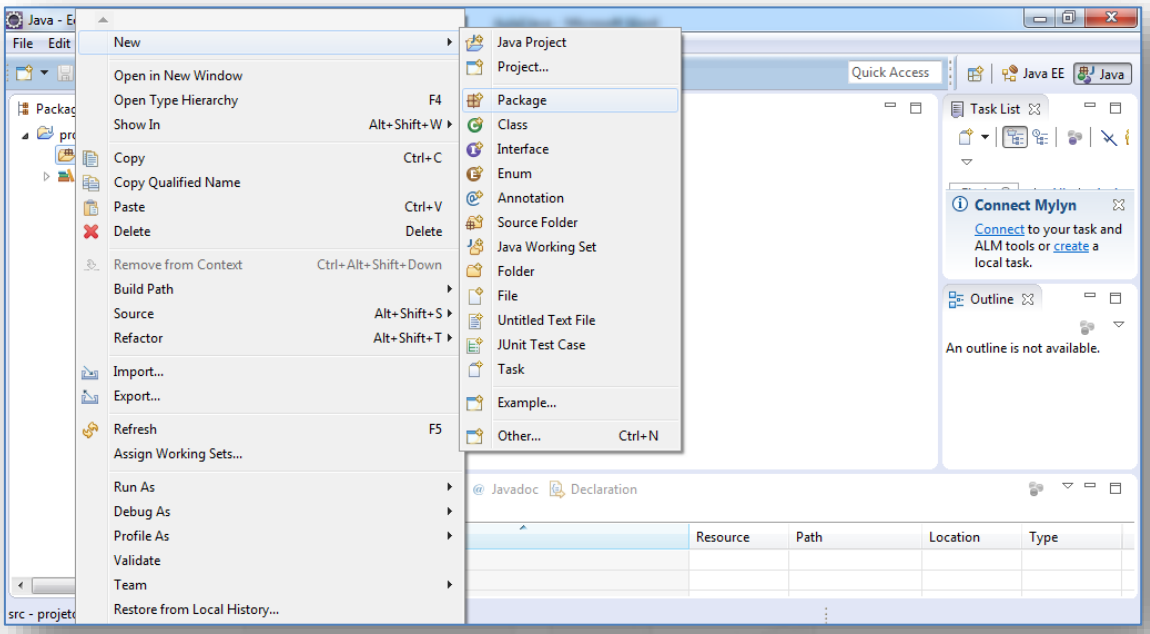

Java EE - Eclipse
Preferences (Filtered)
type filter text
Java
Compiler
Installed JREs
Installed JREs
Add, remove or edit JRE definitions. By default, the checked JRE is added to the build path of newly created Java projects.
Installed JREs:
Name
Location
Type
jdk1.8.0_45 (default)
Program Files (x86)\...
Standard ...
jre7
C:\Program Files\Java\jre7
Standard VM
Add...
Edit...
Duplicate...
Remove
Search...
Apply
OK
Cancel
0 items selected

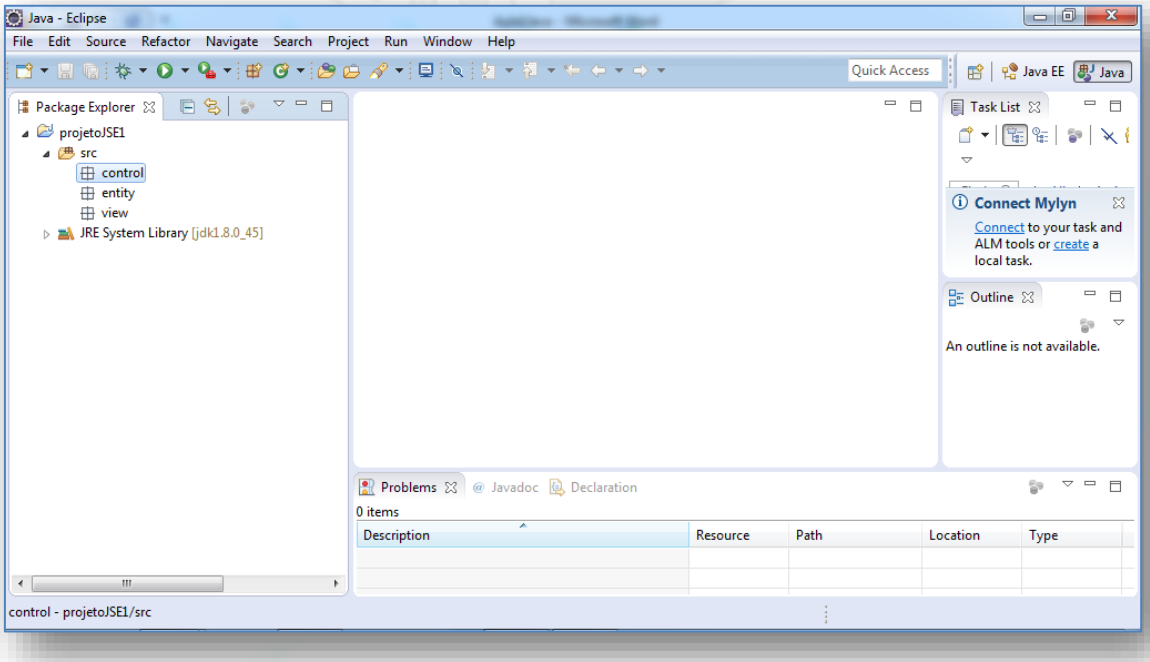

Java EE - Eclipse
New Java Project
Create a Java Project
Create a Java project in the workspace or in an external location.
Project name: projetoJSE1
Use default location
Location: C:\workspacenewnoite\projetoJSE1
Browse...
JRE
Use an execution environment JRE: JavaSE-1.8
Use a project specific JRE: jdk1.8.0_45
Use default JRE (currently 'jdk1.8.0_4
Configure JREs...
Project layout
Use project folder as root for sources and class files
Create separate folders for sources and class files
Configure default...
< Back
Next >
Finish
Cancel
Markers
0 items
Description
0 items selected
Outline
Task List
outline is not available.

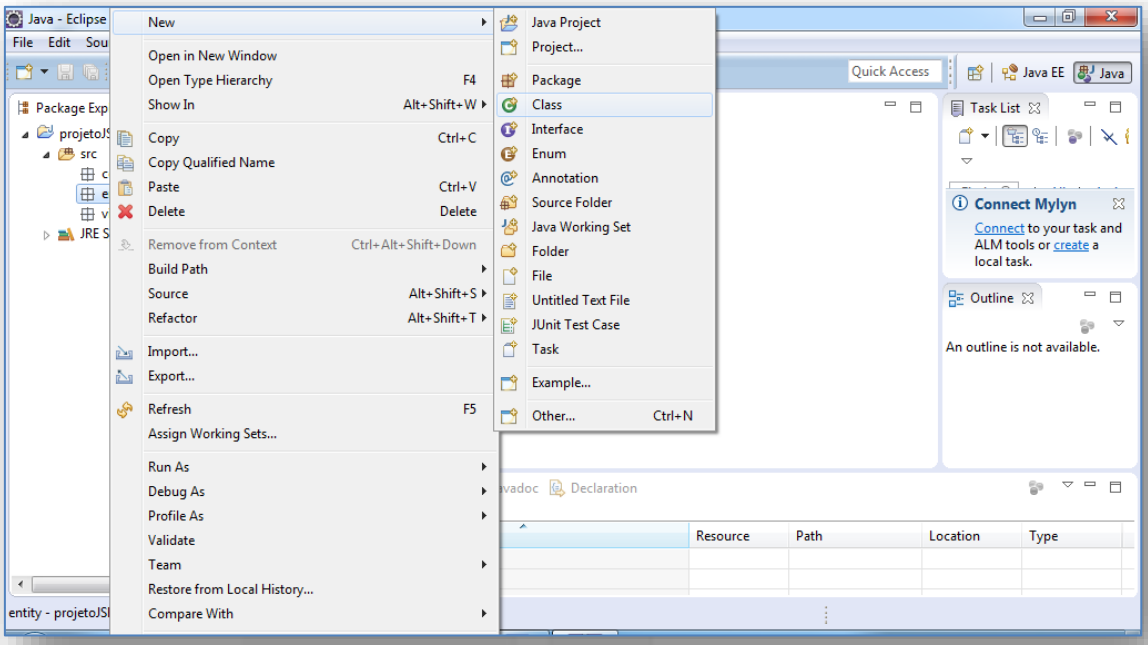

Copy Ctrl+C
Copy Qualified Name
Paste Ctrl+V
Delete Delete
Remove from Context Ctrl+Alt+Shift+Down
Build Path
Source Alt+Shift+S
Refactor Alt+Shift+T
Import...
Export...
Refresh F5
Close Project
Close Unrelated Projects
Assign Working Sets...
Run As
Debug As
Profile As
Validate
Team
Restore from Local History...
Compare With
Configure
Quick Access
Java EE
Java
Task List
Connect Mylyn
Connect to your task and ALM tools or create a local task.
Outline
An outline is not available.
Declaration
Resource
Path
Location
Type
projetoJSE1


Java - Eclipse
Properties for projet
Add Library
Add Library
Select the library type to add.
Connectivity Driver Definition
CXF Runtime
EAR Libraries
JRE System Library
JUnit
Maven Managed Dependencies
Plug-in Dependencies
Server Runtime
User Library
Web App Libraries
< Back
Next >
Finish
Cancel
Resource
Builders
Java Build Path
Java Code Style
Java Compiler
Java Editor
Javadoc Location
Project Facets
Project References
Run/Debug Setting
Server
Task Repository
Task Tags
Validation
WikiText
Add JARs...
Add External JARs...
Add Variable...
Add Library...
Add Class Folder...
External Class Folder...
Edit...
Remove
Migrate JAR File...
Apply
OK
Cancel
projetoJSE1

Criando um projeto...

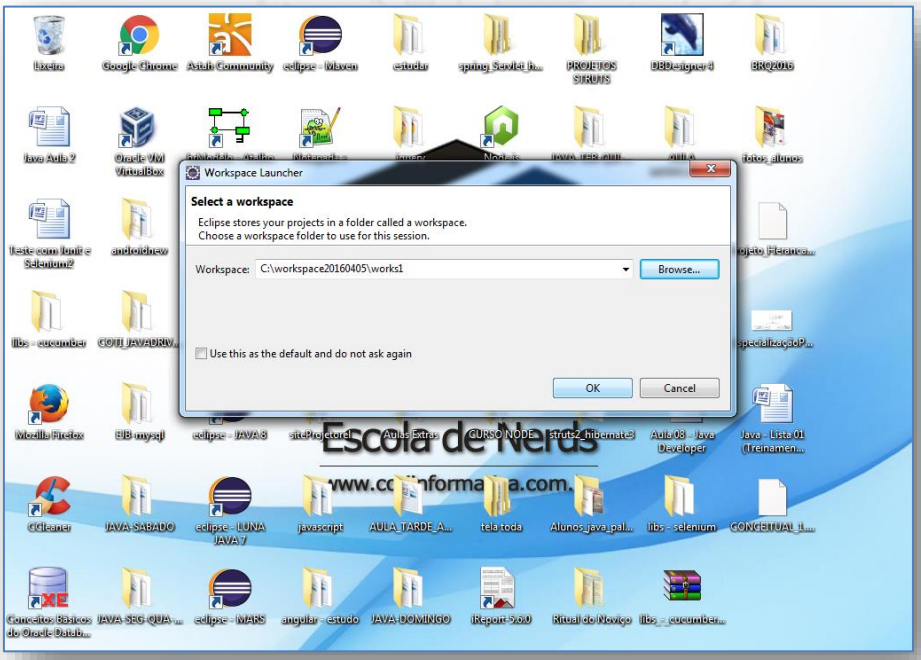






Workspace

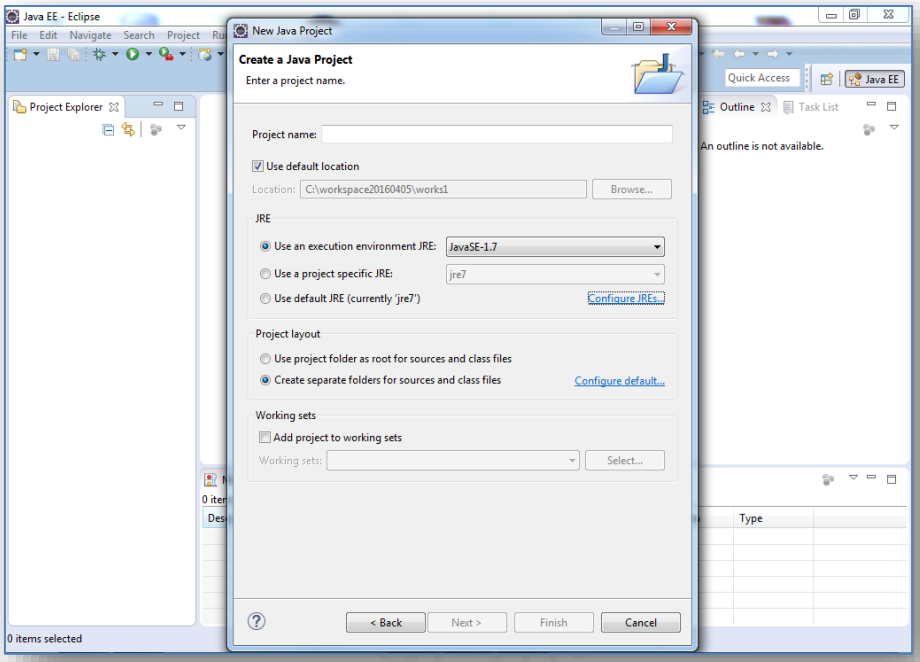
java 8 _ jdk1.8; jre 1.8 /dowload (java) site da Oracle



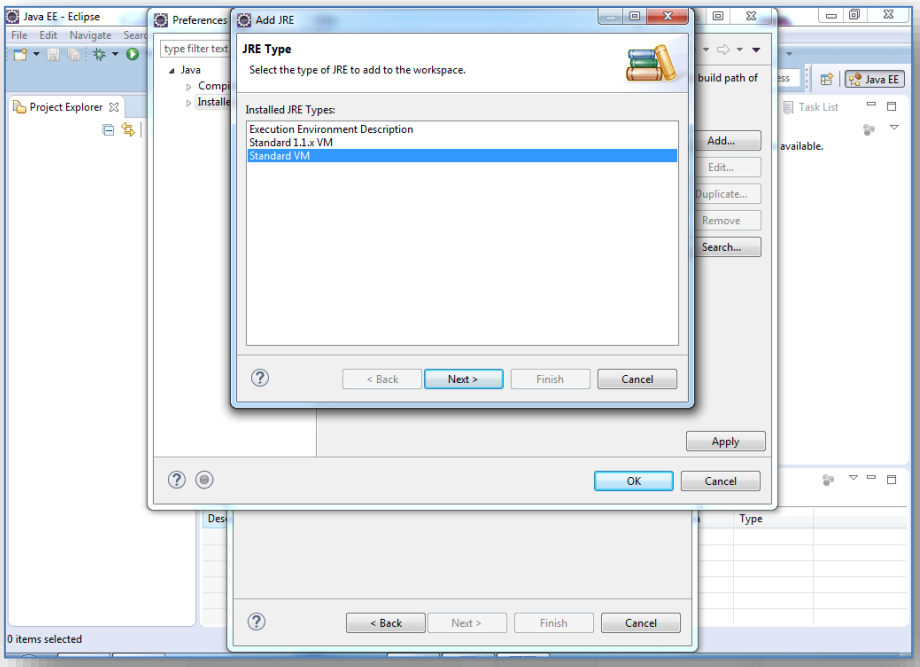
Copy and Paste/ Eclipse

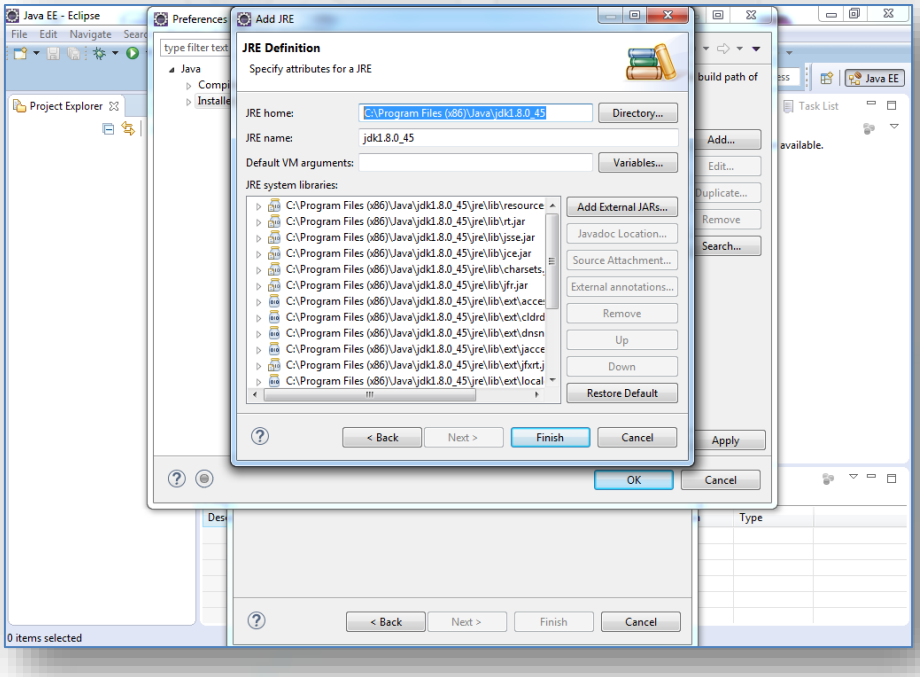
Orientação a Objetos
Java Parte 1

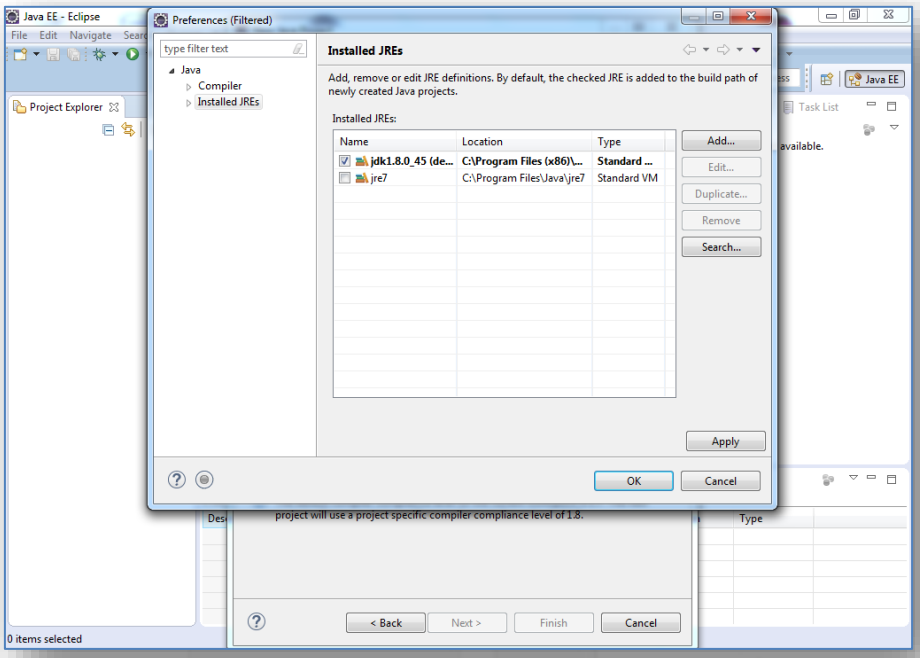
JSE _ Java Project _ Java Local (Foco OO)

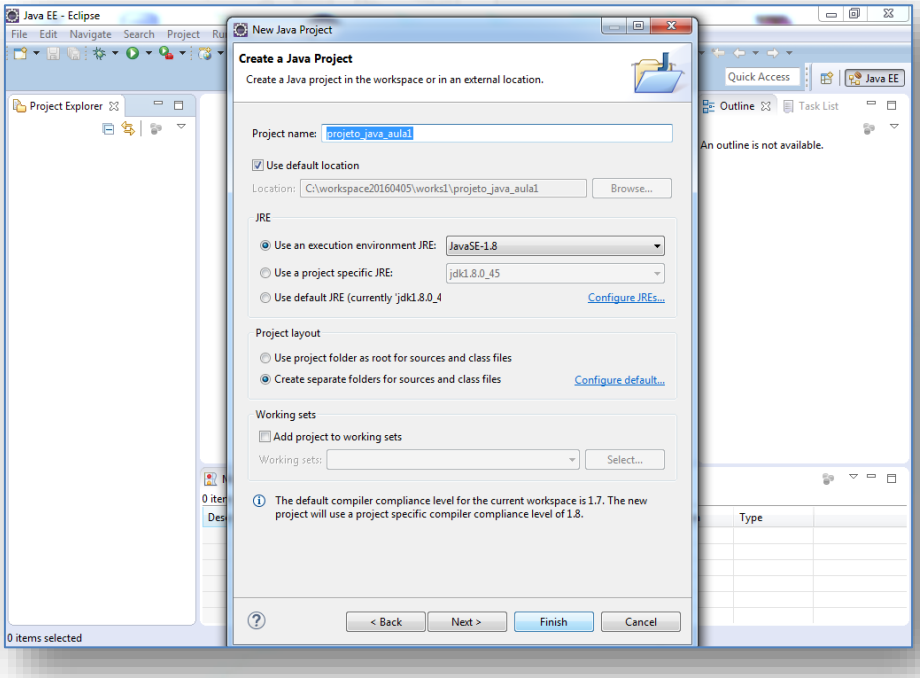


Escola de Nerds
www.cotiinformatica.com.br


Java EE - Eclipse
Preferences
Add JRE
JRE Type
Select the type of JRE to add to the workspace.
Installed JRE Types:
Execution Environment Description
Standard 1.1.x VM
Standard VM
< Back
Next >
Finish
Cancel
Apply
OK
Cancel
Project Explorer
0 items selected

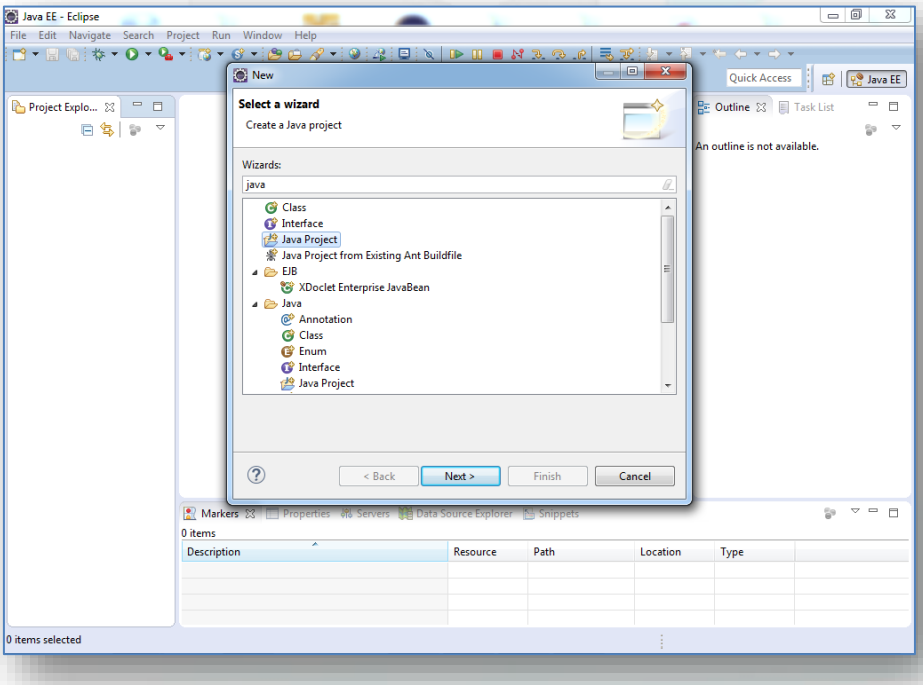

Java EE - Eclipse
Preferences
Add JRE
JRE Definition
Specify attributes for a JRE
JRE home:
C:\Program Files (x86)\Java\jdk1.8.0_45
Directory...
JRE name:
jdk1.8.0_45
Default VM arguments:
Variables...
JRE system libraries:
Add External JARs...
Javadoc Location...
Source Attachment...
External annotations...
Remove
Up
Down
Restore Default
< Back
Next >
Finish
Cancel
OK
Cancel
Project Explorer
0 items selected

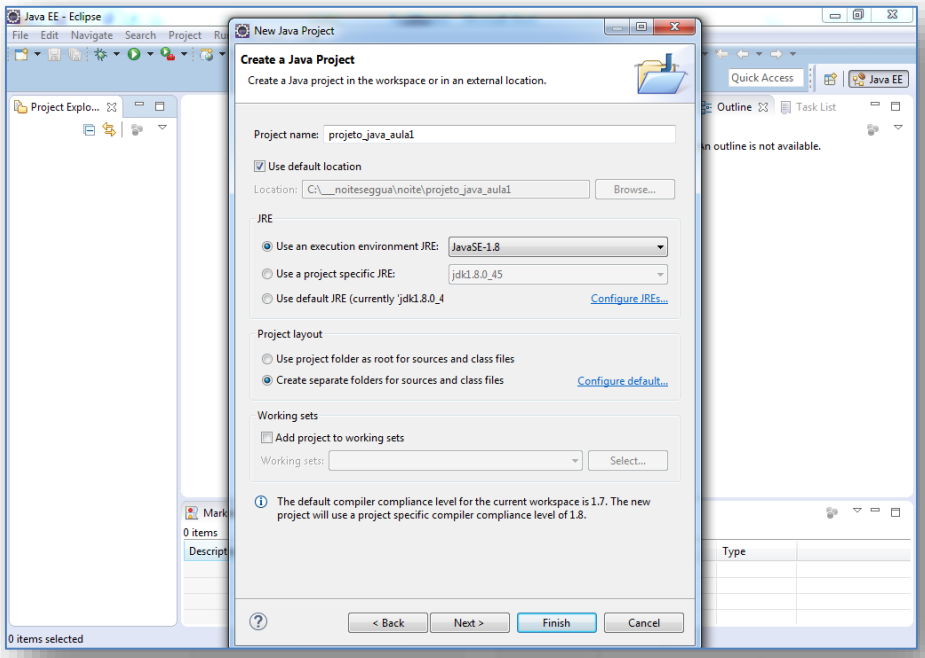

Java EE - Eclipse
Preferences (Filtered)
type filter text
Java
Compiler
Installed JREs
Installed JREs
Add, remove or edit JRE definitions. By default, the checked JRE is added to the build path of newly created Java projects.
Installed JREs:
Name
Location
Type
jdk1.8.0_45 (de...
C:\Program Files (x86)\...
Standard ...
jre7
C:\Program Files\Java\jre7
Standard VM
Add...
Edit...
Duplicate...
Remove
Search...
Apply
OK
Cancel
Project Explorer
Java EE
Task List
Type
0 items selected
< Back
Next >
Finish
Cancel


Java EE - Eclipse
New Java Project
Create a Java Project
Create a Java project in the workspace or in an external location.
Project name:
projeto_java_aula1
Use default location
Location:
C:\workspace20160405\works1\projeto_java_aula1
Browse...
JRE
Use an execution environment JRE:
JavaSE-1.8
Use a project specific JRE:
jdk1.8.0_45
Use default JRE (currently 'jdk1.8.0_4
Configure JREs...
Project layout
Use project folder as root for sources and class files
Create separate folders for sources and class files
Configure default...
Working sets
Add project to working sets
Working sets:
Select...
The default compiler compliance level for the current workspace is 1.7. The new project will use a project specific compiler compliance level of 1.8.
< Back
Next >
Finish
Cancel
Project Explorer
Quick Access
Java EE
Outline
Task List
An outline is not available.
Type
0 items selected

Java _ Parte baixar da oracle jdk8 (Download)
jdk _ compilador
jre _ Interpretador
entro no site da oracle e instalo o jdk 8.0
java 8

JSE _ java standard Edition (Java OO) Programação Local ...
instalar o jdk
_ Programa é o Eclipse ...
---
web (projetos) _ MVC
JSF _ (very easy) Oasis ... (bom)
---struts2 /conhecimento
---spring / conhecimento



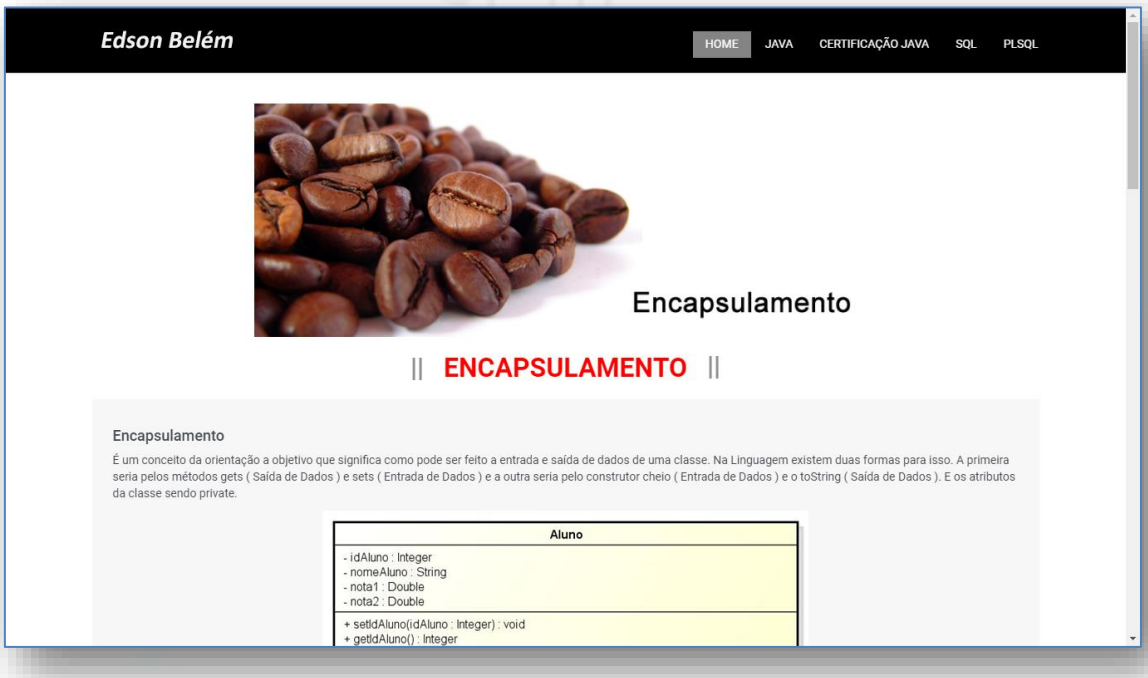


Separar em Camadas ...

Model / View / Control
entity/ view / control
Os Dados que trabalho
view
_input e output
control
lógica

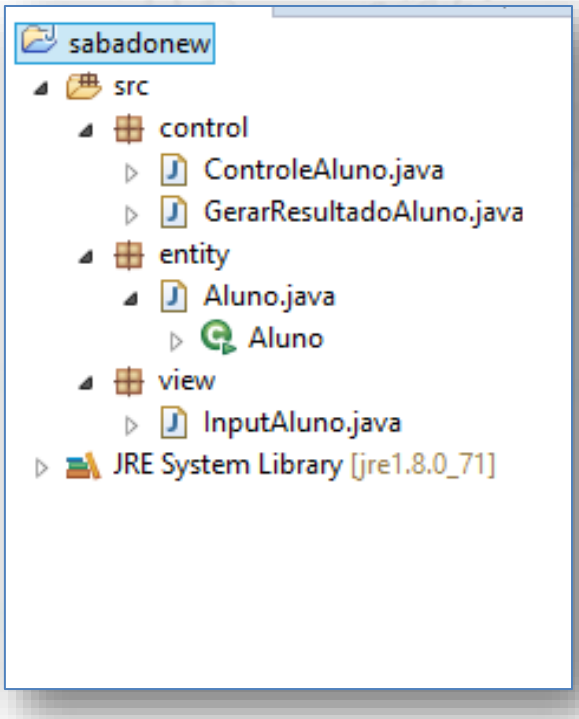
http://blogedsonbelem.com.br/blog/java/artigos/mvc.html

http://blogedsonbelem.com.br/blog/java/artigos/encaps.html



http://blogedsonbelem.com.br/blog/java/artigos/classes.html

Estrutura do projeto:



```
package entity;

public class Aluno {

//pacote, importaÃ§Ã£o e classe
//    modificar de acesso ()
//public, default (), protected
//public e default classe Externa
//public default, protecte, private
//nao nomralizada

	private Integer idAluno;
	//int ou Integer (32)
	//byte, short, int, long
	//Byte, Short, Integer, Long
	private String nome;
	private String disciplina;
	private Double nota1; //. (64)
	private Double nota2;
	private Double media;
	private String situacao;

	public Integer getIdAluno() {
		return idAluno;
	}
	public void setIdAluno(Integer idAluno) {
		this.idAluno = idAluno;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getDisciplina() {
		return disciplina;
	}
	public void setDisciplina(String disciplina) {
		this.disciplina = disciplina;
	}
	public Double getNota1() {
		return nota1;
```

```
	}
	public void setNota1(Double nota1) {
		this.nota1 = nota1;
	}
	public Double getNota2() {
		return nota2;
	}
	public void setNota2(Double nota2) {
		this.nota2 = nota2;
	}
	public Double getMedia() {
		return media;
	}
	public void setMedia(Double media) {
		this.media = media;
	}
	public String getSituacao() {
		return situacao;
	}
	public void setSituacao(String situacao) {
		this.situacao = situacao;
	}

	//atalho (Main + ctrl + espaÃ§o e enter)
	//public static void main(String args[]){

	public static void main(String[] args) {
	 //Para chamar o mÃ©todo, trabaklhar com os dados[
		//eu preciso do Objeto
		Aluno a = new Aluno();

		 a.setNome("jose");
		 a.setDisciplina("java");
		 a.setNota1(10.);
		 a.setNota2(8.); //encapsular os dados

		 //a.nome= "morte do belem ...";

		 System.out.println(a.getNome());
		 //syso + ctrl + espaÃ§o e enter
System.out.println(a.getNome() + "," + a.getDisciplina() );


	Aluno x = new Aluno();
```

```
		x.setNome("nilton");
		x.setDisciplina("fisica");
		System.out.println(x.getNome());
		}
}
```

No console...

```
jose
jose,java
nilton
```

```
package view;

import javax.swing.JOptionPane;

import entity.Aluno;

public class InputAluno {

	public void  lerIdAluno(Aluno a){
		a.setIdAluno(
	new Integer(JOptionPane.showInputDialog("idAluno", null))
				);
	}

	public void  lerNome(Aluno a){
	//encapsulando (eu entro com os dados le encapsula depois)
	//Digito o nome nÃ£o preciso converter ... pq Ã© String
		a.setNome(
	JOptionPane.showInputDialog("nome", null)
				);
	}

	public void  lerDisciplina(Aluno a){
		//encapsulando (eu entro com os dados le encapsula
depois)
		a.setDisciplina(
	 JOptionPane.showInputDialog("disciplina", null)
				);
	}
```

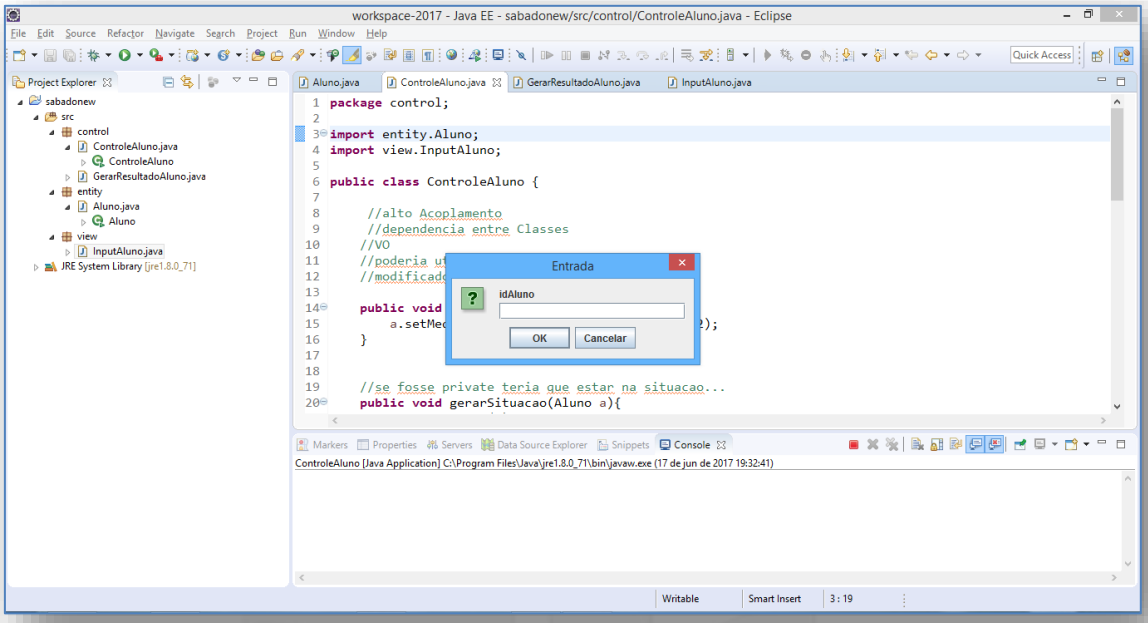
```
	public void  lerNota1(Aluno a){
	//encapsulando (eu entro com os dados le encapsula depois)
		//Converte uma String em real
		 a.setNota1(
	 new Double(JOptionPane.showInputDialog("nota1", null))
					);
	}

	//SÃ³ quando for numero eu preciso Converter ...
	//ele encqapsulando o que estou digitando ...

	public void  lerNota2(Aluno a){
		 a.setNota2(
	  new Double(JOptionPane.showInputDialog("nota2", null))
					);
	}
}
```

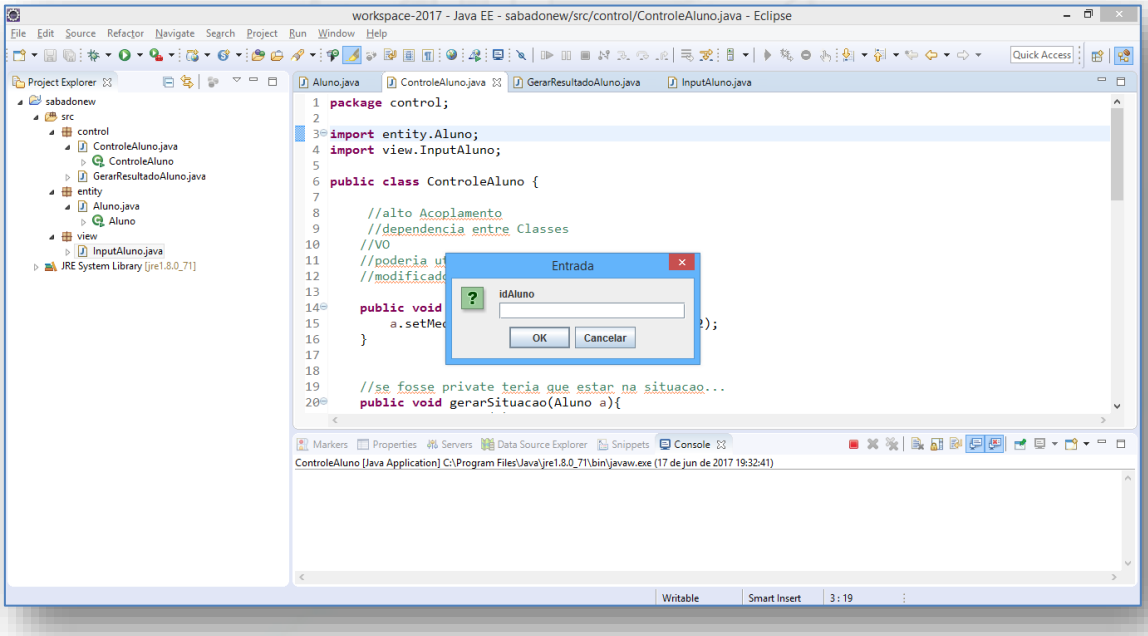
```
package control;

public class GerarResultadoAluno {

	public Double calcularMedia(Double n1, Double n2){
		return (n1+n2)/2;
	}
}
```

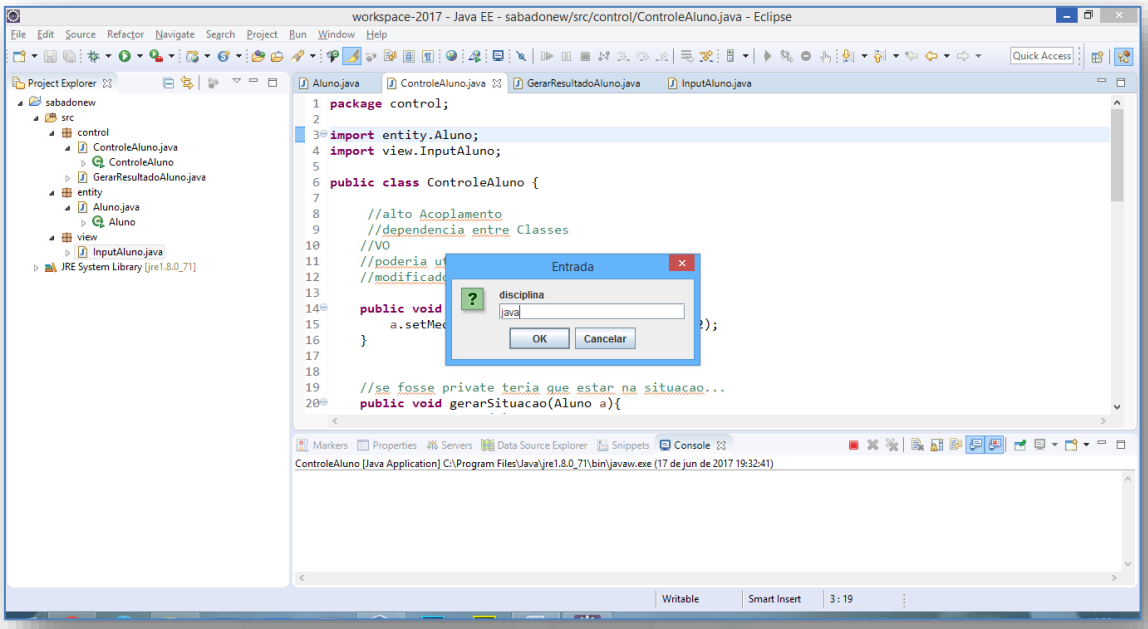
```
package control;

import entity.Aluno;
import view.InputAluno;

public class ControleAluno {

	 //alto Acoplamento
	 //dependencia entre Classes
	//VO
	//poderia utilizar o metodo como private
	//modificado de acesso como private ...
```

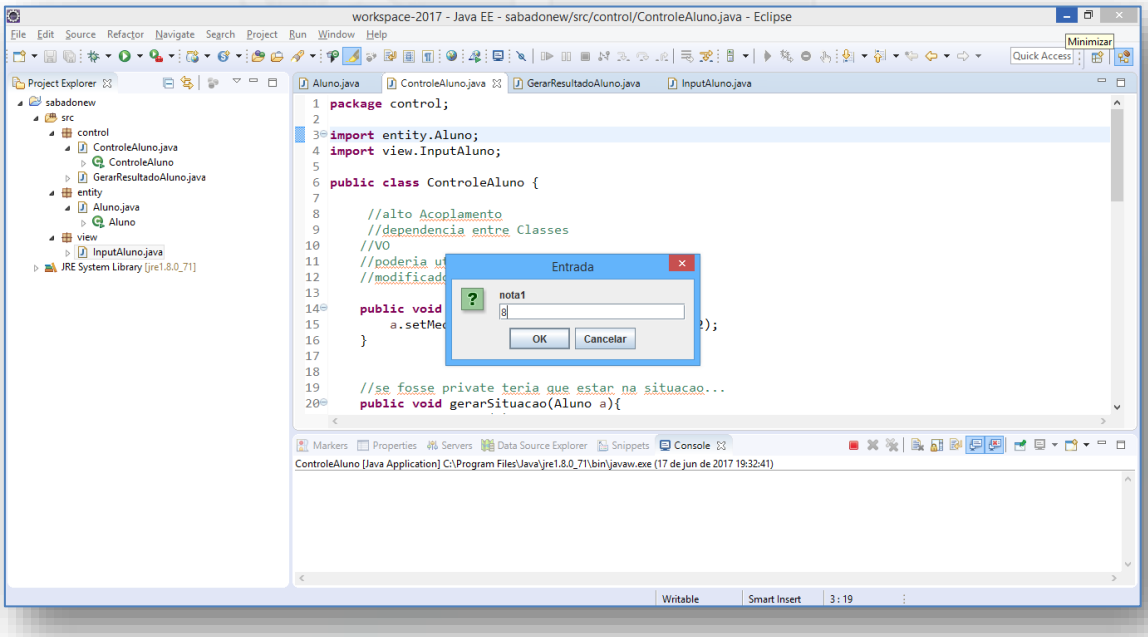
```
                                    //VO
        public void calcularMedia(Aluno a){
              a.setMedia((a.getNota1() + a.getNota2())/2);
        }


        //se fosse private teria que estar na situacao...
        public void gerarSituacao(Aluno a){
              calcularMedia(a);
              if (a.getMedia()>7){
                    a.setSituacao("aprovado");
              }else if (a.getMedia()>=5){
                    a.setSituacao("recuperacao");
              }else{
                    a.setSituacao("reprovado");
              }
        }

        public static void main(String[] args) {
              Aluno a = new Aluno();
              ControleAluno ca = new ControleAluno();
              InputAluno ia = new InputAluno();
              try{
                      ia.lerIdAluno(a);
                      ia.lerNome(a);
                      ia.lerDisciplina(a);
                      ia.lerNota1(a);
                      ia.lerNota2(a);
                       ca.calcularMedia(a); //processamento
                       ca.gerarSituacao(a);
  System.out.println(a.getNome() );
  System.out.println(a.getDisciplina() + "," + a.getMedia());
              }catch(Exception ex){
                    ex.printStackTrace();
              }
        }
}
```

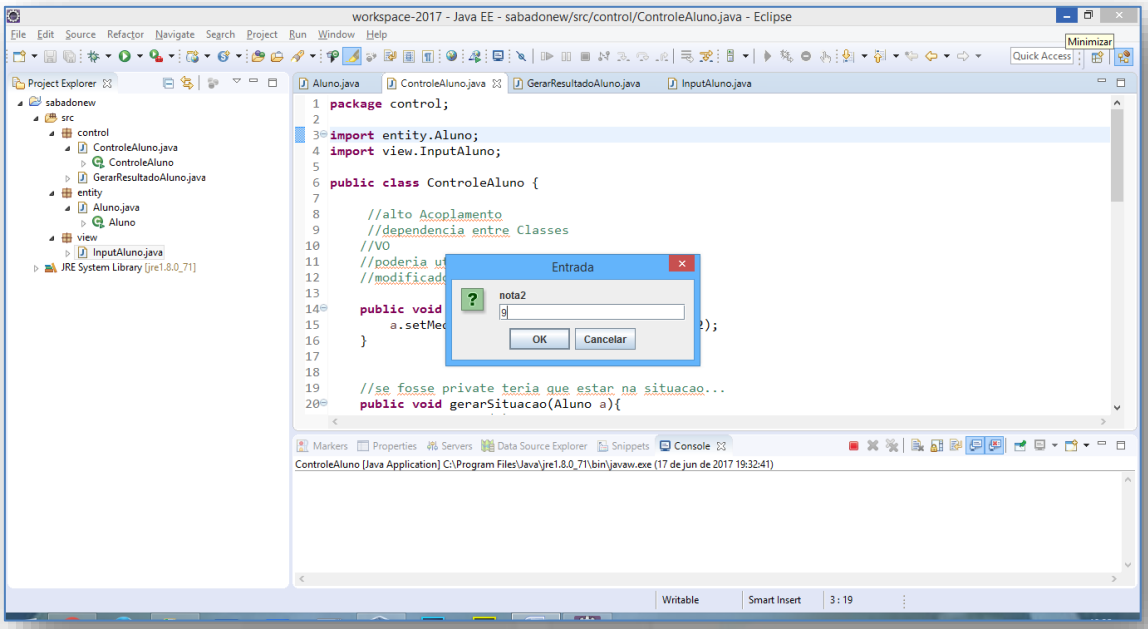
Rodando a classe...





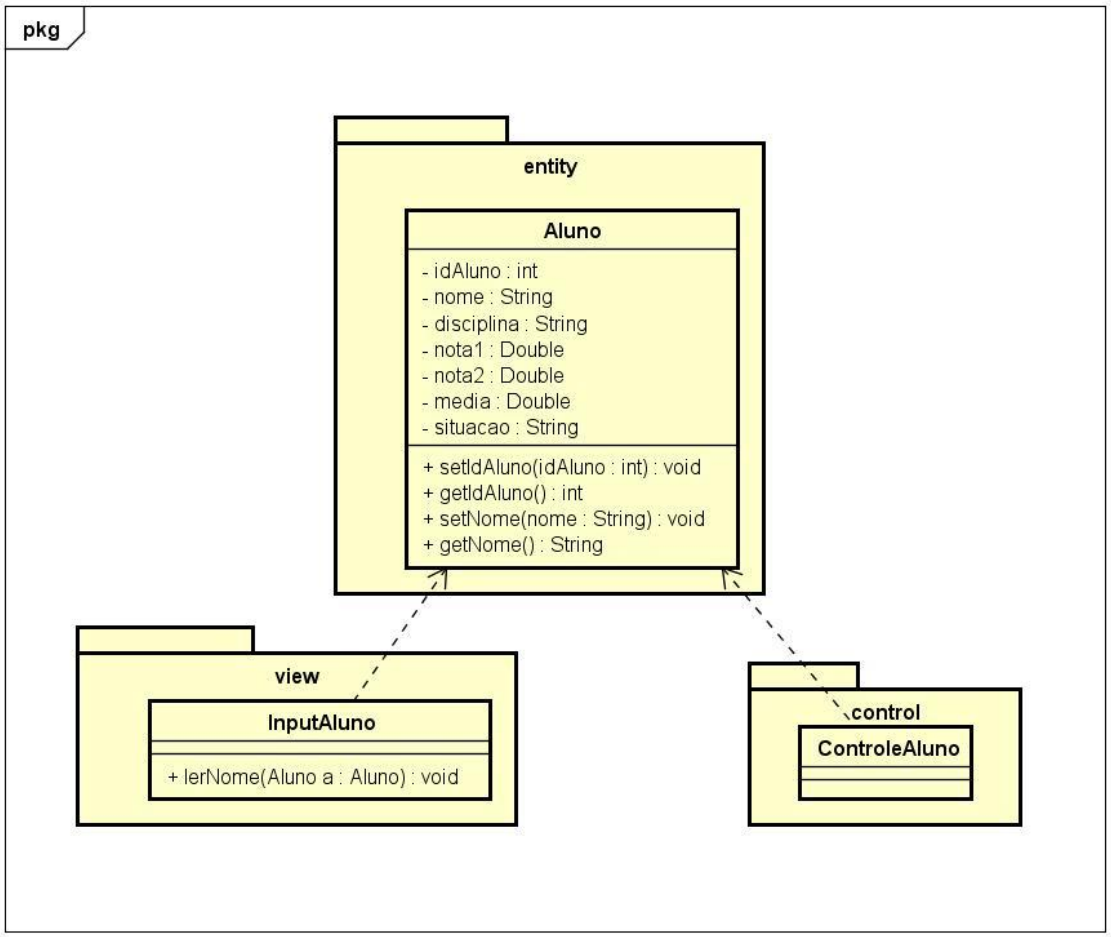

workspace-2017 - Java EE - sabadonew/src/control/ControleAluno.java - Eclipse
Entrada
disciplina
java
OK
Cancelar


workspace-2017 - Java EE - sabadonew/src/control/ControleAluno.java - Eclipse
Entrada
nota1
8
OK
Cancelar



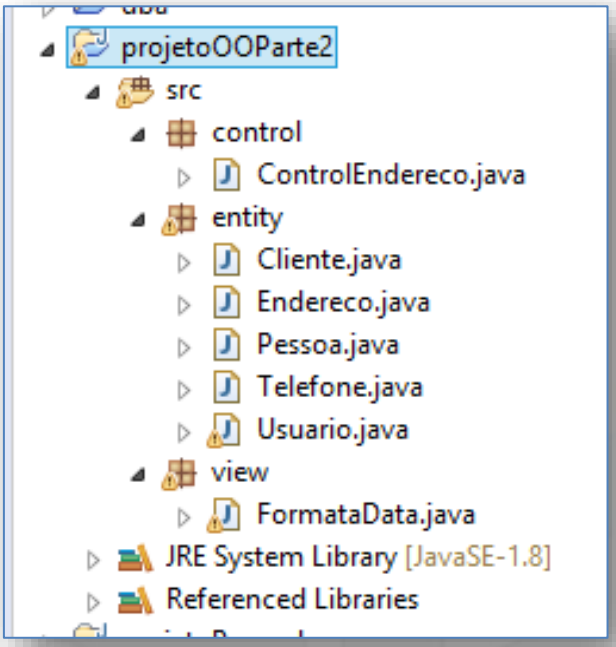
No console...

```
luciana
java,8.5
```



Escola de Nerds
www.cotiinformatica.com.br

Estrutura do projeto:




```
package entity;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public  class Pessoa {

	//Estou utilizando um formatador de Data para colocar a
data
	//no padrÃ£o do Brasil dia/Mes/Ano
	//final Ã© constante (nao muda)
	//static jÃ¡ estÃ¡ na memorria

	//esse recurso Ã© para ver a data no padÃ£o Brasileiro ...
	public final static DateTimeFormatter DF=
			DateTimeFormatter.ofPattern("dd/MM/yyyy");

	private String     nome;
	private String     email;
	private LocalDate  dataNascimento; //java 8
```

```
	public Pessoa() {

	}

	 //Encapsular (alimentar os atributos daqui)
	public Pessoa(String nome, String email, LocalDate
dataNascimento) {
		this.nome = nome;
		this.email = email;
		this.dataNascimento = dataNascimento;
	}

	@Override
	public String toString() {
		return "Pessoa [nome=" + nome + ", email=" + email +
", dataNascimento=" + DF.format(dataNascimento) + "]";
	}

	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public LocalDate getDataNascimento() {
		return dataNascimento;
	}
	public void setDataNascimento(LocalDate dataNascimento) {
		this.dataNascimento = dataNascimento;
	}

	public static void main(String[] args) {
	 Pessoa p = new Pessoa("ilan","ilan@gmail.com",
LocalDate.of(1997,02, 10));
	//  p.setNome("Ilan");
	// p.setEmail("ilan@gmail.com");
	// p.setDataNascimento(LocalDate.of(1997, 02, 10));
```

```
		//Data Ã© o que mais muda no Java ...
			System.out.println(p);
		}

}
```

No console...

```
Pessoa [nome=ilan, email=ilan@gmail.com,
dataNascimento=10/02/1997]
```

```
package entity;

import java.time.LocalDate;

//Ser (extends)
//Cliente Ã© uma Pessoa

public class Cliente extends Pessoa{

//Cliente is a
	private Integer codigo;
	private String  sexo;

	public Cliente() {
   }

	public Cliente(Integer codigo, String nome, String email,
LocalDate dataNascimento, String sexo) {
			super(nome, email, dataNascimento);
			this.codigo = codigo;
			this.sexo = sexo;
		}

		@Override
		public String toString() {
			return "Cliente [codigo=" + codigo + ", sexo=" + sexo
+ ", getNome()=" + getNome() + ", getEmail()=" + getEmail() + ",
getDataNascimento()=" + DF.format(getDataNascimento()) + "]";
		}
```

```
	public Integer getCodigo() {
		return codigo;
	}
	public void setCodigo(Integer codigo) {
		this.codigo = codigo;
	}
	public String getSexo() {
		return sexo;
	}
	public void setSexo(String sexo) {
		this.sexo = sexo;
	}


	public static void main(String[] args) {
		Cliente c= new
Cliente(10,"lu","lu@gmail.com",LocalDate.of(1999, 01, 20),"f");
		System.out.println(c);
	}
}
```

No console...

```
Cliente [codigo=10, sexo=f, getNome()=lu,
getEmail()=lu@gmail.com, getDataNascimento()=20/01/1999]
```

```
package entity;

import java.util.List;

public class Usuario {
 //has a

	//OneToOne
	//Usuario tem um Endereco
	private Endereco endereco;
	//OneToMany
	private Telefone[] telefones;
```

```
	//OneToMany
	private List<Telefone> listaTelefone;
}
```

```
package entity;

public class Endereco {

	private Integer idEndereco;
	private String logradouro;
	private Integer numero;
	private String bairro;
	private String cidade;
	private String uf;
	private String cep;

	public Endereco() {
	}

	public Endereco(Integer idEndereco, String logradouro,
Integer numero, String bairro, String cidade, String uf, String
cep) {
		this.idEndereco = idEndereco;
		this.logradouro = logradouro;
		this.numero = numero;
		this.bairro = bairro;
		this.cidade = cidade;
		this.uf = uf;
		this.cep = cep;
	}

	// esse padrÃ£o Ã© do GoogleMaps

	@Override
	public String toString() {
		return logradouro + "," + numero + "," + bairro + ","
+ cidade + "," + uf + "," + cep;
	}
	// Correio nÃ£o liga para o numero ...

	public Integer getIdEndereco() {
```

```
		return idEndereco;
	}
	public void setIdEndereco(Integer idEndereco) {
		this.idEndereco = idEndereco;
	}
	public String getLogradouro() {
		return logradouro;
	}
	public void setLogradouro(String logradouro) {
		this.logradouro = logradouro;
	}
	public Integer getNumero() {
		return numero;
	}
	public void setNumero(Integer numero) {
		this.numero = numero;
	}
	public String getBairro() {
		return bairro;
	}
	public void setBairro(String bairro) {
		this.bairro = bairro;
	}
	public String getCidade() {
		return cidade;
	}
	public void setCidade(String cidade) {
		this.cidade = cidade;
	}
	public String getUf() {
		return uf;
	}
	public void setUf(String uf) {
		this.uf = uf;
	}
	public String getCep() {
		return cep;
	}
	public void setCep(String cep) {
		this.cep = cep;
	}
}
```

```
package entity;

public class Telefone {


}
```

```
package control;

import org.lavieri.modelutil.cep.WebServiceCep;
import entity.Endereco;

public class ControlEndereco {

  public void buscarCep(String cep,Endereco e){
        WebServiceCep ws = WebServiceCep.searchCep(cep);

        if (ws.wasSuccessful()){
              e.setLogradouro(ws.getLogradouro());
              e.setBairro(ws.getBairro());
              e.setCidade(ws.getCidade());
              e.setUf(ws.getUf());
           e.setCep(ws.getCep());
        }else{
             System.out.println("Cep nao Encontrado ");
             throw new IllegalArgumentException("Cep nao
Encontrado ...");
             //throw lanÃ§ar o erro e vai parar
        }

  }

      //SobreCarga mÃ©todo igual com parametros a maix
  public void buscarCep(String cep,Integer numero, Endereco e){
        WebServiceCep ws = WebServiceCep.searchCep(cep);
    if (ws.wasSuccessful()){
              e.setLogradouro(ws.getLogradouro());
              e.setBairro(ws.getBairro());
              e.setCidade(ws.getCidade());
              e.setUf(ws.getUf());
```

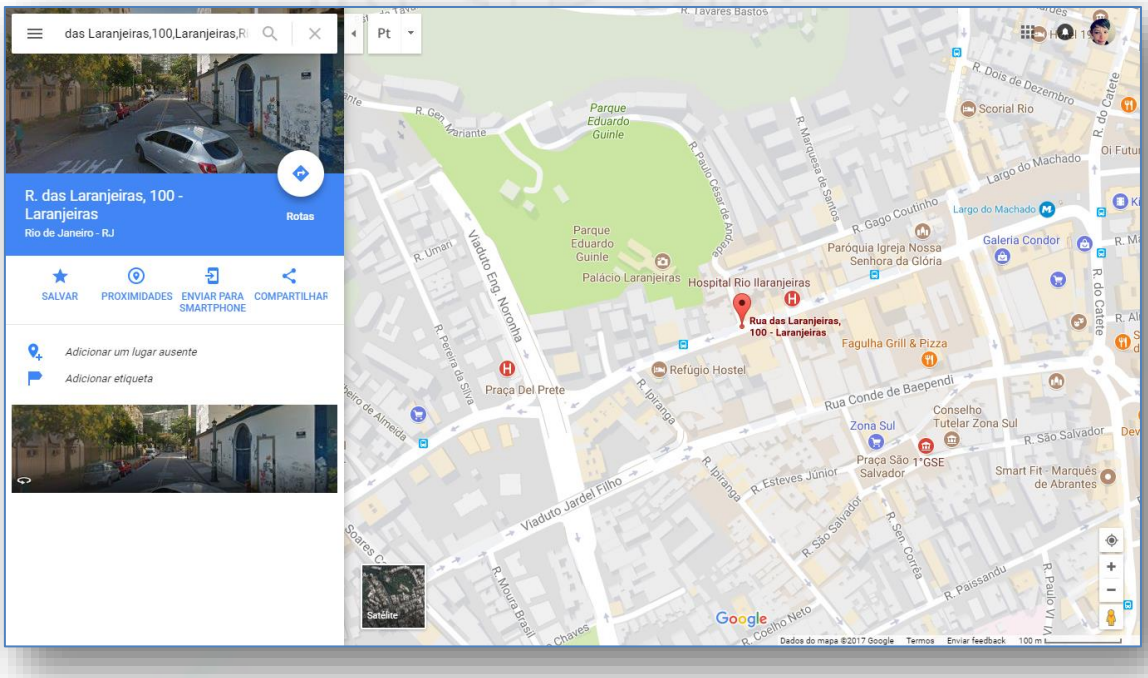
```
            e.setCep(ws.getCep());
            e.setNumero(numero);
        }else{
             System.out.println("Cep nao Encontrado ");
             throw new IllegalArgumentException("Cep nao
Encontrado ...");
        }
  try{
              Runtime.getRuntime().exec(new String[]{"firefox",
            "http://www.google.com.br/maps/place/" +
              e.toString().replace(" ","%20")
             });

        }catch(Exception ex){
            ex.printStackTrace();
        }
  }


  public void buscarCepWindows(String cep,Integer numero,
Endereco e){
        WebServiceCep ws = WebServiceCep.searchCep(cep);
          if (ws.wasSuccessful()){
                  e.setLogradouro(ws.getLogradouro());
                  e.setBairro(ws.getBairro());
                  e.setCidade(ws.getCidade());
                  e.setUf(ws.getUf());
                e.setCep(ws.getCep());
                e.setNumero(numero);
              }else{
                  System.out.println("Cep nao Encontrado ");
 throw new IllegalArgumentException("Cep nao Encontrado ...");
              }
        try{
                  System.out.println("Abrindo Google Maps");
     Runtime.getRuntime().exec(new String[]{"cmd" , "/c", "
start chrome    " +  "http://www.google.com.br/maps/place/"  +
                        e.toString().replaceAll(" ", "%20")
                          });
            }catch(Exception ex){
                ex.printStackTrace();
            }
      }
```

```
        public static void main(String[] args) {
              Endereco e = new Endereco();
              ControlEndereco ce = new ControlEndereco();
                 ce.buscarCep("22240004", 100, e);
                 System.out.println("Endereco :" + e);
        }
}
```

No console...

```
Abrindo Google Maps

Endereco :das Laranjeiras,100,Laranjeiras,Rio de
Janeiro,RJ,22240004
```

No chrome pelo Windows...



```
package view;

import java.text.SimpleDateFormat;
import java.util.Date;
 /**
  *
  * @author aluno
  * @since  FormatarData no Java
  * @version 1.0
  */
public class FormataData {
 /**
  * @param  -Date Data Entrada java.util
  * @return -String Data Formatada padrao Brasil
  */
      public String dataFormataBrasil(Date d){
      SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
            return sdf.format(d);
      }
/**
 * @param -Date Data Entrada java.util
 * @return -Data Formatada padrao EUA
 */
      public String dataFormataEUA(Date d){
      SimpleDateFormat sdf = new SimpleDateFormat("MM-dd-yy");
            return sdf.format(d);
      }
/**
 * @param -Date Data Entrada java.util
 * @return -Data Formata padrao Web
 */
      public String dataFormataWEB(Date d){
      SimpleDateFormat sdf = new SimpleDateFormat("YYYY-MM-dd");
            return sdf.format(d);
      }

      public static void main(String[] args) {
            FormataData  fmt = new FormataData();
//mes-dia-ano
      System.out.println(fmt.dataFormataBrasil(new
Date("01/28/1973")));
      System.out.println(fmt.dataFormataWEB(new Date())); //data
do Sistema
```

```
        System.out.println(fmt.dataFormataEUA(new Date()));
        }
}
```

No console...

```
28/01/1973
2017-06-25
06-25-17
```

Gerando Javadoc de uma classe...

Javadoc

Javadoc é um gerador de documentação criado pela Sun Microsystems para documentar a API dos programas em Java, a partir do código-fonte. O resultado é expresso emHTML. É constituído, basicamente, por algumas marcações muitos simples inseridas nos comentários do programa.

Este sistema é o padrão de documentação de classes em Java, e muitas dos IDEs desta linguagem irão automaticamente gerar um Javadoc em HTML.

Ele também provê uma API para a criação de doclets e taglets, que permitem a análise da estrutura de um aplicativo Java. É assim, por exemplo, que o JDiff consegue gerar relatórios de alterações feitas entre duas versões de uma API.

Tags Javadoc

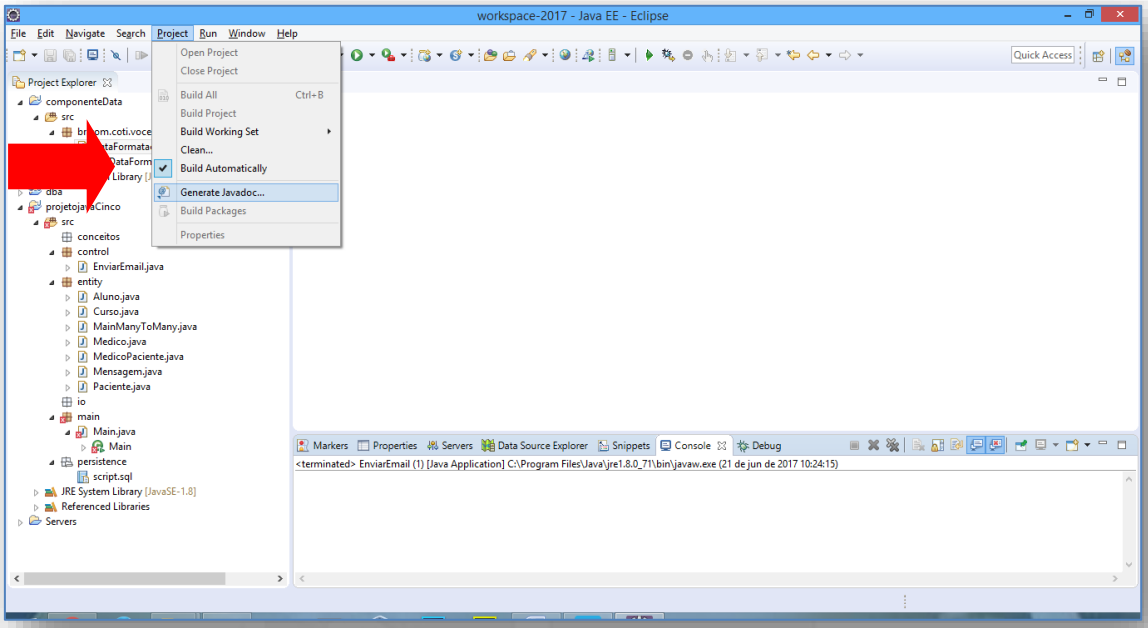
Os desenvolvedores usam certos estilos de comentários e tags Javadoc ao documentar códigos-fonte. Um bloco de comentário em Java iniciado com **/**** irá iniciar um bloco de comentário Javadoc, que será incluído no HTML gerado. Uma tag Javadoc começa com um "@" (arroba). Na tabela abaixo, algumas destas tags.

Tag	Descrição
@author	Nome do desenvolvedor
@category	Você pode criar categorias para seus códigos, como “utilidades” ou “maipulação” ou “services”, etç,
@deprecated	Marca o método como *deprecated*. Algumas IDEs exibirão um alerta de compilação se o método for chamado.
@exception	Documenta uma exceção lançada por um método — veja também @throws.
@link	Possibilita a definição de um link para um outro documento local ou remoto através de um URL.
@param	Define um parâmetro do método. Requerido para cada parâmetro.
@return	Documenta o valor de retorno. Essa tag não deve ser usada para construtores ou métodos definidos com o tipo de retorno *void*.
@see	Documenta uma associação a outro método ou classe.
@since	Documenta quando o método foi adicionado a a classe.
@throws	Documenta uma exceção lançada por um método. É um sinônimo para a @exception introduzida no Javadoc 1.2.
@version	Exibe o número da versão de uma classe ou um método.
{@link XX}	Cria um link para uma classe ou método, o nome da classe ou método deve ser colocado no lugar das letras “XX”

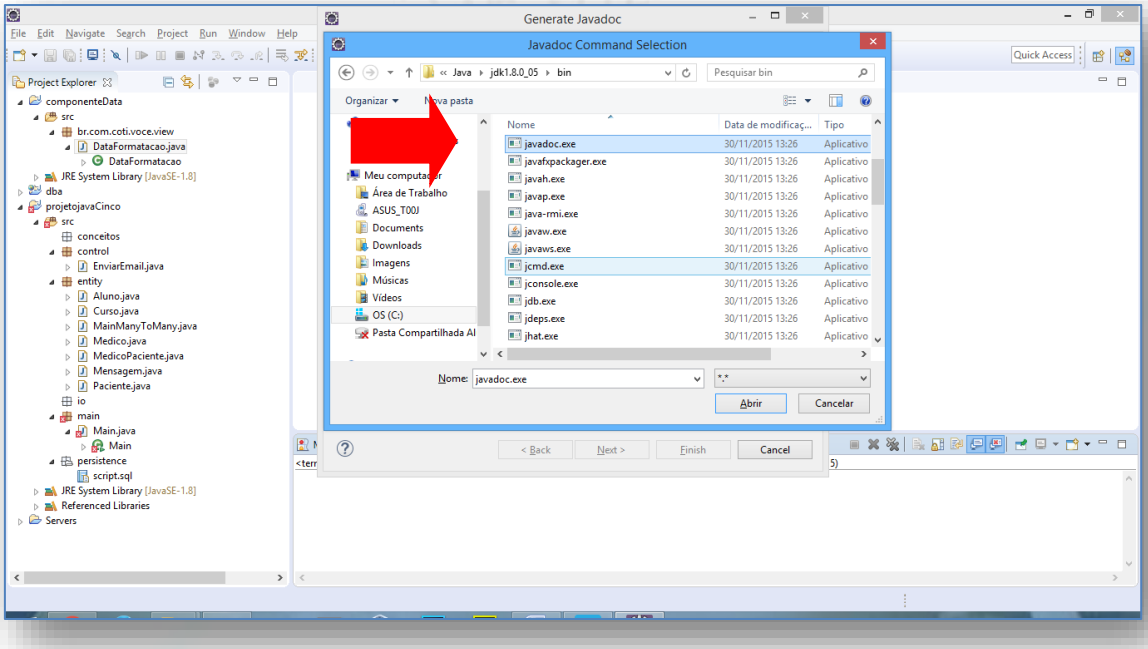
Para inserir o símbolo **@** sem iniciar uma tag Javadoc você pode usar o código de caracter HTML **@** e evitar problemas de *parsing*.

Para criar o javadoc...

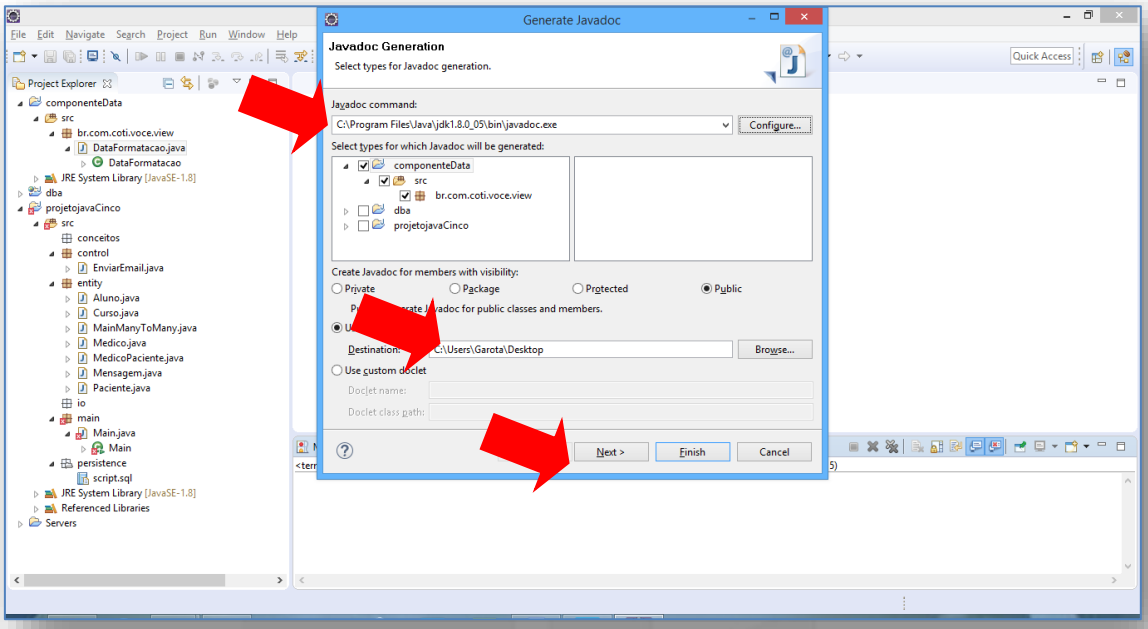
Clicar no Menu -> Project -> GenerateJavadoc...



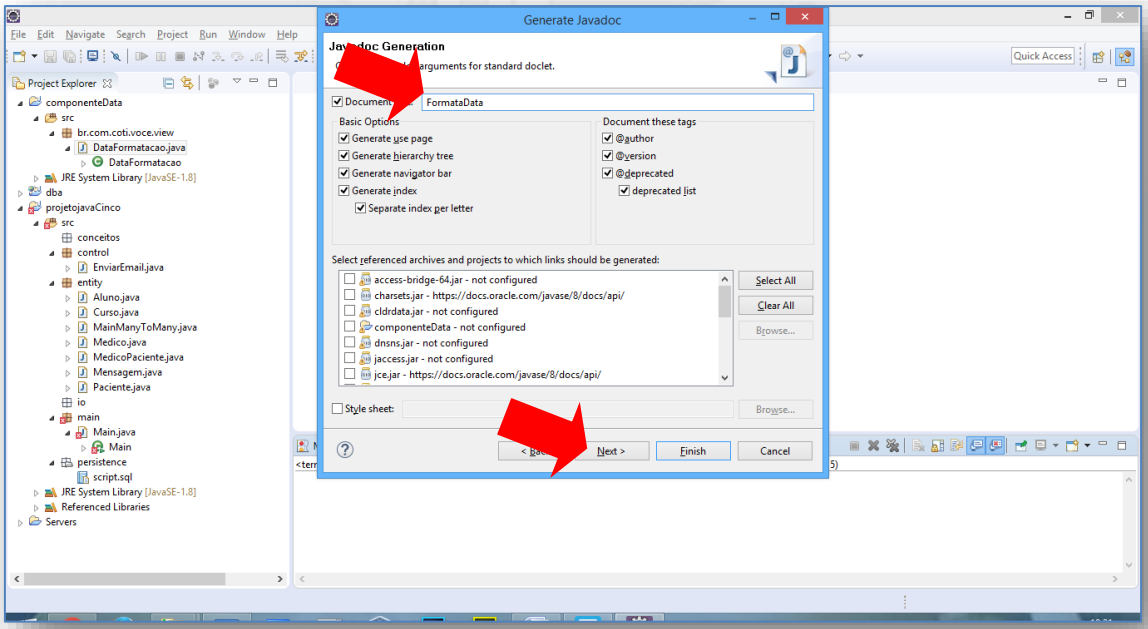
Em javadoc commando indicar onde se encontra o JDK.



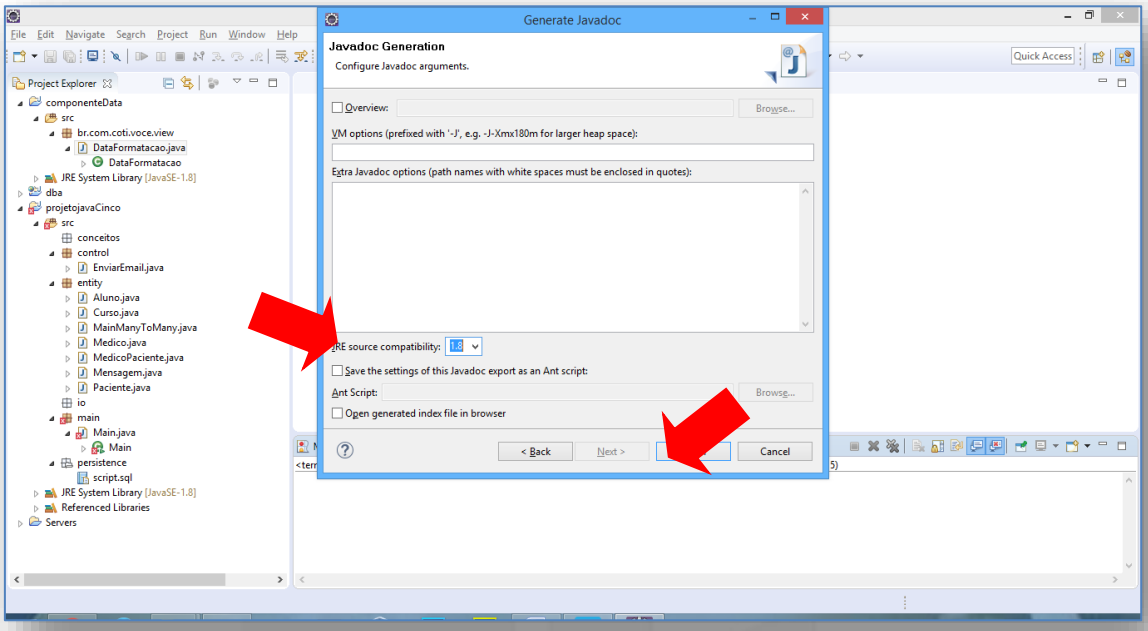
Alterar o destino onde será gravado -> next...



Colocar um nome pro documento...



Conferir o JRE -> Finish...

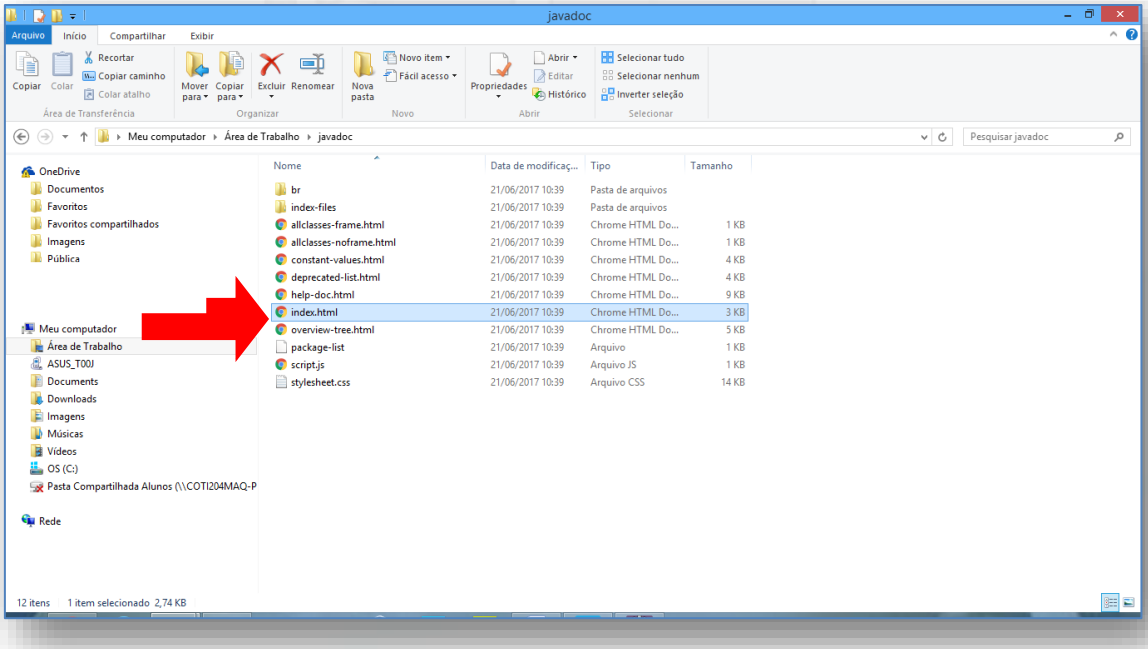


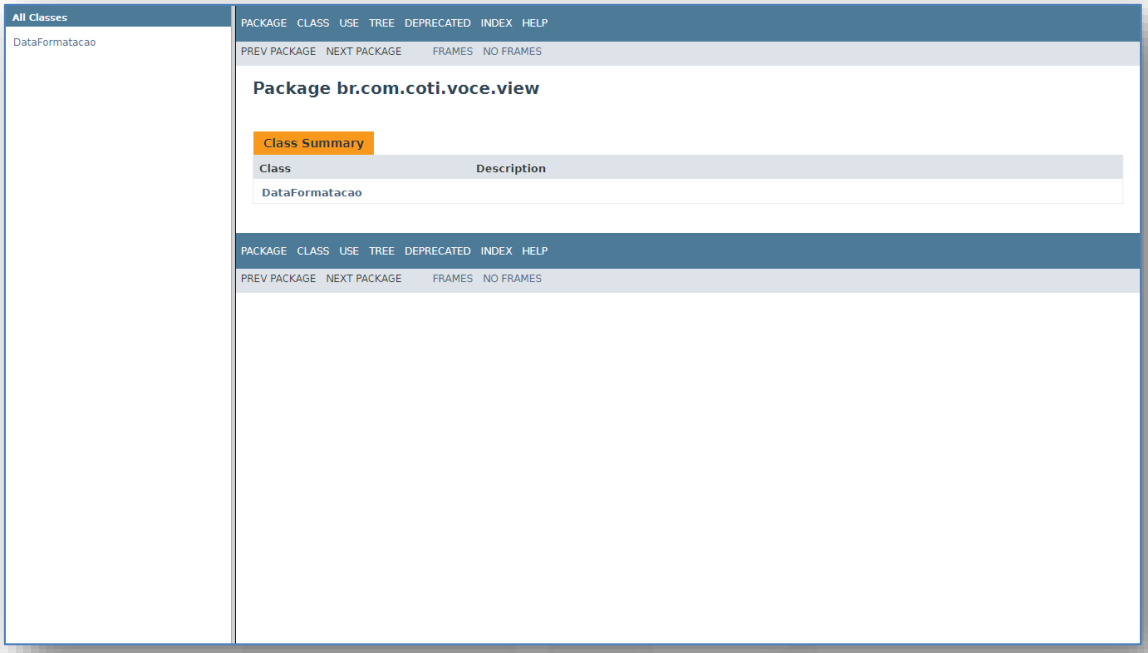
No console...

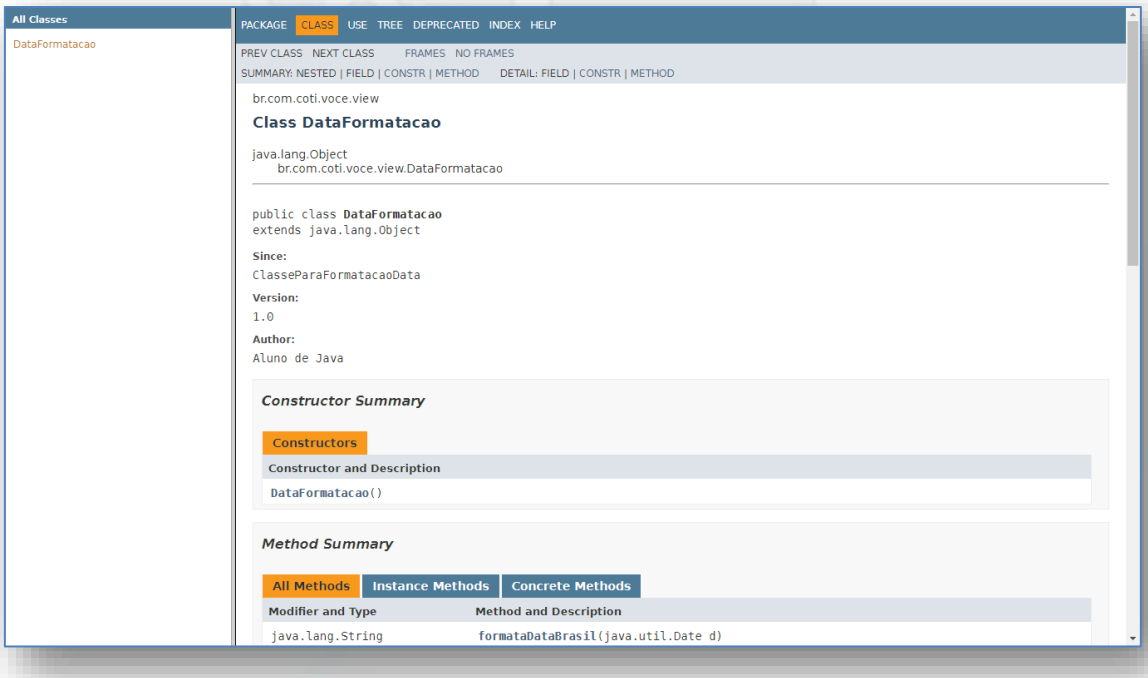
```
Loading source files for package br.com.coti.voce.view...
Constructing Javadoc information...
Standard Doclet version 1.8.0_05
Building tree for all the packages and classes...
Generating
C:\Users\Garota\Desktop\br\com\coti\voce\view\DataFormatacao.htm
l...
Generating
C:\Users\Garota\Desktop\br\com\coti\voce\view\package-
frame.html...
Generating
C:\Users\Garota\Desktop\br\com\coti\voce\view\package-
summary.html...
Generating
C:\Users\Garota\Desktop\br\com\coti\voce\view\package-
tree.html...
Generating C:\Users\Garota\Desktop\constant-values.html...
Generating C:\Users\Garota\Desktop\br\com\coti\voce\view\class-
use\DataFormatacao.html...
```

```
Generating
C:\Users\Garota\Desktop\br\com\coti\voce\view\package-
use.html...
Building index for all the packages and classes...
Generating C:\Users\Garota\Desktop\overview-tree.html...
Generating C:\Users\Garota\Desktop\index-files\index-1.html...
Generating C:\Users\Garota\Desktop\index-files\index-2.html...
Generating C:\Users\Garota\Desktop\index-files\index-3.html...
Generating C:\Users\Garota\Desktop\deprecated-list.html...
Building index for all classes...
Generating C:\Users\Garota\Desktop\allclasses-frame.html...
Generating C:\Users\Garota\Desktop\allclasses-noframe.html...
Generating C:\Users\Garota\Desktop\index.html...
Generating C:\Users\Garota\Desktop\help-doc.html...
```

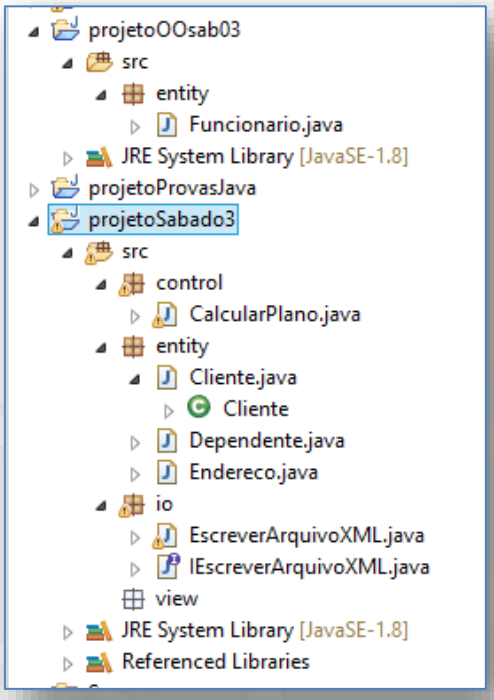
Ir na pasta indicada para criar o doc e abrir o índex.html...



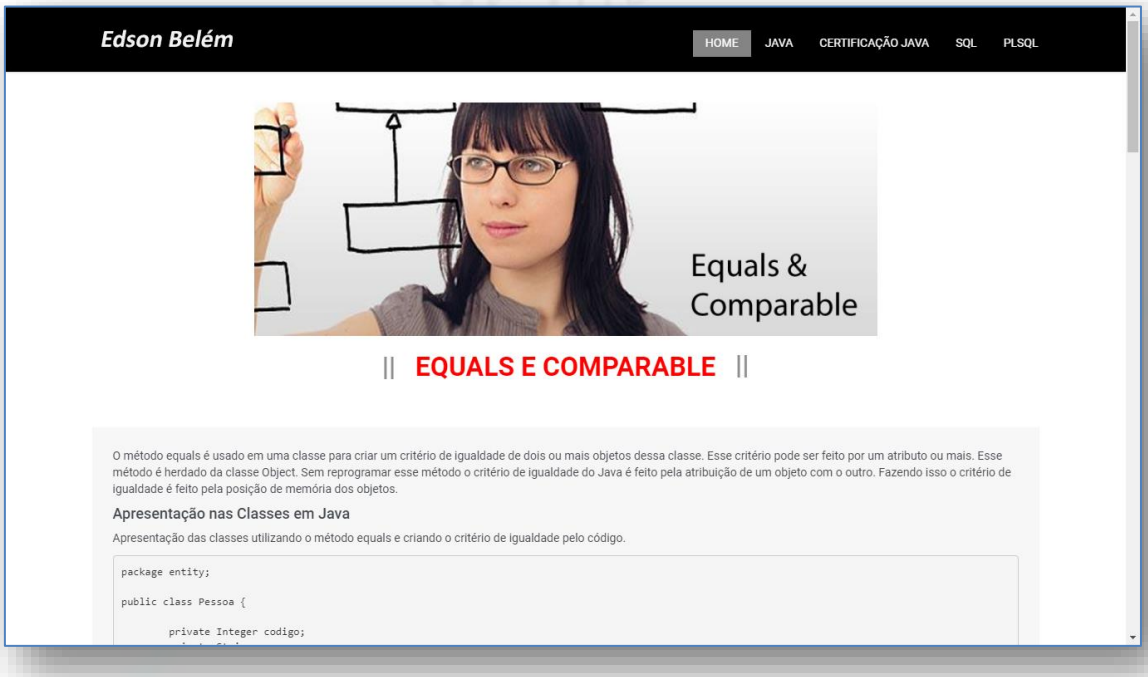

All Classes
DataFormatacao
PACKAGE CLASS USE TREE DEPRECATED INDEX HELP
PREV PACKAGE NEXT PACKAGE FRAMES NO FRAMES
Package br.com.coti.voce.view
Class Summary
Class
Description
DataFormatacao
PACKAGE CLASS USE TREE DEPRECATED INDEX HELP
PREV PACKAGE NEXT PACKAGE FRAMES NO FRAMES


All Classes
DataFormatacao
PACKAGE CLASS USE TREE DEPRECATED INDEX HELP
PREV CLASS NEXT CLASS FRAMES NO FRAMES
SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD
br.com.coti.voce.view
Class DataFormatacao
java.lang.Object
br.com.coti.voce.view.DataFormatacao
public class DataFormatacao
extends java.lang.Object
Since:
ClasseParaFormatacaoData
Version:
1.0
Author:
Aluno de Java
Constructor Summary
Constructors
Constructor and Description
DataFormatacao()
Method Summary
All Methods
Instance Methods
Concrete Methods
Modifier and Type
Method and Description
java.lang.String
formataDataBrasil(java.util.Date d)

Estrutura do projeto:



http://blogedsonbelem.com.br/blog/java/artigos/equal_comp.html



```
package entity;

import java.util.Arrays;

public class Funcionario  implements Comparable<Funcionario>{

	//organizaÃ§Ã£o ordenaÃ§Ã£o ...

	private Integer codigo;
	private String  nome;
	private Double  salario;

	public Funcionario() {

	}

	public Funcionario(Integer codigo, String nome, Double salario) {
		super();
		this.codigo = codigo;
		this.nome = nome;
		this.salario = salario;
	}

	@Override
	public String toString() {
		return "Funcionario [codigo=" + codigo + ", nome=" + nome + ", salario=" + salario + "]";
	}

	public Integer getCodigo() {
		return codigo;
	}
	public void setCodigo(Integer codigo) {
		this.codigo = codigo;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
```

```
	}
	public Double getSalario() {
		return salario;
	}
	public void setSalario(Double salario) {
		this.salario = salario;
	}

	 //BUSCAR os dados (localiza o seu objeto)
	@Override
	public boolean equals(Object o){
	 Funcionario f = (Funcionario) o; //casting
	 //O cÃ³digo desta classe Ã© igual ao da Outra Classe
	 return this.codigo.equals(f.codigo);
	}

//Ordenando a Classe (critÃ©rio de OrdenaÃ§Ã£o)
	@Override
	public int compareTo(Funcionario f) {
		return this.codigo.compareTo(f.getCodigo());
	}

	public static void main(String[] args) {
		Funcionario f1 = new Funcionario(10,"jose",5000.);
		Funcionario f2 = new Funcionario(11,"marcio",3000.);
		Funcionario f3 = new Funcionario(8,"bruno",1000.);
		System.out.println(f1.equals(f2));

		//java.lang.ArrayIndexOutOfBoundsException:
		Funcionario vet[]=new Funcionario[3];
		 vet[0]= f1;
		 vet[1]= f2;
		 vet[2]= f3;
		 Arrays.sort(vet); //ordenou pelo CÃ³digo ...
		 for (Funcionario x: vet){
			 System.out.println(x);
//imprime cada item ordenado
		 }
	}
}
```

No console...

```
false
Funcionario [codigo=8, nome=bruno, salario=1000.0]
Funcionario [codigo=10, nome=jose, salario=5000.0]
Funcionario [codigo=11, nome=marcio, salario=3000.0]
```

```
package entity;
import java.util.ArrayList;
import java.util.List;

public class Cliente {

	private Integer idCliente;
	private String  nome;
	private String  email;
	private Double  valorPlano;
	private Double  totalPago=0.;

	private Endereco endereco;

	private   List<Dependente> dependentes;


	public Cliente() {
	}

	public Cliente(Integer idCliente, String nome, String
email) {
		this.idCliente = idCliente;
		this.nome = nome;
		this.email = email;
	}

  //toString
	public Integer getIdCliente() {
		return idCliente;
	}
```

```
	@Override
	public String toString() {
		return "Cliente [idCliente=" + idCliente + ", nome="
+ nome + ", email=" + email + ", valorPlano=" + valorPlano + ",
totalPago=" + totalPago + "]";
	}

	public void setIdCliente(Integer idCliente) {
		this.idCliente = idCliente;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public Endereco getEndereco() {
		return endereco;
	}
	public void setEndereco(Endereco endereco) {
		this.endereco = endereco;
	}
	public List<Dependente> getDependentes() {
		return dependentes;
	}
	public void setDependentes(List<Dependente> dependentes) {
		this.dependentes = dependentes;
	}
                                    //var args
	public void adicionar(Dependente ...d){
		if (dependentes == null){
			//interface      Classe
		  dependentes = new ArrayList<Dependente>();
		}
		for(Dependente x : d){
			dependentes.add(x);
		}
```

```
	}

	public Double getValorPlano() {
		return valorPlano;
	}
	public void setValorPlano(Double valorPlano) {
		this.valorPlano = valorPlano;
	}
	public Double getTotalPago() {
		return totalPago;
	}
	public void setTotalPago(Double totalPago) {
		this.totalPago = totalPago;
	}
}
```

```
package entity;

import java.util.Date;
import java.text.SimpleDateFormat;

public class Dependente {

	public static final SimpleDateFormat SDF = new
SimpleDateFormat("yyyy-MM-dd");

	private Integer idDependente;
	private String nome;
	private String parentesco;
	private String sexo;
	private Date dataAniversario;

	private Cliente cliente;

	public Dependente() {
	}

	public Dependente(Integer idDependente, String nome, String
parentesco, String sexo, Date dataAniversario) {
		this.idDependente = idDependente;
		this.nome = nome;
		this.parentesco = parentesco;
```

```
		this.sexo = sexo;
		this.dataAniversario = dataAniversario;
	}

	@Override
	public String toString() {
		return "Dependente [idDependente=" + idDependente +
", nome=" + nome + ", parentesco=" + parentesco + ", sexo=" +
sexo + ", dataAniversario=" + SDF.format(dataAniversario) + "]";
	}

	public Integer getIdDependente() {
		return idDependente;
	}
	public void setIdDependente(Integer idDependente) {
		this.idDependente = idDependente;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getParentesco() {
		return parentesco;
	}
	public void setParentesco(String parentesco) {
		this.parentesco = parentesco;
	}
	public String getSexo() {
		return sexo;
	}
	public void setSexo(String sexo) {
		this.sexo = sexo;
	}
	public Date getDataAniversario() {
		return dataAniversario;
	}
	public void setDataAniversario(Date dataAniversario) {
		this.dataAniversario = dataAniversario;
	}
	public Cliente getCliente() {
		return cliente;
```

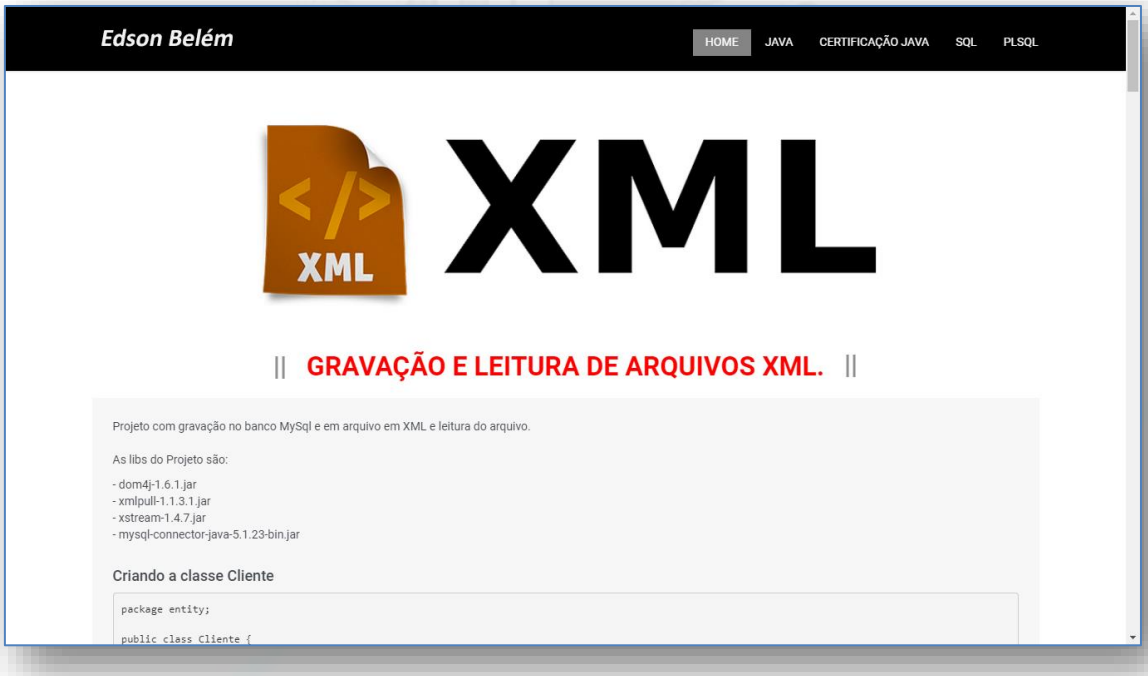
```
	}
	public void setCliente(Cliente cliente) {
		this.cliente = cliente;
	}
}
```

```
package entity;

public class Endereco {

	private Integer  idEndereco;
	private String   bairro;
	private String   cidade;

	private Cliente cliente;

	public Endereco(Integer idEndereco, String bairro, String
cidade) {
		this.idEndereco = idEndereco;
		this.bairro = bairro;
		this.cidade = cidade;
	}

	public Endereco() {

	}

	@Override
	public String toString() {
		return "Endereco [idEndereco=" + idEndereco + ",
bairro=" + bairro + ", cidade=" + cidade + "]";
	}

	public Integer getIdEndereco() {
		return idEndereco;
	}
	public void setIdEndereco(Integer idEndereco) {
		this.idEndereco = idEndereco;
	}
	public String getBairro() {
		return bairro;
```

```
	}
	public void setBairro(String bairro) {
		this.bairro = bairro;
	}
	public String getCidade() {
		return cidade;
	}
	public void setCidade(String cidade) {
		this.cidade = cidade;
	}
	public Cliente getCliente() {
		return cliente;
	}
	public void setCliente(Cliente cliente) {
		this.cliente = cliente;
	}
}
```

---

http://blogedsonbelem.com.br/blog/java/artigos/grav_leit_xml.html



http://blogedsonbelem.com.br/blog/java/artigos/interf.html

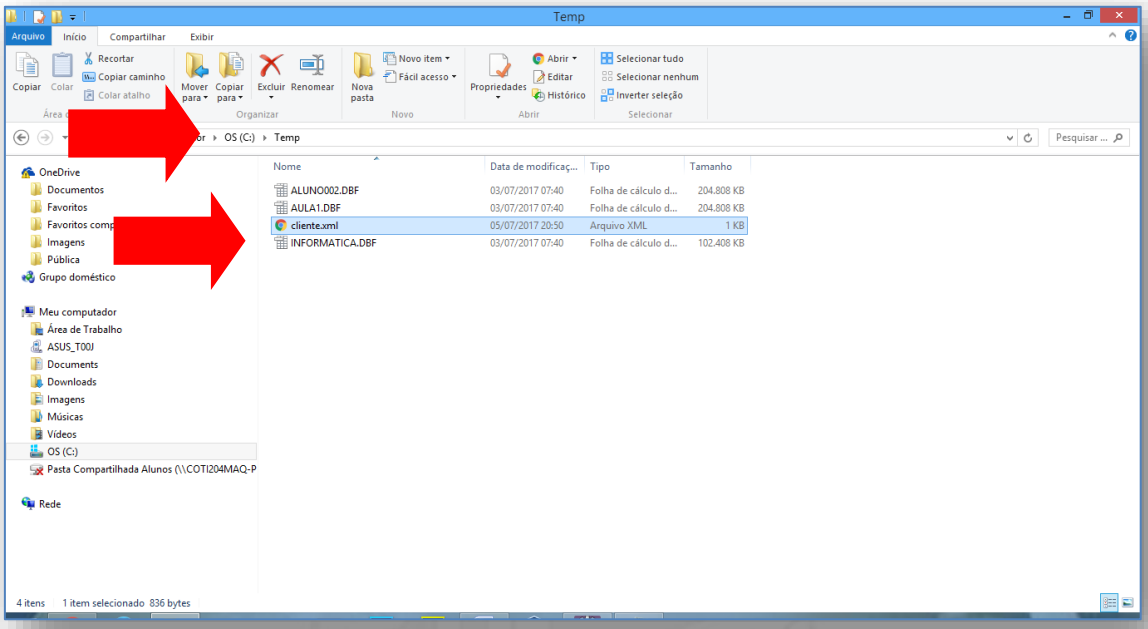
```
package io;

import java.util.List;

import entity.Cliente;

public interface IEscreverArquivoXML {

	public void open() throws Exception;

	public  void  writeXml(List<Cliente> clientes) throws
Exception;

	public void close() throws Exception;

}
```

```
package io;

import java.io.FileWriter;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import com.thoughtworks.xstream.XStream;
import com.thoughtworks.xstream.io.xml.DomDriver;

import entity.Cliente;
import entity.Dependente;
import entity.Endereco;

public class EscreverArquivoXML
implements IEscreverArquivoXML {

      FileWriter fw;

      @Override
      public void open() throws Exception {
       fw = new FileWriter("/tmp/cliente.xml");
      }

      @Override
      public void writeXml(List<Cliente> clientes) throws
Exception {
        if (clientes==null)
              throw new Exception("Lista Vazia ...");
//ele nem comeÃ§a ...
//iso-8859-1 Ã© melhor UT8
      XStream xstream = new XStream(new DomDriver("iso-8859-1"));
                  xstream.alias("dados", List.class);
                  xstream.alias("cliente", Cliente.class);
               fw.write(xstream.toXML(clientes));
               fw.flush();
      }

       @Override
       public void close() throws Exception {
            fw.close();
       }
```

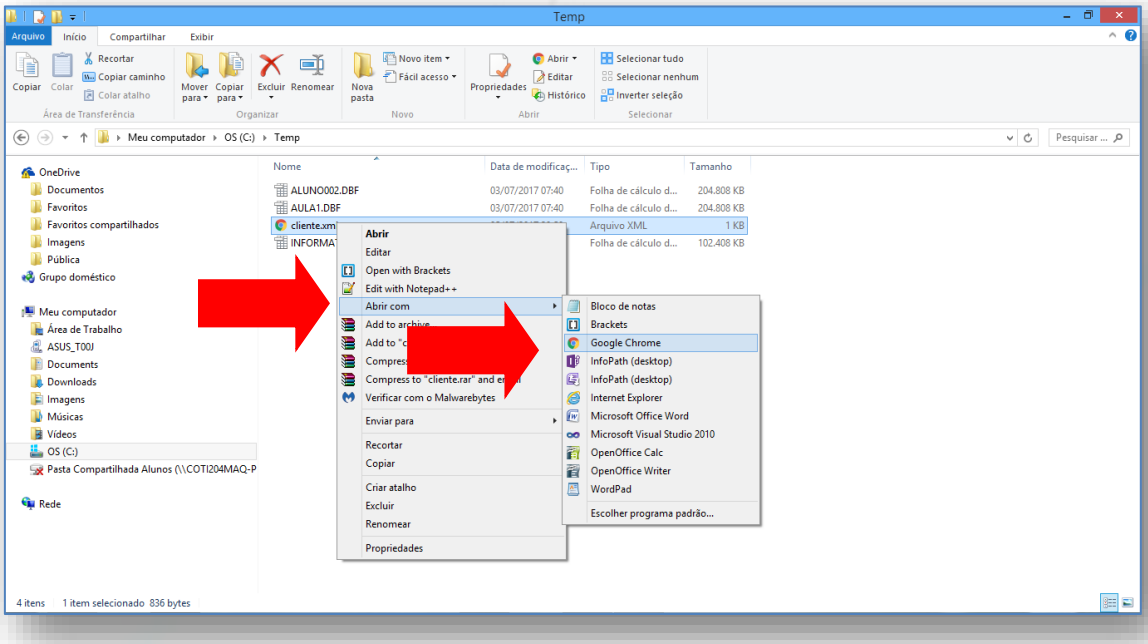
```
        public static void main(String[] args) {
              try{
       Cliente co2 = new Cliente(10,"carlos","carlos@gmail.com");
       Endereco e1= new Endereco(100,"tijuca","Rio de Janeiro");
            Dependente x1 =
             new Dependente(1,"maria","filha","f", new Date());
            Dependente x2 =
 new Dependente(2,"joÃ£o","filho","m", new Date("01/20/2007"));
             co2.setEndereco(e1); //quando for um seta
             co2.adicionar(      x1     ,     x2        );
//adicionar
              List<Cliente> lst1 =new ArrayList<Cliente>();
                        lst1.add(co2);
             EscreverArquivoXML arq  = new EscreverArquivoXML();
                        arq.open();
                        arq.writeXml(lst1);
                        arq.close();
              System.out.println(" Xml Gerado com Sucesso");
             }catch(Exception ex){
                  ex.printStackTrace();
             }
       }
}
```

No console...

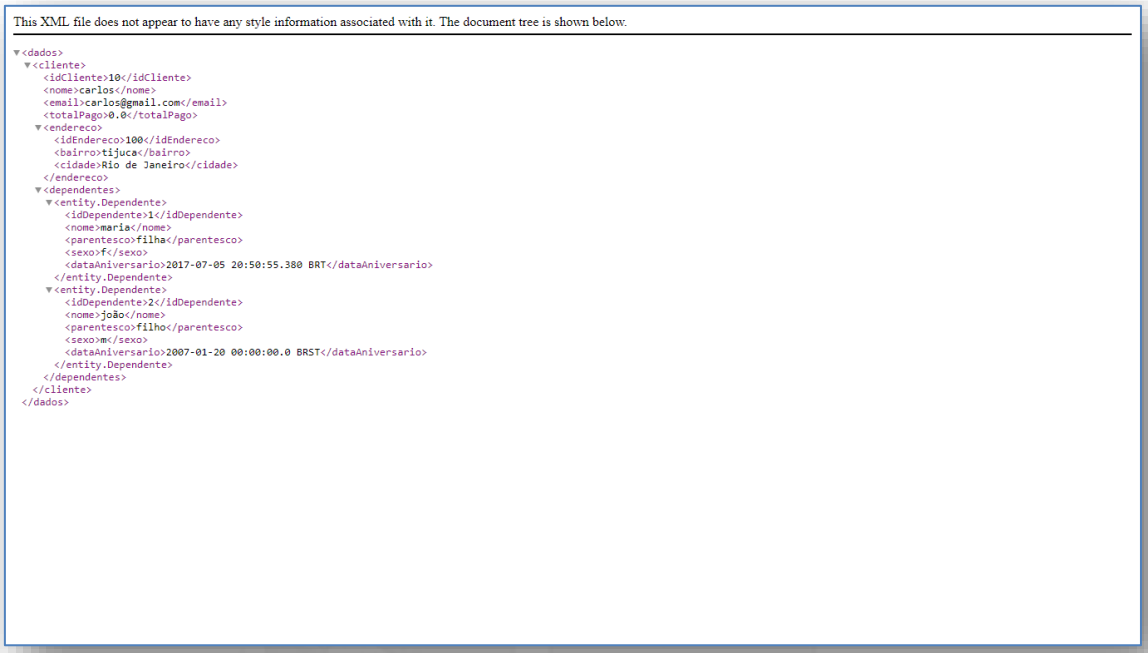
```
Xml Gerado com Sucesso
No explorer...
```




```
Abrir com o chrome...
```



No chrome...




```
package control;

import static java.lang.Math.cos;
import static java.lang.Math.sin;
import static java.lang.Math.tan;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import entity.Cliente;
import entity.Dependente;
import entity.Endereco;

public class CalcularPlano {

    public static void calcularPlano(Cliente c,List<Dependente>
lista,Double taxa){
        c.setValorPlano(taxa);
        c.setDependentes(lista);
```

```
		for(int i=0;i< (lista.size()+ 1);i++){
	c.setTotalPago(c.getTotalPago() + c.getValorPlano());
		}
	}

	public static void calcularPlano(Cliente c, Double taxa){
		c.setValorPlano(taxa);
		for(int i=0;i< (c.getDependentes().size()+ 1);i++){
		c.setTotalPago(c.getTotalPago() + c.getValorPlano());
		}
	}

	public static void main(String[] args) {

		System.out.println(sin(0));//0
		System.out.println(cos(0));//1
		System.out.println(tan(0));//0
		System.out.println(Math.pow(2,3));
		System.out.println(Math.sqrt(100));
		System.out.println(Math.abs(-100));
		System.out.println(Math.exp(2));
		System.out.println(Math.floor(9.9)); //ignora a casa
decimal
		System.out.println(Math.ceil(9.01)); //ir para cima
		System.out.println(Math.round(9.4));

	Cliente c = new Cliente(10,"joao","josoa@gmail.com");
	Endereco e = new Endereco(200,"taquara","jacarepagua");
	Dependente d1 = new Dependente(10,"maria","filha","f",new
Date());
	Dependente d2 = new Dependente(11,"carlos","filho","m",new
Date());
	//essa lista sÃ³ tem valor para o primeiro mÃ©todo ...
	List<Dependente> lista = new ArrayList<Dependente>();
		lista.add(d1);
		lista.add(d2);
		 c.setEndereco(e);
		 c.setDependentes(lista);

	CalcularPlano cp = new CalcularPlano();
	   cp.calcularPlano(c,         lista         , 300.);
		System.out.println(c);
	System.out.println("valor do Plano :" + c.getValorPlano());
```

```
        System.out.println("Total do Plano :" + c.getTotalPago());
        System.out.println("------------------------------");

        Cliente co2 = new Cliente(10,"carlos","carlos@gmail.com");
        Endereco e1= new Endereco(100,"tijuca","Rio de Janeiro");
             Dependente x1 =
               new Dependente(1,"maria","filha","f", new Date());
           Dependente x2 =
    new Dependente(2,"joao","filho","m", new Date("01/20/2007"));
           co2.setEndereco(e1);
           co2.adicionar(      x1     ,     x2        );
       CalcularPlano.calcularPlano(co2,  250.);
        System.out.println("Cliente:"+ co2);
        System.out.println("Valor do Plano:"+ co2.getValorPlano());
        System.out.println("Total do Plano:"+ co2.getTotalPago());
        System.out.println(co2.getDependentes());
        System.out.println(co2.getEndereco());
        }
}
```

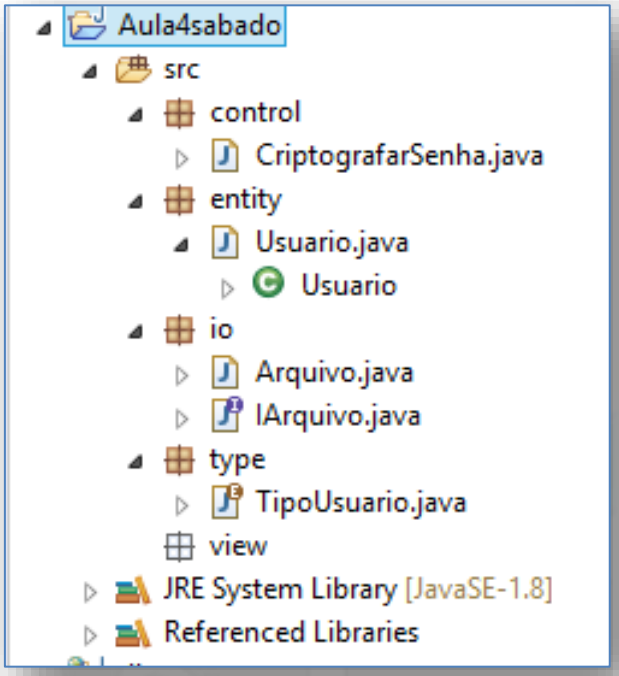
No console...

```
0.0
1.0
0.0
8.0
10.0
100
7.38905609893065
9.0
10.0
9
Cliente [idCliente=10, nome=joao, email=josoa@gmail.com,
valorPlano=300.0, totalPago=900.0]
valor do Plano :300.0
Total do Plano :900.0
------------------------------
Cliente:Cliente [idCliente=10, nome=carlos,
email=carlos@gmail.com, valorPlano=250.0, totalPago=750.0]
Valor do Plano:250.0
Total do Plano:750.0
```

```
[Dependente [idDependente=1, nome=maria, parentesco=filha,
sexo=f, dataAniversario=2017-07-05], Dependente [idDependente=2,
nome=joao, parentesco=filho, sexo=m, dataAniversario=2007-01-
20]]
Endereco [idEndereco=100, bairro=tijuca, cidade=Rio de Janeiro]
```

Escola de Nerds
www.cotiinformatica.com.br

Estrutura do projeto:




```
package entity;

import java.io.Serializable;

import type.TipoUsuario;

public class Usuario  implements Serializable {
	//Gravar o Objeto (persistir)

	private static final long serialVersionUID = 1L;

	private Integer      idUsuario;
	private String       nome;
	private String       email;
	private TipoUsuario  tipo; //enum
	private String       senha;
	private transient  Integer      ativo;
```

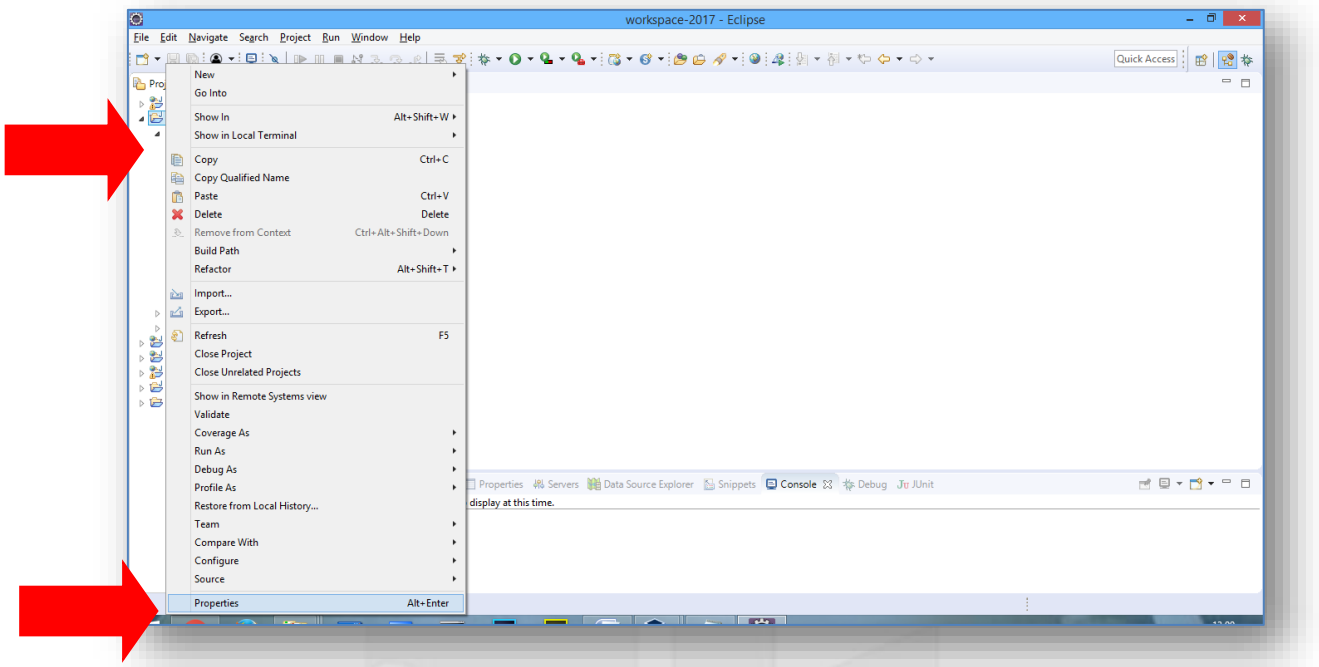
```
	public Usuario() {
	}

	public Usuario(Integer idUsuario, String nome, String
email, TipoUsuario tipo, String senha) {
		this.idUsuario = idUsuario;
		this.nome = nome;
		this.email = email;
		this.tipo = tipo;
		this.senha = senha;
	}

	@Override
	public String toString() {
		return "Usuario [idUsuario=" + idUsuario + ", nome="
+ nome + ", email=" + email + ", tipo=" + tipo + ", senha=" +
senha + "]";
	}

	public Integer getIdUsuario() {
		return idUsuario;
	}
	public void setIdUsuario(Integer idUsuario) {
		this.idUsuario = idUsuario;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public TipoUsuario getTipo() {
		return tipo;
	}
	public void setTipo(TipoUsuario tipo) {
		this.tipo = tipo;
```

```
	}
	public String getSenha() {
		return senha;
	}
	public void setSenha(String senha) {
		this.senha = senha;
	}
	public Integer getAtivo() {
		return ativo;
	}
	public void setAtivo(Integer ativo) {
		this.ativo = ativo;
	}
}
```

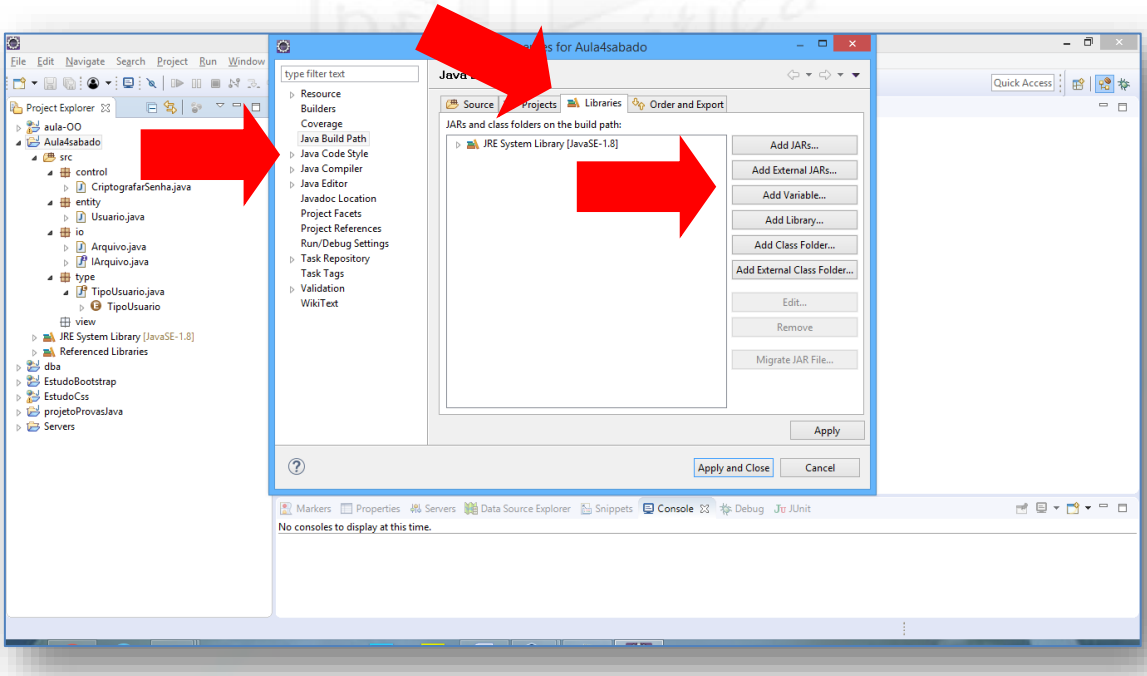
```
package type;

public enum TipoUsuario {

	USUARIO,ADMINISTRADOR;

}
```

Para colocar as libs:

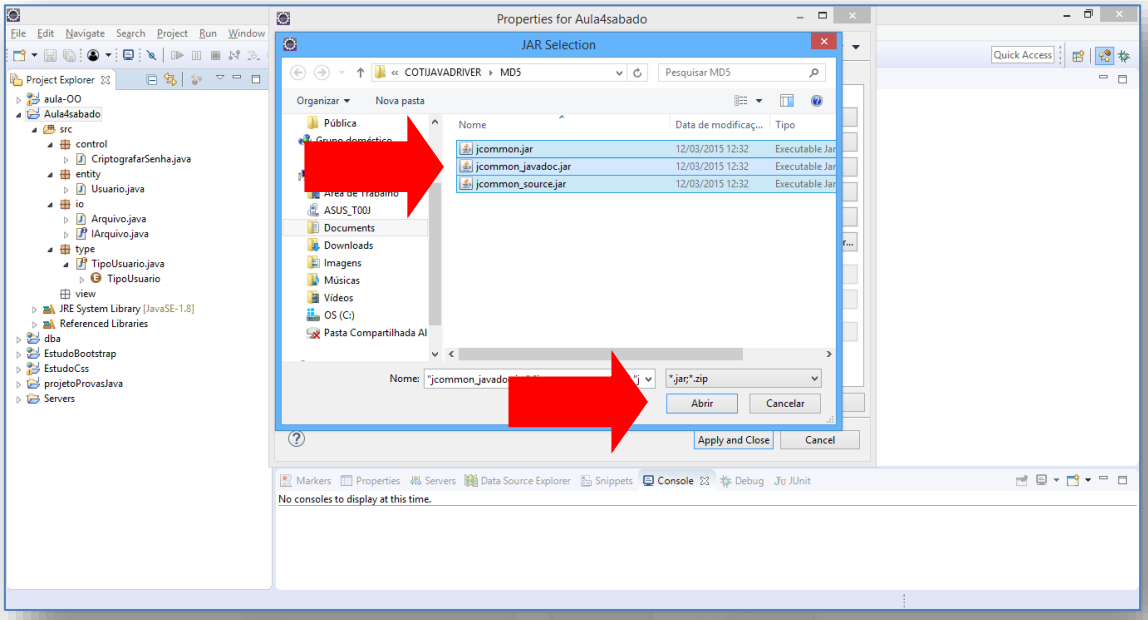
Clicar no projeto com o botão direito -> properties...



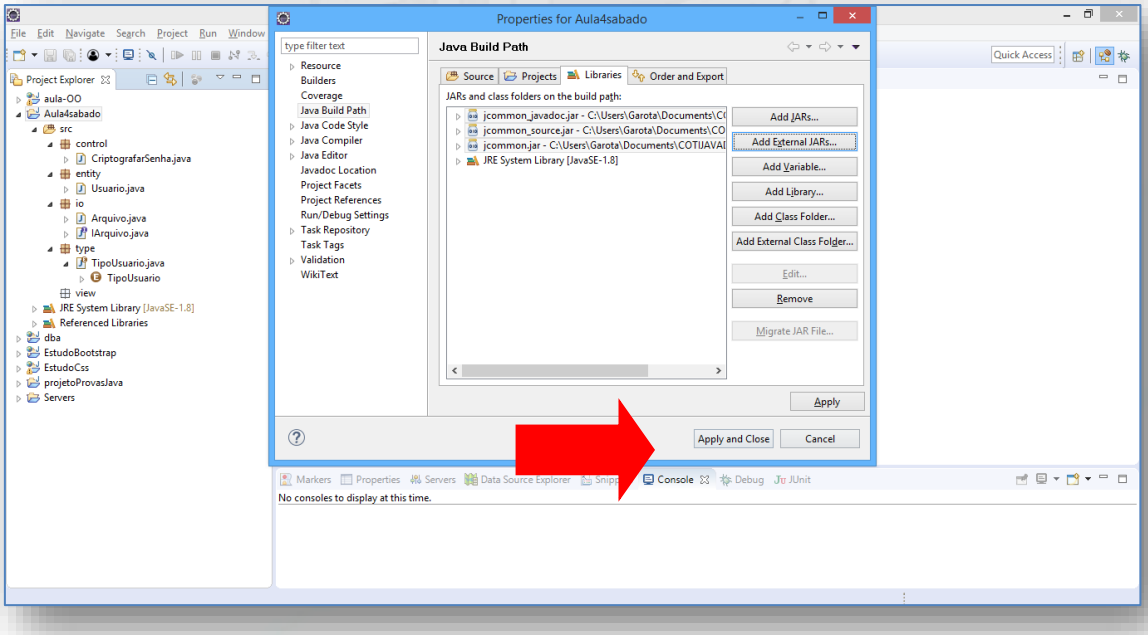
Clicar em Java build path -> na aba Libraries -> Add External Jar...



Selecionar as libs do MD5 -> abrir...



Apply and close....



```
package control;

import org.jcommon.encryption.SimpleMD5;

import entity.Usuario;
import type.TipoUsuario;

public class CriptografarSenha {

//profesdonbelem@gmail.com123456789!$TESTE
      //MD5 Ã© fraco ...

     public static void criptografar(Usuario u){
          SimpleMD5 md5 = new SimpleMD5(u.getSenha(),
           "profesdonbelem@gmail.com123456789!$TESTE");
          u.setSenha(md5.toHexString());
     }


 public static void main(String[] args) {

      Usuario u = new
Usuario(100,"lu","lu@gmail.com",TipoUsuario.USUARIO,"123");
      CriptografarSenha.criptografar(u);
      System.out.println(u.getNome() + "," + u.getSenha());
}
}
```

No console...

```
lu,11a443c21f5fa837257138f705ae1d44
```

```
package io;

import entity.Usuario;
```

```
public interface IArquivo {

	public void openWrite() throws Exception;

	public void writeDados(Usuario u) throws Exception;

	public void closeWrite() throws Exception;

	//gravar e buscar ...

    public void openRead() throws Exception;

	public Object readDados() throws Exception;

	public void closeRead() throws Exception;
}
```

```
package io;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

import control.CriptografarSenha;
import entity.Usuario;
import type.TipoUsuario;

public class Arquivo implements IArquivo {

	FileOutputStream  fos;
	ObjectOutputStream  oos;

	@Override
	public void openWrite() throws Exception {
		fos = new FileOutputStream("/tmp/usuario.dat");
		oos = new ObjectOutputStream(fos);
```

```
}
@Override
public void writeDados(Usuario u) throws Exception {
   oos.writeObject(u); //nao grava transient
   oos.flush();
}
@Override
public void closeWrite() throws Exception {
      oos.close();
      fos.close();
}

FileInputStream  fis;
ObjectInputStream  ois;

@Override
public void openRead() throws Exception {
      fis = new FileInputStream("/tmp/usuario.dat");
      ois = new ObjectInputStream(fis);
}

@Override
public Object readDados() throws Exception {
      return ois.readObject();
}

@Override
public void closeRead() throws Exception {
   ois.close();
   fis.close();
}

public static void main(String[] args) {
      Usuario u =
            new Usuario(100, "ilan","ilan@gmail.com",
                        TipoUsuario.USUARIO, "123");
               u.setAtivo(1);
         CriptografarSenha.criptografar(u);
      IArquivo arq = new Arquivo();
       try{
            arq.openWrite(); //abre
```

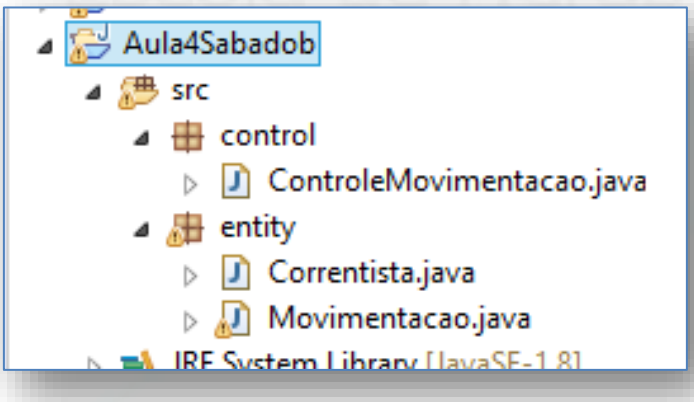
```
                arq.writeDados(u); //grava
                arq.closeWrite(); //fecha

                arq.openRead();
                Usuario resp = (Usuario) arq.readDados();
                System.out.println(resp);
                System.out.println(resp.getAtivo());
                arq.closeRead();
            }catch(Exception ex){
                ex.printStackTrace();
            }
        }
}
```

No console...

```
Usuario [idUsuario=100, nome=ilan, email=ilan@gmail.com,
tipo=USUARIO, senha=11a443c21f5fa837257138f705ae1d44]
null
```

OUTRO PROJETO...

Estrutura do projeto...



```
package entity;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class Correntista implements Serializable {

	private static final long serialVersionUID = 1L;

	private  Integer idCorrentista;
	private  String   nome;
	private String    email;
	private Double    saldo = 0.;

	//O Relacionamento entre Correntista e Movimentacao
	//Um para Muitos
	//OneToMany
	private List<Movimentacao> movimentacoes;
//Correntista
	//tem uma Lista de Movimentacao
	//chamado movimentacoes ...
	//Um para mUitos

	public Correntista() {
	}

	public Correntista(Integer idCorrentista, String nome,
String email, Double saldo) {
		super();
		this.idCorrentista = idCorrentista;
		this.nome = nome;
		this.email = email;
		this.saldo = saldo;
	}

	@Override
	public String toString() {
```

```
		return "Correntista [idCorrentista=" + idCorrentista
+ ", nome=" + nome + ", email=" + email + ", saldo=" + saldo +
"]";
	}

	public Integer getIdCorrentista() {
		return idCorrentista;
	}
	public void setIdCorrentista(Integer idCorrentista) {
		this.idCorrentista = idCorrentista;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public Double getSaldo() {
		return saldo;
	}
	public void setSaldo(Double saldo) {
		this.saldo = saldo;
	}

	public List<Movimentacao> getMovimentacoes() {
		return movimentacoes;
	}
	public void setMovimentacoes(List<Movimentacao>
movimentacoes) {
		this.movimentacoes = movimentacoes;
	}

    public void adicionar(Movimentacao ...m){
	  if (movimentacoes ==null){
		movimentacoes = new ArrayList<Movimentacao>();
	  }
```

```
		for (Movimentacao x : m){
			movimentacoes.add(x);
		}
	}
}
```

```
package entity;

import java.io.Serializable;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;

public class Movimentacao implements Serializable{

	private static final long serialVersionUID = 1L;

	public static SimpleDateFormat SDF =
			new SimpleDateFormat("dd/MM/yyyy hh:mm:ss");

	private Integer idMovimentacao;
	private String  tipo;
	private Double  deposito;
	private Double  retirada;
	private Date    dataMovimentacao;

	//ManyToOne
	private Correntista correntista;

	public Movimentacao() {

	}

	public Movimentacao(Integer idMovimentacao, String tipo,
Double deposito, Double retirada, Date dataMovimentacao) {
		super();
		this.idMovimentacao = idMovimentacao;
		this.tipo = tipo;
```

```
		this.deposito = deposito;
		this.retirada = retirada;
		this.dataMovimentacao = dataMovimentacao;
	}

	@Override
	public String toString() {
		return "Movimentacao [idMovimentacao=" +
idMovimentacao + ", tipo=" + tipo + ", deposito=" + deposito +
", retirada=" + retirada + ", dataMovimentacao=" +
 SDF.format(dataMovimentacao) + "]";
	}

	public Integer getIdMovimentacao() {
		return idMovimentacao;
	}
	public void setIdMovimentacao(Integer idMovimentacao) {
		this.idMovimentacao = idMovimentacao;
	}
	public String getTipo() {
		return tipo;
	}
	public void setTipo(String tipo) {
		this.tipo = tipo;
	}
	public Double getDeposito() {
		return deposito;
	}
	public void setDeposito(Double deposito) {
		this.deposito = deposito;
	}
	public Double getRetirada() {
		return retirada;
	}
	public void setRetirada(Double retirada) {
		this.retirada = retirada;
	}
	public Date getDataMovimentacao() {
		return dataMovimentacao;
	}
	public void setDataMovimentacao(Date dataMovimentacao) {
		this.dataMovimentacao = dataMovimentacao;
```

```
	}
	public Correntista getCorrentista() {
		return correntista;
	}
	public void setCorrentista(Correntista correntista) {
		this.correntista = correntista;
	}
}
```

```
package control;

import java.util.Date;

import entity.Correntista;
import entity.Movimentacao;

public class ControleMovimentacao {

  public void calcularDeposito(Correntista c,Movimentacao m){
		c.setSaldo(c.getSaldo() + m.getDeposito());
		 if (m.getDeposito()>=10000){
	 System.out.println("Radar da Receita Federal ...");
		 }
		 m.setDataMovimentacao(new Date());
		 m.setTipo("deposito");
	  }


	public void calcularRetirada(Correntista c,Movimentacao m){
	  if (c.getSaldo()<m.getRetirada()){
			System.out.println("Saldo insuficiente ...");
	  }
	  else{
			c.setSaldo( c.getSaldo() - m.getRetirada());
	  }
	  m.setDataMovimentacao(new Date());
		  m.setTipo("retirada");
	 }
	 //transferencia
	 //acrescentrar juros
	 //operacao com limite ...
```

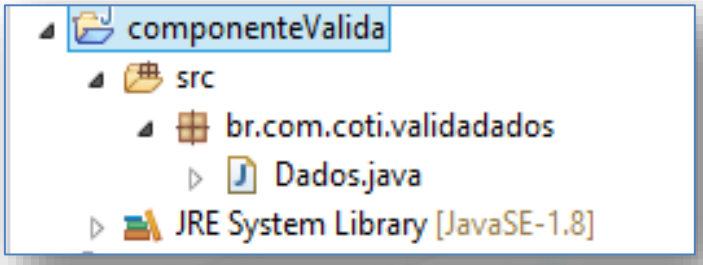
```
        public static void main(String[] args) {

         try{
          Correntista c=
              new Correntista(1001,"joao","joao@gmail.com", 1000.);
          Movimentacao m1 =
     new Movimentacao(500, "deposito",5000.,  0.,new Date());
          Movimentacao m2 =
      new Movimentacao(501, "deposito",300.,  0.,new Date());
          Movimentacao m3 =
       new Movimentacao(502, "retirada",0.,  9000.,new Date());
        ControleMovimentacao cm = new ControleMovimentacao();
          cm.calcularDeposito(c, m1);
          cm.calcularDeposito(c, m2);
          cm.calcularRetirada(c, m3);
          c.adicionar(m1 , m2 , m3);
            System.out.println(c);
          System.out.println(c.getMovimentacoes());
          }catch(Exception ex){
           ex.printStackTrace();
          }
        }
}
```

No console...

```
Saldo insuficiente ...
Correntista [idCorrentista=1001, nome=joao,
email=joao@gmail.com, saldo=6300.0]
[Movimentacao [idMovimentacao=500, tipo=deposito,
deposito=5000.0, retirada=0.0, dataMovimentacao=10/07/2017
01:21:16], Movimentacao [idMovimentacao=501, tipo=deposito,
deposito=300.0, retirada=0.0, dataMovimentacao=10/07/2017
01:21:16], Movimentacao [idMovimentacao=502, tipo=retirada,
deposito=0.0, retirada=9000.0, dataMovimentacao=10/07/2017
01:21:16]]
```

PROJETO DE CRIAÇÃO DO COMPONENTE:

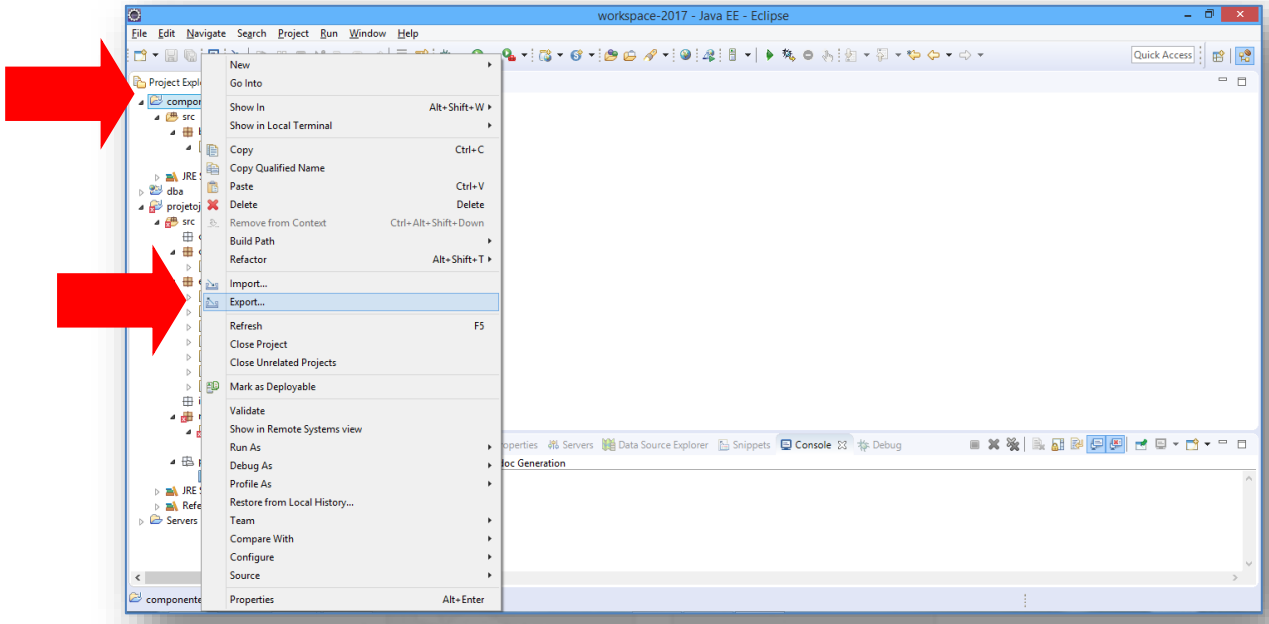
Estrutura do projeto:



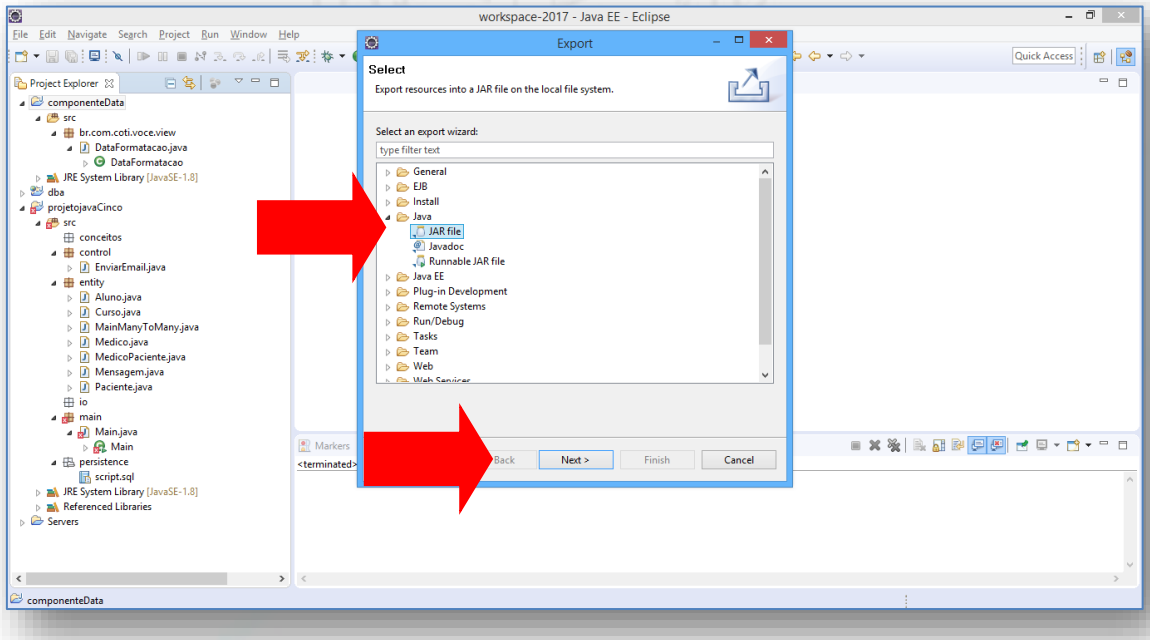

```
package br.com.coti.validadados;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Dados {

	public Boolean isNome(String nome){
		Pattern p = Pattern.compile("[a-z A-Z]{2,50}");
		Matcher m = p.matcher(nome);
		return m.matches();
	}
	public Boolean isEmail(String email){
		Pattern p=Pattern.compile(".+@.+\\.[a-z]+");
		Matcher m = p.matcher(email);
		return m.matches();
	}
	public Boolean isSexo(String sexo){
		Pattern p=Pattern.compile("M|F|m|f");
		Matcher m = p.matcher(sexo);
		return m.matches();
	}


}
```

Para criar o componente...

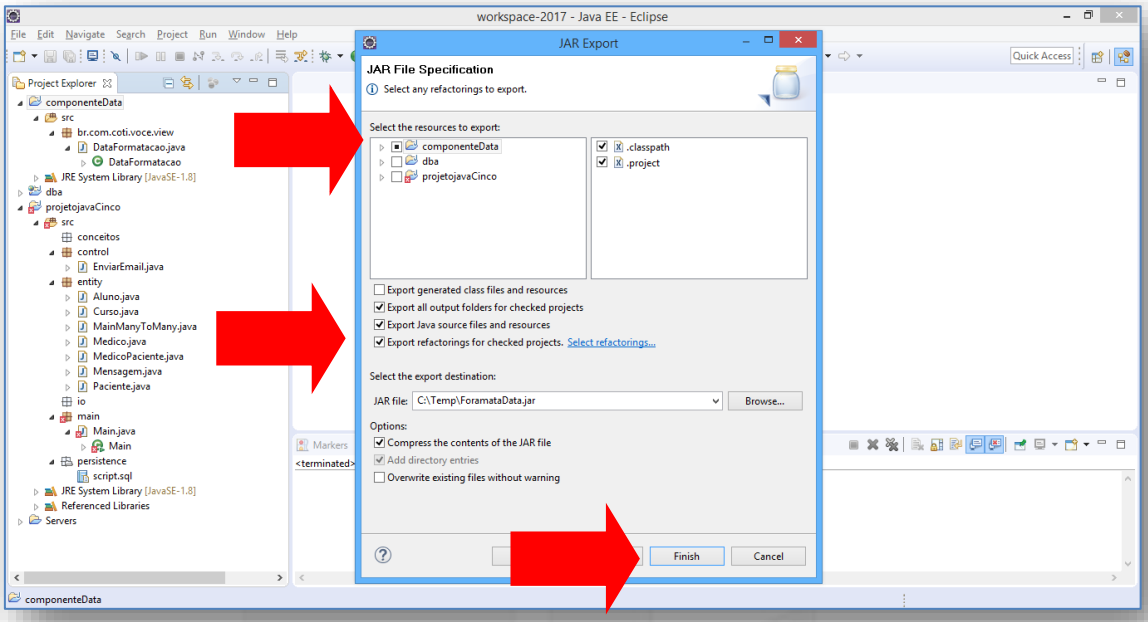
Clicar no projeto com o botão direito -> Export...



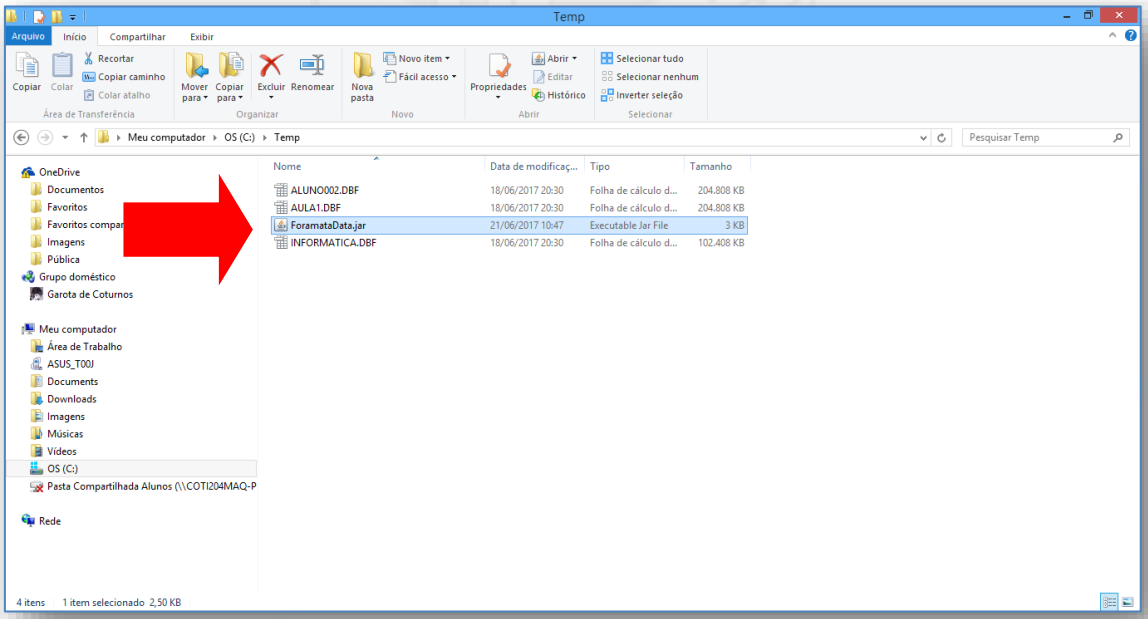
Java -> JAR File – Next...



Marcar o projeto que irá ser exportado -> Finish...

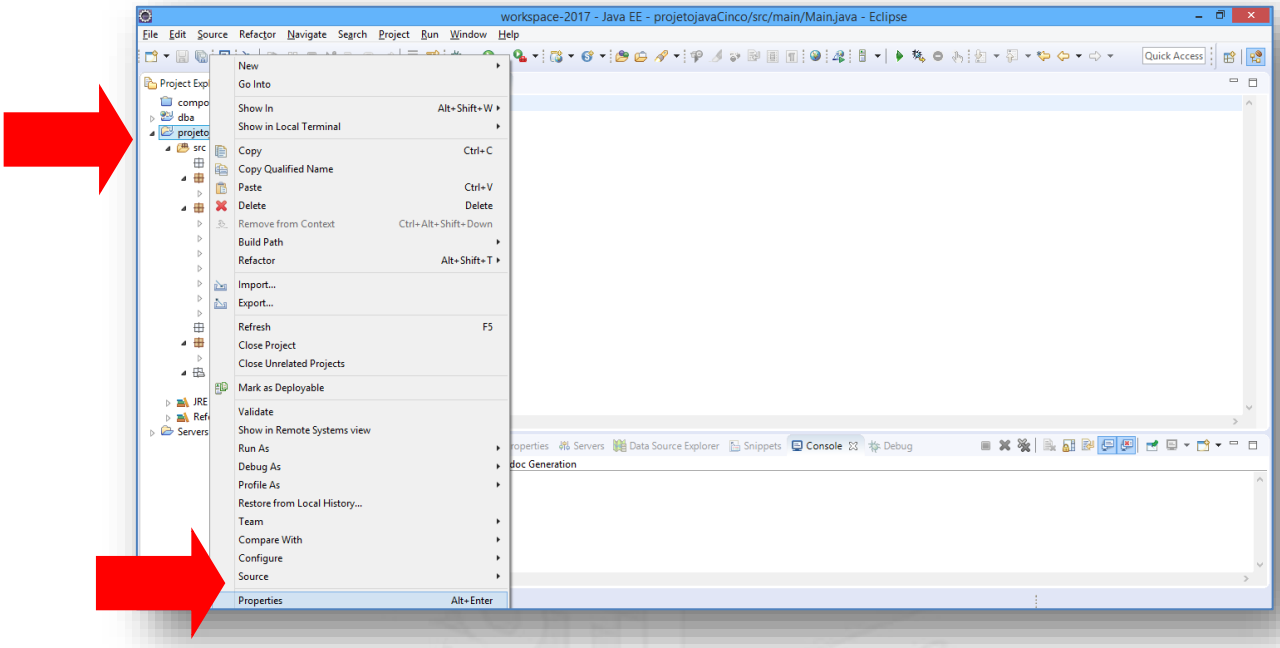


No Explorer...

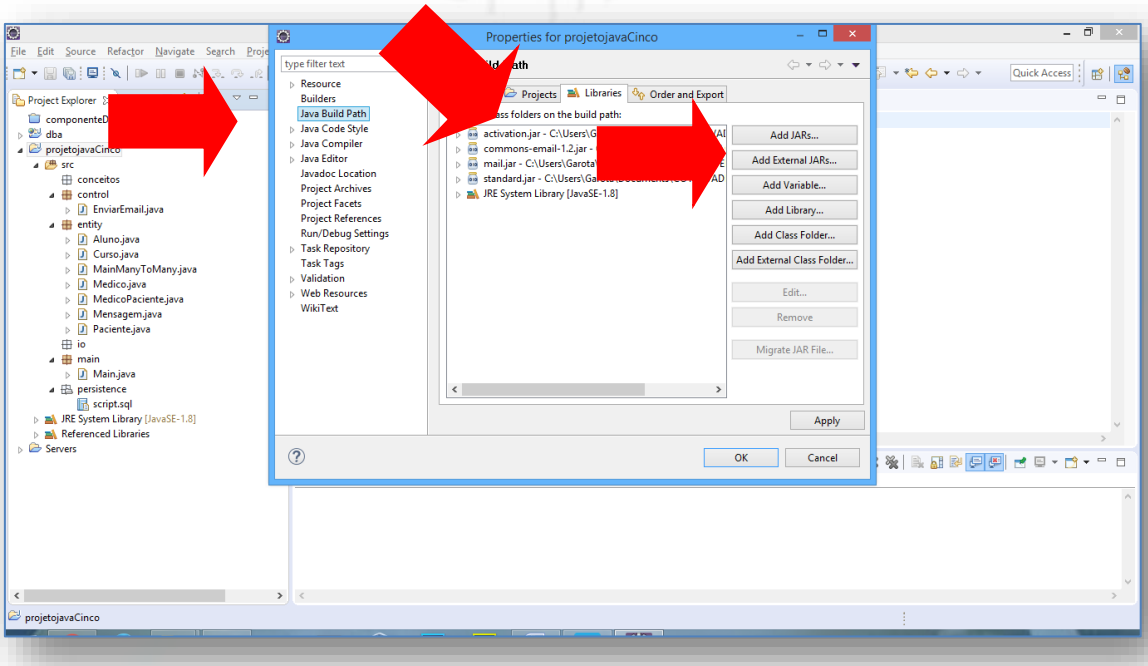


Voltando para o projeto anterior e incluindo o componente....

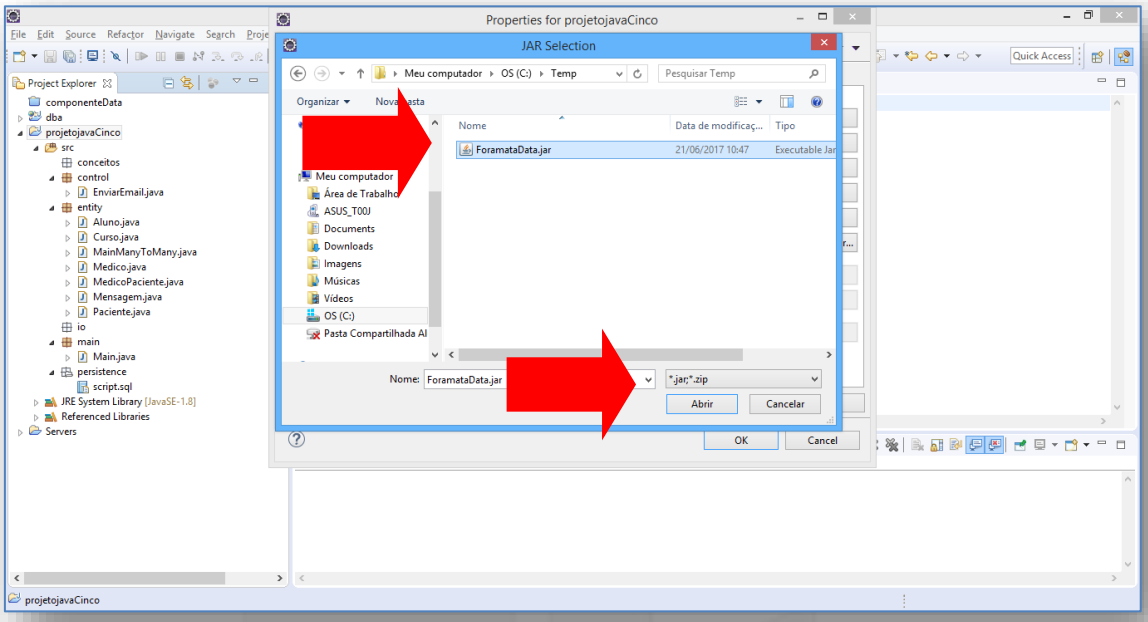
Clicar no projeto -> Properties...



Java BuilPath _. Libraries -> Add External jar...

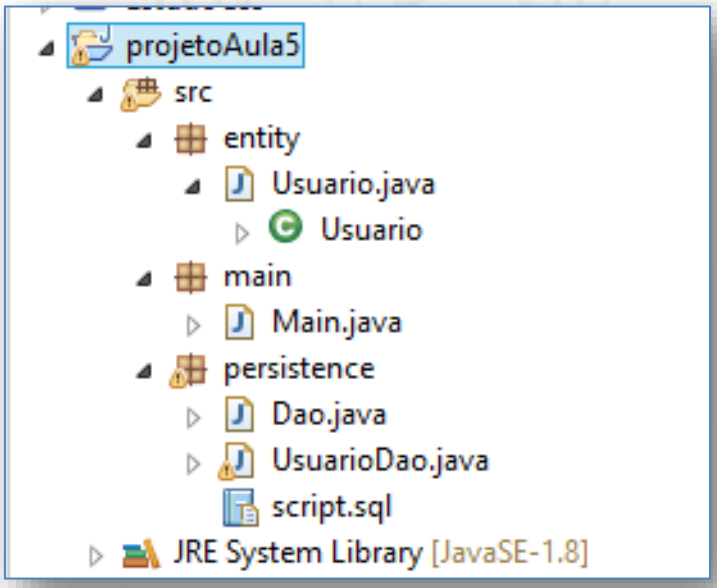


Abrir na pasta onde está o componente -> Abrir...



Novo projeto:

Estrutura do projeto...



```
package entity;

public class Usuario {

	private Integer idUsuario;
	private String  nome;
	private String  email;
	private String  sexo;
	private String  perfil;
	private String  senha;

	public Usuario() {

	}

	public Usuario(Integer idUsuario, String nome, String
email, String sexo, String perfil, String senha) {
		super();
		this.idUsuario = idUsuario;
		this.nome = nome;
		this.email = email;
		this.sexo = sexo;
		this.perfil = perfil;
		this.senha = senha;
	}

	@Override
	public String toString() {
		return "Usuario [idUsuario=" + idUsuario + ", nome="
+ nome + ", email=" + email + ", sexo=" + sexo  + ", perfil=" +
perfil + "]";
	}

	public Integer getIdUsuario() {
		return idUsuario;
	}
	public void setIdUsuario(Integer idUsuario) {
		this.idUsuario = idUsuario;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
```

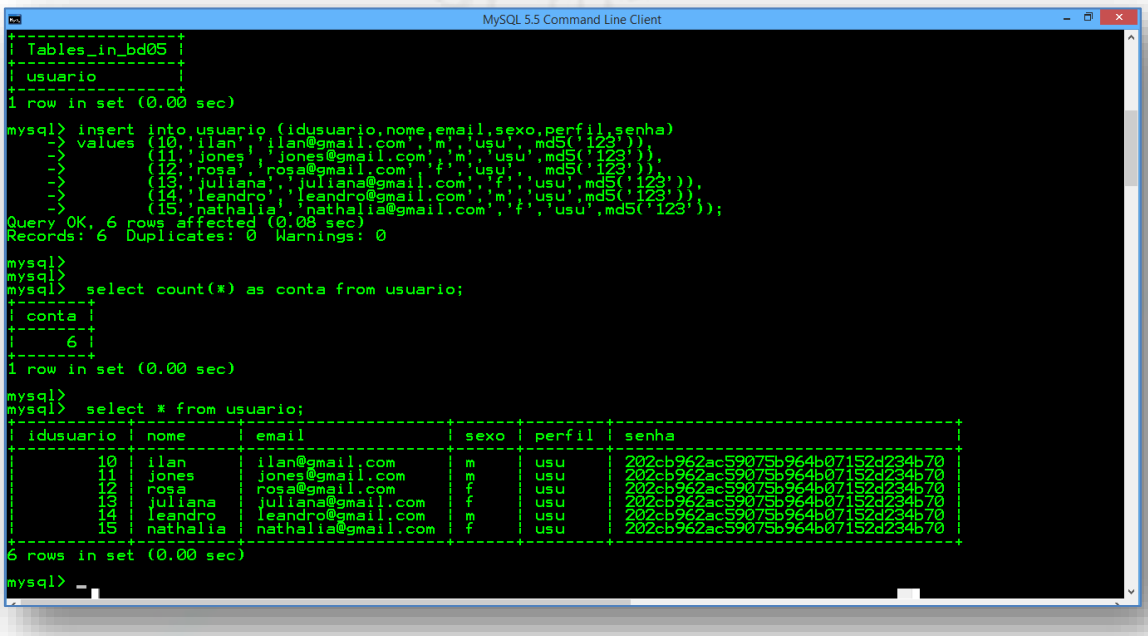
```
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public String getSexo() {
		return sexo;
	}
	public void setSexo(String sexo) {
		this.sexo = sexo;
	}
	public String getPerfil() {
		return perfil;
	}
	public void setPerfil(String perfil) {
		this.perfil = perfil;
	}
	public String getSenha() {
		return senha;
	}
	public void setSenha(String senha) {
		this.senha = senha;
	}
}
```

---

script.sql

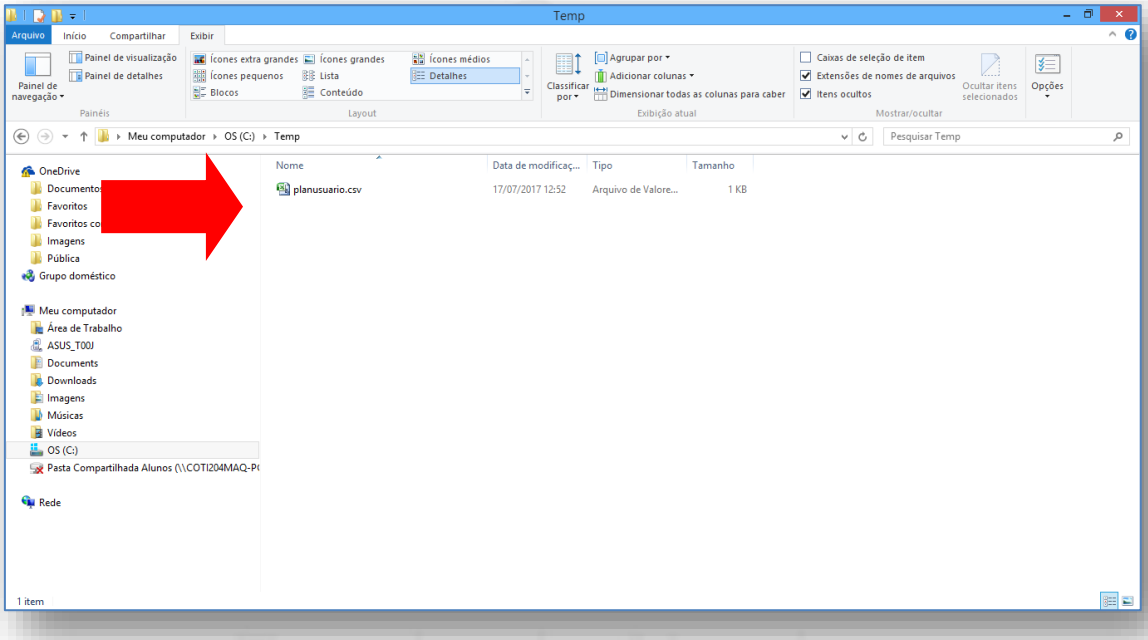
```
#so aqui
# Aqui da ruim
#### drop database if exists bd05;
#eu nao ensinei
#nao facam  o de cima ....

create database bd05;
use bd05;

#se conectar
# comandos como delete,truncate (esquecer) ...
```

```
create table usuario(
  idusuario int primary key auto_increment,
  nome varchar (50) not null,
  email varchar (60) not null unique,
  sexo   enum ('m','f') not null,
  perfil enum ('adm','usu') not null,
  senha varchar (200)
);

show tables;

insert into usuario (idusuario,nome,email,sexo,perfil,senha)
values (10,'ilan','ilan@gmail.com','m','usu', md5('123')),
       (11,'jones','jones@gmail.com','m','usu',md5('123')),
       (12,'rosa','rosa@gmail.com','f','usu',  md5('123')),
       (13,'juliana','juliana@gmail.com','f','usu',md5('123')),
       (14,'leandro','leandro@gmail.com','m','usu',md5('123')),

(15,'nathalia','nathalia@gmail.com','f','usu',md5('123'));


 select count(*) as conta from usuario;

 select * from usuario;
```



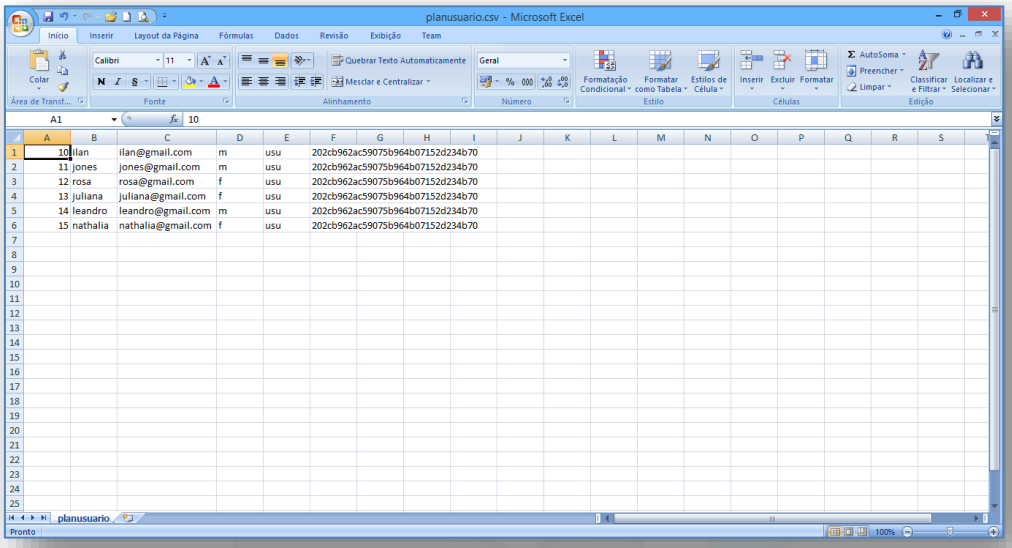
```
--exportando os dados (gerar a planilha)
select idusuario,nome,email, sexo, perfil, senha
        into outfile '/tmp/planusuario.csv'
        fields terminated by ';'
        lines terminated by '\n'
        from usuario;
```

No Explorer...



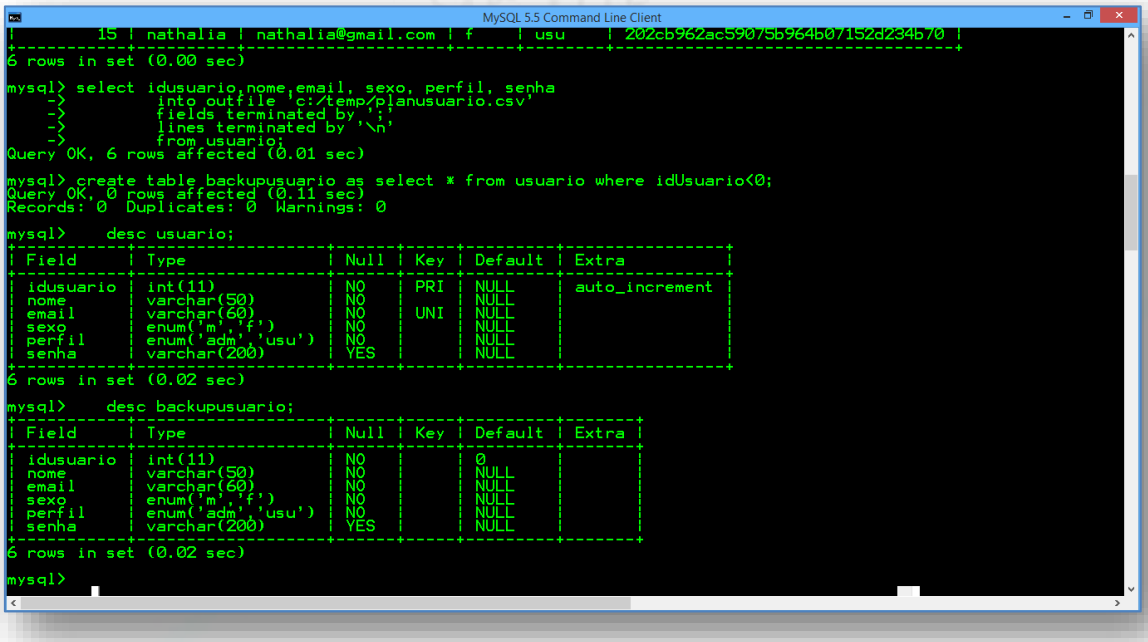
Escola de Nerds
www.cotiinformatica.com.br

No Excel...

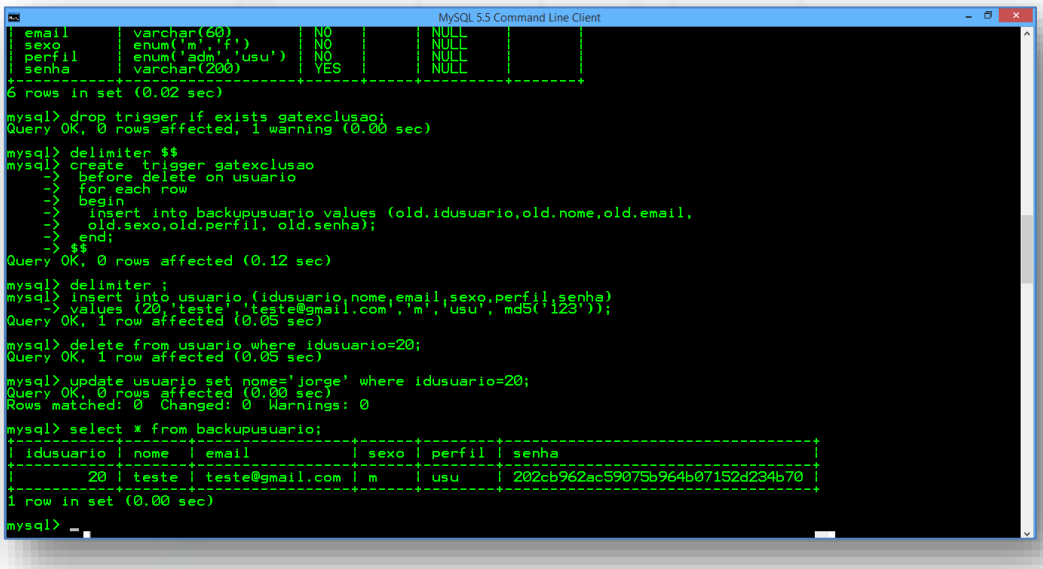



```
create table backupusuario as select * from usuario where
idUsuario<0;
```

```
 # idusuario <0 nao existe entao ele irea criar uma tabela vazia
   desc usuario;
   desc backupusuario;
```



```
drop trigger if exists gatexclusao;

delimiter $$
create  trigger gatexclusao
 before delete on usuario
 for each row
 begin
       insert into backupusuario values
(old.idusuario,old.nome,old.email,
       old.sexo,old.perfil, old.senha);
 end;
$$
delimiter ;

insert into usuario (idusuario,nome,email,sexo,perfil,senha)
values (20,'teste','teste@gmail.com','m','usu', md5('123'));

#delete e update sempre com where

delete from usuario where idusuario=20;
update usuario set nome='jorge' where idusuario=20;

throws Exception
select * from backupusuario;
```



```
package persistence;

import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;

//data access object
public class Dao {

	Connection con; //banco
	PreparedStatement stmt; //tabela (crud)
	Statement  state; //tabela (ddl)
	ResultSet rs; //Consulta
	CallableStatement call; // Strored Procedure

	public void open() throws Exception{
		Class.forName("com.mysql.jdbc.Driver");
		con = DriverManager.getConnection
("jdbc:mysql://localhost:3306/bd05","root","coti");
	}

	public void close() throws Exception{
		con.close();
	}
}
```

```
package persistence;

import java.sql.PreparedStatement;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import entity.Usuario;

public class UsuarioDao extends Dao{
```

```
      public void create(Usuario u) throws Exception{
        open();
         stmt = con.prepareStatement("insert into usuario values
(null,?,?,?,?, md5(?) )" ,
PreparedStatement.RETURN_GENERATED_KEYS);
         stmt.setString(1, u.getNome());
         stmt.setString(2, u.getEmail());
         stmt.setString(3, u.getSexo());
         stmt.setString(4, u.getPerfil());
         stmt.setString(5, u.getSenha());
         stmt.execute();
         rs = stmt.getGeneratedKeys();
         rs.next();
         //qual o numero da chave que ele estÃ¡ buscando ?
        u.setIdUsuario(   rs.getInt(1)   ); //qual o numero que
foi gerado
       close();
      }

      public void exportaExcel() throws Exception{
             open();
 SimpleDateFormat sdf = new SimpleDateFormat("yyyyMMddHHmmss");
             String msg ="select idusuario,nome,email, sexo,
perfil,senha "+
    " into outfile '/tmp/"+ sdf.format(new Date())
+"usuario.csv' "+ "  fields terminated by ';' " + "  lines
terminated by '\n' " +  " from usuario";
        state = con.createStatement();
        rs = state.executeQuery(msg);
            close();
      }

      public List<Usuario> findAll() throws Exception{
           open();
        stmt = con.prepareStatement("select * from usuario");
             //rs=stmt.executeQuery();
             rs = stmt.executeQuery();
//existe nos programas de consulta
             List<Usuario> lst= new ArrayList<Usuario>();
             while(rs.next()){ //enquanto existir faÃ§a
                 Usuario u = new Usuario(
                                 rs.getInt(1) ,
```

```
                                    rs.getString(2),
                                    rs.getString(3),
                                    rs.getString(4),
                                    rs.getString(5),
                                    rs.getString(6)
                                );
                    lst.add(u);
            }
                close();
                return lst;
        }

        public static void main(String[] args) {
                Usuario u = new Usuario(null,"edson",
"edson@gmail.com","m","usu","123");
                try{
                    UsuarioDao dao = new UsuarioDao();
                    dao.findAll().forEach(System.out::println);
                       //java 8
                    System.out.println(dao.findAll());
                }catch(Exception ex){
                    System.out.println("Error :" + ex.getMessage());
                }
        }
}
```

No console...

```
Usuario [idUsuario=10, nome=ilan, email=ilan@gmail.com, sexo=m,
perfil=usu]
Usuario [idUsuario=11, nome=jones, email=jones@gmail.com,
sexo=m, perfil=usu]
Usuario [idUsuario=12, nome=rosa, email=rosa@gmail.com, sexo=f,
perfil=usu]
Usuario [idUsuario=13, nome=juliana, email=juliana@gmail.com,
sexo=f, perfil=usu]
Usuario [idUsuario=14, nome=leandro, email=leandro@gmail.com,
sexo=m, perfil=usu]
Usuario [idUsuario=15, nome=nathalia, email=nathalia@gmail.com,
sexo=f, perfil=usu]
[Usuario [idUsuario=10, nome=ilan, email=ilan@gmail.com, sexo=m,
perfil=usu], Usuario [idUsuario=11, nome=jones,
```

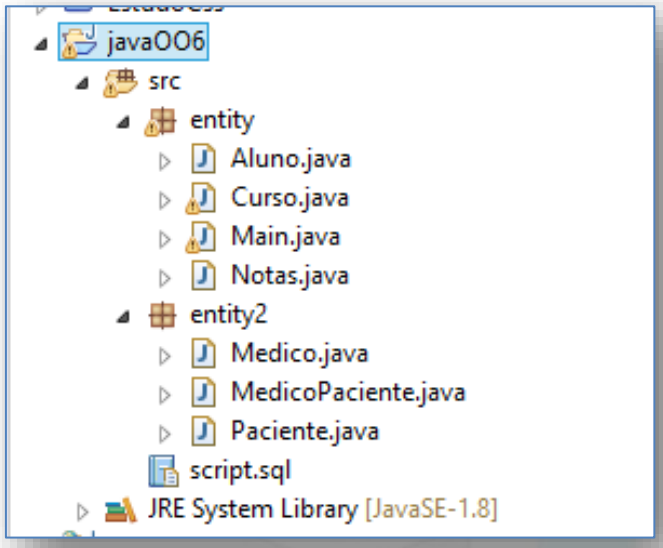
```
email=jones@gmail.com, sexo=m, perfil=usu], Usuario
[idUsuario=12, nome=rosa, email=rosa@gmail.com, sexo=f,
perfil=usu], Usuario [idUsuario=13, nome=juliana,
email=juliana@gmail.com, sexo=f, perfil=usu], Usuario
[idUsuario=14, nome=leandro, email=leandro@gmail.com, sexo=m,
perfil=usu], Usuario [idUsuario=15, nome=nathalia,
email=nathalia@gmail.com, sexo=f, perfil=usu]]
```

```
package main;

import br.com.coti.validadados.Dados;
import entity.Usuario;

public class Main {

	public static void main(String[] args) {

		Dados d = new Dados();
		Usuario u = new Usuario();
		try{
			u.setIdUsuario(100);
			u.setPerfil("usu");
			u.setNome("luciana");
			u.setEmail("lu@gmail.com");
			u.setSexo("f");
			u.setSenha("123");
	 System.out.println(d.isNome(    u.getNome()      ));
	 System.out.println(d.isSexo(    u.getSexo()    ));
	 System.out.println(d.isEmail(   u.getEmail()   ));
		}catch(Exception ex){

		}
	}
}
```

No console...

```
true
true
true
```

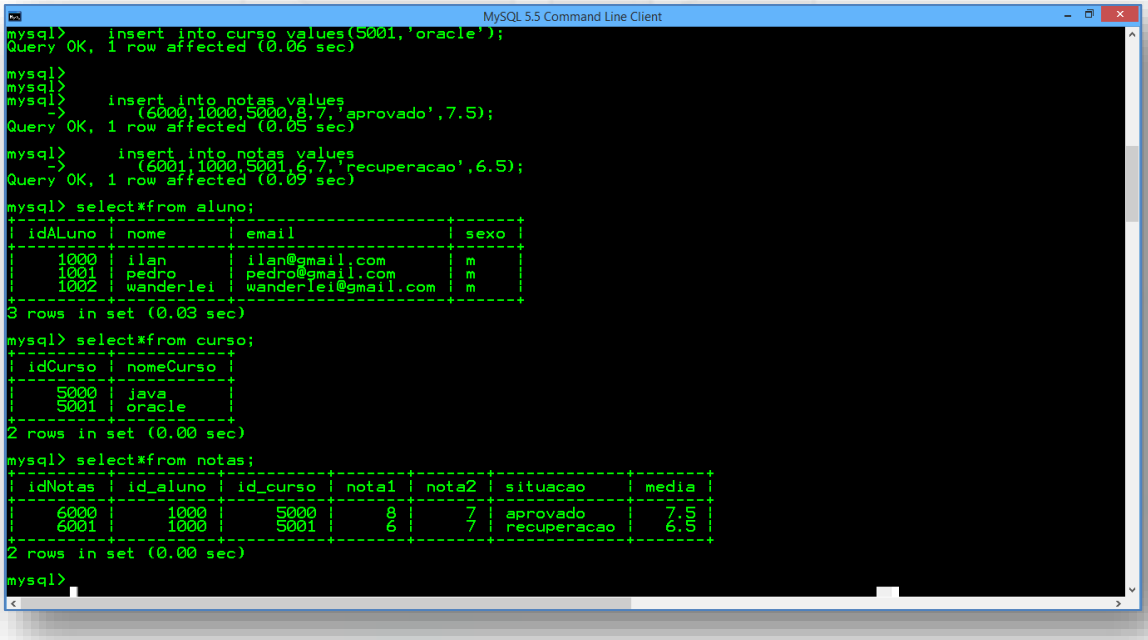
Estrutura do projeto:



script.sql

```
create database banco6;

use banco6;


create table aluno(idALuno int primary key,
                   nome varchar (50),
                   email varchar (50) unique,
                   sexo varchar (1));
 create table curso(idCurso int primary key,
                   nomeCurso varchar (50)
                   );

create table notas (idNotas int primary key,
                   id_aluno int,
                   id_curso int,
                   nota1 float,
                   nota2 float,
                   situacao varchar (15),
                   media float,
   foreign key(id_aluno) references aluno(idaluno),
   foreign key(id_curso) references curso(idcurso));
```

```
   insert into aluno values(1000,'ilan','ilan@gmail.com','m');
   insert into aluno values(1001,'pedro','pedro@gmail.com','m');
   insert into aluno
values(1002,'wanderlei','wanderlei@gmail.com','m');
   insert into curso values(5000,'java');
   insert into curso values(5001,'oracle');


   insert into notas values
      (6000,1000,5000,8,7,'aprovado',7.5);
    insert into notas values
      (6001,1000,5001,6,7,'recuperacao',6.5);

select*from aluno;

select*from curso;

select*from notas;
```



```
package entity;

public class Aluno {

	private Integer idAluno;
	private String nome;
	private String email;
	private String sexo;

	public Aluno() {

	}

	public Integer getIdAluno() {
		return idAluno;
	}
	public Aluno(Integer idAluno, String nome, String email,
String sexo) {
		this.idAluno = idAluno;
		this.nome = nome;
		this.email = email;
		this.sexo = sexo;
	}

	@Override
	public String toString() {
		return "Aluno [idAluno=" + idAluno + ", nome=" + nome
+ ", email=" + email + ", sexo=" + sexo + "]";
	}

	public void setIdAluno(Integer idAluno) {
		this.idAluno = idAluno;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
```

```
	}
	public String getSexo() {
		return sexo;
	}
	public void setSexo(String sexo) {
		this.sexo = sexo;
	}
}
```

```
package entity;

import java.util.List;

public class Curso {

	private Integer idCurso;
	private String  nomeCurso;

	public Curso() {

	}

	public Curso(Integer idCurso, String nomeCurso) {
		this.idCurso = idCurso;
		this.nomeCurso = nomeCurso;
	}

	@Override
	public String toString() {
		return "Curso [idCurso=" + idCurso + ", nomeCurso=" +
nomeCurso + "]";
	}

	public Integer getIdCurso() {
		return idCurso;
	}
	public void setIdCurso(Integer idCurso) {
		this.idCurso = idCurso;
	}
	public String getNomeCurso() {
		return nomeCurso;
	}
	public void setNomeCurso(String nomeCurso) {
```

```
            this.nomeCurso = nomeCurso;
      }
}
```

```
package entity;

public class Notas {

      private Integer idNotas;
      private Double nota1;
      private Double nota2;
      private Double media;
      private String situacao;

      private Aluno aluno;
      private Curso curso;

      public Notas(Integer idNotas, Double nota1, Double nota2,
Double media, String situacao) {
            this.idNotas = idNotas;
            this.nota1 = nota1;
            this.nota2 = nota2;
            this.media = media;
            this.situacao = situacao;
      }

      public Notas() {
      }

      @Override
      public String toString() {
            return "Notas [idNotas=" + idNotas + ", nota1=" +
nota1 + ", nota2=" + nota2 + ", media=" + media + ", situacao="
+ situacao + ", aluno=" + aluno + ", curso=" + curso + "]";
      }

      public Integer getIdNotas() {
            return idNotas;
      }
      public void setIdNotas(Integer idNotas) {
            this.idNotas = idNotas;
      }
      public Double getNota1() {
```

```
		return nota1;
	}
	public void setNota1(Double nota1) {
		this.nota1 = nota1;
	}
	public Double getNota2() {
		return nota2;
	}
	public void setNota2(Double nota2) {
		this.nota2 = nota2;
	}
	public Double getMedia() {
		return media;
	}
	public void setMedia(Double media) {
		this.media = media;
	}
	public String getSituacao() {
		return situacao;
	}
	public void setSituacao(String situacao) {
		this.situacao = situacao;
	}
	public Aluno getAluno() {
		return aluno;
	}
	public void setAluno(Aluno aluno) {
		this.aluno = aluno;
	}
	public Curso getCurso() {
		return curso;
	}
	public void setCurso(Curso curso) {
		this.curso = curso;
	}
}
```

```
package entity;

public class Main {

	public static void main(String[] args) {

		Aluno a1 = new Aluno(10,"jose","jose@gmail.com","m");
		Aluno a2 = new
Aluno(11,"frutinha","frutinha@gmail.com","f");
		Aluno a3 = new
Aluno(12,"loide","loide@gmail.com","m");

		Curso c1 = new Curso(1000,"java");
		Curso c2 = new Curso(2000,"Oracle");

		Notas n1 = new Notas(3000,5.,6.,5.5,"recuperacao");
			n1.setAluno(a1);
			n1.setCurso(c1);
		Notas n2 = new Notas(3001,7.,8.,7.5,"aprovado");
			n2.setAluno(a1);
			n2.setCurso(c2);
	}

}
```

```
package entity2;

import java.util.ArrayList;
import java.util.List;

public class Medico {

	private  Integer  idMedico;
	private  String   nomeMedico;
	private  String   especializao;
	//ManyToMany

	private List<Paciente> pacientes;
	private List<MedicoPaciente> medicoPacientes;

	public Medico() {
```

```
	}

	public Medico(Integer idMedico, String nomeMedico, String
especializao) {
		this.idMedico = idMedico;
		this.nomeMedico = nomeMedico;
		this.especializao = especializao;
	}

	@Override
	public String toString() {
		return "Medico [idMedico=" + idMedico + ",
nomeMedico=" + nomeMedico + ", especializao=" + especializao +
"]";
	}

	public Integer getIdMedico() {
		return idMedico;
	}
	public void setIdMedico(Integer idMedico) {
		this.idMedico = idMedico;
	}
	public String getNomeMedico() {
		return nomeMedico;
	}
	public void setNomeMedico(String nomeMedico) {
		this.nomeMedico = nomeMedico;
	}
	public String getEspecializao() {
		return especializao;
	}
	public void setEspecializao(String especializao) {
		this.especializao = especializao;
	}
	public List<Paciente> getPacientes() {
		return pacientes;
	}
	public void setPacientes(List<Paciente> pacientes) {
		this.pacientes = pacientes;
	}
	public List<MedicoPaciente> getMedicoPacientes() {
		return medicoPacientes;
	}
	public void setMedicoPacientes(List<MedicoPaciente>
medicoPacientes) {
```

```
            this.medicoPacientes = medicoPacientes;
      }

      public void adicionarMedicoPaciente(Paciente p){
         if (medicoPacientes ==null){
               medicoPacientes = new ArrayList<MedicoPaciente>();
         }
            medicoPacientes.add(new MedicoPaciente(this, p));
      }

      public static void main(String[] args) {
            Medico m1= new Medico(10,"jose","clinico");
            Medico m2= new Medico(11,"carlos","ortopedista");

Paciente p1 = new Paciente(1000,"teste1","teste1@gmail.com");
Paciente p2 = new Paciente(1001,"teste2","teste2@gmail.com");
            m1.adicionarMedicoPaciente(p1);
            m1.adicionarMedicoPaciente(p2);
            m2.adicionarMedicoPaciente(p1);
            System.out.println(m1);
            System.out.println(m1.getMedicoPacientes());
            System.out.println("-----");
            System.out.println(m2);
            System.out.println(m2.getMedicoPacientes());
      }
}
```

No console...

```
Medico [idMedico=10, nomeMedico=jose, especializao=clinico]
[MedicoPaciente [medico=Medico [idMedico=10, nomeMedico=jose,
especializao=clinico], paciente=Paciente [idPaciente=1000,
nome=teste1, email=teste1@gmail.com]], MedicoPaciente
[medico=Medico [idMedico=10, nomeMedico=jose,
especializao=clinico], paciente=Paciente [idPaciente=1001,
nome=teste2, email=teste2@gmail.com]]]
-----
Medico [idMedico=11, nomeMedico=carlos,
especializao=ortopedista]
[MedicoPaciente [medico=Medico [idMedico=11, nomeMedico=carlos,
especializao=ortopedista], paciente=Paciente [idPaciente=1000,
nome=teste1, email=teste1@gmail.com]]]
```

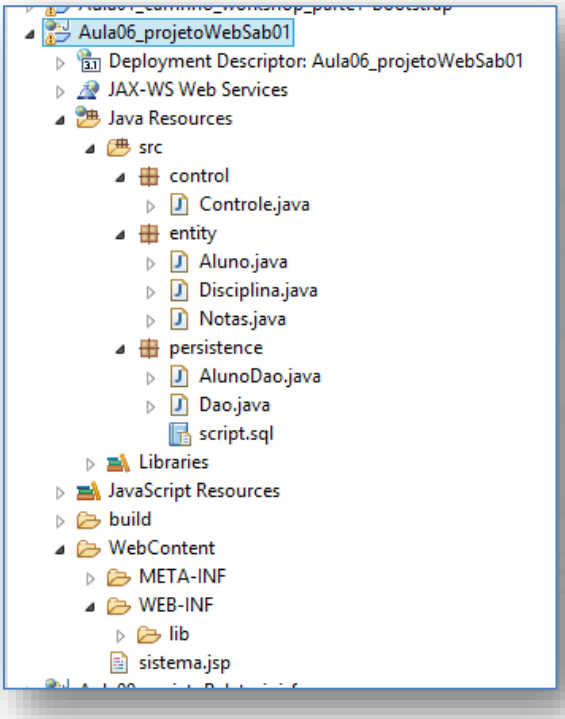
```
package entity2;

import java.util.List;

public class Paciente {

	private Integer idPaciente;
	private String nome;
	private String email;

    private List<Medico> medicos;

	public Paciente() {
	}

	public Paciente(Integer idPaciente, String nome, String
email) {
		this.idPaciente = idPaciente;
		this.nome = nome;
		this.email = email;
	}

	@Override
	public String toString() {
		return "Paciente [idPaciente=" + idPaciente +    ",
nome=" + nome + ", email=" + email + "]";
	}

	public Integer getIdPaciente() {
		return idPaciente;
	}
	public void setIdPaciente(Integer idPaciente) {
		this.idPaciente = idPaciente;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
```

```
	public void setEmail(String email) {
		this.email = email;
	}
	public List<Medico> getMedicos() {
		return medicos;
	}

	public void setMedicos(List<Medico> medicos) {
		this.medicos = medicos;
	}
}
```

```
package entity2;

public class MedicoPaciente {

	private Medico medico;
	private Paciente paciente;

	public MedicoPaciente() {

	}

	public MedicoPaciente(Medico medico, Paciente paciente) {
		this.medico = medico;
		this.paciente = paciente;
	}

	@Override
	public String toString() {
		return "MedicoPaciente [medico=" + medico + ",
paciente=" + paciente + "]";
	}

	public Medico getMedico() {
		return medico;
	}
	public void setMedico(Medico medico) {
		this.medico = medico;
	}
	public Paciente getPaciente() {
		return paciente;
	}
```

```
    public void setPaciente(Paciente paciente) {
        this.paciente = paciente;
    }
}
```

Estrutura do projeto:



LIBS:

gson-2.2.4-javadoc.jar
gson-2.2.4-sources.jar
gson-2.2.4.jar
jstl.jar
ojdbc14-10.2.0.3.jar
servlet3-api.jar
standard.jar

```
package entity;

import java.io.Serializable;

public class Aluno implements Serializable{

	private static final long serialVersionUID =2l;

	private Integer idAluno;
	private String  nome;
	private String  email;
	private String  sexo;
	private Integer ativo;

	public Aluno() {

	}

	public Aluno(Integer idAluno, String nome, String email,
String sexo, Integer ativo) {
		super();
		this.idAluno = idAluno;
		this.nome = nome;
		this.email = email;
		this.sexo = sexo;
		this.ativo = ativo;
	}


	@Override
	public String toString() {
		return "Aluno [idAluno=" + idAluno + ", nome=" + nome
+ ", email=" + email + ", sexo=" + sexo + ", ativo=" + ativo +
"]";
	}

	public Integer getIdAluno() {
		return idAluno;
	}
	public void setIdAluno(Integer idAluno) {
		this.idAluno = idAluno;
	}
```

```
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public String getSexo() {
		return sexo;
	}
	public void setSexo(String sexo) {
		this.sexo = sexo;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
	}
	public Integer getAtivo() {
		return ativo;
	}
	public void setAtivo(Integer ativo) {
		this.ativo = ativo;
	}
}
```

```
package entity;

import java.io.Serializable;

public class Disciplina  implements Serializable{

	private static final long serialVersionUID = 1L;
	private Integer idDisciplina;
	private String nomeDisciplina;
	private Integer ativo;

	public Disciplina() {
```

```
	}

	public Disciplina(Integer idDisciplina, String
nomeDisciplina, Integer ativo) {
		super();
		this.idDisciplina = idDisciplina;
		this.nomeDisciplina = nomeDisciplina;
		this.ativo = ativo;
	}



	@Override
	public String toString() {
		return "Disciplina [idDisciplina=" + idDisciplina +
", nomeDisciplina=" + nomeDisciplina + ", ativo=" + ativo + "]";
	}

	public Integer getIdDisciplina() {
		return idDisciplina;
	}
	public void setIdDisciplina(Integer idDisciplina) {
		this.idDisciplina = idDisciplina;
	}
	public String getNomeDisciplina() {
		return nomeDisciplina;
	}
	public void setNomeDisciplina(String nomeDisciplina) {
		this.nomeDisciplina = nomeDisciplina;
	}
	public Integer getAtivo() {
		return ativo;
	}
	public void setAtivo(Integer ativo) {
		this.ativo = ativo;
	}
}
```

```
package entity;

import java.io.Serializable;

public class Notas implements Serializable{

	private static final long serialVersionUID = 1L;
	private Integer idNotas;
	private Double  nota1;
	private Double  nota2;
	private Double  media;
	private String  situacao;

	public Notas() {
	}

	public Notas(Integer idNotas, Double nota1, Double nota2,
Double media, String situacao) {
		this.idNotas = idNotas;
		this.nota1 = nota1;
		this.nota2 = nota2;
		this.media = media;
		this.situacao = situacao;
	}

	@Override
	public String toString() {
		return "Notas [idNotas=" + idNotas + ", nota1=" +
nota1 + ", nota2=" + nota2 + ", media=" + media + ", situacao="
+ situacao + "]";
	}

	public Integer getIdNotas() {
		return idNotas;
	}
	public void setIdNotas(Integer idNotas) {
		this.idNotas = idNotas;
	}
	public Double getNota1() {
		return nota1;
	}
	public void setNota1(Double nota1) {
```

```
		this.nota1 = nota1;
	}
	public Double getNota2() {
		return nota2;
	}
	public void setNota2(Double nota2) {
		this.nota2 = nota2;
	}
	public Double getMedia() {
		return media;
	}
	public void setMedia(Double media) {
		this.media = media;
	}
	public String getSituacao() {
		return situacao;
	}
	public void setSituacao(String situacao) {
		this.situacao = situacao;
	}
}
```

---

script.sql

```
conn system/coti

show user;

create user alunosab identified by coti
 default tablespace users
 quota 30m on users;

grant create table, create session, create procedure,
      create trigger, create view, create sequence
       to alunosab;

conn alunosab/coti

show user;

create table aluno (
	idAluno number (15) primary key,
	nome varchar2 (35),
```

```
      email varchar2 (50) unique,
      sexo varchar (1) check (sexo in ('m','f','M','F')),
      ativo number (5)
);

show user;
USER Ú "ALUNOSAB"

select * from cat;

TABLE_NAME                     TABLE_TYPE
------------------------------ -----------
ALUNO                          TABLE
SEQ_ALUNO                      SEQUENCE

set linesize 1000;
column nome format a15;
column email format a25;

create sequence seq_aluno;

insert into aluno values
(seq_aluno.nextval,'ilan','ilan@gmail.com', 'm',1);
 --atualizar na tabela ...
 commit;

  select * from aluno;

 IDALUNO NOME            EMAIL                     S      ATIVO
-------- --------------- ------------------------- - ----------
       1 ilan            ilan@gmail.com            m          1

  select seq_aluno.currval from dual;

CURRVAL
-------
      1
```

```
package persistence;

import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
```

```
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;

public class Dao {

	Connection con;
	PreparedStatement stmt;
	ResultSet rs;
	CallableStatement call;
	Statement state;

	 public void open() throws Exception{
		Class.forName("oracle.jdbc.driver.OracleDriver");
		con = DriverManager.
	   getConnection("jdbc:oracle:thin:@127.0.0.1:1521:XE",
				"alunosab","coti");
	 }

	 public void close() throws Exception{
		con.close();
	 }

	public static void main(String[] args) {
		try{
			Dao d = new Dao();
			  d.open();
			  d.close();
		 System.out.println("Banco de Dados Abertos ...");
		}catch(Exception ex){
			ex.printStackTrace();
		}
	}
}
```

No console...

```
Banco de Dados Abertos ...
```

```
package persistence;

import entity.Aluno;

public class AlunoDao extends Dao {

       public void create(Aluno a)throws Exception{
        open();
             stmt = con.prepareStatement
     ("insert into aluno  values  (seq_aluno.nextval,?,?,?,?)");
             stmt.setString(1, a.getNome());
             stmt.setString(2, a.getEmail());
             stmt.setString(3, a.getSexo());
             stmt.setInt(4, a.getAtivo());
            stmt.execute();
        close();
       }


}
```

```
package control;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import entity.Aluno;
import persistence.AlunoDao;


@WebServlet("/Controle")
public class Controle extends
HttpServlet {

     private static final long serialVersionUID = 1L;
```

```
      protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
      }
      protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
            String cmd = request.getParameter("cmd");
            if (cmd.equalsIgnoreCase("gravar")){
                  gravar(request,response);
            }
      }

    protected void gravar(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
           try{
            Aluno a = new Aluno();
                  a.setNome(request.getParameter("nome"));
                  a.setEmail(request.getParameter("email"));
                  a.setSexo(request.getParameter("sexo"));
                a.setAtivo(1);
                 new AlunoDao().create(a);
           request.setAttribute("msg", "Dados Gravados ...");
           }catch(Exception ex){
                  ex.printStackTrace();
      request.setAttribute("msg", "Error:"+ ex.getMessage());
           }
      finally{
           request.getRequestDispatcher("sistema.jsp").
              forward(request, response);
      }
      }
}
```

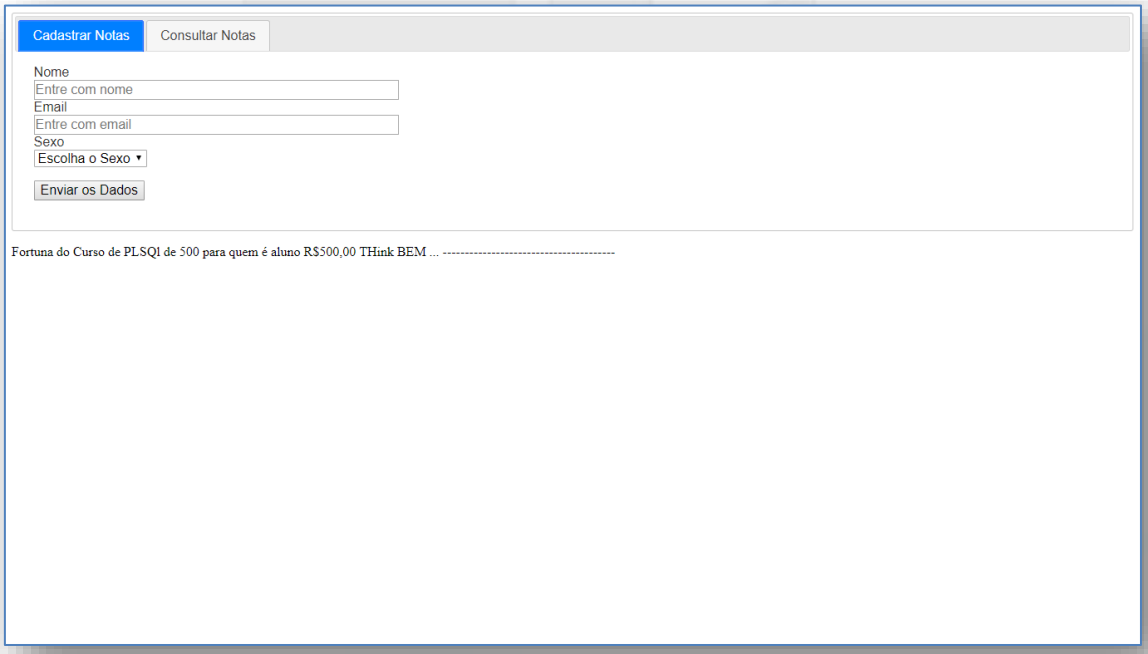
---

sistema.jsp

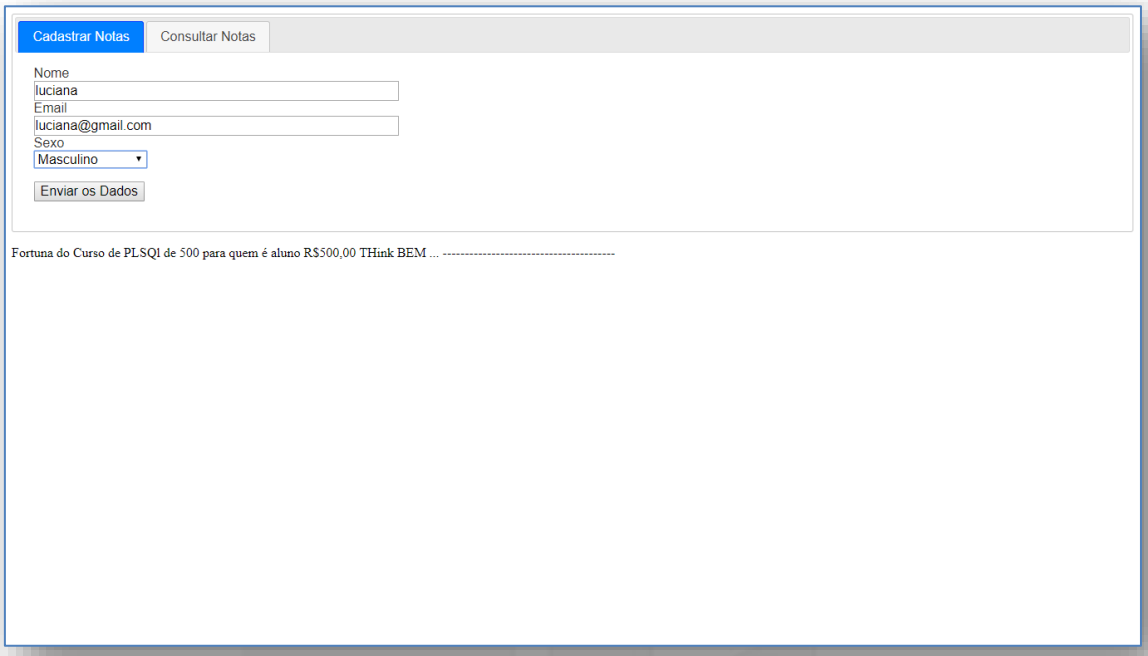
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html >
<html>
<head>
```

```
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.0/jqu
ery-ui.css"/>
<script type="text/javascript"
src="https://code.jquery.com/jquery-3.1.0.js"  ></script>
<script type="text/javascript"
src="https://code.jquery.com/ui/1.12.0/jquery-ui.js"></script>
<script type="text/javascript"
src="https://cdnjs.cloudflare.com/ajax/libs/jquery-
validate/1.16.0/jquery.validate.js"></script>

<script>
$(document).ready(function(){
   $("#tabs").tabs();
});
</script>

</head>
<body>
 <div id="tabs">
   <ul>
    <li><a href="#tab1">Cadastrar Notas</a></li>
   <li><a  href="#tab2">Consultar Notas</a></li>
   </ul>
<div id="tab1">

 <form method="post" action="Controle?cmd=gravar">
   Nome<br/>
   <input type="text" name="nome" size="50" id="nome"
     placeholder="Entre com nome" pattern="[a-z A-Z]+"
     title="digite o nome" required="required"
   />
   <br/>Email<br/>
   <input type="email" name="email" size="50" id="email"
     placeholder="Entre com email" title="digite o email"
     required="required"
    />
    <br/>Sexo<br/>
   <select name="sexo" id="sexo" required>
      <option value="">Escolha o Sexo</option>
      <option value="m">Masculino</option>
      <option value="f">Feminino</option>
   </select>
   <P/>
    <button id="btn1" type="submit">Enviar os Dados</button>
```

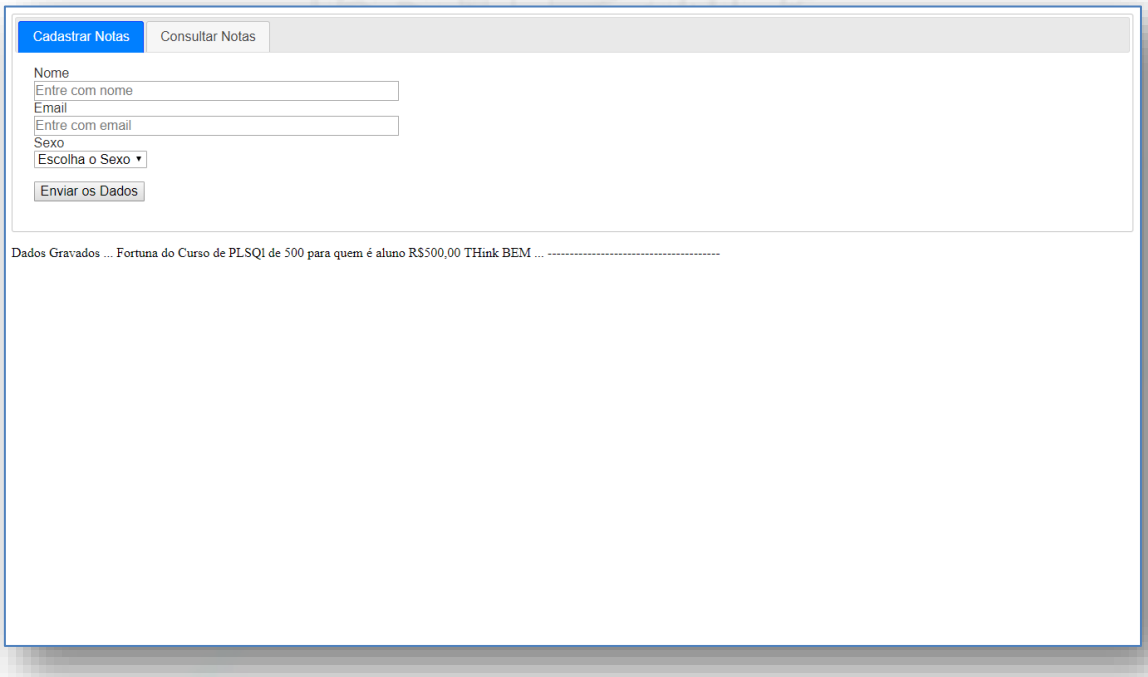
```
 </form>
</div>
<div id="tab2">
 Consultar os Dados
</div>
 </div>
<p/>

${msg}

    Fortuna do Curso de PLSQl de 500 para quem é aluno
    R$500,00 THink BEM ...

    ----------------------------------------

</body>
</html>
```



Cadastrando...



Gravado....

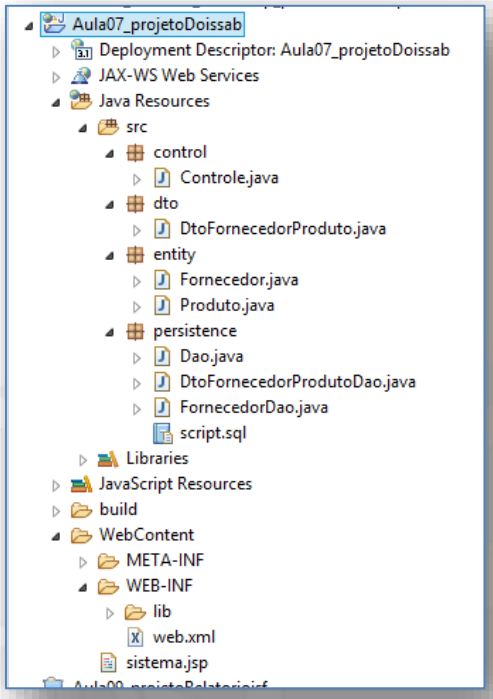


script.sql

```
conn system/coti
--conectei como dba

 create user sabado001 identified by coti
  default tablespace users
  quota 30m on users;

 grant create table, create session, create sequence,
  create procedure, create trigger, create view to sabado001;

  conn sabado001/coti

 show user;
USER Ú "SABADO001"

 select user from dual;

USER
----------
SABADO001

  create table  fornecedor(
      idFornecedor number (15) primary key,
      nome varchar2 (35) unique
);

  create table produto(
      idProduto number (15) primary key,
      nomeProduto varchar2 (50),
      preco number (15,2),
      id_fornecedor number (15),
      foreign key(id_fornecedor) references fornecedor
 on delete cascade
  );

  create sequence seq_fornecedor start with 100;

  create sequence seq_produto start    with 300;
```

```
  select * from cat;

TABLE_NAME                     TABLE_TYPE
------------------------------ -----------
FORNECEDOR                     TABLE
PRODUTO                        TABLE
SEQ_FORNECEDOR                 SEQUENCE
SEQ_PRODUTO                    SEQUENCE


--programas (mysql / oracle express / mongodb)
--eclipse oxigÃªnio / java 1.8
-- tomcat 8
--visual studio (android)

 ------------------------
 --Aqui Ã© view no java DTO...

 create or replace view  buscarTudo as
     select  idFornecedor,nome ,    idProduto, nomeProduto,
preco
     from fornecedor f inner join produto p
     on f.idfornecedor = p.id_fornecedor;

  select * from buscarTudo;
 --aqui no oracle view
 --no java dto


  --View ======== no java dto (DtoForncedorProduto)

  select * from cat;


TABLE_NAME                     TABLE_TYPE
------------------------------ -----------
BUSCARTUDO                     VIEW
FORNECEDOR                     TABLE
PRODUTO                        TABLE
SEQ_FORNECEDOR                 SEQUENCE
SEQ_PRODUTO                    SEQUENCE
```

Estrutura do projeto:



LIBS:

gson-2.2.4.jar
jstl-1.2.jar
ojdbc14-10.2.0.3.jar
servlet3-api.jar

```
package entity;

import java.util.ArrayList;
import java.util.List;

public class Fornecedor {

	private Integer idFornecedor;
	private String  nome;
	//OneToMany

	private List<Produto> produtos;
```

```
	public Fornecedor() {
	}

	public Fornecedor(Integer idFornecedor, String nome) {
		this.idFornecedor = idFornecedor;
		this.nome = nome;
	}

	@Override
	public String toString() {
		return "Fornecedor [idFornecedor=" + idFornecedor +
", nome=" + nome + "]";
	}

	public Integer getIdFornecedor() {
		return idFornecedor;
	}
	public void setIdFornecedor(Integer idFornecedor) {
		this.idFornecedor = idFornecedor;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public List<Produto> getProdutos() {
		return produtos;
	}
	public void setProdutos(List<Produto> produtos) {
		this.produtos = produtos;
	}

	public void adicionar(Produto p){
		if (produtos==null){
			produtos = new ArrayList<Produto>();
		}
		produtos.add(p);
	}
}
```

```
package entity;

public class Produto {

	private Integer idProduto;
	private String  nomeProduto;
	private Double  preco;
	//Muitos para UM
	private Fornecedor fornecedor;

	public Produto() {
	}

	public Produto(Integer idProduto, String nomeProduto,
Double preco) {
		this.idProduto = idProduto;
		this.nomeProduto = nomeProduto;
		this.preco = preco;
	}

	@Override
	public String toString() {
		return "Produto [idProduto=" + idProduto + ",
nomeProduto=" + nomeProduto + ", preco=" + preco + "]";

	}
	public Integer getIdProduto() {
		return idProduto;
	}
	public void setIdProduto(Integer idProduto) {
		this.idProduto = idProduto;
	}
	public String getNomeProduto() {
		return nomeProduto;
	}
	public void setNomeProduto(String nomeProduto) {
		this.nomeProduto = nomeProduto;
	}
	public Double getPreco() {
		return preco;
	}
	public void setPreco(Double preco) {
		this.preco = preco;
	}
```

```
	public Fornecedor getFornecedor() {
		return fornecedor;
	}
	public void setFornecedor(Fornecedor fornecedor) {
		this.fornecedor = fornecedor;
	}
}
```

```
package persistence;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Dao {

	 Connection con;
	PreparedStatement stmt;
	ResultSet rs;

	public void open() throws Exception{
	 Class.forName("oracle.jdbc.OracleDriver");
	 con = DriverManager.getConnection
	("jdbc:oracle:thin:@127.0.0.1:1521:XE","sabado001","coti");

	}

	public void close() throws Exception{
	 con.close();
	}
}
```

```
package persistence;

import java.util.List;

import entity.Fornecedor;
import entity.Produto;
```

```
public class FornecedorDao extends Dao{

      public void create(Fornecedor f, List<Produto> produtos
)throws Exception{
            open();
        stmt = con.
  prepareStatement("insert into fornecedor
values(seq_fornecedor.nextval,?) ");
         stmt.setString(1, f.getNome());
        stmt.execute();
        stmt.close();

       stmt = con.prepareStatement("select seq_fornecedor.currval
from dual");
       rs = stmt.executeQuery();
       rs.next();
       int chave = rs.getInt(1); //numero do fornecedor
       stmt.close();

       for (Produto p : produtos){
           stmt = con.prepareStatement("insert into produto "
              +  "   values(seq_produto.nextval,?,?,?)   ");
           stmt.setString(1,p.getNomeProduto());
           stmt.setDouble(2, p.getPreco());
           stmt.setInt(3, chave);
           stmt.execute();
         stmt.close();
       }
        close();
      }

      public static void main(String[] args) {
           try{
                Dao d = new Dao();
                d.open();
                d.close();
                System.out.println("Banco OK");
               Fornecedor f = new Fornecedor(null,"uruguaiana");
                    Produto p1= new Produto(null,"ocrei",10.);
                    Produto p2= new Produto(null,"hiban",20.);
                    f.adicionar(p1);
                    f.adicionar(p2);
```

```
                new FornecedorDao().create(f, f.getProdutos());
                      System.out.println("Dados Gravados :" + f);
                      //set linesize 1000;
                      //column nome format a15;
                      //column nomeProduto format a15;
                      //select * from fornecedor;
                      //select * from produto;
                }catch(Exception ex){
                     ex.printStackTrace();
                }
          }
}
```

No console...

```
Banco OK
Dados Gravados :Fornecedor [idFornecedor=null, nome=uruguaiana]
```

Data Transfer Object – DTO - Objeto de Transferência de Dados

O próprio nome já diz muito: um objeto simples usado para transferir dados de um local a outro na aplicação, sem lógica de negócios em seus objetos e comumente associado à transferência de dados entre uma camada de visão (view layer) e outra de persistência dos dados (model layer). Muito frequentemente você verá esse padrão sendo usado em conjunto com um DAO. Veja na**Figura 2** um exemplo claro dessa representação e desse conjunto entre os dois padrões.

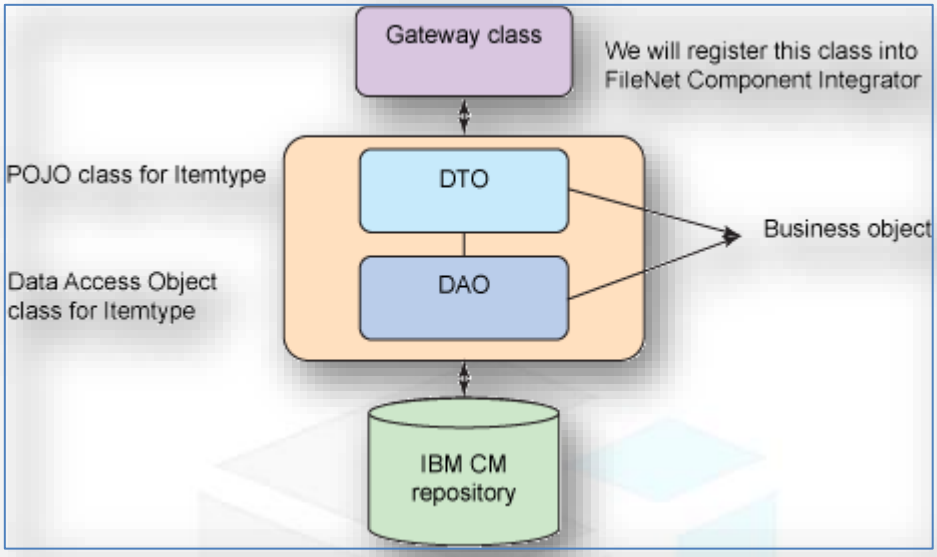


Figura 2: Exemplo de união entre os padrões DTO e DAO

Esse padrão também é bastante usado quando não se deseja expor a camada de persistência, porém é preciso que sejam exibidos os mesmos dados na camada de apresentação. Por exemplo, considere uma tela de uma aplicação que necessite listar os dados de 10 pessoas cadastradas em uma tabela. Para acessar estes dados, a camada de persistência assim o faz com a listagem configurada em um ArrayList de 10 PO's (vide padrão acima). Para passar esses valores à tela, a mesma lista antes tem de ser convertida para uma lista de DTO's com mesmos atributos e métodos get's/set's. Tudo isso porque a mesma aplicação faz uso do JPA, por exemplo, ou Hibernate e os mesmos frameworks não permitem que os dados tidos como "lazy (preguiçosos)" perdurem até depois de a conexão ter sido fechada. Por tal razão a conversão se faz necessária e assim os dados poderão fazer o trajeto sem serem perdidos ou sem que nenhum erro de conexão venha a acontecer.

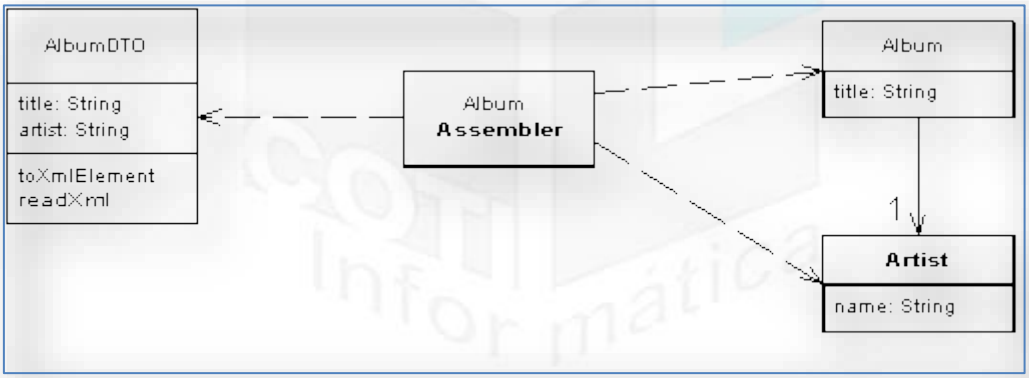
Observação: os padrões de projeto também não devem ser usados em detrimento do ambiente onde se está executando o mesmo projeto, a ideia é que eles sejam abstratos o suficiente para se adaptar, porém você será o autor principal disso, então não desconsidere o ambiente na hora de pensar em todos os cenários adaptáveis.

É um padrão de projeto de software usado para transferir dados entre subsistemas de um software. DTOs são frequentemente usados em conjunção com objetos de acesso a dados para obter dados de um banco de dados.

A diferença entre objetos de transferência de dados e objetos de negócio ou objetos de acesso a dados é que um DTO não possui comportamento algum, exceto o de armazenamento e obtenção de seus próprios dados. DTOs são objetos simples que não contêm qualquer lógica de negócio que requeira testes.[1]

Em uma arquitetura EJB tradicional, os DTOs servem para dois propósitos: primeiro, contornar o problema que entidades do EJB não são serializáveis; segundo, eles definem implicitamente uma fase de montagem onde todos os dados que serão usados para apresentação passam por marshalling antes de irem efetivamente para a camada de apresentação. Uma terceira razão para usar DTOs é que camadas não-adjacentes à camada de dados da aplicação não podem acessar estes objetos e modificá-los.

DTO (Martin Fowler)



```
package dto;

import entity.Fornecedor;
import entity.Produto;

public class DtoFornecedorProduto {

	private Fornecedor fornecedor;
	private Produto    produto;

	public DtoFornecedorProduto() {
	}
```

```
	public DtoFornecedorProduto(Fornecedor fornecedor, Produto
produto) {
		this.fornecedor = fornecedor;
		this.produto = produto;
	}

	@Override
	public String toString() {
		return "DtoFornecedorProduto [fornecedor=" +
fornecedor + ", produto=" + produto + "]";
	}

	public Fornecedor getFornecedor() {
		return fornecedor;
	}
	public void setFornecedor(Fornecedor fornecedor) {
		this.fornecedor = fornecedor;
	}
	public Produto getProduto() {
		return produto;
	}
	public void setProduto(Produto produto) {
		this.produto = produto;
	}
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;

import dto.DtoFornecedorProduto;
import entity.Fornecedor;
import entity.Produto;

public class DtoFornecedorProdutoDao
		extends Dao {


	public List<DtoFornecedorProduto> findAll() throws
Exception{
```

```
            open();
    stmt = con.prepareStatement("select * from buscarTudo");
//view
          rs = stmt.executeQuery();
          List<DtoFornecedorProduto> lst =
               new ArrayList<DtoFornecedorProduto>();
         while(rs.next()){
         DtoFornecedorProduto dto= new DtoFornecedorProduto();
               Fornecedor f = new Fornecedor();
                  f.setIdFornecedor(rs.getInt("idFornecedor"));
                  f.setNome(rs.getString("nome"));

                Produto p = new Produto();
                          p.setIdProduto(rs.getInt("idProduto"));
p.setNomeProduto(rs.getString("nomeProduto"));
                          p.setPreco(rs.getDouble("preco"));
                     dto.setFornecedor(f);
                     dto.setProduto(p);
               lst.add(dto);
         }
             close();
             return lst;
       }

     public static void main(String[] args) {
          try{
           System.out.println(
                     new DtoFornecedorProdutoDao().findAll()
                          );
          }catch(Exception ex){
               ex.printStackTrace();
          }
     }
}
```

No console...

```
[DtoFornecedorProduto [fornecedor=Fornecedor [idFornecedor=100,
nome=uruguaiana], produto=Produto [idProduto=300,
nomeProduto=ocrei, preco=10.0]], DtoFornecedorProduto
[fornecedor=Fornecedor [idFornecedor=100, nome=uruguaiana],
produto=Produto [idProduto=301, nomeProduto=hiban, preco=20.0]]]
```

JavaServer Pages

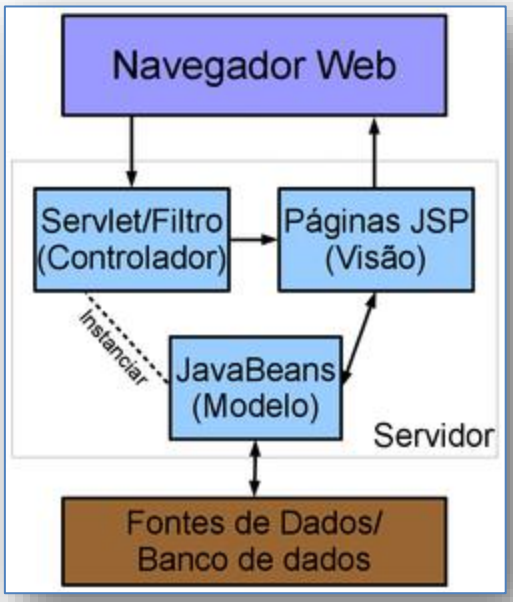
JavaServer Pages (**JSP**) é uma tecnologia que ajuda os desenvolvedores de software a criarem páginas web geradas dinamicamente baseadas em HTML, XML ou outros tipos de documentos. Lançada em 1999 pela Sun Microsystems, JSP é similar ao PHP, mas usa a linguagem de programação Java.

Para implantar e executar JavaServer Pages, um servidor web compatível com um container servlet, como Apache Tomcat, Jetty ou Glassfish, é requerido.
Por ser baseada na linguagem de programação Java, tem a vantagem da portabilidade de plataforma, que permite a sua execução em diversos sistemas operacionais, como oWindows da Microsoft, Unix e Linux.

Esta tecnologia permite ao desenvolvedor de páginas para Internet produzir aplicações que acessem o banco de dados, manipulem arquivos no formato texto, capturem informações a partir de formulários e captem informações sobre o visitante e sobre o servidor.

Uma página criada com a tecnologia JSP, após instalada em um servidor de aplicação compatível com a tecnologia Java EE, é transformada em um Servlet.

Visão geral:



A arquitetura JSP Modelo 2.

Arquitetonicamente, a JSP pode ser vista como uma abstração de alto nível de servlets Java. JSPs são traduzidas em servlets em tempo de execução. Cada servlet JSP é armazenado em cache e reutilizado até que a JSP original seja modificada.

JSP pode ser usada independentemente ou como o componente de visão de um projeto modelo-visão-controlador do lado do servidor, normalmente com JavaBeans como modelo e servlets Java (ou um framework como o Apache Struts) como o controlador. Este é um tipo de arquitetura Modelo 2.
JSP permite que o código Java e algumas ações pré-definidas sejam intercalados com conteúdo de marcação web estático, com a página resultante sendo compilada e executada no servidor que irá entregar um documento. As páginas compiladas, assim como as bibliotecas Java dependentes, utilizam o bytecode Java em vez de um formato de software nativo. Como qualquer outro programa Java, elas devem ser executadas em uma máquina virtual Java (JVM), que se integra com o sistema operacional do host servidor para fornecer um ambiente de plataforma neutra abstrato.
JSPs são normalmente utilizadas para entregar documentos HTML e XML, mas através do uso de OutputStream, elas também podem proporcionar outros tipos de dados.

O recipiente web cria objetos JSP implícitos como pageContext, servletContext, session, request & response.

A linguagem:

Em relação a arquitetura, JSP pode ser visto como uma abstração de alto nível dos Java servlets. Páginas JSP são carregadas dentro do servidor e processado a partir de uma estrutura especial do servidor Java, chamada Java EE Web Application, muitas vezes como um arquivo `.war` `.ear`.

JSP permite que código Java e certas ações pré-definidas sejam intercaladas com conteúdo de marcação estático, resultando na página sendo compilada e executada no servidor para ser entregue num documento HTML ou XML. As páginas compiladas e qualquer dependência de biblioteca Java usa os bytecodes primeiro que o formato nativo de software, e deve assim ser executado com uma JVM, a máquina virtual Java, integrada com o host do sistema operacional para prover um ambiente de abstração de plataforma.

A sintaxe de JSP é uma mistura de dois tipos básicos de conteúdo: *scriptlet elements* e *markup*. Markup é tipicamente um padrão HTML ou XML, enquanto os elementos scriptlet são blocos de código Java os quais podem ser unidos com o tipo de marcação, markup. Quando a página é requisitada o código Java é executado e sua saída é adicionada, in loco, com o ambiente de marcação para gerar a página final. Códigos JSP devem ser compiladas para criação das classes bytecodes antes de serem executadas, mas essa compilação é necessária apenas quando é feita uma mudança no código.

Sintaxe:

Páginas JSP utilizam vários delimitadores para funções de script. A mais básica é **<% ... %>**, que delimita um *scriptlet* JSP. Um scriptlet é um fragmento de código Java que é executado quando o usuário solicita uma página. Outros delimitadores comuns incluem **<%= ... %>** para *expressões*, onde o valor da expressão é colocado na página entregue ao usuário, e *diretivas*, denotadas com **<%@ ... %>**.

O código Java não é necessário ser completo ou auto-contido dentro de seu bloco de elemento scriptlet, mas pode abranger conteúdos de marcação fornecendo a página como um todo é sintaticamente correto. Por exemplo, quaisquer bloco Java if/for/while aberto em um elemento scriptlet deve ser corretamente fechado em um elemento posterior para que a página seja

compilada com sucesso. A marcação que cai dentro de um bloco de código quebrado está sujeito aquele código, desta forma a marcação dentro do bloco *if*aparecerá apenas na saída quando a condição *if* for avaliada como verdadeira (true). Do mesmo modo, a marcação dentro de uma construção de loop pode aparecer várias vezes na saída dependendo de quantas vezes o corpo do loop for executado.

Comentários JSP:

O JSP tem um tipo de comentário próprio, por exemplo: <%-- comentário --%>
Nota: este comentário só aparece do lado do servidor, não aparecendo por isso no código-fonte do browser !!

Tags JSP:

No JSP existem 5 tags:

1 - Declaration tag

Esta tag permite declarar variáveis e métodos. A sua sintaxe é: <%! …mais alguma coisa.. %> Um exemplo,

```
<%!
    private int contador= 0 ;
%>
```

2 - Expression tag

Esta tag permite mostrar algum resultado. A sua sintaxe é: <%=..algo para mostrar…%> Por exemplo, para mostrar o valor da variável contador bastava apenas:

<%=contador %>

3 - Directive tag
Esta tag permite dar informação sobre a página ao motor JSP. Existe 3 tipos de directivas:
page - processa informação para esta página. Existem 11 atributos opcionais. Sintaxe: <%@ page atributo=”valor” %>
Include – arquivos para serem incluidos. Sintaxe: <%@ include file=”pagina.jsp” %>

Taglib- Define uma biblioteca a ser usada. Precisa de um prefixo e de um url:
Sintaxe: <%@ taglib prefix = "prefixo" uri = "taglib.tld" %>

4 - Scriptlet tag

Todo o código entre <% %> é chamado de scriptlet. Sintaxe: **<% …código %>**

5 - Action tag

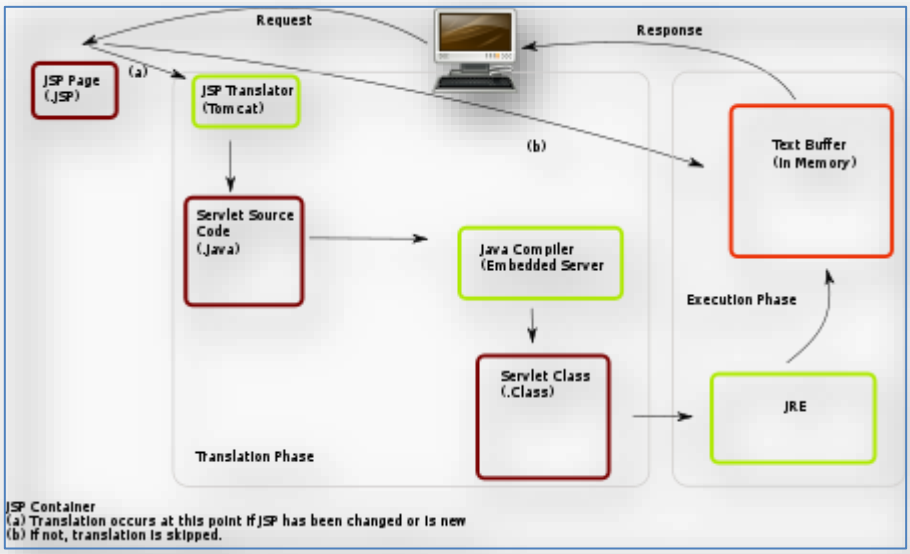
Esta tag tem como principais funcionalidades:

1. Acessar
2. para a inclusão de applets Java em páginas, independentemente e alterar propriedades de JavaBeans (`jsp:useBean`, `jsp:getProperty` e `jsp:setProperty`);
3. Redirecionar a requisição para outra página JSP (`jsp:forward`);
4. Uma única tag do browser cliente (`jsp:plugin` e `jsp:fallback`).

Servlet:

Servlet (servidorzinho em tradução livre) é uma classe Java usada para estender as funcionalidades de um servidor. Apesar dos servlets poderem responder a quaisquer tipos de requisições, eles normalmente são usados para estender as aplicações hospedadas por servidores web, desta forma eles podem ser imaginados como Applets Java que rodam em servidores em vez de rodarem nosnavegadores web. Estes tipos de servlets são os equivalentes Java a outras tecnologias de conteúdo Web dinâmico, como PHP e ASP.NET.

Também pode ser definido como um componente semelhante um servidor, que gera dados HTML eXML para a camada de apresentação de uma aplicação Web. Ele processa dinamicamenterequisições e respostas.



Ciclo de vida de um componente JSP.

Descrição:

A API Java Servlet (do pacote javax.servlet) proporciona ao desenvolvedor a possibilidade de adicionar conteúdo dinâmico em um servidor web usando a plataforma Java.

Esta tecnologia disponibiliza ao programador da linguagem Java uma interface para o servidor web (ou servidor de aplicação), através de uma API. As aplicações baseadas no Servlet geram conteúdo dinâmico (normalmente HTML) e interagem com os clientes, utilizando o modelo *requisição-resposta*.

Os servlets normalmente utilizam o protocolo HTTP, apesar de não serem restritos a ele. Um Servlet necessita de um container Web para ser executado.

Eles são frequentemente usados para:

- Processar ou armazenar dados que foram submetidos de um formulário HTML
- Fornecer conteúdo dinâmico, como os resultados de uma consulta a um banco de dados

- Gerenciar a informação de estado que não existe no protocolo sem estado HTTP, como inserir/retirar os itens de uma cesta de compras de um cliente específico

Tecnicamente falando, um "servlet" é uma classe Java no Java EE que obedece à API Java Servlet, um protocolo pelo qual uma classe Java pode responder a requisições. Servlets podem, em princípio, se comunicar sobre qualquer protocolo cliente-servidor, mas eles são na maioria das vezes usados com o protocolo HTTP. Desta forma, "servlet" normalmente é usado como uma abreviação para "servlet HTTP". Assim, um desenvolvedor de software pode usar um servlet para adicionar conteúdo dinâmico para umservidor web usando a plataforma Java. O conteúdo gerado normalmente é HTML, mas pode ser outro dado como XML. Servlets podem manter estado em variáveis de sessãoatravés de muitas transações de servidores, usando cookies HTTP ou reescrita de URL.

```
package control;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import entity.Fornecedor;
import entity.Produto;
import persistence.DtoFornecedorProdutoDao;
import persistence.FornecedorDao;


@WebServlet("/Controle")
public class Controle extends
HttpServlet {

	private static final long serialVersionUID = 1L;

	protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
	}
```

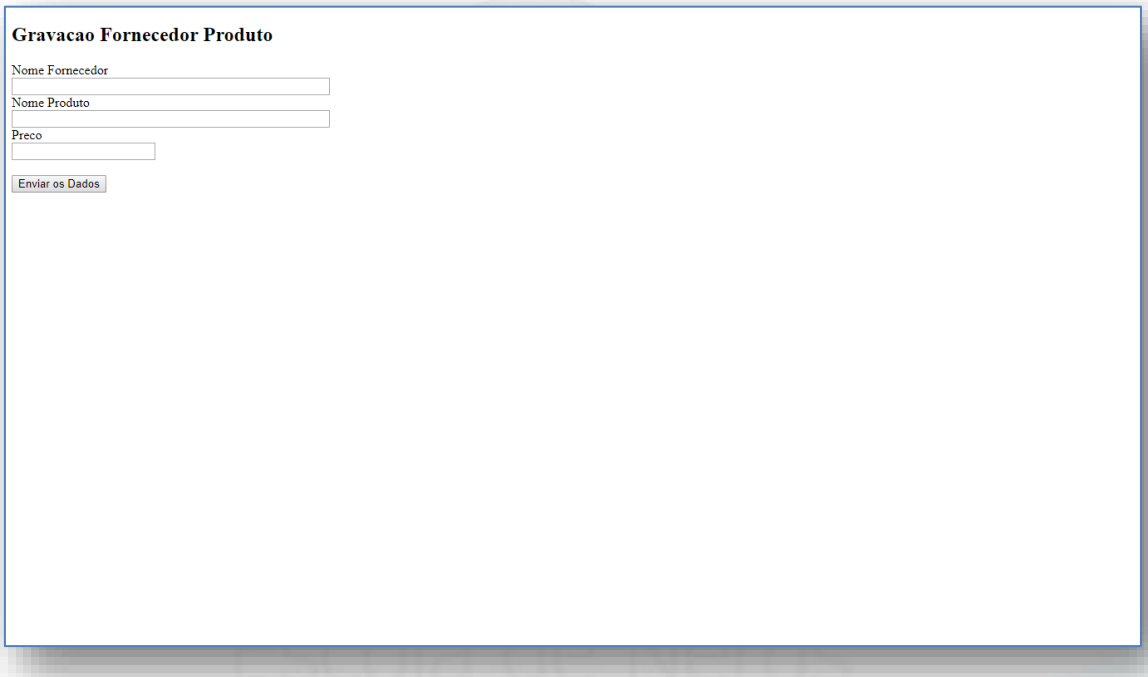
```
      protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
      try{
        Fornecedor f = new Fornecedor();
          f.setNome( request.getParameter("nomeFornecedor") );

       Produto p = new Produto();
          p.setNomeProduto(request.getParameter("nomeProduto"));
          p.setPreco(new Double(request.getParameter("preco")));
          f.adicionar(p);
         FornecedorDao dao = new FornecedorDao();
          dao.create(f, f.getProdutos());
      request.setAttribute("msg", new
DtoFornecedorProdutoDao().findAll());
      }catch(Exception ex){
          request.setAttribute("msg", "error :  + " +
ex.getMessage() );
      }finally{
request.getRequestDispatcher("sistema.jsp").forward(request,
response);
      }
     }
}
```

---

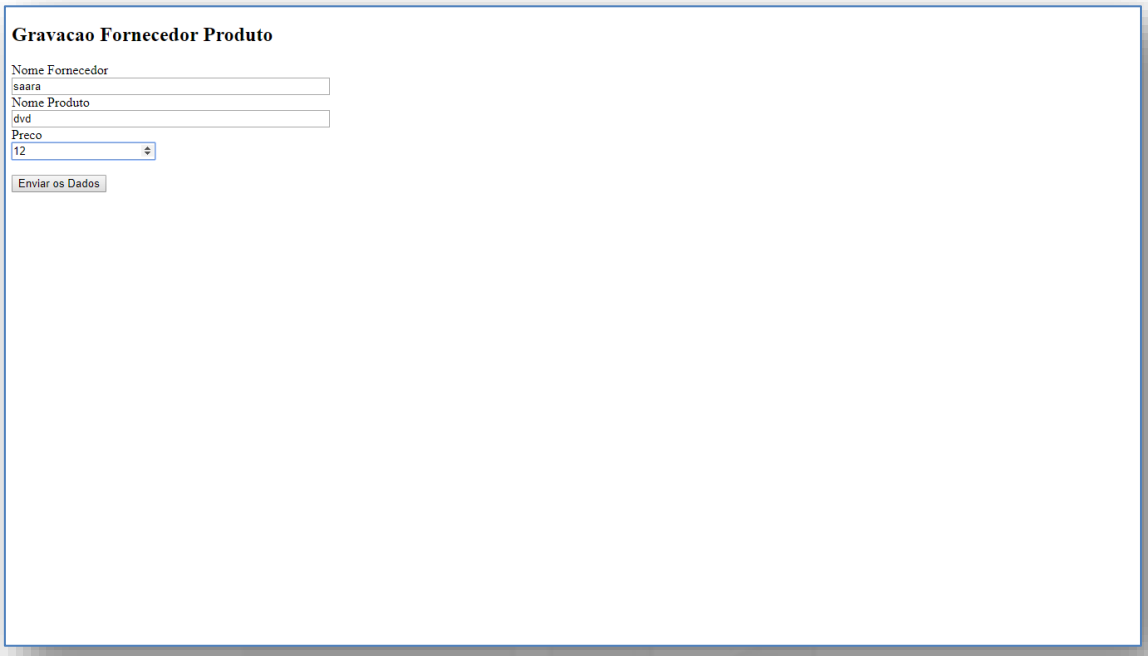
sistema.jsp

```
<!doctype html>
<html>
<head>

</head>

<body>
<h2>Gravacao Fornecedor Produto </h2>
<form method="post" action="Controle?cmd=gravar">
  Nome Fornecedor <br/>
 <input type="text" name="nomeFornecedor" required="required"
       size="50" value="" />
 <br/>Nome Produto<br/>
<input type="text" name="nomeProduto" required="required"
       size="50" value="" />
```

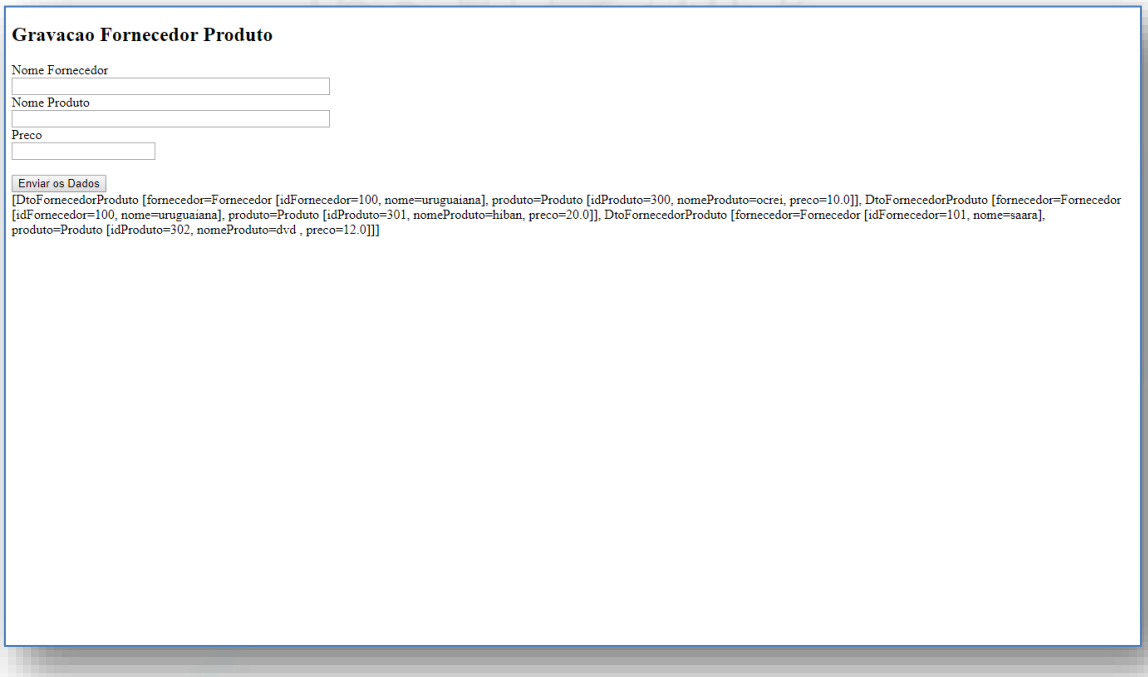
```
<br/>Preco<br/>
<input type="number" step="0.1" name="preco" required="required"
       size="50" value="" />
       <br/><br/>
<input type="submit" value="Enviar os Dados" />
</form>
${msg}
</body>
</html>
```



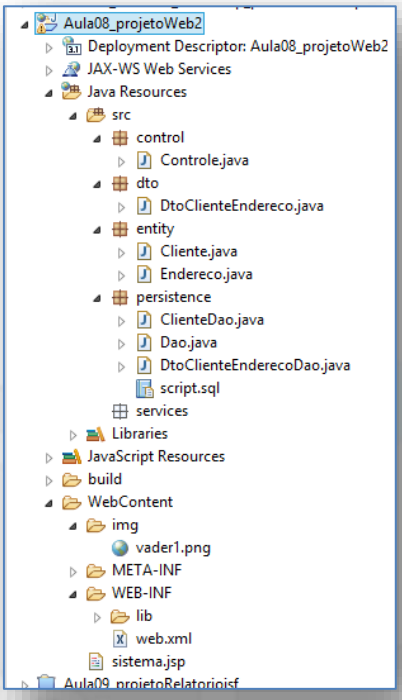
Gravando produto...



Resultado...



Estrutura do projeto:



LIBS:

```
asm-3.1.jar
commons-codec-1.6.jar
commons-logging-1.1.1.jar
fluent-hc-4.2.5.jar
gson-2.2.4.jar
httpclient-4.2.5.jar
httpclient-cache-4.2.5.jar
httpcore-4.2.4.jar
httpmime-4.2.5.jar
jackson-core-asl-1.9.2.jar
jackson-jaxrs-1.9.2.jar
jackson-mapper-asl-1.9.2.jar
jackson-xc-1.9.2.jar
jaxen-1.1.6.jar
jdom-2.0.5.jar
jersey-client-1.12.jar
jersey-core-1.12.jar
jersey-json-1.12.jar
jersey-server-1.12.jar
```

```
jersey-servlet-1.12.jar
jettison-1.1.jar
jsr311-api-1.1.1.jar
jstl.jar
mysql-connector-java-5.1.23-bin.jar
servlet3-api.jar
standard.jar
xercesImpl.jar
xml-apis.jar
```

```
package entity;

import java.io.Serializable;

public class Cliente implements Serializable{

	private static final long serialVersionUID = 1L;

	private Integer idCliente;
	private String  nome;
	private String  email;

	  //Cliente tem 1 Endereco ...
	//OneToOne
	  private Endereco endereco;

	public Cliente() {

	}

	public Cliente(Integer idCliente, String nome, String
email) {
		super();
		this.idCliente = idCliente;
		this.nome = nome;
		this.email = email;
	}

	@Override
	public String toString() {
```

```
            return "Cliente [idCliente=" + idCliente + ", nome="
+ nome + ", email=" + email + "]";
      }

      public Integer getIdCliente() {
            return idCliente;
      }
      public void setIdCliente(Integer idCliente) {
            this.idCliente = idCliente;
      }
      public String getNome() {
            return nome;
      }
      public void setNome(String nome) {
            this.nome = nome;
      }
      public String getEmail() {
            return email;
      }
      public void setEmail(String email) {
            this.email = email;
      }
      public Endereco getEndereco() {
            return endereco;
      }
      public void setEndereco(Endereco endereco) {
            this.endereco = endereco;
      }
}
```

```
package entity;

public class Endereco {

      private Integer idEndereco;
      private String  logradouro;
      private String  bairro;
      private String  cidade;
      private String  estado;
      private String  cep; //Google Maps ... toString() ...
```

```
	//1 para 1
	//OneToOne
	 private Cliente cliente;

	public Endereco() {

	}

	public Endereco(Integer idEndereco, String logradouro,
String bairro, String cidade, String estado, String cep) {
		this.idEndereco = idEndereco;
		this.logradouro = logradouro;
		this.bairro = bairro;
		this.cidade = cidade;
		this.estado = estado;
		this.cep = cep;
	}

	@Override
	public String toString() {
		return  logradouro + "," +  bairro + "," + cidade +
"," + estado + "," + cep  ;
	}

	public Integer getIdEndereco() {
		return idEndereco;
	}
	public void setIdEndereco(Integer idEndereco) {
		this.idEndereco = idEndereco;
	}
	public String getLogradouro() {
		return logradouro;
	}
	public void setLogradouro(String logradouro) {
		this.logradouro = logradouro;
	}
	public String getBairro() {
		return bairro;
	}
	public void setBairro(String bairro) {
		this.bairro = bairro;
	}
	public String getCidade() {
```

```
		return cidade;
	}
	public void setCidade(String cidade) {
		this.cidade = cidade;
	}
	public String getEstado() {
		return estado;
	}
	public void setEstado(String estado) {
		this.estado = estado;
	}
	public String getCep() {
		return cep;
	}
	public void setCep(String cep) {
		this.cep = cep;
	}
	public Cliente getCliente() {
		return cliente;
	}
	public void setCliente(Cliente cliente) {
		this.cliente = cliente;
	}
}
```

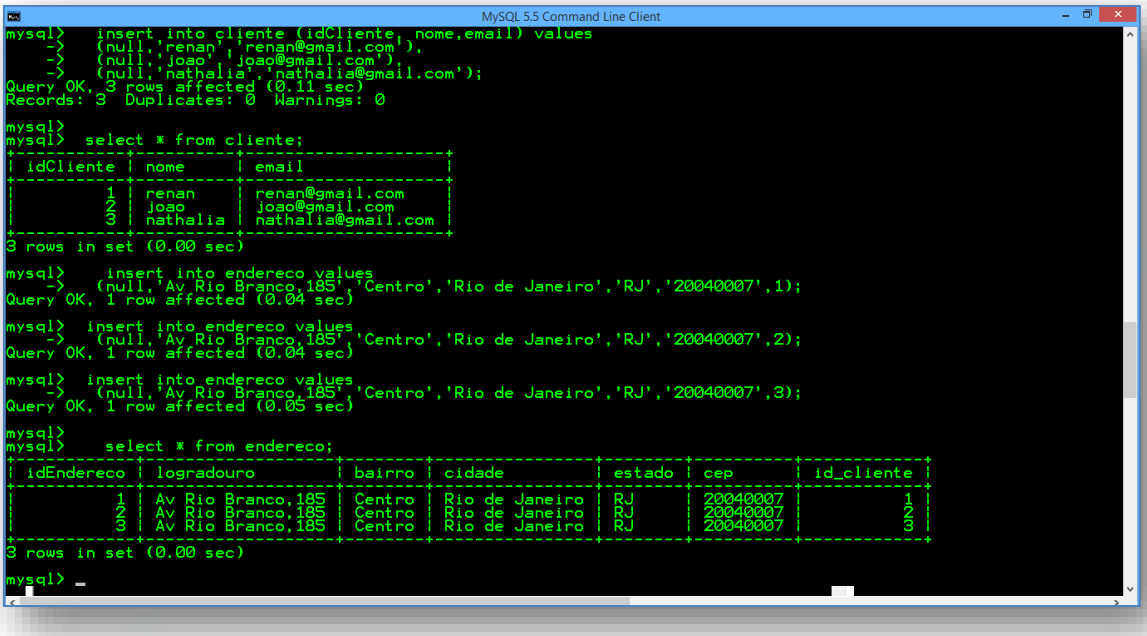
---

script.sql

```
#lista de jar
# mysql.jar, Servet3.jar, jstl.jar, Gson.jar, jersey
(WebServices)
#alterar o web.xml dele

create database web02;

use web02;

drop table if exists cliente;
drop table if exists endereco;

 create table cliente(
	idCliente int primary key auto_increment,
	nome varchar (50),
```

```
      email varchar (50) unique
      );

 create table endereco(
      idEndereco int primary key auto_increment,
      logradouro varchar (50),
      bairro varchar (50),
      cidade varchar (50),
      estado varchar (2),
      cep varchar (20),
      id_cliente int unique,
      foreign key(id_Cliente) references cliente(idCliente)
      );

    create view V$clienteEndereco  as
    select idCliente, nome, email,
           idEndereco, logradouro, bairro,cidade, estado, cep
          from cliente c inner join
          endereco e
          on c.idCliente = e.id_cliente;


--mostrar tabelas
  show tables;

+-------------------+
| Tables_in_web02   |
+-------------------+
| cliente           |
| endereco          |
| v$clienteendereco |
+-------------------+

--mostrar bancos
  show databases;

--mostrar usuário conectado
  select user();
+----------------+
| user()         |
+----------------+
| root@localhost |
+----------------+
```

```
 insert into cliente (idCliente, nome,email) values
 (null,'renan','renan@gmail.com'),
 (null,'joao','joao@gmail.com'),
 (null,'nathalia','nathalia@gmail.com');

select * from cliente;
+-----------+----------+--------------------+
| idCliente | nome     | email              |
+-----------+----------+--------------------+
|         1 | renan    | renan@gmail.com    |
|         2 | joao     | joao@gmail.com     |
|         3 | nathalia | nathalia@gmail.com |
+-----------+----------+--------------------+

  insert into endereco values
 (null,'Av Rio Branco,185','Centro','Rio de
Janeiro','RJ','20040007',1);
insert into endereco values
 (null,'Av Rio Branco,185','Centro','Rio de
Janeiro','RJ','20040007',2);
insert into endereco values
 (null,'Av Rio Branco,185','Centro','Rio de
Janeiro','RJ','20040007',3);

  select * from endereco;
```



```
  select nome, logradouro, bairro from V$clienteEndereco;
```

```
+----------+--------------------+--------+
| nome     | logradouro         | bairro |
+----------+--------------------+--------+
| renan    | Av Rio Branco,185  | Centro |
| joao     | Av Rio Branco,185  | Centro |
| nathalia | Av Rio Branco,185  | Centro |
|          |                    |        |
```

DAO - **Objeto de acesso a dados:**

Esse padrão tem por finalidade criar uma camada de acesso ao banco de dados e proporcionando o isolamento da camada de persistência das demais camadas do projeto. Essa camada e composta pela classe DAO e a classe que terá os métodos para o CRUD (Inserir, Deletar, Excluir e Lista) para cada entidade da camada de modelo. No exemplo abaixo a classe PessoaDao representa essa lógica e a entidade a ser persistida no banco de dados é Pessoa. A classe Dao é composta por dois métodos que são: Open() para abrir o banco e o Close() para fechar o banco. E alguns atributos como Connection (Conexão ao Banco de Dados), PreparedStatement (Acessar a Tabela), ResultSet (Consulta a Tabela) e o CallaBleStatement (Usar Procedures e Function).

Objeto de acesso a dados (ou simplesmente DAO, acrônimo de Data Access Object), é um padrão para persistência de dados que permite separar regras de negócio das regras de acesso a banco de dados. Numa aplicação que utilize a arquitetura MVC, todas as funcionalidades de bancos de dados, tais como obter as conexões, mapear objetosJava para tipos de dados SQL ou executar comandos SQL, devem ser feitas por classes DAO.

```
  package persistence;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
```

```
public class Dao {

	Connection con;
	PreparedStatement stmt;
	ResultSet rs;

	public static final String URL =
"jdbc:mysql://localhost:3306/web02";
	public static final String USER = "root";
	public static final String PASSWORD = "coti";
	public static final String BANCO = "com.mysql.jdbc.Driver";

	public void open() throws Exception {
		Class.forName(BANCO);
	con = DriverManager.getConnection(URL, USER, PASSWORD);
	}

	public void close() throws Exception {
		con.close();
	}
}
```

```
package persistence;

import java.sql.PreparedStatement;
import java.util.ArrayList;
import java.util.List;

import entity.Cliente;
import entity.Endereco;

public class ClienteDao  extends Dao{

	public void create (Cliente c) throws Exception{
		open();
		 stmt = con.prepareStatement("insert into cliente
values (null,?,?) ", PreparedStatement.RETURN_GENERATED_KEYS);
		 stmt.setString(1, c.getNome());
		 stmt.setString(2, c.getEmail());
```

```
            stmt.execute();
               rs = stmt.getGeneratedKeys();
               rs.next();
               c.setIdCliente(rs.getInt(1));
            close();
      }

      public List<Cliente> findAll() throws Exception{
           open();
        stmt = con.prepareStatement("select * from cliente");
            rs = stmt.executeQuery();
            List<Cliente> lista =new ArrayList<Cliente>();
            while(rs.next()){
                Cliente c = new Cliente(
                              rs.getInt(1),
                              rs.getString(2),
                              rs.getString(3)
                                        );
                lista.add(c);
            }
           close();
           return lista;
      }



      public void create (Cliente c, Endereco e) throws
Exception{
            open();
            con.setAutoCommit(false);
//transação em banco de dados
             stmt = con.prepareStatement("insert into cliente
values (null,?,?) ", PreparedStatement.RETURN_GENERATED_KEYS);
             stmt.setString(1, c.getNome());
             stmt.setString(2, c.getEmail());
             stmt.execute();
                rs = stmt.getGeneratedKeys();
                rs.next();
                c.setIdCliente(rs.getInt(1));
             stmt.close();

             stmt = con.prepareStatement("insert into endereco
values (null,?,?,?,?,?,?) ");
```

```
                stmt.setString(1, e.getLogradouro());
                stmt.setString(2, e.getBairro());
                stmt.setString(3, e.getCidade());
                stmt.setString(4, e.getEstado());
                stmt.setString(5, e.getCep());
                stmt.setInt(6, c.getIdCliente());
                stmt.execute();
                     stmt.close();
            con.setAutoCommit(true);
            close();
      }
}
```

```
package dto;

import entity.Cliente;
import entity.Endereco;

public class DtoClienteEndereco {

      private Cliente cliente;
      private Endereco endereco;

      public DtoClienteEndereco(Cliente cliente, Endereco
endereco) {
            this.cliente = cliente;
            this.endereco = endereco;
      }

      public DtoClienteEndereco() {
      }


      @Override
      public String toString() {
            return "DtoClienteEndereco [cliente=" + cliente + ",
endereco=" + endereco + "]";
      }

      public Cliente getCliente() {
            return cliente;
```

```
	}
	public void setCliente(Cliente cliente) {
		this.cliente = cliente;
	}
	public Endereco getEndereco() {
		return endereco;
	}
	public void setEndereco(Endereco endereco) {
		this.endereco = endereco;
	}
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;

import dto.DtoClienteEndereco;
import entity.Cliente;
import entity.Endereco;

public class DtoClienteEnderecoDao
extends Dao{

	public List<DtoClienteEndereco> findAll() throws
Exception{
		open();
		 stmt = con.prepareStatement("select * from
V$clienteEndereco");
		 List<DtoClienteEndereco> lista =
				new ArrayList<DtoClienteEndereco>();
		 rs = stmt.executeQuery();
		 while(rs.next()){
			Cliente c = new Cliente(rs.getInt(1),
					rs.getString(2),
					rs.getString(3)
					);
			Endereco e = new Endereco(rs.getInt(4),
						rs.getString(5),
						rs.getString(6),
```

```
                                                rs.getString(7),
                                                rs.getString(8),
                                                rs.getString(9)
                                );
                        DtoClienteEndereco dto = new
DtoClienteEndereco(c, e);
                        lista.add(dto);
                    }
                close();
                return lista;
        }
}
```

```
package control;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import entity.Cliente;
import entity.Endereco;
import persistence.ClienteDao;

@WebServlet("/Controle")
public class Controle extends
HttpServlet {

      private static final long serialVersionUID = 1L;

      protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
      }

      protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
       String cmd = request.getParameter("cmd");
```

```
        if (cmd.equalsIgnoreCase("gravar")){
             gravar(request,response);
        }
       }

      protected void gravar(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
        try{
             Cliente c = new Cliente(null,
                              request.getParameter("nome"),
                              request.getParameter("email")
                        );
             Endereco e = new Endereco(null,
                       request.getParameter("logradouro"),
                           request.getParameter("bairro"),
                           request.getParameter("cidade"),
                           request.getParameter("estado"),
                           request.getParameter("cep")
                    );
          ClienteDao cd = new ClienteDao();
                       cd.create(c,e);
           String url="http://www.google.com.br/maps/place/" +
                       e.toString().replaceAll(" ", "%20");
             String link="<a href="+url+">Google Maps</a>";
             String voltar="<a href=sistema.jsp>Voltar</a>";
             response.setContentType("text/html");
             response.getWriter().print(" Dados gravados <p/>");
             response.getWriter().print(link +"<br/>");
             response.getWriter().print(voltar +"<br/>");
        }catch(Exception ex){
              ex.printStackTrace();
        }
    }
}
```

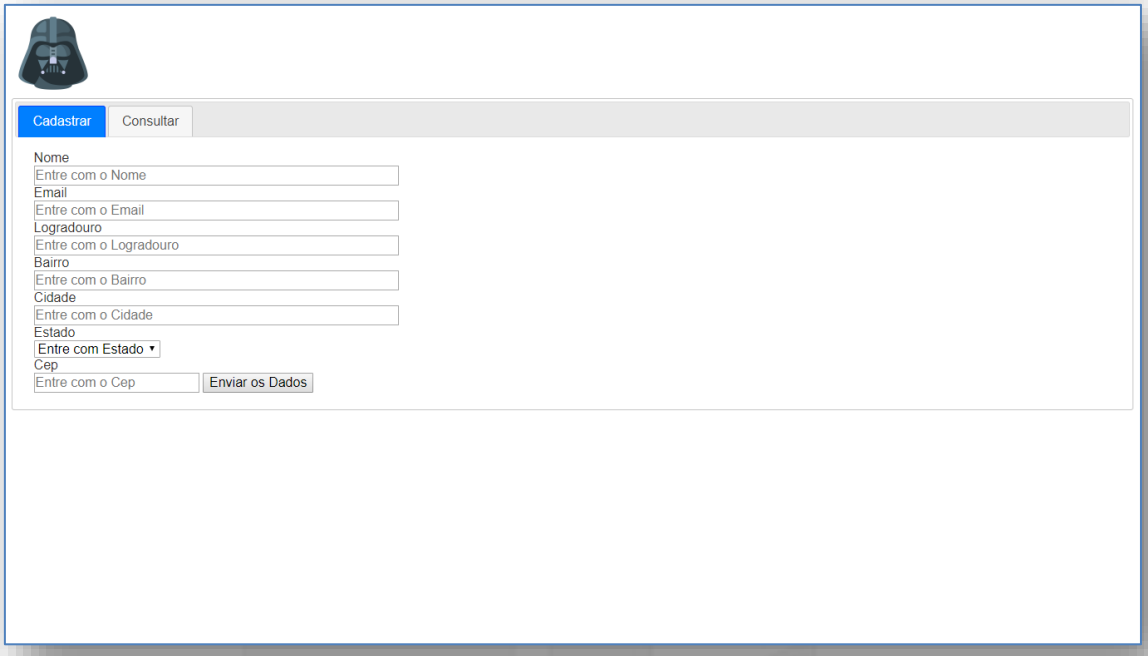
---

sistema.jsp

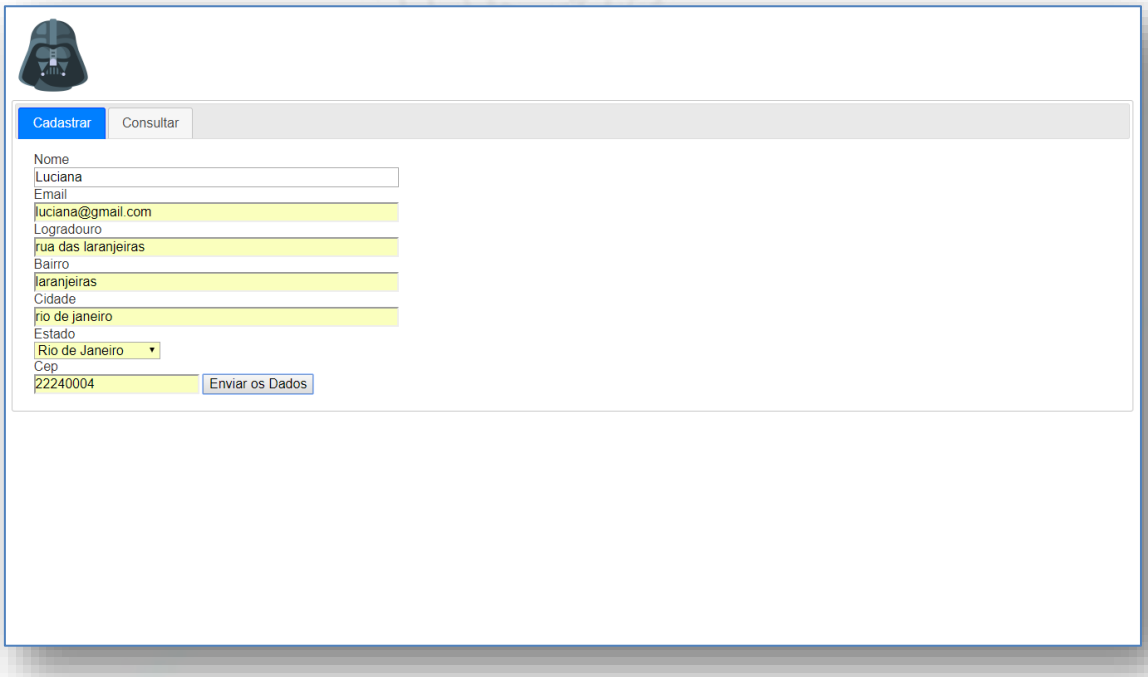
```
<!doctype html>
<html>
<head>
```

```
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.0/jqu
ery-ui.css"/>
<script type="text/javascript"
src="https://code.jquery.com/jquery-3.1.0.js"  ></script>
<script type="text/javascript"
src="https://code.jquery.com/ui/1.12.0/jquery-ui.js"></script>
<script type="text/javascript"
src="https://cdnjs.cloudflare.com/ajax/libs/jquery-
validate/1.16.0/jquery.validate.js"></script>
<script>
$(document).ready(function(){
      $("#tabs").tabs();
});
</script>
</head>
<body>

<img src="img/vader1.png" width="100px" height="100px"    >

 <div id="tabs">
  <ul>
     <li><a href="#cadastro">Cadastrar</a></li>
     <li><a href="#consulta">Consultar</a></li>
  </ul>

<div id="cadastro">

  <form method="post" action="Controle?cmd=gravar" name="form1">
   Nome<br/>
   <input type="text" name="nome" id="nome" size="50"
      required="required"
      placeholder="Entre com o Nome"
   />
  <br/>Email<br/>
   <input type="email" name="email" id="email" size="50"
      required="required"
      placeholder="Entre com o Email"
   />
 <br/>Logradouro<br/>
   <input type="text" name="logradouro" id="logradouro"
size="50"
      required="required"
```

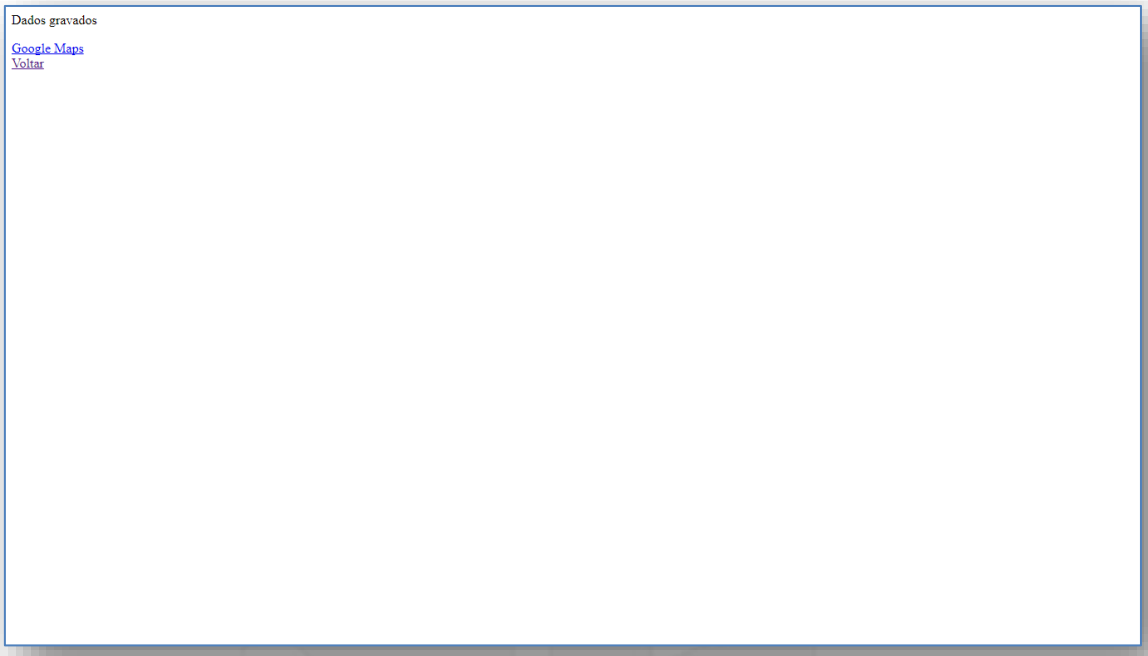
```
      placeholder="Entre com o Logradouro"
    />
 <br/>Bairro<br/>
    <input type="text" name="bairro" id="bairro" size="50"
      required="required"
      placeholder="Entre com o Bairro"
    />
 <br/>Cidade<br/>
    <input type="text" name="cidade" id="cidade" size="50"
      required="required"
      placeholder="Entre com o Cidade"
    />
<br/>Estado<br/>
    <select name="estado" id="estado" required="required">
      <option value="">Entre com Estado</option>
      <option value="RJ">Rio de Janeiro</option>
      <option value="MG">Minas Gerais</option>
      <option value="SP">Sao Paulo</option>
    </select>
<br/>Cep<br/>
    <input type="text" name="cep" id="cep" size="20"
      required="required"
      placeholder="Entre com o Cep"
    />

 <button id="btn1" type="submit">Enviar os Dados</button>

  </form>
</div>

<div id="consulta">
  Consultar os Dados
  <br/>
  ${msg}
</div>
 </div>
</body>
</html>
```



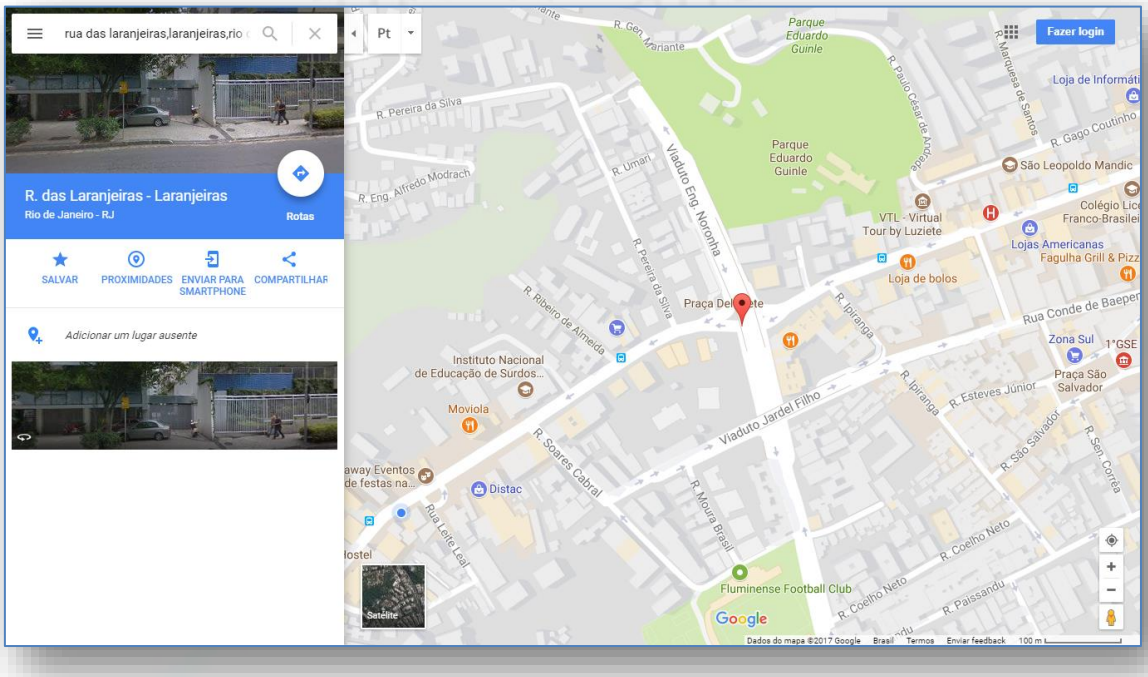
Cadastrando...



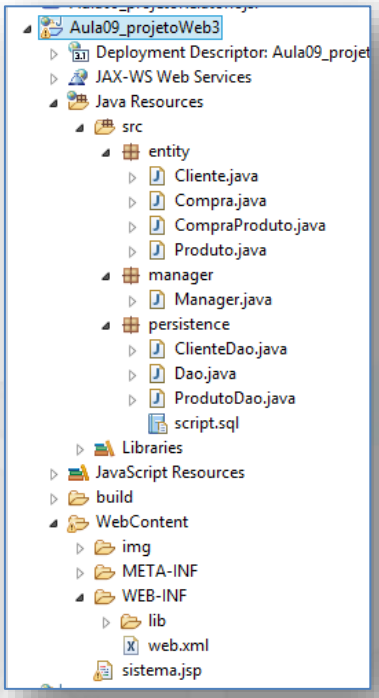
Gravado...



Clicando no link do mapa...



Estrutura do projeto:



LIBS:

gson-2.2.4.jar
jstl.jar
ojdbc14-10.2.0.3.jar
servlet3-api.jar
standard.jar

---

script.sql

Usando o Oracle...

Conectando como system...
```
conn system/coti
```

Criando um usuário...
```
create user aluno03 identified by coti
   default tablespace users
   quota 30m on users;
```

Dando as permissões para esse usuário...
```
grant create table, create session, create procedure,
      create sequence, create trigger, create view
      to aluno03;
```

Conectando como aluno03...
```
conn aluno03/coti
```

Mostrando quem é o usuário conectado...
```
show user;
```

Criando tabela cliente...
```
  create table cliente(
    idCliente number  (15) primary key,
    nome varchar (50),
    email varchar (50) unique
  );
```

Criando tabela compra...
```
  create table compra(
     idCompra number (15) primary key,
     total number (25,2),
     dataCompra timestamp,
     id_Cliente number (15),
     foreign key (id_Cliente) references cliente
  );
```

Apagando a tabela produto e tudo relacionado com ela...

```
drop table produto cascade constraint;
```

Apagando a tabela compra_produto e tudo relacionado com ela...

```
drop table compra_produto cascade constraint;
```

Criando a tabela produto...

```
create table produto(
    idProduto number (15) primary key,
    nomeProduto varchar (25),
    preco number (15,2),
    quantidade number (15),
    img varchar (50)
);
```

Criando a tabela compra_produto...

```
create table compra_produto(
    id_Produto number (15) ,
    id_Compra number (15),
    primary key(id_Produto,id_Compra),
    foreign key (id_Compra) references Compra,
    foreign key (id_Produto ) references Produto
);
```

Criando as sequencias...

```
create sequence seq_cliente;
create sequence seq_produto;
```

Criando a sequencia e começando a numeração pelo 2000...

```
create sequence seq_compra start with 2000;
```

Criando uma função para gravar o cliente...

```
 create or replace function gravCliente(vnome in varchar,
    vemail in varchar) return number
    as
     vcodigo number (15):=0;
    begin
        insert into cliente values
(seq_cliente.nextval,vnome,vemail);
        select seq_cliente.currval into vcodigo from dual;
         commit;
         return vcodigo;
         exception when others then
         rollback;
         return -1;
    end;
/
```

Criando uma variável do tipo "number" para receber "resp"

```
--no orasql
 var resp number;
```

Executando a função, passando os dados para gravar o cliente...

```
 exec  :resp :=  gravCliente('lu','lu@gmail.com');
```

Imprimindo a reposta

```
 print resp;
```

Conseguiu gravar...

```
 RESP
-----
   1
```

Mostrando as tabelas que existem nesse usuário (aluno03)

```
select * from cat;
```

```
TABLE_NAME                     TABLE_TYPE
------------------------------ -----------
CLIENTE                        TABLE
COMPRA                         TABLE
COMPRA_PRODUTO                 TABLE
PRODUTO                        TABLE
SEQ_CLIENTE                    SEQUENCE
SEQ_COMPRA                     SEQUENCE
SEQ_PRODUTO                    SEQUENCE
```

Formatando a largura das colunas para exibir melhor...

```
--formatar para resultado
 column nome format a15;
 column email format a20;
```

Formatando a largura da exibição na tela

```
 set linesize 1500;
```

Formatando a quantidade de linha na pagina

```
 set pagesize 2000;

 select * from cliente;
```

```
IDCLIENTE    NOME            EMAIL
---------- --------------- ---------------
         1 lu              lu@gmail.com
```

Vendo o conteúdo da função criada...

```
 select text from  user_source  where type in ('FUNCTION')
   and upper(name)=upper('gravcliente');
```

```
TEXT
------------------------------------------------------------
function gravCliente(vnome in varchar,
   vemail in varchar) return number
   as
    vcodigo number (15):=0;
   begin
        insert into cliente values (seq_cliente.nextval,vnome,vemail);
        select seq_cliente.currval into vcodigo from dual;
         commit;
         return vcodigo;
         exception when others then
         rollback;
         return -1;
   end;
```

Colocando a palavra Jose em Caixa Alta...

```
 select upper('jose') from dual;
```

```
UPPE
----
JOSE
```

Inserindo dados na tabela...

```
  insert into produto values
    (seq_produto.nextval,'hiphone pera',200,1,'hiphone1.jpg');
  insert into produto values
    (seq_produto.nextval,'hitable
orange',400,1,'hitablet1.jpg');
  insert into produto values
(seq_produto.nextval,'polystation',200,1,'polistation1.jpg');
  insert into produto values
    (seq_produto.nextval,'tamangoshi',100,1,'tamangoshi1.jpg');
 insert into produto values
    (seq_produto.nextval,'tamangoshi',100,1,'tamangoshi1.jpg');
 insert into produto values
    (seq_produto.nextval,'funstation',100,1,'funstation1.jpg');

 commit;
```

```
package entity;

public class Cliente {

	private Integer idCliente;
	private String nome;
	private String email;
	   //um Cliente para uma Compra
	private Compra compra;

	public Cliente() {
	}

	public Cliente(Integer idCliente, String nome, String
email) {
		super();
		this.idCliente = idCliente;
		this.nome = nome;
		this.email = email;
	}

	@Override
	public String toString() {
		return "Cliente [idCliente=" + idCliente + ", nome="
+ nome + ", email=" + email + "]";
	}

	public Integer getIdCliente() {
		return idCliente;
	}
	public void setIdCliente(Integer idCliente) {
		this.idCliente = idCliente;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
```

```
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public Compra getCompra() {
		return compra;
	}
	public void setCompra(Compra compra) {
		this.compra = compra;
	}
}
```

```
package entity;

public class Produto {

	private Integer idProduto;
	private String nomeProduto;
	private Double preco;
	private Integer quantidade;
	private String img;

	public Produto() {

	}

	public Produto(Integer idProduto, String nomeProduto,
Double preco, Integer quantidade, String img) {
		this.idProduto = idProduto;
		this.nomeProduto = nomeProduto;
		this.preco = preco;
		this.quantidade = quantidade;
		this.img = img;
	}

	@Override
	public String toString() {
		return "Produto [idProduto=" + idProduto + ",
nomeProduto=" + nomeProduto + ", preco=" + preco + ",
quantidade="+ quantidade + ", img=" + img + "]";
	}
```

```
	public Integer getIdProduto() {
		return idProduto;
	}
	public void setIdProduto(Integer idProduto) {
		this.idProduto = idProduto;
	}
	public String getNomeProduto() {
		return nomeProduto;
	}
	public void setNomeProduto(String nomeProduto) {
		this.nomeProduto = nomeProduto;
	}
	public Double getPreco() {
		return preco;
	}
	public void setPreco(Double preco) {
		this.preco = preco;
	}
	public Integer getQuantidade() {
		return quantidade;
	}
	public void setQuantidade(Integer quantidade) {
		this.quantidade = quantidade;
	}
	public String getImg() {
		return img;
	}
	public void setImg(String img) {
		this.img = img;
	}
}
```

```
package entity;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class Compra {

	private Integer idCompra;
	private Double total = 0.;
```

```
	private Date data;

	private Cliente cliente;

	private List<Produto> produtos;

	public Compra() {
	}

	public Compra(Integer idCompra, Double total, Date data) {
		this.idCompra = idCompra;
		this.total = total;
		this.data = data;
	}

	@Override
	public String toString() {
		return "Compra [idCompra=" + idCompra + ", total=" +
total + ", data=" + data + "]";
	}

	public Integer getIdCompra() {
		return idCompra;
	}
	public void setIdCompra(Integer idCompra) {
		this.idCompra = idCompra;
	}
	public Double getTotal() {
		return total;
	}
	public void setTotal(Double total) {
		this.total = total;
	}
	public Date getData() {
		return data;
	}
	public void setData(Date data) {
		this.data = data;
	}
	public Cliente getCliente() {
		return cliente;
	}
	public void setCliente(Cliente cliente) {
```

```
		this.cliente = cliente;
	}
	public List<Produto> getProdutos() {
		return produtos;
	}
	public void setProdutos(List<Produto> produtos) {
		this.produtos = produtos;
	}

	public void adicionarProduto(Produto ...p){
		 if (produtos==null){
			produtos = new ArrayList<Produto>();
		 }
		 double total = 0.;
		for(Produto x : p){
		  total +=  (x.getPreco() * x.getQuantidade());
		  produtos.add(x);
		}
		  setTotal(total);
	}
}
```

```
package entity;

public class CompraProduto {

	private Produto produto;
	private Compra compra;

	public CompraProduto() {
	}

	public CompraProduto(Produto produto, Compra compra) {
		this.produto = produto;
		this.compra = compra;
	}

	@Override
	public String toString() {
		return "CompraProduto [produto=" + produto + ",
compra=" + compra + "]";
```

```
	}

	public Produto getProduto() {
		return produto;
	}
	public void setProduto(Produto produto) {
		this.produto = produto;
	}
	public Compra getCompra() {
		return compra;
	}
	public void setCompra(Compra compra) {
		this.compra = compra;
	}
}
```

```
package persistence;

import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Dao {

	public static final String
URL="jdbc:oracle:thin:@localhost:1521:XE";
	public static final String USER="aluno03";
	public static final String PASSWORD="coti";

    Connection con;
    PreparedStatement stmt;
    ResultSet rs;
    CallableStatement call;

    public void open() throws Exception{
      Class.forName("oracle.jdbc.OracleDriver");
      con = DriverManager.getConnection(URL,USER, PASSWORD);
    }

    public void close() throws Exception{
```

```
      con.close();
    }
}
```

```
package persistence;

import java.sql.Types;

import entity.Cliente;

public class ClienteDao extends Dao{

     public void create(Cliente c)throws Exception{
           open();
    call= con.prepareCall("{ ? = call gravCliente(?,?) }");
    call.registerOutParameter(1,  Types.INTEGER);
    call.setString(2, c.getNome());
    call.setString(3, c.getEmail());
    call.execute();
    int resp = call.getInt(1);
      if (resp!=-1){
           c.setIdCliente(resp);
      }else{
       throw new Exception("Gravacao nao Autorizada ...");
      }
      close();
  }

 public static void main(String[] args) {
 try{
      Cliente c = new
Cliente(null,"dainana","daiana@gmail.com");
       new ClienteDao().create(c);
       System.out.println(c);
}catch(Exception ex){
      System.out.println("error:" + ex.getMessage());
      ex.printStackTrace();
 }
}
}
```

No console...

```
Cliente [idCliente=2, nome=dainana, email=daiana@gmail.com]
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;

import entity.Produto;

public class ProdutoDao extends Dao{

	public List<Produto> findAll() throws Exception{
		open();
	  stmt = con.prepareStatement("select * from produto");
		rs = stmt.executeQuery();
		List<Produto>lst = new ArrayList<Produto>();
		while(rs.next()){
		    Produto p = new Produto();
			p.setIdProduto(rs.getInt("idProduto"));
p.setNomeProduto(rs.getString("nomeProduto"));
			p.setPreco(rs.getDouble("preco"));
p.setQuantidade(rs.getInt("quantidade"));
			p.setImg(rs.getString("img"));
			lst.add(p);
		}
		close();
		return lst;
	}
}
```

```
package manager;

import java.util.List;

import entity.Produto;
import persistence.ProdutoDao;
```

public class Manager {

```
	private List<Produto> lista;

	public List<Produto> getLista() {
		try{
		   lista = new ProdutoDao().findAll();
		}catch(Exception ex){
		}
		  return lista;
	}


	public void setLista(List<Produto> lista) {
		this.lista = lista;
	}
}
```

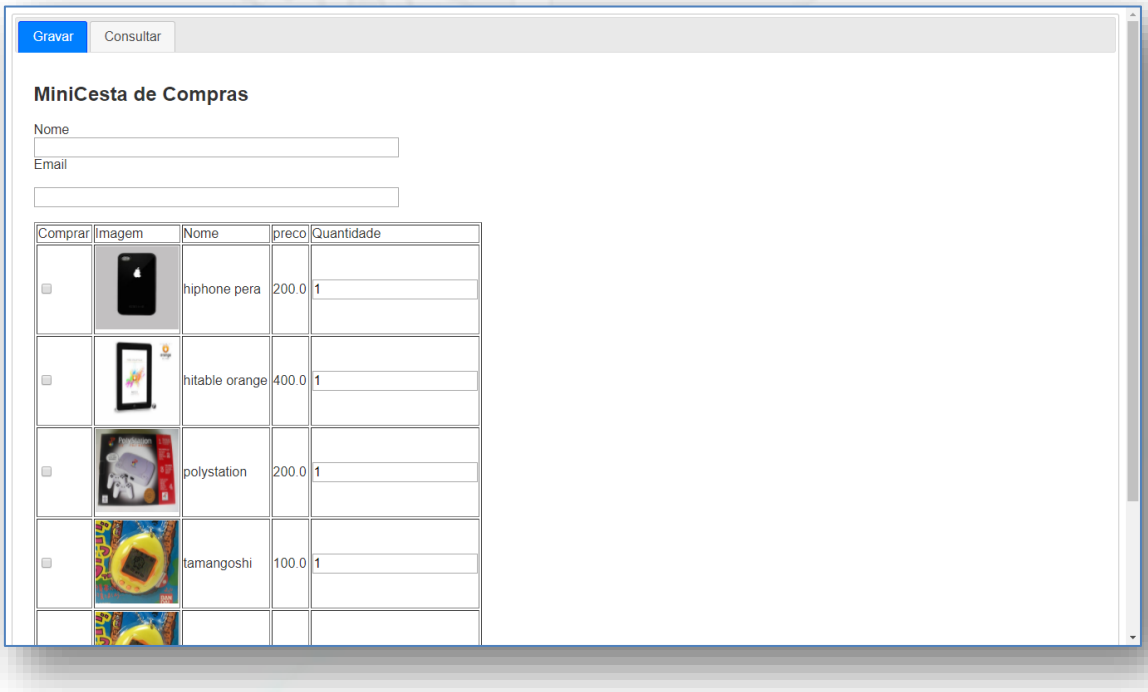
---

sistema.jsp

```
<!doctype html>
<html>
<head>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.0/jquery-ui.css"/>
<script type="text/javascript"
src="https://code.jquery.com/jquery-3.1.0.js"  ></script>
<script type="text/javascript"
src="https://code.jquery.com/ui/1.12.0/jquery-ui.js"></script>
<script type="text/javascript"
src="https://cdnjs.cloudflare.com/ajax/libs/jquery-validate/1.16.0/jquery.validate.js"></script>
<jsp:useBean id="mb"   class="manager.Manager"  scope="request"
></jsp:useBean>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<script>
 $(document).ready(function(){
	$("#tabs").tabs();
 })
```

```
</script>
</head>
<body>
<div id="tabs">
  <ul>
     <li><a href="#tabs-1">Gravar</a></li>
     <li><a href="#tabs-2">Consultar</a></li>
  </ul>

<div id="tabs-1">
<h2>MiniCesta de Compras</h2>


 <form method="post" action="Controle?cmd=gravar" name="form1">

 Nome<br/>
 <input type="text" name="nome" id="nome" value="" size="50"
   required="required"/>
<br/>Email<br/><br/>
 <input type="email" name="email" id="email" value="" size="50"
  required="required"
  />
<br/><br/>

  <table border="1">
  <thead>
   <tr>
     <td>Comprar</td>
     <td>Imagem</td>
     <td>Nome</td>
     <td>preco</td>
     <td>Quantidade</td>
   </tr>
   </thead>
    <c:forEach items="${mb.lista}" var="linha">
      <tr>
      <td><input type="checkbox" name="produto"
value="${linha.idProduto}"
                 /></td>
      <td><img src="img/${linha.img}" width="100px"
height="100px" /></td>
      <td>${linha.nomeProduto}</td>
      <td>${linha.preco}</td>
```

```
      <td><input type="number"
name="quantidade${linha.idProduto}"
                 value="1"  size="3"     /></td>
    </tr>

    </c:forEach>

  </table>



 </form>

 </div>
 <div id="tabs-2">
  Consulta da Gravacao
 </div>
 </div>
</body>
</html>
```

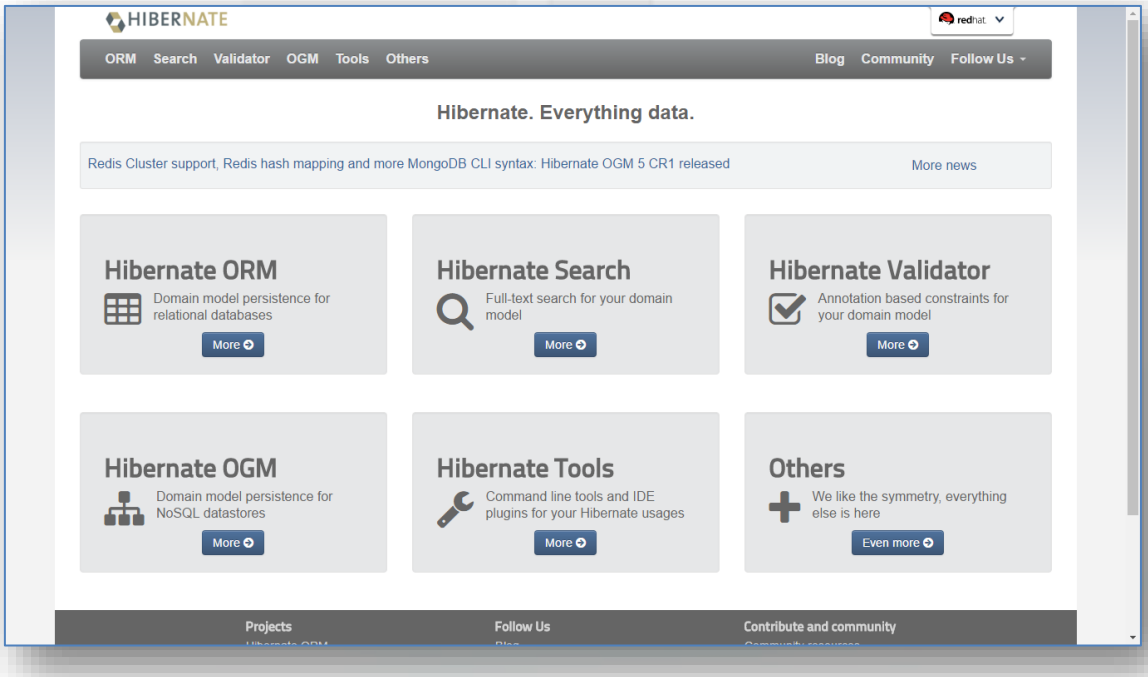


Hibernate:

- FrameWork para Trabalhar com o Banco de dados ... (Atua Na camada de Persistencia), então na hora de Gravar, você uma aplicação ao invés do DER , você Grava ORM)
Hibernate (3...5)
Hibernate 3.0 _ Trabalha muito com o FrameWork de Banco chamado Hibernate (Comandos do Hibernate) ...
Hibernate 4.0 _ Trabalha JPA ... (Anotação de Classe, serve para Persistir os Dados)

Hibernate é um frameWrok de Banco substitui o JDBC....

http://hibernate.org/



Hibernate:

O ***Hibernate*** é um *framework* para o mapeamento objeto-relacional escrito na linguagem Java, mas também é disponível em .Netcomo o nome NHibernate. Este framework facilita o mapeamento dos atributos entre uma base tradicional de dados relacionais e o modelo objeto de uma aplicação, mediante o uso de arquivos (XML) ou anotações Java (veja Annotation (java)).

Hibernate é um software livre de código aberto distribuído com a licença LGPL.

Características:

O objetivo do Hibernate é diminuir a complexidade entre os programas Java, baseado no modelo orientado a objeto, que precisam trabalhar com um banco de dados do modelo relacional (presente na maioria dos SGBDs). Em especial, no desenvolvimento de consultas e atualizações dos dados.

Sua principal característica é a transformação das classes em Java para tabelas de dados (e dos tipos de dados Java para os da SQL). O Hibernate gera as chamadas SQL e libera o desenvolvedor do trabalho manual da conversão dos dados resultante, mantendo o programa portável para quaisquer bancos de dados SQL, porém causando um pequeno aumento no tempo de execução.

Nas questões relacionadas para o gerenciamento de transações e na tecnologia de acesso à base de dados são de responsabilidade de outros elementos na infraestrutura do programa. Apesar de existirem API no Hibernate para possuir operações de controle transacional, ele simplesmente delegará estas funções para a infraestrutura na qual foi instalada.

No caso de aplicações construídas para serem executadas em servidores de aplicação, o gerenciamento das transações é realizado segundo o padrão JTA. Já nas aplicaçõesstandalone, o programa delega o tratamento transacional ao driver JDBC.

Hibernate pode ser utilizado em aplicações Java standalone ou em aplicações Java EE, utilizando servlet ou sessões Enterprise Java Beans.

História:

Hibernate foi criado por desenvolvedores Java, espalhados ao redor do mundo, e liderado por Gavin King. Posteriormente, JBoss Inc (empresa comprada pela Red Hat) contratou os principais desenvolvedores do programa para fazer o seu suporte.

A atual versão do Hibernate é a 4.x, que incorporou características como a nova arquitetura Interceptor/Callback, filtros definidos pelo usuário e anotações JDK 5.0 (Metadados do Java), que substitui os arquivos XML. Hibernate 3 também se aproxima das especificações EJB 3.0 e atua como a espinha dorsal das implementações EJB 3.0 em JBoss.

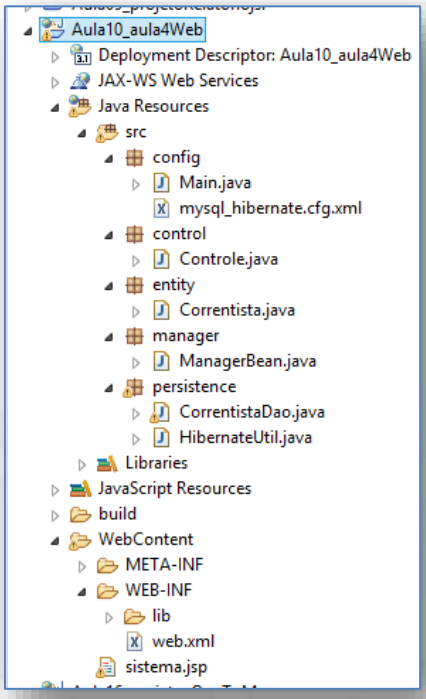
HQL:

A HQL (Hibernate Query Language) é um dialeto SQL para o Hibernate. Ela é uma poderosa linguagem de consulta que se parece muito com a SQL, mas a HQL é totalmente orientada a objeto, incluindo os paradigmas de herança, polimorfismo e encapsulamento.

No Hibernate, você pode escolher tanto usar a SQL quanto a HQL. Escolhendo a HQL, você poderá executar os pedidos SQL sobre as classes de persistência do Java ao invés de tabelas no banco de dados.

Utilizando o HQL temos a vantagem de portabilidade de banco, ou seja, suponha que estamos utilizando um banco de dados A, ao trocarmos para um banco B o HQL automaticamente cria comandos referentes a cada banco de dados. Isso facilita, pois no SQL teríamos que rastreara alterar vários códigos no sistema.

Estrutura do projeto:



LIBS:

```
antlr-2.7.6.jar
asm-attrs.jar
asm.jar
c3p0-0.9.0.jar
cglib-2.1.3.jar
commons-beanutils-1.8.0.jar
commons-codec-1.3.jar
commons-collections-3.2.1.jar
commons-digester-2.0.jar
commons-el-1.0.jar
commons-fileupload-1.0.jar
commons-lang-2.1.jar
commons-logging-1.1.1.jar
commons-validator.jar
dom4j-1.6.1.jar
ejb3-persistence.jar
hibernate-annotations.jar
hibernate-commons-annotations.jar
hibernate3.jar
```

```
jakarta-oro.jar
jstl-1.1.0.jar
jstl.jar
jta.jar
log4j-1.2.15.jar
mysql-connector-java-5.1.23-bin.jar
oro-2.0.8.jar
servlet3-api.jar
standard.jar
```

```
package entity;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
```

Fazer as anotações do Hibernate na classe e nos atributos para serem gerados no banco.

@Entity - A classe vira tabela no banco
@Table - Nomeio a tabela para "correntista" com letra minuscula
@Column - O atributo virará uma coluna na tabela
Length - numero maximo de caracteres na coluna
Unique - o email não irá se repetir

Implementar a classe Serializable para trabalhar com objeto.
Atenção para os imports!
Na classe serão sempre "javax."*

```
@Entity
@Table(name="correntista")
public class Correntista implements Serializable  {

	private static final long serialVersionUID = 1L;
```

```
	@Id                                    //chave primaria
	private  Integer idCorrentista;

	@Column(length=50) //tamanho 50
	private  String  nome;

	@Column(length=50, unique=true)
	private  String email; //tamanho 50 (nao pode duplicar)

    @Column
	private Double saldo=0.;

    //Hibernate FrameWork para trabalhar com PersistÃªncia, o
principal
    //motivo Ã© gravar OO trabalhar OO na gravaÃ§Ã£o ...

	public Correntista() {
	}

	public Correntista(Integer idCorrentista, String nome,
String email, Double saldo) {
		this.idCorrentista = idCorrentista;
		this.nome = nome;
		this.email = email;
		this.saldo = saldo;
	}

	@Override
	public String toString() {
		return "Correntista [idCorrentista=" + idCorrentista
+ ", nome=" + nome + ", email=" + email + ", saldo="
				+ saldo + "]";
	}

	public Integer getIdCorrentista() {
		return idCorrentista;
	}
	public void setIdCorrentista(Integer idCorrentista) {
		this.idCorrentista = idCorrentista;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
```

```
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public Double getSaldo() {
		return saldo;
	}
	public void setSaldo(Double saldo) {
		this.saldo = saldo;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
	}
}
```

Configurar esse arquivo com as configrações para gerar o banco...

mysql_hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</pro
perty>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</
property>
```

Criar o banco "hiberaula10" se não existir...

```
    <property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/hibaula10?createDatabaseIfNotExist=t
rue</property>
```

Usuário "root"

```
    <property
name="hibernate.connection.username">root</property>
```

Senha "coti"

```
    <property
name="hibernate.connection.password">coti</property>
```

Propriedade para mostrar o SQL de criação da tabela...

```
    <property name="hibernate.show_sql">true</property>
```

Propriedade para mostrar o SQL formatdo...

```
    <property name="hibernate.format_sql">true</property>
```

Propriedade para atualizar as tabelas caso seja necessário...

```
    <property name="hibernate.hbm2ddl.auto">update</property>
```

Mapear as classe que serão tabelas no banco...

```
    <mapping class="entity.Correntista"/>

</session-factory>
</hibernate-configuration>
```

Classe "Main" para gerar o banco e as tabelas.

```
package config;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.cfg.Configuration;
import org.hibernate.tool.hbm2ddl.SchemaExport;

public class Main {

    public static void main(String[] args) {
```

```
            Configuration cfg = new
AnnotationConfiguration().configure("config/mysql_hibernate.cfg.
xml");
             new SchemaExport(cfg).create(true,true);
```

"Create(true, true)", onde o 1º "true" criar o banco e o 2º "true" criar a(s) tabela(s)

```
            /*
             *
             *
    drop table if exists correntista

    create table correntista (
        idCorrentista integer not null,
        email varchar(50) unique,
        nome varchar(50),
        saldo double precision,
        primary key (idCorrentista)
    )
            */
     }
}
```

```
package persistence;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;

public class HibernateUtil {

    private static final SessionFactory sessionFactory;

    static {
        try {
```

Indicar onde se encontra o arquivo XML de configração do banco...

```
            sessionFactory = new
AnnotationConfiguration().configure("config/mysql_hibernate.cfg.
xml").buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation
failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

```
package persistence;

import java.util.List;
//imports tudo é org...

import org.hibernate.Criteria;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.Transaction;

import entity.Correntista;

public class CorrentistaDao {

     Session session; //comandos para trabalhar com a tabela
     Transaction transaction; //transação no banco (salvar,
alterar, excluir)
     Criteria criteria; //consulta
     Query query; //consulta

       public void create(Correntista c) throws Exception{
   session = HibernateUtil.getSessionFactory().openSession();
               transaction = session.beginTransaction();
                   session.save(c);
//grava e comita no Banco de Dados
               transaction.commit();
            session.close();
       }
```

```
        public void update(Correntista c) throws Exception{
    session = HibernateUtil.getSessionFactory().openSession();
                transaction = session.beginTransaction();
                    session.update(c);
//grava e comita no Banco de Dados
                transaction.commit();
             session.close();
        }

        public void delete(Correntista c) throws Exception{
    session = HibernateUtil.getSessionFactory().openSession();
                transaction = session.beginTransaction();
                    session.delete(c);   //grava e comita no
Banco de Dados
                transaction.commit();
             session.close();
        }

    public List<Correntista> findAll(){
     session = HibernateUtil.getSessionFactory().openSession();
            List<Correntista> lista = session.createQuery("select
obj from Correntista as obj order by obj.saldo desc ").list();
              session.close();
            return lista;
       }

       public Correntista findByCode(Integer cod){
     session = HibernateUtil.getSessionFactory().openSession();
     Correntista correntista = (Correntista)
session.get(Correntista.class, cod);
                    session.close();
           return correntista;
       }

       //jdbc
//     public void create2(Correntista c)throws Exception{
//          //open();
//          //antes
//           stmt = con.prepareStatement("insert into
correntista values (?,?,?,?) ");
//            stmt.setInt(1,c.getIdCorrentista());
//            stmt.setString(2,c.getNome());
//            stmt.setString(3,c.getEmail());
//            stmt.setDouble(4,c.getSaldo());
//            stmt.execute();
```

```
 //close();
//     }


       public static void main(String[] args) {
           try{
                 Correntista c1 = new
Correntista(1000,"lu","lu@gmail.com",5000.);
                 Correntista c2 = new
Correntista(2000,"gilson","gilson@gmail.com",15000.);
                 Correntista c3 = new
Correntista(3000,"ilan","ilan@gmail.com",50000.);
                 CorrentistaDao dao = new CorrentistaDao();
                 //  dao.create(c1);
           //      dao.create(c2);
           //      dao.create(c3);
                  System.out.println("Listar Banco :" +
dao.findAll());
                  Correntista co =   dao.findByCode(3000);
               System.out.println("Cliente :" + co);
                  //time
        }catch(Exception ex){
                 ex.printStackTrace();
           }
     }
}
```

No console..

```
log4j:WARN No appenders could be found for logger
(org.hibernate.cfg.annotations.Version).
log4j:WARN Please initialize the log4j system properly.
Hibernate:
    select
        correntist0_.idCorrentista as idCorren1_0_,
        correntist0_.email as email0_,
        correntist0_.nome as nome0_,
        correntist0_.saldo as saldo0_
    from
        correntista correntist0_
    order by
        correntist0_.saldo desc
```

```
Listar Banco :[Correntista [idCorrentista=3000, nome=ilan,
email=ilan@gmail.com, saldo=50000.0], Correntista
[idCorrentista=2000, nome=gilson, email=gilson@gmail.com,
saldo=15000.0], Correntista [idCorrentista=1000, nome=lu,
email=lu@gmail.com, saldo=5000.0]]
Hibernate:
    select
        correntist0_.idCorrentista as idCorren1_0_0_,
        correntist0_.email as email0_0_,
        correntist0_.nome as nome0_0_,
        correntist0_.saldo as saldo0_0_
    from
        correntista correntist0_
    where
        correntist0_.idCorrentista=?
Cliente :Correntista [idCorrentista=3000, nome=ilan,
email=ilan@gmail.com, saldo=50000.0]
```

```
package manager;

import java.util.List;
import entity.Correntista;
import persistence.CorrentistaDao;

public class ManagerBean {

  private List<Correntista> correntistas;

public List<Correntista> getCorrentistas() {
      try{
            correntistas = new CorrentistaDao().findAll();
      }catch(Exception ex){
      }
      return correntistas;
}

public void setCorrentistas(List<Correntista> correntistas) {
      this.correntistas = correntistas;
}
}
```

```
package control;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import entity.Correntista;
import persistence.CorrentistaDao;


@WebServlet("/Controle")
public class Controle extends HttpServlet {

	private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {

	}

	protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
		Correntista co =new Correntista();
		CorrentistaDao dao = new CorrentistaDao();
				try {
			co.setIdCorrentista(new
Integer(request.getParameter("idCorrentista")));
			co.setNome(request.getParameter("nome"));
			co.setEmail(request.getParameter("email"));
			co.setSaldo(new
Double(request.getParameter("saldo")));
				dao.create(co);
			request.setAttribute("msg", "Dados Gravados");
			}catch(Exception ex){
				ex.printStackTrace();
			}finally{
```

```
request.getRequestDispatcher("sistema.jsp").forward(request,
response);
                    }
      }
}
```

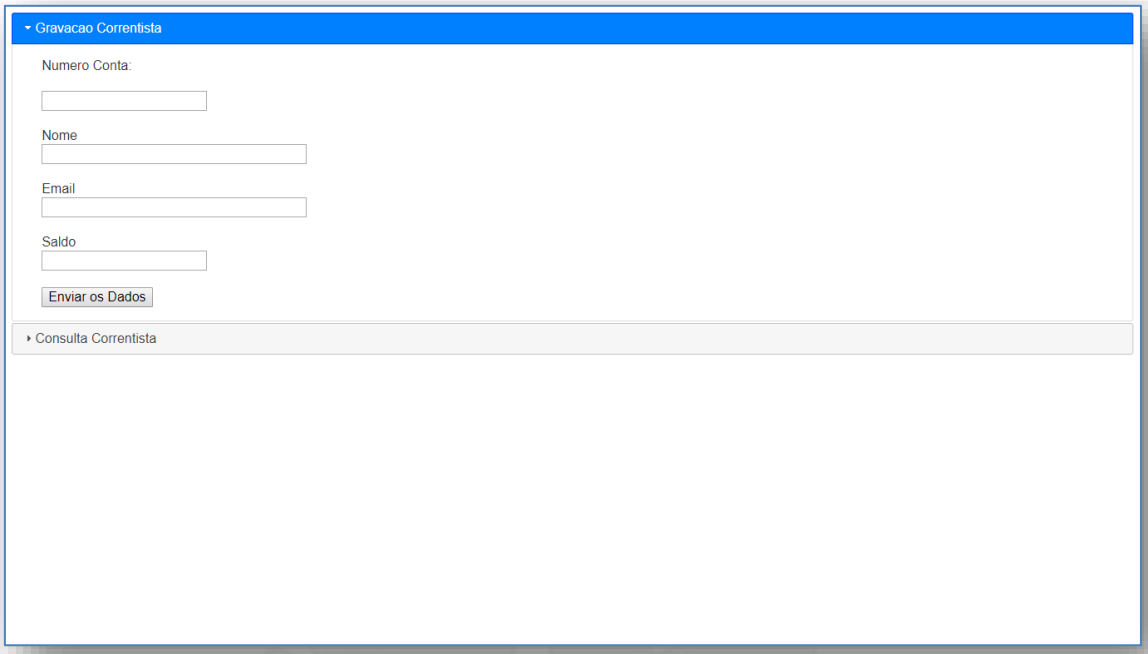
---

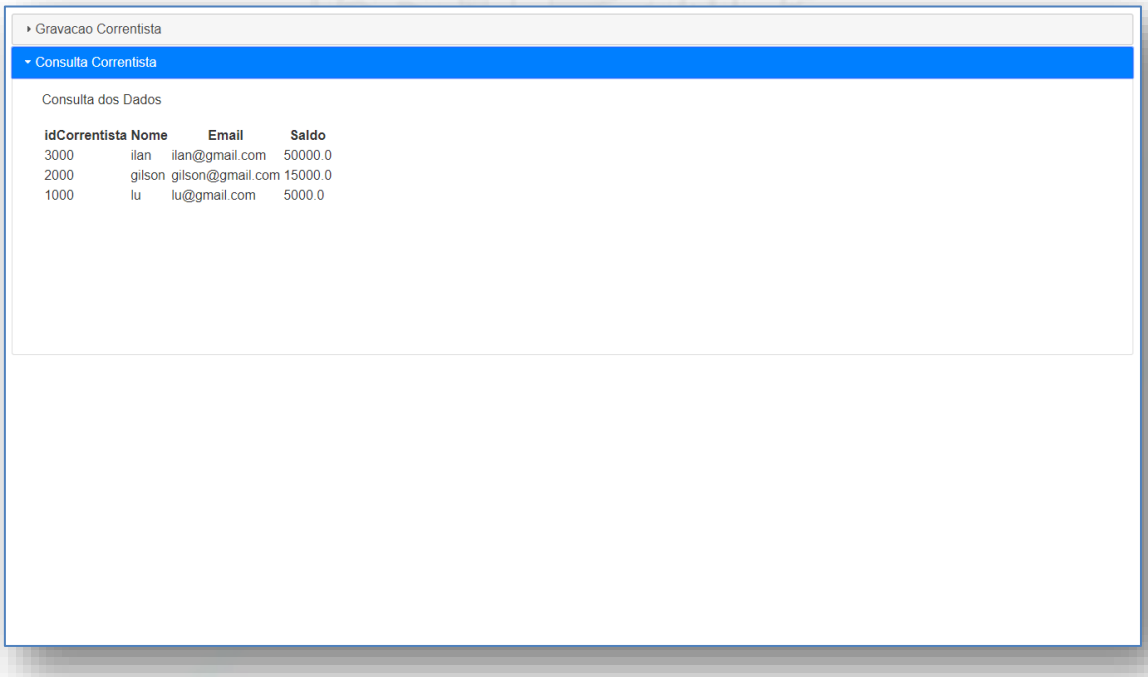
sistema.jsp

```
<!doctype html >
<html>
<head>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.0/jqu
ery-ui.css"/>
<script type="text/javascript"
src="https://code.jquery.com/jquery-3.1.0.js"  ></script>
<script type="text/javascript"
src="https://code.jquery.com/ui/1.12.0/jquery-ui.js"></script>
<script type="text/javascript"
src="https://cdnjs.cloudflare.com/ajax/libs/jquery-
validate/1.16.0/jquery.validate.js"></script>
<script>
 $(document).ready(function(){
        $("#accordion").accordion();
 });
</script>
</head>
<body>
 <div id="accordion">

<h3>Gravacao Correntista</h3>
<div>
      <form method="post" action="Controle?cmd=gravar">
      Numero Conta:<br/><br/>
      <Input type="number" name="idCorrentista"
id="idCorrentista" value="" size="10"
         required="required"/>
      <br/><br/>Nome <br/>
      <input type="text" name="nome" id="nome"  value=""
size="35" required="required" />
      <br/><br/>Email <br/>
      <input type="email" name="email" id="email"  value=""
size="35"  required="required" />
      <br/><br/>Saldo <br/>
```

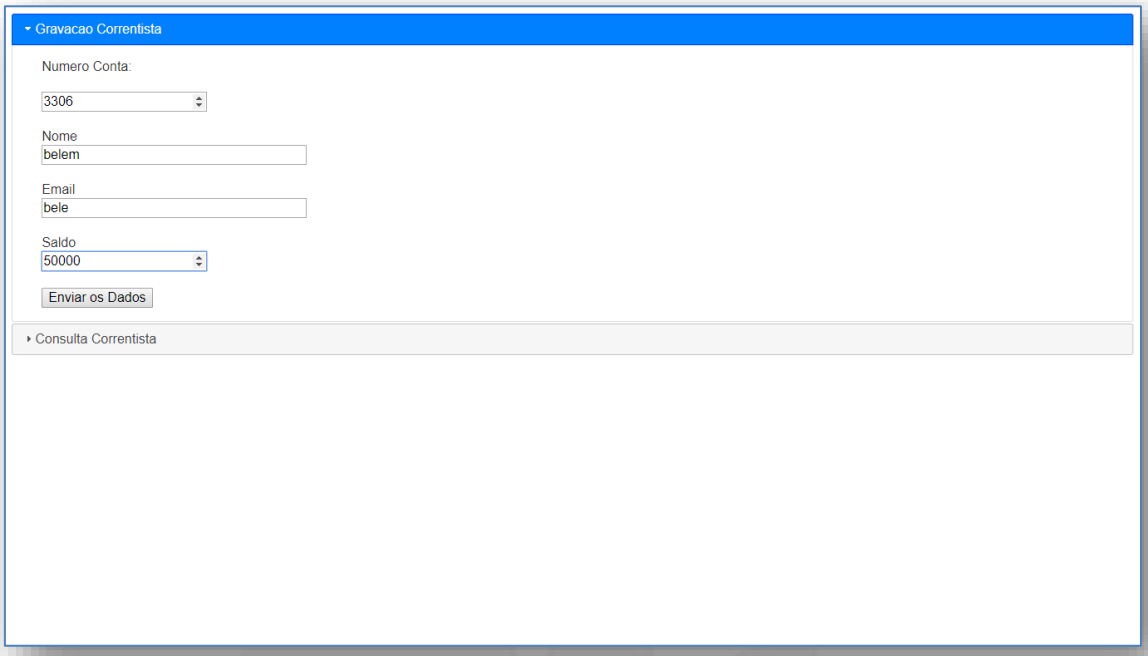
```
      <input type="number" step="0.1" name="saldo" id="saldo"
value="" size="15"
      required="required"
       />
      <br/><br/>
      <button type="submit">Enviar os Dados</button>
</form>
 </div>



 <h3>Consulta Correntista</h3>
<div>
      <jsp:useBean id="mb" class="manager.ManagerBean"
scope="page"></jsp:useBean>
      <%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core" %>
      Consulta dos Dados<br/><br/>
      <table border="0">
      <thead>
      <tr>
       <th>idCorrentista</th>
       <th>Nome</th>
       <th>Email</th>
       <th>Saldo</th>
      </tr>
      </thead>
      <tbody>
      <c:forEach items="${mb.correntistas}" var="linha">
      <tr>
       <td>${linha.idCorrentista}</td>
       <td>${linha.nome}</td>
       <td>${linha.email}</td>
       <td>${linha.saldo}</td>
      </tr>
      </c:forEach>
      </tbody>
      </table>
</div>

</div>
<br/><br/>
${msg}

</body>
```

```
</html>
```

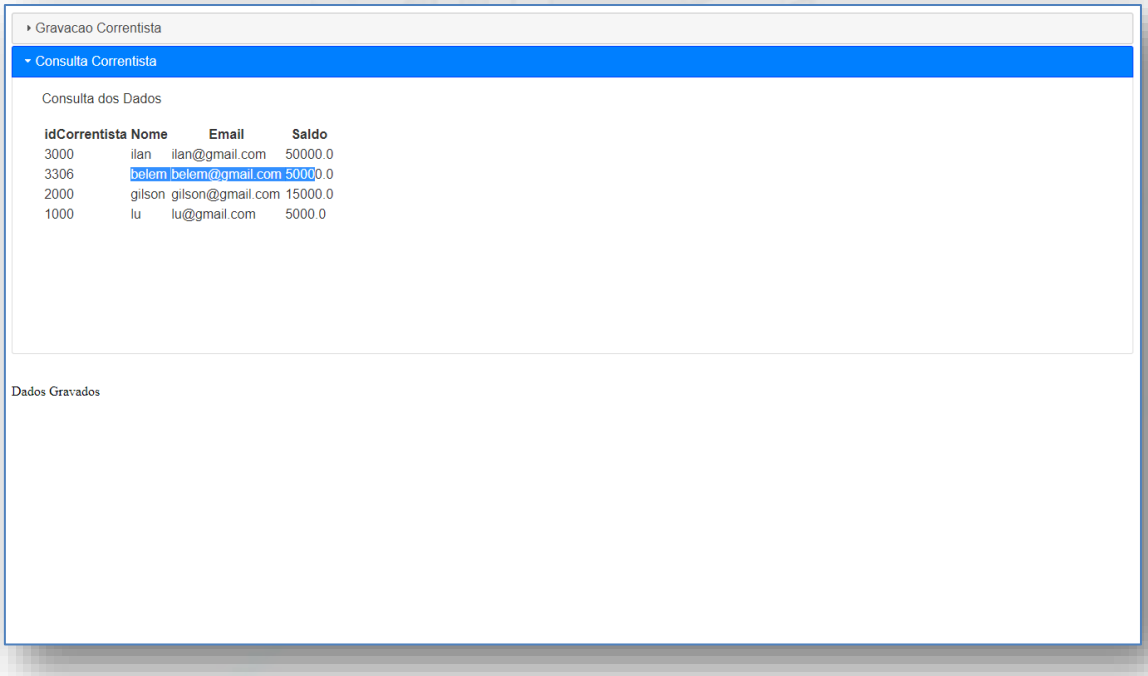

Gravacao Correntista
Numero Conta:
Nome
Email
Saldo
Enviar os Dados
Consulta Correntista


Gravacao Correntista
Consulta Correntista
Consulta dos Dados
idCorrentista Nome Email Saldo
3000 ilan ilan@gmail.com 50000.0
2000 gilson gilson@gmail.com 15000.0
1000 lu lu@gmail.com 5000.0

Gravando...



Cadastrado...



Web service:

Web service é uma solução utilizada na integração de sistemas e na comunicação entre aplicações diferentes. Com esta tecnologia é possível que novas aplicações possam interagir com aquelas que já existem e que sistemas desenvolvidos em plataformas diferentes sejam compatíveis. Os *Web services* são componentes que permitem às aplicações enviar e receber dados em formato XML. Cada aplicação pode ter a sua própria "linguagem", que é traduzida para uma linguagem universal, um formato intermediário como *XML, Json, CSV, etc*.

Para as empresas, os *Web services* podem trazer agilidade para os processos e eficiência na comunicação entre cadeias de produção ou de logística. Toda e qualquer comunicação entre sistemas passa a ser dinâmica e principalmente segura, pois não há intervenção humana.

Essencialmente, o Web Service faz com que os recursos da aplicação do software estejam disponíveis sobre a rede de uma forma normalizada. Outras tecnologias fazem a mesma coisa, como por exemplo, os browsers da Internet acessam às páginas Web disponíveis usando por norma as tecnologias da Internet, HTTP e HTML. No entanto, estas tecnologias não são bem sucedidas na comunicação e integração de aplicações. Existe uma grande motivação sobre a tecnologia Web Service pois possibilita que diferentes aplicações comuniquem - se entre si e utilizem recursos diferentes.

Utilizando a tecnologia Web Service, uma aplicação pode invocar outra para efetuar tarefas simples ou complexas mesmo que as duas aplicações estejam em diferentes sistemas e escritas em linguagens diferentes. Por outras palavras, os Web Services fazem com que os seus recursos estejam disponíveis para que qualquer aplicação cliente possa operar e extrair os recursos fornecidos pelo Web Service.

Os Web Services são identificados por um URI (Uniform Resource Identifier), descritos e definidos usando XML (Extensible Markup Language). Um dos motivos que tornam os Web Services atractivos é o facto deste modelo ser baseado em tecnologias standards, em particular XML e HTTP (Hypertext Transfer Protocol). Os Web Services são utilizados para disponibilizar serviços interactivos na Web, podendo ser acessados por outras aplicações usando, por exemplo, o protocolo SOAP (Simple Object Access Protocol).

O **objetivo dos Web Services** é a comunicação de aplicações através da Internet. Esta comunicação é realizada com intuito de facilitar a EAI (Enterprise Application Integration) que significa a integração das aplicações de uma empresa, ou seja, interoperabilidade entre a informação que circula numa organização nas diferentes aplicações como, por exemplo, o comércio electrónico com os seus clientes e seus fornecedores. Esta interação

constitui o sistema de informação de uma empresa. E para além da interoperabilidade entre as aplicações, a EAI permite definir um workflow entre as aplicações e pode constituir uma alternativa aos ERP (Enterprise Resource Planning). Com umworkflow é possível otimizar e controlar processos e tarefas de uma determinada organização.

Padrão:

A W3C, OASIS são as instituições responsáveis pela padronização dos *Web Services*. Empresas como IBM e Microsoft, duas das maiores do setor de tecnologia, apoiam o desenvolvimento deste padrão.

Segundo o W3C (World Wide Web Consortium) um Web Service define-se como: um sistema de software projectado para suportar a interoperabilidade entre máquinas sobre rede.

Tem uma relação descritiva num formato machine-processable, especificamente WSDL (Webservice Description Language).

Outros sistemas interagem com o Web Service usando as mensagens SOAP, tipicamente sobre HTTP com XML na junção com outros standards da Web.

Tecnologias:

As bases para a construção de um *Web service* são os padrões XML e SOAP. O transporte dos dados é realizado normalmente via protocolo HTTP ou HTTPS para conexões seguras (o padrão não determina o protocolo de transporte). Os dados são transferidos no formato XML, encapsulados pelo protocolo SOAP. Também é bastante comum usar o protocolo REST(Representational transfer protocol), para transferir o estado do dado para a aplicação.

Segurança:

Muitas empresas temiam, no passado, prover funcionalidades na Internet devido ao medo de expor seus dados. Mas com advento dos *Web Services* elas podem publicar serviços de forma simples e que são totalmente isolados da base de dados.

A segurança dos Web Services é um dos pontos fracos desta tecnologia. O problema não é a falta de mecanismos de segurança mas sim a falta de consenso em qual deve ser o mecanismo a ser adaptado pela tecnologia Web Service. As questões mais relevantes na segurança são as seguintes:

- Autenticidade (ter a certeza que uma transação do Web Service ocorreu entre o servidor e seu cliente;
- Privacidade (todas as mensagens trocadas entre o servidor e o cliente não são interceptadas por uma pessoa não autorizada);
- Integridade (as mensagens enviadas tanto pelo servidor ao cliente, como o contrário, devem permanecer inalteradas).

Integração de sistemas:

Muitas pessoas consideram que os *Web services* corrigem um grande problema da informática: a falta de integração de sistemas.

Os *Web services* permitem que a integração de sistemas seja realizada de maneira compreensível, reutilizável e padronizada.

É uma tentativa de organizar um cenário cercado por uma grande variedade de diferentes aplicativos, fornecedores e plataformas.

Tecnologias Utilizadas:

Para a representação e estruturação dos dados nas mensagens recebidas/enviadas é utilizado o XML . As chamadas às operações, incluindo os parâmetros de entrada/saída, são codificadas no protocolo SOAP. Os serviços (operações, mensagens, parâmetros, etc.) são descritos usando a linguagem WSDL. O processo de publicação/pesquisa/descoberta de *Web Services* utiliza o protocolo UDDI.

XML:

Extensible Markup Language (XML) é a base em que os Web Services são construídos. O XML fornece a descrição, o armazenamento, o formato da transmissão para trocar os dados através dos Web Services e também para criar tecnologias Web Services para a troca dos dados.

A sintaxe de XML usada nas tecnologias dos Web Services especifica como os dados são representados genericamente, define como e com que qualidades de serviço os dados são transmitidos, pormenoriza como os serviços são publicados e descobertos. Os Web Services decodificam as várias partes de XML para interagir com as várias aplicações.

SOAP:

O SOAP (Simple Object Access Protocol) baseia-se numa invocação remota de um método e para tal necessita especificar o endereço do componente, o nome do método e os argumentos para esse método. Estes dados são formatados em XML com determinadas regras e enviados normalmente por HTTP para esse componente. Não define ou impõe qualquer semântica, quer seja o modelo de programação, quer seja a semântica específica da implementação. Este aspecto é extremamente importante, pois permite que quer o serviço, quer o cliente que invoca o serviço sejam aplicações desenvolvidas sobre diferentes linguagens de programação. Por esta razão, o SOAP tornou-se uma norma aceita para se utilizar com Web Services, uma tecnologia construída com base em XML e HTTP. Desta forma, pretende-se garantir a interoperabilidade e intercomunicação entre diferentes sistemas, através da utilização da linguagem XML e do mecanismo de transporte HTTP ou outro como, por exemplo, SMTP. O SOAP permite que os documentos XML de envio e de recepção sobre a Web suportem um protocolo comum de transferência de dados para uma comunicação de rede eficaz, ou seja, o SOAP providencia o transporte de dados para os Web Services.

Em relação a Web, o SOAP é um protocolo de RPC que funciona sobre HTTP (ou SMTP, ou outro) de forma a ultrapassar as restrições de segurança/firewalls normalmente impostas aos sistemas clássicos de RPC (RMI, DCOM, CORBA/IIOP) suportando mensagens XML. Em vez de usar HTTP para pedir uma página HTML para ser visualizada num browser, o SOAP envia uma mensagem de XML através do pedido HTTP e recebe uma resposta, se existir, através da resposta do HTTP. Para assegurar corretamente a transmissão da mensagem de XML, o servidor de HTTP, tais como Apache ou IIS (Microsoft Internet Information Server), recebe mensagens SOAP e deve validar e compreender o formato do documento XML definido na especificação SOAP v1.1.

WSDL:

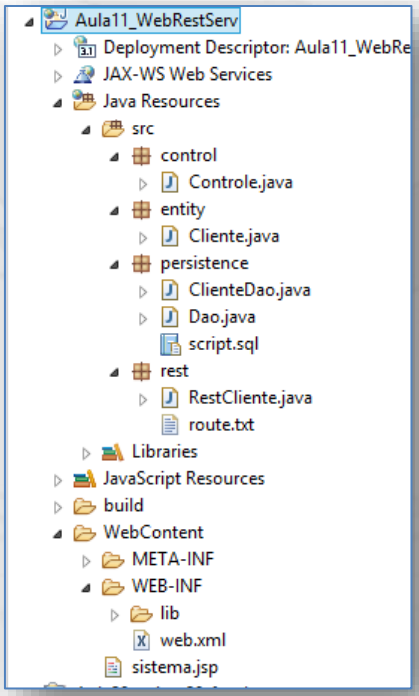
É a sigla de *Web Services Description Language*, padrão baseado em XML para descrever o serviço como no COM, onde ele traz os métodos do *Web Service*. Funciona como uma espécie de "*TypeLibrary*" do *Web Service*, além de ser usado para a validação das chamadas dos métodos.

O WSDL é uma especificação desenvolvida pelo W3C.

O WSDL é extensível para permitir a descrição dos serviços e suas mensagens, independentemente dos formatos de mensagem e dos protocolos de rede que sejam usados. No entanto, é comum usar-se o MIME (Multipurpose Internet Mail Extensions) e o HTtp://SOAP.

O WSDL descreve os serviços disponibilizados à rede através de uma semântica XML, este providencia a documentação necessária para se chamar um sistema distribuído e o procedimento necessário para que esta comunicação se estabeleça. Enquanto que o SOAP especifica a comunicação entre um cliente e um servidor, o WSDL descreve os serviços oferecidos.

Estrutura do projeto:



LIBS:

```
asm-3.1.jar
commons-codec-1.6.jar
commons-logging-1.1.1.jar
fluent-hc-4.2.5.jar
gson-2.2.4.jar
httpclient-4.2.5.jar
```

```
httpclient-cache-4.2.5.jar
httpcore-4.2.4.jar
httpmime-4.2.5.jar
jackson-core-asl-1.9.2.jar
jackson-jaxrs-1.9.2.jar
jackson-mapper-asl-1.9.2.jar
jackson-xc-1.9.2.jar
jaxen-1.1.6.jar
jdom-2.0.5.jar
jersey-client-1.12.jar
jersey-core-1.12.jar
jersey-json-1.12.jar
jersey-server-1.12.jar
jersey-servlet-1.12.jar
jettison-1.1.jar
jsr311-api-1.1.1.jar
mysql-connector-java-5.1.23-bin.jar
xercesImpl.jar
xml-apis.jar
```

```
package entity;

public class Cliente {

	private Integer idCliente;
	private String nome;
	private String email;

	public Cliente() {
	}

	public Cliente(Integer idCliente, String nome, String
email) {
		super();
		this.idCliente = idCliente;
		this.nome = nome;
		this.email = email;
	}

	@Override
	public String toString() {
		return "Cliente [idCliente=" + idCliente + ", nome="
+ nome + ", email=" + email + "]";
```

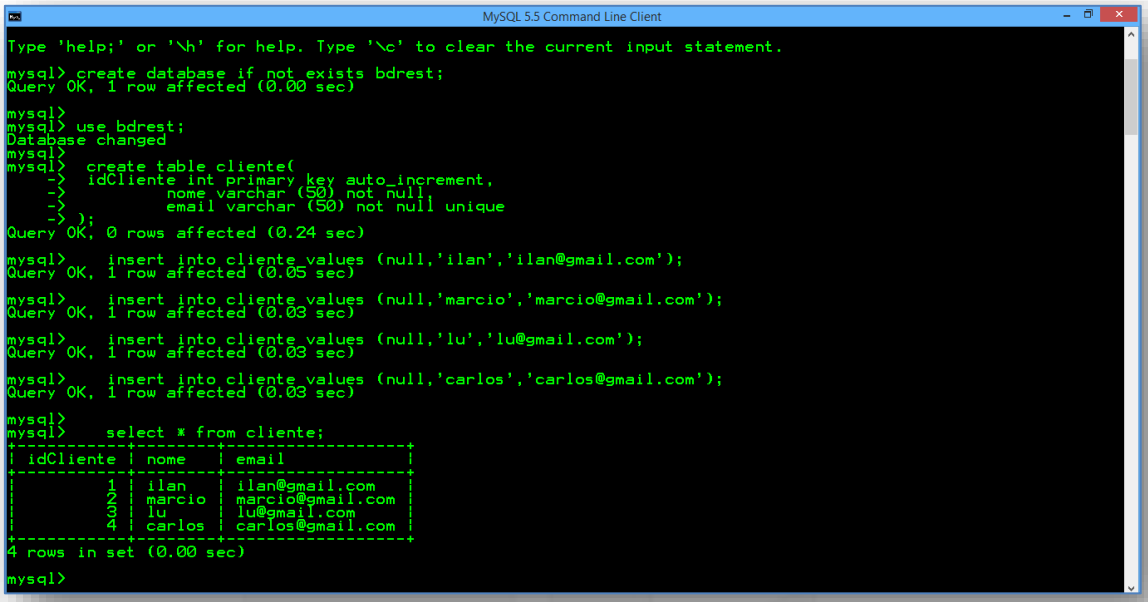
```
	}

	public Integer getIdCliente() {
		return idCliente;
	}
	public void setIdCliente(Integer idCliente) {
		this.idCliente = idCliente;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
}
```

---

script.sql

```
# drop database if exists bdrest;    --Ai MEU DEUS

create database if not exists bdrest;

use bdrest;

 create table cliente(
	idCliente int primary key auto_increment,
	nome varchar (50) not null,
	email varchar (50) not null unique
);

   insert into cliente values (null,'ilan','ilan@gmail.com');
   insert into cliente values
(null,'marcio','marcio@gmail.com');
   insert into cliente values (null,'lu','lu@gmail.com');
   insert into cliente values
(null,'carlos','carlos@gmail.com');
```

```
select * from cliente;
```



```
package persistence;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Dao {

      Connection con;
      PreparedStatement stmt;
      ResultSet rs;

          public static final String BANCO = "bdrest";
          public static final String  USER = "root";
          public static final String  SENHA = "coti";
          public static final String
URL="jdbc:mysql://localhost:3306/"+ BANCO;

        public void open() throws Exception{
             Class.forName("com.mysql.jdbc.Driver");
             con = DriverManager.getConnection(URL,USER,SENHA);
```

```
        }

        public void close() throws Exception{
              con.close();
        }
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;

import entity.Cliente;

public class ClienteDao  extends Dao{

      public void create(Cliente c) throws Exception{
            open();
              stmt = con.prepareStatement("insert into cliente
values (null,?,?)");
               stmt.setString(1, c.getNome());
               stmt.setString(2, c.getEmail());
               stmt.execute();
              stmt.close();
            close();
      }

      public void update(Cliente c) throws Exception{
            open();
       stmt = con.
           prepareStatement("update cliente set nome=?,email=?
where idCliente=?");
               stmt.setString(1, c.getNome());
               stmt.setString(2, c.getEmail());
               stmt.setInt(3, c.getIdCliente());
               stmt.execute();
              stmt.close();
            close();
      }

      public void delete(Integer cod) throws Exception{
            open();
```

```
    stmt = con.prepareStatement("delete from cliente where
idCliente=?");
                stmt.setInt(1, cod);
                stmt.execute();
               stmt.close();
              close();
       }

       public List<Cliente> findAll() throws Exception {
             open();
          stmt = con.prepareStatement("select * from cliente");
               rs = stmt.executeQuery();
               List<Cliente> lst = new ArrayList<Cliente>();
               while(rs.next()){
                Cliente c= new Cliente();
                 c.setIdCliente(rs.getInt(1));
                 c.setNome(rs.getString(2));
                 c.setEmail(rs.getString(3));
                 lst.add(c);
               }
             close();
             return lst;
       }

       public Cliente findByCode(Integer cod) throws Exception {
             open();
                stmt = con.prepareStatement("select * from
cliente where idCliente=?");
               stmt.setInt(1, cod);
               rs = stmt.executeQuery();
               Cliente c=null;
               if(rs.next()){
                 c= new Cliente();
                  c.setIdCliente(rs.getInt(1));
                  c.setNome(rs.getString(2));
                  c.setEmail(rs.getString(3));
               }
             close();
             return c;
       }
}
```

route.txt

Mapear as suas rotas

```
listar = http://localhost:8080/WebRestServ/json/cliente/listar
listarCodigo =
http://localhost:8080/WebRestServ/json/cliente/listarCodigo/1
(GET) gravar =
http://localhost:8080/WebRestServ/json/cliente/gravar/nome/email
(POST)gravar =
http://localhost:8080/WebRestServ/json/cliente/gravar
```

```
package rest;

import javax.ws.rs.Consumes;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;

import com.google.gson.Gson;

import entity.Cliente;
import persistence.ClienteDao;

//  /json/cliente
@Path("/cliente")
public class RestCliente {

      //projeto/json/cliente/gravar/nome/email _ route ...
(gravação)

      @GET
      @Produces("text/plain")
      @Path("/gravar/{nome}/{email}")
      public String gravar(@PathParam("nome") String nome,
                           @PathParam("email") String email){
            try{
                  Cliente c = new Cliente(null,nome,email);
```

```
            new ClienteDao().create(c);
                  return "dados gravados ";
            }catch(Exception ex){
                  return "Error : " + ex.getMessage();
            }
    }

        //veio do Formulário ... (fechado ) ---protecao
        @POST
        @Produces("application/json")
        @Consumes("application/json")
        @Path("/gravar")
        public String gravar(String cliente){
              try   {
        Cliente x = new Gson().fromJson(cliente, Cliente.class);
                  new ClienteDao().create(x);
                  return "Dados Gravados";
              }catch(Exception ex){
                  return "Error :" + ex.getMessage();
              }
        }

          @GET
          @Produces("application/json")
          @Path("/listar")
          public String listar(){
              try{
              return new Gson().toJson(new ClienteDao().findAll());
               }catch(Exception ex){
                  return "Error : " + ex.getMessage();
              }
        }

          @GET
          @Produces("application/json")
          @Path("/listarCodigo/{codigo}")
          public String listarCodigo(
                        @PathParam("codigo") String codigo){
              try{
             return new Gson().toJson(new ClienteDao().findByCode(
new Integer(codigo)  ));
               }catch(Exception ex){
                  return "Error : " + ex.getMessage();
              }
        }
```

```
}
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <servlet>
    <servlet-name>Jersey REST Service</servlet-name>
    <servlet-
class>com.sun.jersey.spi.container.servlet.ServletContainer</ser
vlet-class>
    <init-param>
      <param-
name>com.sun.jersey.config.property.packages</param-name>
```

Colocar o nome exato do pacote onde está a classe Rest...

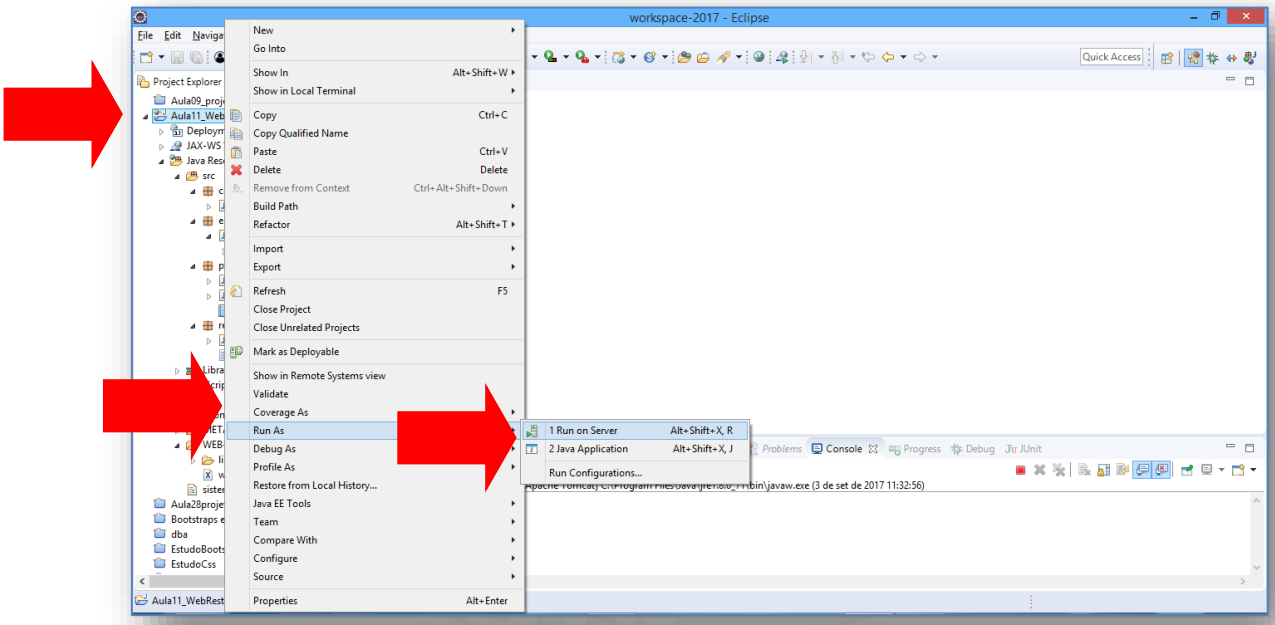
```
      <param-value>rest</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Jersey REST Service</servlet-name>
```

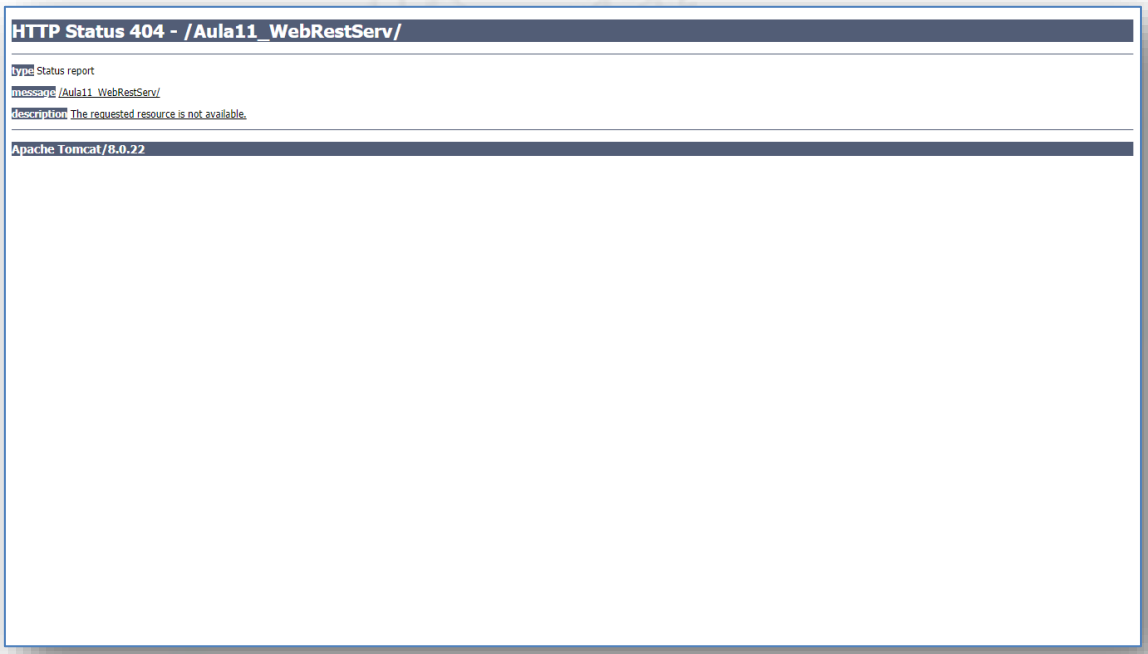
Digitar na URL..

```
    <url-pattern>/json/*</url-pattern>
  </servlet-mapping>
</web-app>
```

Rodando pelo projeto...

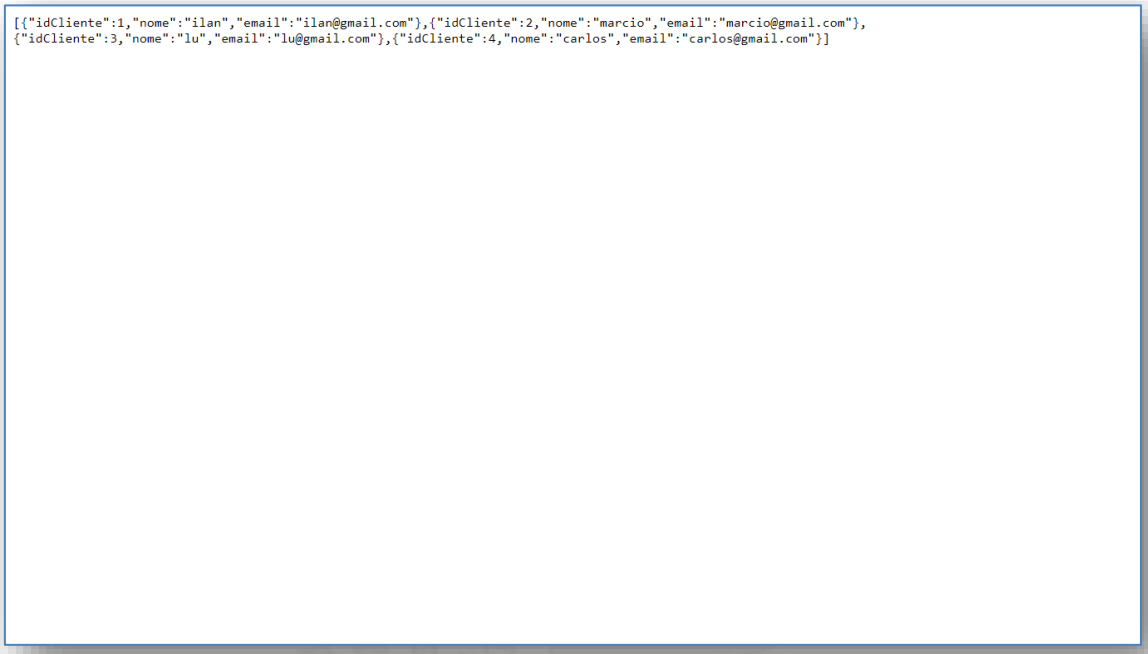
Clicar no projeto com o botão direito -> Run as -> Run on Server...





Digitar na URL o caminho...

http://localhost:8088/Aula11_WebRestServ/json/cliente/listar



```
package control;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.google.gson.Gson;
import com.sun.jersey.api.client.Client;
import com.sun.jersey.api.client.WebResource;

import entity.Cliente;


@WebServlet("/Controle")
```

```
public class Controle extends HttpServlet {
      private static final long serialVersionUID = 1L;
    Client cl;
    Gson gson;

      protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
      }

      protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
           //VIA POST
           try{
           String cmd= request.getParameter("cmd");
               if (cmd.equalsIgnoreCase("gravar")){
                  Cliente  cliente = new Cliente();
cliente.setNome(request.getParameter("nome"));
cliente.setEmail(request.getParameter("email"));
                           cl = Client.create();
                           gson= new Gson();
```

Atenção ao passar a rota!

```
           WebResource wr =
cl.resource("http://localhost:8080/WebRestServ/json/cliente/grav
ar");
                    wr.type("application/json").
                           accept("application/json").
                              post(gson.toJson(  cliente
)); //gravar e converter
                  request.setAttribute("msg", "Dados Gravados");
               }
           }catch(Exception ex){
                ex.printStackTrace();
           }finally{

      request.getRequestDispatcher("sistema.jsp").forward(request
, response);
           }
      }
```

```
}
```

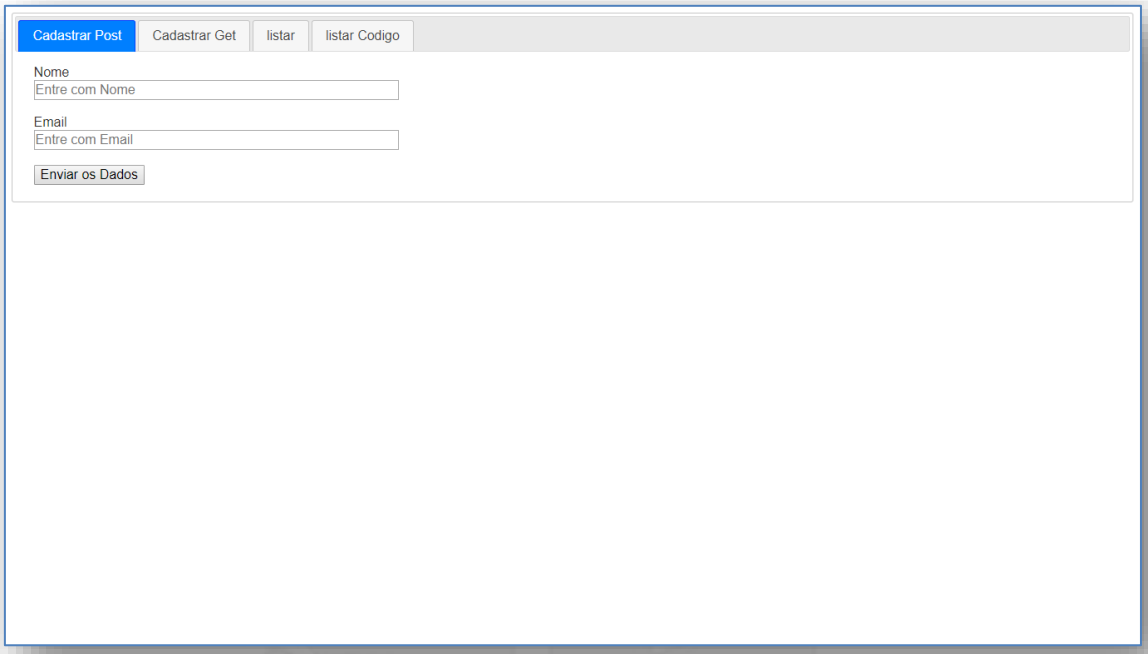
---

sistema.xhtml

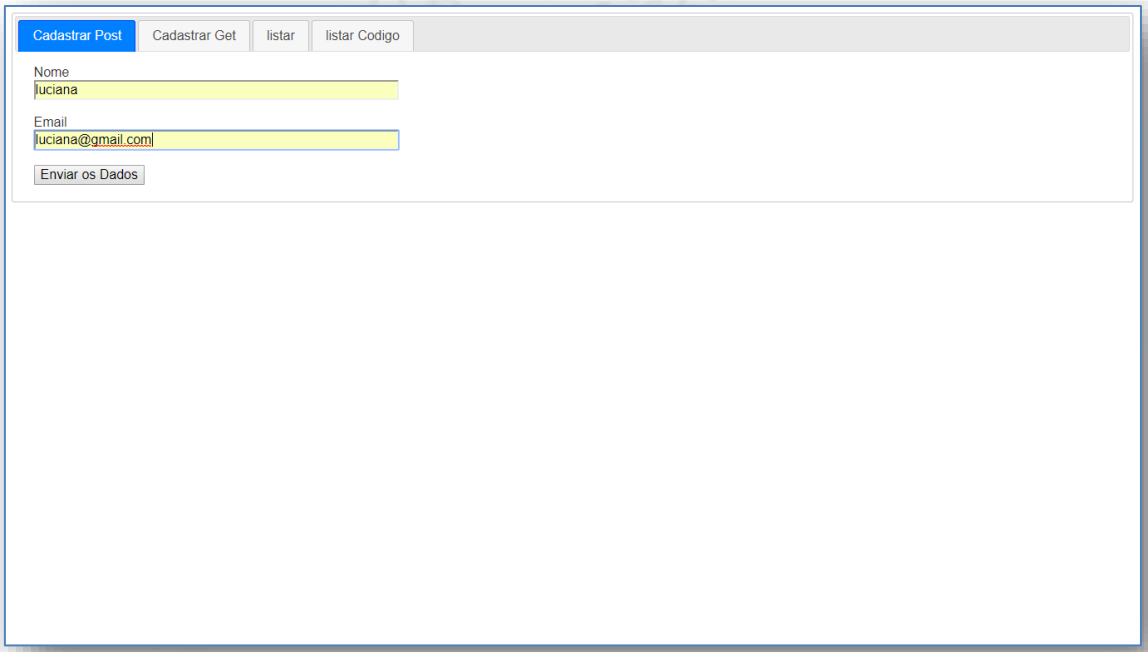
```
<!Doctype html>
<html>
<head>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.0/jquery-ui.css"/>
<script type="text/javascript"
src="https://code.jquery.com/jquery-3.1.0.js"  ></script>
<script type="text/javascript"
src="https://code.jquery.com/ui/1.12.0/jquery-ui.js"></script>
<script type="text/javascript"
src="https://cdnjs.cloudflare.com/ajax/libs/jquery-validate/1.16.0/jquery.validate.js"></script>
<script>
 $(document).ready(function(){
       $("#tabs").tabs();
 });
</script>
</head>
<body>
  <div id="tabs">
  <ul>
   <li><a href="#tab1">Cadastrar Post</a></li>
   <li><a href="#tab2">Cadastrar Get</a></li>
   <li><a href="#tab3">listar</a></li>
   <li><a href="#tab4">listar Codigo</a></li>
  </ul>
 <div id="tab1">
   <form method="post" action="Controle?cmd=gravar" name="form1"
id="form1">
     Nome<br/>
     <input type="text" name="nome" id="nome" value="" size="50"
required="required"
       pattern="[a-z A-Z]{2,50}" placeholder="Entre com Nome"
title="nome"
     />
    <br/><br/> Email<br/>
```

```
      <input type="email" name="email" id="email" value=""
size="50"
         placeholder="Entre com Email" title="email"
       /><br/><br/>
       <button id="btn1" type="submit">Enviar os Dados</button>
    </form>
 </div>
    <div id="tab2">
    <form method="get" action="Controle" name="form2" id="form2">
     <input type="hidden" name="cmd" value="gravarGet" />
      Nome<br/><br/>
      <input type="text" name="nome" id="nome" value="" size="50"
required="required"
        pattern="[a-z A-Z]{2,50}" placeholder="Entre com Nome"
title="nome"
      />
      Email<br/><br/>
      <input type="email" name="email" id="email" value=""
size="50"
         placeholder="Entre com Email" title="email"
       />
       <button id="btn1" type="submit">Enviar os Dados</button>
    </form>
 </div>
  </div>
  ${msg}
</body>
</html>
```

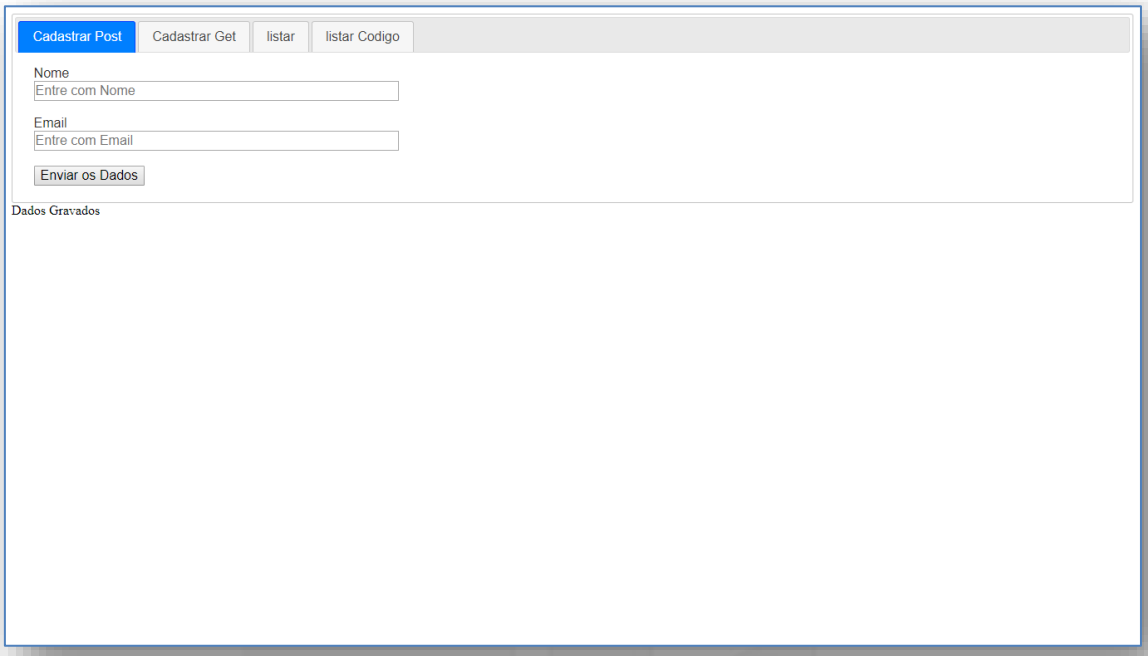
Rodando pela página sistema.jsp...



Cadastrando via método POST...



Gravado...


Cadastrar Post
Cadastrar Get
listar
listar Codigo
Nome
Entre com Nome
Email
Entre com Email
Enviar os Dados
Dados Gravados

Spring MVC

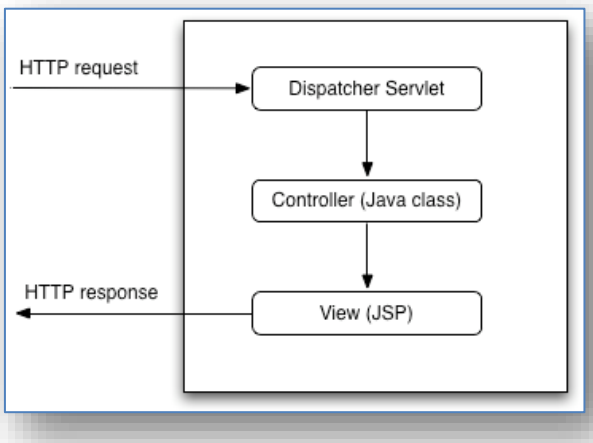
O Spring MVC pertence ao Spring Framework e tem como principal proposta possibilitar uma maior produtividade no desenvolvimento de aplicações *web* corporativas utilizando a linguagem Java. Baseando-se no já comentado padrão de projeto MVC, ele consegue separar as principais fronteiras de uma aplicação *web*, fazendo com que nosso código fique bastante organizado.

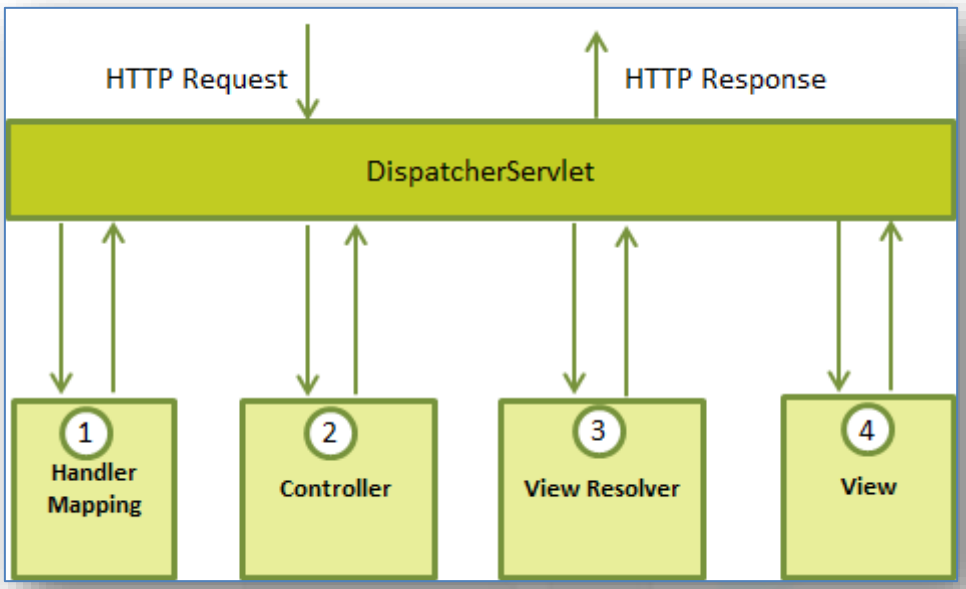
Por se tratar de um framework leve e não intrusivo, ou seja, não requer que classes da aplicação tenham contato com as classes da biblioteca, e pelo fato de estar integrado ao Spring Framework, o Spring MVC tem à disposição serviços como injeção de dependências, gerenciamento de transações e de segurança, além da integração com frameworks de mapeamento objeto-relacional como o Hibernate.

O funcionamento básico do Spring MVC se dá através da classe **DispatcherServlet**, que é um *Front Controller* e estende **HttpServlet**. Ela tem como funções principais direcionar as requisições para outros controladores, tratá-las e repassar as respostas para o usuário.

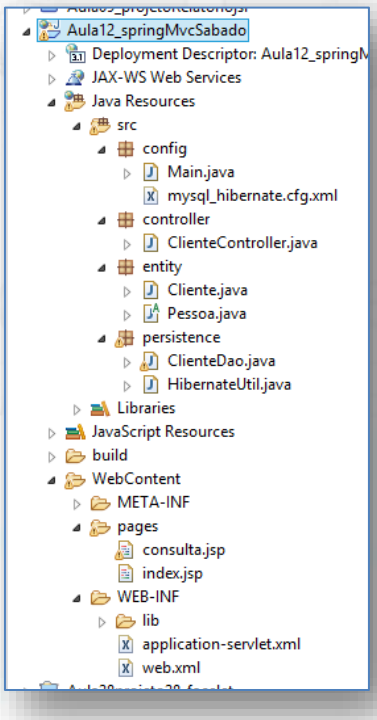
Após o recebimento da solicitação HTTP, o **DispatcherServlet** consulta o **HandlerMapping**, que redireciona a solicitação para outro controlador de acordo com suas configurações de comportamento.

A **Figura 1** mostra como se dá o fluxo de uma requisição com o **DispatcherServlet**.





Estrutura do projeto:



LIBS:

```
antlr-2.7.6.jar
aopalliance-1.0.jar
asm-attrs.jar
asm.jar
c3p0-0.9.0.jar
cglib-2.1.3.jar
commons-codec-1.3.jar
commons-collections-3.2.1.jar
commons-el-1.0.jar
commons-lang-2.1.jar
commons-logging-1.1.1.jar
commons-logging-1.1.3.jar
commons-validator.jar
dom4j-1.6.1.jar
ejb3-persistence.jar
hibernate-annotations.jar
hibernate-commons-annotations.jar
hibernate-validator-3.0.0.ga.jar
hibernate3.jar
hsqldb.jar
jakarta-oro.jar
jcl-over-slf4j-1.6.1.jar
jstl-api-1.2.jar
jstl-impl-1.2.jar
jta.jar
log4j-1.2.15.jar
log4j-1.2.16.jar
mysql-connector-java-5.1.23-bin.jar
oro-2.0.8.jar
servlet3-api.jar
slf4j-api-1.6.1.jar
slf4j-log4j12-1.6.1.jar
spring-aop-4.1.0.RELEASE.jar
spring-beans-4.1.0.RELEASE.jar
spring-context-4.1.0.RELEASE.jar
spring-core-4.1.0.RELEASE.jar
spring-expression-4.1.0.RELEASE.jar
spring-web-4.1.0.RELEASE.jar
spring-webmvc-4.1.0.RELEASE.jar
```

```
package entity;

import javax.persistence.Column;
import javax.persistence.MappedSuperclass;


@MappedSuperclass
public abstract class Pessoa {

	@Column(length=50)
	 private String nome;
	@Column(length=50,unique=true)
	 private String email;

	 public Pessoa() {
	}

	public Pessoa(String nome, String email) {
		this.nome = nome;
		this.email = email;
	}


	@Override
	public String toString() {
		return "Pessoa [nome=" + nome + ", email=" + email +
"]";
	}

	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
}
```

```
package entity;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Cliente extends Pessoa{

	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	  private Integer idCliente;
	@Column(columnDefinition=" enum ('m','f') ")
	  private String  sexo;

	public Cliente() {
	}

	public Cliente( Integer idCliente,String nome, String
email, String sexo) {
		super(nome, email);
		this.idCliente = idCliente;
		this.sexo = sexo;
	}

	@Override
	public String toString() {
		return "Cliente [idCliente=" + idCliente +",
getNome()=" + getNome() + ", sexo=" + sexo  + ", getEmail()=" +
getEmail() + "]";
	}

	public Integer getIdCliente() {
		return idCliente;
	}
	public void setIdCliente(Integer idCliente) {
		this.idCliente = idCliente;
	}
	public String getSexo() {
		return sexo;
```

```
	}
	public void setSexo(String sexo) {
		this.sexo = sexo;
	}
}
```

---

mysql_hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</pro
perty>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</
property>
    <property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/hib02?createDatabaseIfNotExist=true<
/property>
    <property
name="hibernate.connection.username">root</property>
    <property
name="hibernate.connection.password">coti</property>

    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

       <mapping class="entity.Pessoa"/>
      <mapping class="entity.Cliente"/>

 </session-factory>
</hibernate-configuration>
```

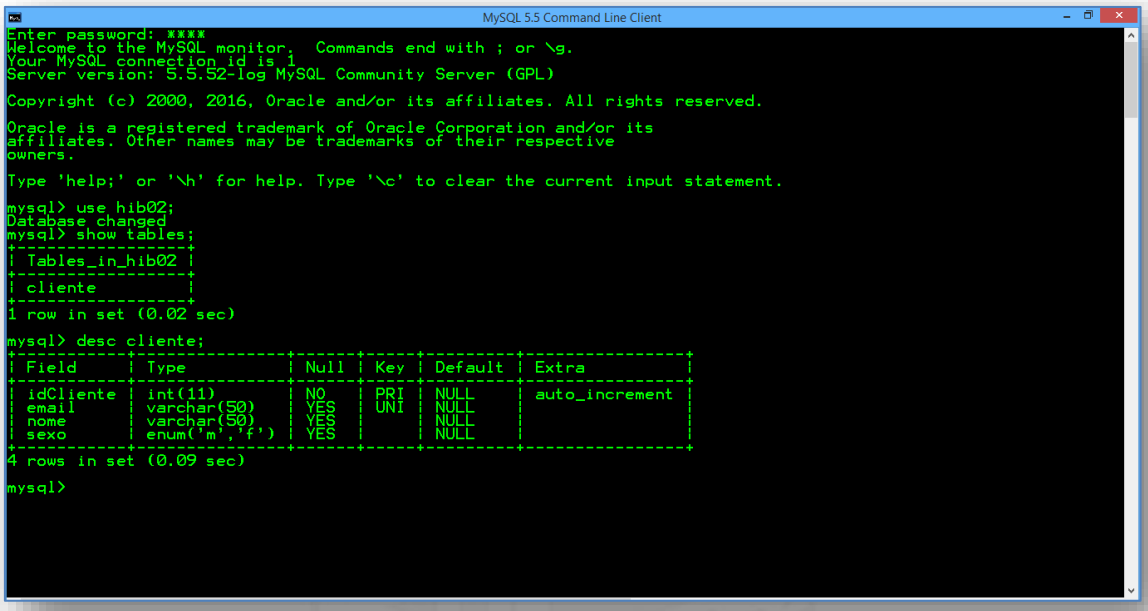
```
package config;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.cfg.Configuration;
import org.hibernate.tool.hbm2ddl.SchemaExport;

public class Main {

      public static void main(String[] args) {

            Configuration cfg = new AnnotationConfiguration().
     configure("config/mysql_hibernate.cfg.xml");
            new SchemaExport(cfg).create(true,true);
          System.out.println("Tabela Gerada com Sucesso !!!");
      }
}
```

No console...

```
log4j:WARN No appenders could be found for logger
(org.hibernate.cfg.annotations.Version).
log4j:WARN Please initialize the log4j system properly.

    drop table if exists Cliente

    create table Cliente (
        idCliente integer not null auto_increment,
        email varchar(50) unique,
        nome varchar(50),
        sexo  enum ('m','f') ,
        primary key (idCliente)
    )
Tabela Gerada com Sucesso !!!
```

No banco...

Criou somente uma tabela com os dados das duas classes...




```
package persistence;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;

public class HibernateUtil {

    private static final SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new AnnotationConfiguration().
configure("config/mysql_hibernate.cfg.xml").buildSessionFactory(
);
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation
failed." + ex);
            throw new ExceptionInInitializerError(ex);
```

```
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

```
package persistence;

import java.util.List;

import org.hibernate.Criteria;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.Transaction;

import entity.Cliente;

public class ClienteDao {

        Session session;
        Query query;
        Transaction transaction;
        Criteria criteria;

    public void create(Cliente c) throws Exception{
session =  HibernateUtil.getSessionFactory().openSession();
            transaction =  session.beginTransaction();
                    session.save(c);
                transaction.commit();
            session.close();
    }

    public void update(Cliente c) throws Exception{
session =  HibernateUtil.getSessionFactory().openSession();
            transaction =  session.beginTransaction();
                    session.update(c);
                transaction.commit();
            session.close();
    }
```

```
        public void delete(Cliente c) throws Exception{
    session =  HibernateUtil.getSessionFactory().openSession();
                transaction =  session.beginTransaction();
                       session.delete(c);
                transaction.commit();
            session.close();
        }

        public List<Cliente> findAll() {
     session =  HibernateUtil.getSessionFactory().openSession();
                    List<Cliente> lista =
session.createQuery("from Cliente").list();
            session.close();
             return  lista;
        }

        public Cliente findByCode(Integer cod) {
     session =  HibernateUtil.getSessionFactory().openSession();
          Cliente c = (Cliente) session.get(Cliente.class, cod);
           session.close();
              return  c;
        }
}
```

```
package controller;

import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.servlet.ModelAndView;

import entity.Cliente;
import persistence.ClienteDao;

@Controller
public class ClienteController {
      @RequestMapping(value="/", method=RequestMethod.GET)
      public ModelAndView iniciar(){
            return new ModelAndView("index"); //abrir
```

```
	}

@RequestMapping(value="/cadastrar.html",method=RequestMethod.POST)
	public  ModelAndView cadastrar(Cliente c){
		ModelAndView mv = new ModelAndView("index");
//voltar
		 try{
		     new ClienteDao().create(c);
		  mv.addObject( "msg", " Dados Gravados  ...");
		 }catch(Exception ex){
	   mv.addObject( "msg", "Error :" + ex.getMessage());
		 }
		return mv;
	} //volta para o index.jsp

@RequestMapping(value="/lista.html",method=RequestMethod.GET)
	public  ModelAndView lista(){
		ModelAndView mv = new ModelAndView("consulta");
//voltar
		 try{
		List <Cliente> lst = new ClienteDao().findAll();
		        mv.addObject( "lst", lst);
		 }catch(Exception ex){
		 mv.addObject( "msg", "Error :" + ex.getMessage());
		 }
		return mv;
	}

	//link que irÃ¡ para consulta.jsp

@RequestMapping(value="/excluir.html/{codigo}",method=RequestMethod.GET)
	public  ModelAndView excluir(@PathVariable("codigo")
Integer codigo){
		ModelAndView mv = new ModelAndView("consulta");
		 try{
		      ClienteDao dao = new ClienteDao();
		 new ClienteDao().delete(dao.findByCode(codigo));
		       mv.addObject( "lst", dao.findAll());
		       mv.addObject("msg","Registro Excluido");
		 }catch(Exception ex){
		 mv.addObject( "msg", "Error :" + ex.getMessage());
```

```
            }
            return mv;
        }
}
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <display-name>springMvcSabado</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>


  <servlet>
          <servlet-name>appServlet</servlet-name>
          <servlet-
class>org.springframework.web.servlet.DispatcherServlet</servlet
-class>
          <init-param>
               <param-name>contextConfigLocation</param-name>
               <param-value>/WEB-INF/application-
servlet.xml</param-value>
          </init-param>
          <load-on-startup>1</load-on-startup>
     </servlet>
     <servlet-mapping>
          <servlet-name>appServlet</servlet-name>
          <url-pattern>/</url-pattern>
     </servlet-mapping>
</web-app>
```

---

application-servlet.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:context="http://www.springframework.org/schema/contex
t"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:mvc="http://www.springframework.org/schema/mvc"
      xmlns:jpa="http://www.springframework.org/schema/data/jpa"
      xsi:schemaLocation="
http://www.springframework.org/schema/beans
      http://www.springframework.org/schema/beans/spring-beans-
4.0.xsd
      http://www.springframework.org/schema/data/jpa
    http://www.springframework.org/schema/data/jpa/spring-
jpa.xsd
      http://www.springframework.org/schema/context
      http://www.springframework.org/schema/context/spring-
context-4.0.xsd
      http://www.springframework.org/schema/mvc
    http://www.springframework.org/schema/mvc/spring-mvc-
4.0.xsd"
      >
```

Atenção ao nome do pacote onde está a classe de controle...

```
      <context:component-scan base-
package="controller"></context:component-scan>
      <mvc:annotation-driven></mvc:annotation-driven>

      <bean

      class="org.springframework.web.servlet.view.InternalResourc
eViewResolver">
            <property name="prefix">
```

Atenção ao nome do pacote onde se encontram as páginas web...

```
            <value>/pages/</value>
        </property>
        <property name="suffix">
            <value>.jsp</value>
        </property>
    </bean>
</beans>
```

Criar a pasta "pages" dentro de "WebContent"...

index.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-
8">
<title>Insert title here</title>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.0/jqu
ery-ui.css"/>
<script type="text/javascript"
src="https://code.jquery.com/jquery-3.1.0.js"  ></script>
<script type="text/javascript"
src="https://code.jquery.com/ui/1.12.0/jquery-ui.js"></script>
<script>
    $(document).ready(function(){
        $("#accordion").accordion();
    });
</script>

</head>
<body>
<div id="accordion">
<h3>Cadastrar os Cliente</h3>
<div>
   <form method="post" action="cadastrar.html">
     Nome : <br/><br/>
```

```
      <input type="text" name="nome" id="nome"   size="50"
required="required" />
    <br/>Email : <br/><br/>
     <input type="email" name="email" id="email"   size="50"
required="required" />
    <br/>Sexo : <br/><br/>
     <select name="sexo"  required="required">
        <option value="">Escolha o Sexo</option>
         <option value="m"> Masculino</option>
         <option value="f"> Feminino</option>
     </select>
      <button id="btn1" type="submit">Enviar os Dados</button>
      <br/><br/>
          ${msg}
       <br/><br/>
       <a href="lista.html">Consultar Cliente</a>
   </form>
   </div>
  </div>
</body>
</html>
```

---

consulta.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-
8">
<title>Insert title here</title>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.0/jqu
ery-ui.css"/>
<script type="text/javascript"
src="https://code.jquery.com/jquery-3.1.0.js"  ></script>
<script type="text/javascript"
src="https://code.jquery.com/ui/1.12.0/jquery-ui.js"></script>
<script>
    $(document).ready(function(){
```

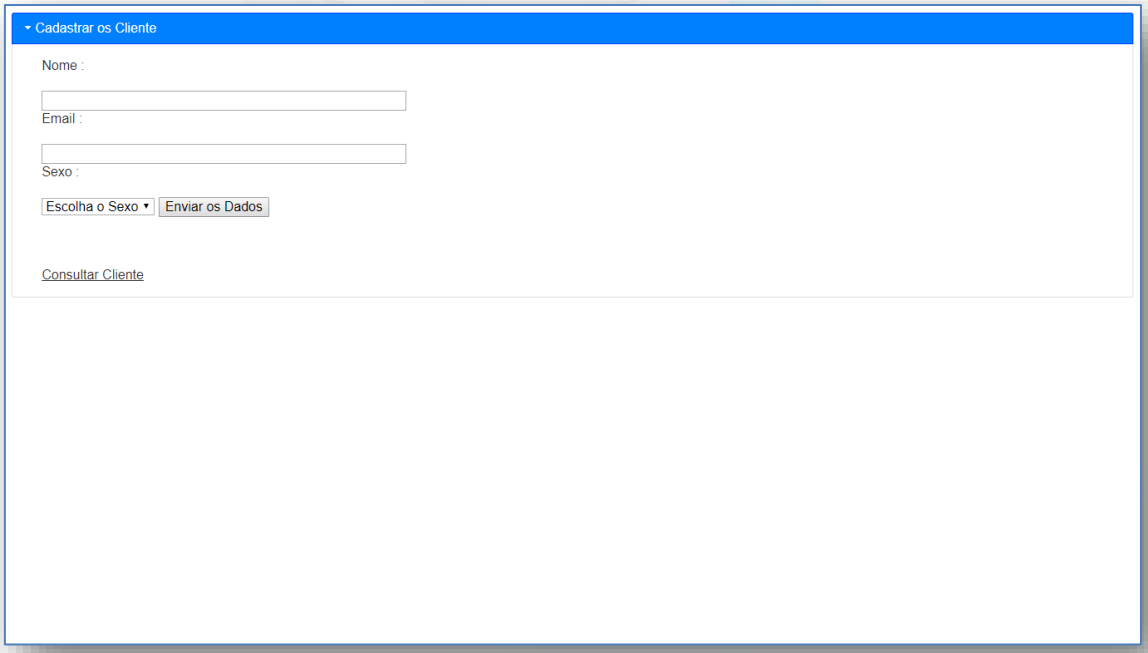
```
        $("#accordion").accordion({active:0});
    });
</script>

</head>
<body>
<%@  taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"
%>
<%@  taglib prefix="fn"
uri="http://java.sun.com/jsp/jstl/functions" %>
<div id="accordion">
 <h3>Consultar  Cliente</h3>
  <div>
     <table class="table table-hover  table-bordered">
          <thead>
            <tr>
              <td>idCliente</td>
              <td>Nome</td>
              <td>Email</td>
              <td>Sexo</td>
              <td>Excluir</td>
              <td>Alterar</td>
            </tr>
          </thead>
               <tbody>
               <c:forEach items="${lst}"  var="linha">
             <tr>
                <td>${linha.idCliente}</td>
                <td>${linha.nome}</td>
               <td>${linha.email}</td>
                <td>${linha.sexo}</td>
                <td><a
href="/springMvcSabado/excluir.html/${linha.idCliente}">Excluir<
/a></td>
                <td>Alterar</td>
            </tr>
            </c:forEach>
          </tbody>
     </table>


   </div>
  </div>
```

```
  <br/><br/>
```

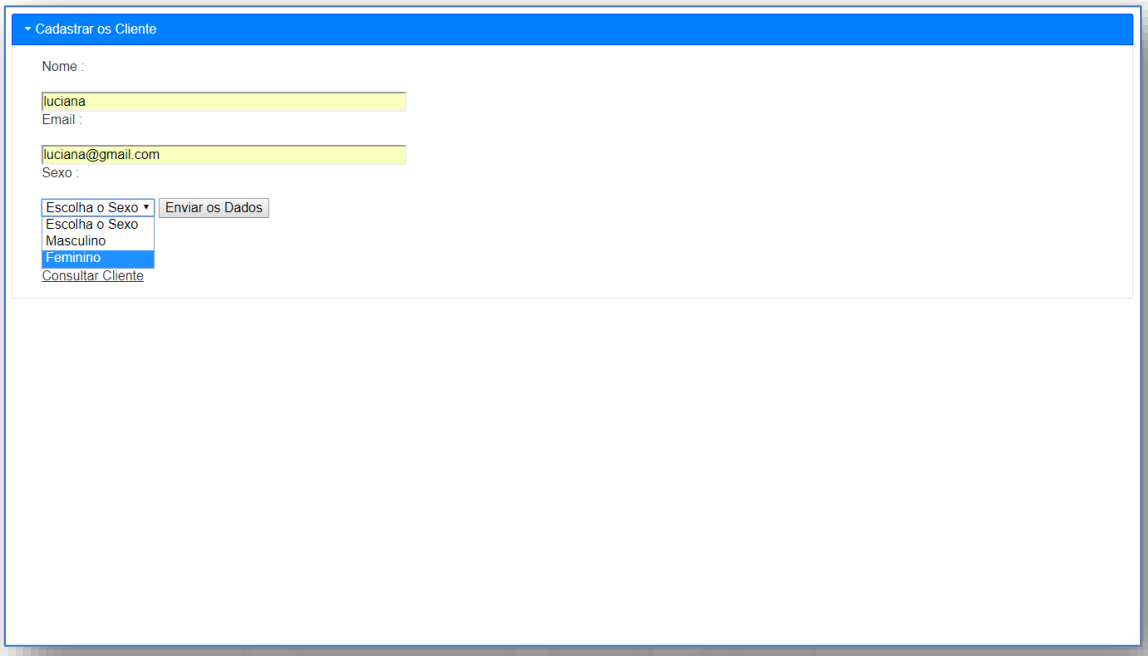
Atenção no link, é o nome do projeto...

```
  <a href="/springMvcSabado/">Voltar para Principal</a>
<br/>
</body>
</html>
```

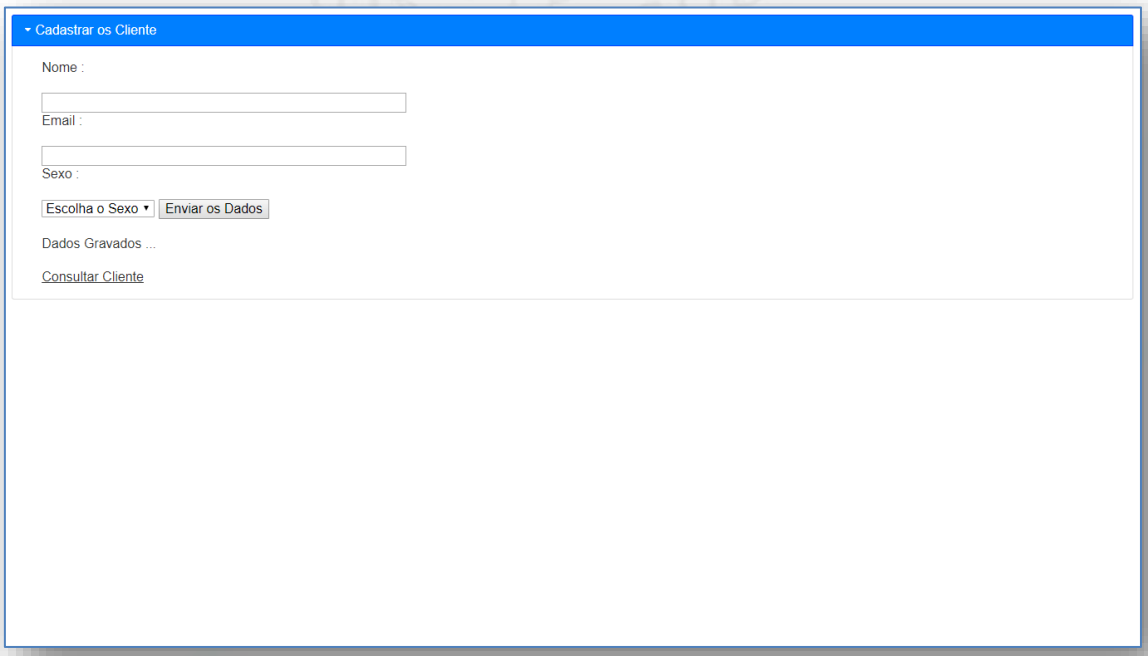
Para rodar o sistema tem que ser pelo projeto. Clicando no projeto com o botão direito...



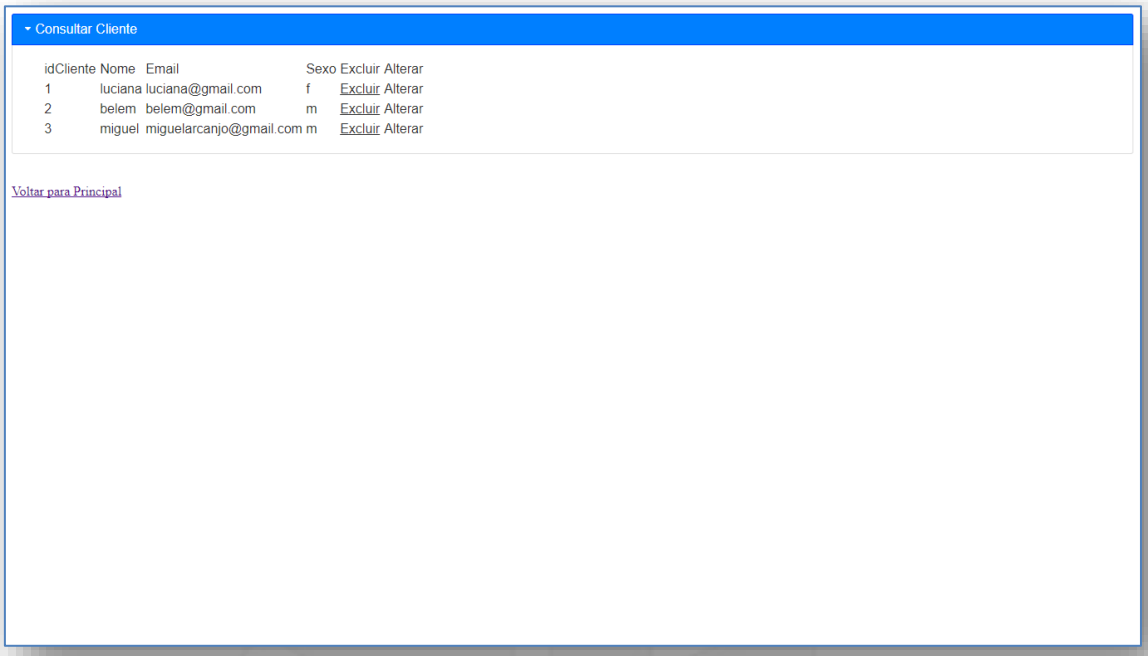
Cadastrando um cliente...



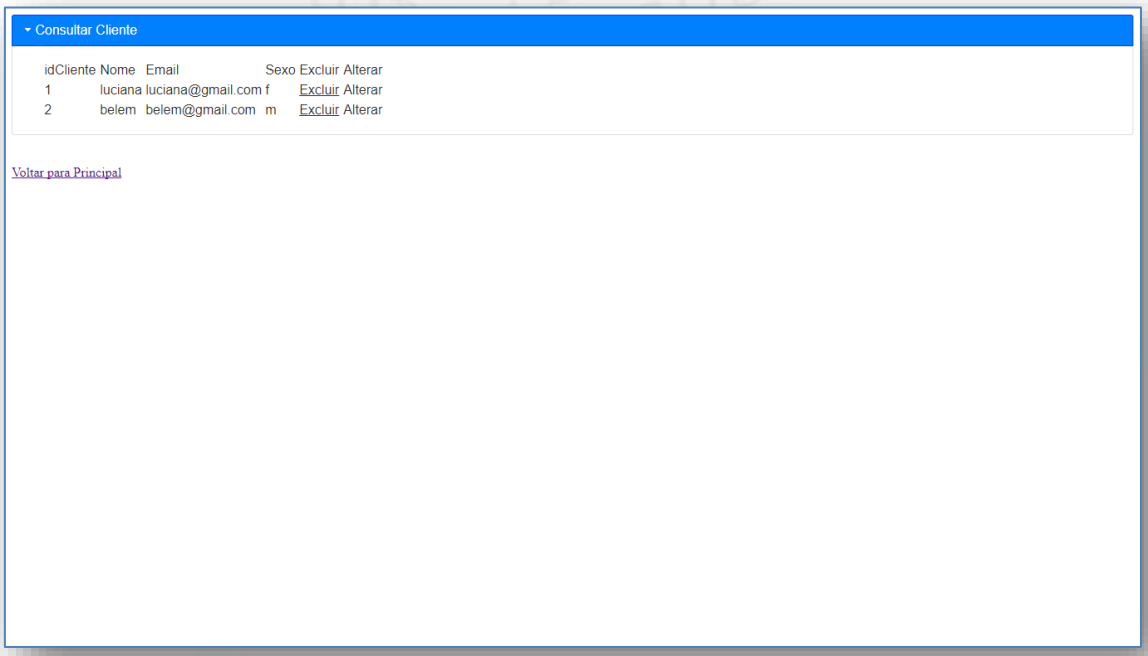
Gravado...



Clicando em consultar...



Excluindo Miguel...



JavaServer Faces:

JavaServer Faces (**JSF**) é uma especificação Java para a construção **JavaServer Faces** (**JSF**) é uma especificação Java para a construção de interfaces de usuário baseadas em componentes paraaplicações web. Possui um modelo de programação dirigido a eventos, abstraindo os detalhes da manipulação dos eventos e organização dos componentes, permitindo que o programador se concentre na lógica da aplicação.

Foi formalizada como um padrão através do Java Community Process e faz parte da Java Platform, Enterprise Edition.

JSF 2 utiliza Facelets como seu sistema de template padrão. Outras tecnologias da camada de visão, como XUL também podem ser empregadas. Em contraste, JSF 1.x utiliza JavaServer Pages (JSP) como seu sistema de template padrão.

Histórico:

O JavaServer Faces ganhou expressão na versão 1.1 quando implementado pela comunidade utilizando a especificação 127 [1] do Java Community Process [2] , evidenciando maturidade e segurança.

Hoje ele está na versão 2.0 da especificação 252 [3] do JCP. A fundação Apache vem realizando esforços na implementação da especificação através do projeto MyFaces. O reconhecimento do trabalho é visto por diversas empresas, tanto é que a Oracle doou os fontes do ADF Faces, conjunto de mais de 100 componentes JSF, para o projeto MyFaces que o denominará de Trinidad.

O JSF é atualmente considerado pela comunidade Java como a última palavra em termos de desenvolvimento de aplicações Web utilizando Java, resultado da experiência e maturidade adquiridas com o JSP/Servlet (Model1), Model2 (MVC) e Struts.

Como trabalha:

JavaServer Faces é baseada em um modelo de desenho de IU (interface de usuário) baseada em componentes, usando arquivos XML chamados de modelos de visão ouFacelets views. Os pedidos são processados pelo

FacesServlet, que carrega o modelo de visão adequado, constrói uma árvore de componentes, processa os eventos e apresenta a resposta, normalmente na linguagem HTML, para o cliente. O estado de componentes de interface do usuário e outros objetos de interesse de escopo, é salvo no final de cada pedido em um processo chamado **stateSaving** (nota: *transiente* **true**) e restaurado na próxima criação desta visão. Objetos e estados podem ser salvos ou no cliente ou no servidor.

Características:

- Permite que o desenvolvedor crie UIs através de um conjunto de componentes UIs pré-definidos;
- Fornece um conjunto de tags JSP para acessar os componentes;
- Reutiliza componentes da página;
- Associa os eventos do lado cliente com os manipuladores dos eventos do lado do servidor (os componentes de entrada possuem um valor local representando o estado no lado servidor);
- Fornece separação de funções que envolvem a construção de aplicações Web.
- Utiliza Ajax em alguns de seus componentes tornando alguns processos mais rápidos e eficientes.

Ciclo de vida:

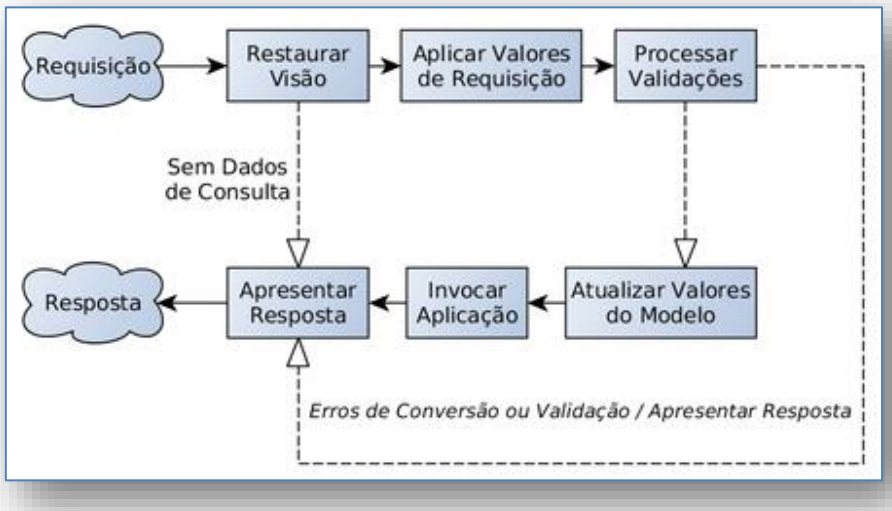


Diagrama mostrando o fluxo do ciclo de vida do JSF.

Ciclo de vida do JSF é o nome dado à sequência de processamento realizada na implementação JSF para a geração das visões. O JSF, diferente de outros frameworks, possui um processamento de requisição dividido em seis fases:

1. Restauração da visão - o objetivo principal desta fase é construir a árvores de componentes.[7]Ela utiliza o template e cria a árvore inicial através da análise da requisição. Após isto, ela salva o estado da árvore no objeto `FacesContext`. Nas requisições subsequentes, ela cria a árvore do estado e procede a execução do resto do ciclo de vida.
2. Aplicação dos valores de requisição - **o JSF pega cada componente da árvore** começando com a raiz e a cria ou recupera do objeto `FacesContext`. Cada componente na árvore gerencia seus próprios valores e toma-os dos parâmetros, cookies e cabeçalhos da requisição HTTP.
3. Validações de processo - o JSF realiza a conversão e validação sobre todos os componentes começando com o raiz. O valor submetido de cada componente é convertido em um objeto e validado chamando-se o validador registrado. O JSF salva o valor submetido. Se ocorrer um erro durante a conversão ou validação, o ciclo de vida escapa diretamente para a fase de "apresentação da resposta".
4. Atualização de valores de modelo - durante esta fase, o valor do componente é passado para o modelo através da atualização das propriedades dos backing beans.
5. Invocação da aplicação - a manipulação de evento para cada ação e ouvidor de ação é executada começando com o(s) ouvidor(es) de ação e então a chamada do método de ação.
6. Apresentação da resposta

Recursos visuais:

O framework JSF permite a inserção, via IDE, de:

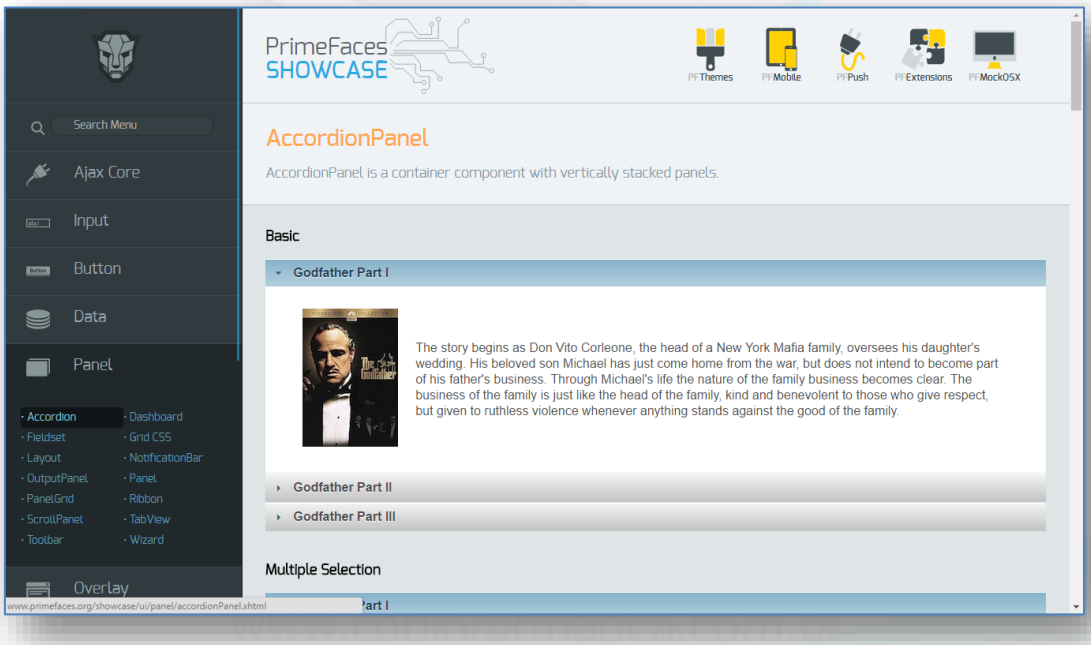
- Folhas de estilo (CSS);
- Comandos em JavaScript;
- Metodologia Ajax.

JSF inclui:

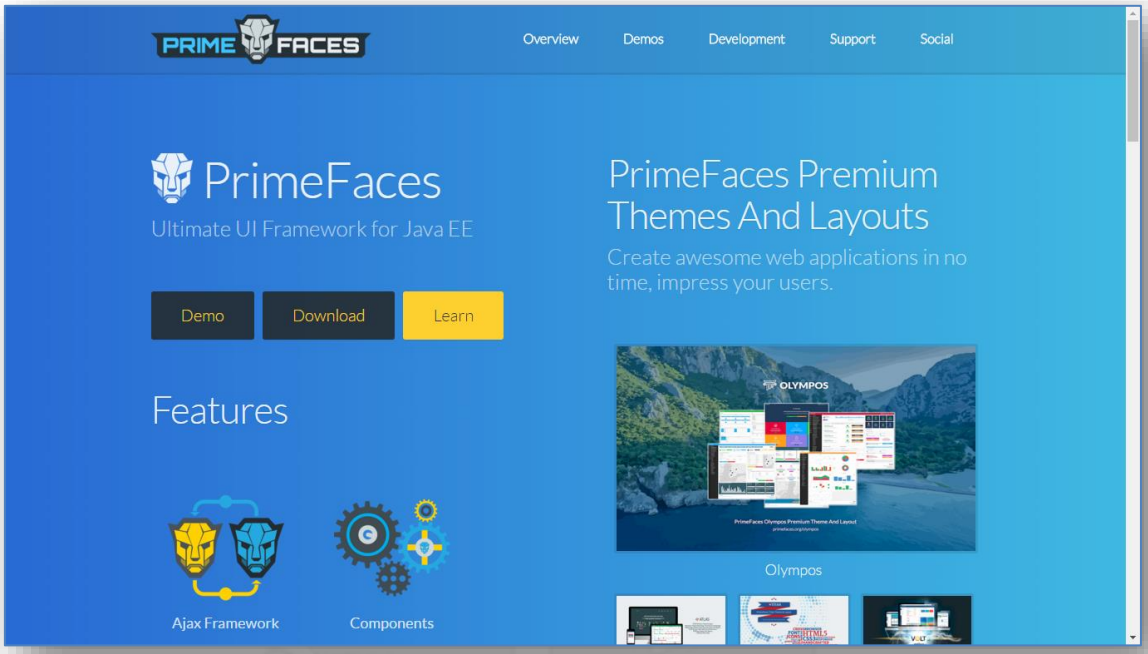
- Suporte a internacionalização e acessibilidade;

- Um conjunto padrão de componentes de interface de usuário que possibilitam validação padronizada;
- Duas bibliotecas de etiqueta ("tag libraries") especiais do JavaServer Pages (JSP) para expressar a interface do JavaServer Faces dentro de uma página JSP;
- Um modelo de eventos do lado servidor ("server-side event model");
- Gerência de estados;
- Managed Beans;
- Linguagem de Expressão Unificada ("Unified Expression Language") para JSP 2.0 e JSF 1.2.

http://www.primefaces.org/showcase/ui/panel/accordionPanel.xhtml



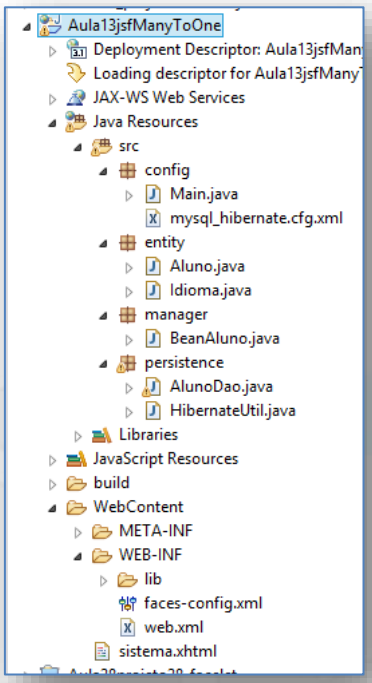
PrimeFaces
http://primefaces.org/



http://www.primefaces.org/showcase/

Onde pode encontrar as opções (menus, panel, buttons)

Estrutura do projeto:



LIBS:

```
all-themes-1.0.10.jar
antlr-2.7.6.jar
asm-attrs.jar
asm.jar
c3p0-0.9.0.jar
cglib-2.1.3.jar
commons-beanutils-1.8.0.jar
commons-codec-1.3.jar
commons-collections-3.2.1.jar
commons-digester-2.0.jar
commons-el-1.0.jar
commons-lang-2.1.jar
commons-logging-1.1.1.jar
commons-validator.jar
dom4j-1.6.1.jar
ejb3-persistence.jar
hibernate-annotations.jar
hibernate-commons-annotations.jar
hibernate-validator-3.0.0.ga.jar
```

```
hibernate3.jar
hsqldb.jar
jakarta-oro.jar
javax.faces-2.0.11.jar
jstl.jar
jta.jar
log4j-1.2.15.jar
mysql-connector-java-5.1.23-bin.jar
oro-2.0.8.jar
primefaces-5.0-sources.jar
primefaces-5.0.jar
standard.jar
```

```
package entity;

import java.util.ArrayList;
import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.FetchType;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToMany;
import javax.persistence.Table;

@Entity
@Table(name = "tblaluno")
public class Aluno {

	@Id
	@GeneratedValue(strategy = GenerationType.AUTO)
	private Integer idAluno;
	@Column(length = 50)
	private String nome;
	@Column(length = 50, unique = true)
	private String email;
	@OneToMany(mappedBy = "aluno", fetch = FetchType.EAGER,
cascade = CascadeType.ALL)
	private List<Idioma> idiomas;
```

```
	public Aluno() {
	}

	public Aluno(Integer idAluno, String nome, String email) {
		this.idAluno = idAluno;
		this.nome = nome;
		this.email = email;
	}

	@Override
	public String toString() {
		return "Aluno [idAluno=" + idAluno + ", nome=" + nome
+ ", email=" + email + "]";
	}

	public Integer getIdAluno() {
		return idAluno;
	}
	public void setIdAluno(Integer idAluno) {
		this.idAluno = idAluno;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public List<Idioma> getIdiomas() {
		return idiomas;
	}
	public void setIdiomas(List<Idioma> idiomas) {
		this.idiomas = idiomas;
	}

	public void adicionar(Idioma idioma) {
		if (idiomas == null) {
			idiomas = new ArrayList<Idioma>();
		}
		idiomas.add(idioma);
```

```
	}
}
```

```
package entity;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.Table;

@Entity
@Table(name="tblidioma")
public class Idioma {

	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	 private Integer idIdioma;

	 @Column(length=50)
	 private String  nomeIdioma;

	@ManyToOne
	@JoinColumn(name="id_aluno")
	private Aluno aluno;

	public Idioma() {

	}

	public Idioma(Integer idIdioma, String nomeIdioma) {
		this.idIdioma = idIdioma;
		this.nomeIdioma = nomeIdioma;
	}

	@Override
	public String toString() {
		return  idIdioma + "," + nomeIdioma ;
	}
	public Integer getIdIdioma() {
```

```
            return idIdioma;
      }
      public void setIdIdioma(Integer idIdioma) {
            this.idIdioma = idIdioma;
      }
      public String getNomeIdioma() {
            return nomeIdioma;
      }
      public void setNomeIdioma(String nomeIdioma) {
            this.nomeIdioma = nomeIdioma;
      }
      public Aluno getAluno() {
            return aluno;
      }
      public void setAluno(Aluno aluno) {
            this.aluno = aluno;
      }
}
```

---

mysql_hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</pro
perty>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</
property>
    <property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/rir?createDatabaseIfNotExist=true</p
roperty>
    <property
name="hibernate.connection.username">root</property>
    <property
name="hibernate.connection.password">coti</property>

        <property name="hibernate.show_sql">true</property>
```

```
        <property name="hibernate.format_sql">true</property>
        <property
name="hibernate.hbm2ddl.auto">update</property>

    <mapping class="entity.Aluno"/>
    <mapping class="entity.Idioma"/>

</session-factory>
</hibernate-configuration>
```

```
package config;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.cfg.Configuration;
import org.hibernate.tool.hbm2ddl.SchemaExport;

public class Main {

     public static void main(String[] args) {
          Configuration cfg =  new AnnotationConfiguration().
      configure("config/mysql_hibernate.cfg.xml");
          new SchemaExport(cfg).create(true, true);
          System.out.println("Gerar as Tabelas no banco de
Dados RIR");

          /*
           *  alter table tblidioma
       drop
       foreign key FKFD35EA13D6C4A3C7

    drop table if exists tblaluno

    drop table if exists tblidioma

    create table tblaluno (
        idAluno integer not null auto_increment,
        email varchar(50) unique,
        nome varchar(50),
        primary key (idAluno)
    )

    create table tblidioma (
        idIdioma integer not null auto_increment,
```

```
        nomeIdioma varchar(50),
        id_aluno integer,
        primary key (idIdioma)
    )

    alter table tblidioma
        add index FKFD35EA13D6C4A3C7 (id_aluno),
        add constraint FKFD35EA13D6C4A3C7
        foreign key (id_aluno)
        references tblaluno (idAluno)
Gerar as Tabelas no banco de Dados RIR

            *
            */

     }
}
```

```
package persistence;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;

public class HibernateUtil {

    private static final SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new AnnotationConfiguration().
configure("config/mysql_hibernate.cfg.xml").buildSessionFactory(
);
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation
failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Criteria;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.Transaction;

import entity.Aluno;
import entity.Idioma;

public class AlunoDao {

	Session session;
	Transaction transaction;
	Query query;
	Criteria criteria;

	   public void create(Aluno a,List<Idioma>lst) throws
Exception{
	session = HibernateUtil.getSessionFactory().openSession();
		transaction = session.beginTransaction();
		    session.save(a); //gravei o Aluno _ 10
		    for (Idioma idi : lst){
		       idi.setAluno(a);  // 10 (busca a primÃ¡ria)
		       session.save(idi);
//2000, ingles, 10 _ 2001, hebraico, 10
		    }
		transaction.commit();
	     session.close();
	   }

	   public void create(Aluno a){
	session = HibernateUtil.getSessionFactory().openSession();
		transaction = session.beginTransaction();
		      session.save(a);
//SÃ³ gravo pelo mais importante o Rel jÃ¡ foir Realizado
		     transaction.commit();
	     session.close();
	}
```

```
        public void update(Aluno a,List<Idioma>lst) throws
Exception{
      session = HibernateUtil.getSessionFactory().openSession();
            transaction = session.beginTransaction();
                session.update(a);
                for (Idioma idi : lst){
                   idi.setAluno(a);
                   session.update(idi);
                }
            transaction.commit();
          session.close();
         }

        public void delete(Aluno a)throws Exception{
      session = HibernateUtil.getSessionFactory().openSession();
            transaction = session.beginTransaction();
                  session.delete(a);
//quando apagar o aluno ele apaga todos (relacionado)
                  transaction.commit();
          session.close();
        }

        public List<Aluno> findAll(){
      session = HibernateUtil.getSessionFactory().openSession();
  List<Aluno> lst = session.createQuery("from Aluno").list();
           session.close();
           return lst;
        }

        public Aluno findByCode(Integer cod){
      session = HibernateUtil.getSessionFactory().openSession();
       Aluno aluno = (Aluno) session.get(Aluno.class, cod);
           session.close();
           return aluno;
        }

        public static void main(String[] args) {
         try {
              Aluno a = new Aluno(null,"ilan","ilan@gmail.com");
              List<Idioma> lst = new ArrayList<Idioma>();
                 Idioma idi1 = new Idioma(null,"ingles");
                    idi1.setAluno(a);
                 Idioma idi2 = new Idioma(null,"espanhol");
                    idi2.setAluno(a);
                 Idioma idi3 = new Idioma(null,"hebraico");
```

```
                    idi3.setAluno(a);
                    lst.add(idi1); lst.add(idi2); lst.add(idi3);
                    a.setIdiomas(lst);
//assim gravo pelo segundo MÃ©todo ...
                    new AlunoDao().create(a);
                    System.out.println("Dados Gravados ");
            }  catch (Exception e) {
            e.printStackTrace();
            }
            }
}
```

No console...

```
log4j:WARN No appenders could be found for logger
(org.hibernate.cfg.annotations.Version).
log4j:WARN Please initialize the log4j system properly.
Hibernate:
    insert
    into
        tblaluno
        (email, nome)
    values
        (?, ?)
Hibernate:
    insert
    into
        tblidioma
        (id_aluno, nomeIdioma)
    values
        (?, ?)
Hibernate:
    insert
    into
        tblidioma
        (id_aluno, nomeIdioma)
    values
        (?, ?)
Hibernate:
    insert
    into
        tblidioma
        (id_aluno, nomeIdioma)
```

```
    values
        (?, ?)
Dados Gravados
```

```
package manager;

import java.util.ArrayList;
import java.util.List;

import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.RequestScoped;
import javax.faces.context.FacesContext;

import entity.Aluno;
import entity.Idioma;
import persistence.AlunoDao;

@ManagedBean(name="mb")
@RequestScoped
public class BeanAluno {

	private Aluno aluno; //para trabalhar com Aluno
	private List<String> idiomaSelecionado;  //Para adicionar
os Idiomar
	private List<Aluno> alunos;

	public BeanAluno() {
	    aluno = new Aluno();
	    idiomaSelecionado = new ArrayList<String>();
	}

	public void cadastrar(){
		FacesContext fc = FacesContext.getCurrentInstance();
		try{
			List<Idioma> lst = new ArrayList<Idioma>();
			for (String idi :   idiomaSelecionado ){
				Idioma idioma = new Idioma(null, idi);
				  idioma.setAluno(aluno); //Muitos
				  lst.add(idioma);
			}
  //UM
			aluno.setIdiomas(lst);
```

```
//aluno busca uma vez a Lista
                        new AlunoDao().create(aluno);
                    aluno = new Aluno();
                    idiomaSelecionado= new ArrayList<String>();
            fc.addMessage(null, new FacesMessage("Dados Gravados
Aluno e Idioma"));
             }catch(Exception ex){
             fc.addMessage(null, new FacesMessage("Error:" +
ex.getMessage()));
             }
      }

      public Aluno getAluno() {
            return aluno;
      }
      public void setAluno(Aluno aluno) {
            this.aluno = aluno;
      }
      public List<String> getIdiomaSelecionado() {
            return idiomaSelecionado;
      }
      public void setIdiomaSelecionado(List<String>
idiomaSelecionado) {
            this.idiomaSelecionado = idiomaSelecionado;
      }
      //Listagem dos alunos e idiomas ...
      public List<Aluno> getAlunos() {
               alunos = new AlunoDao().findAll();
            return alunos;
      }
      public void setAlunos(List<Aluno> alunos) {
            this.alunos = alunos;
      }
}
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
```

```
  <display-name>Aula13jsfManyToOne</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>
</web-app>
```

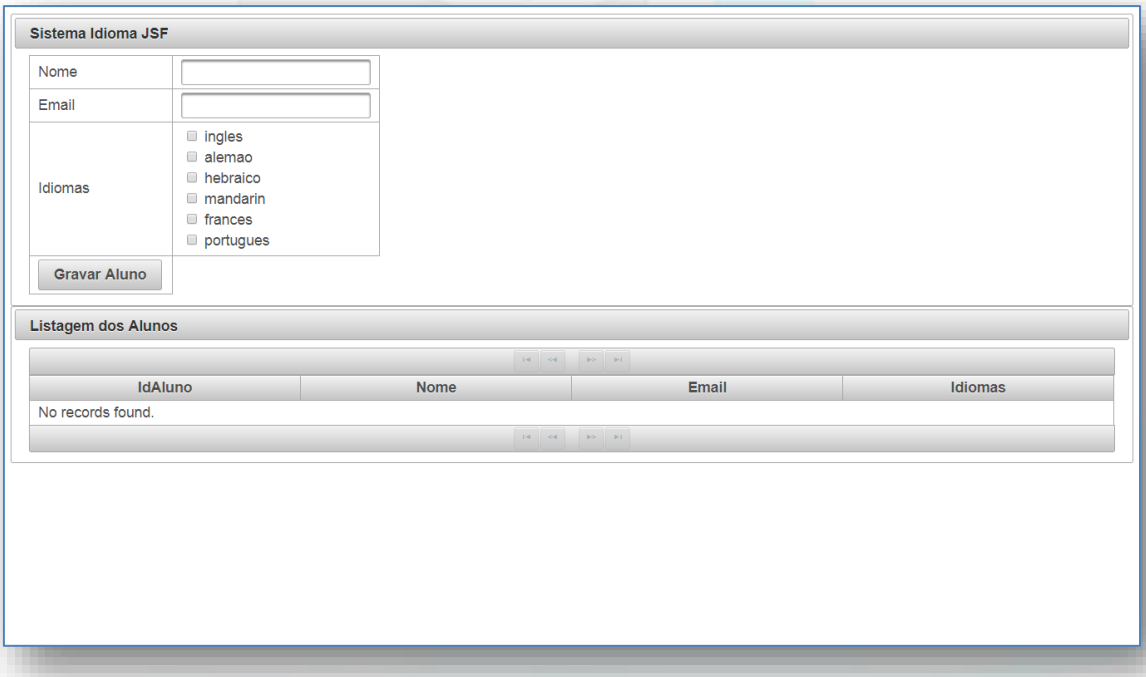
---

sistema.xhtml

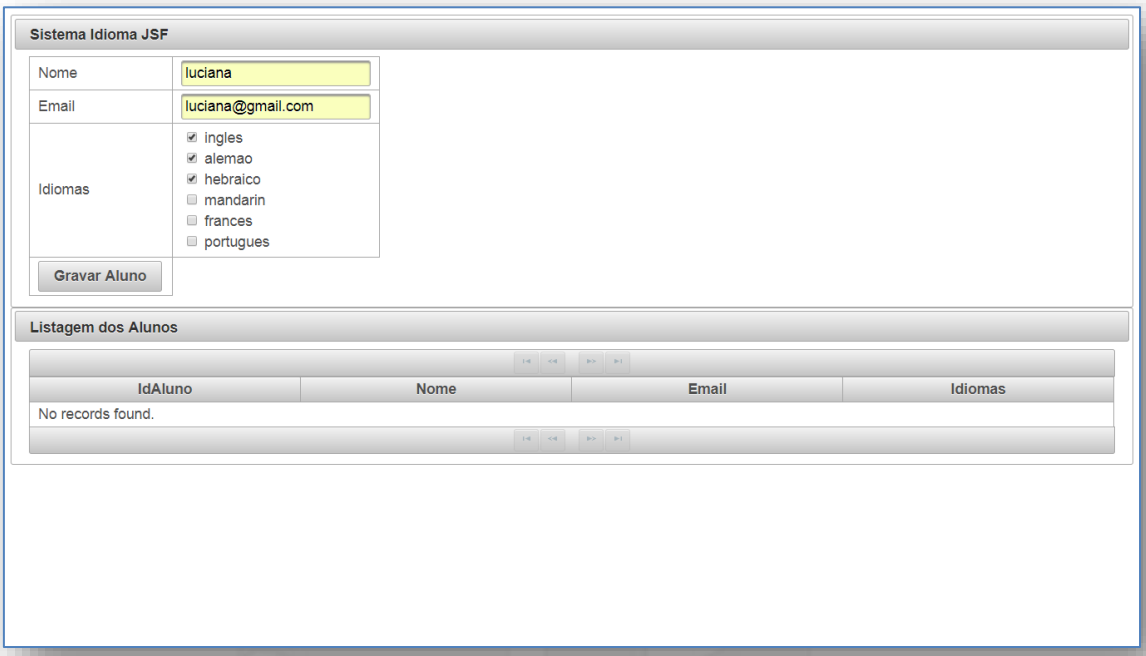
```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head>
</h:head>

<h:body>
  <p:panel header="Sistema Idioma JSF">
    <h:form id="form1">
        <p:panelGrid columns="2">
           <h:outputText value="Nome" />
           <p:inputText    value="#{mb.aluno.nome}"
required="true"></p:inputText>
           <h:outputText value="Email" />
```

```
            <p:inputText     value="#{mb.aluno.email}"
required="true">
                <f:validateRegex pattern="^.+@.+\.[a-z]+$"
></f:validateRegex>
            </p:inputText>

          <h:outputText value="Idiomas"></h:outputText>
          <h:selectManyCheckbox value="#{mb.idiomaSelecionado}"

layout="pageDirection">
              <f:selectItem itemValue="ingles"
itemLabel="ingles"></f:selectItem>
              <f:selectItem itemValue="alemao"
itemLabel="alemao"></f:selectItem>
              <f:selectItem itemValue="hebraico"
itemLabel="hebraico"></f:selectItem>
              <f:selectItem itemValue="mandarin"
itemLabel="mandarin"></f:selectItem>
             <f:selectItem itemValue="frances"
itemLabel="frances"></f:selectItem>
              <f:selectItem itemValue="portugues"
itemLabel="portugues"></f:selectItem>
          </h:selectManyCheckbox>
          <p:commandButton value="Gravar Aluno"
action="#{mb.cadastrar}"
                                      update=":form1, :form2"
                                          ajax="true"
          ></p:commandButton>

        </p:panelGrid>
        <p:growl></p:growl>
     </h:form>
 </p:panel>

 <p:panel header="Listagem dos Alunos">
  <h:form id="form2">
    <p:dataTable value="#{mb.alunos}" var="linha" rows="3"
paginator="true" >
       <p:column headerText="IdAluno">
             <h:outputText value="#{linha.idAluno}" />
       </p:column>
      <p:column headerText="Nome">
           <h:outputText value="#{linha.nome}" />
       </p:column>
      <p:column headerText="Email">
```

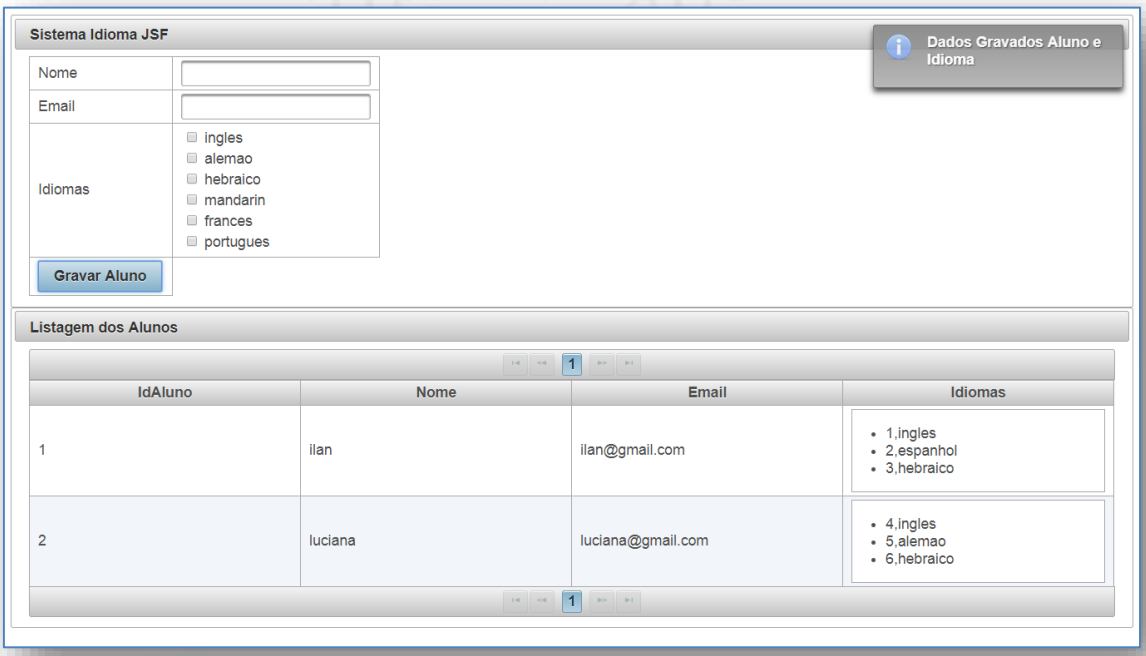
```
            <h:outputText value="#{linha.email}" />
        </p:column>
        <p:column headerText="Idiomas">
            <p:dataList value="#{linha.idiomas}" var="idi">
                #{idi}
            </p:dataList>
        </p:column>
    </p:dataTable>
    </h:form>
 </p:panel>

  <!-- Ficamos ... -->
</h:body>
</html>
```



Cadastrando um aluno...



Gravado...



Meu banco de dados na vida real, fica no integrator.com.br

WebService Rest JSON_ Velocidade (SaÃ-da)
-->Problema seguranÃ§a ... (ficar atento) (seguranÃ§a)
--> Quem Resgate ...

. Angular
.Java (JSF / spring)
. Android
. Qualquer linguagem de programaÃ§Ã£o do Mundo (php, c#)

Serv nao pode pode sair do on-Line ... (24 hr)

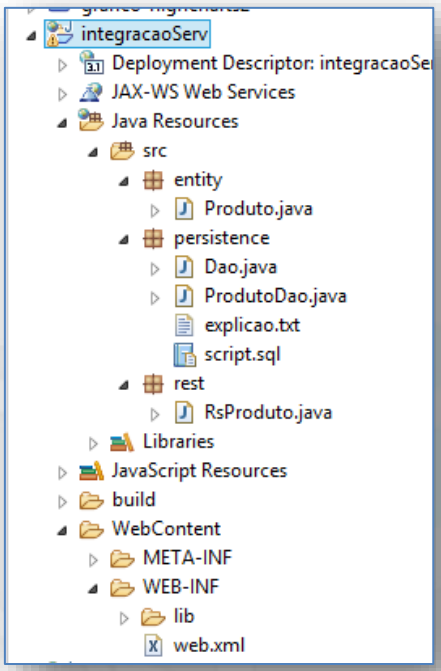
Rotas
---

rotas do gravar
---
http://localhost:8080/integracaoServ/json/produto/gravar/cdpiaf/10/1

rotas do listar
---
http://localhost:8080/integracaoServ/json/produto/listar

PROJETO SERVER NO ECLIPSE

Estrutura do projeto:



LIBS:

```
asm-3.1.jar
commons-codec-1.6.jar
commons-logging-1.1.1.jar
fluent-hc-4.2.5.jar
gson-2.2.4.jar
httpclient-4.2.5.jar
httpclient-cache-4.2.5.jar
httpcore-4.2.4.jar
httpmime-4.2.5.jar
jackson-core-asl-1.9.2.jar
jackson-jaxrs-1.9.2.jar
jackson-mapper-asl-1.9.2.jar
jackson-xc-1.9.2.jar
jaxen-1.1.6.jar
jdom-2.0.5.jar
jersey-client-1.12.jar
jersey-core-1.12.jar
jersey-json-1.12.jar
```

```
jersey-server-1.12.jar
jersey-servlet-1.12.jar
jettison-1.1.jar
jsr311-api-1.1.1.jar
mysql-connector-java-5.1.23-bin.jar
xercesImpl.jar
xml-apis.jar
```

```
package entity;

import java.io.Serializable;

import javax.xml.bind.annotation.XmlRootElement;

@XmlRootElement
public class Produto implements Serializable {

	private static final long serialVersionUID = 1L;

	private Integer idProduto;
	 private String nome;
	 private Double preco;
	 private Integer quantidade;

	 public Produto() {
	}

	public Produto(Integer idProduto, String nome, Double preco, Integer quantidade) {
		super();
		this.idProduto = idProduto;
		this.nome = nome;
		this.preco = preco;
		this.quantidade = quantidade;
	}

	@Override
	public String toString() {
		return "Produto [idProduto=" + idProduto + ", nome=" + nome + ", preco=" + preco + ", quantidade=" + quantidade + "]";
	}
```

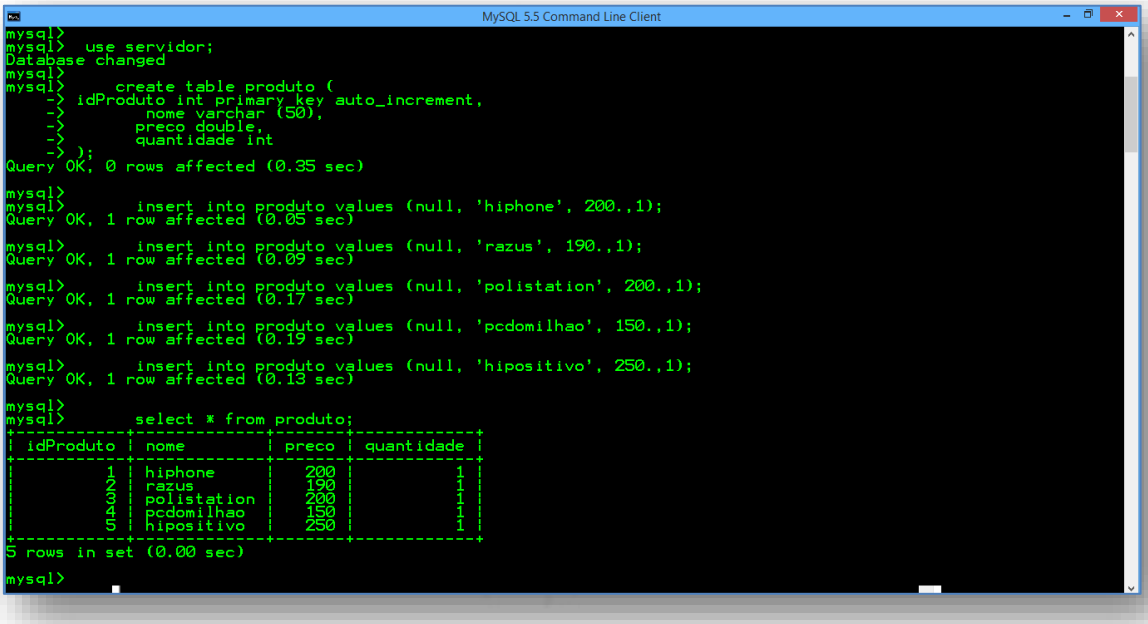
```
	public Integer getIdProduto() {
		return idProduto;
	}
	public void setIdProduto(Integer idProduto) {
		this.idProduto = idProduto;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public Double getPreco() {
		return preco;
	}
	public void setPreco(Double preco) {
		this.preco = preco;
	}
	public Integer getQuantidade() {
		return quantidade;
	}
	public void setQuantidade(Integer quantidade) {
		this.quantidade = quantidade;
	}
}
```

---

script.sql

```
drop database if exists servidor;

create database servidor;

 use servidor;

    create table produto (
      idProduto int primary key auto_increment,
        nome varchar (50),
       preco double,
       quantidade int
);
```

```
insert into produto values (null, 'hiphone', 200.,1);
insert into produto values (null, 'razus', 190.,1);
insert into produto values (null, 'polistation', 200.,1);
insert into produto values (null, 'pcdomilhao', 150.,1);
insert into produto values (null, 'hipositivo', 250.,1);

select * from produto;
```



Escola de Nerds
www.cotiinformatica.com.br

```
package persistence;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Dao {

 Connection con;  //banco
 PreparedStatement stmt; //tabela
 ResultSet rs; //consulta

      public void open() throws Exception{
        Class.forName("com.mysql.jdbc.Driver");
```

```
        con = DriverManager.getConnection
("jdbc:mysql://localhost:3306/servidor","root","coti");
        }

        public void close() throws Exception{
         con.close();
        }
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;

import entity.Produto;

public class ProdutoDao  extends Dao{

        public void create(Produto p) throws Exception{
              open();
                stmt = con.prepareStatement("insert into produto
values (null, ?,?,?)  ");
                  stmt.setString(1, p.getNome());
                  stmt.setDouble(2, p.getPreco());
                  stmt.setInt(3, p.getQuantidade());
                    stmt.execute();
                    stmt.close();
              close();
        }

        public List<Produto>  findAll() throws Exception{
        open();
          stmt = con.prepareStatement("select * from produto");
                rs = stmt.executeQuery();
                List<Produto> lst = new ArrayList<Produto>();
                while(rs.next()){
                  Produto p = new Produto();
                                p.setIdProduto(rs.getInt(1));
                                p.setNome(rs.getString(2));
                            p.setPreco(rs.getDouble(3));
                            p.setQuantidade(rs.getInt(4));
                            lst.add(p);
                }
        close();
```

```
        return lst;
        }
}
```

---

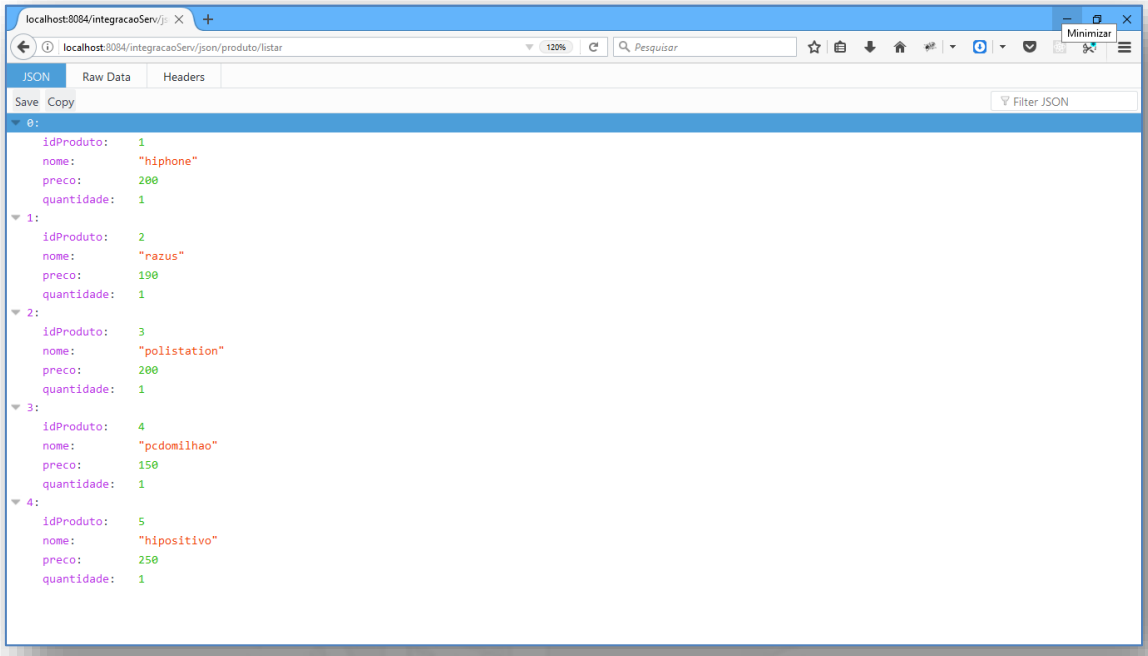
web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app id="WebApp_ID" version="3.1"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd">

<!-- comentario mudar o nome do param-value rest -->

 <servlet>
      <servlet-name>Jersey REST Service</servlet-name>
      <servlet-
class>com.sun.jersey.spi.container.servlet.ServletContainer</ser
vlet-class>
      <init-param>
            <param-
name>com.sun.jersey.config.property.packages</param-name>
            <param-value>rest</param-value>
      </init-param>
      <load-on-startup>1</load-on-startup>
  </servlet>

  <servlet-mapping>
      <servlet-name>Jersey REST Service</servlet-name>
      <url-pattern>/json/*</url-pattern>
  </servlet-mapping>

</web-app>
```

```
package rest;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
```

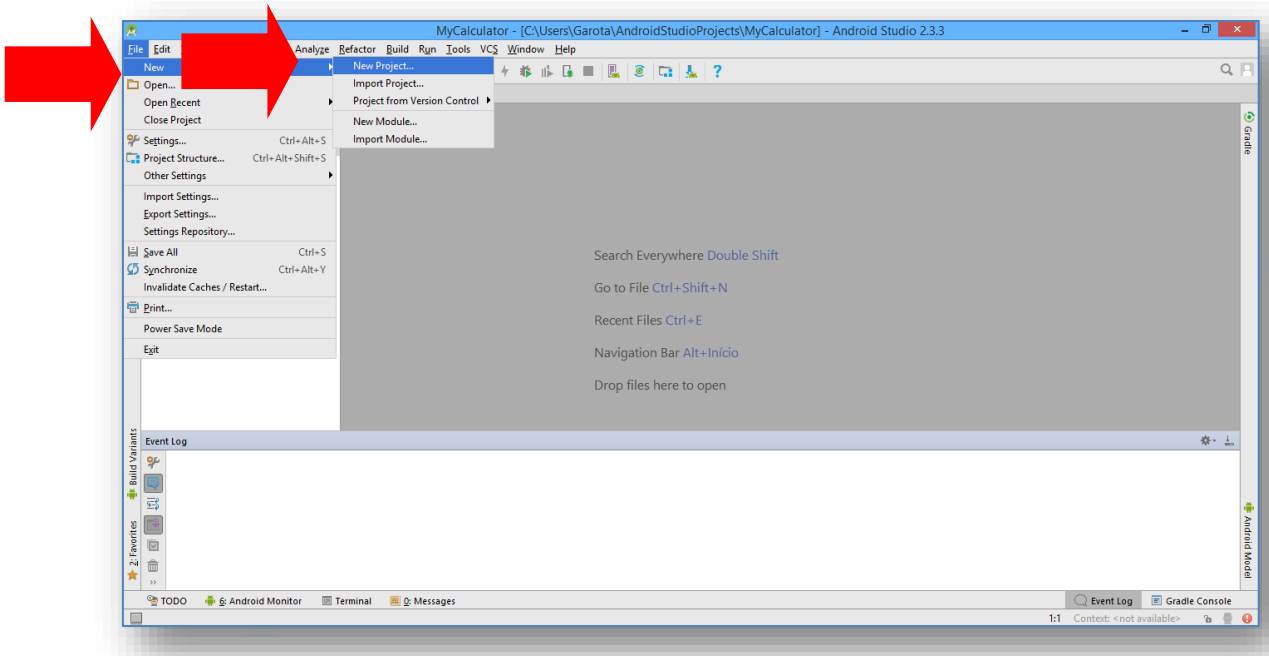
```
import com.google.gson.Gson;

import entity.Produto;
import persistence.ProdutoDao;

@Path("/produto")
public class RsProduto {

	@GET
	@Path("/gravar/{nome}/{preco}/{quantidade}")
	@Produces("text/plain")
	public String gravar(@PathParam("nome") String nome,
			@PathParam("preco") String preco,
			@PathParam("quantidade") String quantidade
				){
		try{
		    ProdutoDao dao = new ProdutoDao();
		      dao.create(new Produto(null, nome, new
Double(preco), new Integer(quantidade)));
		      return " Dados Gravados ... ";
		}catch(Exception ex){
		  return "Error :" + ex.getMessage();
		}
	}

	@GET
	@Path("/listar")
	@Produces("application/json")
	public String listar( ){
		try{
		    ProdutoDao dao = new ProdutoDao();
		        return new Gson().toJson( dao.findAll() );
		}catch(Exception ex){
		     return "Error :" + ex.getMessage();
		}
	}
}
```

Rodar o projeto clicar no projeto com o botão direito...
Digitar a URL da rota...
http://localhost:8084/integracaoServ/json/produto/listar

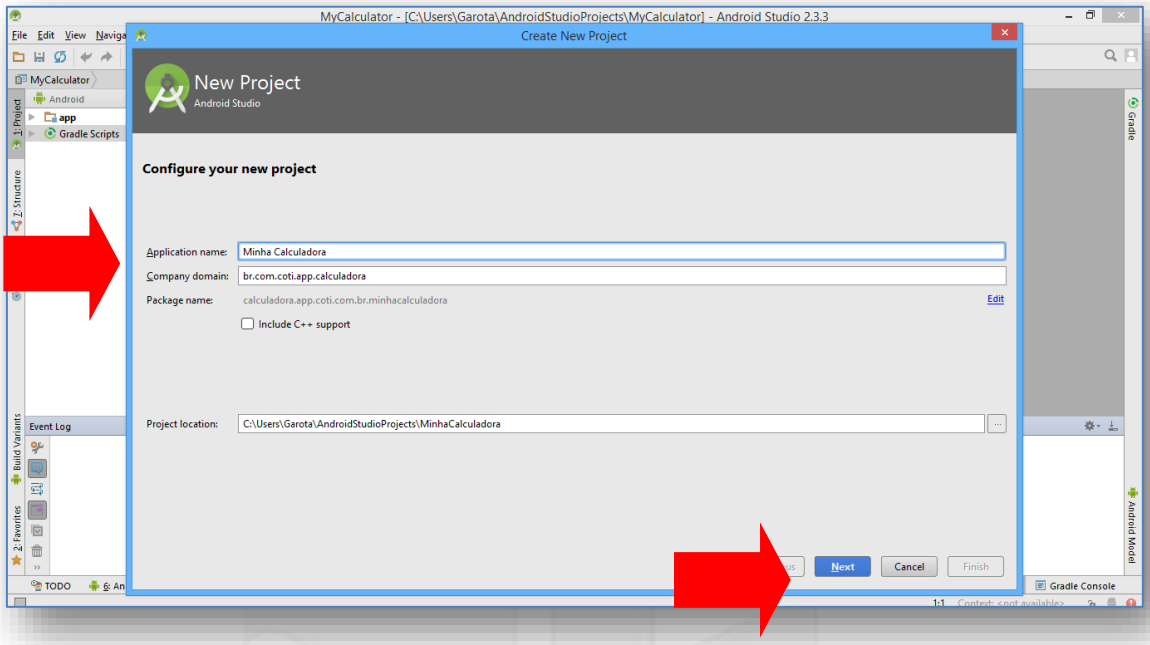


Para criar um projeto no Android Studio...

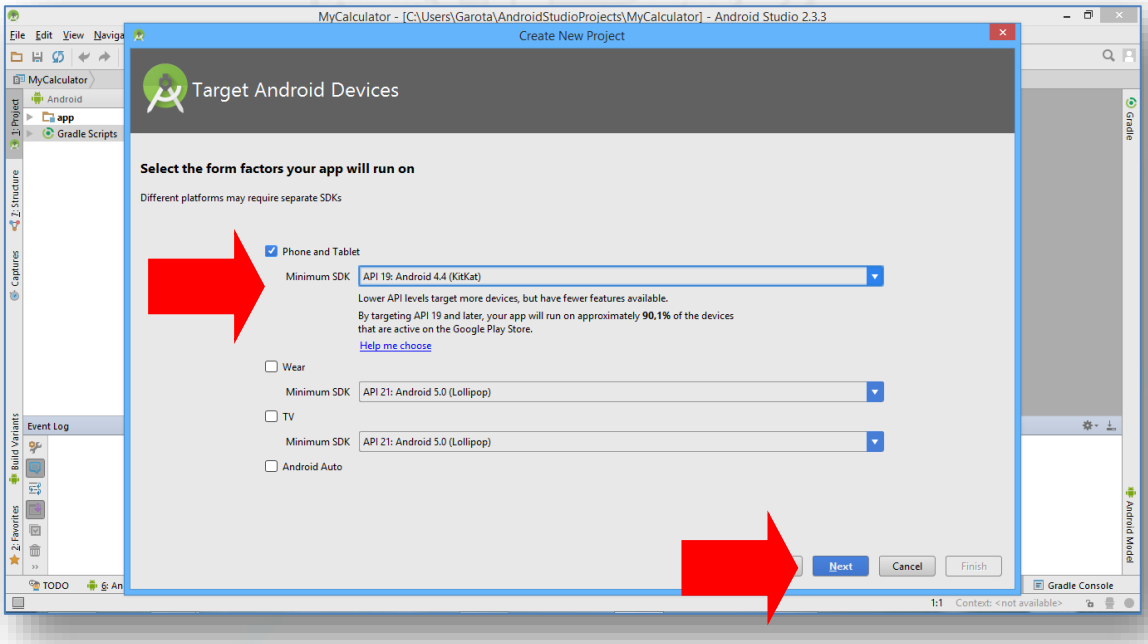
File -> new -> New Project...



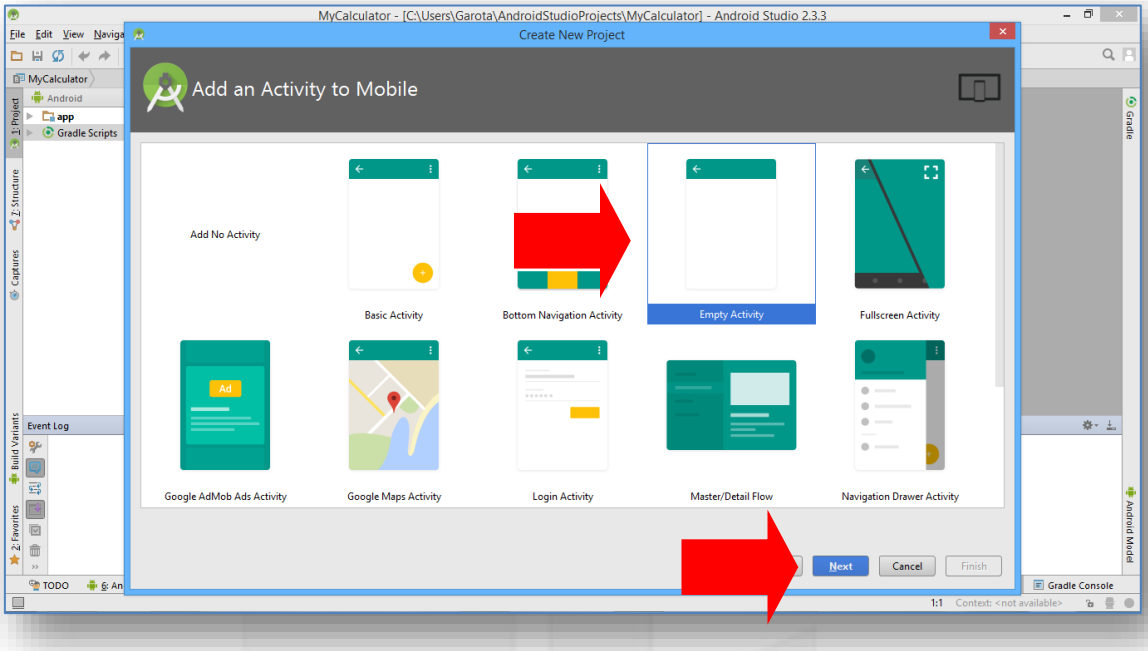
Digitar um nome para o projeto em aplication name -> o endereço de domínio -> next...



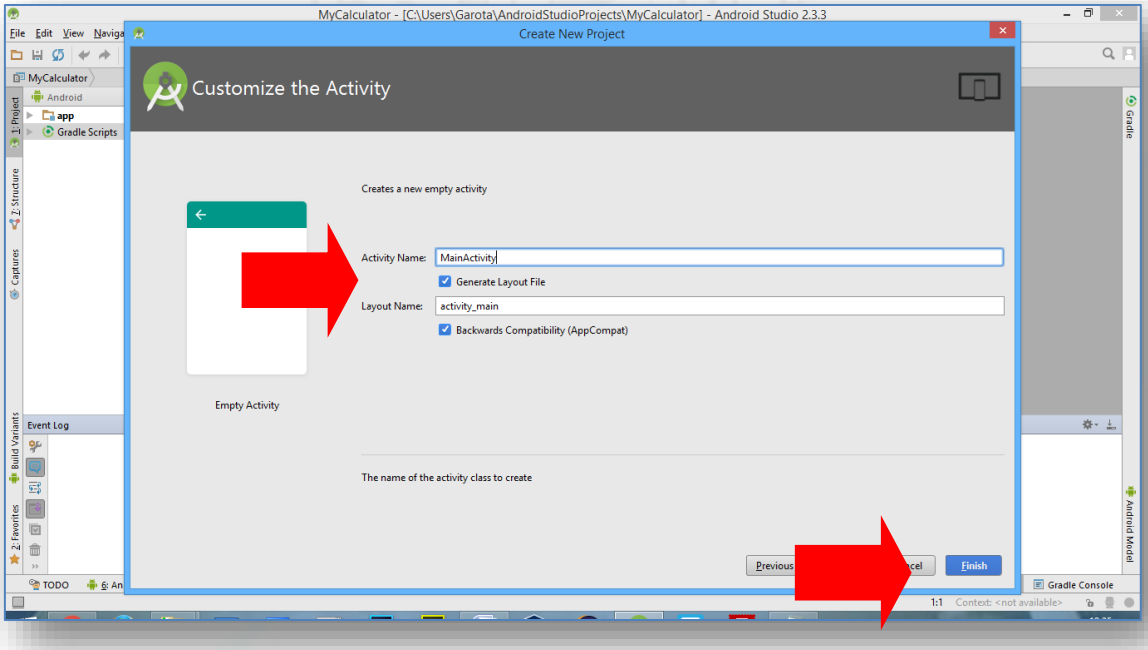
Selecionar o SDK -> Kitcat -> next...



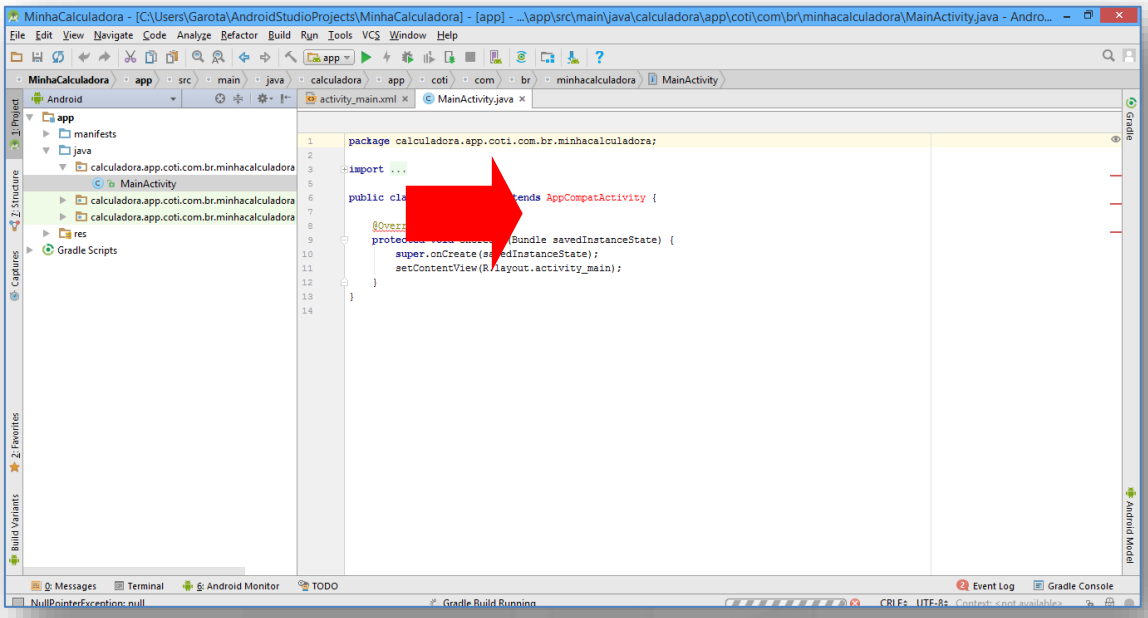
Escolher o layput -> Empty Activity -> next...



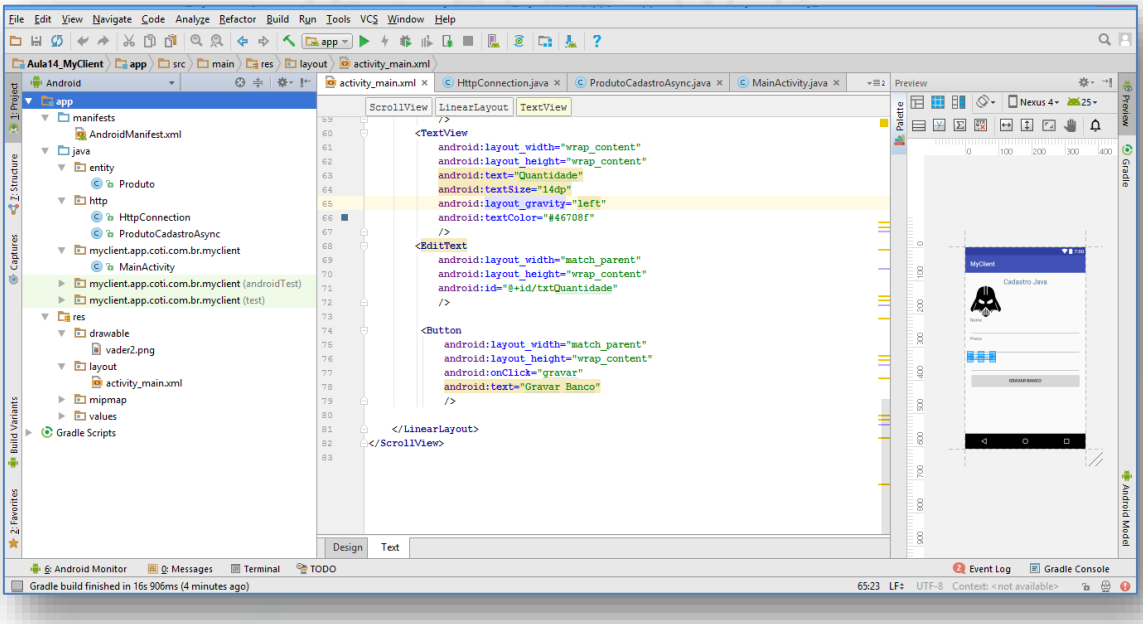
Manter os nome padrões das classes -> Finish....



Esperar montar o projeto todo -> Repare que as classes ainda estão dando erro pois não está totalmente montado...



Estrutura do projeto:



Para programar a tela vá em:

Activity_main.xml

Salvar uma imagem e colar em “drawable”...

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context="myclient.app.coti.com.br.myclient.MainActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        >

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Cadastro Java"
        android:textSize="22dp"
        android:layout_gravity="center_horizontal"
        android:textColor="#46708f"
        />

     <ImageView
         android:layout_width="100sp"
         android:layout_height="100sp"
         android:src="@drawable/vader2"
         android:layout_gravity="left"
         />

    <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Nome"
            android:textSize="14dp"
            android:layout_gravity="left"
            android:textColor="#46708f"
            />
      <EditText
          android:layout_width="match_parent"
          android:layout_height="wrap_content"
          android:id="@+id/txtNome"
          />
```

```
    <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Preco"
            android:textSize="14dp"
            android:layout_gravity="left"
            android:textColor="#46708f"
            />
        <EditText
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:id="@+id/txtPreco"
            />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Quantidade"
            android:textSize="14dp"
            android:layout_gravity="left"
            android:textColor="#46708f"
            />
        <EditText
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:id="@+id/txtQuantidade"
            />

         <Button
             android:layout_width="match_parent"
             android:layout_height="wrap_content"
             android:onClick="gravar"
             android:text="Gravar Banco"
             />

    </LinearLayout>
</ScrollView>
```

Alterar o AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="myclient.app.coti.com.br.myclient">
```

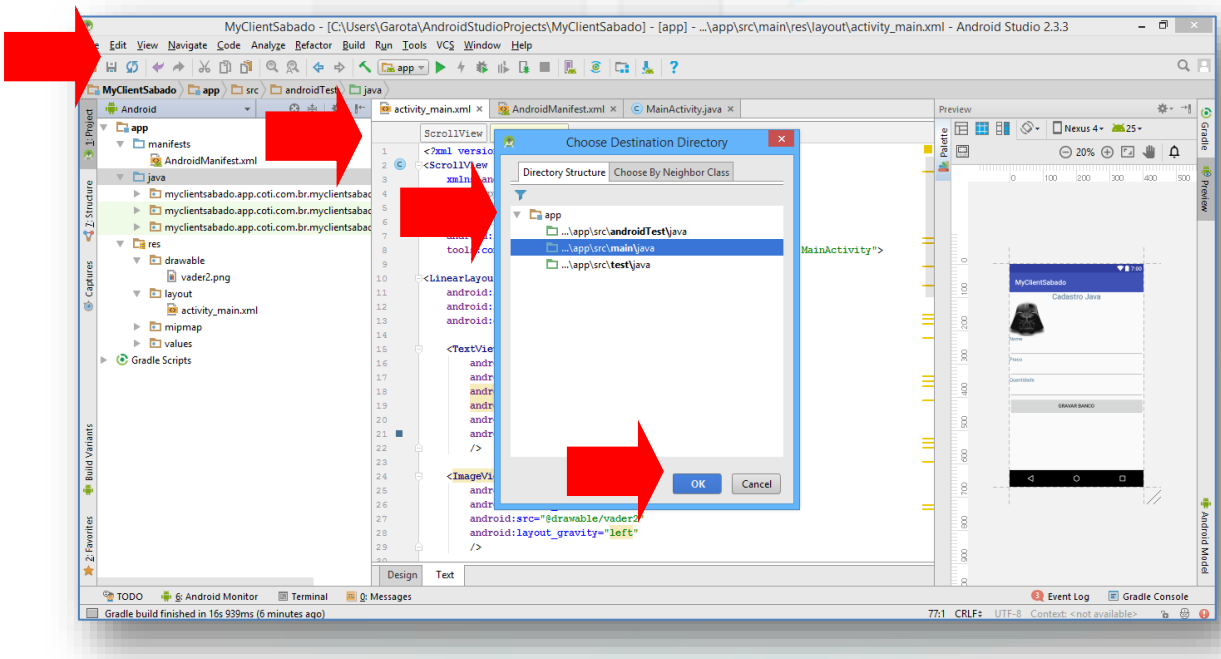
Dando permissão para o acesso a internet...

```
  <uses-permission android:name="android.permission.INTERNET"></uses-
permission>

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
```

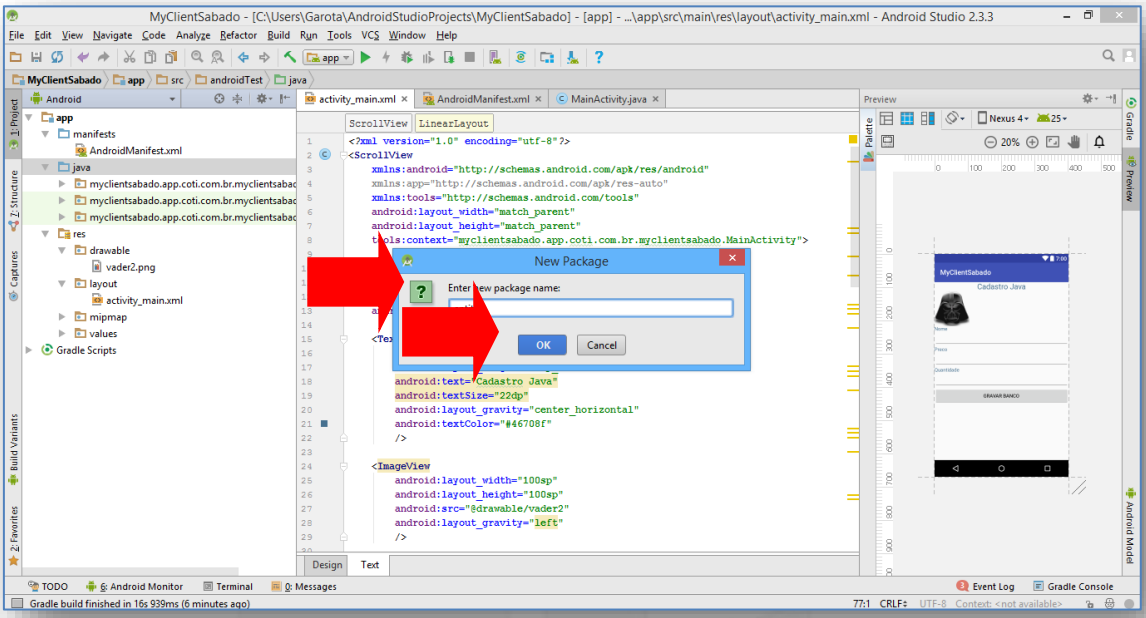
```
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Criar o pacote entity em Java...

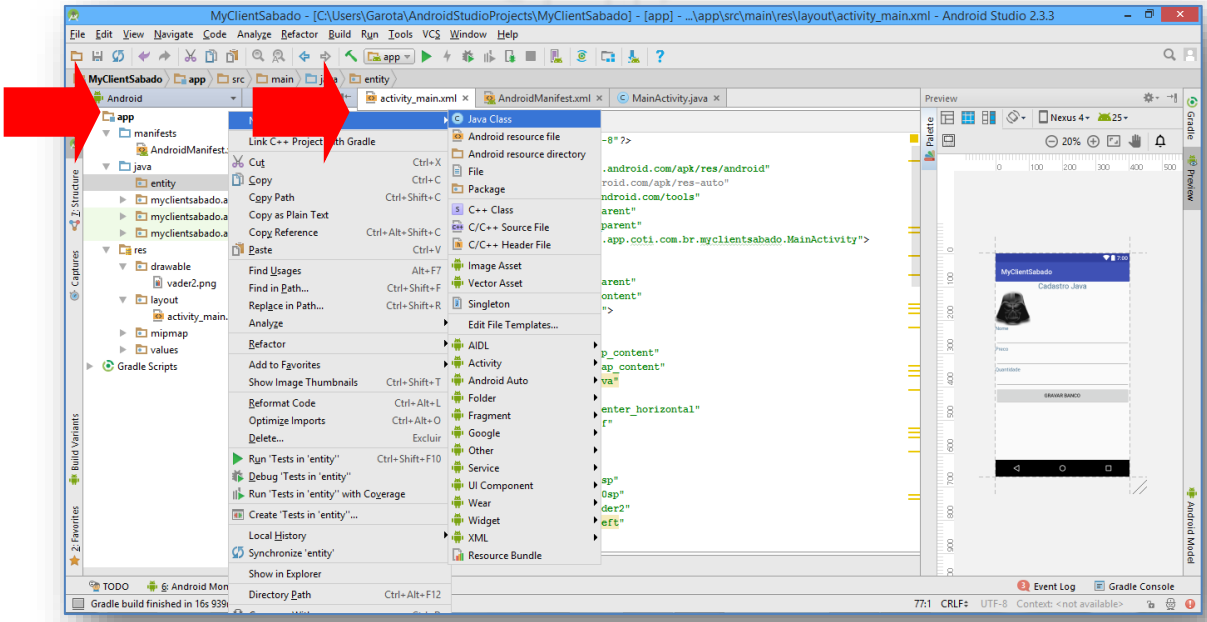
Clicar na opção main...



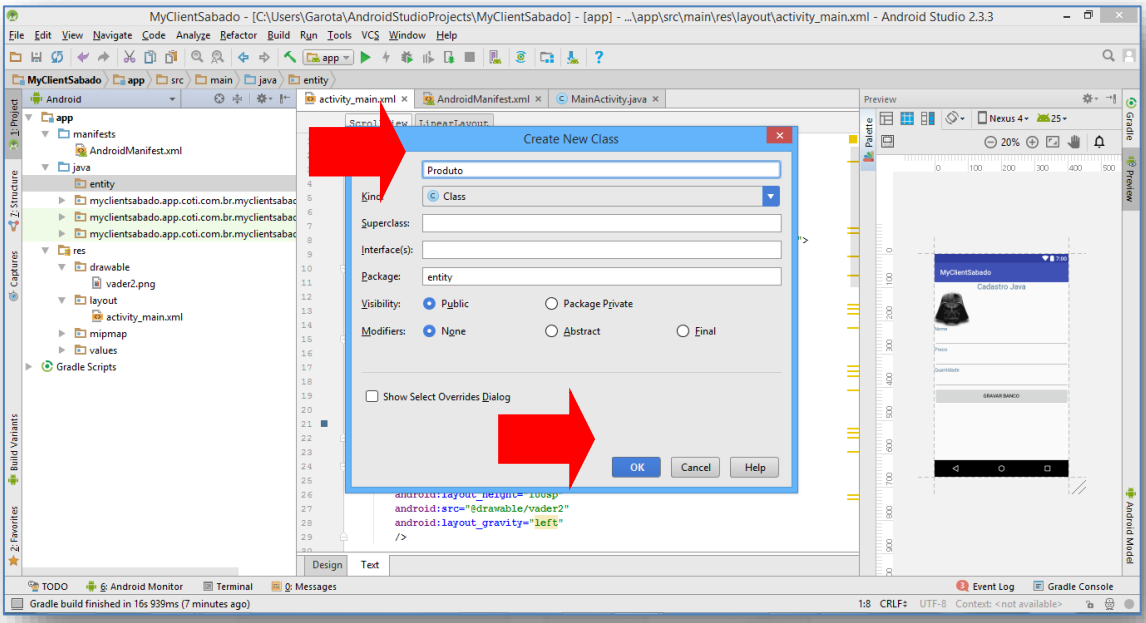
Digitar o nome do pacote....



Para criar a classe...
Clicar no pacote entity e criar a classe...



Digitar o nome da classe...

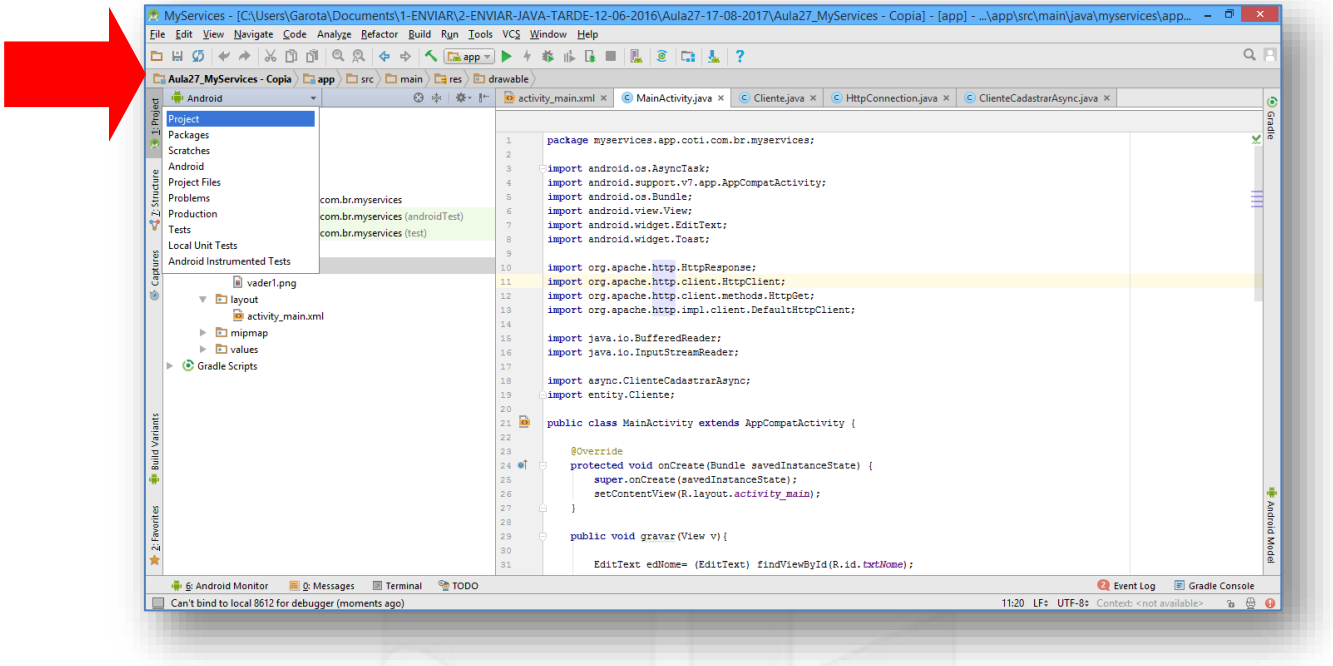



```
package entity;

public class Produto {

    private Integer idProduto;
    private String nome;
    private Double preco;
    private Integer quantidade;

    public Produto() {
    }


    public Produto(Integer idProduto, String nome, Double preco,
Integer quantidade) {
        this.idProduto = idProduto;
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
    }

    @Override
    public String toString() {
        return "Produto{" +
                "idProduto=" + idProduto +
                ", nome='" + nome + '\'' +
                ", preco=" + preco +
```

```
                ", quantidade=" + quantidade +
                '}';
    }

    public Integer getIdProduto() {
        return idProduto;
    }

    public void setIdProduto(Integer idProduto) {
        this.idProduto = idProduto;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public Double getPreco() {
        return preco;
    }

    public void setPreco(Double preco) {
        this.preco = preco;
    }

    public Integer getQuantidade() {
        return quantidade;
    }

    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
    }
}
```

www.cotiinformatica.com.br

Para colocar as libs, mudar a perspectiva para Project...

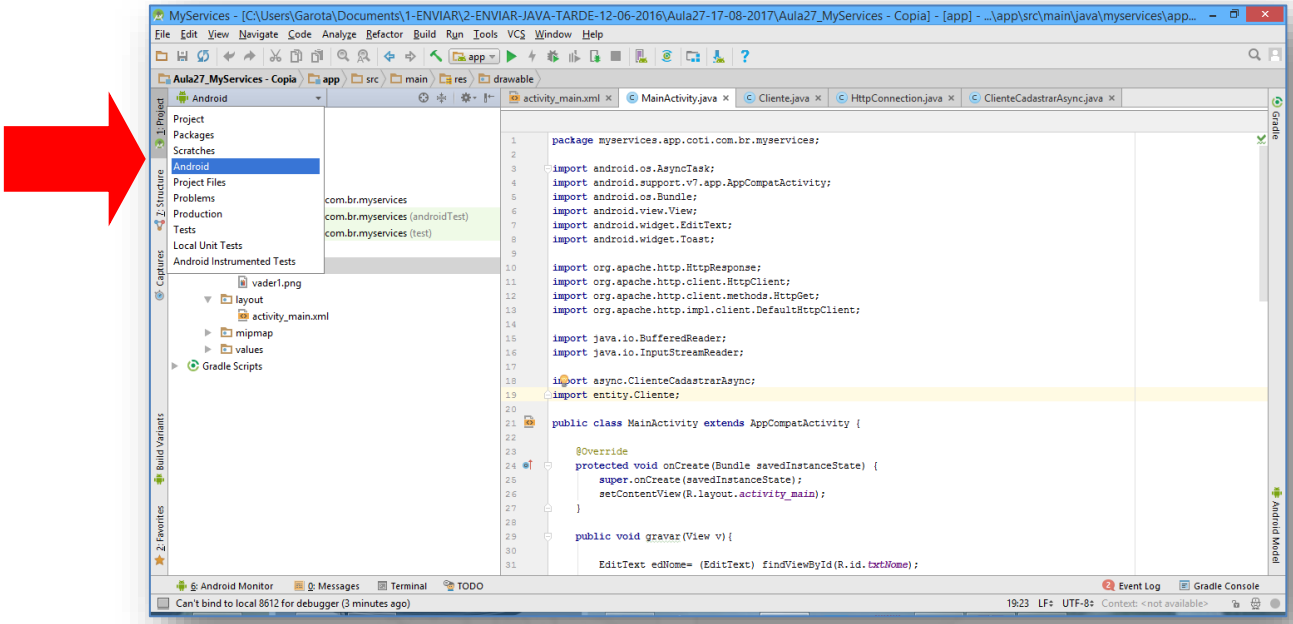


Abrir a pasta libs e colar as 4 libs...

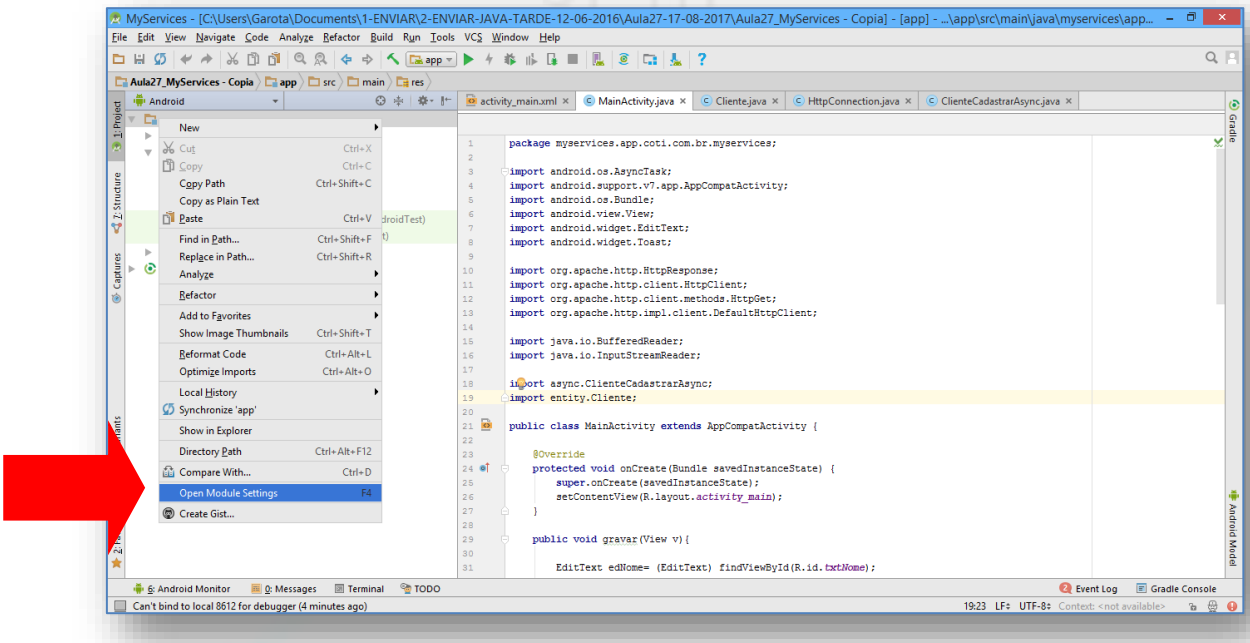
httpclient-4.2.5.jar
httpclient-cache-4.2.5.jar
httpcore-4.2.4.jar
httpmime-4.2.5.jar

Para incorporar as libs ao projeto...

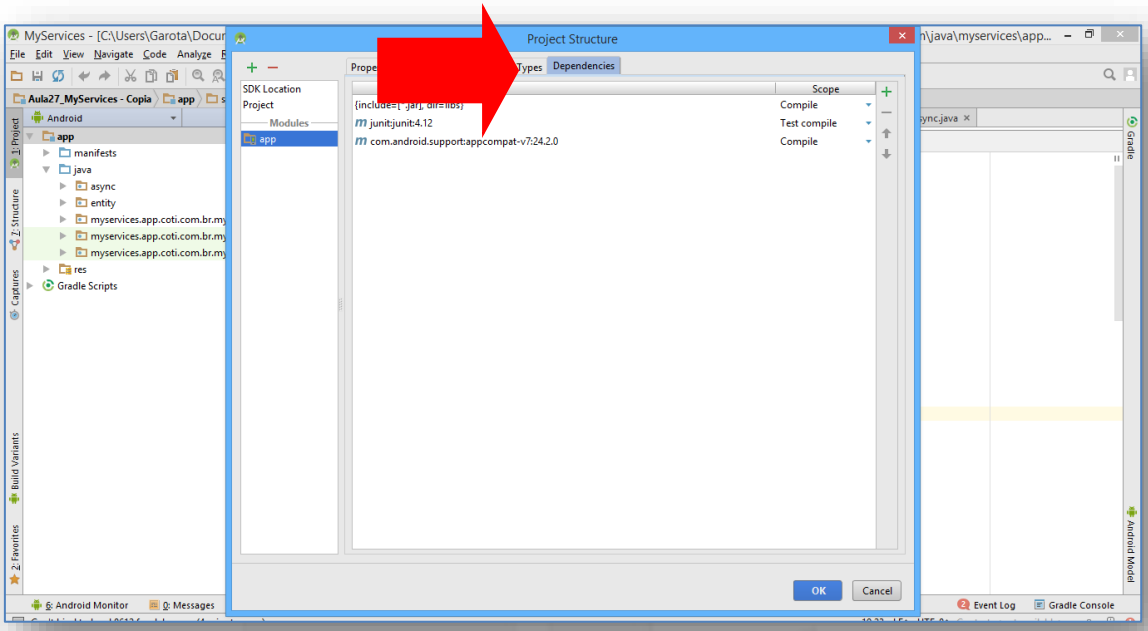
Voltar para a perspectiva android



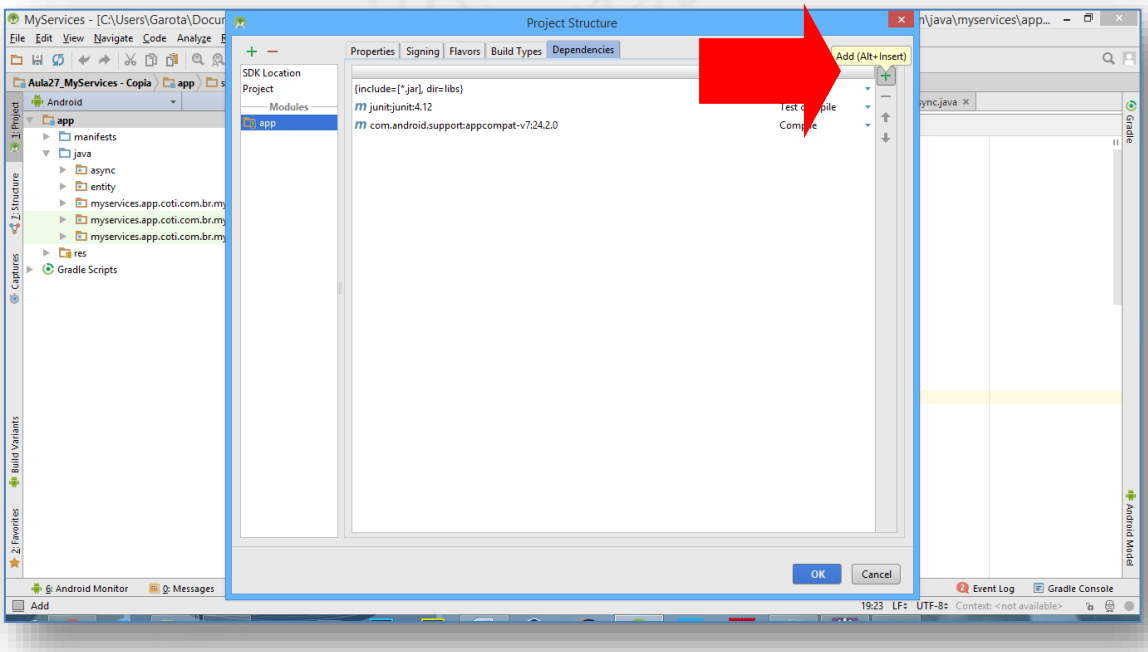
Clicar em Open Module Settings...



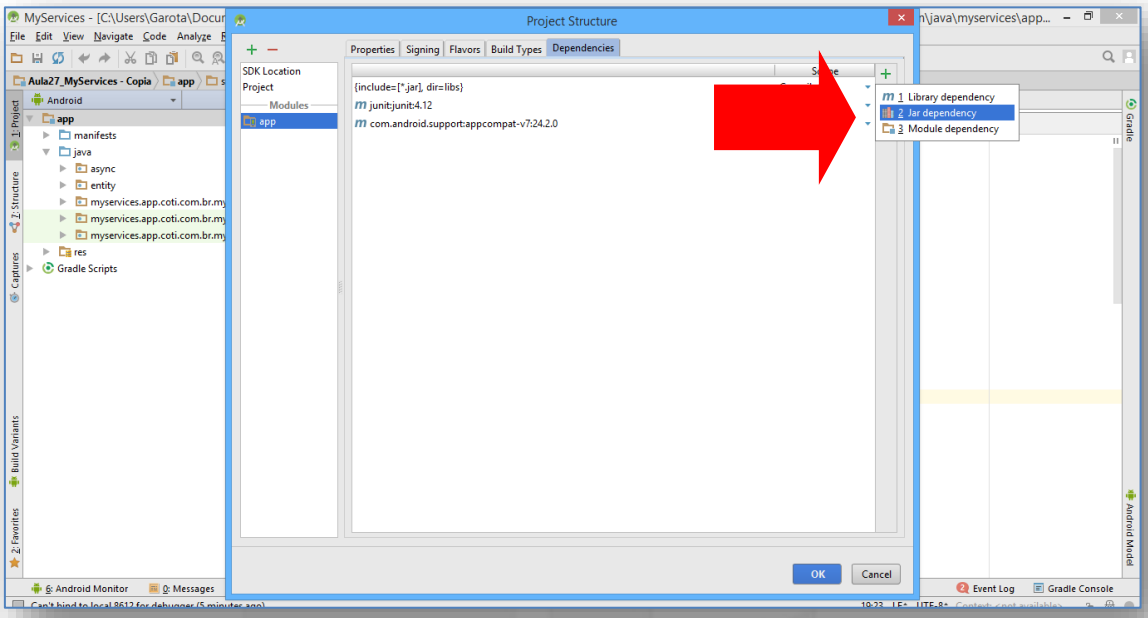
Clicar na aba "dependencies"...



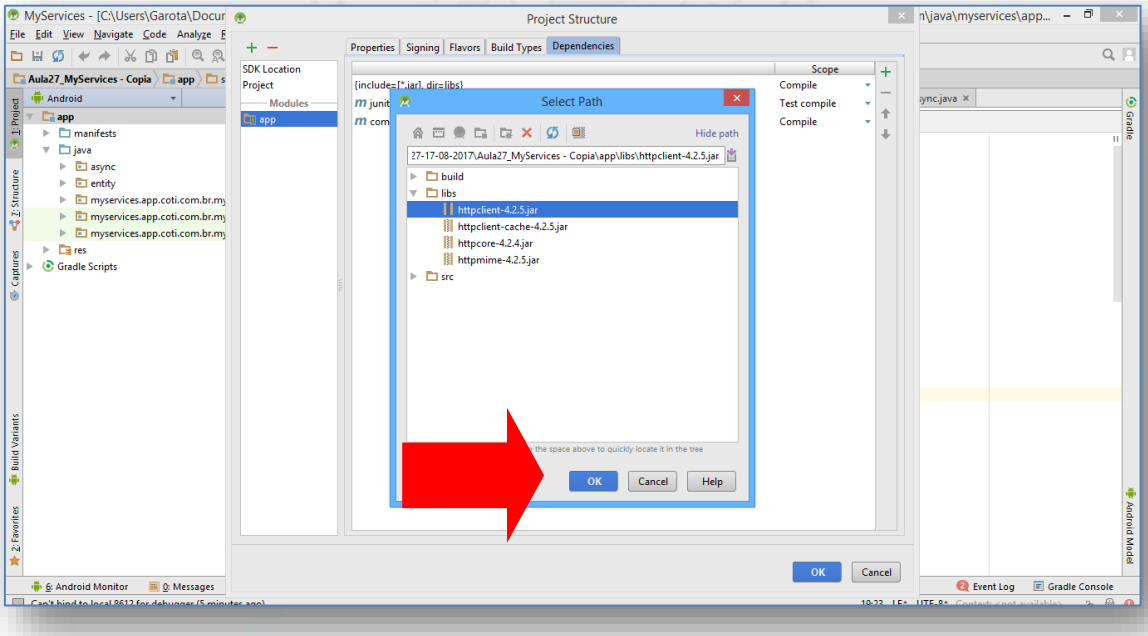
Adicionar uma a uma as libs. Para adicionar clicar no sinal de "+"...



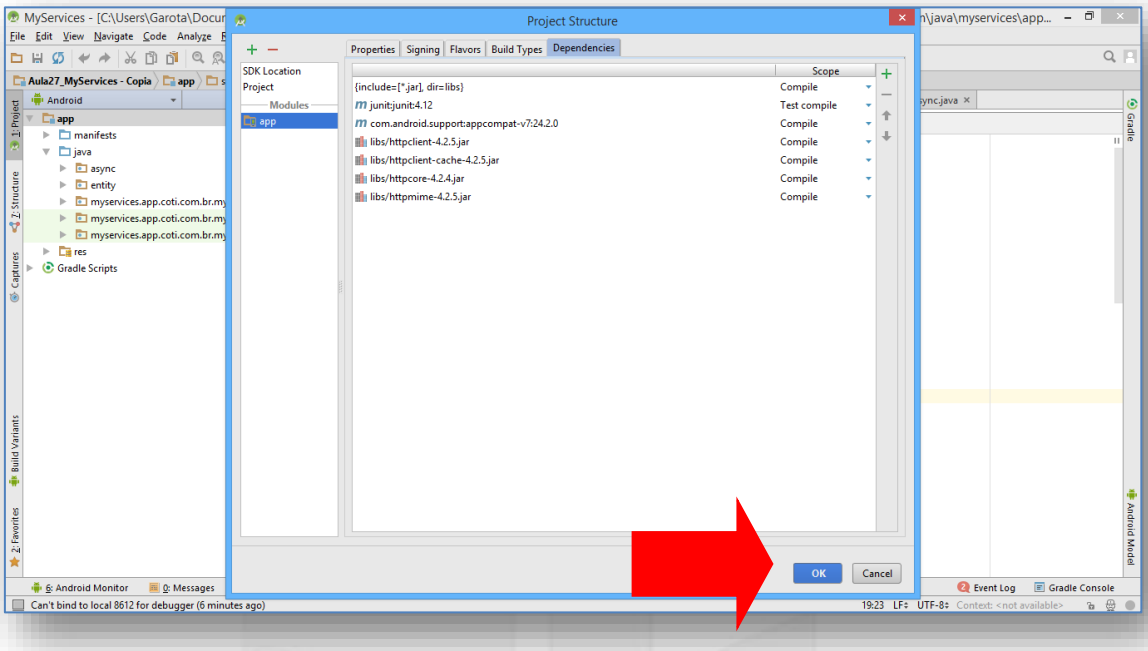
Clicar em jar dependencies...



Clicar na lib -> ok...



Fazer o mesmo para as outras -> ok...



Criar o pacote "http"...
Criar a classe "HttpConnection"

```
package http;


import org.apache.http.HttpResponse;
import org.apache.http.client.HttpClient;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.impl.client.DefaultHttpClient;

import java.io.BufferedReader;
import java.io.InputStreamReader;

public class HttpConnection {


    public static String httpGet(String url){
        try{
            HttpClient client = new DefaultHttpClient();
            HttpGet httpGet = new HttpGet(url);
            HttpResponse resp = client.execute(httpGet);
            BufferedReader bf = new BufferedReader(
     new InputStreamReader( resp.getEntity().getContent() )
            );
```

```
            return bf.readLine();
        }catch(Exception ex){
            return ex.getMessage();
        }
    }


}
```

```
package http;


import android.content.Context;
import android.os.AsyncTask;
import android.widget.Toast;

import entity.Produto;

public class ProdutoCadastroAsync
extends AsyncTask<String,Void,String> {

    private Context context;
    private Produto produto;


    public ProdutoCadastroAsync(Context context, Produto produto) {
        this.context = context;
        this.produto = produto;
    }

    @Override
    protected void onPostExecute(String resposta){
    Toast.makeText(context, resposta, Toast.LENGTH_LONG).show();
    }


    @Override
    protected String doInBackground(String... params) {
        try{
```

Atenção para o endereço do WebService...

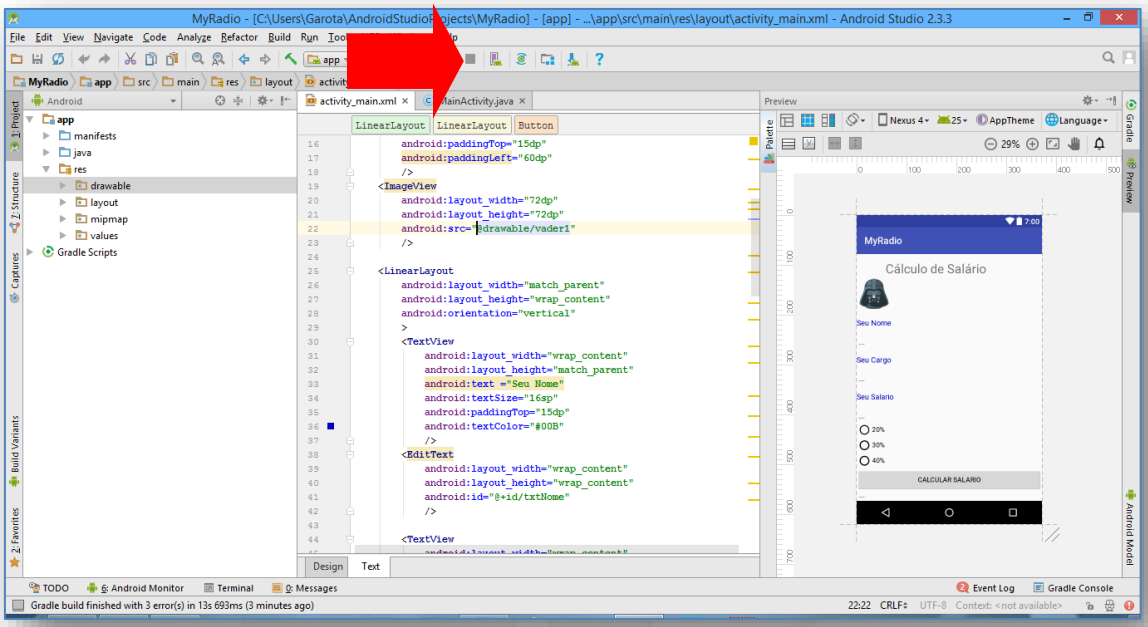
```
          String resposta =
HttpConnection.httpGet("http://192.168.0.132:8080/integracaoServ/json/
produto/gravar/"
    +produto.getNome() + "/" +  produto.getPreco() + "/" +
produto.getQuantidade()
                          );

         return resposta;
        }catch(Exception ex){
              return ex.getMessage();
        }
```

```
    }
}
```

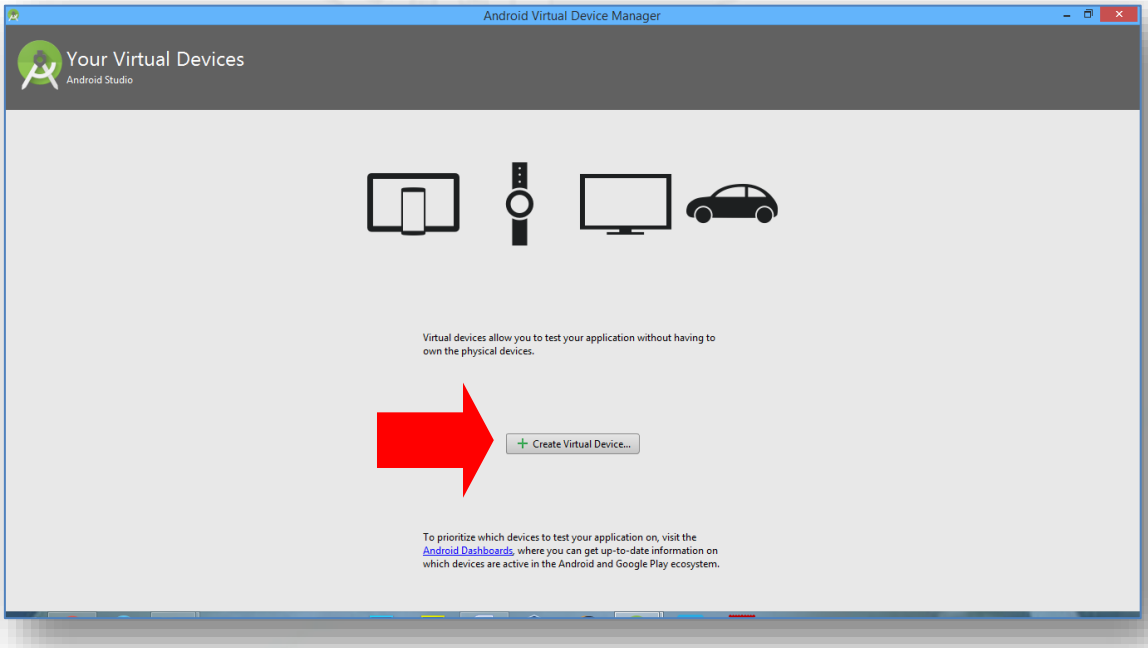
No MainActivity...

```
package myclient.app.coti.com.br.myclient;

import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;

import entity.Produto;
import http.ProdutoCadastroAsync;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }


    public void gravar(View v){
        EditText txtNome = (EditText) findViewById(R.id.txtNome);
        EditText txtPreco = (EditText) findViewById(R.id.txtPreco);
        EditText txtQuantidade = (EditText)
findViewById(R.id.txtQuantidade);

        Produto p = new Produto();
              p.setNome(txtNome.getText().toString());
              p.setPreco(new Double(txtPreco.getText().toString()));
              p.setQuantidade(new
Integer(txtQuantidade.getText().toString()));

        new ProdutoCadastroAsync(getApplicationContext() ,   p
).execute();
         p.setNome("");
         p.setPreco(0.);
         p.setQuantidade(0);
        //limpar a tela
        txtNome.setText("");
        txtPreco.setText("");
        txtQuantidade.setText("");
    }
}
```

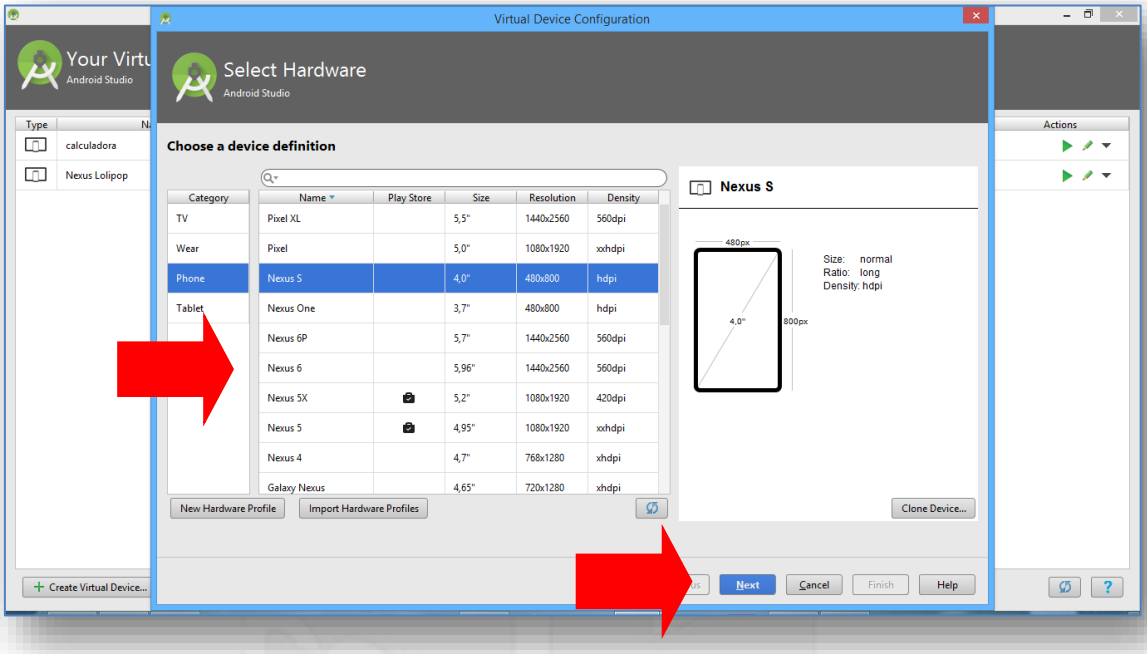
Clicar em ADV Manager para escolher o emulador...



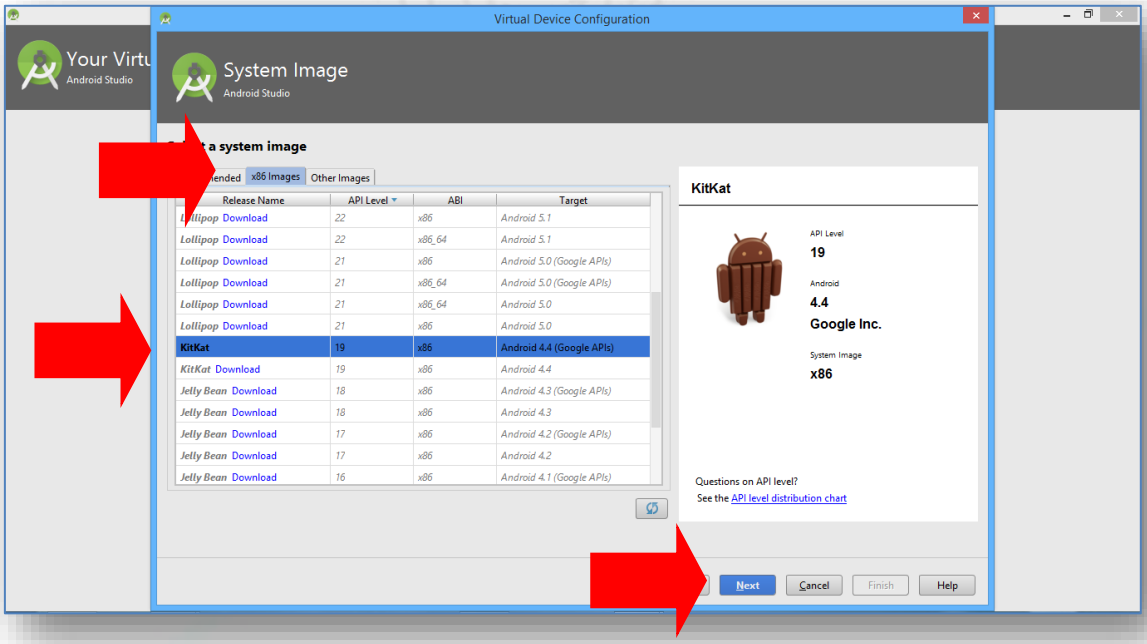
Clicar em create virtual device...



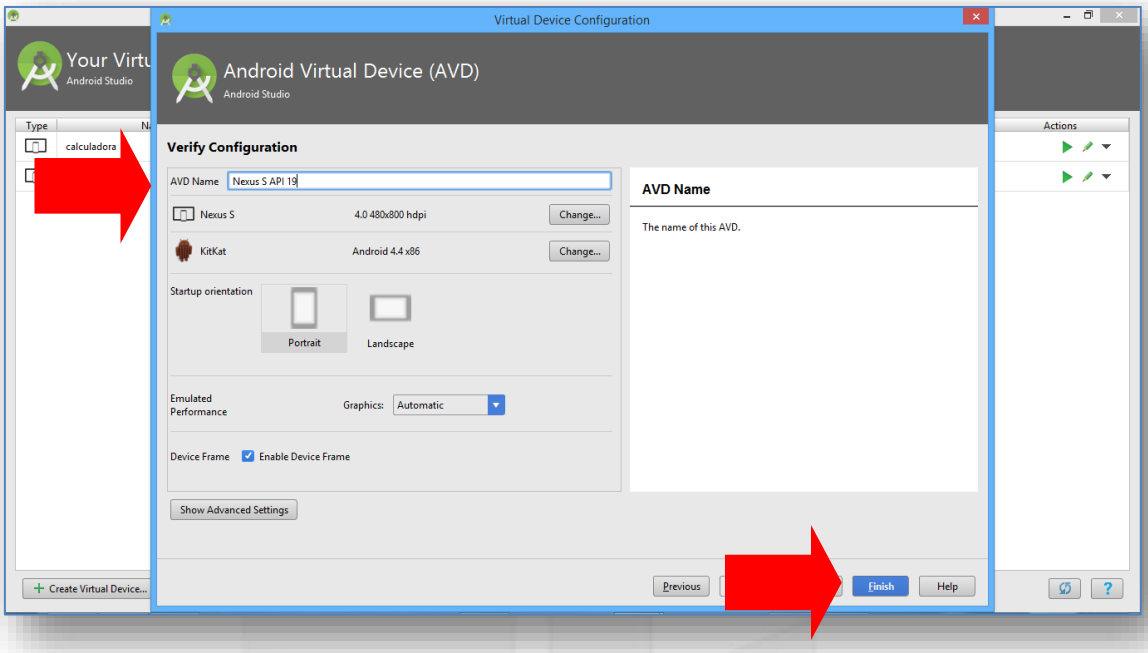
Escolhi o nexus S por ser mais rápido para compilar -> next...



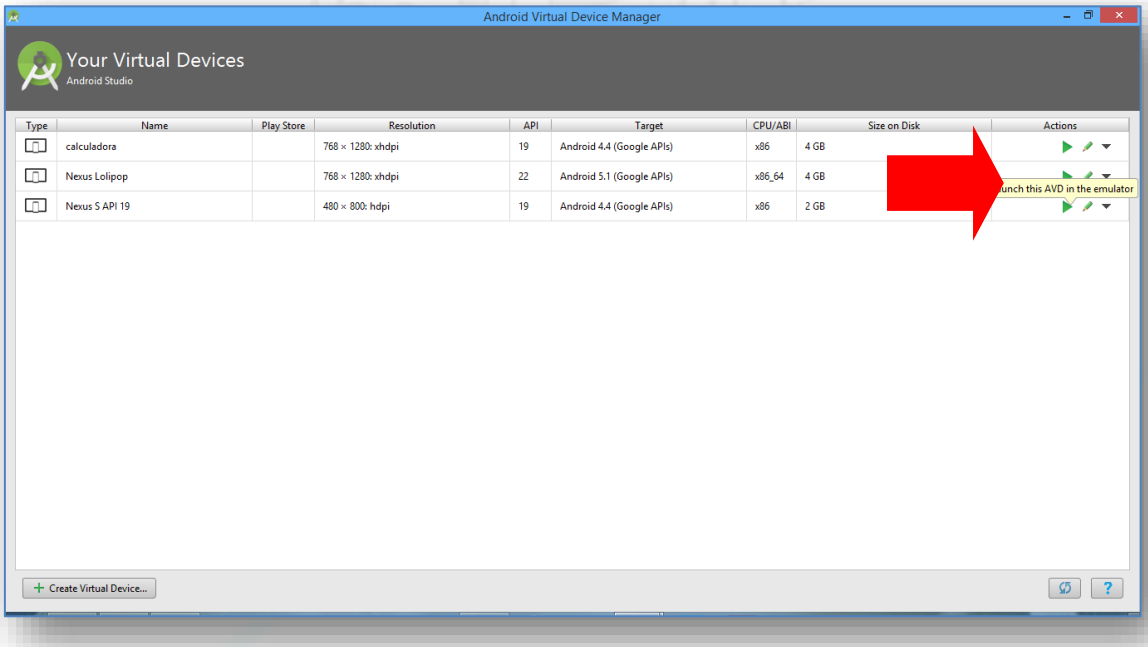
Como criamos o projeto com base no SDK Kitkat, escolher a versão compatível (KitKat) -> Next...



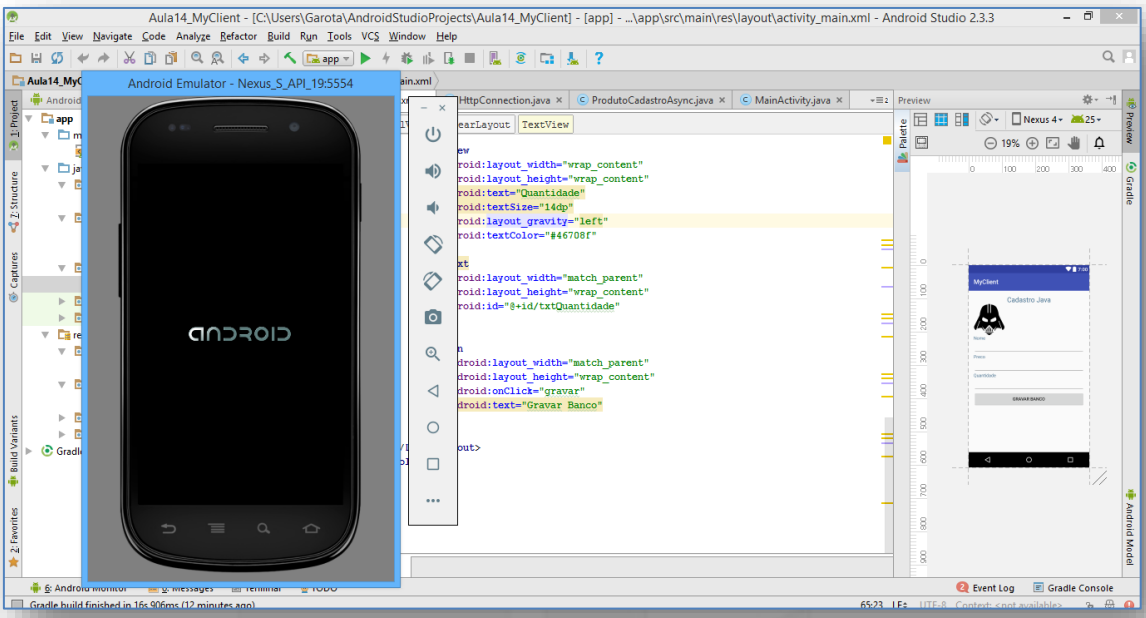
Dar um nome para o emulador -> Finish...



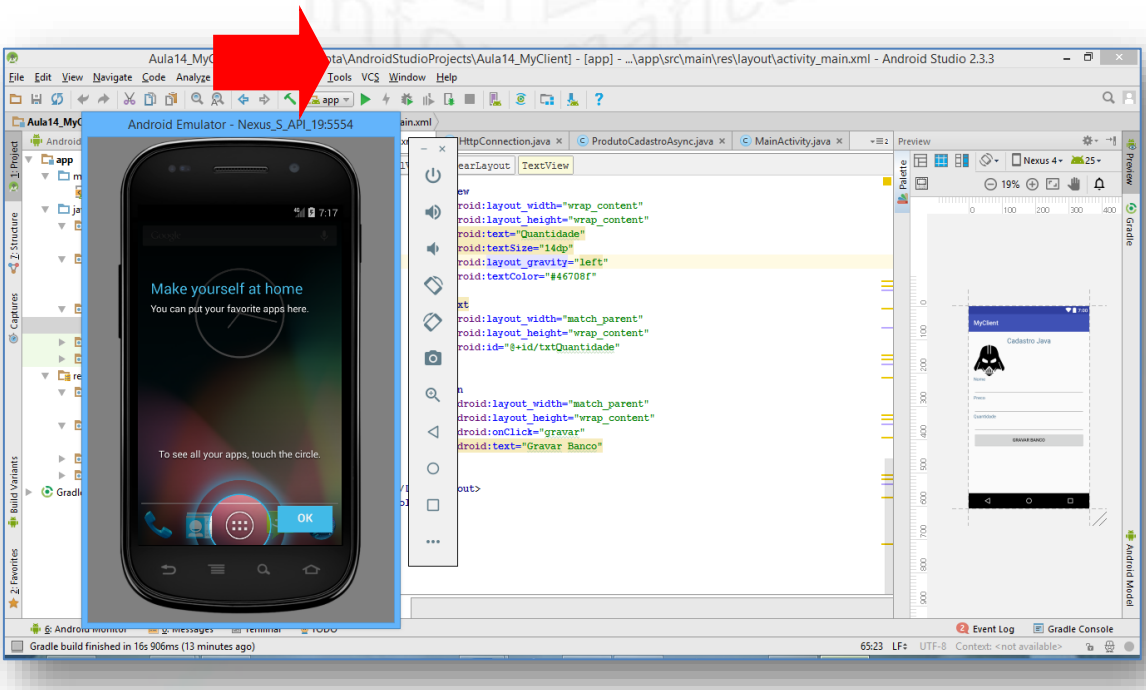
Quando abrir a janela, escolher o emulador -> clicar na seta...



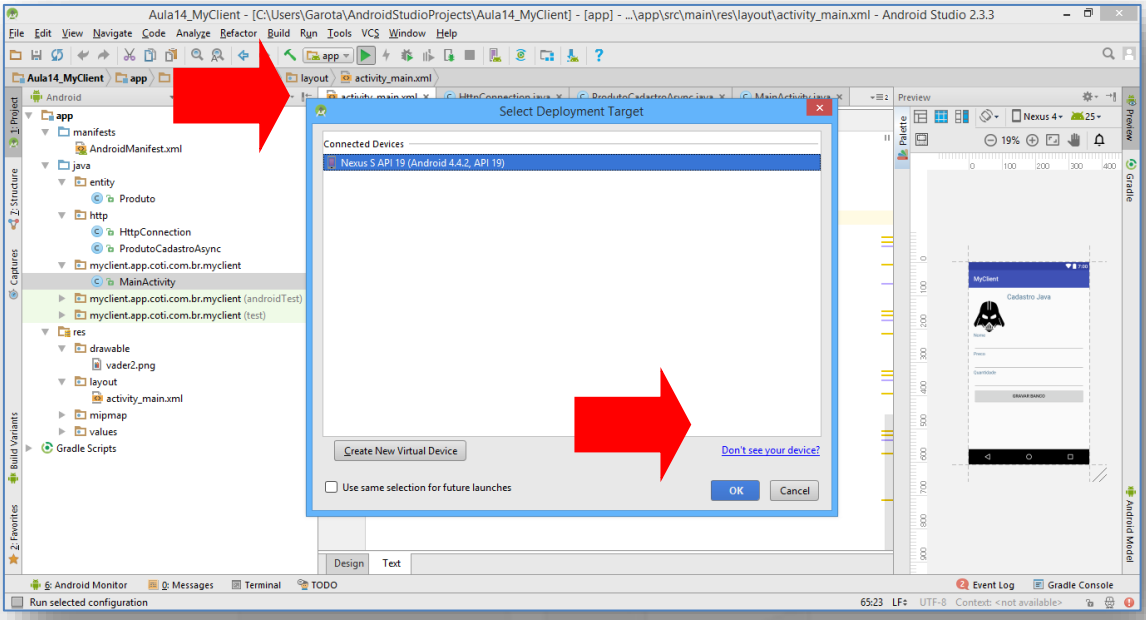
Esperar abrir o emulador...



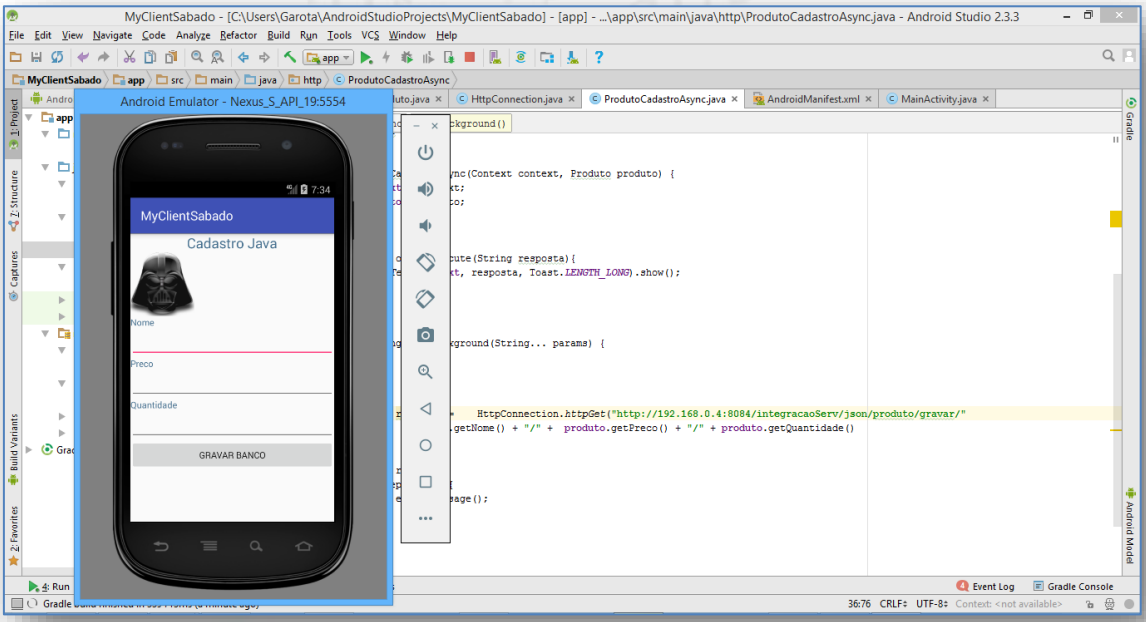
Clicar em run...



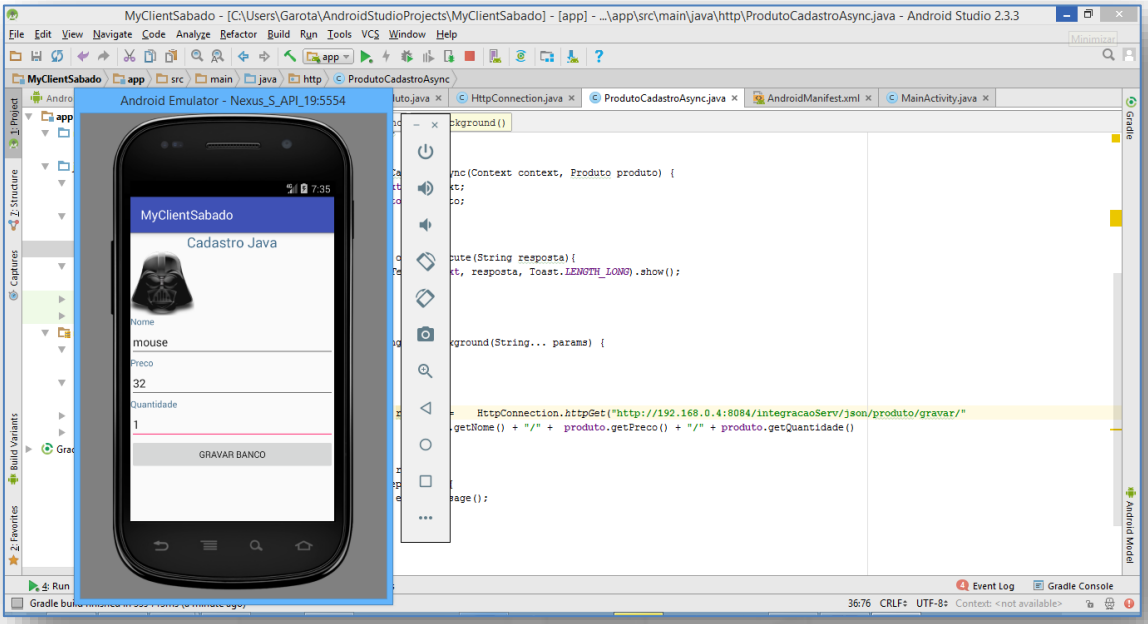
Escolher o emulador -> ok...



Depois de aberto, digitar os dados...

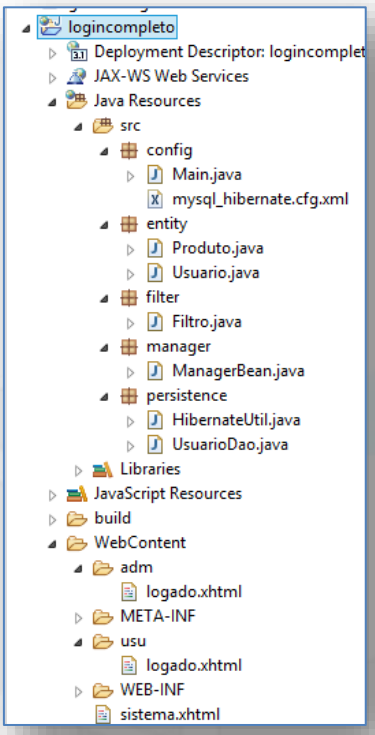


Cadastrando o produto...



Escola de Nerds
www.cotiinformatica.com.br

Estrutura do projeto:



LIBS:

```
all-themes-1.0.10.jar
antlr-2.7.6.jar
asm-attrs.jar
asm.jar
c3p0-0.9.0.jar
cglib-2.1.3.jar
commons-beanutils-1.8.0.jar
commons-codec-1.3.jar
commons-collections-3.2.1.jar
commons-digester-2.0.jar
commons-el-1.0.jar
commons-fileupload-1.0.jar
commons-lang-2.1.jar
commons-logging-1.1.1.jar
commons-validator.jar
dom4j-1.6.1.jar
ejb3-persistence.jar
hibernate-annotations.jar
```

```
hibernate-commons-annotations.jar
hibernate3.jar
jakarta-oro.jar
javax.faces-2.0.11.jar
jcommon.jar
jstl-1.1.0.jar
jstl.jar
jta.jar
log4j-1.2.15.jar
mysql-connector-java-5.1.23-bin.jar
oro-2.0.8.jar
primefaces-5.0-sources.jar
primefaces-5.0.jar
servlet3-api.jar
standard.jar
```

```
package entity;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="produto")
public class Produto {

	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	@Column
	private Integer idProduto;
	@Column(length=50)
	private String  nomeProduto;
	@Column
	private Double preco;

	public Produto() {
	}

	public Produto(Integer idProduto, String nomeProduto,
Double preco) {
```

```
		this.idProduto = idProduto;
		this.nomeProduto = nomeProduto;
		this.preco = preco;
	}

	public Integer getIdProduto() {
		return idProduto;
	}

	public void setIdProduto(Integer idProduto) {
		this.idProduto = idProduto;
	}

	public String getNomeProduto() {
		return nomeProduto;
	}

	public void setNomeProduto(String nomeProduto) {
		this.nomeProduto = nomeProduto;
	}

	public Double getPreco() {
		return preco;
	}

	public void setPreco(Double preco) {
		this.preco = preco;
	}


}
```

```
package entity;

import java.io.Serializable;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="usuario")
```

```
public class Usuario implements Serializable {

private static final long serialVersionUID = 1L;

	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	@Column
	private Integer codigo;
	@Column(length=50)
	private String nome;
	@Column(length=50, unique=true)
	private String email;
	@Column
	private String senha;
	@Column (columnDefinition="   enum ( 'usu'  ,  'adm' )  ")
	private String perfil;

	public Usuario() {

	}

	public Usuario(Integer codigo, String nome, String email,
String senha, String perfil) {
		this.codigo = codigo;
		this.nome = nome;
		this.email = email;
		this.senha = senha;
		this.perfil = perfil;
	}

	@Override
	public String toString() {
		return "Usuario [codigo=" + codigo + ", nome=" + nome
+ ", email=" + email + ", perfil=" + perfil + "]";
	}

	public Integer getCodigo() {
		return codigo;
	}

	public void setCodigo(Integer codigo) {
		this.codigo = codigo;
```

```
	}

	public String getNome() {
		return nome;
	}

	public void setNome(String nome) {
		this.nome = nome;
	}

	public String getEmail() {
		return email;
	}

	public void setEmail(String email) {
		this.email = email;
	}

	public String getSenha() {
		return senha;
	}

	public void setSenha(String senha) {
		this.senha = senha;
	}

	public String getPerfil() {
		return perfil;
	}

	public void setPerfil(String perfil) {
		this.perfil = perfil;
	}

}
```

---

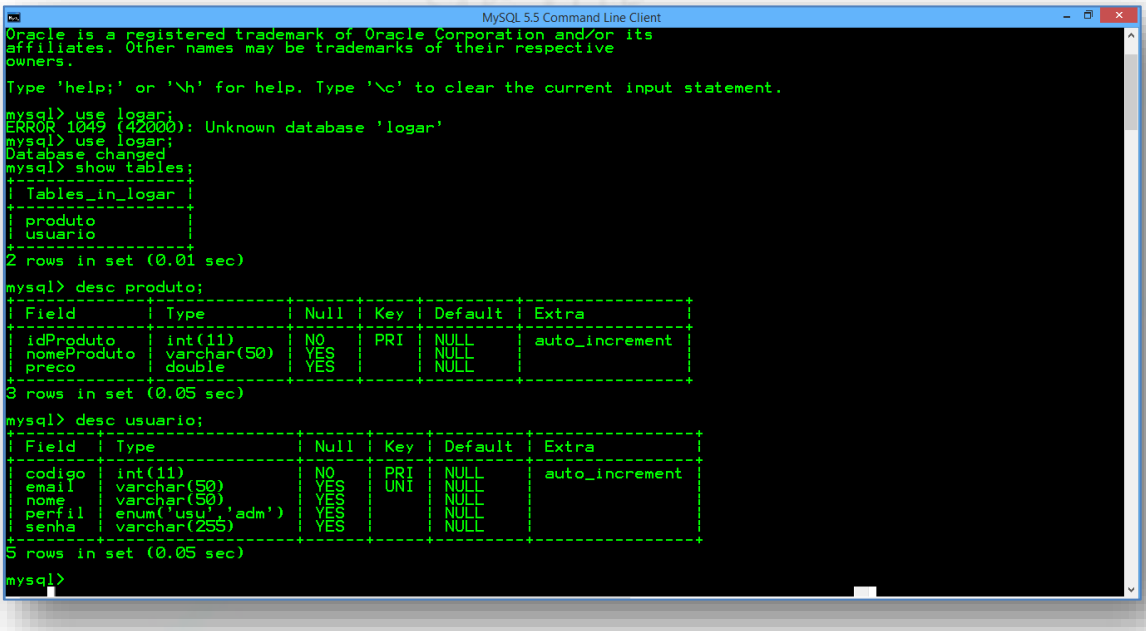
mysql_hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
```

```
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</pro
perty>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</
property>
    <property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/logar?createDatabaseIfNotExist=true<
/property>
    <property
name="hibernate.connection.username">root</property>
    <property
name="hibernate.connection.password">coti</property>

      <property name="hibernate.show_sql">true</property>
      <property name="hibernate.format_sql">true</property>
      <property name="hibernate.hbm2ddl.auto">update</property>

        <mapping class="entity.Usuario"/>
        <mapping class="entity.Produto"/>

 </session-factory>
</hibernate-configuration>
```

```
package config;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.cfg.Configuration;
import org.hibernate.tool.hbm2ddl.SchemaExport;

public class Main {

     public static void main(String[] args) {
          Configuration cfg = new AnnotationConfiguration().
               configure("config/mysql_hibernate.cfg.xml");
          new SchemaExport(cfg).create(true,true);
         System.out.println("Tabelas Geradas ... Com Sucesso");

        /*
         *
    drop table if exists produto
```

```
    drop table if exists usuario

    create table produto (
        idProduto integer not null auto_increment,
        nomeProduto varchar(50),
        preco double precision,
        primary key (idProduto)
    )

    create table usuario (
        codigo integer not null auto_increment,
        email varchar(50) unique,
        nome varchar(50),
        perfil    enum ( 'usu'  ,  'adm' )  ,
        senha varchar(255),
        primary key (codigo)
    )
Tabelas Geradas ... Com Sucesso
         */
     }

}
```



```
package persistence;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;

public class HibernateUtil {

    private static final SessionFactory sessionFactory;
    static {
        try {
            sessionFactory = new AnnotationConfiguration().
configure("config/mysql_hibernate.cfg.xml").buildSessionFactory(
);

        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation
failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

```
package persistence;

import org.hibernate.Criteria;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.Transaction;
import org.jcommon.encryption.SimpleMD5;

import entity.Usuario;

public class UsuarioDao {

      //13/10 _ EJB _ Aluno (180) _bootstrap / angular (13:30 -
19:30 ou 20:30)
      // 07/10 _ Turma Nova (tarde)
```

```
      // 08/10 _ certificacao _ 09:00 as 13:00 (presencial) _
gravado ... (aulas gravados) _ 500
      // 18/10 seg/qua/sex ( nova ) 10:00 as 13:00
      // 27/10 _ Android (Sexta-Noite) _ 500


      Session session;
      Transaction transaction;
      Criteria criteria;
      Query query;

        public void criptografar(Usuario u){
            SimpleMD5 md5 = new SimpleMD5(u.getSenha(),
     "profedsonbelem@gmail.com1234567890@$");
             u.setSenha(md5.toHexString());
        }

       public void create(Usuario u) {
                  criptografar(u);
//criptografa para gravar depois
    session = HibernateUtil.getSessionFactory().openSession();
                   transaction = session.beginTransaction();
                        session.save(u);
//gravo o usuario com a senha Criptografada ... Cafe
                   transaction.commit();
               session.close();
       }

public Usuario findByLogin(Usuario u) {
           criptografar(u);
    session = HibernateUtil.getSessionFactory().openSession();
         //email nao pode errar nem a senha ...
             query = session.
   createQuery("select u from Usuario as u where u.email=:param1
and u.senha=:param2");
          query.setParameter("param1", u.getEmail());
          query.setParameter("param2",  u.getSenha());
          Usuario resp = (Usuario) query.uniqueResult();
         session.close();
         return resp;
  }

       public static void main(String[] args) {
          try{
```

```
      Usuario usu = new
Usuario(null,"lu","lu@gmail.com","123","adm");
           //     new UsuarioDao().create(usu); //Cadastro ...
       Usuario resposta = new UsuarioDao().findByLogin(usu);
         if (resposta == null){
               System.out.println("...Nao Logado");
         }else{
              System.out.println("...Logado :" + resposta );
         }
           }catch(Exception ex){
             ex.printStackTrace();
           }
      }
}
```

No console...

```
log4j:WARN No appenders could be found for logger
(org.hibernate.cfg.annotations.Version).
log4j:WARN Please initialize the log4j system properly.
Hibernate:
    select
        usuario0_.codigo as codigo0_,
        usuario0_.email as email0_,
        usuario0_.nome as nome0_,
        usuario0_.perfil as perfil0_,
        usuario0_.senha as senha0_
    from
        usuario usuario0_
    where
        usuario0_.email=?
        and usuario0_.senha=?
...Logado :Usuario [codigo=1, nome=lu, email=lu@gmail.com,
perfil=adm]
```

```
package filter;

import java.io.IOException;

import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.FilterConfig;
```

```
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebFilter( urlPatterns={"/usu/*","/adm/*"} )
public class Filtro implements Filter
{

    public Filtro() {
    }

     public void destroy() {
     }

     public void doFilter(ServletRequest request,
ServletResponse response, FilterChain chain) throws IOException,
ServletException {
        HttpServletRequest   req= (HttpServletRequest) request;
   HttpServletResponse resp= (HttpServletResponse) response;
             HttpSession session = req.getSession();
//ele so consegue entrar nas pastas usu e adm se estiver logado
                if (session.getAttribute("logado")!=null){
                  chain.doFilter(request, response);
//continuar    (IR)
                }else{
                   resp.sendRedirect("../sistema.jsf");
//senao volta
                }
     }

     public void init(FilterConfig fConfig) throws
ServletException {
     }
}
```

```
package manager;

import javax.annotation.PostConstruct;
import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.RequestScoped;
import javax.faces.context.FacesContext;
import javax.servlet.http.HttpSession;

import entity.Usuario;
import persistence.UsuarioDao;

@ManagedBean(name="mb")
@RequestScoped
public class ManagerBean {

	private Usuario usuario;
	private Usuario logado; //sÃ³ logado (sem espaÃ§o) Nao ganha espaÃ§o ainda

	@PostConstruct
	public void init(){
		usuario = new Usuario(); //espaÃ§o
	}

	public Usuario getUsuario() {
		return usuario;
	}

	public void setUsuario(Usuario usuario) {
		this.usuario = usuario;
	}

	public Usuario getLogado() {
		return logado;
	}

	public void setLogado(Usuario logado) {
		this.logado = logado;
	}

	public void gravar(){
		FacesContext fc = FacesContext.getCurrentInstance();
		try{
```

```
                //validar ...
               new UsuarioDao().create(usuario);
//gravar com criptografia
               usuario = new Usuario();
//da espaço de mem (limpar os Dados)
fc.addMessage(null, new FacesMessage("Dados Gravados ..."));
//Mensagem
          }catch(Exception ex){
               fc.addMessage(null, new FacesMessage("Error :" +
ex.getMessage()));
          }
     }

     public String logar(){
          FacesContext fc  =FacesContext.getCurrentInstance();
           try{
                HttpSession session = (HttpSession)
fc.getExternalContext().getSession(true);
                logado = new
UsuarioDao().findByLogin(usuario);
                if (logado !=null){
                 session.setAttribute("logado", logado);
                 return logado.getPerfil() + "/logado.jsf";
                }else{
                  fc.addMessage(null,  new FacesMessage("Nao
Logado, Acesso Invalido..."));
                      return null;
                }
           }catch(Exception ex){
                fc.addMessage(null,  new FacesMessage("error :"
+ ex.getMessage()));
           }
          return null;
     }

     //Limpar ou excluir a SessÃ£o (Logou deve apagar isso)
Obrigatoriamente ...
     public String logout(){
          FacesContext fc = FacesContext.getCurrentInstance();
          HttpSession session = (HttpSession)
fc.getExternalContext().getSession(true);
          try{
               //Limpar o logado (Limpar o logado ) ...
                session.setAttribute("logado", null);
//invalida a sessao
```

```
  fc.addMessage(null, new FacesMessage("Logout no Sistema"));
//da a MSG
                  //volta ele rediciona (estÃ¡ tentando entrar
na marra no sistema)
                return "/sistema.jsf?faces-redirect=true";
//volta para o Sistema
            }catch(Exception ex){
                  fc.addMessage(null, new FacesMessage("Error :"
+ ex.getMessage()));
                return "/sistema.jsf?faces-redirect=true";
            }
      }
         //Tudo ComeÃ§a por aqui ...
       //Aqui a porta de tudo que faÃ§o ...
}
```

---

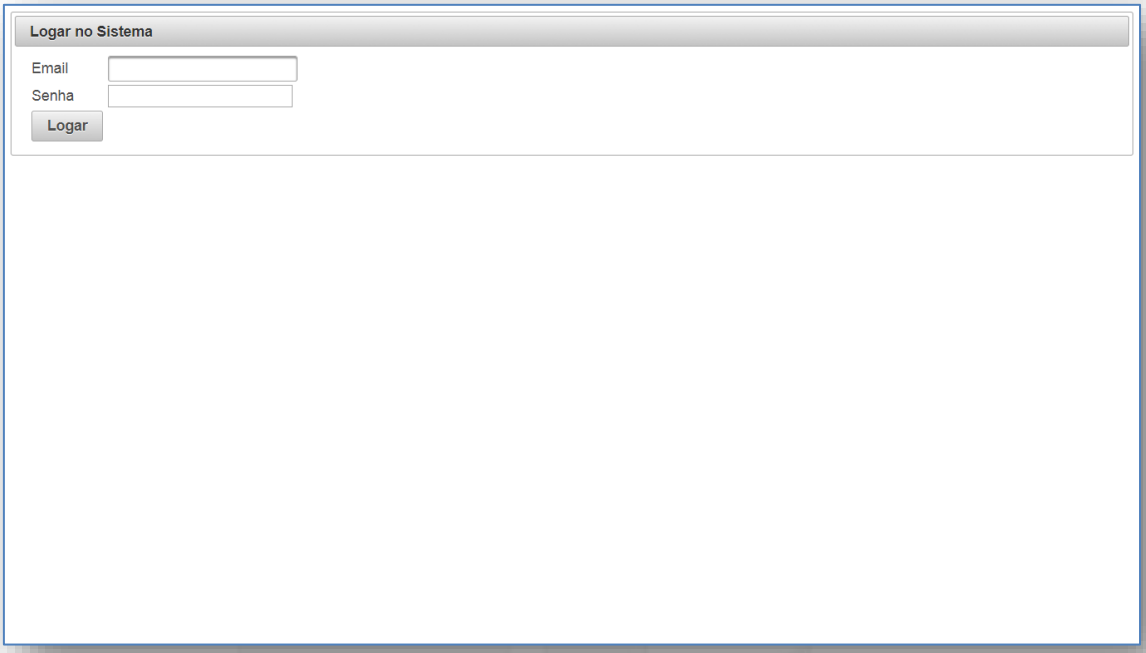
web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
version="3.1">
  <display-name>logincompleto</display-name>
  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>
</web-app>
```

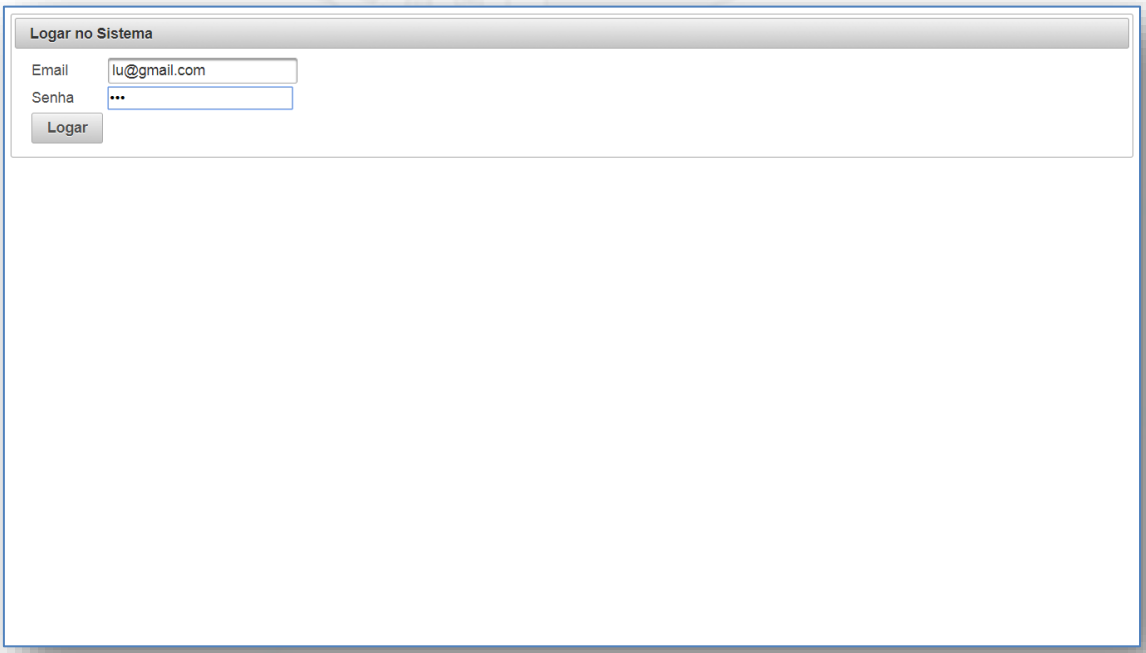
---

sistema.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head>
</h:head>
<h:body>
    <p:panel header="Logar no Sistema">
        <h:form id="form1">
             <h:panelGrid columns="2">
                  <h:outputText value="Email" />
                  <p:inputText value="#{mb.usuario.email}" />
                  <h:outputText value="Senha" />
                  <h:inputSecret value="#{mb.usuario.senha}" />
                  <p:commandButton value="Logar"
action="#{mb.logar}"
ajax="false"
                  ></p:commandButton>
             </h:panelGrid>
        </h:form>
        <p:growl></p:growl>
     </p:panel>
 </h:body>

</html>
```

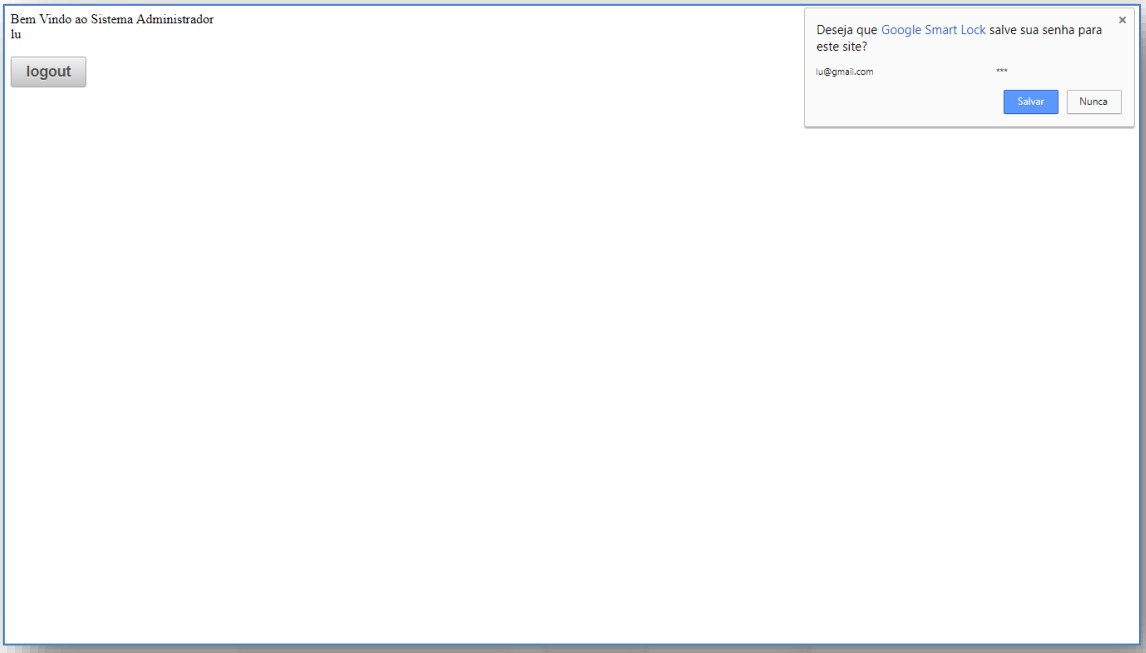


Logando como adm...



adm/logado.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head>
</h:head>

<h:body>
 <h:form id="form3">
    <h:outputText value="Bem Vindo ao Sistema
Administrador"></h:outputText>
 <br/>
     <h:outputText value="#{logado.nome}" />
  <br/><br/>
    <p:commandButton value="logout" action="#{mb.logout}"
      ajax="false"
    ></p:commandButton>
    <p:growl></p:growl>
   </h:form>
</h:body>
</html>
```



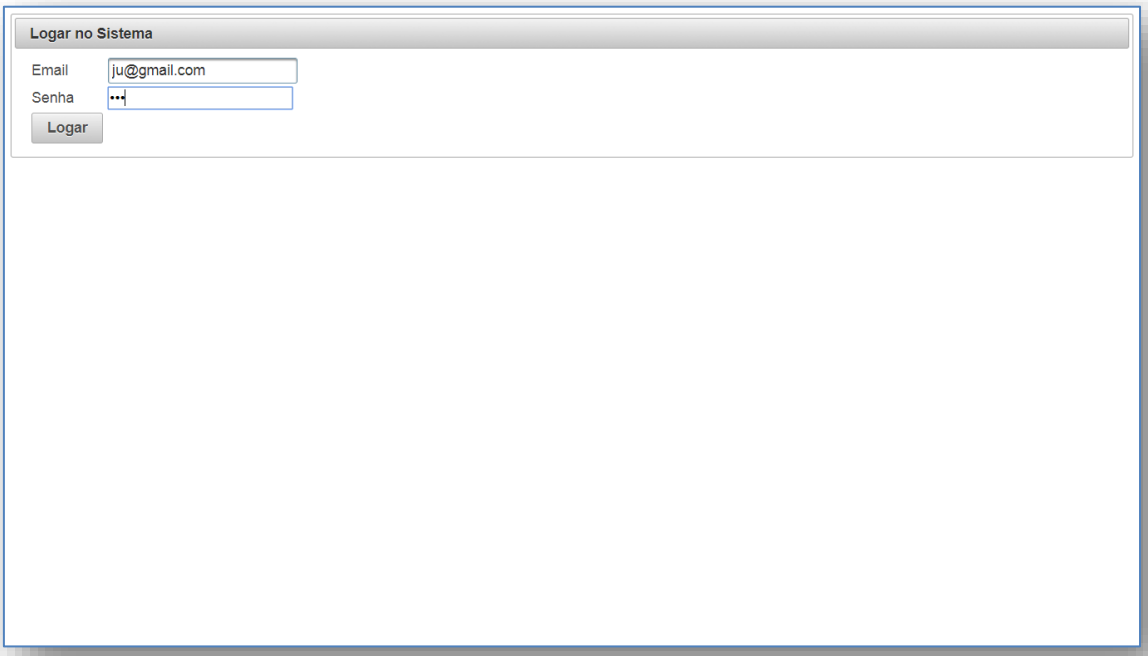
---

usu/logado.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head>
</h:head>

<h:body>
 <h:form id="form3">
    <h:outputText value="Bem Vindo ao Sistema
Usuario"></h:outputText>
 <br/>
     <h:outputText value="#{logado.nome}" />
  <br/><br/>
    <p:commandButton value="logout" action="#{mb.logout}"
```

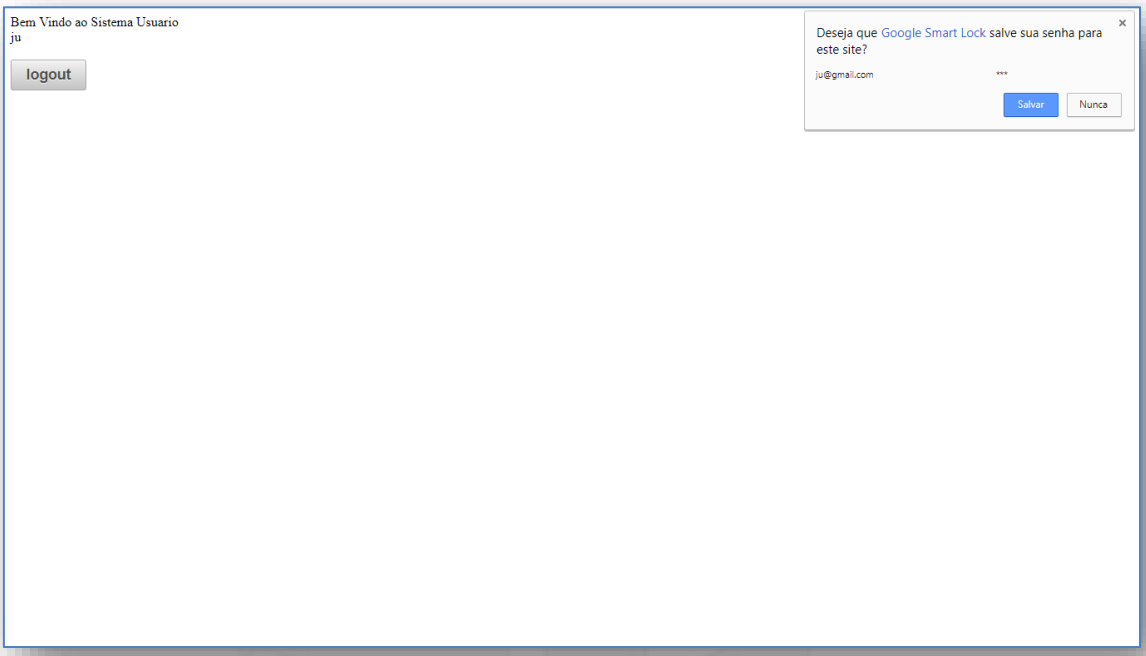
```
        ajax="false"
      ></p:commandButton>
      <p:growl></p:growl>
    </h:form>
</h:body>
</html>
```

Logando como usu…

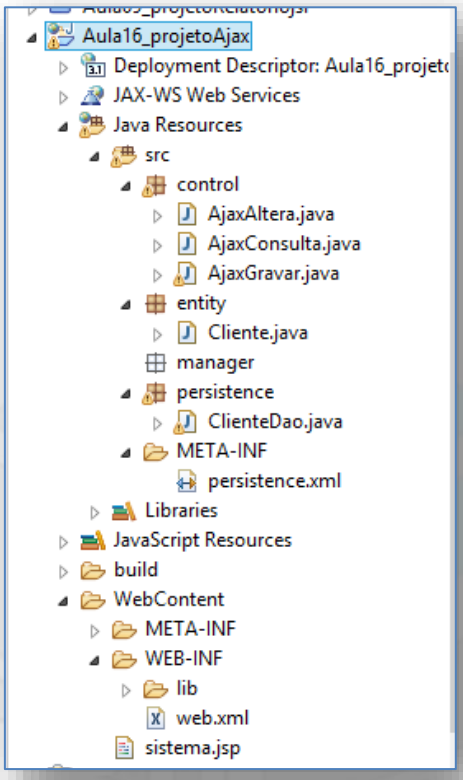


Escola de Nerds
www.cotiinformatica.com.br

Logado...


Bem Vindo ao Sistema Usuario
ju
logout
Deseja que Google Smart Lock salve sua senha para este site?
ju@gmail.com
***
Salvar
Nunca

Estrutura do projeto:



LIBS:

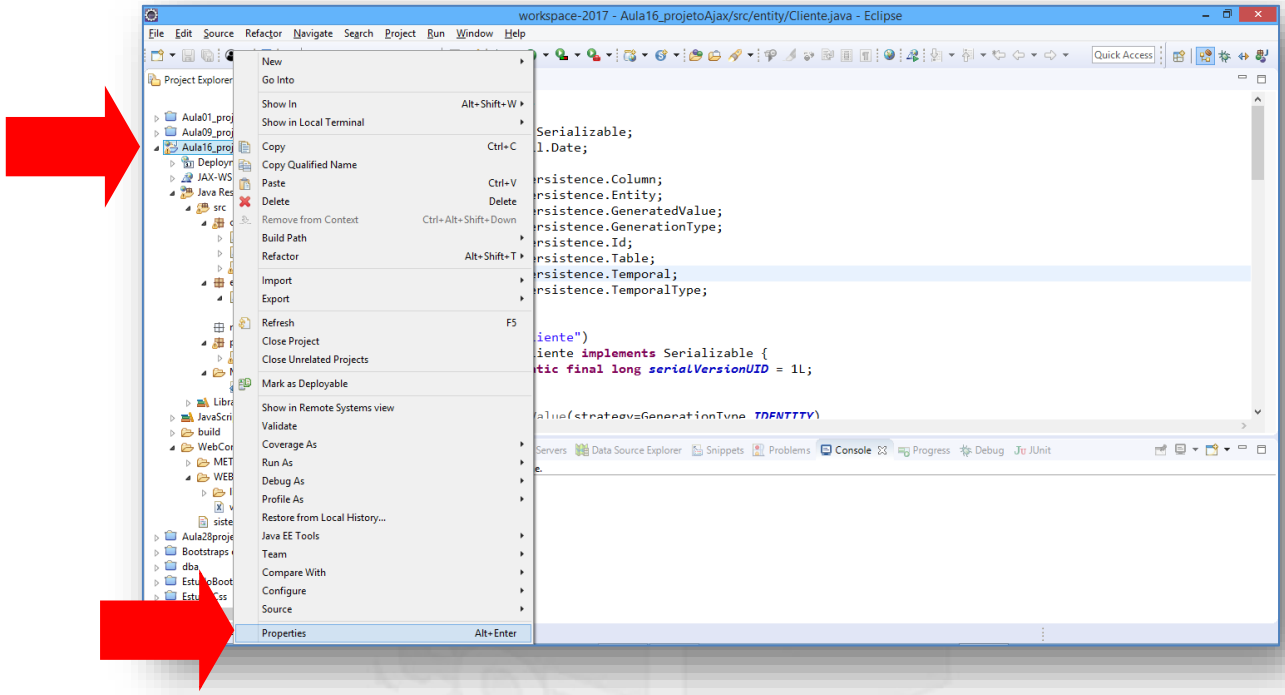
```
antlr-2.7.7.jar
c3p0-0.9.2.1.jar
commons-collections-3.2.1-1.0.0.jar
dom4j-1.6.1.jar
ehcache-core-2.4.3.jar
hibernate-c3p0-4.3.1.Final.jar
hibernate-commons-annotations-4.0.4.Final.jar
hibernate-core-4.3.1.Final.jar
hibernate-ehcache-4.3.1.Final.jar
hibernate-entitymanager-4.3.1.Final.jar
hibernate-jpa-2.1-api-1.0.0.Final.jar
hibernate-validator-4.3.0.Final.jar
hibernate-validator-annotation-processor-4.3.0.Final.jar
jandex-1.1.0.Final.jar
javassist-3.18.1-GA.jar
jboss-logging-3.1.3.GA.jar
jboss-logging-annotations-1.2.0.Beta1.jar
```

```
jboss-transaction-api_1.2_spec-1.0.0.Final.jar
log4j-1.2.15.jar
mysql-connector-java-5.1.23-bin.jar
servlet3-api.jar
slf4j-api-1.6.1.jar
slf4j-log4j12-1.6.0-rc0.jar
validation-api-1.0.0.GA.jar
```

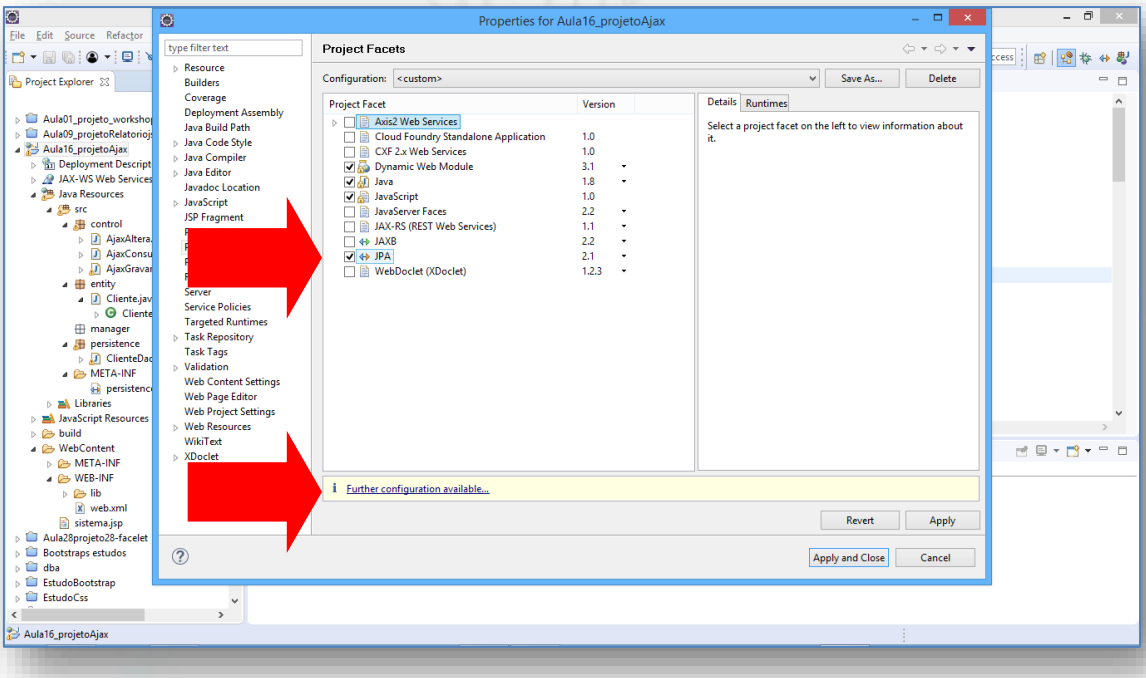
```
package entity;

import java.io.Serializable;
import java.util.Date;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
import javax.persistence.Temporal;
import javax.persistence.TemporalType;

@Entity
@Table(name="cliente")
public class Cliente implements Serializable {

	private static final long serialVersionUID = 1L;

	@Id
	@GeneratedValue(strategy=GenerationType.IDENTITY)
	private	Integer idCliente;
	@Column(length=50)
	private	String	nome;
	@Column(unique=true,length=50)
	private	String	email;
	@Temporal(TemporalType.DATE)
	private	Date	dataNascimento;

	public Cliente() {

	}
```

```
	public Cliente(Integer idCliente, String nome, String
email, Date dataNascimento) {
		super();
		this.idCliente = idCliente;
		this.nome = nome;
		this.email = email;
		this.dataNascimento = dataNascimento;
	}

	@Override
	public String toString() {
		return "Cliente [idCliente=" + idCliente + ", nome="
+ nome + ", email=" + email + ", dataNascimento=" +
dataNascimento + "]";
	}

	public Integer getIdCliente() {
		return idCliente;
	}
	public void setIdCliente(Integer idCliente) {
		this.idCliente = idCliente;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public Date getDataNascimento() {
		return dataNascimento;
	}
	public void setDataNascimento(Date dataNascimento) {
		this.dataNascimento = dataNascimento;
	}
}
```

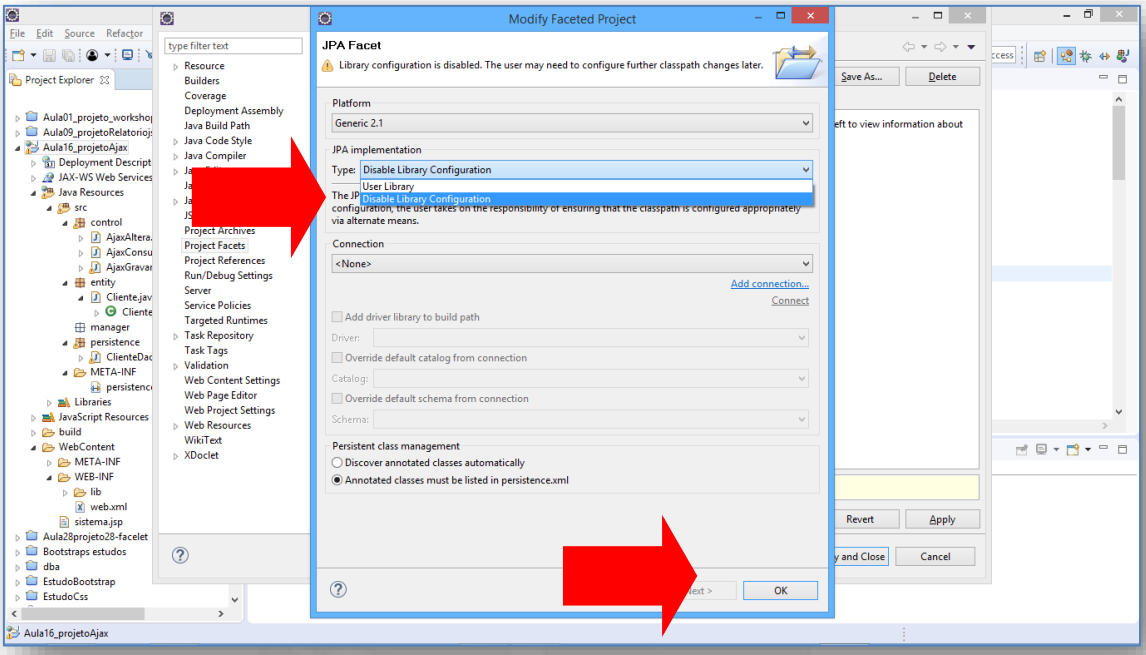
Habilitar o JPA…
Clicar com o botão direito no projeto -> properties...



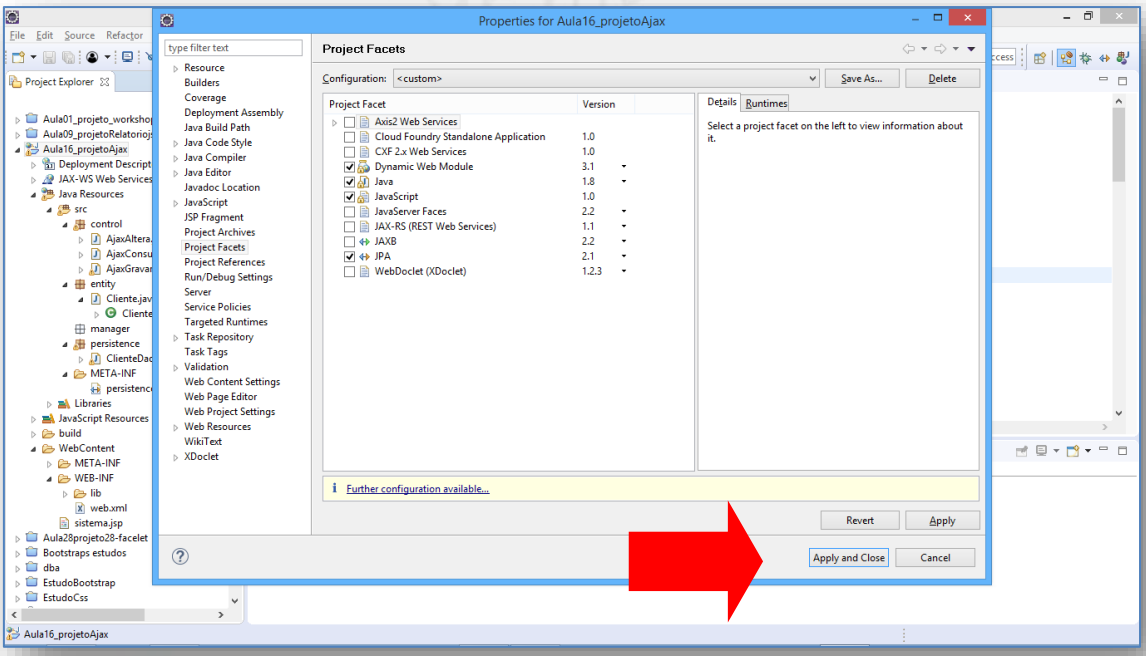
Clicar em JPA -> Futher configuration...



Em Type, desabilitar -> ok...



Apply and close...



persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1"
xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd">

Atenção com o nome da conexão!!!
    <persistence-unit name="AJAX"  transaction-
type="RESOURCE_LOCAL">

<provider>org.hibernate.ejb.HibernatePersistence</provider>

 <class>entity.Cliente</class>

        <properties>
            <property name="hibernate.dialect"
value="org.hibernate.dialect.MySQL5InnoDBDialect" />
            <property name="javax.persistence.jdbc.driver"
value="com.mysql.jdbc.Driver" />
            <property name="javax.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/ajax?createDatabaseIfNotExist
=true" />

            <property name="javax.persistence.jdbc.user"
value="root" />
             <property name="javax.persistence.jdbc.password"
value="coti" />

           <property name="hibernate.show_sql" value="true" />
           <property name="hibernate.format_sql" value="true" />
           <property name="hibernate.hbm2ddl.auto"
value="update" />
        </properties>
    </persistence-unit>
</persistence>
```

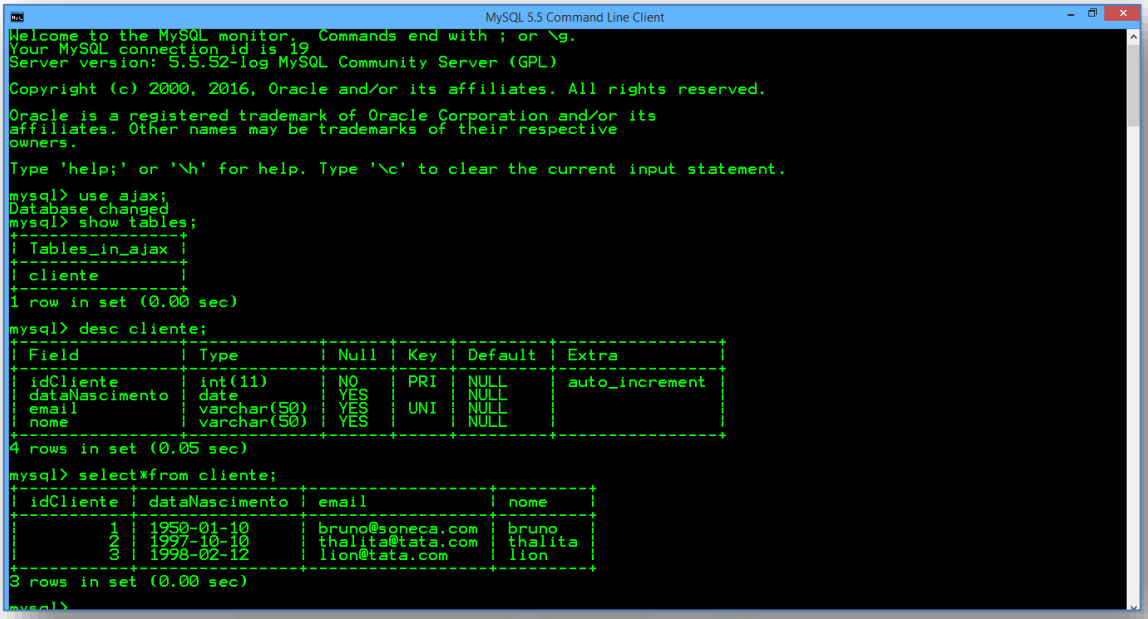
```
package persistence;

import java.util.Date;
import java.util.List;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.EntityTransaction;
import javax.persistence.Persistence;
import javax.persistence.Query;

import entity.Cliente;

public class ClienteDao {

	EntityManager em;
	EntityTransaction transaction;

	public ClienteDao() {
		EntityManagerFactory factory =
Persistence.createEntityManagerFactory("AJAX");
		em = factory.createEntityManager();
	}

	public void create(Cliente c) throws Exception {
		transaction = em.getTransaction();
		transaction.begin();
		em.persist(c);
		transaction.commit();
	}

	public void update(Cliente c) throws Exception {
		transaction = em.getTransaction();
		transaction.begin();
		em.merge(c);
		transaction.commit();
	}

	public void delete(Cliente c) {
		transaction = em.getTransaction();
		transaction.begin();
		   em.remove( em.getReference(Cliente.class,
c.getIdCliente()));
		transaction.commit();
```

```
	}

	public List<Cliente> findAll() {
		List<Cliente> lista = em.createQuery("select obj from
Cliente as obj").getResultList();
		return lista;
	}

	public Cliente findByCode(Integer cod) {
		Query q = em.createQuery("select c from Cliente as c
where  c.idCliente= :param1 ");
				q.setParameter("param1", cod);
		Cliente c = (Cliente) q.getSingleResult();
		return c;
	}


	public static void main(String[] args) {
		try{
//			Cliente c1 = new
Cliente(null,"bruno","bruno@soneca.com", new
Date("01/10/1950"));
//			Cliente c2 = new
Cliente(null,"thalita","thalita@tata.com", new
Date("10/10/1997"));
//			Cliente c3 = new
Cliente(null,"lion","lion@tata.com", new Date("14/12/1997"));

//
//			new ClienteDao().create(c1);
//			new ClienteDao().create(c2);
//			new ClienteDao().create(c3);
//
//		 System.out.println("Dados gravados.... "+ c1 + ","+
c2 +"," +c3);
//
//		 new ClienteDao().create(new
Cliente(null,"carlinho","carlinho@gmail", new
Date(20/01/1995)));
//

		new
ClienteDao().findAll().forEach(System.out::println);
```

```
            }catch(Exception ex){
                  ex.printStackTrace();
            }
      }
}
```

No console…

```
log4j:WARN No appenders could be found for logger
(org.jboss.logging).
log4j:WARN Please initialize the log4j system properly.
Hibernate:
    select
        cliente0_.idCliente as idClient1_0_,
        cliente0_.dataNascimento as dataNasc2_0_,
        cliente0_.email as email3_0_,
        cliente0_.nome as nome4_0_
    from
        cliente cliente0_
Cliente [idCliente=1, nome=bruno, email=bruno@soneca.com,
dataNascimento=1950-01-10]
Cliente [idCliente=2, nome=thalita, email=thalita@tata.com,
dataNascimento=1997-10-10]
Cliente [idCliente=3, nome=lion, email=lion@tata.com,
dataNascimento=1998-02-12]
```



```
package control;

import java.io.IOException;
import java.util.Date;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import entity.Cliente;
import persistence.ClienteDao;


@WebServlet("/AjaxGravar")
public class AjaxGravar extends HttpServlet {

	private static final long serialVersionUID = 1L;
```

```
    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {

    }

    protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
      try{
                  String nome = request.getParameter("nome");
      String email =  request.getParameter("email");
      String data[]  = request.getParameter("data").split("-");
                  //pos[0]= ano, pos[1]= mes, pos[2]= dia

                  Cliente c = new Cliente(null, nome, email,
new Date(data[1]+ "/" + data[2] + "/" + data[0]));
                   new ClienteDao().create(c);
 response.getWriter().print("Servlet ...Dados Gravados ..."+ c);
      }catch(Exception ex){
           response.getWriter().print("Error ..."+
ex.getMessage());
      }
    }
}
```

```
package control;

import java.io.IOException;
import java.text.SimpleDateFormat;
import java.util.List;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import entity.Cliente;
import persistence.ClienteDao;

@WebServlet("/AjaxConsulta")
```

```
public class AjaxConsulta extends HttpServlet {

	private static final long serialVersionUID = 1L;

	protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {

	}

	protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
		try{
			List <Cliente> lst =  new ClienteDao().findAll();
				SimpleDateFormat sdf= new
SimpleDateFormat("dd/MM/yyyy");
			response.getWriter().print("<table border=1>");
				response.getWriter().print("<thead>");
				response.getWriter().print("<tr>");
response.getWriter().print("<th>idCliente</th>");
response.getWriter().print("<th>Nome</th>");
response.getWriter().print("<th>Email</th>");
response.getWriter().print("<th>Data</th>");
response.getWriter().print("<th>Excluir</th>");
response.getWriter().print("<th>Alterar</th>");
				response.getWriter().print("</tr>");
				response.getWriter().print("</thead>");
				for(Cliente x : lst){
					response.getWriter().print("<tr>");
response.getWriter().print("<td>"+x.getIdCliente()+"</td>");
response.getWriter().print("<td>"+x.getNome()+"</td>");
response.getWriter().print("<td>"+x.getEmail()+"</td>");
response.getWriter().print("<td>"+sdf.format(x.getDataNascimento
())+"</td>");
				response.getWriter().print("<th><button
class='btn3' type='button'" + "  data-id="+
x.getIdCliente()+">Excluir</button></th>");
			response.getWriter().print("<th><button
class='btn4' type='button'"
+ "    data-
id="+ x.getIdCliente()+">Alterar</button></th>");
```

```
                        response.getWriter().print("</tr>");
                    }
                    response.getWriter().print("</table>");
        }catch(Exception ex){
             response.getWriter().print("Error ..."+
ex.getMessage());
        }
    }
}
```

```
package control;

import java.io.IOException;
import java.text.SimpleDateFormat;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import entity.Cliente;
import persistence.ClienteDao;


@WebServlet("/AjaxAltera")
public class AjaxAltera extends
HttpServlet {

    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
    }

    protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
```

```
            Integer codigo = new
Integer(request.getParameter("codigo"));
            try{
                  Cliente cliente = new
ClienteDao().findByCode(codigo);
                  SimpleDateFormat sdf  = new
SimpleDateFormat("yyyy-MM-dd");

           response.getWriter().print("codigo<br/>");
           response.getWriter().print("<input type='text'
name='idCliente' readonly='true' class='aidCliente'
value="+codigo+">");
           response.getWriter().print("<br/>Nome<br/>");
  response.getWriter().print("<input type='text' name='nome'
class='anome'  value="+cliente.getNome()+">");
  response.getWriter().print("<br/>Email<br/>");
  response.getWriter().print("<input type='email' name='email'
class='aemail'  value="+cliente.getEmail()+">");
  response.getWriter().print("<br/>Email<br/>");
  response.getWriter().print("<input type='date' name='data'
class='adata' " + " value=" +
sdf.format(cliente.getDataNascimento())+">");
            }catch(Exception ex){
            }
     }
}
```

---

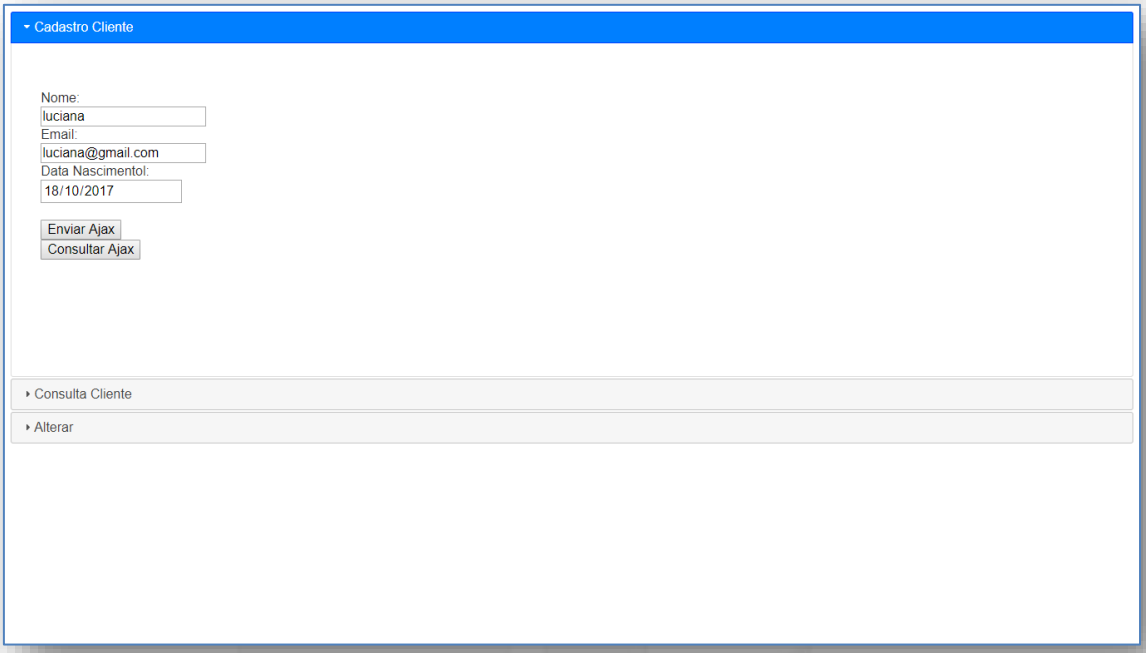
sistema.jsp

```
<!doctype html>
<html>
<head>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.12.0/jqu
ery-ui.css"/>
<script type="text/javascript"
src="https://code.jquery.com/jquery-3.1.0.js"  ></script>
<script type="text/javascript"
src="https://code.jquery.com/ui/1.12.0/jquery-ui.js"></script>

<script>
$(document).ready(function(){
      $("#accordion").accordion();
```

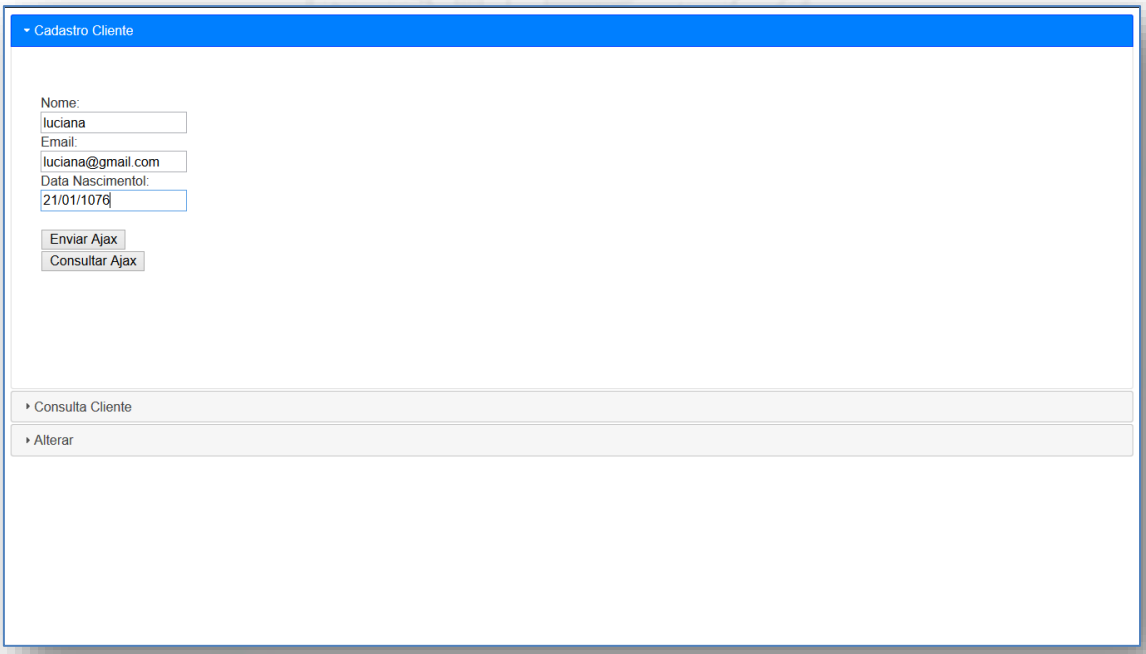
```
 $(".tela1").hide();
 var gid = "";
 var gnome  = "";
 var gemail   = "";
 var gdata   = "";

function resgate(){
 gnome = $("#nome").val();
 gemail =  $("#email").val();
 gdata =    $("#data").val();
}
  function resgateCodigo(){
 gid = $("#idCliente").val();
 }

   function limpar(){
      $("#nome").val("");
      $("#email").val("");
      $("#data").val("");
   }

   $("#btn1").on('click',function(){
       resgate();
       $.ajax({
                url : "AjaxGravar",
                method : "POST",
                data : {nome :  gnome,
                                email :   gemail,
                                data :     gdata
                                }
       }).done(function(resultado){
              $(".resposta").html(resultado);
       });
           limpar();
    });

   $("#btn2").on('click',function(){
       $.ajax({
                url : "AjaxConsulta",
                method : "POST",
                data : {  }
       }).done(function(resultado){
              $(".tabela").html(resultado);
       });
       $("#accordion").accordion({active : 1 });
```

```
            });

           $(document).on('click',  '.btn4' ,function(){
                var id = $(this).attr('data-id');
               $.ajax({
                        url : "AjaxAltera",
                        method : "POST",
                        data : {  codigo : id}
               }).done(function(resultado){
                      $(".formulario").html(resultado);
               });
               $("#accordion").accordion({active : 2 });
            });
});

</script>
</head>

<body>
<div id="accordion">
<h3>Cadastro Cliente</h3>
<div>
<form method="post" id="form1" name="form1">


<div class="tela1">
      <br/> idCliente:<br/>
         <input type="text"    id="idCliente" name="idCliente"
            title="Entre com o Nome no Padrao"
required="required" readonly="readonly" />
</div>

  <br/><br/>
   Nome:<br/>
    <input type="text" placeholder="Entre com Nome" id="nome"
name="nome"
      title="Entre com o Nome no Padrao" required="required" />
  <br/>Email:<br/>
 <input type="email" placeholder="Entre com Email" id="email"
name="email"
    title="Entre com o Email no Padrao" required="required" />
 <br/>Data Nascimentol:<br/>
  <input type="date" placeholder="Entre com Data Nascimento"
id="data" name="data"
```

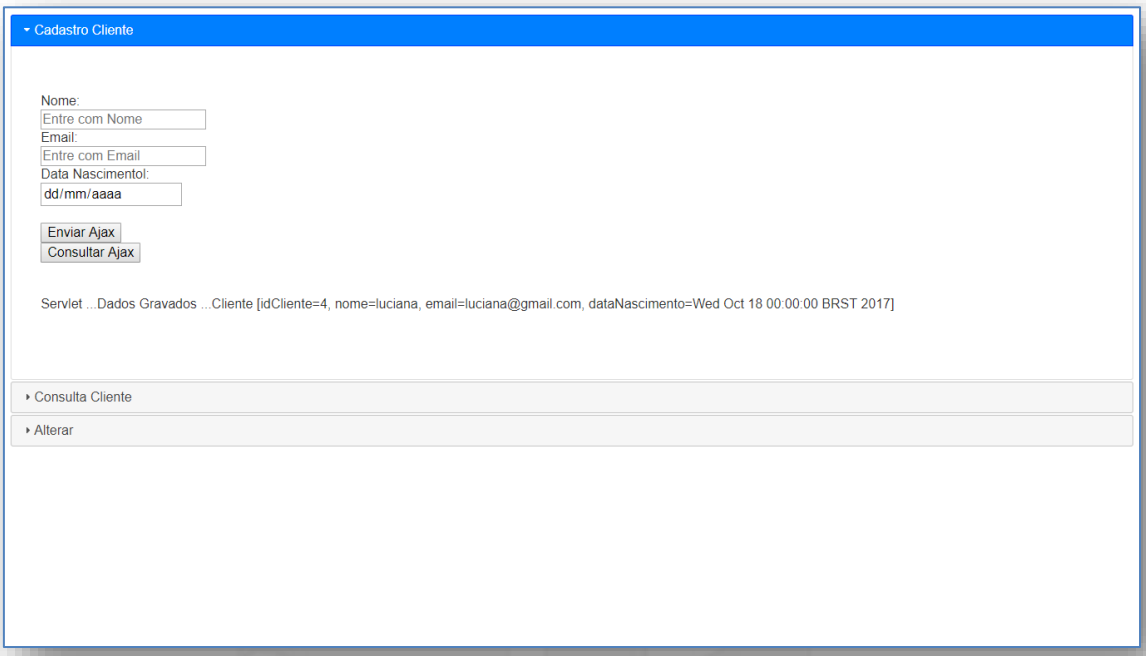
```
                title="Entre com a Data no Padrao"
required="required" />
<br/><br/>
  <button id="btn1" type="button">Enviar Ajax</button>
  <br/>
  <button id="btn2" type="button">Consultar Ajax</button>
</form>

<br/><br/>
 <div class="resposta"></div>
 <br/>
 </div>

 <h3>Consulta Cliente</h3>
 <div>


 <div class="tabela">Consulta dados</div>
  </div>

 <h3>Alterar</h3>
 <div>

 <div class="formulario">Formulario</div>

 </div>
</div>
</body>
</html>
```



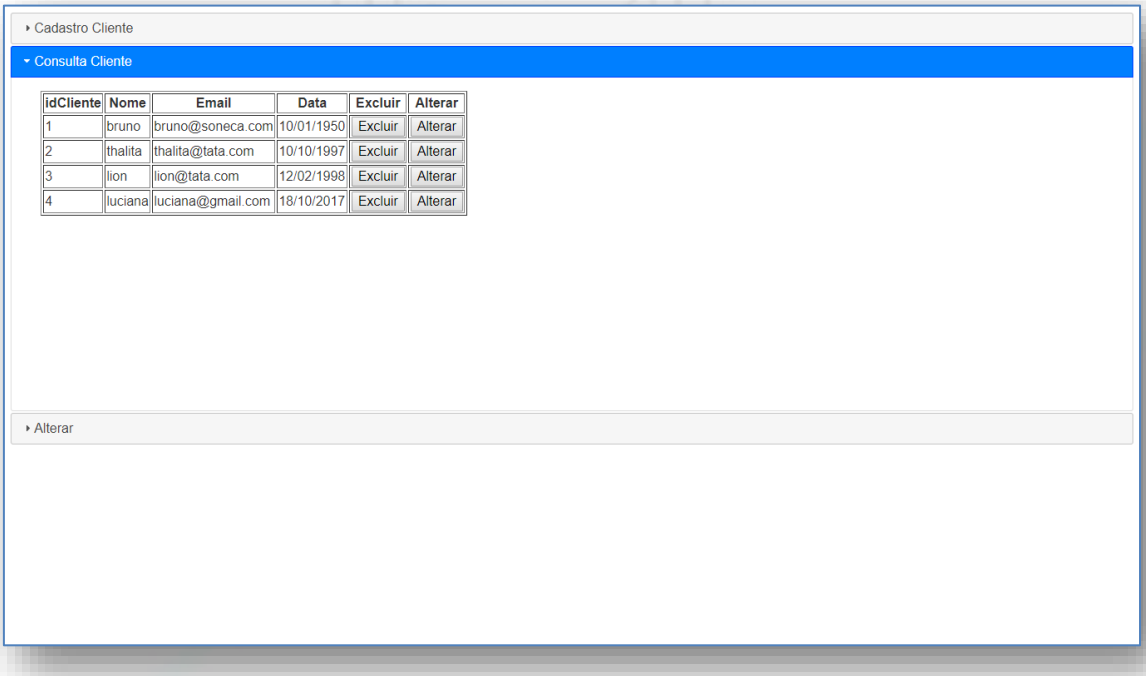
Cadastrando...



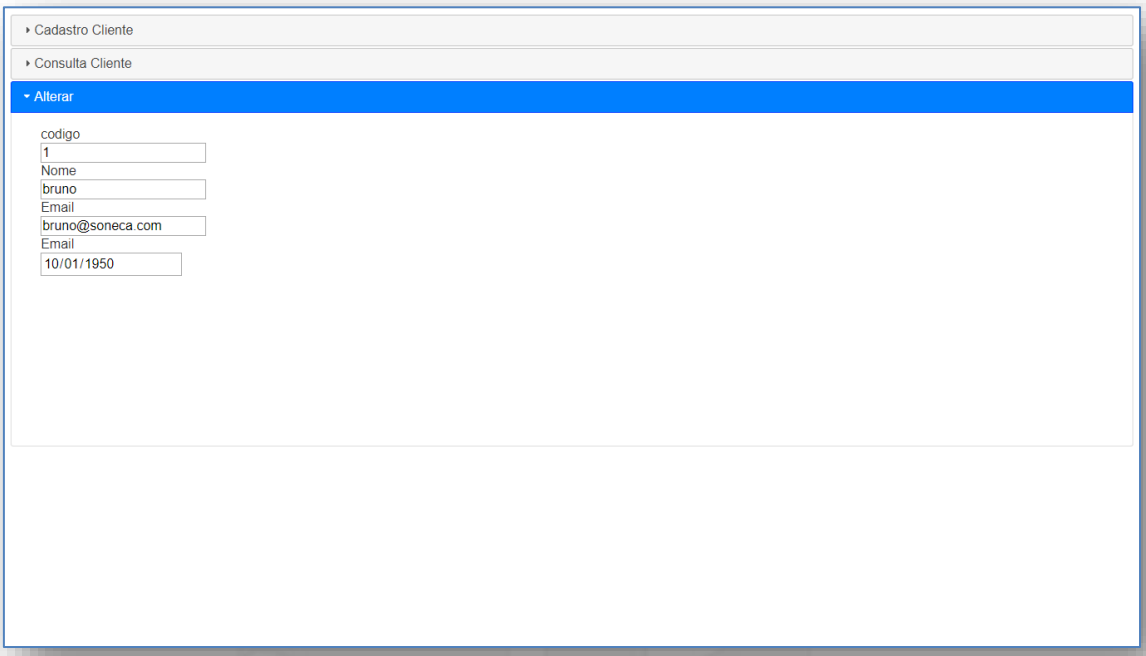
Clicando em ajax...



Clicando em consultar...

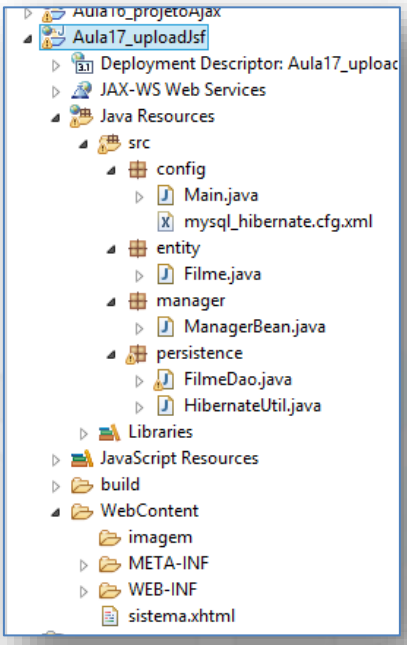


Clicando em alterar bruno...



Escola de Nerds
www.cotiinformatica.com.br

Estrutura do projeto:



LIBS:

```
all-themes-1.0.10.jar
antlr-2.7.6.jar
asm-attrs.jar
asm.jar
c3p0-0.9.0.jar
cglib-2.1.3.jar
commons-beanutils-1.8.0.jar
commons-codec-1.3.jar
commons-collections-3.2.1.jar
commons-digester-2.0.jar
commons-el-1.0.jar
commons-fileupload-1.0.jar
commons-fileupload-1.2.2.jar
commons-io-2.0.1.jar
commons-lang-2.1.jar
commons-logging-1.1.1.jar
commons-validator.jar
dom4j-1.6.1.jar
ejb3-persistence.jar
```

```
hibernate-annotations.jar
hibernate-commons-annotations.jar
hibernate3.jar
jakarta-oro.jar
javax.faces-2.0.11.jar
jstl.jar
jta.jar
log4j-1.2.15.jar
mysql-connector-java-5.1.23-bin.jar
oro-2.0.8.jar
primefaces-5.0-sources.jar
primefaces-5.0.jar
servlet3-api.jar
standard.jar
```

```
package entity;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="filme")
public class Filme implements Serializable {

	private static final long serialVersionUID = 1L;


	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	private Integer idFilme;
	@Column(length=50)
	private  String  nomeFilme;
	@Column(length=50)
	private  String   autor;
```

```
	@Column(length=50)
	private  String  caracteristica;
	@Column(length=255)
	private  String   descricao;
	@Column
	private String    imagem; //annabelle.jpg
	@Column
	private String    trailler; //www.youtube.com/ajsdhaisj

	public Filme() {
	}

	public Filme(Integer idFilme, String nomeFilme, String
autor, String caracteristica, String descricao,
			String imagem, String trailler) {
		this.idFilme = idFilme;
		this.nomeFilme = nomeFilme;
		this.autor = autor;
		this.caracteristica = caracteristica;
		this.descricao = descricao;
		this.imagem = imagem;
		this.trailler = trailler;
	}

	@Override
	public String toString() {
		return "Filme [idFilme=" + idFilme + ", nomeFilme=" +
nomeFilme + ", autor=" + autor + ", caracteristica="  +
caracteristica + ", descricao=" + descricao + ", imagem=" +
imagem + ", trailler=" + trailler + "]";
	}

	public Integer getIdFilme() {
		return idFilme;
	}
	public void setIdFilme(Integer idFilme) {
		this.idFilme = idFilme;
	}
	public String getNomeFilme() {
		return nomeFilme;
	}
	public void setNomeFilme(String nomeFilme) {
		this.nomeFilme = nomeFilme;
```

```
	}
	public String getAutor() {
		return autor;
	}
	public void setAutor(String autor) {
		this.autor = autor;
	}
	public String getCaracteristica() {
		return caracteristica;
	}
	public void setCaracteristica(String caracteristica) {
		this.caracteristica = caracteristica;
	}
	public String getDescricao() {
		return descricao;
	}
	public void setDescricao(String descricao) {
		this.descricao = descricao;
	}
	public String getImagem() {
		return imagem;
	}
	public void setImagem(String imagem) {
		this.imagem = imagem;
	}
	public String getTrailler() {
		return trailler;
	}
	public void setTrailler(String trailler) {
		this.trailler = trailler;
	}
}
```

---

mysql_hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
```

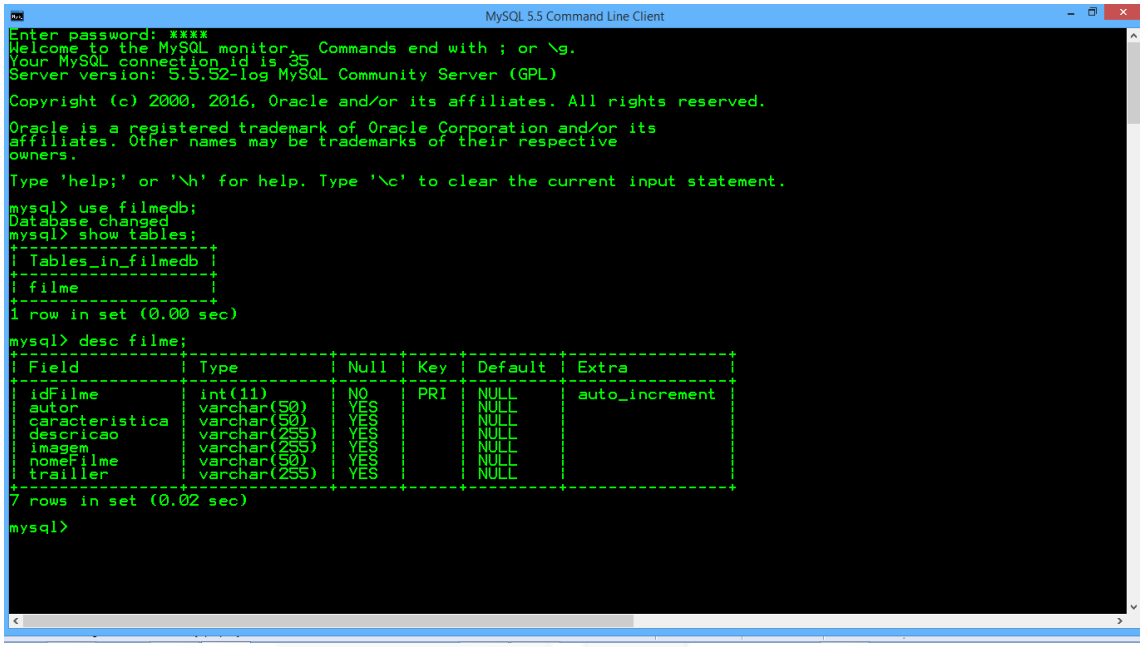
```
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</pro
perty>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</
property>
    <property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/filmedb?createDatabaseIfNotExist=tru
e</property>
    <property
name="hibernate.connection.username">root</property>
    <property
name="hibernate.connection.password">coti</property>

    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

    <mapping class="entity.Filme"/>

 </session-factory>
</hibernate-configuration>
```

```
package config;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.cfg.Configuration;
import org.hibernate.tool.hbm2ddl.SchemaExport;

public class Main {

    public static void main(String[] args) {
        Configuration cfg = new AnnotationConfiguration().
            configure("config/mysql_hibernate.cfg.xml");
        new SchemaExport(cfg).create(true,true);
        System.out.println("Tabelas Geradas ...");

       /*
        *     drop table if exists filme

    create table filme (
```

```
        idFilme integer not null auto_increment,
        autor varchar(50),
        caracteristica varchar(50),
        descricao varchar(255),
        imagem varchar(255),
        nomeFilme varchar(50),
        trailler varchar(255),
        primary key (idFilme)
    )
Tabelas Geradas ...
            *
            */
     }
}
```

No console...

```
log4j:WARN No appenders could be found for logger
(org.hibernate.cfg.annotations.Version).
log4j:WARN Please initialize the log4j system properly.

    drop table if exists filme

    create table filme (
        idFilme integer not null auto_increment,
        autor varchar(50),
        caracteristica varchar(50),
        descricao varchar(255),
        imagem varchar(255),
        nomeFilme varchar(50),
        trailler varchar(255),
        primary key (idFilme)
    )
Tabelas Geradas ...
```



```
package persistence;


import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;

//Abrir  e fechar o Banco de Dados

public class HibernateUtil {

    private static final SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new AnnotationConfiguration().
configure("config/mysql_hibernate.cfg.xml").buildSessionFactory(
);
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation
failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }
```

```
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

```
package persistence;

import java.util.List;

import org.hibernate.Criteria;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.Transaction;
import entity.Filme;

public class FilmeDao {

        Session      session;
    Transaction transaction;
        Query          query;
        Criteria        criteria;


        public void create(Filme f) throws Exception{
    session = HibernateUtil.getSessionFactory().openSession();
            transaction = session.beginTransaction();
                session.save(f);     //gravar
             transaction.commit();
          session.close();
        }

        public void update(Filme f) throws Exception{
    session = HibernateUtil.getSessionFactory().openSession();
            transaction = session.beginTransaction();
                 session.update(f);    //alterar
             transaction.commit();
          session.close();
        }

        public void delete(Filme f) throws Exception{
    session = HibernateUtil.getSessionFactory().openSession();
            transaction = session.beginTransaction();
```

```
                session.delete(f);    //excluir
            transaction.commit();
          session.close();
         }

         public List<Filme> findAll(){
      session = HibernateUtil.getSessionFactory().openSession();
            List<Filme> lista = session.createQuery("from
Filme").list();
          session.close();
          return lista;
         }

         public Filme findByCode(Integer cod){
      session = HibernateUtil.getSessionFactory().openSession();
           Filme filme =(Filme) session.get(Filme.class,cod);
          session.close();
          return filme;
         }
}
```

```
package manager;

import java.io.FileOutputStream;
import java.io.Serializable;
import java.util.List;

import javax.annotation.PostConstruct;
import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.ViewScoped;
import javax.faces.context.FacesContext;
import javax.servlet.ServletContext;

import org.primefaces.model.UploadedFile;

import entity.Filme;
import persistence.FilmeDao;

@ManagedBean(name="mb")
@ViewScoped
```

```
public class ManagerBean  implements Serializable{

      private static final long serialVersionUID = 1L;

        private Filme              filme;
        private UploadedFile arquivo;
        private List<Filme>   filmes;

        @PostConstruct
          public void init(){
            filme = new Filme();
        }

        public  void gravar(){
                 FacesContext fc =
FacesContext.getCurrentInstance();
                 try{
                        byte  arq[]= arquivo.getContents();
                        ServletContext context = (ServletContext)
fc.getExternalContext().getContext();
                   String path= context.getRealPath("/imagem");
             System.out.println("Caminho real  (Onde):" + path);
                        //ele grava como arquivo txt
filme.setImagem(filme.getNomeFilme()+".jpg");
    FileOutputStream fos = new FileOutputStream(path + "/" +
filme.getImagem());
                         fos.write(arq); //escreve byte a byte
                         fos.close();
                         new FilmeDao().create(filme);
                         filme = new Filme();
    fc.addMessage(null, new FacesMessage("Dados Gravados ..."));
                 }catch(Exception ex){
                fc.addMessage(null, new FacesMessage("Error:" +
ex.getMessage()));
                 }
        }

      public Filme getFilme() {
            return filme;
      }
      public void setFilme(Filme filme) {
```

```
            this.filme = filme;
      }
      public UploadedFile getArquivo() {
            return arquivo;
      }
      public void setArquivo(UploadedFile arquivo) {
            this.arquivo = arquivo;
      }
      public List<Filme> getFilmes() {
            filmes = new FilmeDao().findAll();
            return filmes;
      }
      public void setFilmes(List<Filme> filmes) {
            this.filmes = filmes;
      }
}
```

---

faces-config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<faces-config
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-facesconfig_2_2.xsd"
    version="2.2">

</faces-config>
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
version="3.1">
  <display-name>uploadJsf</display-name>
  <servlet>
```

```
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>


  <filter>
      <filter-name>PrimeFaces FileUpload Filter</filter-name>
      <filter-
class>org.primefaces.webapp.filter.FileUploadFilter</filter-
class>
  </filter>

  <filter-mapping>
      <filter-name>PrimeFaces FileUpload Filter</filter-name>
      <servlet-name>Faces Servlet</servlet-name>
  </filter-mapping>

</web-app>
```

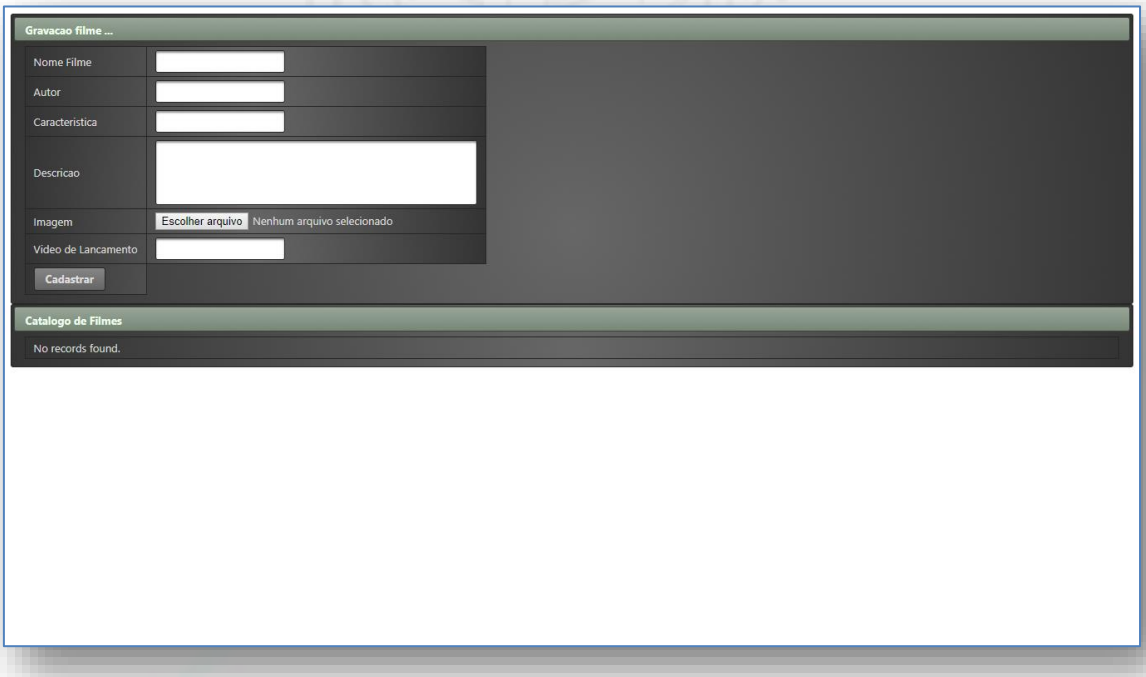
---

sistema.xhtml

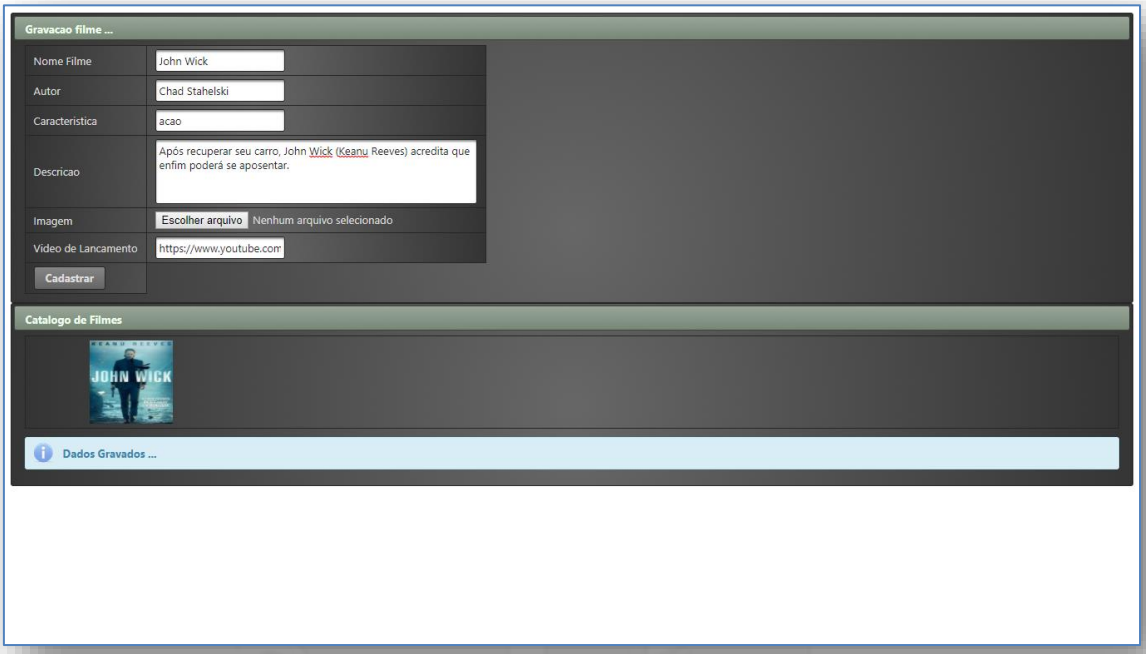
```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head>
</h:head>

<h:body>
      <p:panel header="Gravacao filme ...">
        <h:form id="form1"  enctype="multipart/form-data">
              <p:panelGrid columns="2">
```

```
                    <h:outputText value="Nome Filme" />
                    <p:inputText value="#{mb.filme.nomeFilme}" />
                    <h:outputText value="Autor" />
                    <p:inputText value="#{mb.filme.autor}" />
                    <h:outputText value="Caracteristica" />
                    <p:inputText
value="#{mb.filme.caracteristica}" />
                   <h:outputText value="Descricao" />
                    <p:inputTextarea rows="4" cols="60"
autoResize="true"
                     value="#{mb.filme.descricao}"
                    />
                    <h:outputText value="Imagem" />
              <p:fileUpload value="#{mb.arquivo}"  mode="simple">
                    </p:fileUpload>

                    <h:outputText value="Video de Lancamento" />
                    <p:inputText value="#{mb.filme.trailler}"  >
                    </p:inputText>

                  <p:commandButton value="Cadastrar" ajax="false"
                    update=":form1" action="#{mb.gravar}"
                    ></p:commandButton>

               </p:panelGrid>
             </h:form>
       </p:panel>

 <p:panel header="Catalogo de Filmes">
  <h:form id="form2">
  <p:dataGrid value="#{mb.filmes}" var="linha" columns="2">
    <h:panelGroup>
     <p:commandLink update=":form3"
                 onclick="PF('janela').show()">
      <p:graphicImage value="/imagem/#{linha.imagem}"
       style="width:100px;height:100px" >
      </p:graphicImage>
      <f:setPropertyActionListener value="#{linha}"
                                   target="#{mb.filme}">
     </f:setPropertyActionListener>
    </p:commandLink>

    </h:panelGroup>
```

```
  </p:dataGrid>
  </h:form>
 </p:panel>

  <p:dialog widgetVar="janela" modal="true" position="center
top"
    resizable="false" draggable="false" >
  <h:form id="form3">
    <p:panelGrid columns="2">
     <f:facet name="header">
        <p:graphicImage  value="/imagem/#{mb.filme.imagem}">
        </p:graphicImage>
     </f:facet>
      <h:outputText value="nomeFilme" />
      <p:outputLabel value="#{mb.filme.nomeFilme}" />
          <h:outputText value="Vi-deo" />
      <p:media value="#{mb.filme.trailler}" player="flash" />
     </p:panelGrid>
   </h:form>
 </p:dialog>
</h:body>
</html>
```

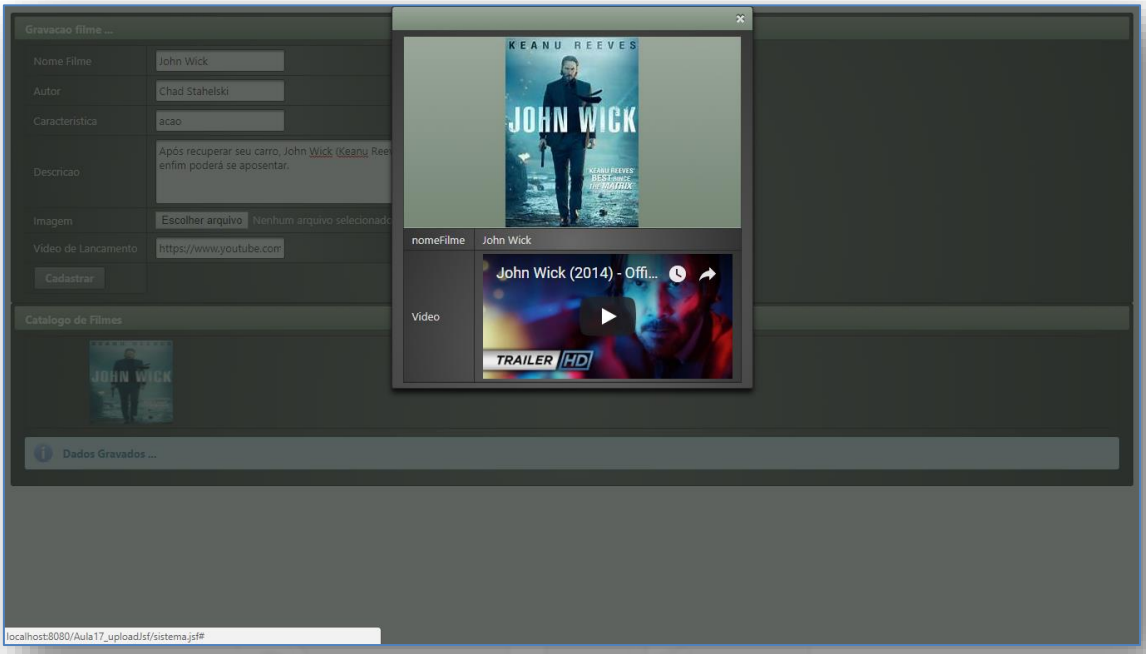


Gravando um filme…



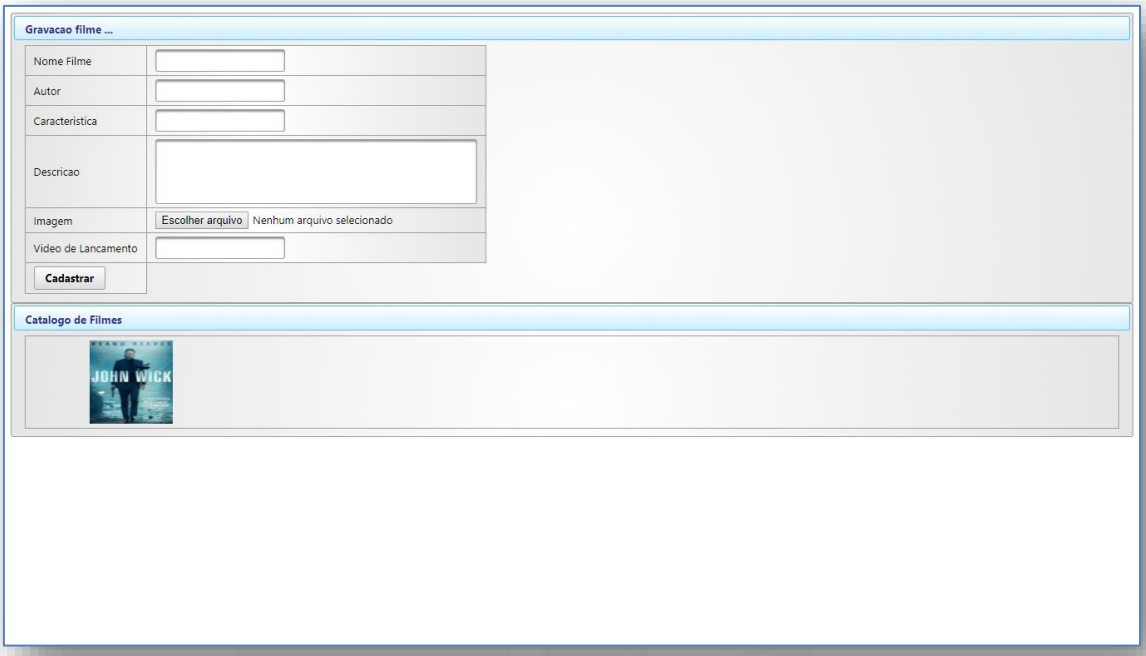
Gravado...

Clicando na imagem do filme…

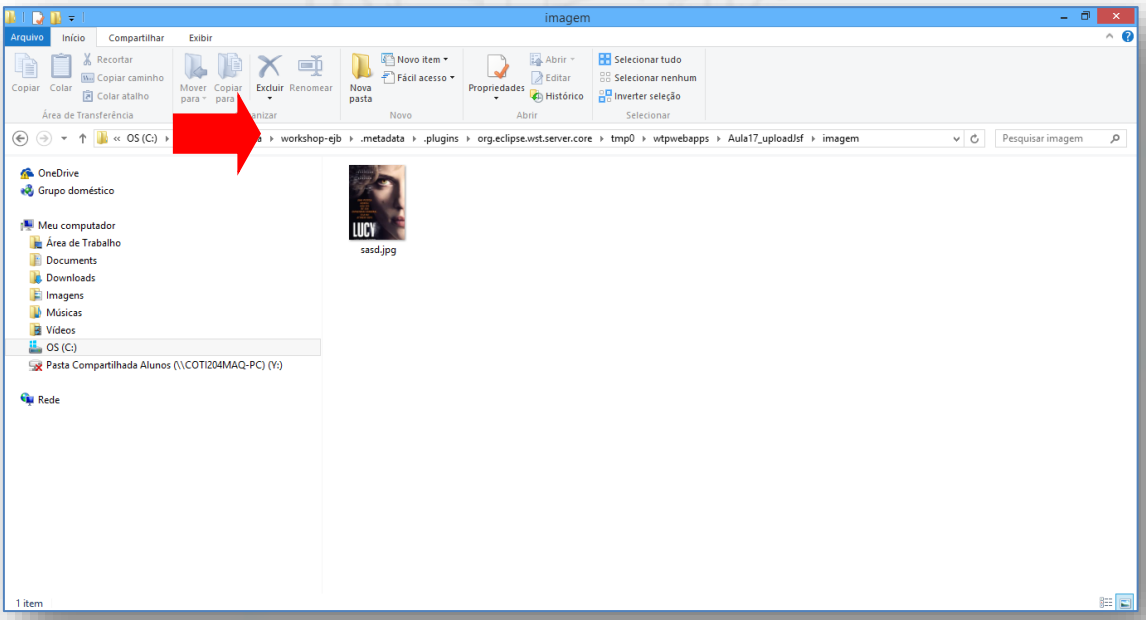


trocando o tema...

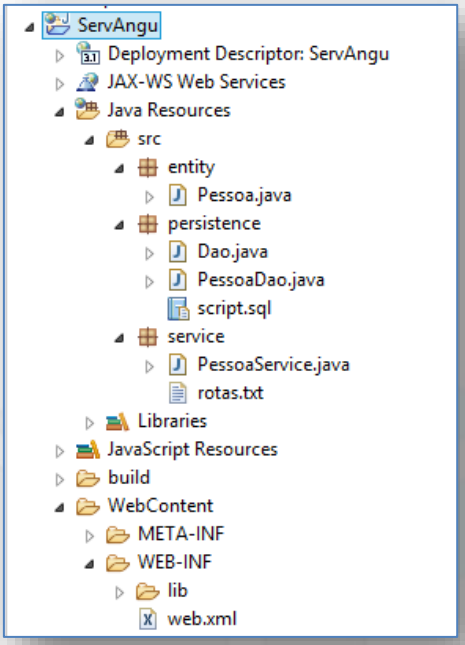
```
<context-param>
    <param-name>primefaces.THEME</param-name>
    <param-value>afternoon</param-value>
</context-param>
```



Endereço da imgem gravado...



Estrutura do projeto:



LIBS:

```
asm-3.1.jar
commons-codec-1.6.jar
commons-logging-1.1.1.jar
fluent-hc-4.2.5.jar
gson-2.2.4.jar
httpclient-4.2.5.jar
httpclient-cache-4.2.5.jar
httpcore-4.2.4.jar
httpmime-4.2.5.jar
jackson-core-asl-1.9.2.jar
jackson-jaxrs-1.9.2.jar
jackson-mapper-asl-1.9.2.jar
jackson-xc-1.9.2.jar
jaxen-1.1.6.jar
jdom-2.0.5.jar
jersey-client-1.12.jar
jersey-core-1.12.jar
jersey-json-1.12.jar
jersey-server-1.12.jar
jersey-servlet-1.12.jar
```

```
jettison-1.1.jar
jsr311-api-1.1.1.jar
mysql-connector-java-5.1.23-bin.jar
xercesImpl.jar
xml-apis.jar
```

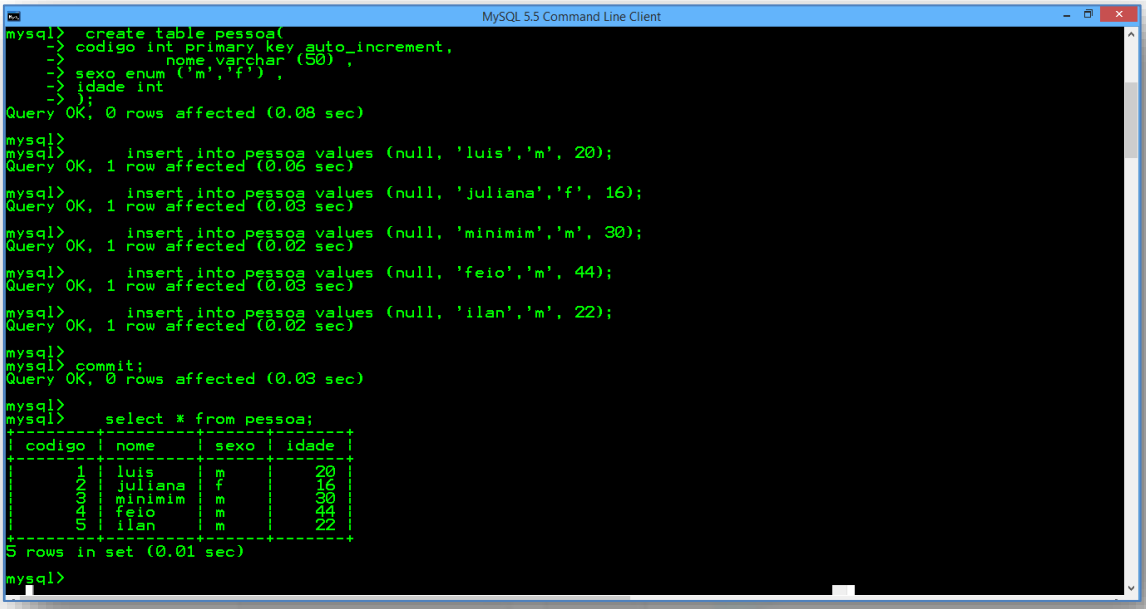
```
package entity;

public class Pessoa {

	private  Integer codigo;
	private  String   nome;
	private String    sexo;
	private Integer  idade;

	public Pessoa(Integer codigo, String nome, String sexo,
Integer idade) {
		this.codigo = codigo;
		this.nome = nome;
		this.sexo = sexo;
		this.idade = idade;
	}

	public Pessoa() {
	}

	@Override
	public String toString() {
		return "Pessoa [codigo=" + codigo + ", nome=" + nome
+ ", sexo=" + sexo + ", idade=" + idade + "]";
	}

	public Integer getCodigo() {
		return codigo;
	}
	public void setCodigo(Integer codigo) {
		this.codigo = codigo;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
```

```
	}
	public String getSexo() {
		return sexo;
	}
	public void setSexo(String sexo) {
		this.sexo = sexo;
	}
	public Integer getIdade() {
		return idade;
	}
	public void setIdade(Integer idade) {
		this.idade = idade;
	}
}
```

---

script.sql

```
drop database angung;

create database angung;

use angung;

 create table pessoa(
	codigo int primary key auto_increment,
	nome varchar (50) ,
	sexo enum ('m','f') ,
	idade int
	);

	insert into pessoa values (null, 'luis','m', 20);
	insert into pessoa values (null, 'juliana','f', 16);
	insert into pessoa values (null, 'minimim','m', 30);
	insert into pessoa values (null, 'feio','m', 44);
	insert into pessoa values (null, 'ilan','m', 22);

commit;

   select * from pessoa;
```




```
package persistence;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Dao {

   Connection con;
   PreparedStatement stmt;
   ResultSet rs;

      public void open() throws Exception{
         Class.forName("com.mysql.jdbc.Driver");
          con = DriverManager.
getConnection("jdbc:mysql://localhost:3306/angung","root","coti"
);
      }

      public void close() throws Exception{
        con.close();
      }
```

```
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;
import entity.Pessoa;

public class PessoaDao  extends Dao{

	public void create(Pessoa p) throws Exception{
		open();
		  stmt = con.prepareStatement("insert into pessoa
values (null,?,?,?)");
		    stmt.setString(1, p.getNome());
		    stmt.setString(2, p.getSexo());
		    stmt.setInt(3 ,  p.getIdade());
		   stmt.execute();
		close();
	 }

	public List<Pessoa> findAll() throws Exception{
		open();
	    stmt = con.prepareStatement("select * from pessoa");
		    rs = stmt.executeQuery();
		    List<Pessoa> lst = new ArrayList<Pessoa>();
		    while(rs.next()){
		     Pessoa p = new Pessoa(rs.getInt(1),
rs.getString(2),
rs.getString(3),
rs.getInt(4));
		     lst.add(p);
		    }
		close();
		return lst;
	}

	public Pessoa findByCode(Integer cod) throws Exception{
		open();
		  stmt = con.prepareStatement("select * from pessoa
where codigo = ?");
		  stmt.setInt(1, cod);
		    rs = stmt.executeQuery();
```

```
                    Pessoa p = null;
                    if(rs.next()){
                     p = new Pessoa(rs.getInt(1),
                         rs.getString(2),
                         rs.getString(3),
                        rs.getInt(4));
                    }
                close();
                return p;
    }
       public void delete(Integer cod) throws Exception{
                open();
                  stmt = con.prepareStatement("delete from pessoa
where codigo=?");
                   stmt.setInt(1,cod);
                  stmt.execute();
                close();
        }
}
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <display-name>ServAngu</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>

    <servlet>
      <servlet-name>Jersey REST Service</servlet-name>
      <servlet-
class>com.sun.jersey.spi.container.servlet.ServletContainer</ser
vlet-class>
```

```
      <init-param>
           <param-
name>com.sun.jersey.config.property.packages</param-name>
```

Atenção para o nome do pacote...

```
           <param-value>service</param-value>
      </init-param>
      <load-on-startup>1</load-on-startup>
  </servlet>

  <servlet-mapping>
      <servlet-name>Jersey REST Service</servlet-name>
      <url-pattern>/json/*</url-pattern>
  </servlet-mapping>


</web-app>
```

```
package service;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;

import com.google.gson.Gson;

import entity.Pessoa;
import persistence.PessoaDao;

@Path("/pessoa")
public class PessoaService {

      //webService vem como String
      @GET
      @Path("/gravar/{nome}/{sexo}/{idade}")
       @Produces("text/plain")
       public String gravar(@PathParam("nome")String nome,
                            @PathParam("sexo" )String sexo,
                            @PathParam("idade") String idade
                                       ){
            try{
```

```
                Pessoa p= new Pessoa (null, nome, sexo, new
Integer(idade));
                new PessoaDao().create(p);
                  return "Dados Gravados ...";
               }catch(Exception ex){
                     return "Error : " + ex.getMessage();
               }
        }

       @GET
       @Path("/listar")
        @Produces("Application/json")
        public String listar(){
              try{
       return  new Gson().toJson(new PessoaDao().findAll()) ;
              }catch(Exception ex){
                     return "Error : " + ex.getMessage();
              }
        }

       @GET
       @Path("/buscarCodigo/{codigo}")
        @Produces("application/json")
        public String buscarCodigo(@PathParam("codigo")String
codigo){
              try{
             return  new Gson().toJson(new
PessoaDao().findByCode(new Integer(codigo))) ;
              }catch(Exception ex){
                     return "Error : " + ex.getMessage();
              }
        }

       @GET
       @Path("/excluir/{codigo}")
        @Produces("text/plain")
        public String excluir(@PathParam("codigo") String codigo){
              try{
                  new PessoaDao().delete(new Integer(codigo));
                  return "Registro Excluido pelo Codigo";
              }catch(Exception ex){
                     return "Error : " + ex.getMessage();
              }
        }
```

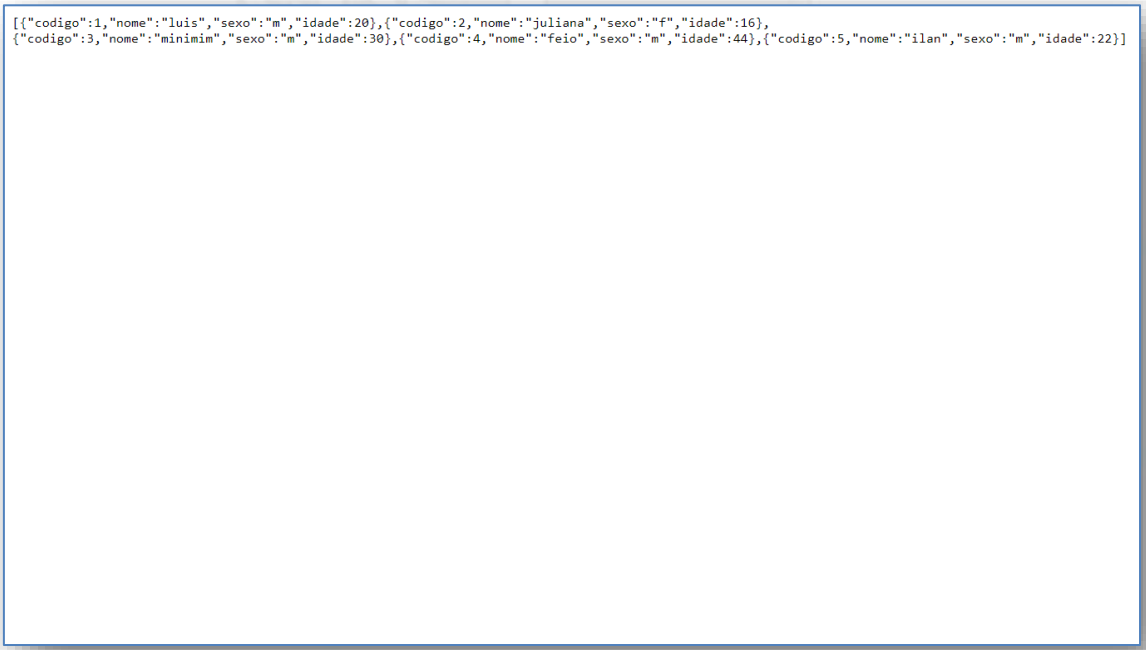
```
}
```

Rotas.txt

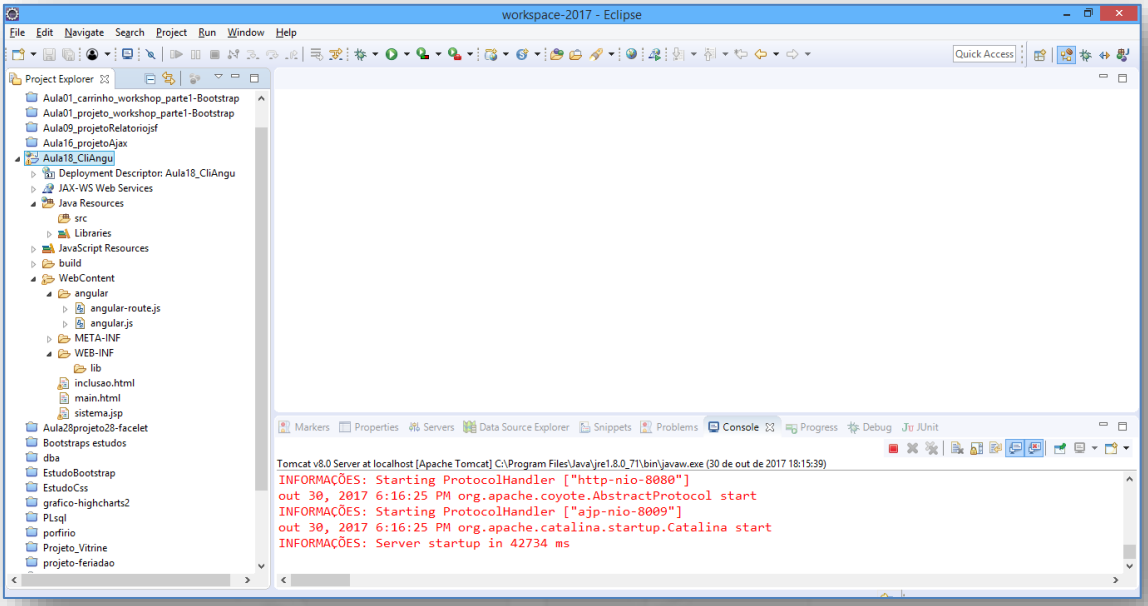
```
listar = http://localhost:8080/ServAngu/json/pessoa/listar
gravar
=http://localhost:8080/ServAngu/json/pessoa/gravar/gustavo/m/22
buscarCodigo=http://localhost:8080/ServAngu/json/pessoa/buscarCo
digo/3
excluir = http://localhost:8080/ServAngu/json/pessoa/excluir/8
```

Rodando o projeto...

http://localhost:8080/ServAngu/json/pessoa/listar



NOVO PROJETO:
Estrutura do projeto:



LIBS:

angular-route.js
angular.js

---

sistema.jsp

```
<!DOCTYPE html>
<html ng-app="app">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.4/angular.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.6.4/angular-route.js"></script>
</head>
<body>
   <p/>
     <ul>
```

```
            <li><a href="#/!">main</a></li>
            <li><a href="#!inclusao">inclusao</a></li>
            <li><a href="#!listar">listar</a></li>
        </ul>

<br/><br/>Respostas :<br/>
 <hr/>

<div  ng-view></div>
</body>
<script>
   var app= angular.module('app',["ngRoute"]);

   app.config(function ($routeProvider){
                    $routeProvider
           .when("/", {
                templateUrl :    "main.html" ,
           })
           .when("/inclusao", {
              templateUrl : "inclusao.html" ,
              controller :  "gravarctl"
           })
      .when("/listar", {
              templateUrl : "listar.html" ,
              controller :  "listarctl"
           });
  });

  app.controller("gravarctl", function($scope, $http){
        //essa é a classe
            $scope.model={ codigo : '',      nome : '',   sexo :
'',  idade : ''      };
              $scope.cadastrar = function(){

$http.get("http://localhost:8080/ServAngu/json/pessoa/gravar/" +
              $scope.model.nome + "/" +$scope.model.sexo + "/"+
$scope.model.idade
                        )
                  .success(function(msg){
                        $scope.message = msg;
                       $scope.model.nome ='';
                       $scope.model.idade ='';
                       $scope.model.sexo ='';
                  });
                 //limpar
```

```
            }
  }) ;
  app.controller("listarctl", function($scope, $http){
$http.get("http://localhost:8080/ServAngu/json/pessoa/listar"
                        )
                .success(function(dados){
                      $scope.listagem= dados;
                });
  }) ;
</script>

</html>
```

---

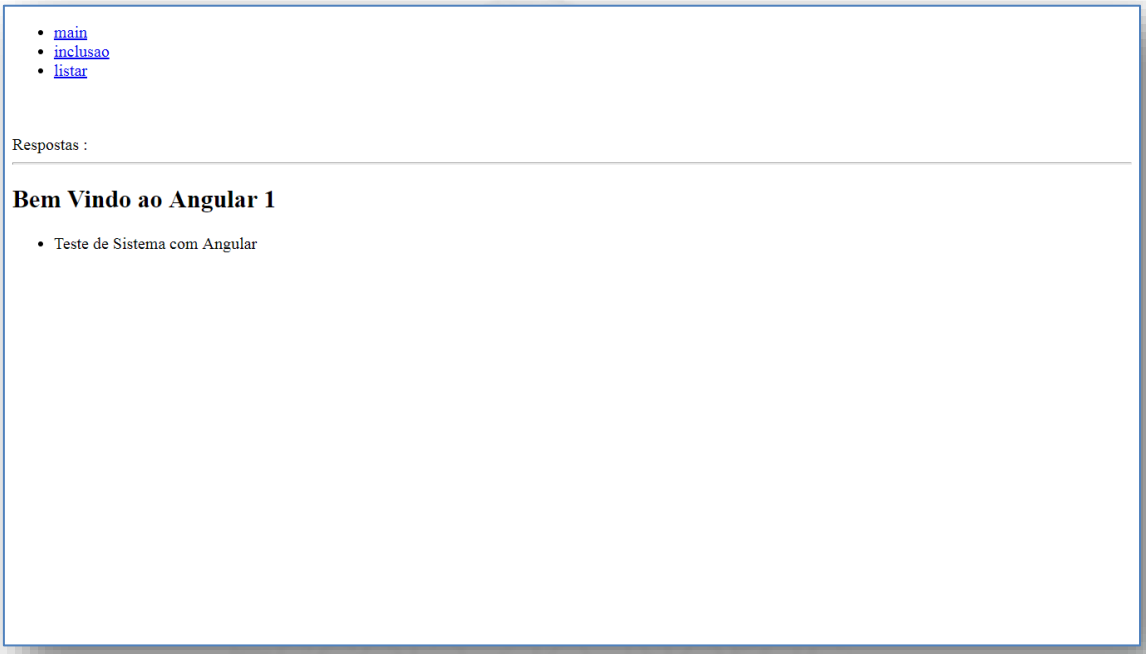
inclusao.html

```
<div>
 <h3>Cadastro com NG</h3>
 <form ng-controller="gravarctl">
     Nome<br/>
     <input type="text" ng-model="model.nome"
required="required" />
    <br/>Sexo<br/>
     <select ng-model="model.sexo"  required="required">
         <option value="m">Masculino</option>
          <option value="f">Feminino</option>
     </select>
    <br/>Idade<br/>
     <input type="number" ng-model="model.idade"
required="required" />
     <button type="button" ng-
click="cadastrar()">Cadastrar</button>
     <br/><br/>
           {{message}}
 </form>
</div>
```

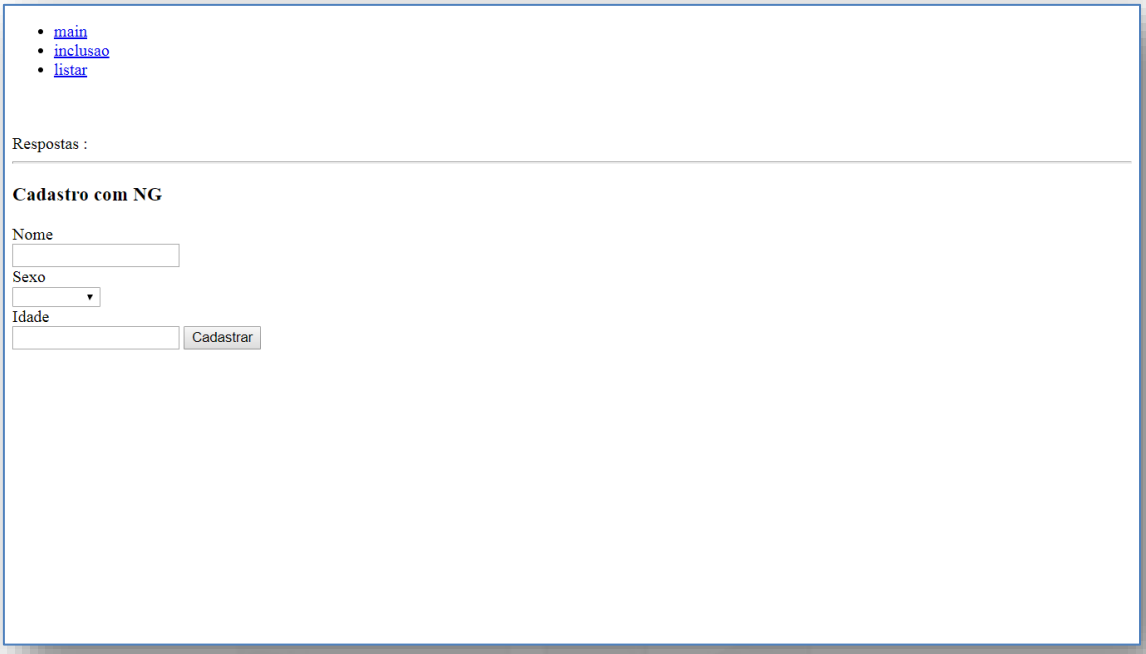
---

main.html

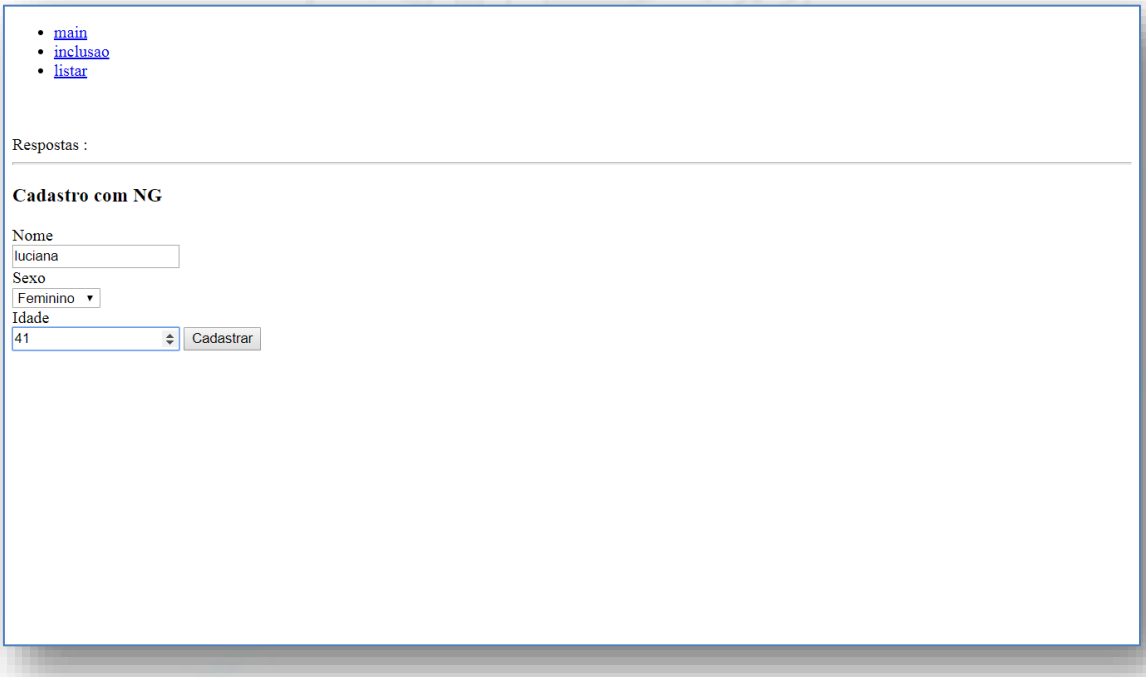
```
<div>
<h2>Bem Vindo ao Angular 1</h2>
<P/>
<ul>
     <li>Teste  de Sistema com Angular</li>
</ul>
</div>
```



Clicando em inclusão...

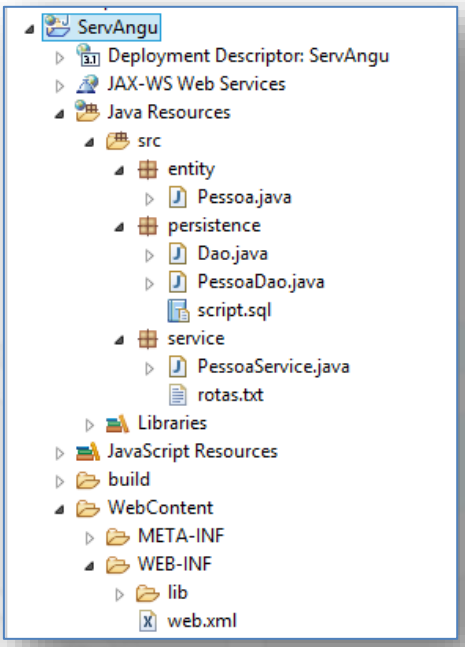


sCadastrando...



PROJETO SERVER:

Estrutura do projeto:



LIBS:

```
asm-3.1.jar
commons-codec-1.6.jar
commons-logging-1.1.1.jar
fluent-hc-4.2.5.jar
gson-2.2.4.jar
httpclient-4.2.5.jar
httpclient-cache-4.2.5.jar
httpcore-4.2.4.jar
httpmime-4.2.5.jar
jackson-core-asl-1.9.2.jar
jackson-jaxrs-1.9.2.jar
jackson-mapper-asl-1.9.2.jar
jackson-xc-1.9.2.jar
jaxen-1.1.6.jar
jdom-2.0.5.jar
jersey-client-1.12.jar
jersey-core-1.12.jar
jersey-json-1.12.jar
```

```
jersey-server-1.12.jar
jersey-servlet-1.12.jar
jettison-1.1.jar
jsr311-api-1.1.1.jar
mysql-connector-java-5.1.23-bin.jar
xercesImpl.jar
xml-apis.jar
```

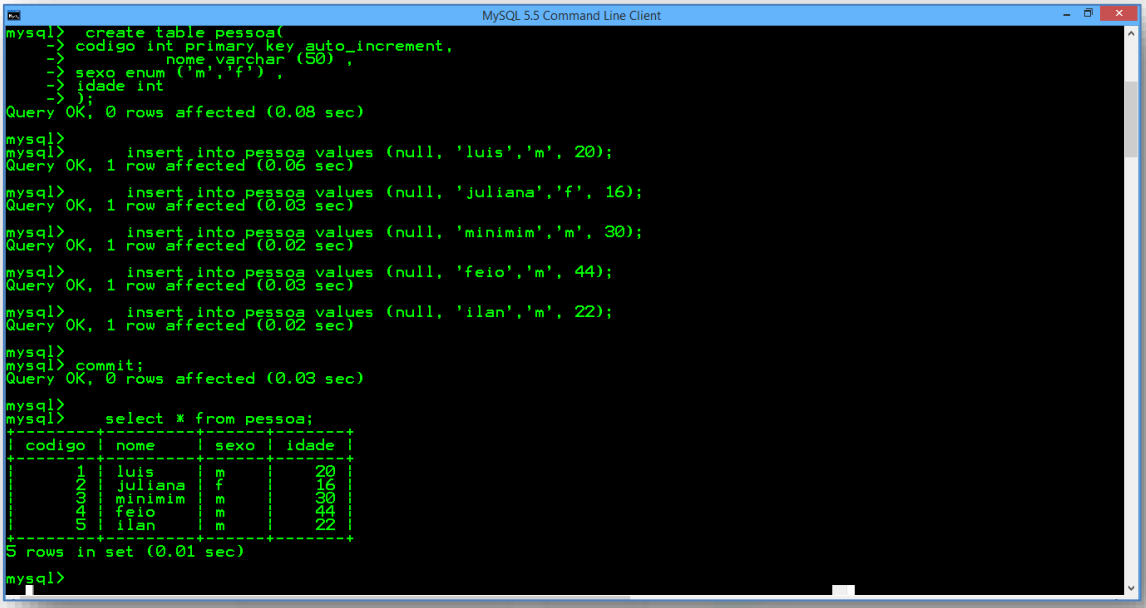
```
package entity;

public class Pessoa {

	private  Integer codigo;
	private  String   nome;
	private String    sexo;
	private Integer  idade;

	public Pessoa(Integer codigo, String nome, String sexo,
Integer idade) {
		this.codigo = codigo;
		this.nome = nome;
		this.sexo = sexo;
		this.idade = idade;
	}

	public Pessoa() {
	}

	@Override
	public String toString() {
		return "Pessoa [codigo=" + codigo + ", nome=" + nome
+ ", sexo=" + sexo + ", idade=" + idade + "]";
	}

	public Integer getCodigo() {
		return codigo;
	}
	public void setCodigo(Integer codigo) {
		this.codigo = codigo;
	}
	public String getNome() {
		return nome;
	}
```

```
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getSexo() {
		return sexo;
	}
	public void setSexo(String sexo) {
		this.sexo = sexo;
	}
	public Integer getIdade() {
		return idade;
	}
	public void setIdade(Integer idade) {
		this.idade = idade;
	}
}
```

---

script.sql

```
drop database angung;

create database angung;

use angung;

 create table pessoa(
	codigo int primary key auto_increment,
	nome varchar (50) ,
	sexo enum ('m','f') ,
	idade int
	);

	insert into pessoa values (null, 'luis','m', 20);
	insert into pessoa values (null, 'juliana','f', 16);
	insert into pessoa values (null, 'minimim','m', 30);
	insert into pessoa values (null, 'feio','m', 44);
	insert into pessoa values (null, 'ilan','m', 22);

commit;

   select * from pessoa;
```




```
package persistence;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Dao {

   Connection con;
   PreparedStatement stmt;
   ResultSet rs;

      public void open() throws Exception{
        Class.forName("com.mysql.jdbc.Driver");
         con = DriverManager.
getConnection("jdbc:mysql://localhost:3306/angung","root","coti"
);
      }

      public void close() throws Exception{
       con.close();
      }
```

```
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;
import entity.Pessoa;

public class PessoaDao  extends Dao{

      public void create(Pessoa p) throws Exception{
              open();
                stmt = con.prepareStatement("insert into pessoa
values (null,?,?,?)");
                  stmt.setString(1, p.getNome());
                  stmt.setString(2, p.getSexo());
                  stmt.setInt(3 ,  p.getIdade());
                 stmt.execute();
              close();
       }

      public List<Pessoa> findAll() throws Exception{
              open();
           stmt = con.prepareStatement("select * from pessoa");
                  rs = stmt.executeQuery();
                  List<Pessoa> lst = new ArrayList<Pessoa>();
                  while(rs.next()){
                    Pessoa p = new Pessoa(rs.getInt(1),
rs.getString(2),
rs.getString(3),
rs.getInt(4));
                    lst.add(p);
                  }
              close();
              return lst;
      }

      public Pessoa findByCode(Integer cod) throws Exception{
              open();
                stmt = con.prepareStatement("select * from pessoa
where codigo = ?");
                stmt.setInt(1, cod);
                   rs = stmt.executeQuery();
```

```
                    Pessoa p = null;
                    if(rs.next()){
                     p = new Pessoa(rs.getInt(1),
                         rs.getString(2),
                         rs.getString(3),
                        rs.getInt(4));
                    }
                close();
                return p;
    }
       public void delete(Integer cod) throws Exception{
                open();
                  stmt = con.prepareStatement("delete from pessoa
where codigo=?");
                   stmt.setInt(1,cod);
                  stmt.execute();
                close();
        }
}
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <display-name>ServAngu</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>

    <servlet>
      <servlet-name>Jersey REST Service</servlet-name>
      <servlet-
class>com.sun.jersey.spi.container.servlet.ServletContainer</ser
vlet-class>
```

```
        <init-param>
            <param-
name>com.sun.jersey.config.property.packages</param-name>
```

Atenção para o nome do pacote...

```
            <param-value>service</param-value>
        </init-param>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>Jersey REST Service</servlet-name>
        <url-pattern>/json/*</url-pattern>
    </servlet-mapping>


</web-app>
```

```
package service;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;

import com.google.gson.Gson;

import entity.Pessoa;
import persistence.PessoaDao;

@Path("/pessoa")
public class PessoaService {

    //webService vem como String
    @GET
    @Path("/gravar/{nome}/{sexo}/{idade}")
     @Produces("text/plain")
     public String gravar(@PathParam("nome")String nome,
                          @PathParam("sexo" )String sexo,
                          @PathParam("idade") String idade
                                  ){
          try{
```

```
                Pessoa p= new Pessoa (null, nome, sexo, new
Integer(idade));
                new PessoaDao().create(p);
                  return "Dados Gravados ...";
               }catch(Exception ex){
                     return "Error : " + ex.getMessage();
               }
        }

       @GET
       @Path("/listar")
        @Produces("Application/json")
        public String listar(){
              try{
       return  new Gson().toJson(new PessoaDao().findAll()) ;
              }catch(Exception ex){
                     return "Error : " + ex.getMessage();
              }
        }

       @GET
       @Path("/buscarCodigo/{codigo}")
        @Produces("application/json")
        public String buscarCodigo(@PathParam("codigo")String
codigo){
              try{
             return  new Gson().toJson(new
PessoaDao().findByCode(new Integer(codigo))) ;
              }catch(Exception ex){
                     return "Error : " + ex.getMessage();
              }
        }

       @GET
       @Path("/excluir/{codigo}")
        @Produces("text/plain")
        public String excluir(@PathParam("codigo") String codigo){
              try{
                  new PessoaDao().delete(new Integer(codigo));
                  return "Registro Excluido pelo Codigo";
              }catch(Exception ex){
                     return "Error : " + ex.getMessage();
              }
        }
```

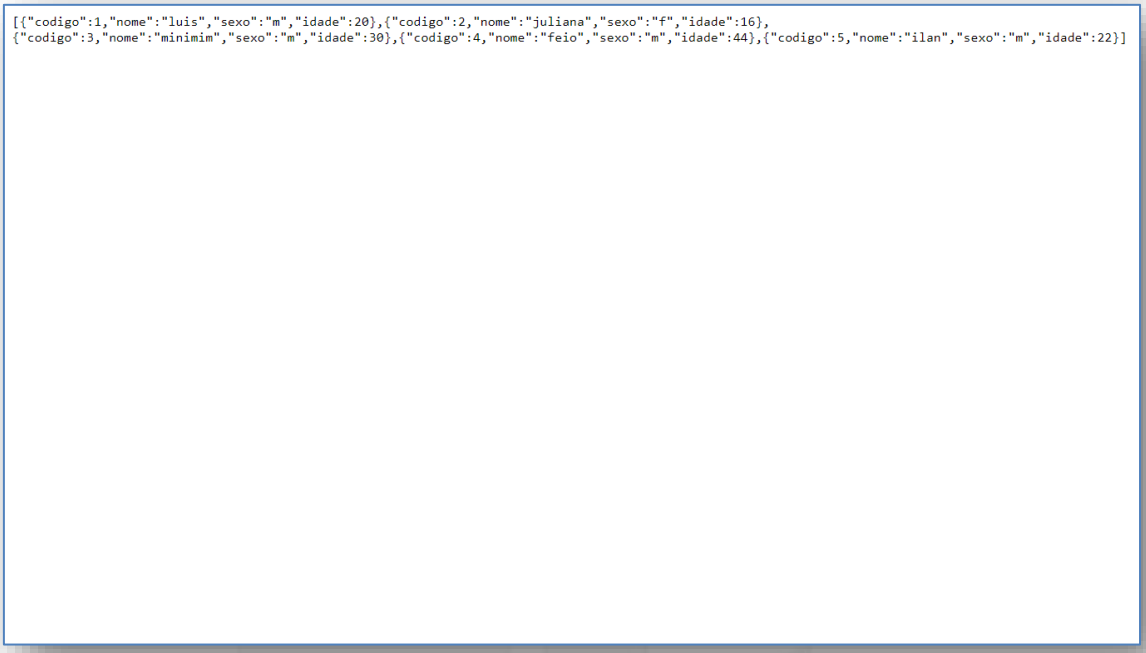
```
        @GET
        @Path("/alterar/{nome}/{sexo}/{idade}/{codigo}")
        @Produces("text/plain")
        public String alterar(
                  @PathParam("nome")String nome,
                  @PathParam("sexo" )String sexo,
                  @PathParam("idade") String idade,
                  @PathParam("codigo") String codigo
                                    ){
              try{
    Pessoa p= new Pessoa (new Integer(codigo),
                          nome, sexo, new Integer(idade));
               new PessoaDao().update(p);
                 return "Dados Alterados ...";
              }catch(Exception ex){
                    return "Error : " + ex.getMessage();
              }
        }
}
```

Rotas.txt

listar = http://localhost:8080/ServAngu/json/pessoa/listar
gravar = http://localhost:8080/ServAngu/json/pessoa/gravar/lu/f/20
buscarCodigo=http://localhost:8080/ServAngu/json/pessoa/buscarCodigo/3
excluir = http://localhost:8080/ServAngu/json/pessoa/excluir/3
alterar=http://localhost:8080/ServAngu/json/pessoa/alterar/gilson/m/21/9

Rodando o projeto...

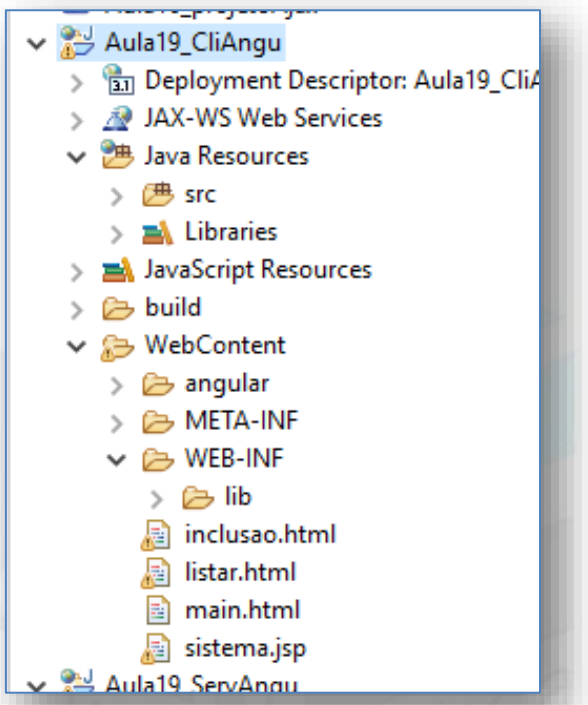
http://localhost:8080/ServAngu/json/pessoa/listar



Escola de Nerds
www.cotiinformatica.com.br

NOVO PROJETO:

Estrutura do projeto:



LIBS:

angular-route.js
angular.js

---

sistema.jsp

```
<!DOCTYPE html>
<html ng-app="app">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.4/angular.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.6.4/angular-route.js"></script>
```

```
</head>
<body>
   <p/>
     <ul>
          <li><a href="#/!">main</a></li>
          <li><a href="#!inclusao">inclusao</a></li>
          <li><a href="#!listar">listar</a></li>
     </ul>

<br/><br/>Respostas :<br/>
 <hr/>
<div  ng-view>
</div>

</body>

<script>
   var app= angular.module('app',["ngRoute"]);

   app.config(function ($routeProvider){
                          $routeProvider
          .when("/", {
              templateUrl :    "main.html" ,
          })
          .when("/inclusao", {
             templateUrl : "inclusao.html" ,
             controller :  "gravarctl"
          })
     .when("/listar", {
             templateUrl : "listar.html" ,
             controller :  "listarctl"
          });
  });

  app.controller("gravarctl", function($scope, $http){
       //essa é a classe
     $scope.model={ codigo : '',   nome : '',   sexo : '',
idade : ''     };
       $scope.cadastrar = function(){
$http.get("http://localhost:8080/ServAngu/json/pessoa/gravar/" +
        $scope.model.nome + "/" +$scope.model.sexo + "/"+
$scope.model.idade
                     )
                     .then(function(msg){
                       $scope.message = msg.data;
```

```
                }).catch(function(msg){
                        $scope.message = msg.data;
                      });

          //area de limpaeza dos Campos ...
                $scope.model.nome="";
                $scope.model.sexo="";
                $scope.model.idade="";
             }
  });

  app.controller("listarctl", function($scope, $http){
        $scope.consultar = function(){
$http.get("http://localhost:8080/ServAngu/json/pessoa/listar"
                      )
                .then(function(dados){
                        $scope.listagem= dados.data;
                });
         }

        $scope.excluir = function(cod){
              var resp = window.confirm('excluir');
              if (resp==true){
$http.get("http://localhost:8080/ServAngu/json/pessoa/excluir/"
               + cod
                   )
             .then(function(dados){
                   $scope.listagem= dados.data;
             });
          window.location.reload();
              }
     }

        $scope.alterar = function(pessoa){
$http.get("http://localhost:8080/ServAngu/json/pessoa/alterar/"
              +pessoa.nome + "/" +     pessoa.sexo + "/"+
              pessoa.idade + "/" +     pessoa.codigo
                   )
             .then(function(dados){
                   $scope.listagem= dados.data;
             });
          window.location.reload();
              }
  });
</script>
```

```
</html>
```

---

main.html

```
<div>
<h2>Bem Vindo ao Angular 1b</h2>
<P/>
<ul>
     <li>Teste  de Sistema com Angular Aula dois</li>
</ul>
</div>
```

---

inclusão.html

```
<div>
 <h3>Cadastro com NG</h3>
 <form ng-controller="gravarctl">

     Nome<br/>
   <input type="text"  ng-model="model.nome" required="required"
    title="Digite o Nome" pattern="[a-z A-Z]{2,50}"
   />
    <br/>Sexo<br/>
     <select ng-model="model.sexo"  required="required"
      title="Escolha o Sexo"
     >
          <option value="m">Masculino</option>
          <option value="f">Feminino</option>
     </select>
    <br/>Idade<br/>
     <input type="number" ng-model="model.idade"
required="required"
     title="Digite a Idade"
     />
  <button type="button" ng-
click="cadastrar()">Cadastrar</button>
     <br/><br/>
          {{message}}
 </form>
</div>
```

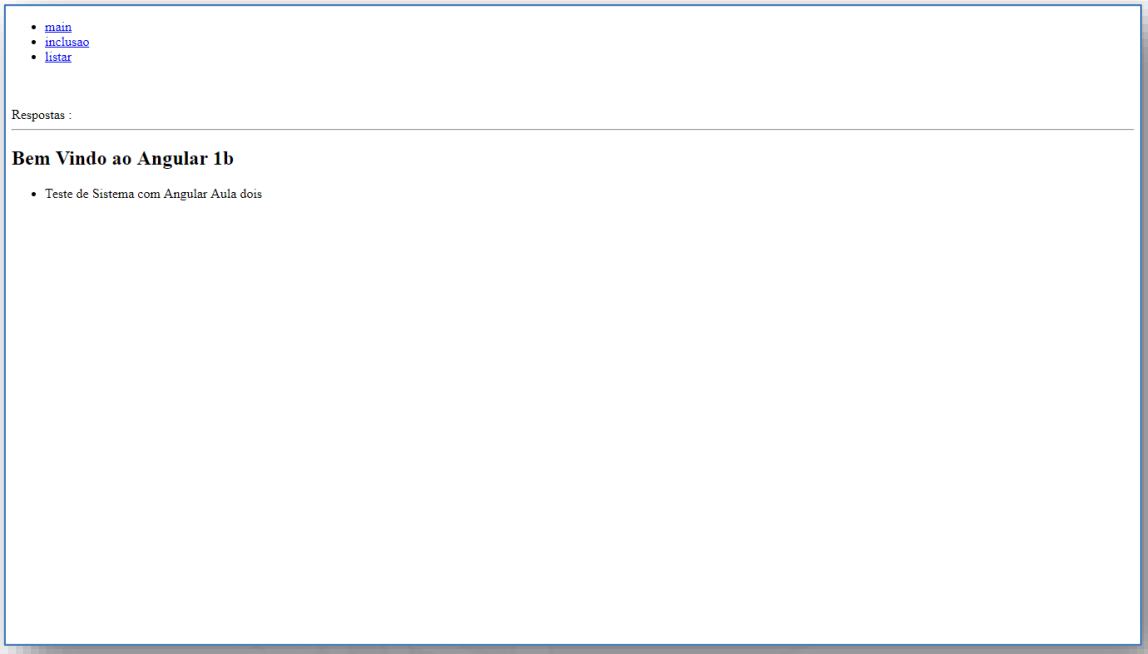
---

listar.html

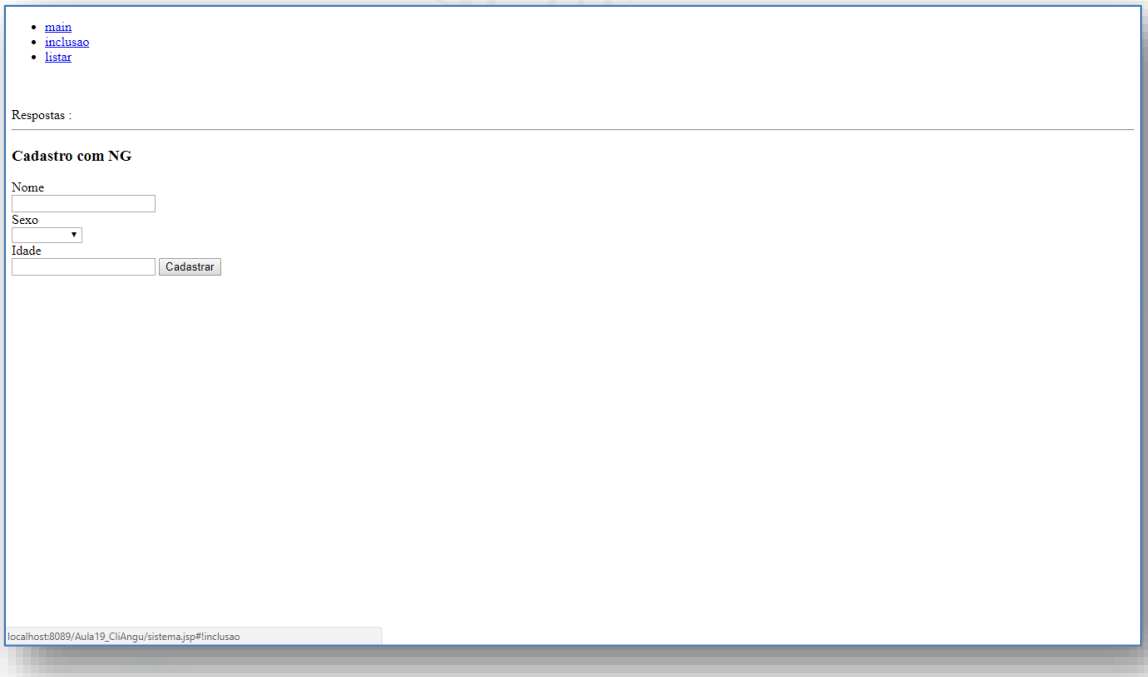
```
<div>
<form ng-controller="listarctl">
{{mensagem}}<p/>
<table ng-init="consultar()" border="1">
 <thead>
   <tr>
       <td>Codigo</td>
       <td>Nome</td>
       <td>Sexo</td>
       <td>Idade</td>
       <td>Exluir</td>
       <td>Alterar</td>
   </tr>
 </thead>
<tbody>
 <tr ng-repeat="p in listagem">
    <td>{{p.codigo}}</td>
    <td><input type="text"   id="nome" ng-model="p.nome"
size="10"></td>
    <td>
      <select ng-model="p.sexo">
           <option value="m">Masculino</option>
           <option value="f">Feminino</option>
    </select>
    </td>
    <td><input type="number" id="idade" ng-model="p.idade"
size="5"></td>
<td><button type="button" ng-
click="excluir(p.codigo)">Excluir</button></td>
<td><button type="button" ng-
click="alterar(p)">Alterar</button></td>

</tr>
</tbody>
<tfoot>
   <tr>
      <td colspan="4"> Registros :{{listagem.length}}
   </tr>
</tfoot>
</table>

</form>
```

```
</div>
```

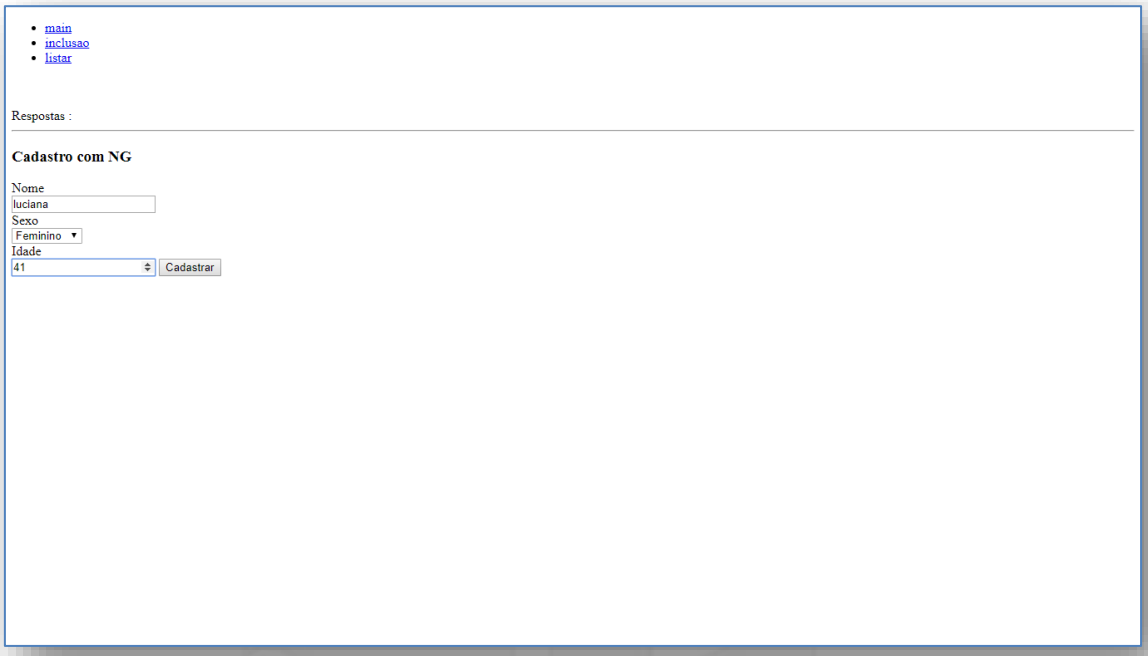
Rodando pelo sistema... Já abre mostrando o main....



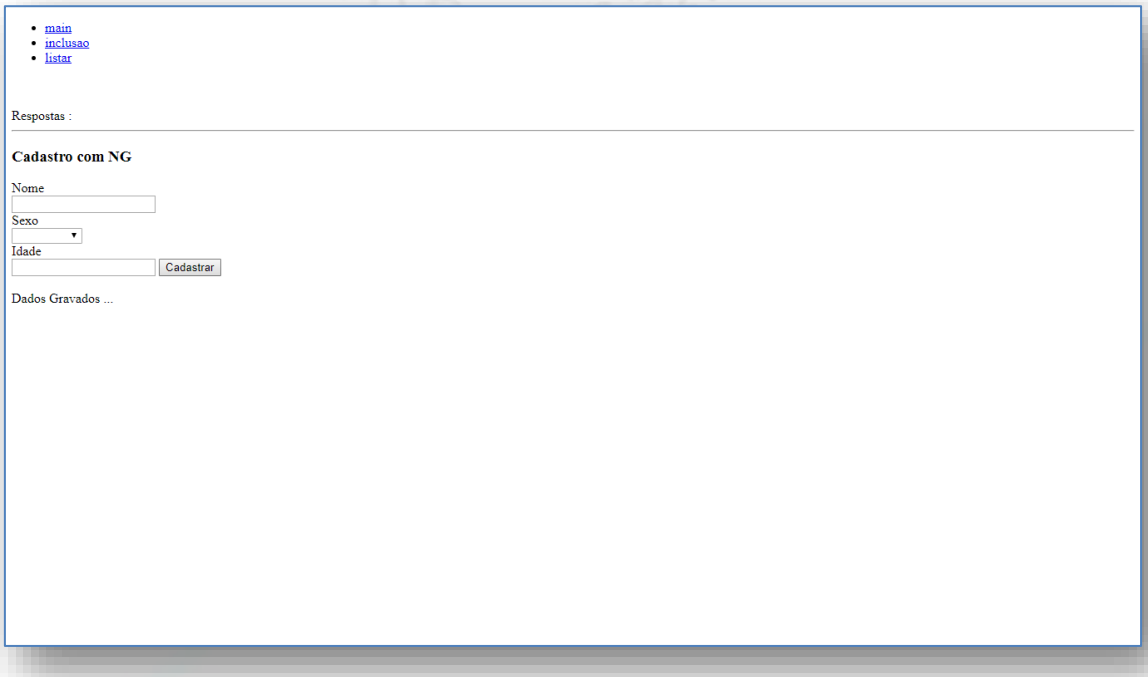
Clicando em inclusão...



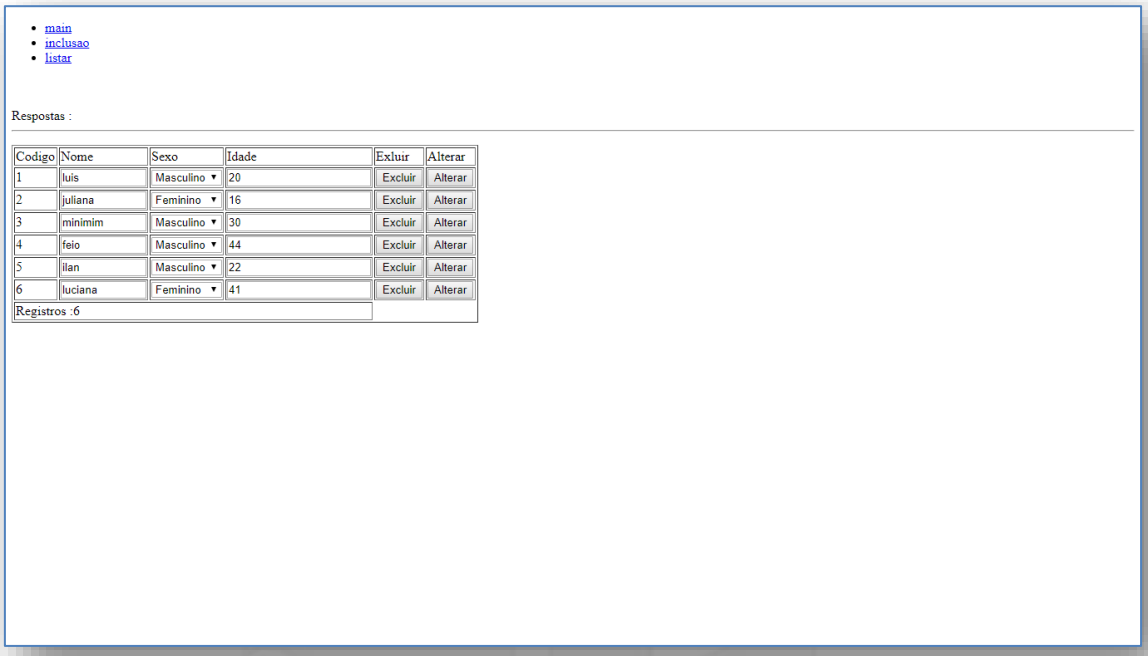
Cadastrando...



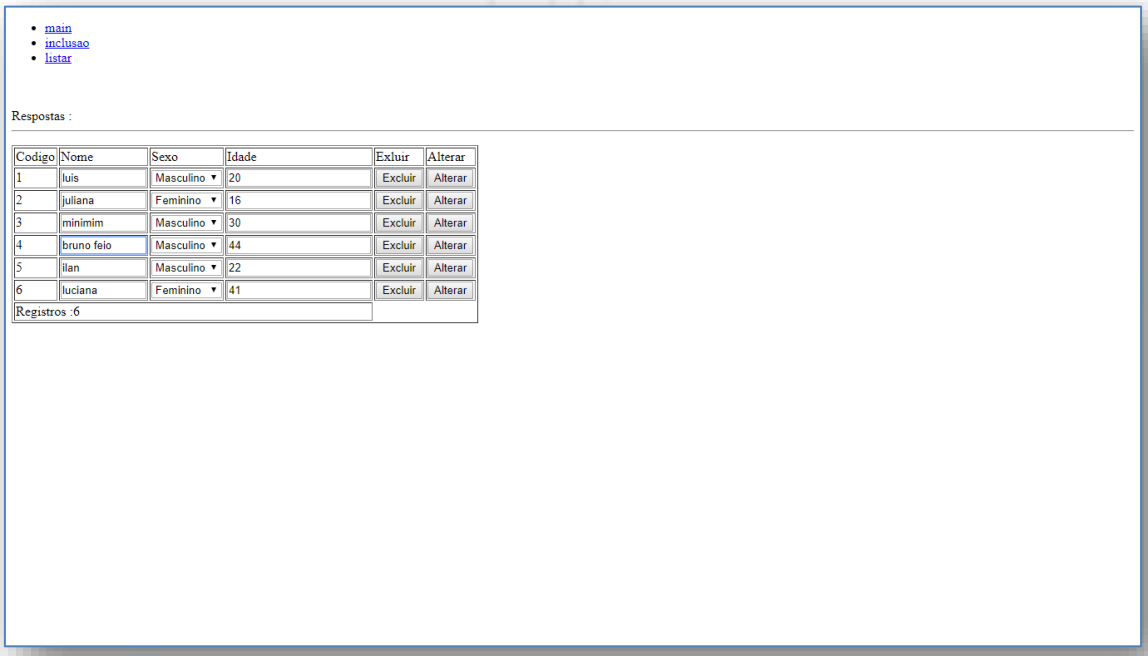
Gravado...



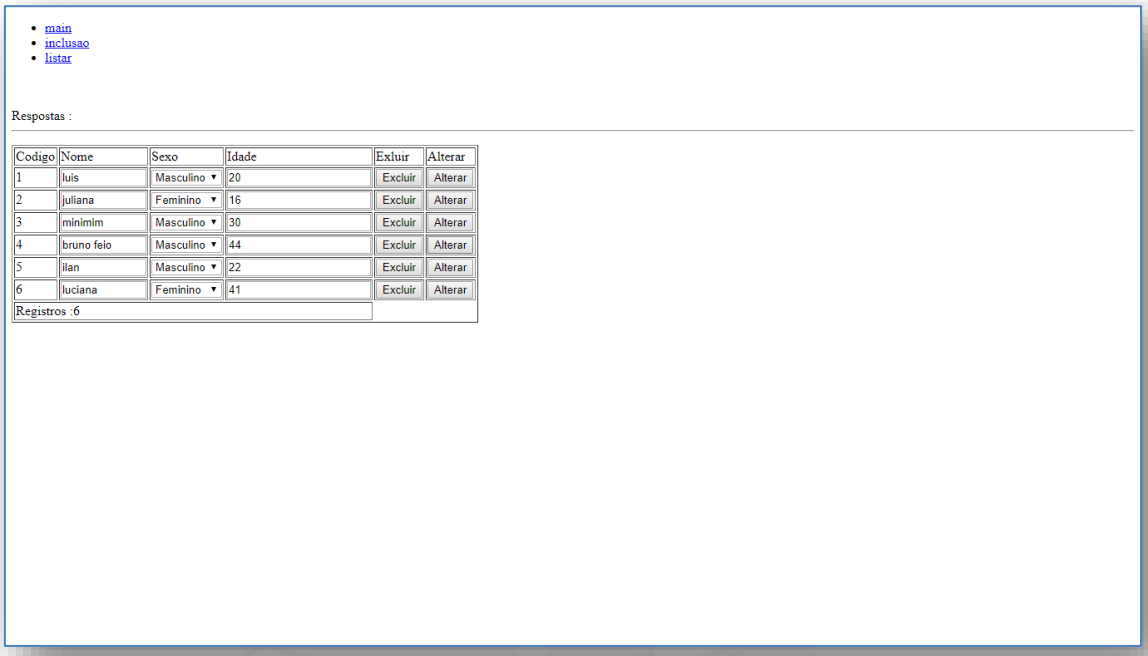
Clicando em listar...



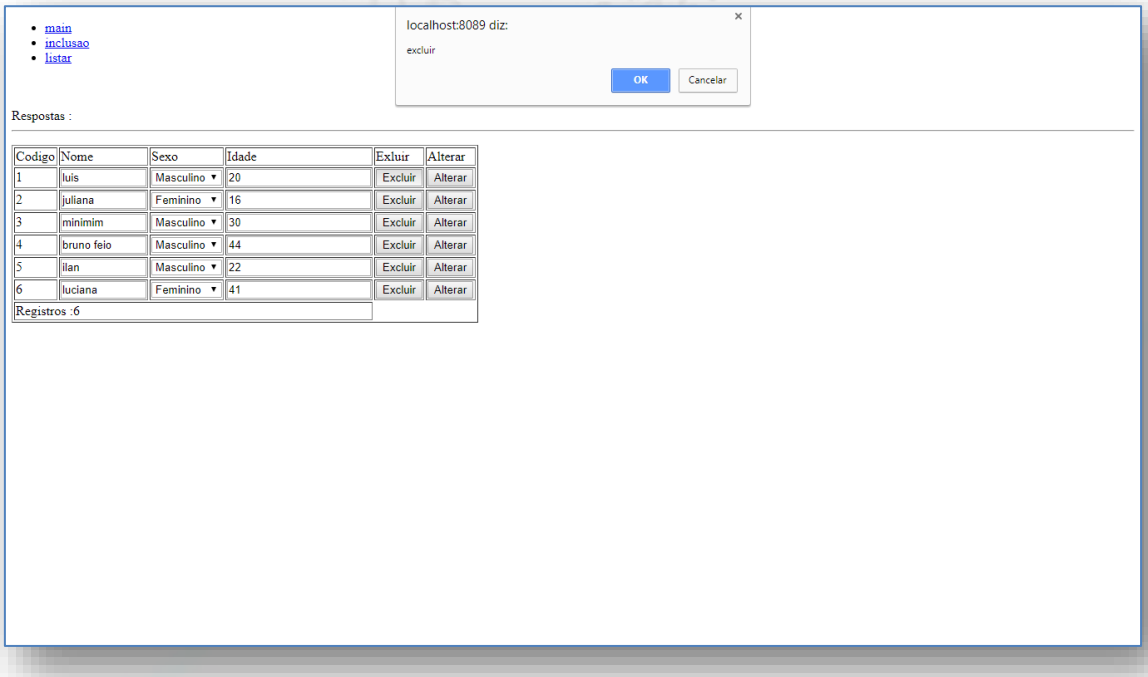
Para alterar, digitar a alteração na linha e depois clicar no botão alterar... alterando o numero 4...



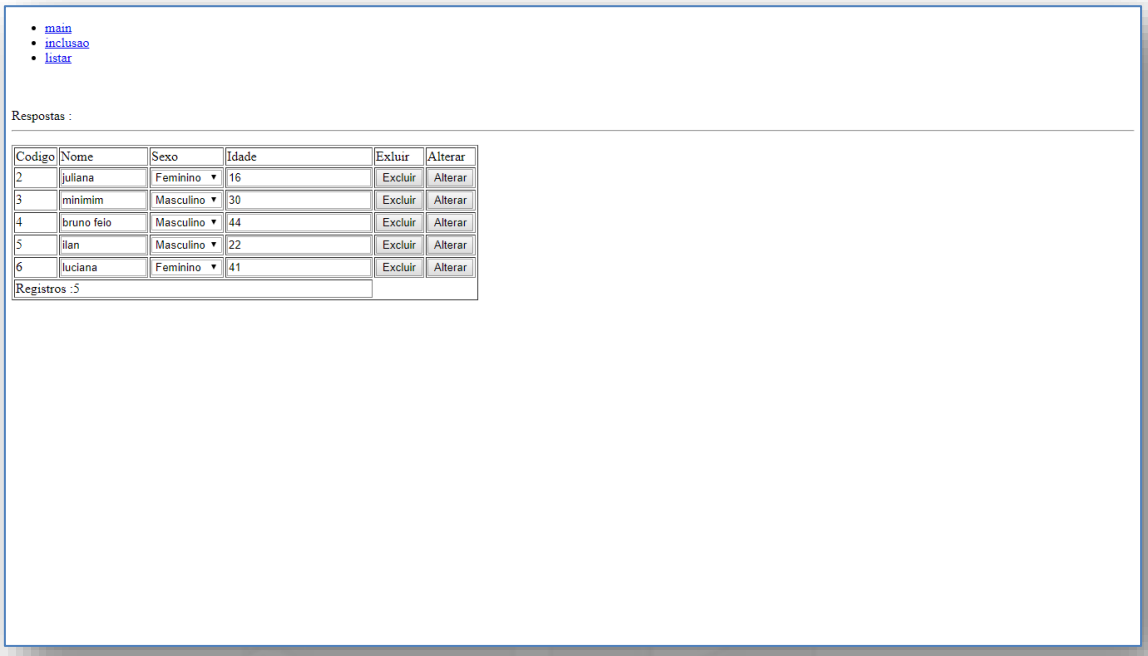
Alterado...



Excluindo luis...



Excluído...


main
inclusao
listar
Respostas :
Codigo Nome Sexo Idade Exluir Alterar
2 juliana Feminino 16 Excluir Alterar
3 minimim Masculino 30 Excluir Alterar
4 bruno feio Masculino 44 Excluir Alterar
5 ilan Masculino 22 Excluir Alterar
6 luciana Feminino 41 Excluir Alterar
Registros :5

O MESMO PROJETO COM BOOTSWATCH...

sistema.jsp

```
<!DOCTYPE html>
<html ng-app="app">
<head>
<script
      src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.4/
angular.min.js"></script>
<script
      src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.6.
4/angular-route.js"></script>

<link rel="stylesheet"
      href="https://maxcdn.bootstrapcdn.com/bootswatch/3.3.7/flat
ly/bootstrap.min.css">
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/font-
awesome/4.7.0/css/font-awesome.min.css">

<script
      src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jqu
ery.min.js"></script>
<script
      src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/boo
tstrap.min.js"></script>
</head>

<body>

<nav class="navbar navbar-default">
  <div class="container-fluid">
    <div class="navbar-header">
      <a class="navbar-brand" href="#">Webservice Angular</a>
    </div>
    <ul class="nav navbar-nav">
     <li class="active"><a href="#/!">Main</a></li>
      <li><a href="#!inclusao">Inclusao</a></li>
      <li><a href="#!listar">Listar</a></li>
    </ul>
  </div>
</nav>
```

```
      <div class="col-md-10 col-md-offset-1">
            <div ng-view></div>
      </div>
</body>

<script>
   var app= angular.module('app',["ngRoute"]);

   app.config(function ($routeProvider){
                          $routeProvider

           .when("/", {
                templateUrl :    "main.html" ,
           })
           .when("/inclusao", {
              templateUrl : "inclusao.html" ,
              controller :  "gravarctl"
           })
     .when("/listar", {
              templateUrl : "listar.html" ,
              controller :  "listarctl"
           });
  });


  app.controller("gravarctl", function($scope, $http){
       //essa é a classe
     $scope.model={ codigo : '',   nome : '',   sexo : '',
idade : ''     };

       $scope.cadastrar = function(){


$http.get("http://localhost:8089/Aula19_ServAngu/json/pessoa/gra
var/" +
        $scope.model.nome + "/" +$scope.model.sexo + "/"+
$scope.model.idade
                      )
                      .then(function(msg){
                        $scope.message = msg.data;
                  }).catch(function(msg){
                        $scope.message = msg.data;
                      });
```

```
        //area de limpaeza dos Campos ...
              $scope.model.nome="";
              $scope.model.sexo="";
              $scope.model.idade="";

          }
  });

  app.controller("listarctl", function($scope, $http){
       $scope.consultar = function(){

$http.get("http://localhost:8089/Aula19_ServAngu/json/pessoa/lis
tar"
                    )
                  .then(function(dados){
                        $scope.listagem= dados.data;
                  });
        }

       $scope.excluir = function(cod){
            var resp = window.confirm('excluir');

            if (resp==true){
$http.get("http://localhost:8089/Aula19_ServAngu/json/pessoa/exc
luir/"
             + cod
                )
            .then(function(dados){
                  $scope.listagem= dados.data;
            });
         window.location.reload();
             }

     }
       $scope.alterar = function(pessoa){
$http.get("http://localhost:8089/Aula19_ServAngu/json/pessoa/alt
erar/"
            +pessoa.nome + "/" +     pessoa.sexo + "/"+
            pessoa.idade + "/" +     pessoa.codigo
                )
            .then(function(dados){
                  $scope.listagem= dados.data;
            });

         window.location.reload();
```

```
            }

  });
</script>

</html>
```

---

main.html

```
<h2>Bem Vindo ao Angular 1b</h2>
<br>

<div class="col-md-12 well">
     Teste  de Sistema com Angular Aula dois
</div>
```

---

inclusão.html

```
<h3 style="text-align: center;">Cadastro com NG</h3>

<div class="col-md-6 col-md-offset-3 well">
     <div class="panel-body">
          <form ng-controller="gravarctl" role="form">

               <label for="nome">Nome:</label>
               <input type="text" ng-model="model.nome"
id="nome" required="required"
                    title="Digite o Nome" pattern="[a-z A-
Z]{2,50}" class="form-control"
                    placeholder="Digitar seu nome" />

               <label for="sexo">Sexo:</label>
               <select ng-model="model.sexo"
required="required" title="Escolha o Sexo"
                    id="sexo" class="form-control">
                    <option value="">Escolha o sexo</option>
                    <option value="m">Masculino</option>
                    <option value="f">Feminino</option>
```

```
                </select>

                <label for="idade">Idade:</label>
                <input type="number" ng-model="model.idade"
required="required" title="Digite a Idade"
                    class="form-control" id="idade"
placeholder="Digite sua Idade" />

                    <br>

                <button type="button" ng-click="cadastrar()"
class="btn btn-primary"
                    title="Clique aqui!">Cadastrar</button>
            </form>
        </div>

        <div class="panel-footer">
            {{message}}
        </div>



</div>
```

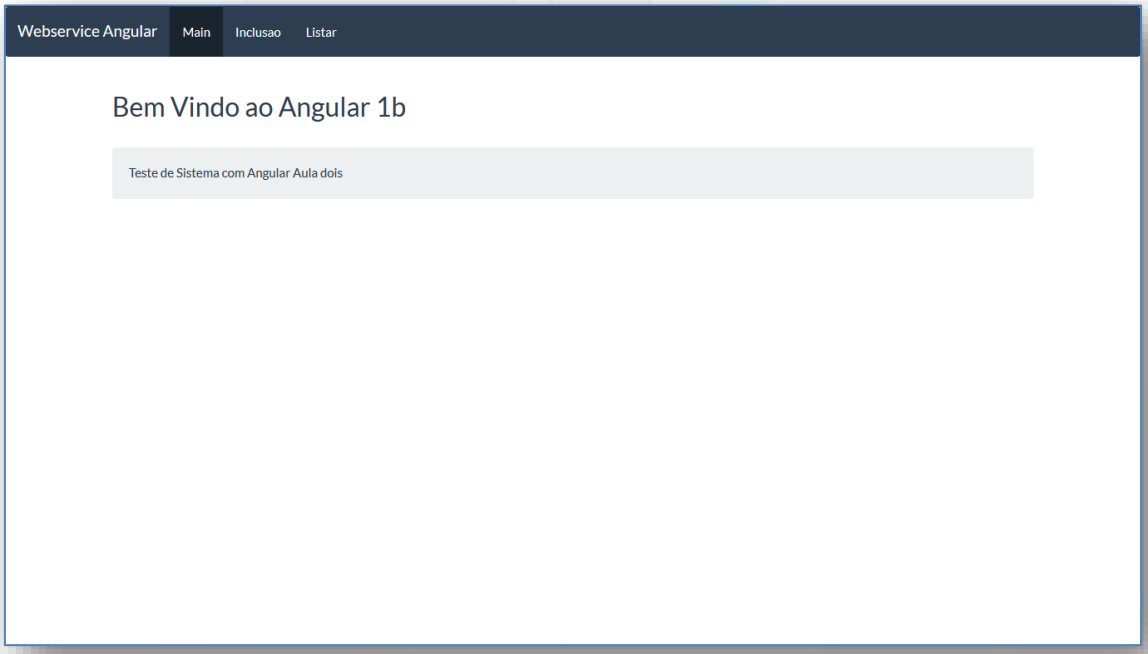
---

listar.html

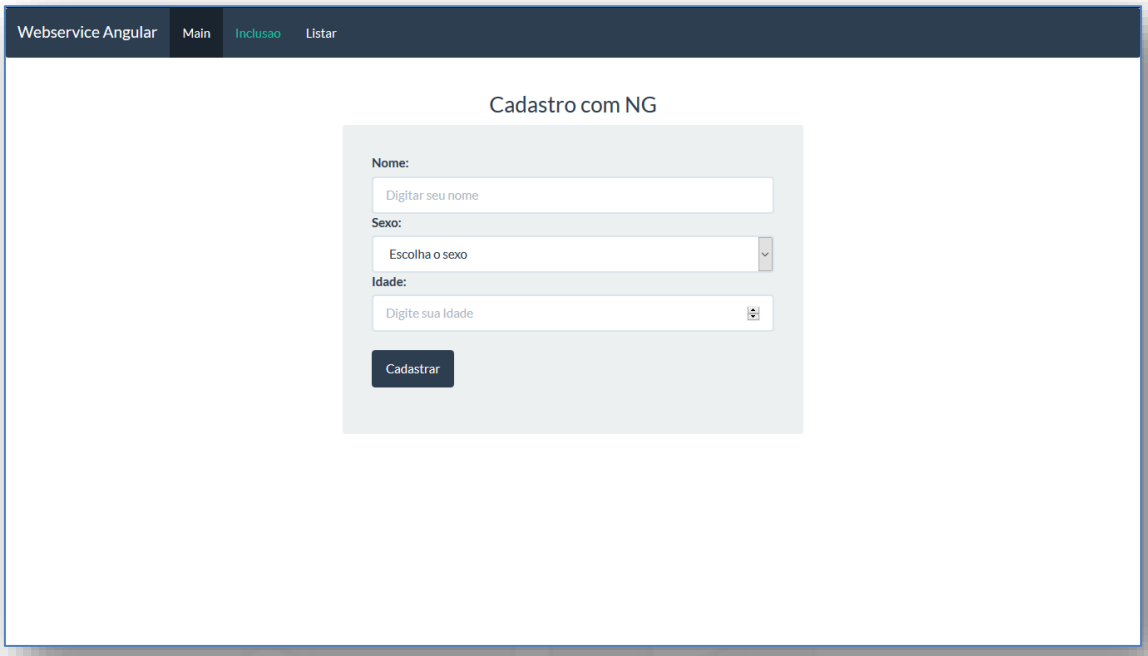
```
<h3 style="text-align: center;">Listar Clientes</h3>

<div class="col-md-10 col-md-offset-1 well">

        <form ng-controller="listarctl">
            {{mensagem}} <br>

            <table ng-init="consultar()" class="table table-
bordered table-hover ">
                <thead>
                    <tr>
                        <td>Codigo</td>
                        <td>Nome</td>
                        <td>Sexo</td>
                        <td>Idade</td>
```

```
                        <td>Excluir</td>
                        <td>Alterar</td>
                    </tr>
                </thead>

                <tbody>
                    <tr ng-repeat="p in listagem">
                        <td>
                            <input type="text"
readonly="readonly" value="{{p.codigo}}" class="form-control">
                        </td>

                        <td>
                            <input type="text" id="nome" ng-
model="p.nome" size="10" class="form-control">
                        </td>

                        <td>
                            <select ng-model="p.sexo"
class="form-control">
                                <option
value="m">Masculino</option>
                                <option
value="f">Feminino</option>
                            </select>
                        </td>

                        <td>
                            <input type="number" id="idade"
ng-model="p.idade"    size="5" class="form-control">
                        </td>

                        <td>
                            <button type="button" ng-
click="excluir(p.codigo)"
                                class="btn btn-success">
                                <i class="fa fa-
trash"></i>
                            </button>
                        </td>
                        <td><button type="button" ng-
click="alterar(p)"
                                class="btn btn-warning">
                                <i class="fa fa-
pencil"></i>
```

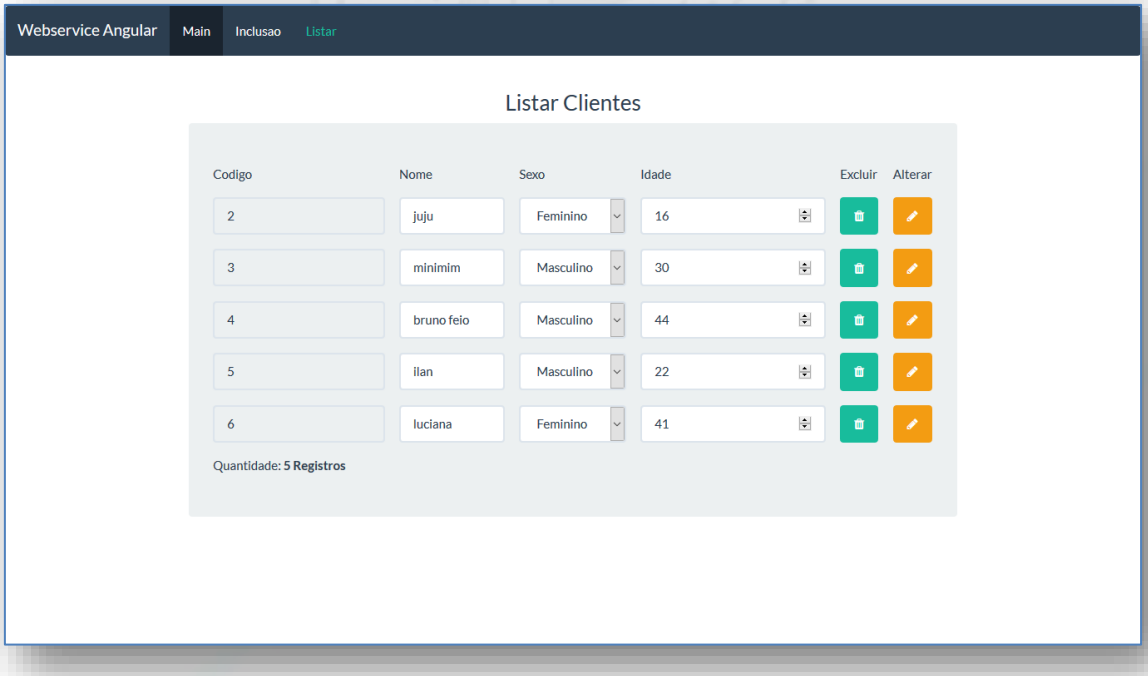
```
                                    </button>
                                    </td>
                              </tr>
                        </tbody>
                        <tfoot>
                              <tr>
                                    <td colspan="4">Quantidade:
<b>{{listagem.length}} Registros</b>
                              </tr>
                        </tfoot>
                  </table>

            </form>

</div>
```



Inclusão.html



Listar.html



JBoss Application Server

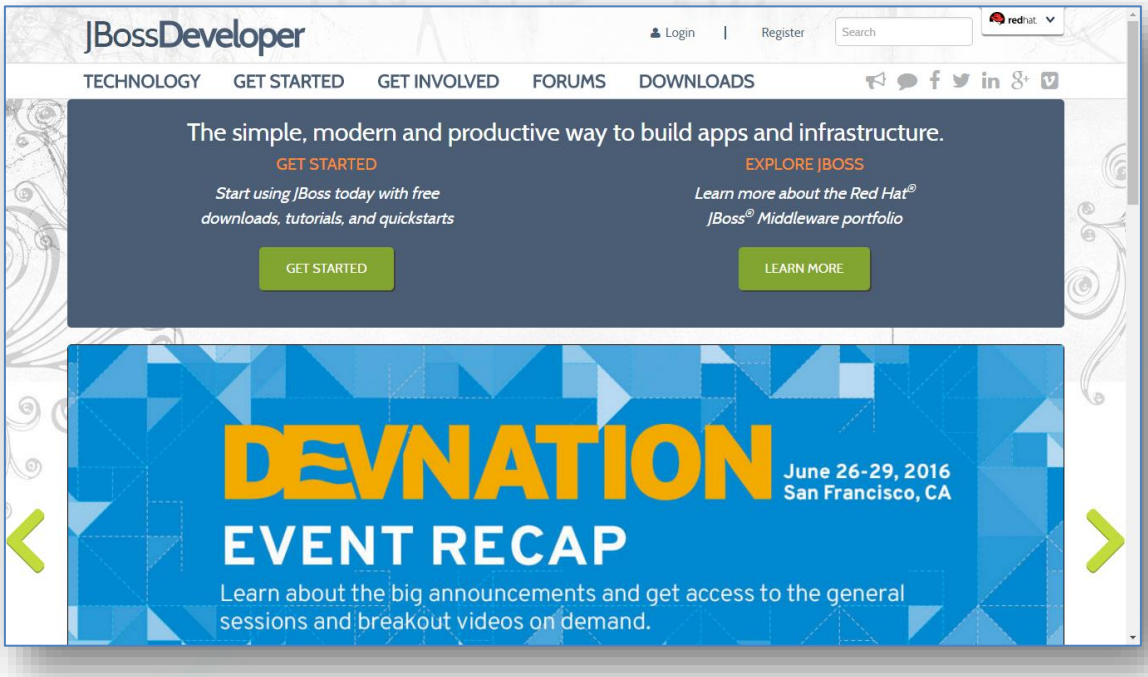
JBoss é um servidor de aplicação de código fonte aberto baseado na plataforma JEE e implementado completamente na linguagem de programação Java. Em virtude disso, ele pode ser usado em qualquer Sistema Operacional que suporte a referida linguagem. O JBoss Application Server 7, utiliza os arquivos: standalone.bat (ou standalone.sh) para prover a sua inicialização.

Os desenvolvedores responsáveis estão em uma empresa de serviços chamada *"JBoss Inc."* fundada por Marc Fleury, o criador da primeira versão do JBoss. O projeto é custeado por uma rede mundial de colaboradores. Em Abril de 2006, foi anunciada sua aquisição pela Red Hat.

A partir da versão 8 o JBoss passou a se chamar ***Wildfly***, além da troca de nomes teve várias melhorias e mudanças como a troca do container que era o JBossWeb para o Undertow.

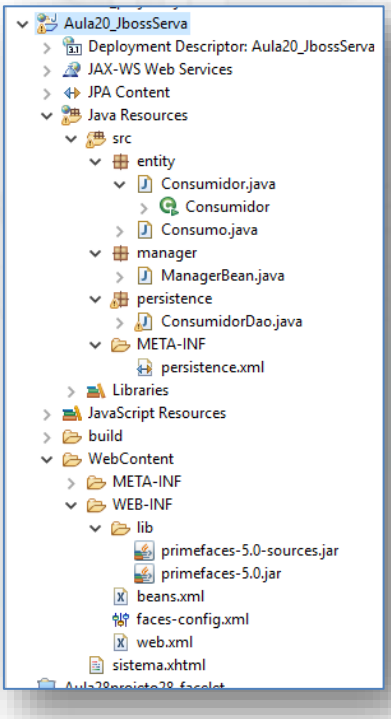
Como alternativa open-source ao JBoss, temos o GlassFish, que é a implementação de referência de toda a especificação Java EE.

http://www.jboss.org/



http://wildfly.org/

Estrutura do projeto:

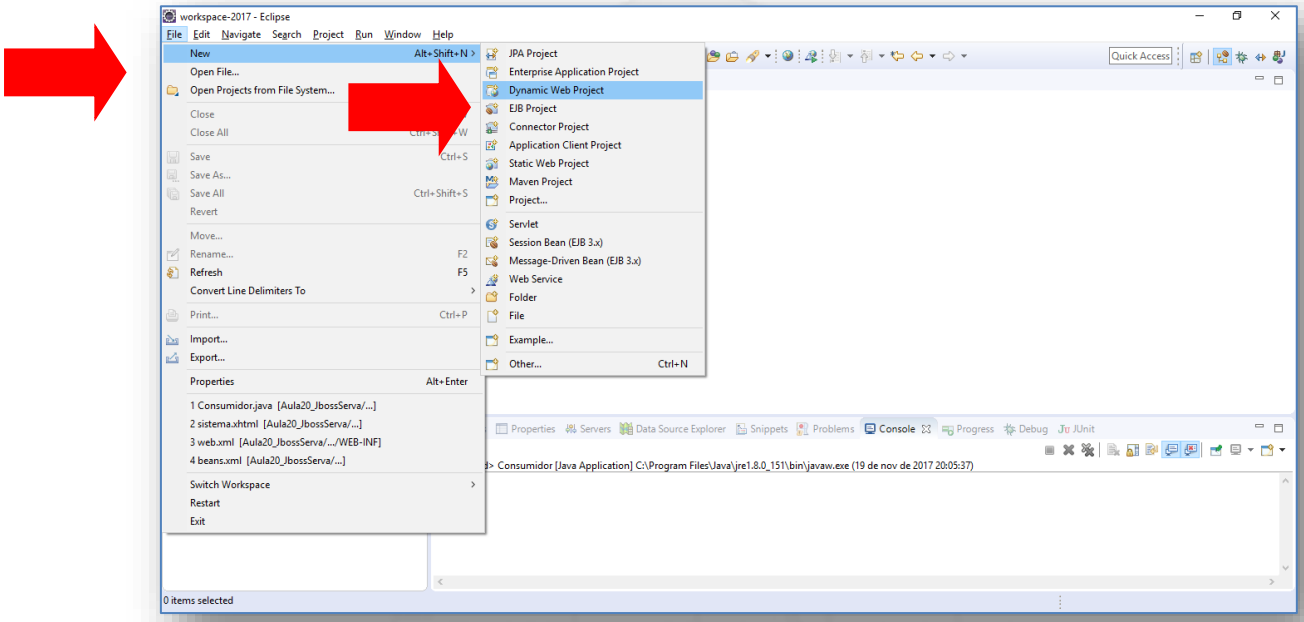


LIBS:

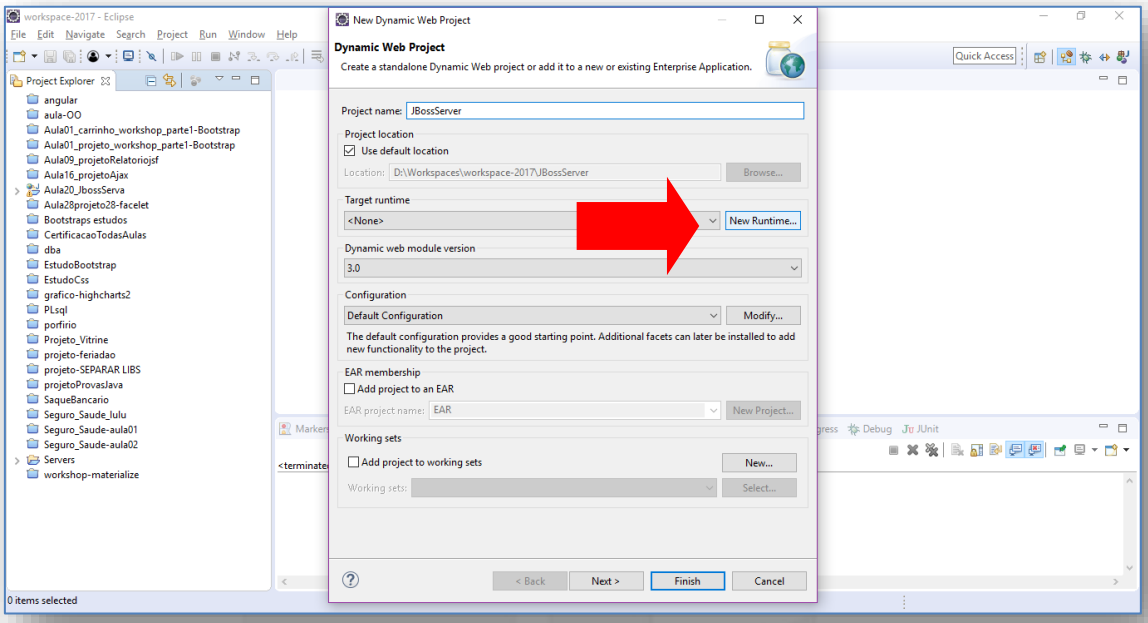
primefaces-5.0-sources.jar
primefaces-5.0.jar

Criando o projeto...

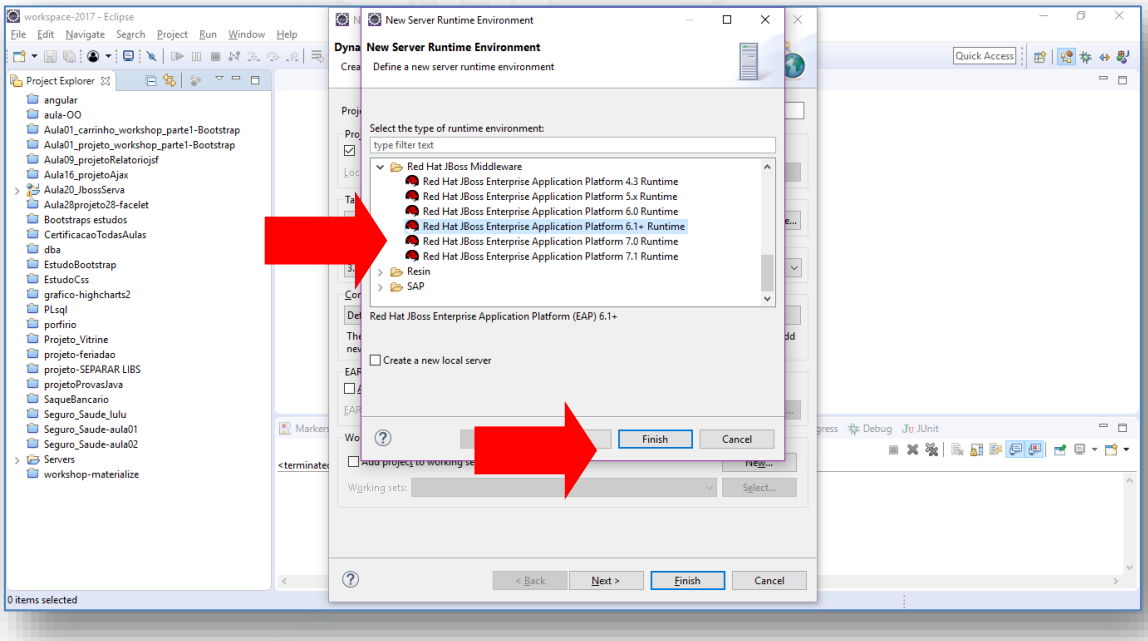
File -> New -> Dynamic Web Project...



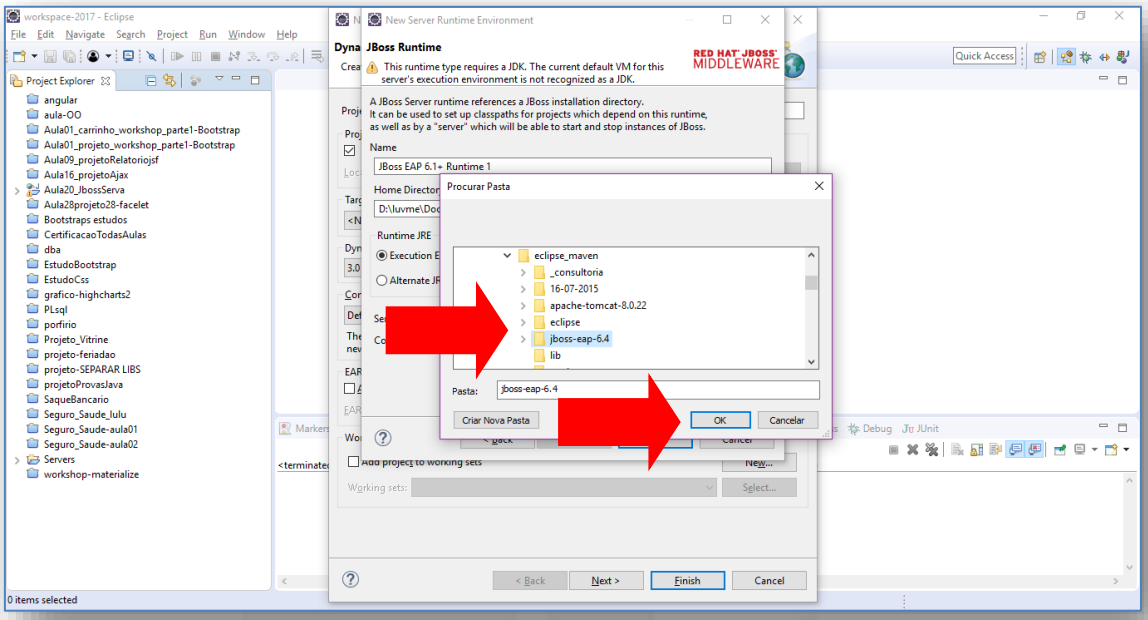
Clicar em new Runtime...



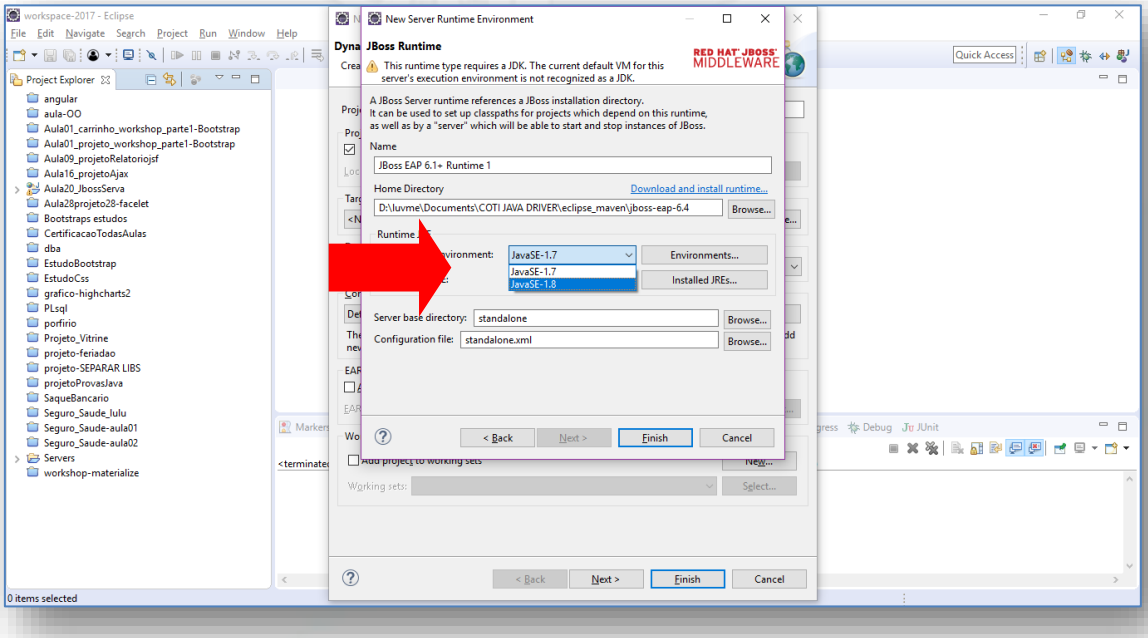
Selecionar Red Hat... -> Jboss 6.1 -> Next...



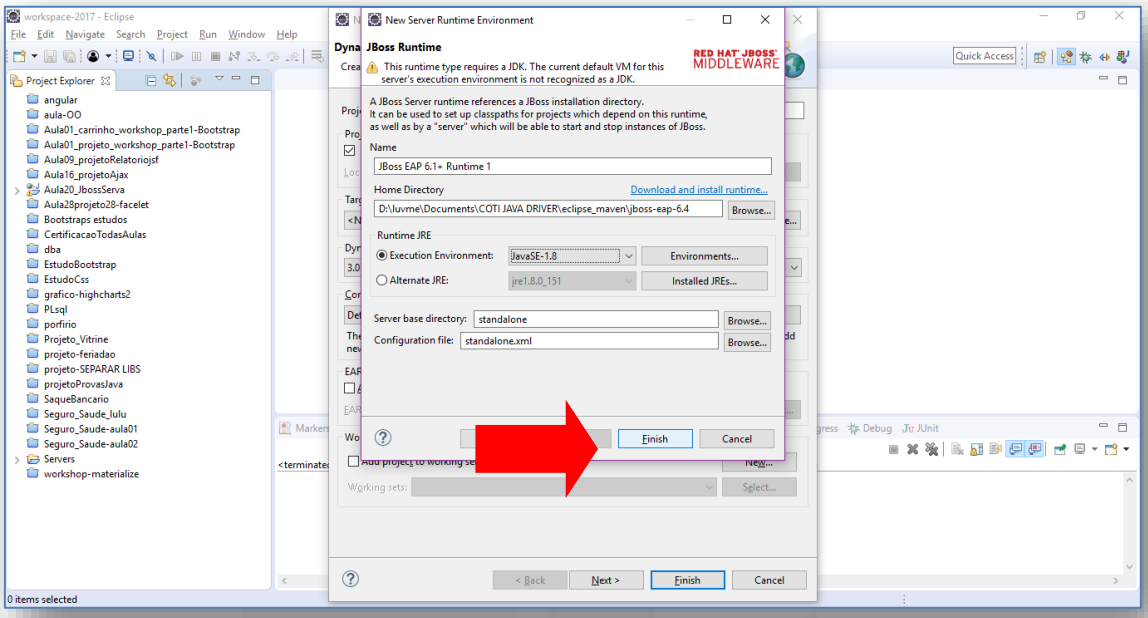
Indicar onde estão as bibliotecas do Jboss -> ok...



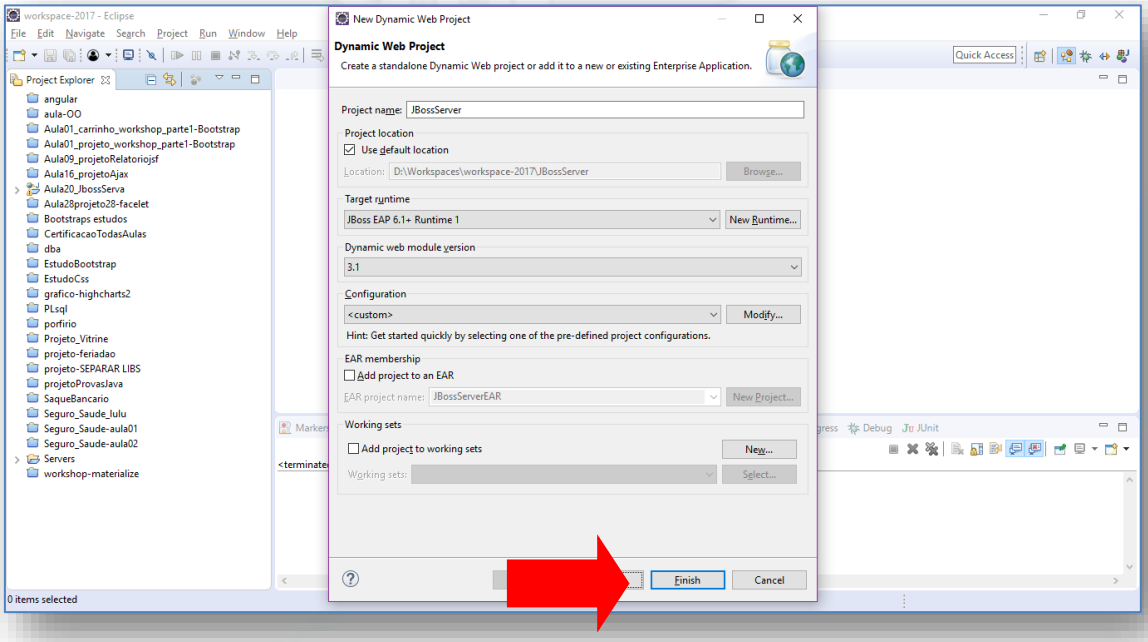
Mudar a JRE para java 1.8...



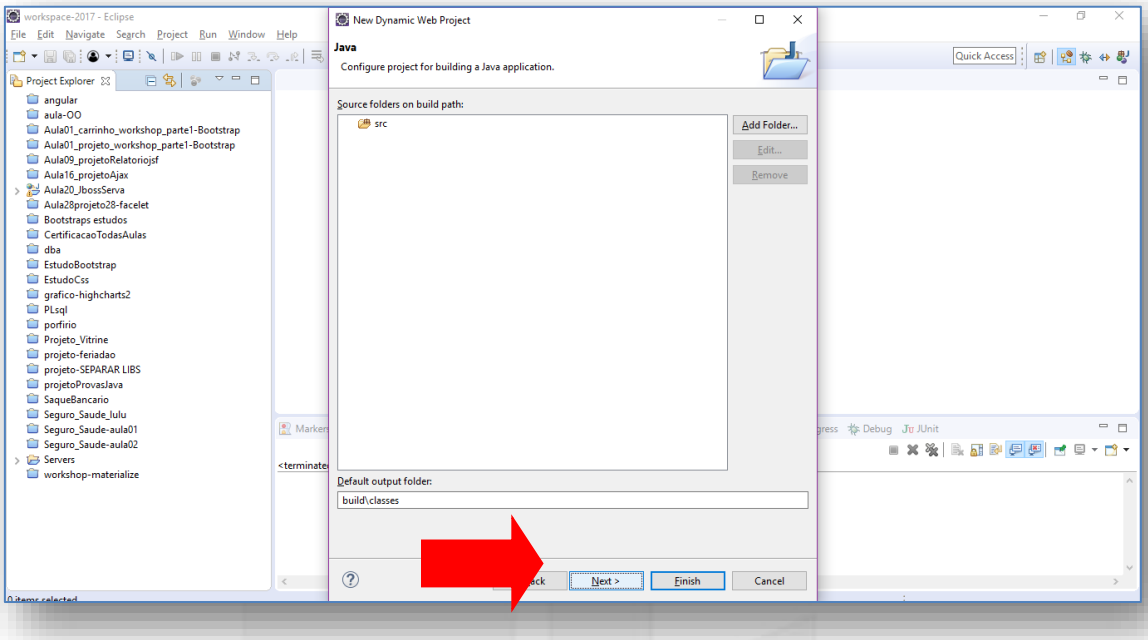
Finish...


New Server Runtime Environment
JBoss Runtime
This runtime type requires a JDK. The current default VM for this server's execution environment is not recognized as a JDK.
A JBoss Server runtime references a JBoss installation directory.
It can be used to set up classpaths for projects which depend on this runtime, as well as by a "server" which will be able to start and stop instances of JBoss.
Name
JBoss EAP 6.1+ Runtime 1
Home Directory
Download and install runtime...
D:\luvme\Documents\COTI JAVA DRIVER\eclipse_maven\jboss-eap-6.4
Browse...
Runtime JRE
Execution Environment:
JavaSE-1.8
Environments...
Alternate JRE:
jre1.8.0_151
Installed JREs...
Server base directory:
standalone
Configuration file:
standalone.xml
Finish
Cancel
< Back
Next >
Finish
Cancel

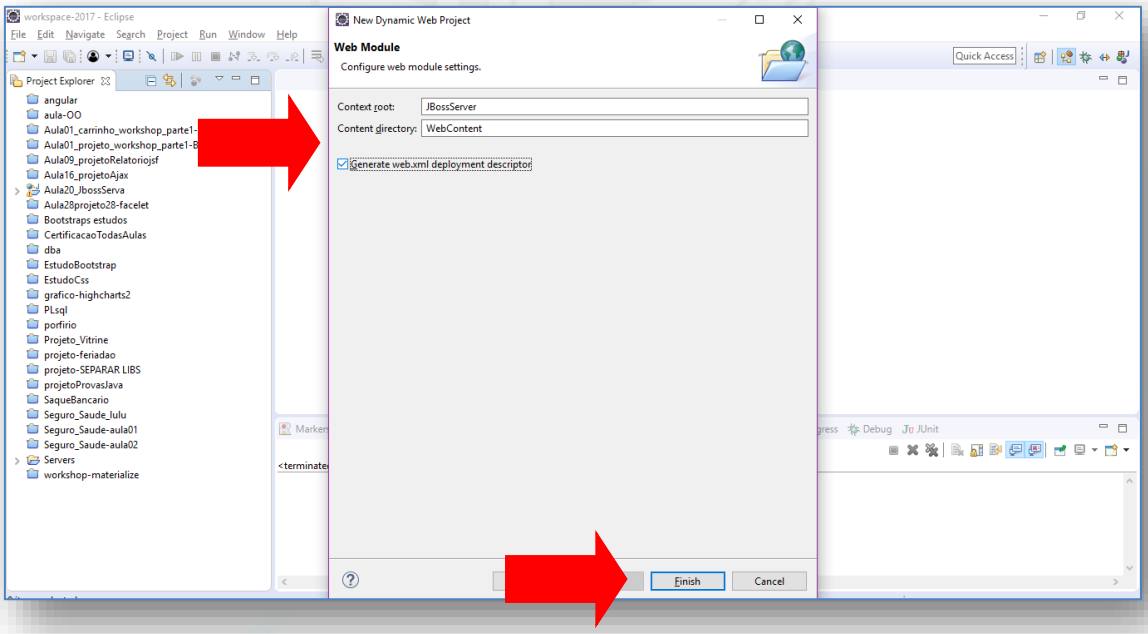
Next...


New Dynamic Web Project
Dynamic Web Project
Create a standalone Dynamic Web project or add it to a new or existing Enterprise Application.
Project name:
JBossServer
Project location
Use default location
Location:
D:\Workspaces\workspace-2017\JBossServer
Browse...
Target runtime
JBoss EAP 6.1+ Runtime 1
New Runtime...
Dynamic web module version
3.1
Configuration
<custom>
Modify...
Hint: Get started quickly by selecting one of the pre-defined project configurations.
EAR membership
Add project to an EAR
EAR project name:
JBossServerEAR
New Project...
Working sets
Add project to working sets
New...
Working sets:
Select...
Finish
Cancel

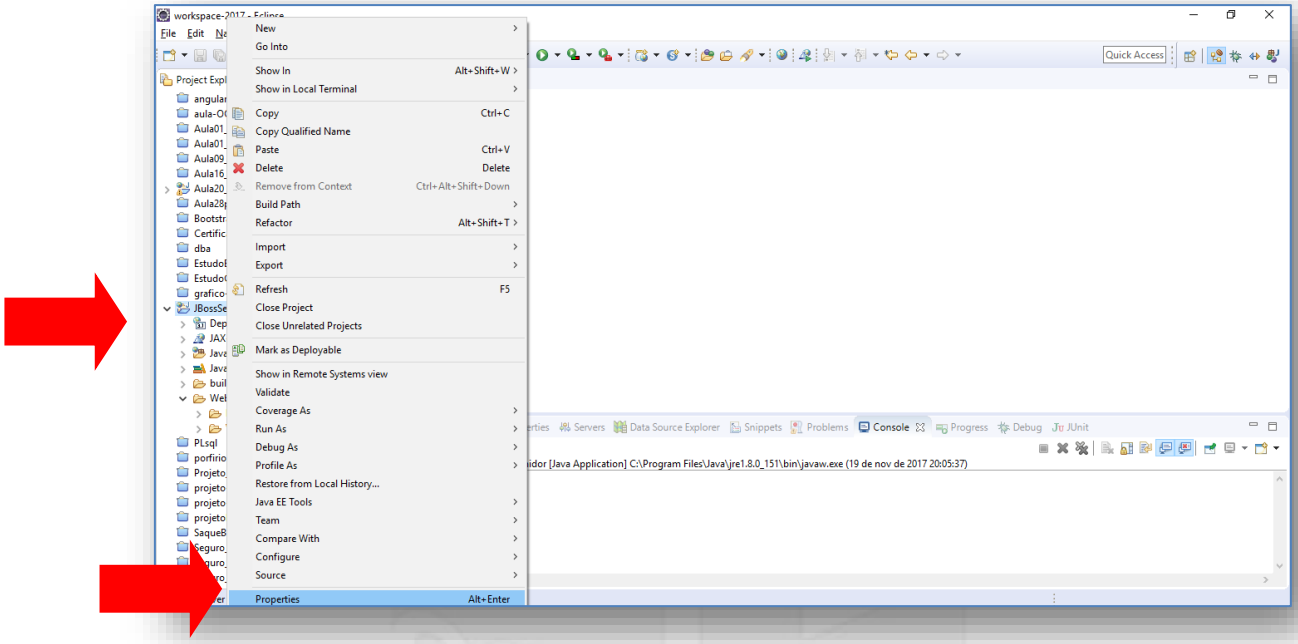
Next...



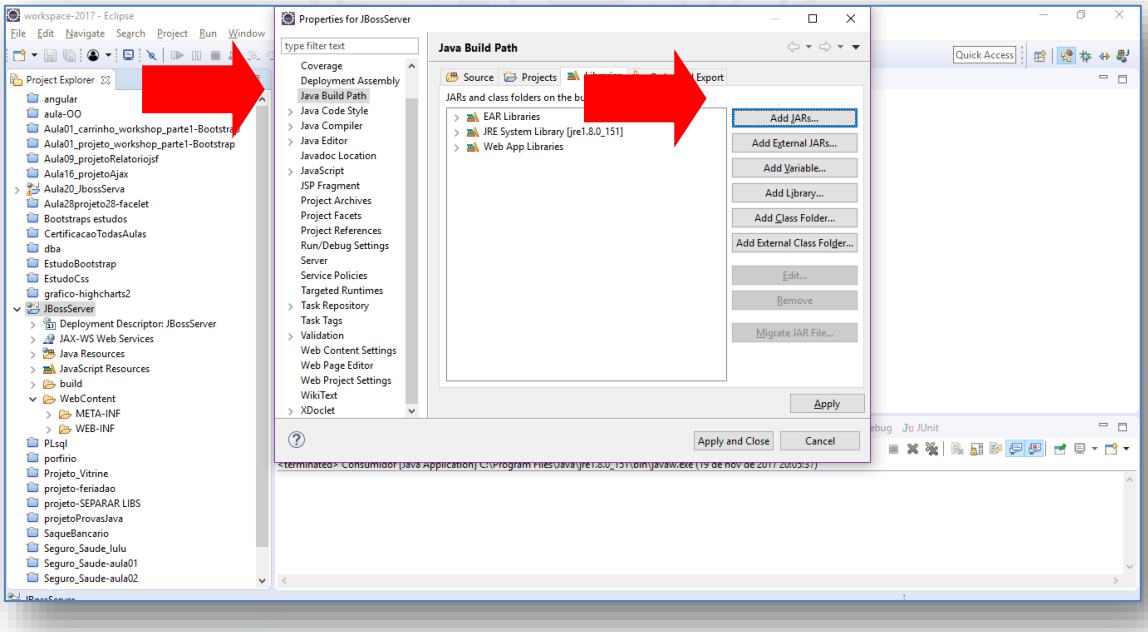
Marcar generate web.xml -> Finish...



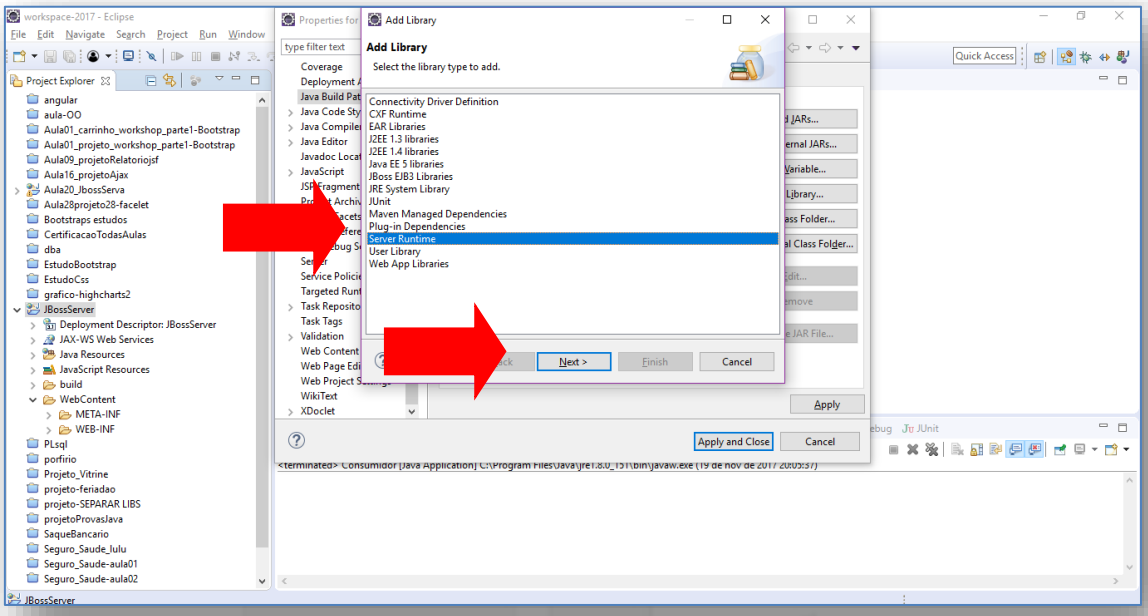
Depois de montado o projeto, clicar com o botão direito > Properties..



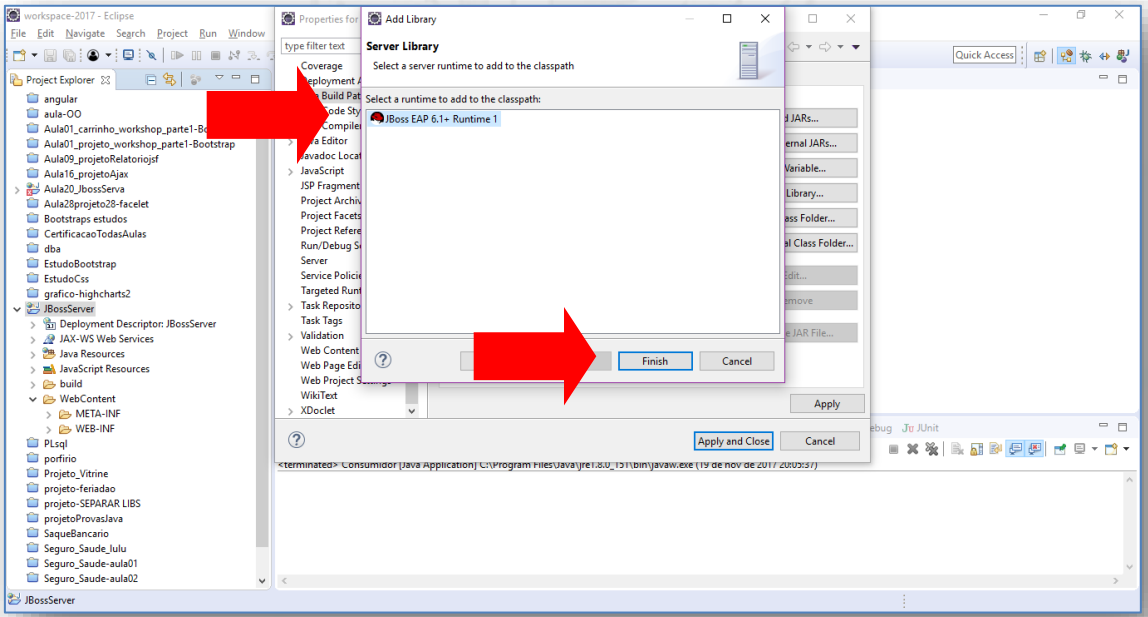
Clicar em Java Build Path -> Add Library



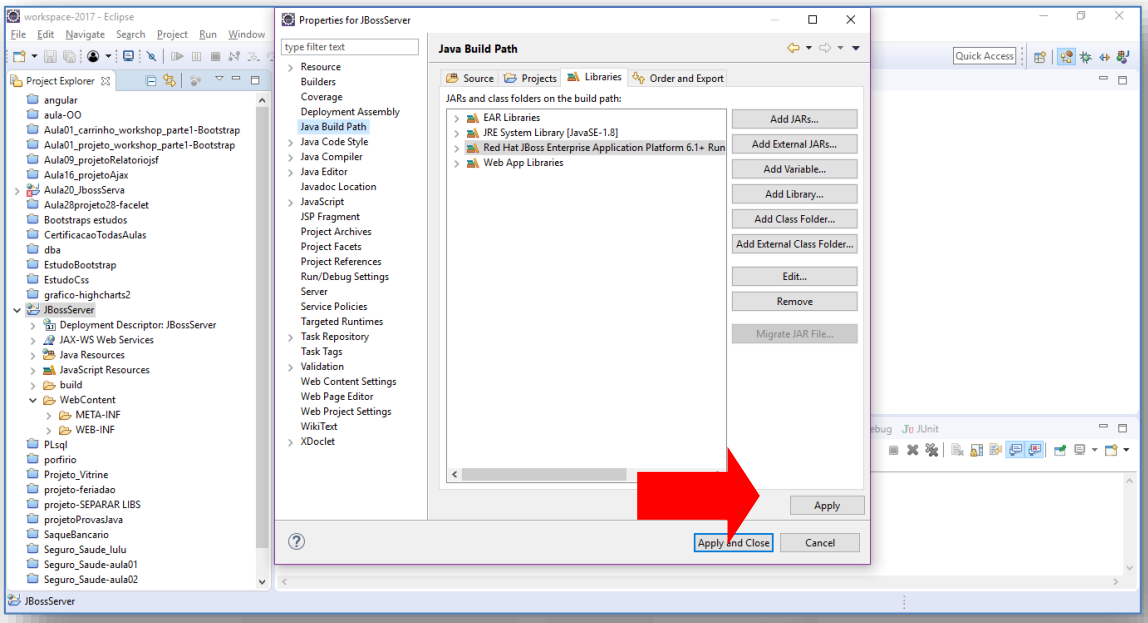
Clicar em server runtime -> next...



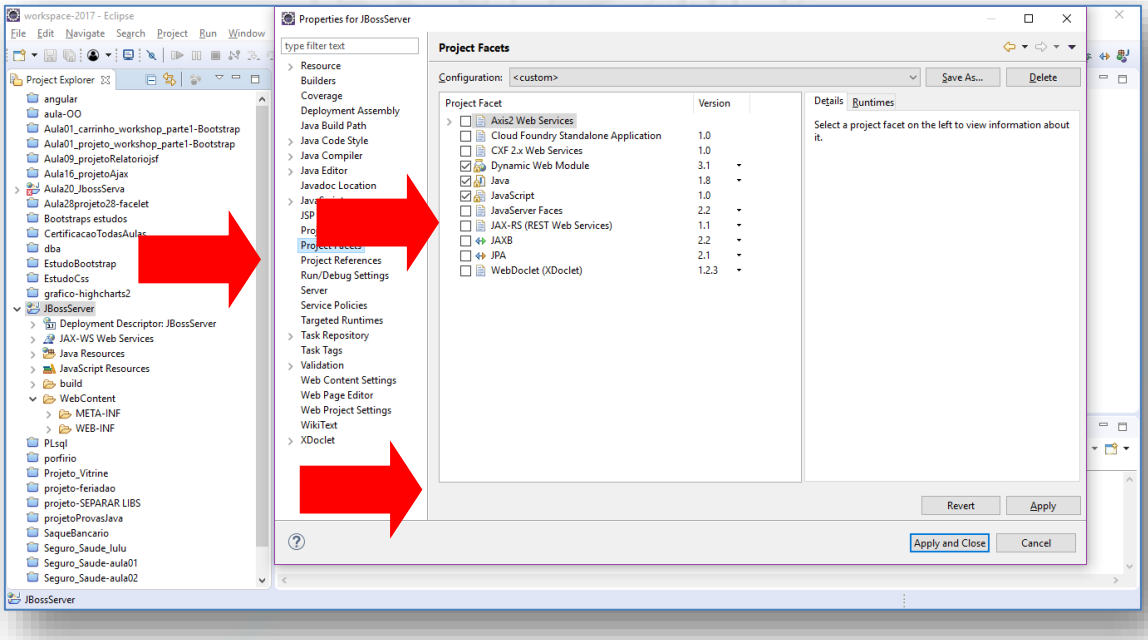
Clicar no jboss -> Finish...



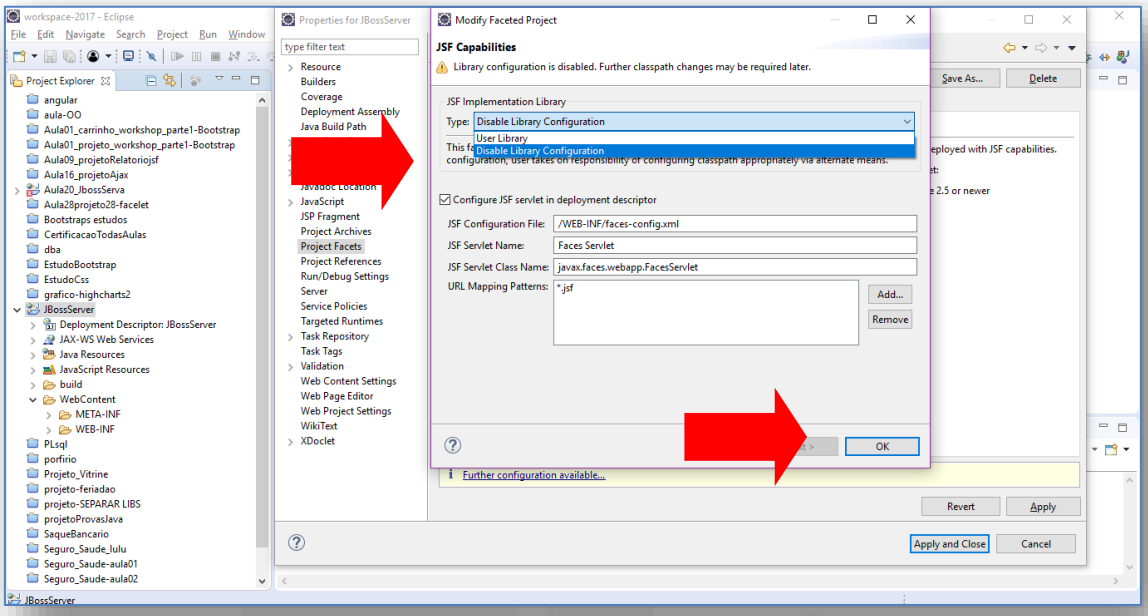
Clicar em apply...



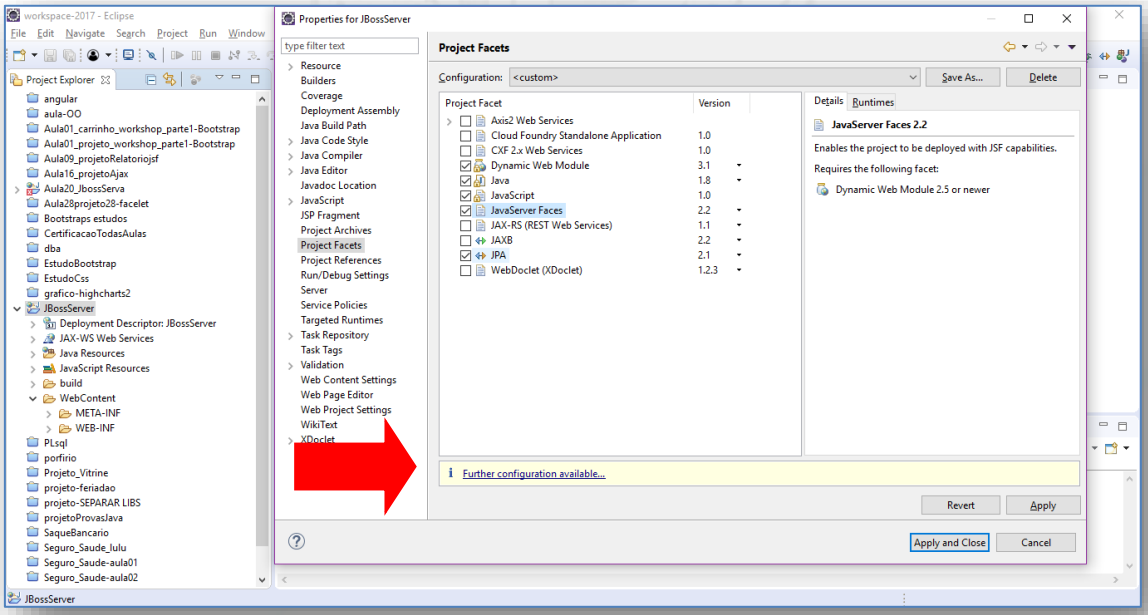
Clicar em Project Facets -> Java server faces -> Futher configuration...



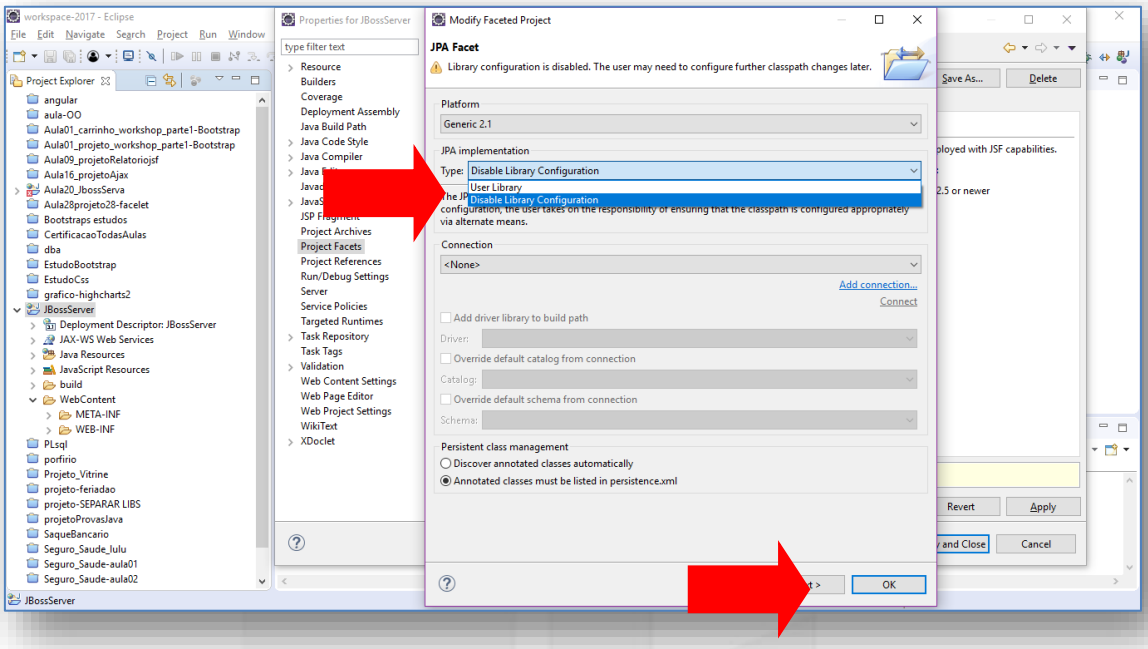
Clicar em disable -> Add ".jsf" -> ok...*



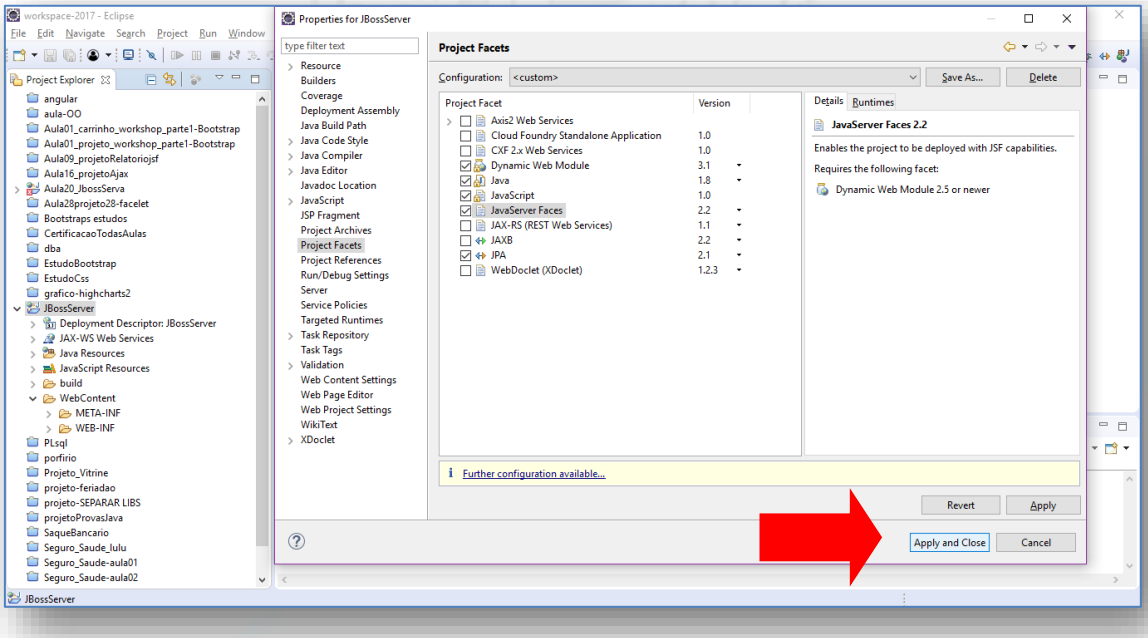
Clicar em JPA > futher configuration...



Clicar em disable -> ok...



Apply and close...



```
package entity;


import java.io.Serializable;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.FetchType;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.NamedQuery;
import javax.persistence.OneToMany;
import javax.persistence.Table;
import javax.persistence.Transient;

@Entity
@Table(name="consumidor")

@NamedQuery(name="mostra",query="select c from Consumidor as c")
public class Consumidor  implements Serializable{

	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy=GenerationType.IDENTITY)
	private Integer id;
	@Column(length=100)
	private String nome;
	@Column(length=100,unique=true)
	private String email;
	@Column(length=25,unique=true)
	private String telefone;
	@Transient
	private transient Double total=0.;

	@OneToMany(mappedBy="consumidor",fetch=FetchType.EAGER)
	private List<Consumo> consumos;

	public Consumidor() {
```

```
	}

	public Consumidor(Integer id, String nome, String email,
String telefone) {
		this.id = id;
		this.nome = nome;
		this.email = email;
		this.telefone = telefone;
	}

	@Override
	public String toString() {
		return "Consumidor [id=" + id + ", nome=" + nome + ",
email=" + email + ", telefone=" + telefone + ", total=" + total
+ "]";
	}

	public Integer getId() {
		return id;
	}
	public void setId(Integer id) {
		this.id = id;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public String getTelefone() {
		return telefone;
	}
	public void setTelefone(String telefone) {
		this.telefone = telefone;
	}
	public Double getTotal() {
		return total;
	}
	public void setTotal(Double total) {
```

```
            this.total = total;
      }
      public static long getSerialversionuid() {
            return serialVersionUID;
      }
      public List<Consumo> getConsumos() {
            return consumos;
      }
      public void setConsumos(List<Consumo> consumos) {
            this.consumos = consumos;
      }

      public void adicionar(Consumo ...c){
            if (consumos==null){
                  consumos = new ArrayList<Consumo>();
            }
               consumos.addAll(Arrays.asList(c));
               calcularSubtotal();
      }

      private void  calcularSubtotal(){
            for(Consumo c: consumos){
            c.setSubtotal( c.getPreco() * c.getQuantidade() );
            }
      }

      public static void main(String[] args) {
    Consumidor c = new
Consumidor(10,"firmino","firmino@gmail.com","3333");
    Consumo c1 =new Consumo(101, "refrigerante", 7., 8);
    Consumo c2 =new Consumo(102, "redbull", 14., 3);
    Consumo c3 =new Consumo(103, "pizza maracana", 55., 5);

            c.adicionar(c1,c2,c3);

            System.out.println(c);
            System.out.println(c.getConsumos());
      }
}
```

No console...

```
Consumidor [id=10, nome=firmino, email=firmino@gmail.com,
telefone=3333, total=0.0]
[Consumo [idConsumo=101, nomeProduto=refrigerante, preco=7.0,
quantidade=8, subtotal=56.0], Consumo [idConsumo=102,
nomeProduto=redbull, preco=14.0, quantidade=3, subtotal=42.0],
Consumo [idConsumo=103, nomeProduto=pizza maracana, preco=55.0,
quantidade=5, subtotal=275.0]]
```

```
package entity;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.Table;
import javax.persistence.Transient;

@Entity
@Table(name="consumo")
public class Consumo implements Serializable{

	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy=GenerationType.IDENTITY)
	private Integer idConsumo;
	@Column(length=50)
	private String nomeProduto;
	@Column
	private Double preco;
	@Column
	private Integer quantidade;
	@Transient
	private Double subtotal=0.;
```

```
	@ManyToOne
	@JoinColumn(name="fkIdConsumidor")
	private Consumidor consumidor;


	public Consumo() {
	}

	public Consumo(Integer idConsumo, String nomeProduto,
Double preco, Integer quantidade) {
		this.idConsumo = idConsumo;
		this.nomeProduto = nomeProduto;
		this.preco = preco;
		this.quantidade = quantidade;
	}

	@Override
	public String toString() {
		return "Consumo [idConsumo=" + idConsumo + ",
nomeProduto=" + nomeProduto + ", preco=" + preco + ",
quantidade=" + quantidade + ", subtotal=" + subtotal + "]";
	}

	public Integer getIdConsumo() {
		return idConsumo;
	}
	public void setIdConsumo(Integer idConsumo) {
		this.idConsumo = idConsumo;
	}
	public String getNomeProduto() {
		return nomeProduto;
	}
	public void setNomeProduto(String nomeProduto) {
		this.nomeProduto = nomeProduto;
	}
	public Double getPreco() {
		return preco;
	}
	public void setPreco(Double preco) {
		this.preco = preco;
	}
	public Integer getQuantidade() {
		return quantidade;
	}
```

```
	public void setQuantidade(Integer quantidade) {
		this.quantidade = quantidade;
	}
	public Double getSubtotal() {
		return subtotal;
	}
	public void setSubtotal(Double subtotal) {
		this.subtotal = subtotal;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
	}
	public Consumidor getConsumidor() {
		return consumidor;
	}
	public void setConsumidor(Consumidor consumidor) {
		this.consumidor = consumidor;
	}
}
```

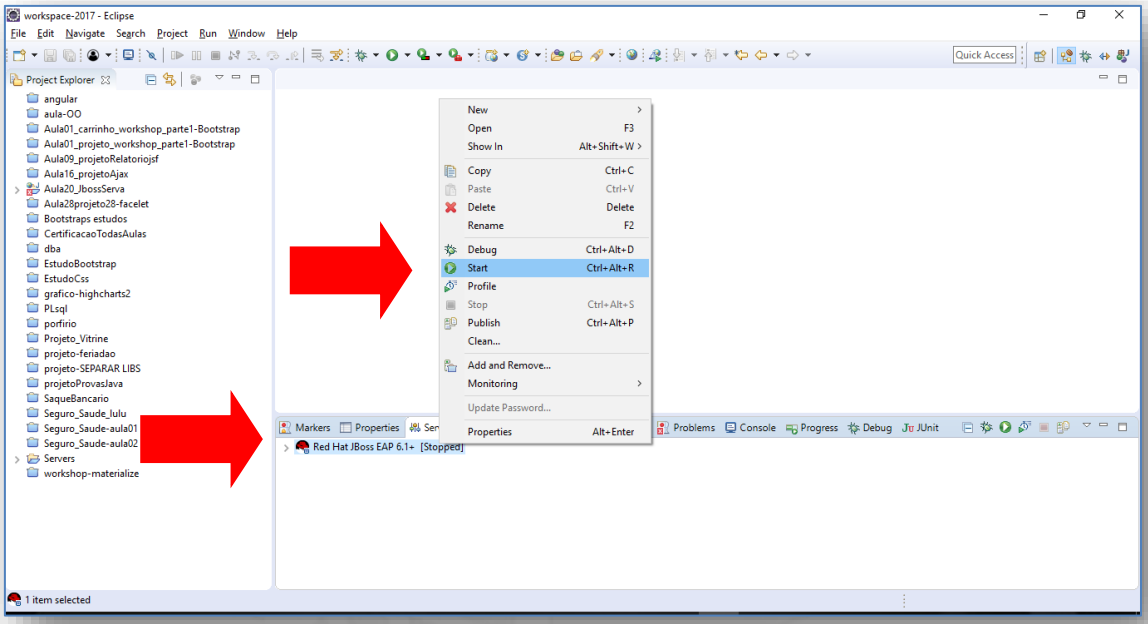
---

persistence.xml

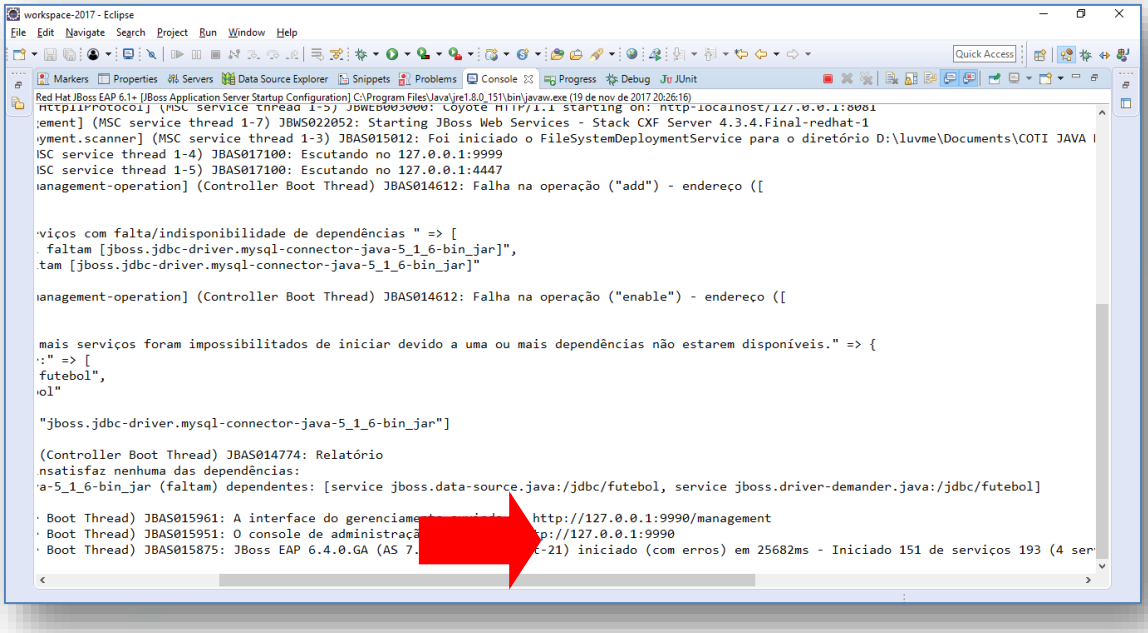
```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1"
xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd">

	<persistence-unit name="bar">

	<jta-data-source>java:/jdbc/bar</jta-data-source>

	 <class>entity.Consumidor</class>
	 <class>entity.Consumo</class>

	<properties>
	   <property name="hibernate.hbm2ddl.auto" value="update"/>
	    <property name="hibernate.show_sql" value="true"/>
	    <property name="hibernate.format_sql" value="true"/>
	</properties>

	</persistence-unit>
```

```
</persistence>
```

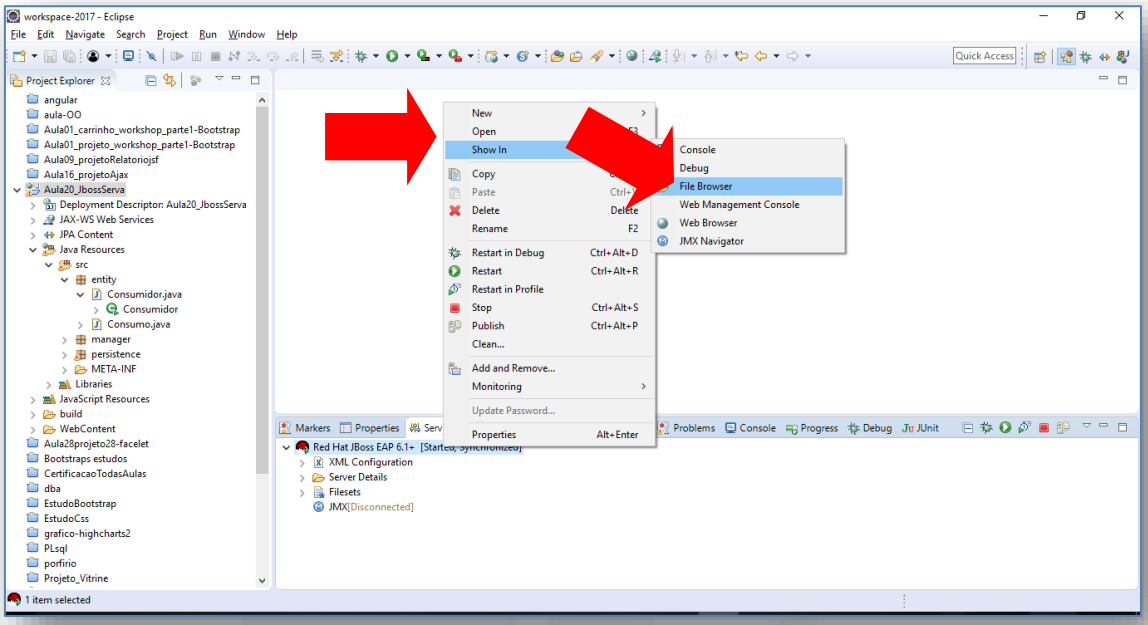
Na aba server, dar start no servidor do Jboss...



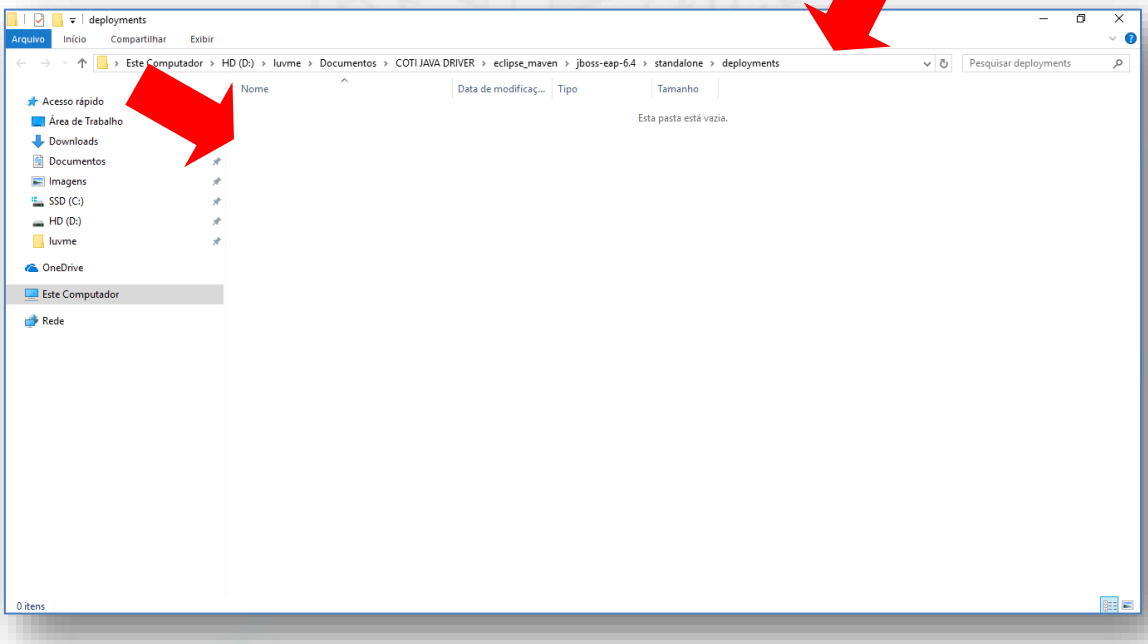
Inciado. (Mesmo que inicie com a mensagem que iniciou com erros não tem problema)...



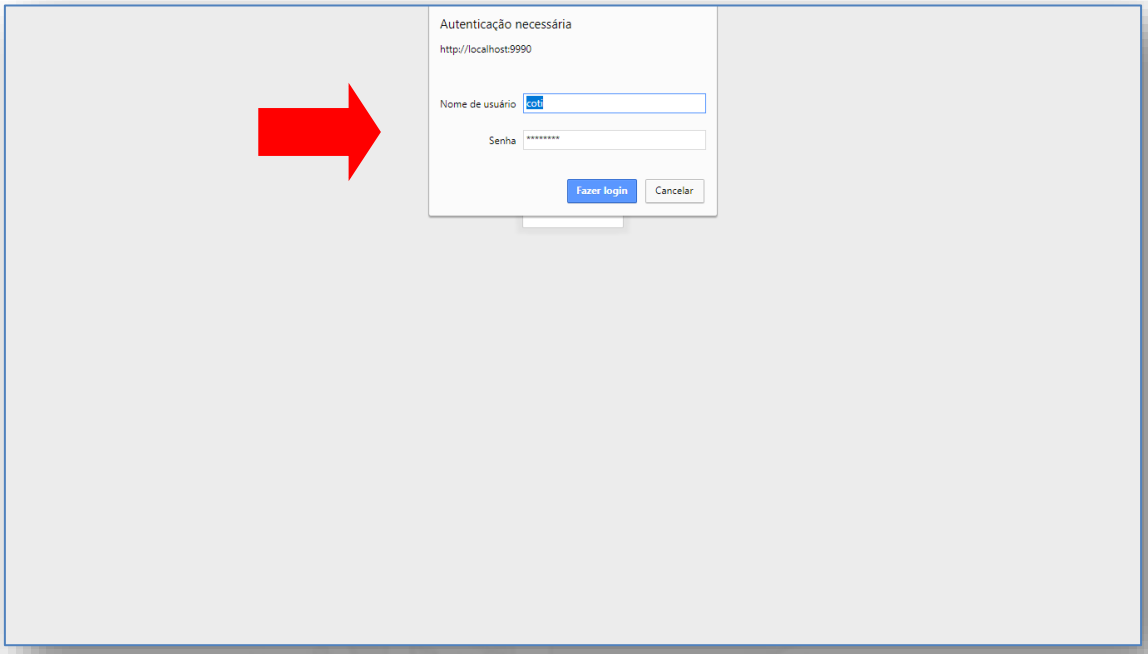
Clicar com o botão direito no servidor -> Show In -> File Browser...



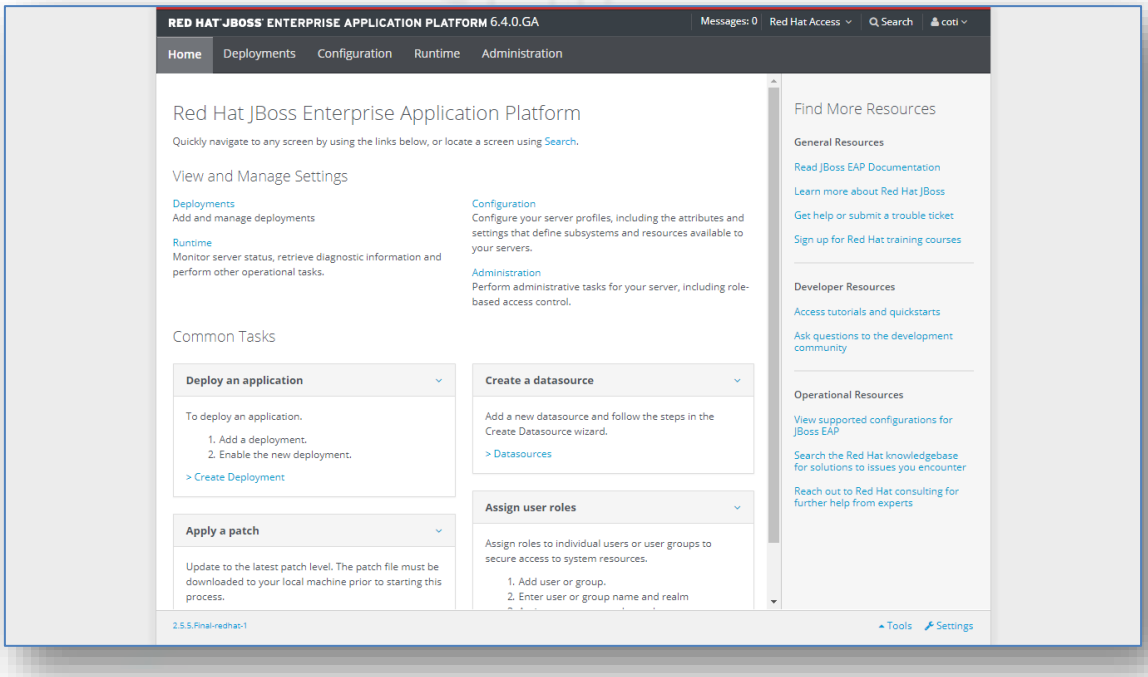
Apagar todos os projetos nesta pasta (se tiver algum)...



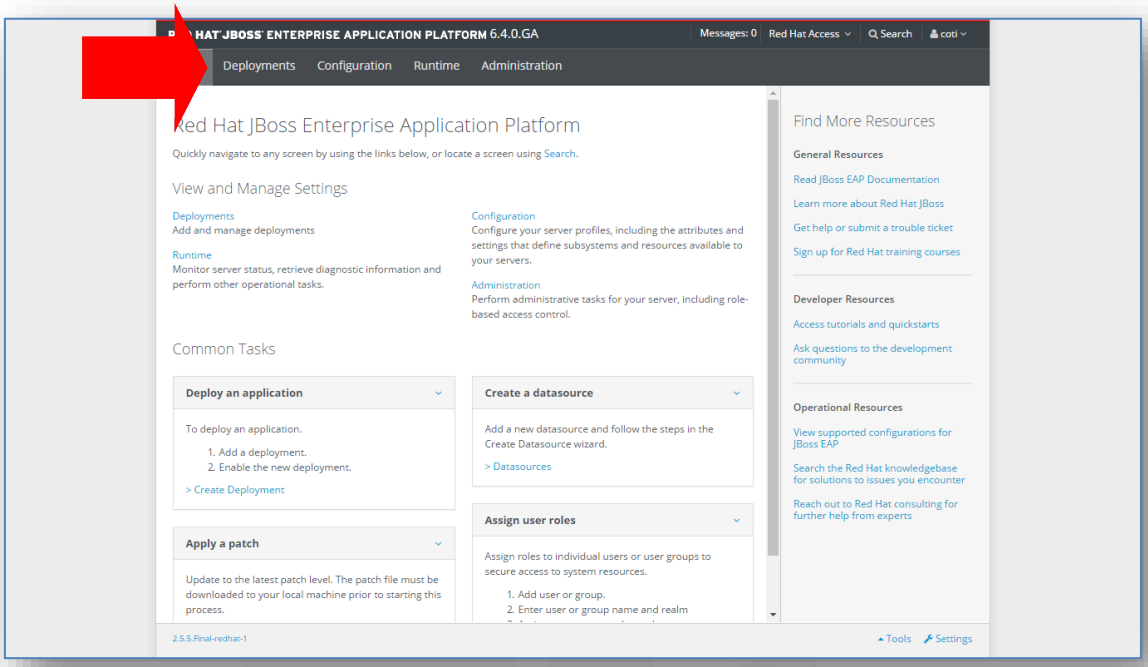
Para Ir para o console do jboss, digite na URL:
http://localhost:9990
usuário: coti - senha: coti@110



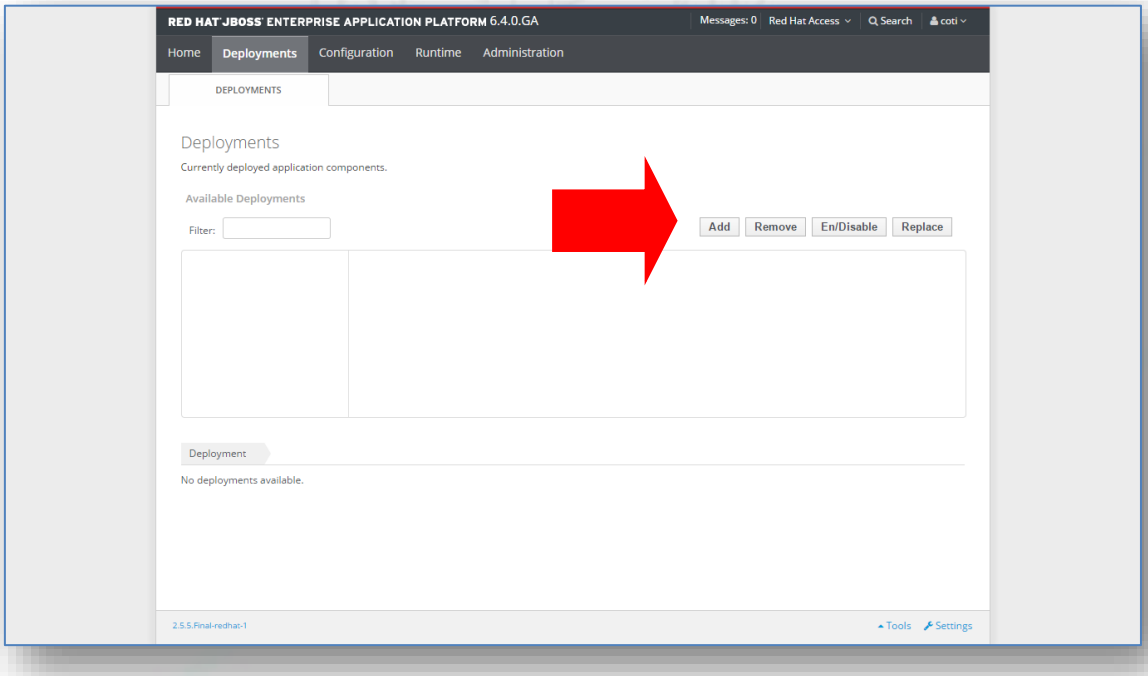
logado...



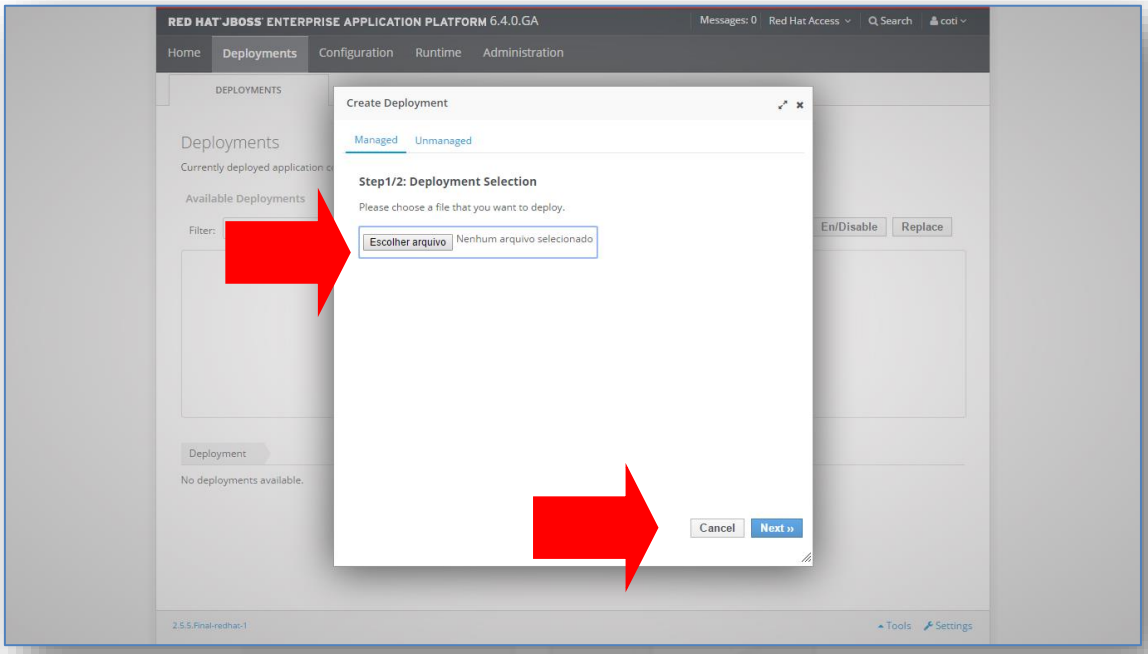
Clicar em Deployments...



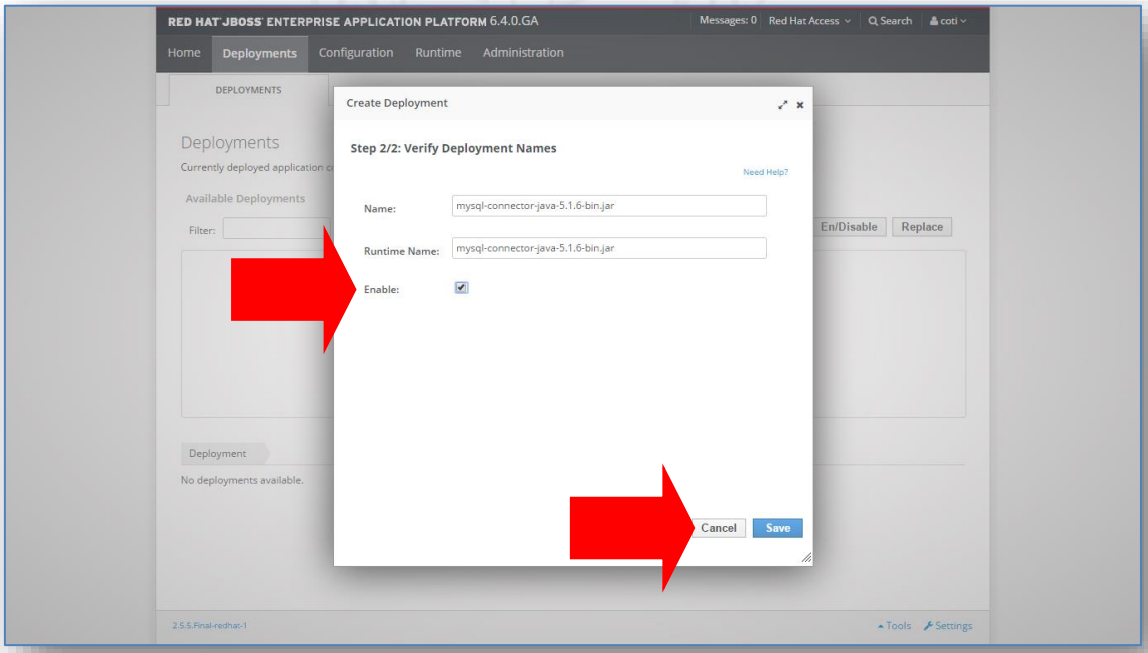
Adicionar o Mysql 5.1.6



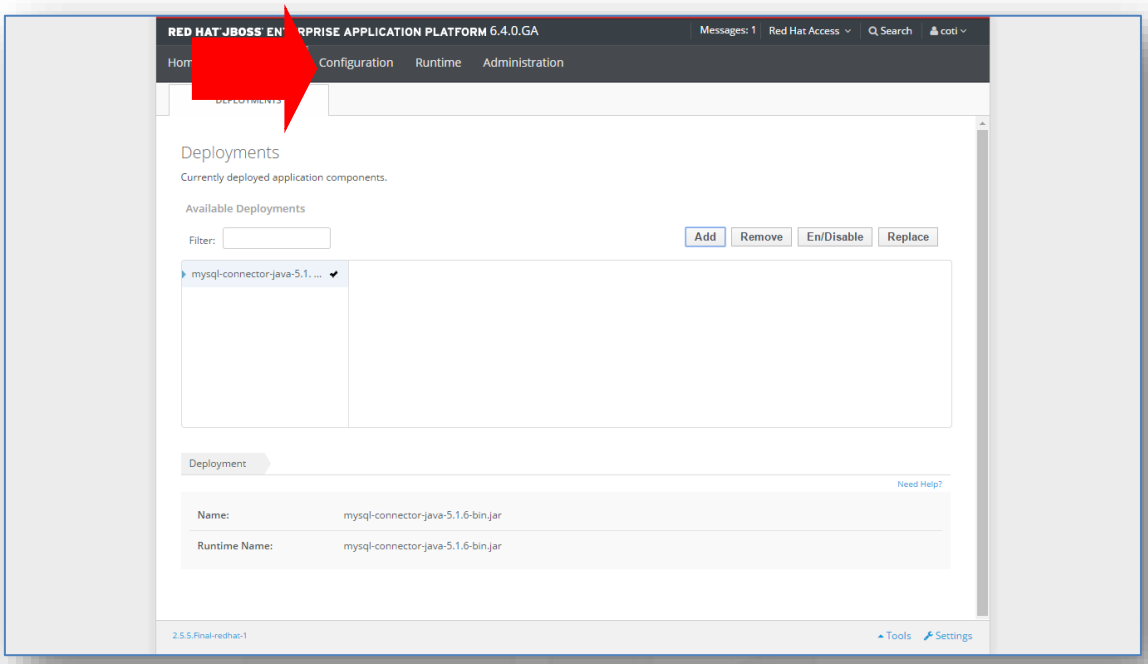
Clicar em add...



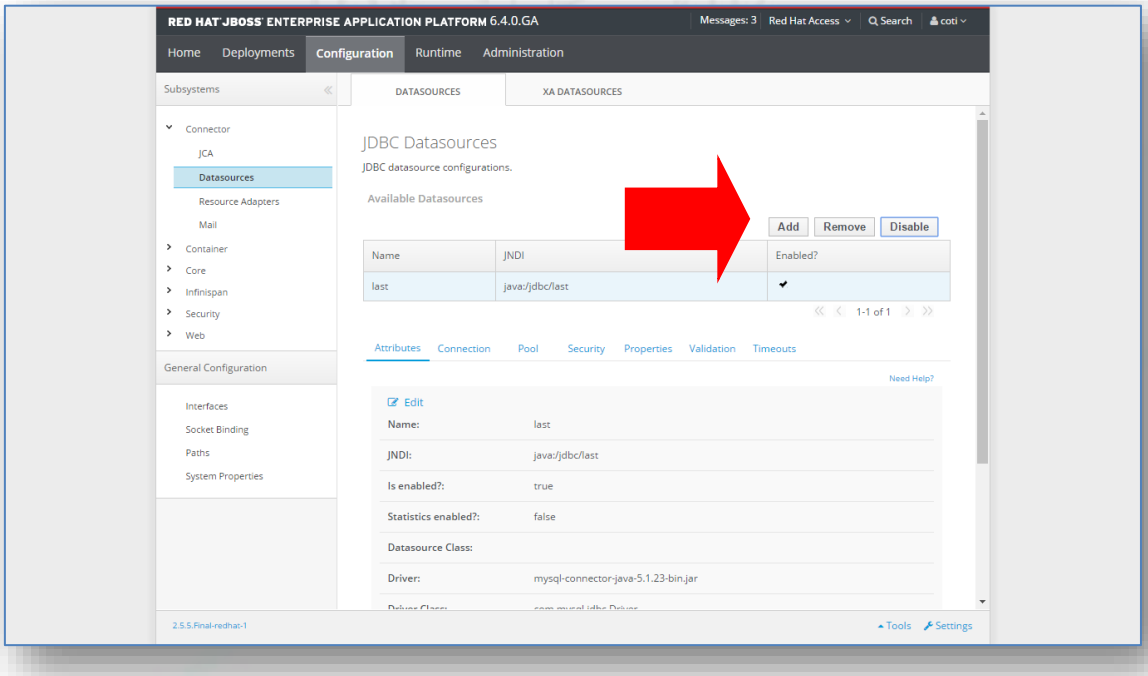
Clicar em enable -> save...



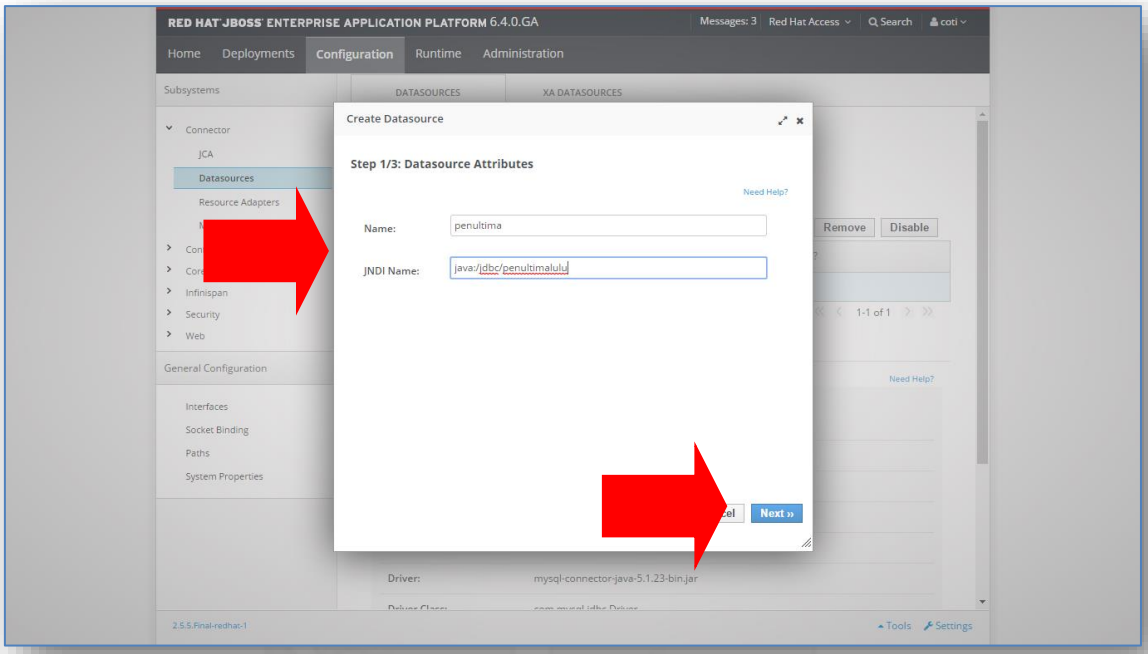
Clicar em configuration...



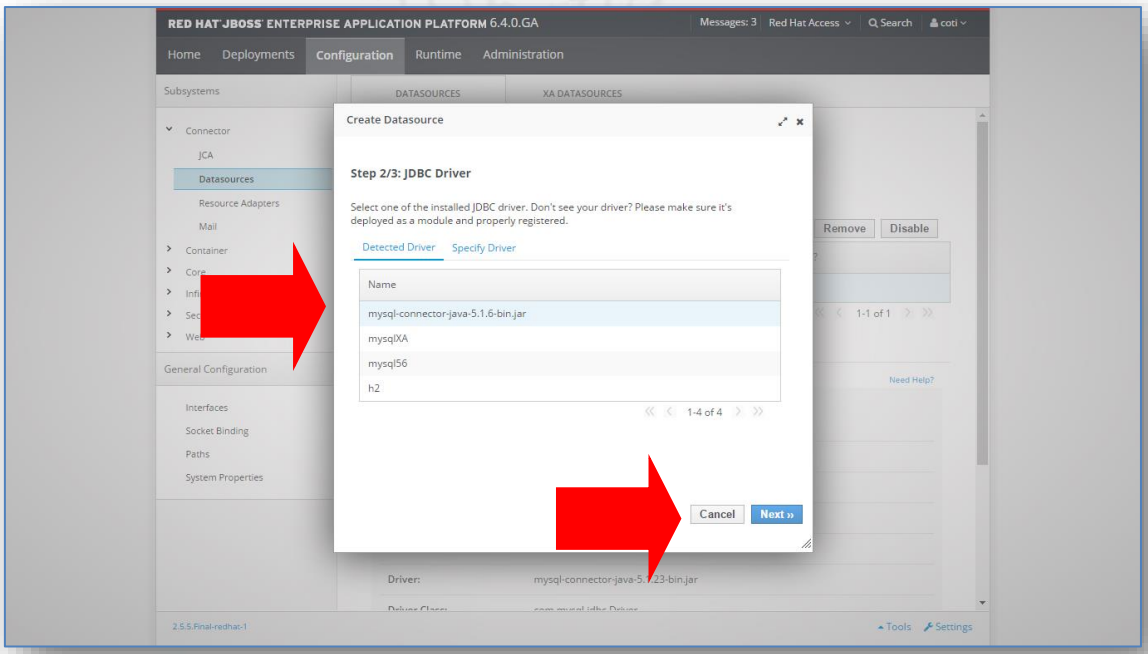
Clicar em "add" para adicionar o JNDI...



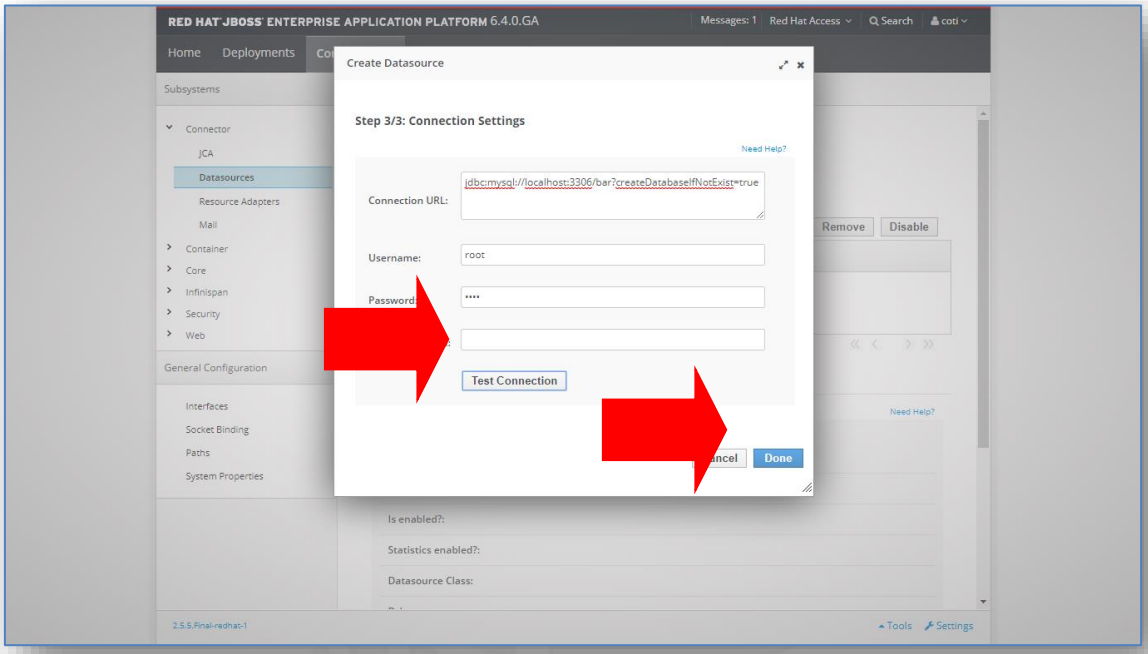
Colocar um nome para conexão -> Digitar o JNDI name "java:/jdbc/bar" -> Next...



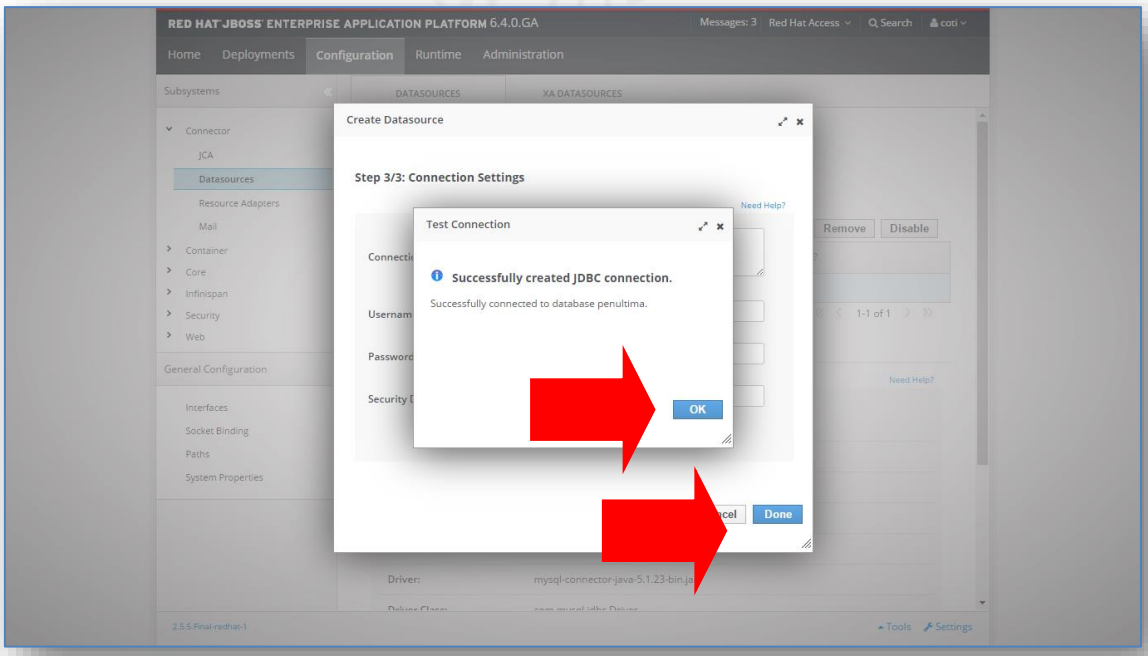
Marcar mysql -> Next...



Digitar as configurações de conexão -> Test Conection...
URL:
jdbc:mysql://localhost:3306/bar?createDatabaseIfNotExist=true



Conexão ok -> ok -> done...



```
package persistence;

import java.util.List;

import javax.ejb.Stateless;
import javax.enterprise.context.Dependent;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;

import entity.Consumidor;

@Stateless
@Dependent
public class ConsumidorDao {

	@PersistenceContext(unitName="bar")
	EntityManager em;

	public void create(Consumidor c){
		em.persist(c);
	}
	public void update(Consumidor c){
		em.merge(c);
	}
	public void delete(Integer cod){
		em.remove(em.getReference(Consumidor.class, cod));
	}
	public List<Consumidor> findAll(){
		return
em.createNamedQuery("mostra").getResultList();
	}
	public Consumidor findByCode(Integer cod){
		return em.find(Consumidor.class, cod);
	}
}
```

```
package manager;

import java.io.Serializable;
import java.util.List;

import javax.enterprise.context.RequestScoped;
import javax.faces.application.FacesMessage;
```

```
import javax.faces.bean.ManagedBean;
import javax.faces.context.FacesContext;
import javax.inject.Inject;

import entity.Consumidor;
import persistence.ConsumidorDao;


@ManagedBean(name="mb")
@RequestScoped
public class ManagerBean  implements Serializable{

	private static final long serialVersionUID = 1L;

	@Inject
	private Consumidor consumidor; //injecao de dependencia que permite new
	@Inject
	private ConsumidorDao dao;  //new nÃ£o
	private List<Consumidor> consumidores;

	public void gravar(){
		FacesContext fc = FacesContext.getCurrentInstance();
		try{
			dao.create(consumidor);
			 consumidor =new Consumidor();
  fc.addMessage(null, new FacesMessage("Dados Gravados ..."));
		}catch(Exception ex){
			fc.addMessage(null, new FacesMessage("error :" + ex.getMessage()));
		}
	}

	public List<Consumidor> getConsumidores() {
		consumidores = dao.findAll();
		return consumidores;
	}

public void setConsumidores(List<Consumidor> consumidores) {
		this.consumidores = consumidores;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
```

```
	}
	public Consumidor getConsumidor() {
		return consumidor;
	}
	public void setConsumidor(Consumidor consumidor) {
		this.consumidor = consumidor;
	}
	public ConsumidorDao getDao() {
		return dao;
	}
	public void setDao(ConsumidorDao dao) {
		this.dao = dao;
	}
}
```

---

beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://xmlns.jcp.org/xml/ns/javaee"
	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/beans_1_1.xsd"
	version="1.1" bean-discovery-mode="annotated">


</beans>
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <display-name>JbossServa</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
```

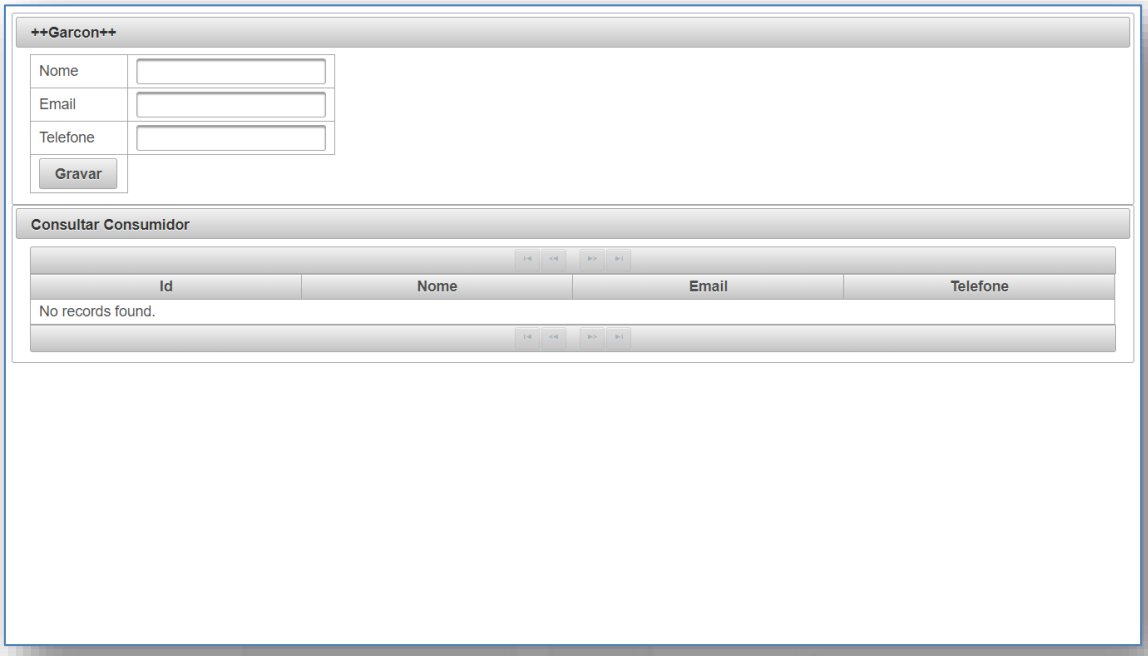
```
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>
</web-app>
```

---

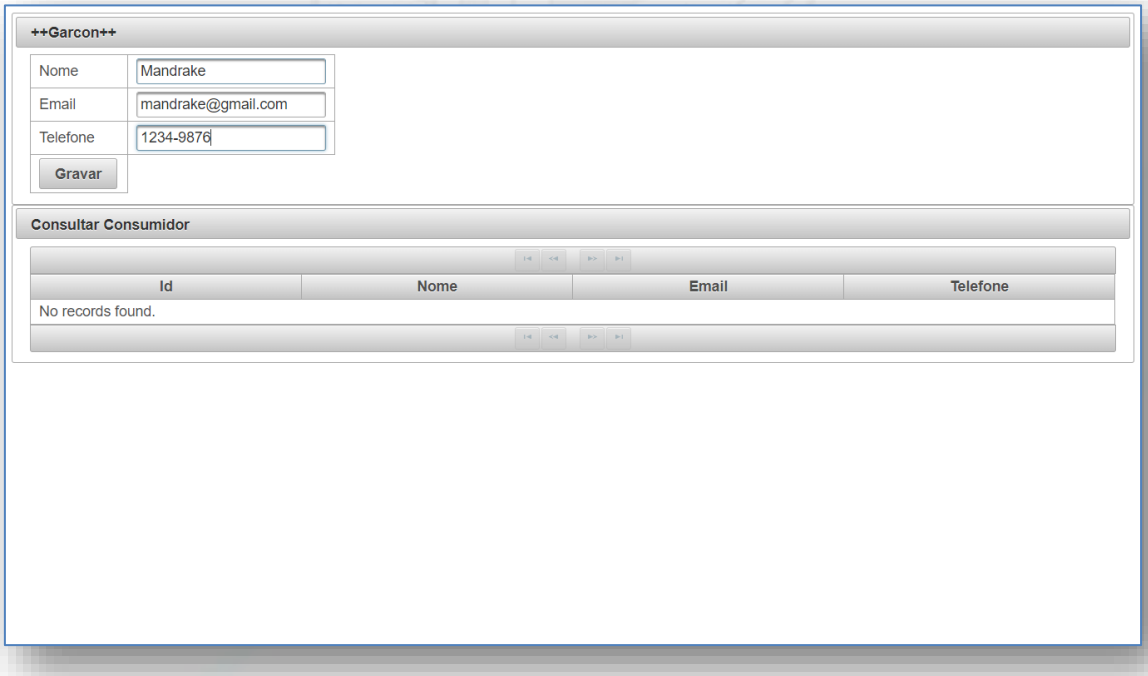
sistema.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head></h:head>

<h:body>
  <p:panel  header="++Garcon++">
    <h:form id="form1">
       <p:panelGrid columns="2">
          <h:outputText value="Nome"></h:outputText>
          <p:inputText value="#{mb.consumidor.nome}"
required="true"></p:inputText>
          <h:outputText value="Email"></h:outputText>
          <p:inputText value="#{mb.consumidor.email}"
required="true"></p:inputText>
          <h:outputText value="Telefone"></h:outputText>
          <p:inputText value="#{mb.consumidor.telefone}"
required="true"></p:inputText>
          <p:commandButton value="Gravar" action="#{mb.gravar}"
```

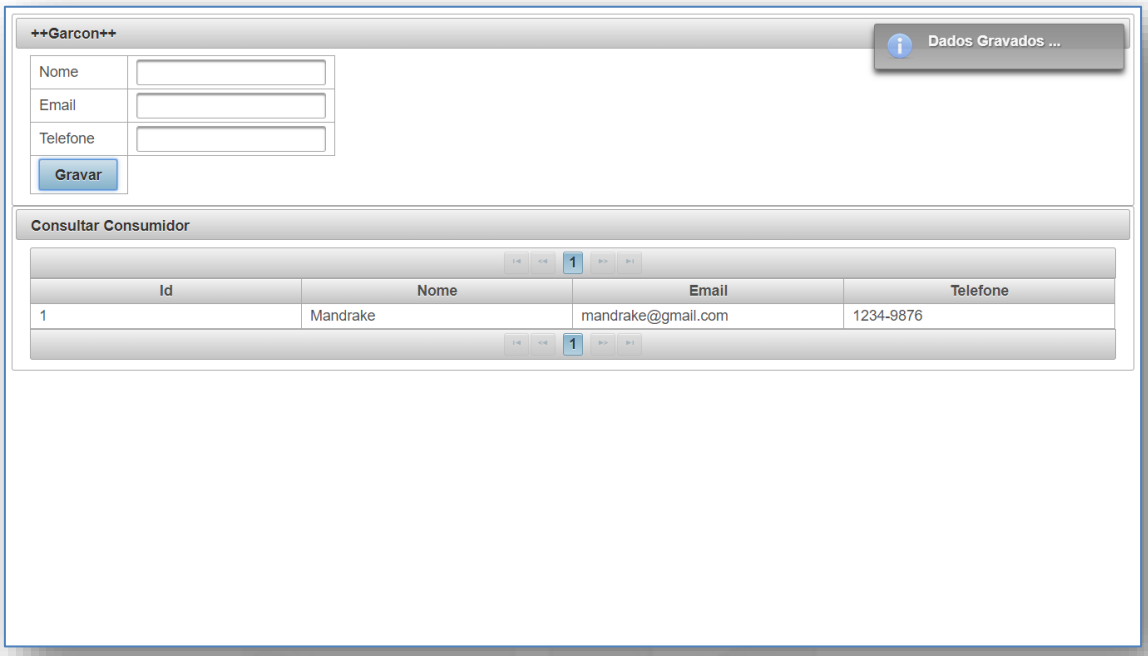
```
                                                    ajax="true"
update=":form1, :form2"></p:commandButton>
        </p:panelGrid>
        <p:growl></p:growl>
    </h:form>
</p:panel>
<p:panel header="Consultar Consumidor">
<h:form id="form2">
    <p:dataTable value="#{mb.consumidores}" var="linha" rows="3"
      paginator="true">
       <p:column headerText="Id">
           <h:outputText value="#{linha.id}" />
        </p:column>
        <p:column headerText="Nome">
           <h:outputText value="#{linha.nome}" />
        </p:column>
         <p:column headerText="Email">
            <h:outputText value="#{linha.email}" />
        </p:column>
        <p:column headerText="Telefone">
            <h:outputText value="#{linha.telefone}" />
         </p:column>
    </p:dataTable>
    </h:form>
</p:panel>
</h:body>
</html>
```



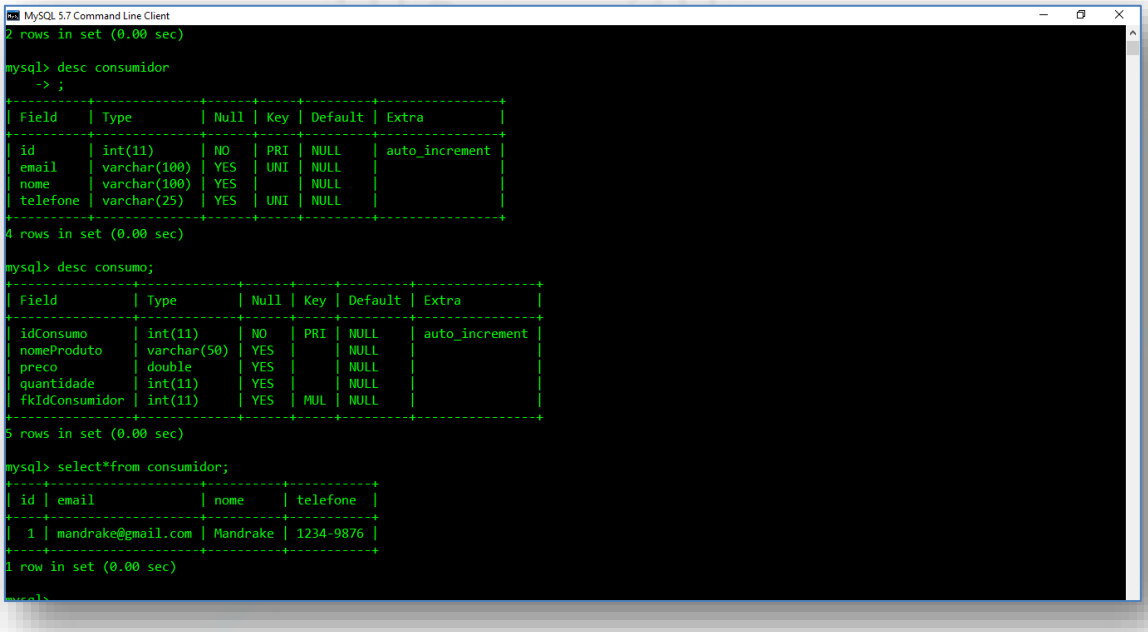
Cadastrando...



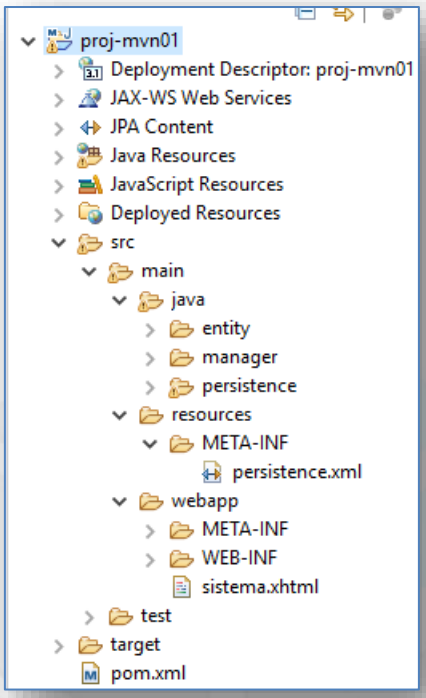
Gravado...



No banco...



Estrutura do projeto:



Maven:

Ant / Maven --> Ant (Módulo antigo), arrastando jar, apontando para jar(set classpath (Eclipse), Ant Botar o Jar sempre no Seu Projeto... Projeto Fica Pesado Ao Extremo

Maven ... (pom.xml) --> configuração dequal e de onde você irá trabalhar com o Java .

Apache Maven:

Apache Maven, ou simplesmente **Maven**, é uma ferramenta de automação de compilação utilizada primariamente em projetos Java. Ela é similar à ferramenta Ant, mas é baseada em conceitos e trabalhos diferentes em um modo diferente. Também é utilizada para construir e gerenciar projetos escritos em C#, Ruby, Scala e outras linguagens. O projeto Maven é hospedado pela Apache Software Foundation, que fazia parte do antigo Projeto Jakarta.

O Maven utiliza um arquivo XML (POM) para descrever o projeto de software sendo construído, suas dependências sobre módulos e componentes externos, a ordem de compilação, diretórios e plug-ins necessários. Ele vem com objetivos pré-definidos para realizar certas tarefas bem definidas como compilação de código e seu empacotamento.

O Maven baixa bibliotecas Java e seus plug-ins dinamicamente de um ou mais repositórios, como o Maven 2 Central Repository, e armazena-os em uma área de cache local.[2] Este cache local de artefatos baixados pode também ser atualizado com artefatos criados por projetos locais. Repositórios públicos podem também ser atualizados.

O Maven é construído utilizando uma arquitetura baseada em plugin, que permite que ele faça uso de qualquer aplicação controlável através da entrada padrão. Teoricamente, isto permitiria qualquer um escrever plugins para fazer interface com ferramentas de construção (compiladores, ferramentas de teste de unidade, etc.) para qualquer outra linguagem. De fato, o suporte e uso para linguagens diferentes de Java tem sido mínimas. Atualmente existe um plugin para o framework .NET e é mantido, e um plugin nativoC/C++ é mantido para o Maven 2.

POM:

Este POM apenas define um identificador único para o projeto (coordenadas) e sua dependência do framework JUnit. No entanto, isto já é suficiente para a construção do projeto e execução dos testes de unidade associados ao projeto. O Maven realiza isto adotando a ideia de Convenção sobre configuração, isto é, ele fornece valores padrões para a configuração do projeto. A estrutura de diretórios de um projeto Maven idiomático normal possui as seguintes entradas de diretório:

Nome do diretório	Propósito
project home	Contém o pom.xml e todos os subdiretórios.
src/main/java	Contém o código-fonte Java entregável para o projeto.
src/main/resources	Contém os recursos entregáveis para o projeto, tais como arquivos de propriedades.
src/test/java	Contém as classes de teste (casos de teste JUnit ou TestNG, por exemplo) para o projeto.
src/test/resources	Contém os recursos necessários para teste.

Conceitos:

Project Object Model:

Um Project Object Model (POM), ou em português Modelo de Projeto de Objeto, fornece todas as configurações para um único projeto. A configuração geral cobre o nome do projeto, seu proprietário e suas dependências de outros projetos. Também pode configurar fases individuais do processo de construção, que são implementados como plugins. Por exemplo, pode configurar o plugin compilador para usar a versão 1.5 do Java para compilação, ou especificar o empacotamento do projeto mesmo que algumas unidades de teste falhem.

Projetos maiores poderiam ser divididos em vários módulos, ou sub-projetos, cada um com seu próprio POM. Cada um pode então escrever um POM raiz através do qual ele pode compilar todos os módulos com um único comando. POMs também podem herdar configuração de outros POMs. Todos os POMs herdam do Super POM por padrão. O Super POM fornece configuração padrão, tal como os diretórios fonte padrões, plugins padrões e assim por diante.

Plugins:

Há plugins Maven para construção, teste, gerenciamento de controle de fonte, execução de um servidor web, geração de arquivos de projeto Eclipse e muito mais. Plugins são introduzidos e configurados em uma seção <plugins> de um arquivo `pom.xml`. Alguns plugins básicos estão incluídos em todos os projetos por padrão e eles possuem configurações padrões sensíveis.

Plugins são o modo primário de estender o Maven. Desenvolver um plugin Maven pode ser feito através da extensão da classe org.apache.maven.plugin.AbstractMojo. Um exemplo simples é dado na página do Centro de Desenvolvedores de Plugin no site do Maven e mais exemplos detalhados são dados no *Apache Maven 3 Cookbook*. Código de exemplo e explicação para o plugin Maven para criar uma máquina virtual baseada em nuvem executando em um servidor de aplicação é dado no artigo *Automate development and management of cloud virtual machines*.

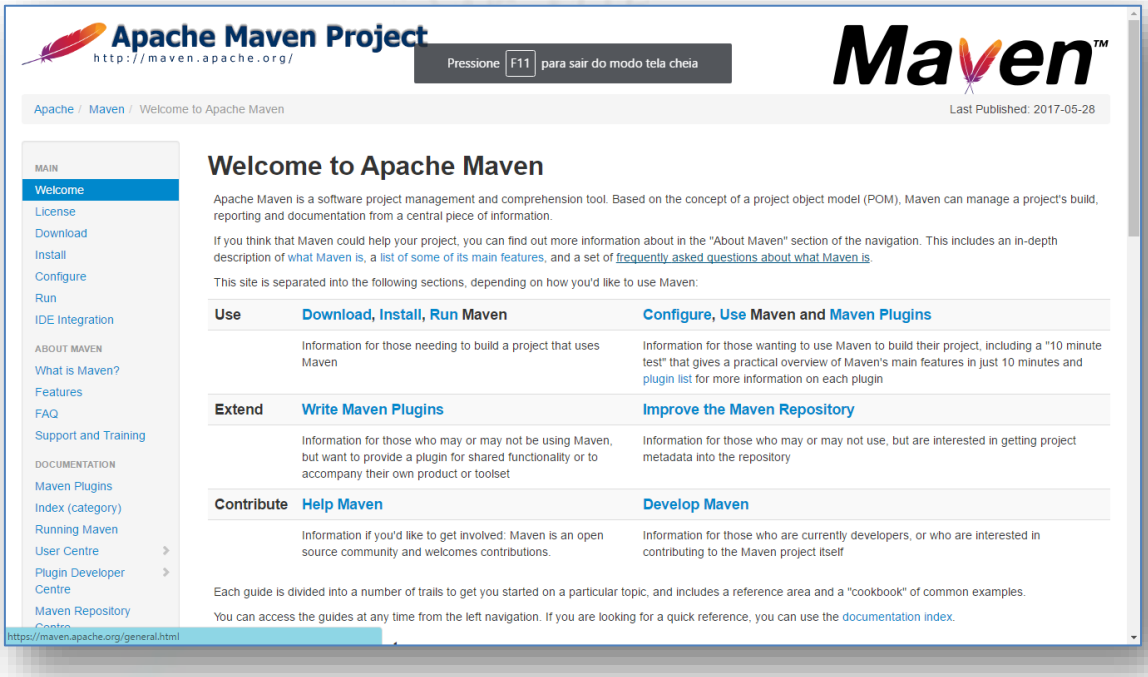
Dependências:

A seção de exemplo sugeriu ao mecanismo de manipulação de dependências do Maven. Digamos que um projeto que necessita da biblioteca Hibernate tem que declarar as coordenadas do projeto do Hibernate em seu POM. O Maven automaticamente baixará a dependência e as dependências que o próprio Hibernate precisa (chamadas dependências transitivas) e as armazenará em um repositório local do usuário. O Repositório Central do Maven 2 é usado por padrão para procurar bibliotecas, mas pode-se configurar os repositórios usados (e.g., repositórios privados de uma empresa) no POM.

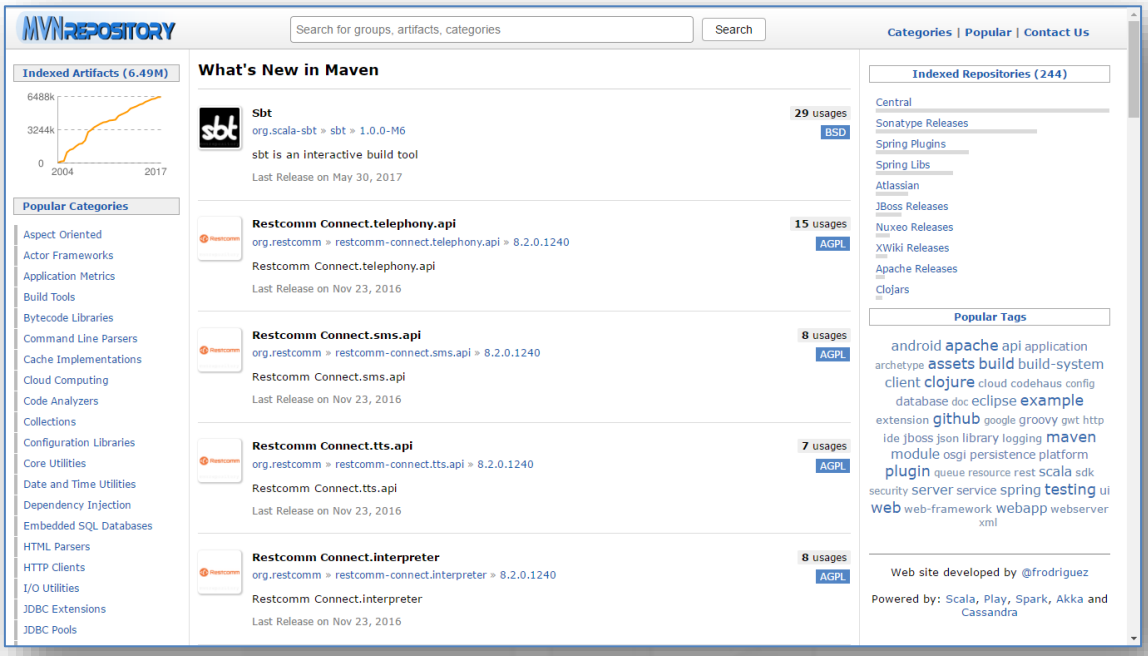
Há motores de busca, como a Central Maven, que podem ser usadas para encontrar coordenadas para diferentes bibliotecas e frameworks de código aberto.

Projetos desenvolvidos em uma única máquina podem depender um do outro através do repositório local. O repositório local é uma estrutura de pasta simples que age tanto como um cache para dependências baixadas e como um local de armazenamento centralizado para construir artefatos localmente. O comando `mvn install` constrói um projeto e coloca seus binários em um repositório local. Então, outros projetos podem utilizar este projeto por especificação de suas coordenadas em seus POMs.

https://maven.apache.org/

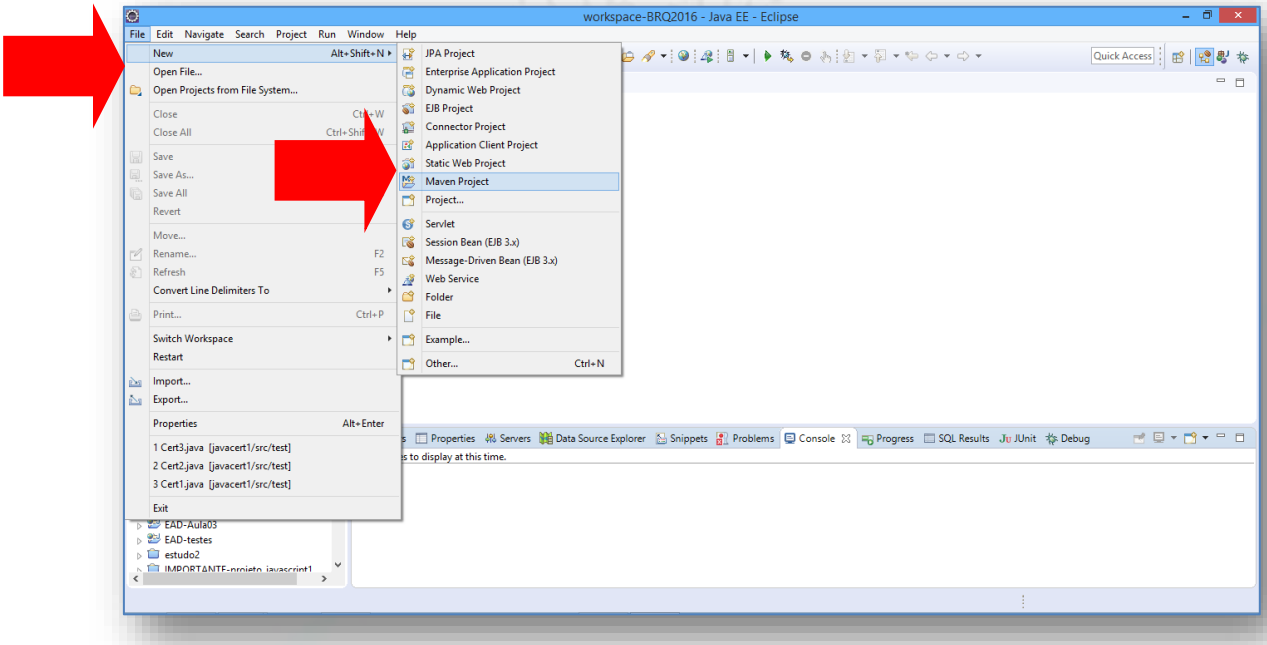


http://mvnrepository.com/

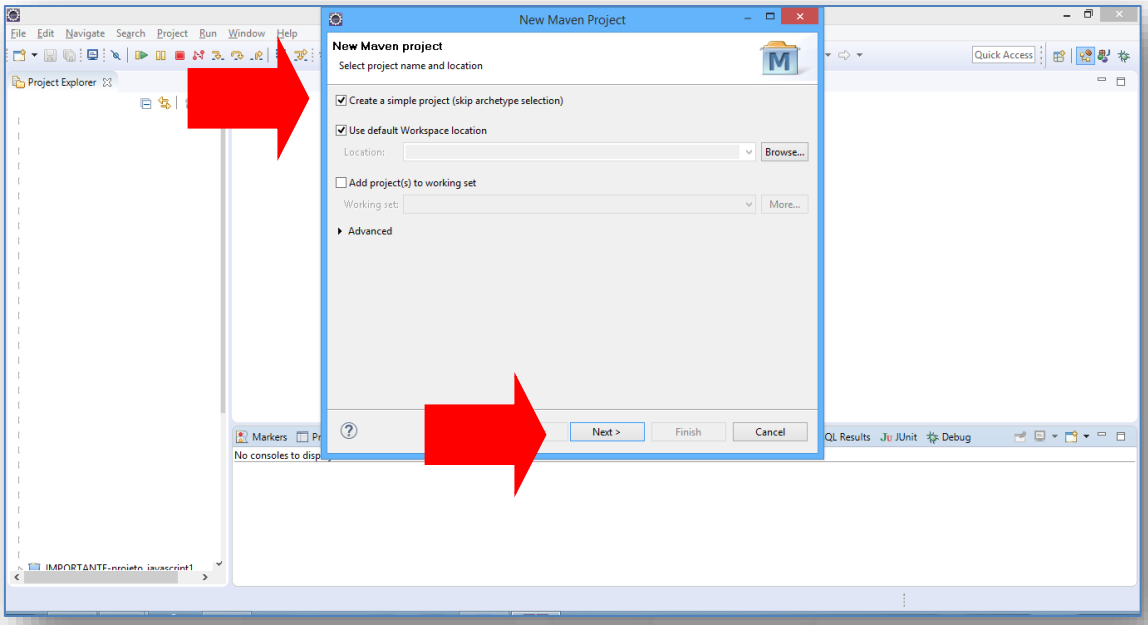


Para criar o projeto maven..

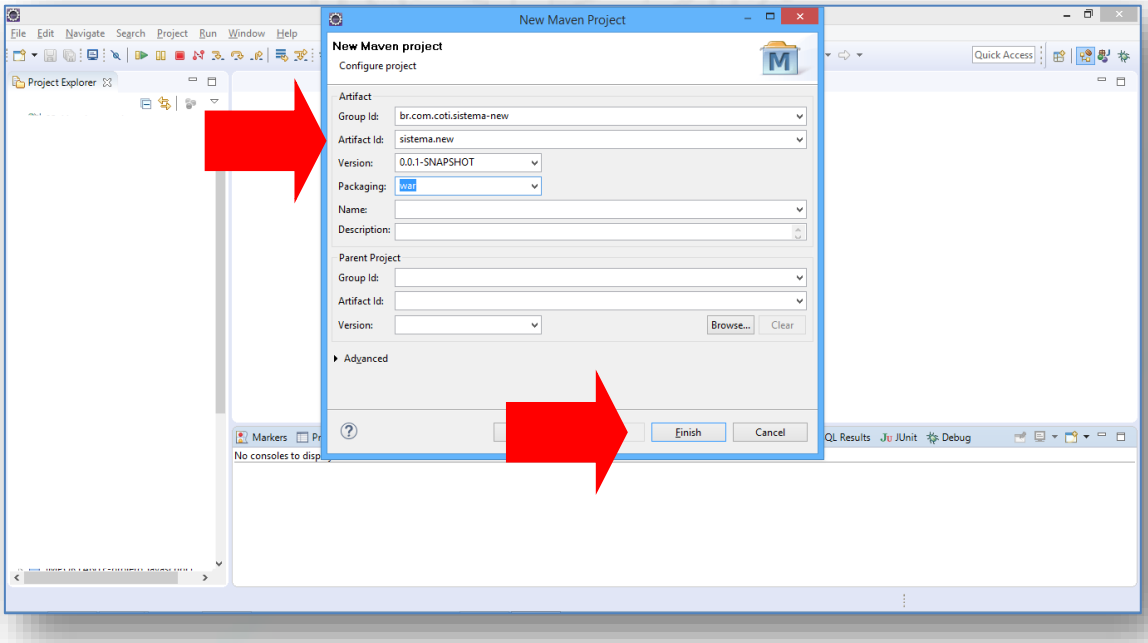
File -> new -> maven Project...



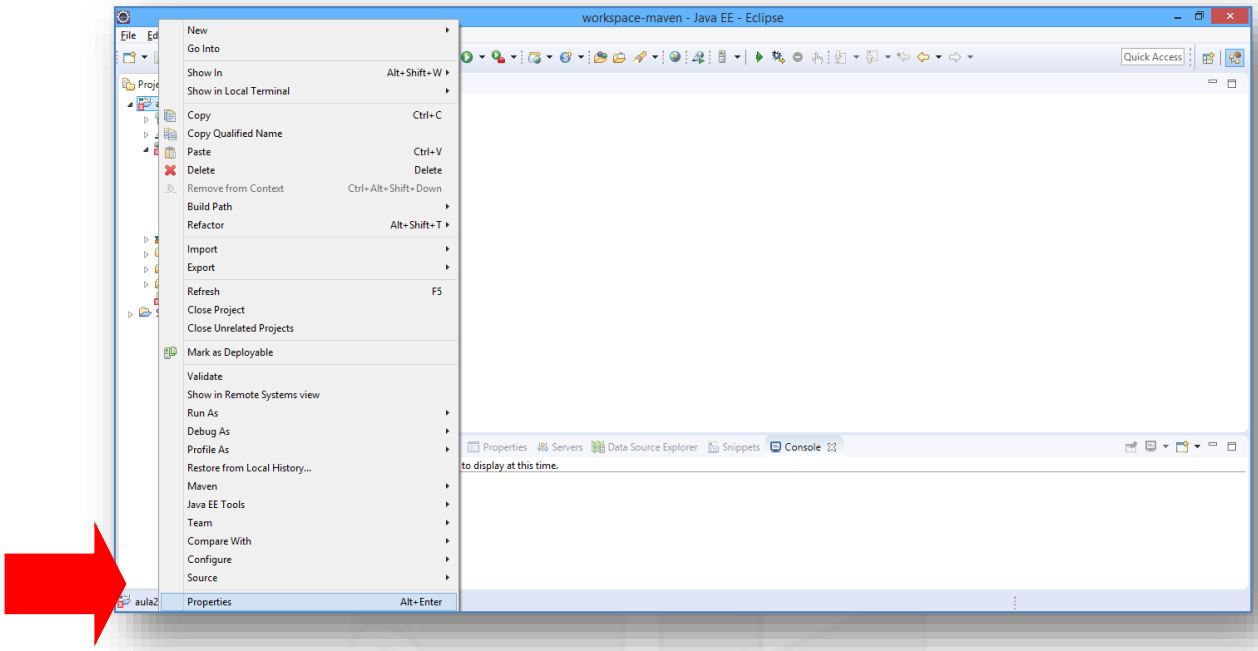
Marcar create a simple Project -> next...



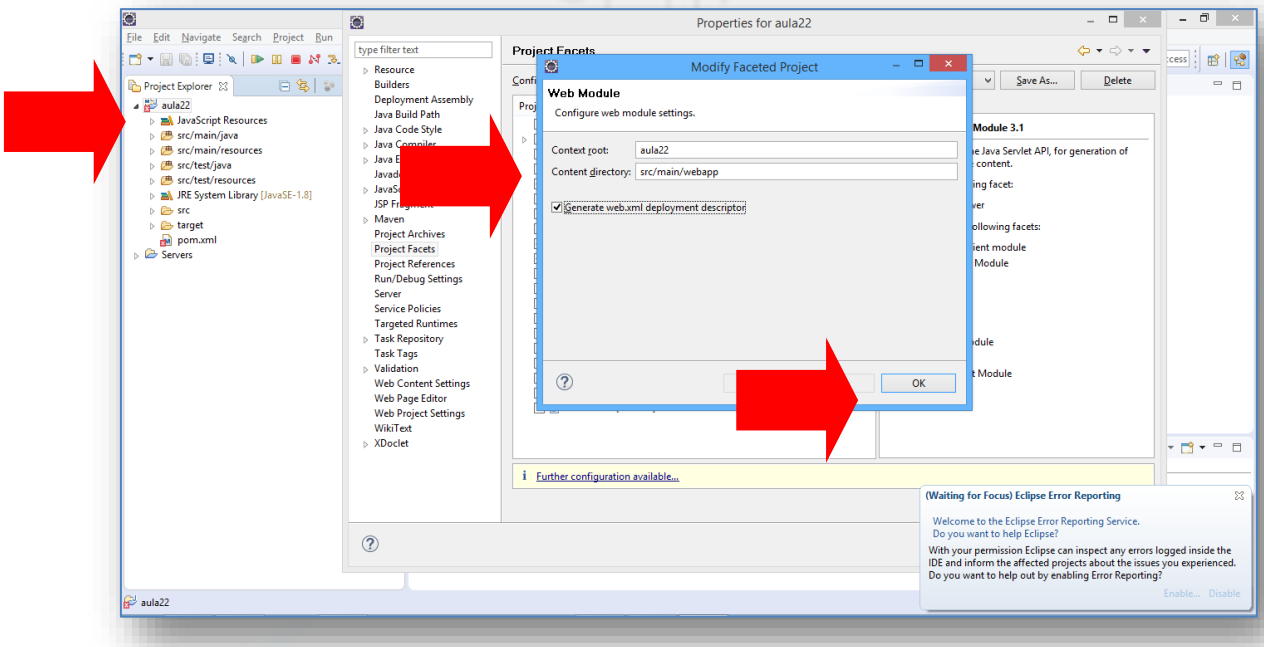
Colocar nome no group id -> artifact id -> packing war -> finish...



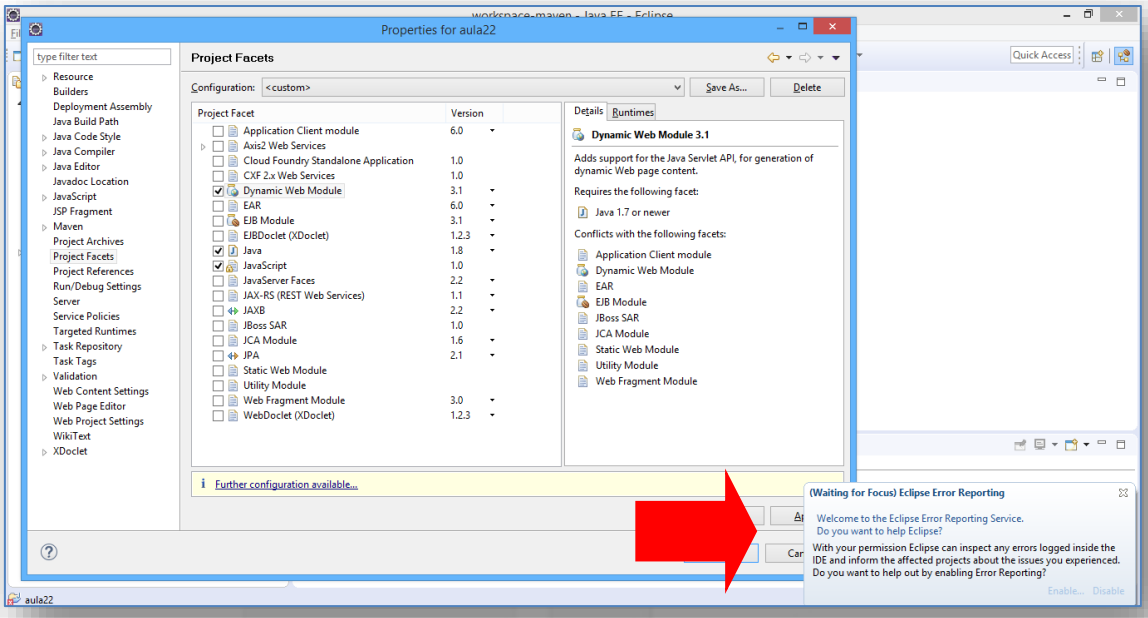
O projeto começará com erro. Precisa configurar...
Clicar no projeto com o botão direito -> Properties...



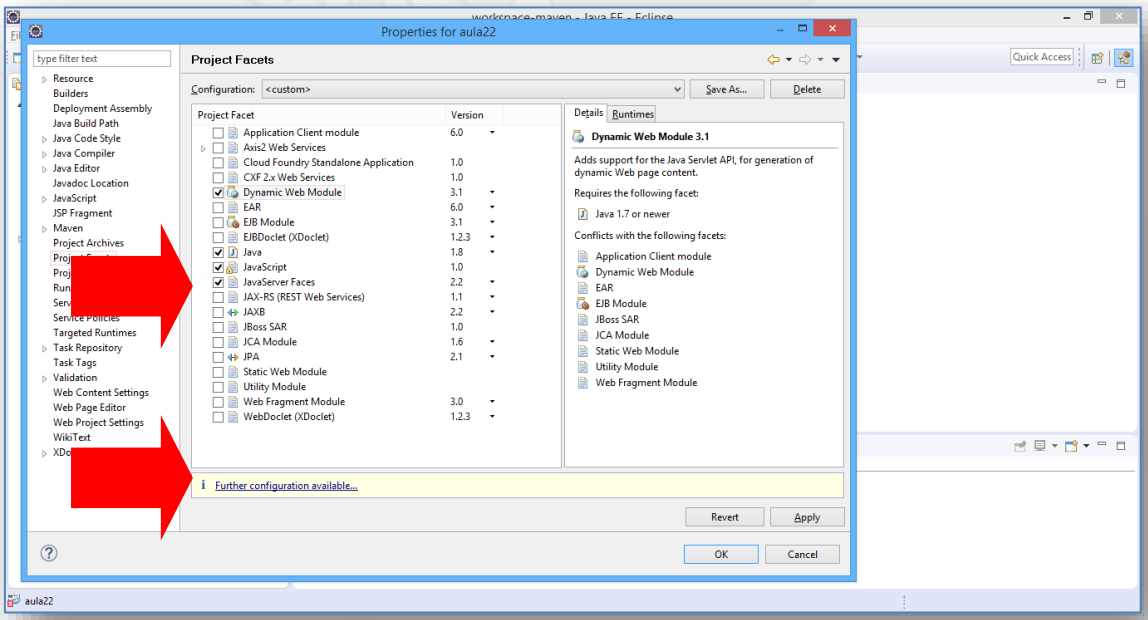
Clicar em Porject Facets -> alterar Java para 1.8 -> Apply -> Dynamic Web Module para 3.1 -> Desmarcar Dynamic -> Apply -> Marcar Dynamic novamente -> Clicar em Futher Configuration -> Alterar Content Directory para src/main/webapp -> Marcar generate web.xml -> ok



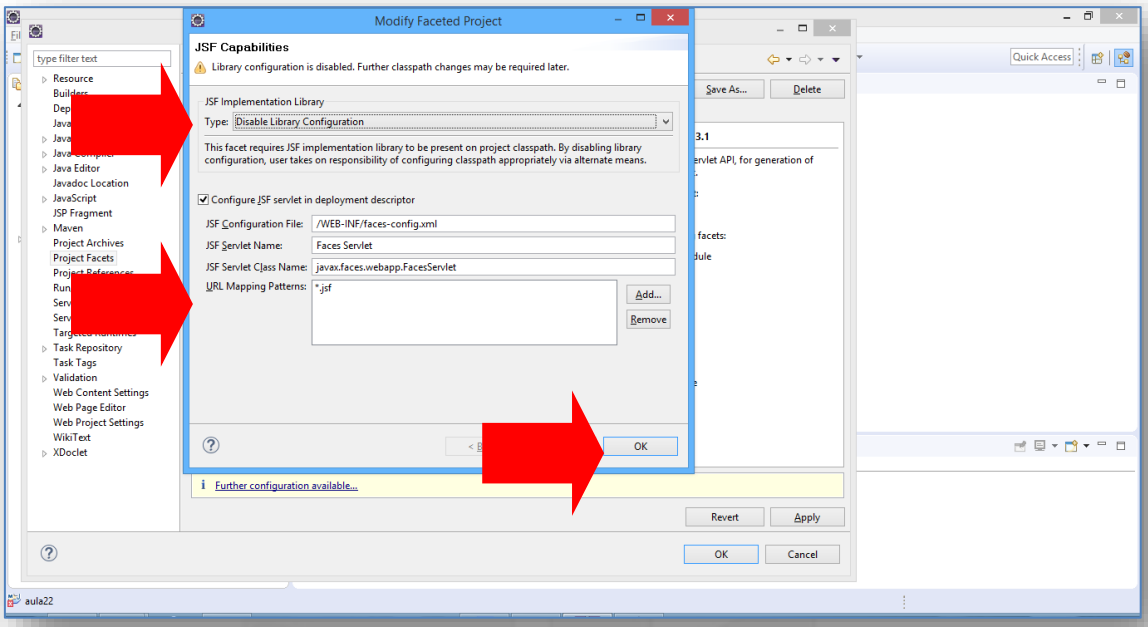
Clicar em Apply...



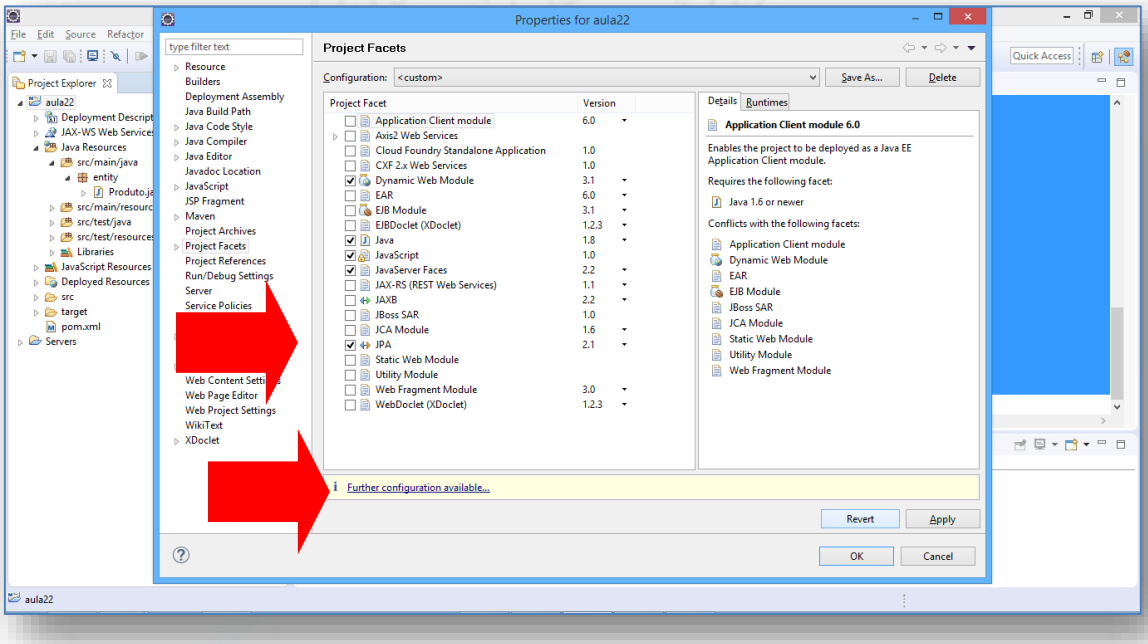
Marcar Java Server Faces -> Futher Configuration...



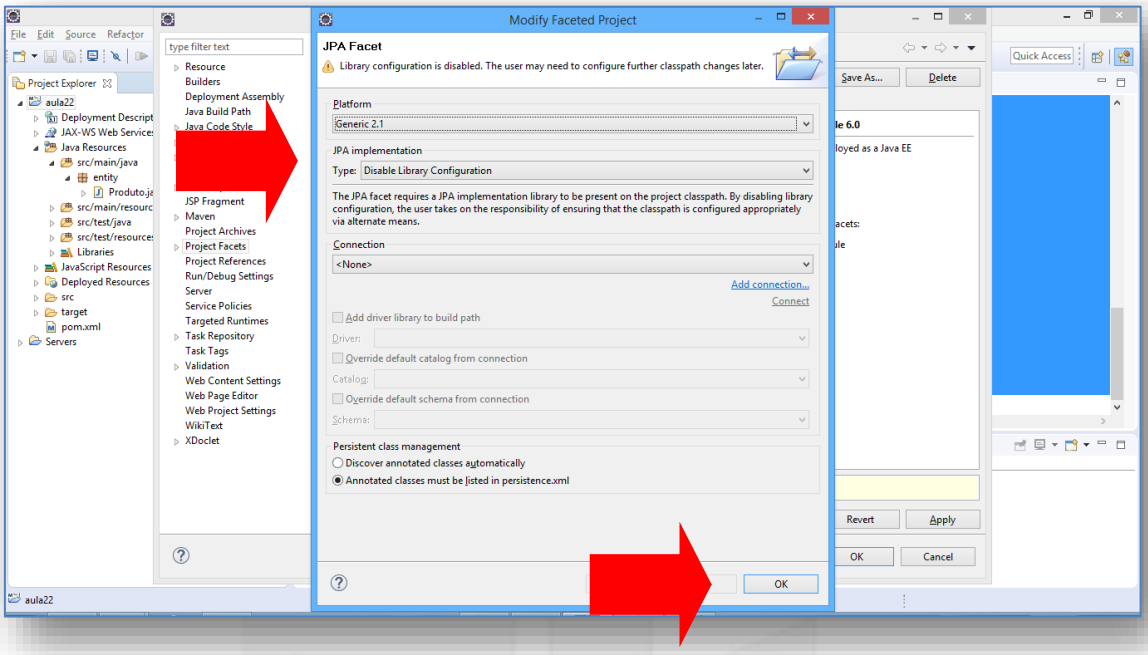
Em Type, clicar em Disable Library... -> URL Mapping Patterns -> Add -> *.jsf -> ok



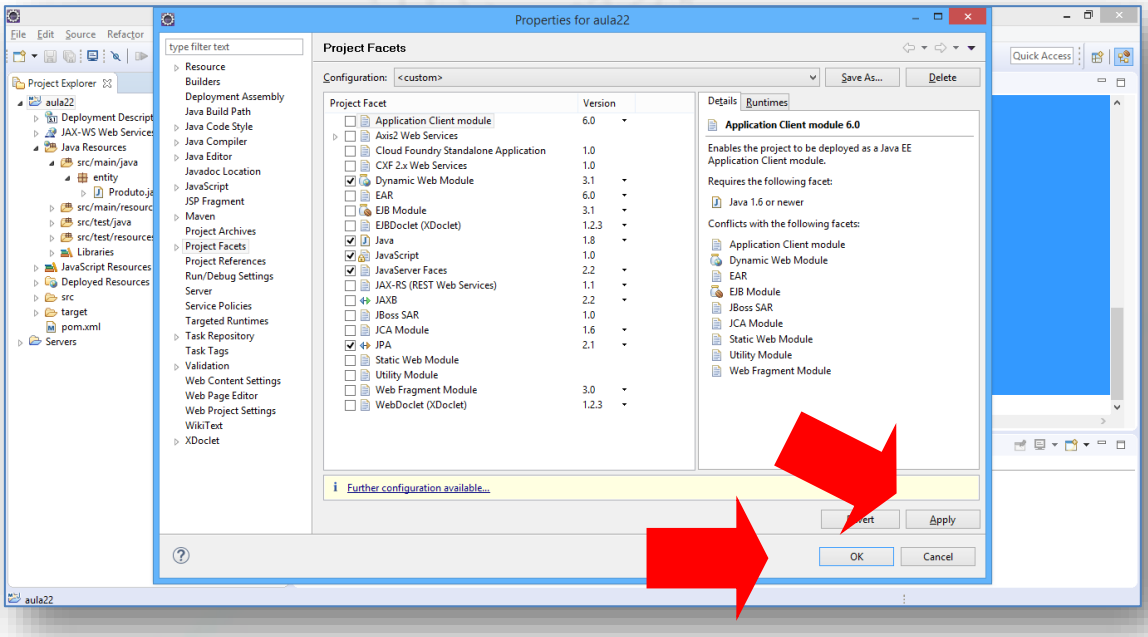
Marcar JPA -> Futher Configuration...



Em Type, clicar em Disable Library -> ok...



Apply -> ok...

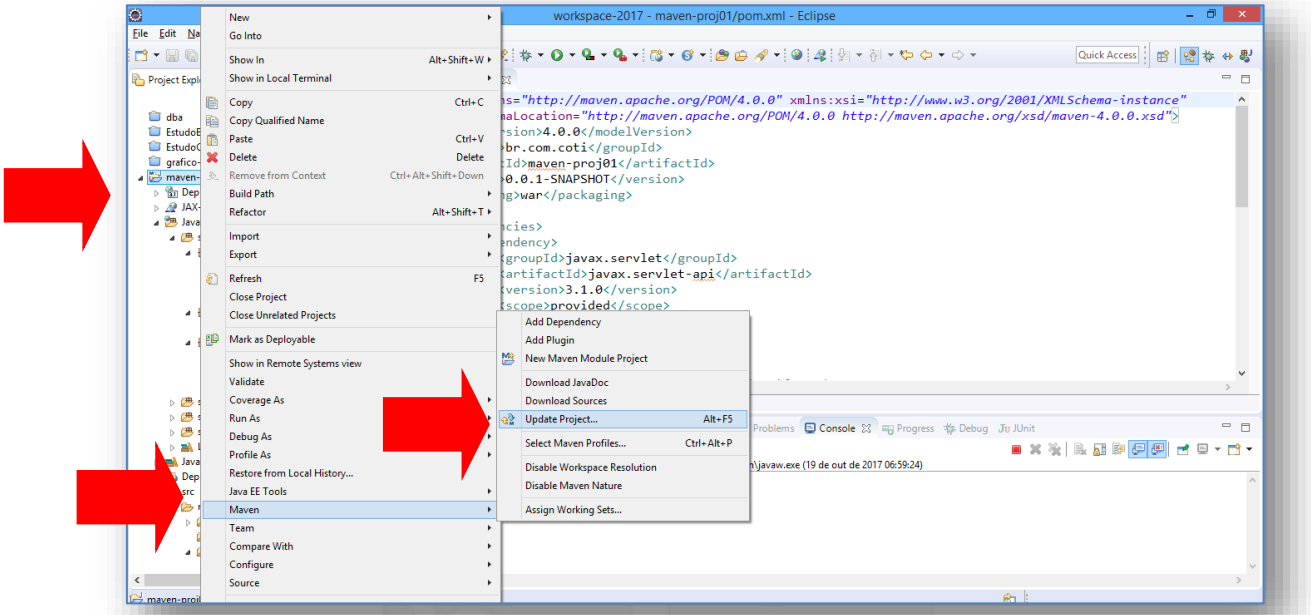


POM.xml

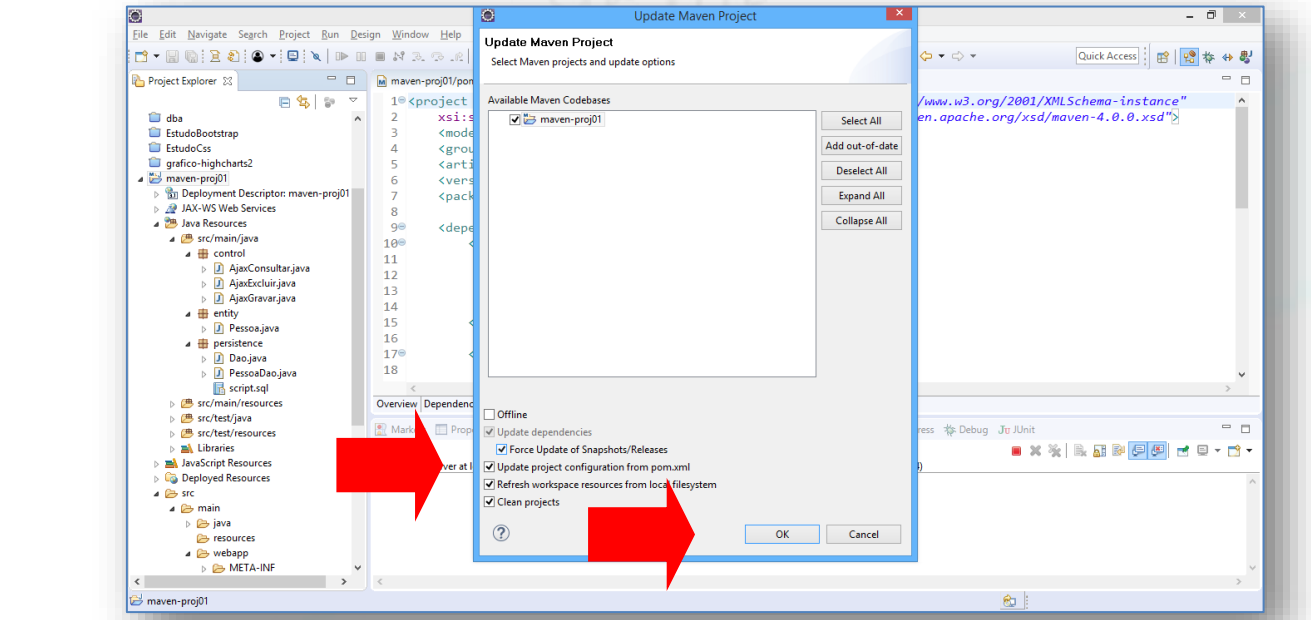
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>br.com.coti.projeto1</groupId>
  <artifactId>proj-mvn01</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>

 <dependencies>
 <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.43</version>
</dependency>

<dependency>
    <groupId>com.sun.faces</groupId>
    <artifactId>jsf-api</artifactId>
    <version>2.2.13</version>
</dependency>
<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
    <version>3.4</version>
</dependency>

<dependency>
    <groupId>com.sun.faces</groupId>
    <artifactId>jsf-impl</artifactId>
    <version>2.2.13</version>
</dependency>

<dependency>
    <groupId>org.primefaces</groupId>
    <artifactId>primefaces</artifactId>
    <version>6.0</version>
</dependency>

<dependency>
    <groupId>org.hibernate</groupId>
```

```
    <artifactId>hibernate-core</artifactId>
    <version>5.2.5.Final</version>
</dependency>

 <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-entitymanager</artifactId>
    <version>5.2.5.Final</version>
</dependency>

<dependency>
    <groupId>org.hibernate.common</groupId>
    <artifactId>hibernate-commons-annotations</artifactId>
    <version>5.0.1.Final</version>
</dependency>

<dependency>
    <groupId>org.hibernate.javax.persistence</groupId>
    <artifactId>hibernate-jpa-2.1-api</artifactId>
    <version>1.0.0.Final</version>
</dependency>

<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
    <scope>provided</scope>
</dependency>

 </dependencies>
  <build>
     <plugins>
        <plugin>
              <groupId>org.apache.maven.plugins</groupId>
               <artifactId>maven-compiler-plugin</artifactId>
               <version>3.1</version>
                <configuration>
                     <source>1.8</source>
                     <target>1.8</target>
               </configuration>
        </plugin>
     </plugins>
  </build>

</project>
```

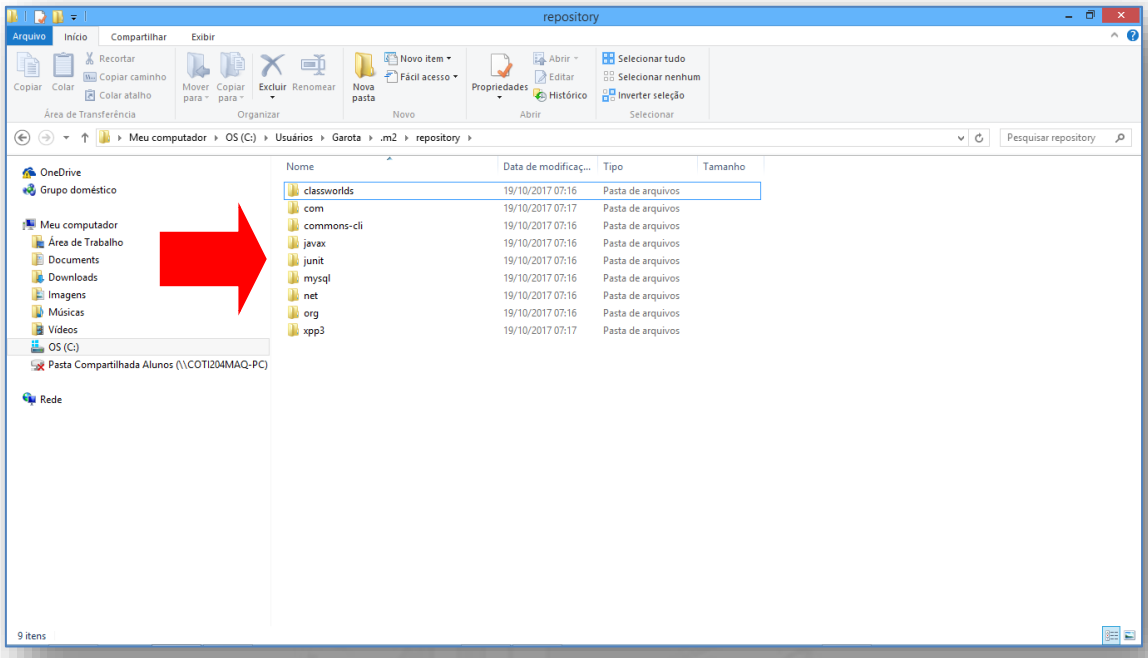
Para fazer um update nas bibliotecas baixadas...
Clicar no projeto com o botão direito -> Maven -> Update Project...



Clicar em Force Update -> ok... E aguardar atualizar...



Para encontrar onde foram baixadas as libs, ir no workspace onde se encontra o projeto maven, voltar uma pasta (está na pasta anterior ao workspace). Procurar a pasta ".m2". Atenção! Ela pode estar oculta.




```
package entity;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="cliente")
public class Cliente implements Serializable{

    private static final long serialVersionUID = 1L;
  @Id
 @GeneratedValue(strategy=GenerationType.IDENTITY)
```

```
	private Integer idCliente;
	@Column(length=50)
	private String  nome;
	@Column(length=50, unique=true)
	private String email;

	public Cliente() {

	}

	public Cliente(Integer idCliente, String nome, String
email) {
		this.idCliente = idCliente;
		this.nome = nome;
		this.email = email;
	}

	@Override
	public String toString() {
		return "Cliente [idCliente=" + idCliente + ", nome="
+ nome + ", email=" + email + "]";
	}

	public Integer getIdCliente() {
		return idCliente;
	}
	public void setIdCliente(Integer idCliente) {
		this.idCliente = idCliente;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
	}
}
```

Mudar a pasta “META-INF” para “src/main/resource”

Persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1"
xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd">
      <persistence-unit name="projmarvel">

<provider>org.hibernate.jpa.HibernatePersistenceProvider</provid
er>
        <class>entity.Cliente</class>

       <properties>
         <property name="javax.persistence.jdbc.driver"
value="com.mysql.jdbc.Driver"  />
         <property name="javax.persistence.jdbc.url"

value="jdbc:mysql://localhost:3306/bd02?createDatabaseIfNotExist
=true"  />
          <property name="javax.persistence.jdbc.user"
value="root"  />
          <property name="javax.persistence.jdbc.password"
value="coti"  />
        <property name="javax.dialect"
value="org.hibernate.dialect.MySQL5InnoDBDialect"  />
         <property name="hibernate.show_sql"       value="true"
/>
         <property name="hibernate.format_sql"
value="true"  />
          <property name="hibernate.hbm2ddl.auto" value="update"
/>
  </properties>

     </persistence-unit>
</persistence>
```

```
package persistence;

import java.util.List;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.EntityTransaction;
import javax.persistence.Persistence;

import entity.Cliente;

public class ClienteDao {

	EntityManager manager;
	EntityTransaction transaction;

	public ClienteDao() {
		EntityManagerFactory factory =
Persistence.createEntityManagerFactory("projmarvel");
		manager = factory.createEntityManager();
	}

	public void create(Cliente c) throws Exception{
	   transaction = manager.getTransaction();
		transaction.begin();
		   manager.persist(c);
	     transaction.commit();
	 }

	public void update(Cliente c) throws Exception{
		transaction = manager.getTransaction();
		     transaction.begin();
			manager.merge(c);
		   transaction.commit();
		}

	public void delete(Cliente c) throws Exception{
		transaction = manager.getTransaction();
		     transaction.begin();
			manager.remove(c);
		   transaction.commit();
		}
```

```
	public Cliente findByCode(Integer cod){
		return manager.find(Cliente.class,cod);
	}

	public List<Cliente> findAll(){
		return (List<Cliente>)
				manager.createQuery("select  c from
Cliente c").getResultList();
	}

	public static void main(String[] args) {
		try{
			Cliente c = new
Cliente(null,"orapoispois","ora@gmail.com");
			ClienteDao dao =new ClienteDao();
			 dao.create(c);
			  System.out.println("Dados Gravados ..." + c);
		}catch(Exception ex){
			System.out.println("Error :" + ex.getMessage());
		}
	}
}
```

No console...

```
dez 04, 2017 11:01:57 AM
org.hibernate.jpa.internal.util.LogHelper
logPersistenceUnitInformation
INFO: HHH000204: Processing PersistenceUnitInfo [
	name: projmarvel
	...]
dez 04, 2017 11:01:57 AM org.hibernate.Version logVersion
INFO: HHH000412: Hibernate Core {5.2.5.Final}
dez 04, 2017 11:01:57 AM org.hibernate.cfg.Environment <clinit>
INFO: HHH000206: hibernate.properties not found
dez 04, 2017 11:01:57 AM
org.hibernate.annotations.common.reflection.java.JavaReflectionM
anager <clinit>
INFO: HCANN000001: Hibernate Commons Annotations {5.0.1.Final}
```

```
dez 04, 2017 11:01:58 AM
org.hibernate.engine.jdbc.connections.internal.DriverManagerConn
ectionProviderImpl configure
WARN: HHH10001002: Using Hibernate built-in connection pool (not
for production use!)
dez 04, 2017 11:01:58 AM
org.hibernate.engine.jdbc.connections.internal.DriverManagerConn
ectionProviderImpl buildCreator
INFO: HHH10001005: using driver [com.mysql.jdbc.Driver] at URL
[jdbc:mysql://localhost:3306/bd02?createDatabaseIfNotExist=true]
dez 04, 2017 11:01:58 AM
org.hibernate.engine.jdbc.connections.internal.DriverManagerConn
ectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: {user=root,
password=****}
dez 04, 2017 11:01:58 AM
org.hibernate.engine.jdbc.connections.internal.DriverManagerConn
ectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
dez 04, 2017 11:01:58 AM
org.hibernate.engine.jdbc.connections.internal.PooledConnections
<init>
INFO: HHH000115: Hibernate connection pool size: 20 (min=1)
Mon Dec 04 11:01:58 BRST 2017 WARN: Establishing SSL connection
without server's identity verification is not recommended.
According to MySQL 5.5.45+, 5.6.26+ and 5.7.6+ requirements SSL
connection must be established by default if explicit option
isn't set. For compliance with existing applications not using
SSL the verifyServerCertificate property is set to 'false'. You
need either to explicitly disable SSL by setting useSSL=false,
or set useSSL=true and provide truststore for server certificate
verification.
dez 04, 2017 11:01:58 AM org.hibernate.dialect.Dialect <init>
INFO: HHH000400: Using dialect:
org.hibernate.dialect.MySQL5Dialect
dez 04, 2017 11:02:00 AM
org.hibernate.resource.transaction.backend.jdbc.internal.DdlTran
sactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess
[org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator
$ConnectionProviderJdbcConnectionAccess@2a640157] for (non-JTA)
DDL execution was not in auto-commit mode; the Connection 'local
transaction' will be committed and the Connection will be set
into auto-commit mode.
Hibernate:
```

```
    insert
    into
        cliente
        (email, nome)
    values
        (?, ?)
Dados Gravados ...Cliente [idCliente=1, nome=orapoispois,
email=ora@gmail.com]
```

```
package manager;

import java.util.List;

import javax.enterprise.context.RequestScoped;
import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.context.FacesContext;

import entity.Cliente;
import persistence.ClienteDao;

@ManagedBean(name="mb")
@RequestScoped
public class ManagerBean {

      //Projeto (serviÃ§os) --> Android (gravar) , Angular1
     //Sistema venda On-line (Provedor )  on-line ... (front)
     //Relatorio
     //login ...
        //Forum ...
        //Chats ...
    //Controle de Estoque ...

      private Cliente cliente;
      private List<Cliente> clientes;

       public ManagerBean() {
         cliente  = new Cliente();
     }

     public Cliente getCliente() {
          return cliente;
     }
```

```
	public void setCliente(Cliente cliente) {
		this.cliente = cliente;
	}

	public void gravar(){
		FacesContext fc =
FacesContext.getCurrentInstance();
		try{
		ClienteDao dao = new ClienteDao();
				dao.create(cliente);
				cliente = new Cliente();
    fc.addMessage(null, new FacesMessage("Dados Gravados ..."));
		 }catch(Exception ex){
		fc.addMessage(null, new FacesMessage("Error :" +
ex.getMessage())) ;
		}
	}

	public List<Cliente> getClientes() {
		try{
			ClienteDao dao = new ClienteDao();
			clientes = dao.findAll();
		}catch(Exception ex){
			ex.printStackTrace();
		}
		return clientes;
	}

	public void setClientes(List<Cliente> clientes) {
		this.clientes = clientes;
	}
}
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <servlet>
```

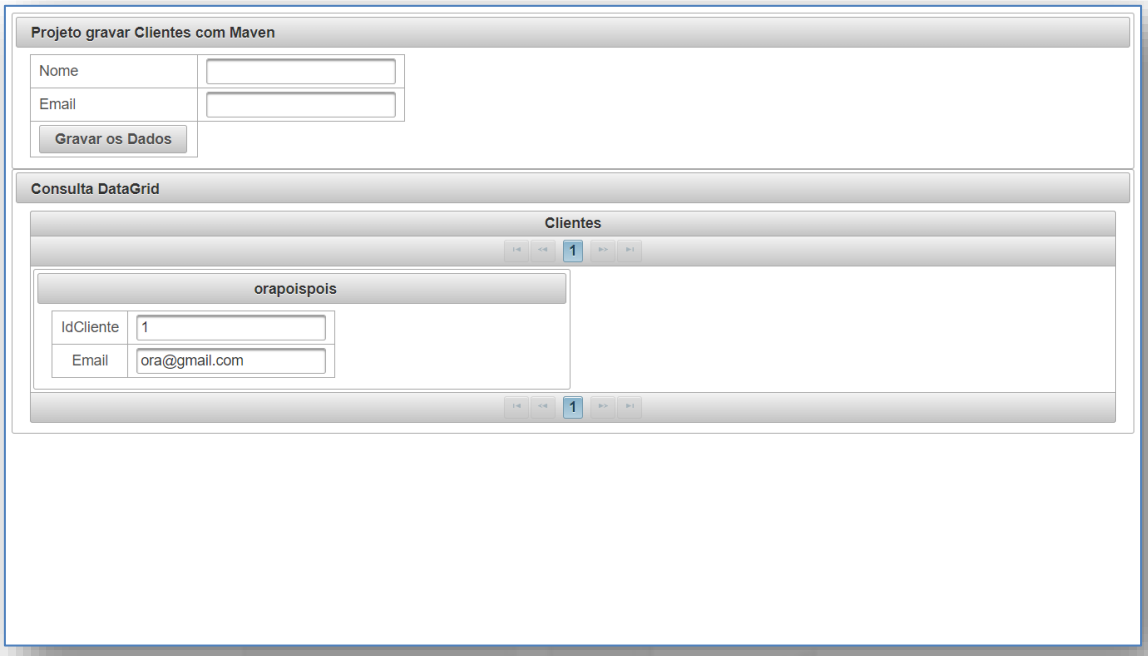
```
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>
</web-app>
```

---

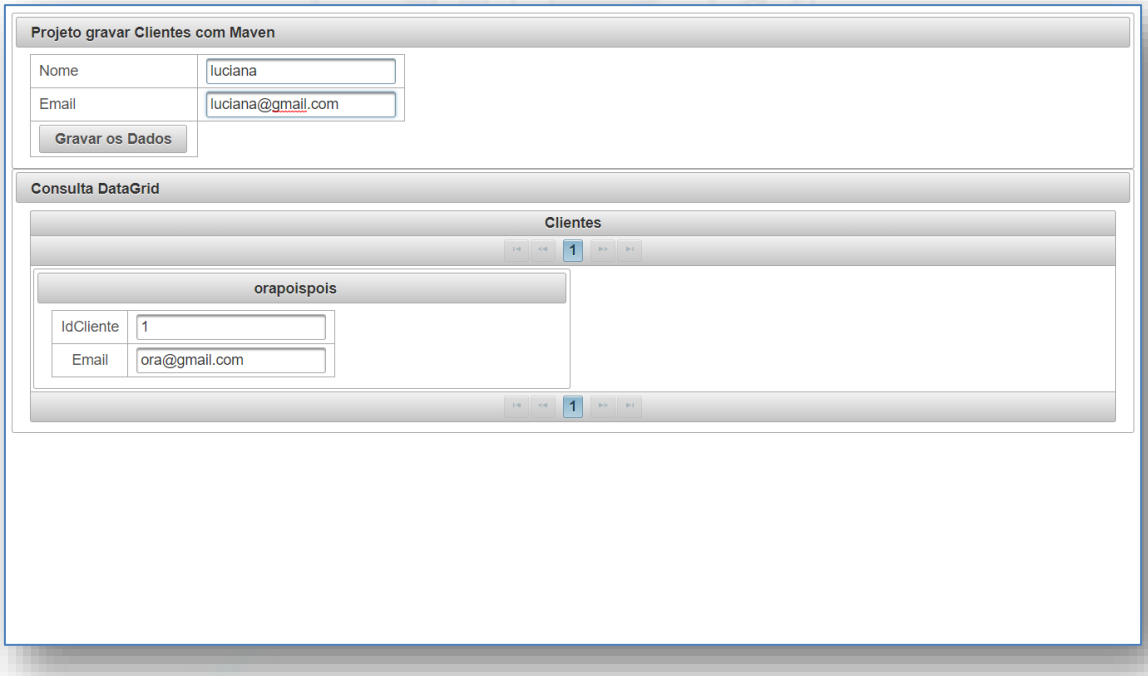
sistema.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">
  <h:head>
  </h:head>

<h:body>
   <p:panel header="Projeto gravar Clientes com Maven">
      <h:form id="form1">
       <p:panelGrid columns="2">
           <h:outputText value="Nome" />
           <p:inputText value="#{mb.cliente.nome}"
required="true"/>

           <h:outputText value="Email" />
           <p:inputText value="#{mb.cliente.email}"
required="true"/>
       <p:commandButton value="Gravar os Dados"
                                        action="#{mb.gravar}"
                        ajax="true" update=":form1, :form2"
                                        ></p:commandButton>
       </p:panelGrid>
       <p:growl></p:growl>
      </h:form>
   </p:panel>
```

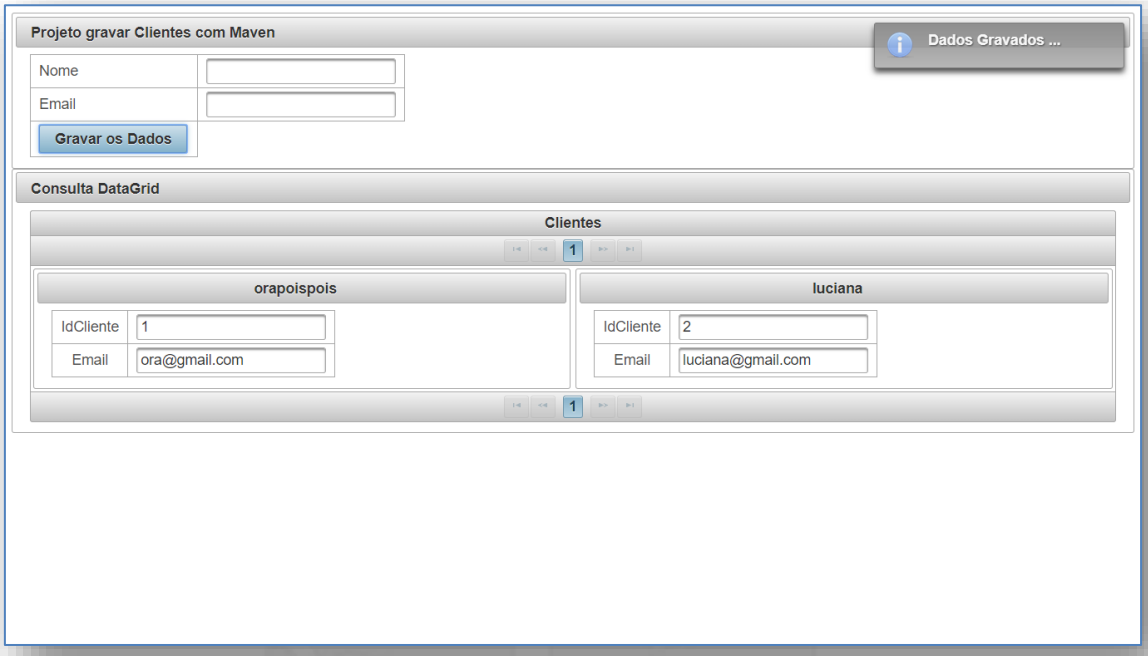
```
   <p:panel header="Consulta DataGrid">
    <h:form id="form2">
       <p:dataGrid   value="#{mb.clientes}"   columns="2"
layout="grid"
           rows="2" paginator="true"  var="cliente"  >
           <f:facet name="header">
                   Clientes
             </f:facet>
    <p:column>
          <p:panel header="#{cliente.nome}" style="text-
align:center" styleClass="width:300px">
              <p:panelGrid columns="2">
                 <h:outputText value="IdCliente" />
                  <p:inputText value="#{cliente.idCliente}"
readonly="true"/>
                  <h:outputText value="Email" />
                  <p:inputText value="#{cliente.email}"
readonly="true" />
              </p:panelGrid>
         </p:panel>
  </p:column>
   </p:dataGrid>

   </h:form>
   </p:panel>
</h:body>
</html>
```



Gravando um cliente...

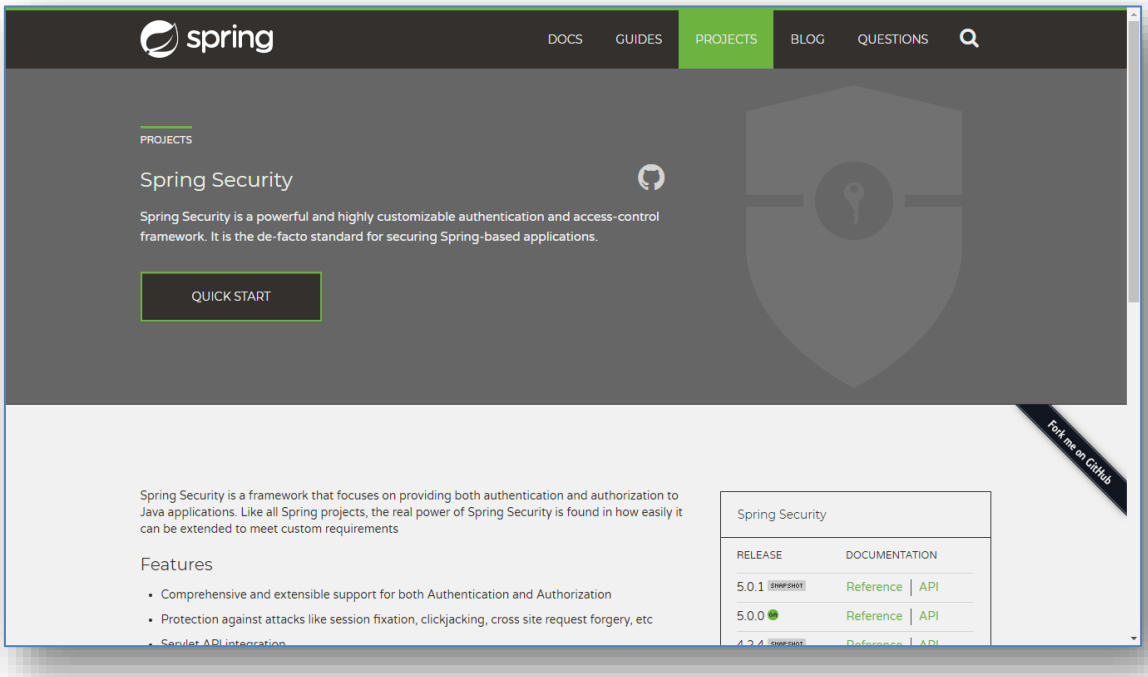


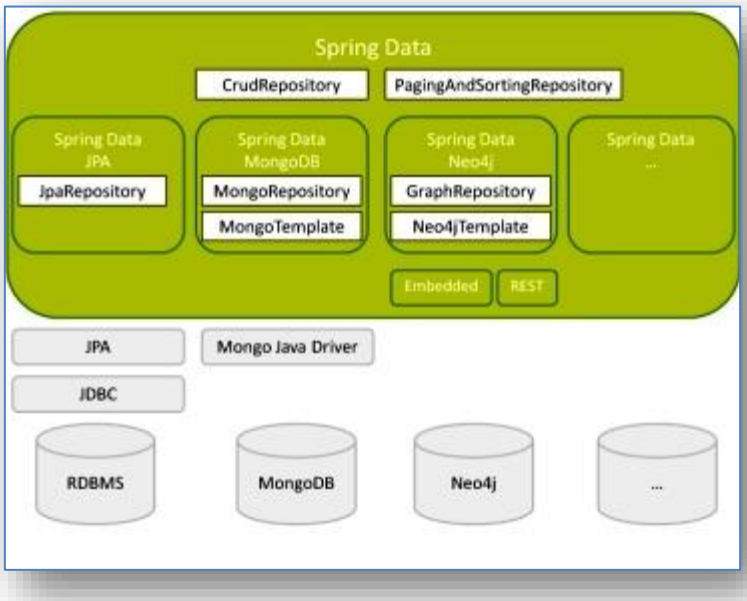
Gravado e atualizado...



Spring:

https://projects.spring.io/spring-security/





É um framework de código aberto (open source), criado por Rod Johnson, em meados de 2002, e apresentado no seu livro Expert One-on-One: JEE Design and Development. Foi criado com o intuito simplificar a programação em Java, possibilitando construir aplicações que antes só era possível utilizando EJB's.

O Spring atualmente possui diversos módulos como Spring Data (trata da persistência), Spring Security (trata da segurança da aplicação) entre outros módulos. Mas o principal (core) pode ser utilizado em qualquer aplicação Java, as principais funcionalidades são a injeção de dependência (CDI) e a programação orientada a aspectos (AOP), cabe ao desenvolvedor dizer ao Spring o que quer usar. O que faz dele uma poderosa ferramenta, pois não existe a necessidade de se arrastar todas as ferramentas do framework para criar uma aplicação simples.

O Spring Data-JPA é útil para desenvolvedores que fazem uso de persistência de dados em suas aplicações e procuram uma forma mais rápida, simples e eficaz de implementá-la.

O Spring é um framework Java que tem grande aceitação na área de desenvolvimento de sistemas. Embora não tenha sido lançado como um framework de persistência de dados, ele provê ferramentas que facilitam o desenvolvimento das classes que contêm as operações de CRUD, tanto com o uso de JDBC puro, como com algum framework de mapeamento objeto relacional (ORM).

O lançamento do projeto Spring Data-JPA conseguiu ir além de apenas mais uma ferramenta de apoio a frameworks de persistência. Este é um poderoso projeto que tem tudo para ganhar espaço e se tornar uma das ferramentas mais adotadas para persistência de dados, transformando os códigos fontes em poucas linhas de muita eficácia.

Quando chega a hora de iniciar um novo projeto de software, a equipe de desenvolvimento começa a ter diversas dúvidas sobre o que usar. Escolher as ferramentas certas, para cada tipo de projeto, entre todas as disponíveis em Java, é sempre um processo bem difícil. Escolher um framework então pode complicar ainda mais a decisão de o que usar em um novo projeto. Os frameworks, às vezes, possuem um objetivo bem específico, como por exemplo: um framework de persistência tem a responsabilidade apenas de persistir dados; um framework MVC tem como finalidade a inclusão do padrão MVC no projeto; frameworks de interface com o usuário (UI) disponibilizam componentes para a camada de visualização. Mas felizmente, para os apreciadores do Spring Framework, este surge como uma solução que se

adapta a vários tipos de projeto. Considerado uma ferramenta de fácil aprendizado, também disponibiliza uma vasta quantidade de APIs que podem ser usadas inclusive na integração com outros frameworks.

Os desenvolvedores do Spring estão sempre trabalhando para aperfeiçoar os recursos já disponíveis no framework, como também para adicionar novos recursos, que possam facilitar ainda mais o processo de desenvolvimento de aplicações. Entre esses processos, as operações de CRUD sempre foram um dos focos do Spring, sejam elas com uso de algum framework de persistência ou mesmo sem o uso deles. Para isso, algumas APIs foram disponibilizadas, como o JdbcTemplate, HibernateTemplate, JpaTemplate, entre outras.

As APIs de persistência sempre estiveram presentes no Spring Framework com a finalidade de facilitar o processo de implementação das classes direcionadas a operações de CRUD. Ainda não satisfeitos com as opções existentes no framework, os desenvolvedores do Spring lançaram nos primeiros meses de 2011 um novo projeto, chamado Spring Data-JPA.

Este projeto tem como objetivo facilitar as operações de CRUD baseadas em implementações JPA 2, reduzindo o esforço do programador para a quantidade de código que realmente se faz necessária. A Java Persistence API, ou apenas JPA, é uma especificação Java (JSR 317) que deve ser seguida por frameworks de mapeamento objeto-relacional (ORM), como o Hibernate, TopLink ou IBatis. Ela padroniza um conjunto de classes e métodos que devem ser implementados pelos frameworks ORM que desejam seguir a especificação. A vantagem em adotar este padrão é que um framework pode ser trocado por outro sem que exista a necessidade de modificar as classes e métodos de persistência da aplicação.

O projeto Spring Data-JPA, embora não seja um framework ORM, foi desenvolvido com base no padrão JPA 2 para trabalhar com qualquer framework que siga tal especificação. Ele é responsável pela implementação dos repositórios (camada de persistência de dados), oferecendo funcionalidades sofisticadas e comuns à maioria dos métodos de acesso a banco de dados. Ao programador, se abstrai a necessidade de criar classes concretas para os repositórios, sendo necessário apenas criar uma interface específica para cada classe de entidade, e nelas, estender a interface JpaRepository. Ao herdar a interface JpaRepository, dois propósitos são preenchidos: no primeiro, uma variedade de métodos como save(), delete(), findAll(), entre outros, são fornecidos; e no segundo, o reconhecimento desta interface como um bean do Spring, o que é útil para a injeção de dependência.

Classe de Entidade: É uma classe do tipo Bean que faz referência a uma entidade do banco de dados, e cada instância desta entidade representa uma linha (registro ou tupla) na tabela (entidade).

Entidade ou Tabela: É onde todas as informações inseridas em um banco de dados relacional são armazenadas. É uma estrutura formada por linhas e colunas onde cada linha consiste de pelo menos uma ou várias colunas.

Outra funcionalidade interessante se refere aos métodos de pesquisa. Em um projeto sem o uso de Spring Data, teríamos geralmente uma interface, e nela, alguns métodos de consulta seriam definidos. Estes métodos posteriormente seriam codificados em uma classe concreta. Quando se faz uso de Spring Data, basta incluir a assinatura do método de pesquisa na interface e adicionar a esta assinatura uma anotação do tipo @Query, com a respectiva consulta no formato JPQL. A implementação deste método será realizada automaticamente pelo Spring em tempo de execução.

Neste contexto, o artigo apresenta uma introdução ao Spring Data-JPA, demonstrando suas principais classes e interfaces. Para exemplificar seu uso, será desenvolvido um pequeno projeto que abordará sua configuração com o framework Hibernate e a implementação de alguns métodos de pesquisa usando os recursos disponibilizados pelo Spring Data. Para o leitor ter uma melhor compreensão do conteúdo tratado aqui, é aconselhável que já tenha um conhecimento básico de Spring e persistência com JPA.

Bean: É uma classe do tipo JavaBean, também conhecida como POJO (Plain Old Java Objects). Possui um construtor, atributos privados e os métodos getters e setters públicos.

Beans do Spring: Para o Spring, beans são classes gerenciadas por ele através de um container IoC (Inversion of Control). Estes beans estarão disponíveis para a aplicação sempre que ela precisar. O controle de instanciar e destruir tais beans é de responsabilidade do container IoC.

O projeto Spring Data-JPA

Os desenvolvedores do Spring tentaram facilitar em muito a vida do programador que irá usar o Spring Data-JPA em suas aplicações. Para isso, eles desenvolveram algumas classes e interfaces que reduzem a complexidade e a quantidade de código fonte que seria de responsabilidade do programador nas operações de CRUD.

O Spring Data está disponível na versão estável 1.0.2-RELEASE e pode ser encontrado no site da comunidade SpringSource – veja a URL para download na seção **Links** – ou através do repositório Maven. Ele pode ser usado com qualquer framework de persistência que siga a especificação JPA 2, como por exemplo, o Hibernate. Sua configuração possui a dependência de algumas bibliotecas do Spring Framework, como spring-core, spring-bean, spring-context, entre outras.

A partir da próxima seção do artigo, vamos demonstrar como funciona esta excelente ferramenta do Spring, e o código fonte aqui apresentado estará disponível, em forma de projeto, no espaço da revista reservado para download.

Conclusões

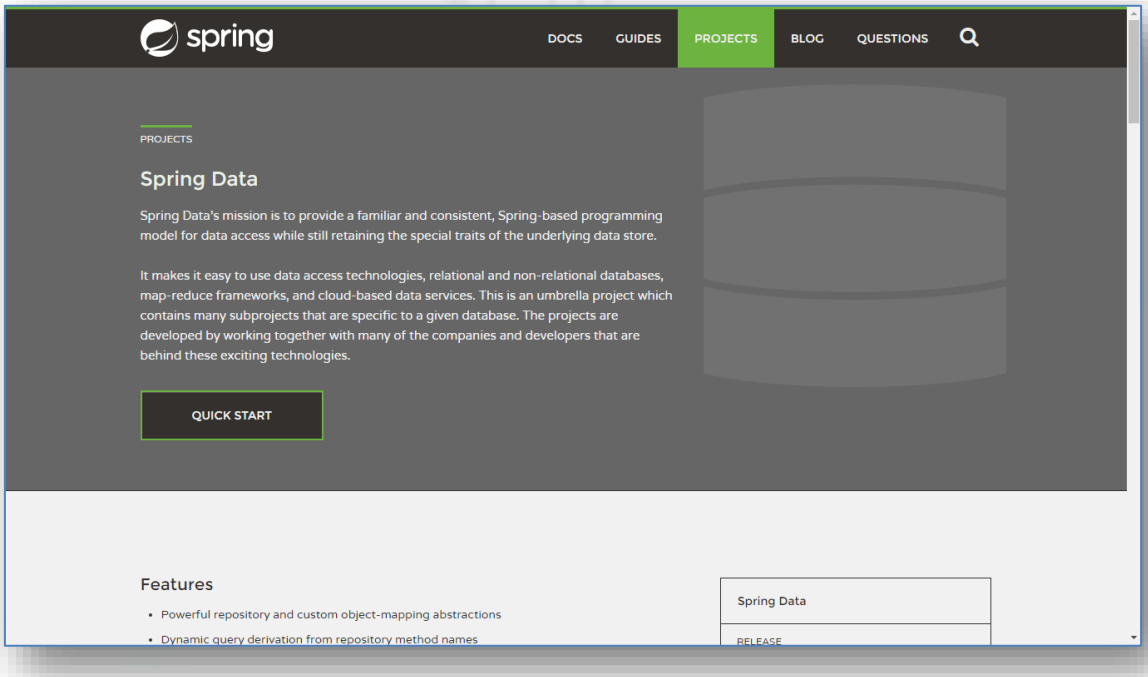
O Spring Framework é um dos projetos mais bem sucedidos da linguagem Java. Seus desenvolvedores estão sempre em busca de novidades que podem facilitar o desenvolvimento de aplicações. Embora o Spring não seja um framework de persistência de dados, sempre forneceu a seus usuários diversas APIs para esse fim, como HibernateTemplate, JdbcTemplate, JpaTemplate, entre outras, e em sua maioria, sempre foram muito bem aceitas e bastante utilizadas em vários projetos. Com o lançamento do Spring Data-JPA, pode-se dizer que houve uma grande evolução no que diz respeito à persistência de dados. Esse projeto facilita em muito o desenvolvimento de repositórios para persistência de dados, diminuindo bastante o número de linhas de código, o que torna o desenvolvimento do projeto mais rápido e fácil para o programador.

A grande facilidade em manipular consultas a dados talvez seja o principal aspecto do Spring Data-JPA, já que necessita apenas que sejam criadas as assinaturas dos métodos de pesquisa, enquanto a implementação destes métodos fica por conta do Spring. Estas consultas podem ser realizadas usando

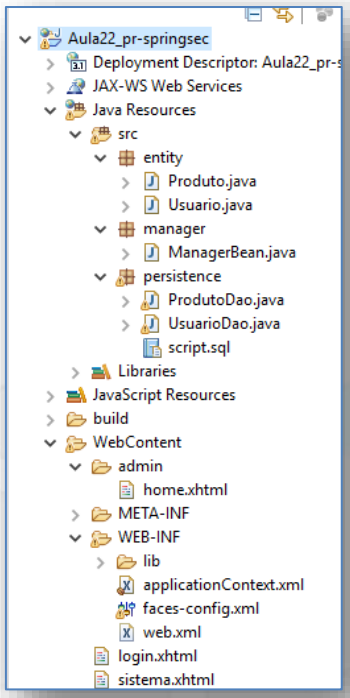
a linguagem JPQL juntamente com NamedQueries, Named Parameters ou Query Methods.

http://spring.io/

http://projects.spring.io/spring-data/



Estrutura do projeto:



LIBS:

antlr-2.7.6.jar
asm-attrs.jar
asm.jar
c3p0-0.9.0.jar
cglib-2.1.3.jar
cglib-2.2.jar
commons-beanutils-1.8.0.jar
commons-chain-1.1.jar
commons-codec-1.3.jar
commons-collections-3.2.1.jar
commons-dbcp-1.2.2.jar
commons-digester-2.0.jar
commons-el-1.0.jar
commons-lang-2.1.jar
commons-logging-1.1.1.jar
commons-pool-1.2.jar
commons-validator-1.3.1.jar
dom4j-1.6.1.jar
ejb3-persistence.jar

freemarker-2.3.8.jar
hibernate-annotations.jar
hibernate-commons-annotations.jar
hibernate-validator-3.0.0.ga.jar
hibernate3.jar
jakarta-oro.jar
javax.faces-2.0.11.jar
jstl-1.1.0.jar
jta.jar
log4j-1.2.15.jar
mysql-connector-java-5.1.23-bin.jar
ognl-2.6.11.jar
ojdbc14.jar
org.springframework.aop-3.1.0.RC1.jar
org.springframework.asm-3.1.0.RC1.jar
org.springframework.beans-3.1.0.RC1.jar
org.springframework.context-3.1.0.RC1.jar
org.springframework.core-3.1.0.RC1.jar
org.springframework.expression-3.1.0.RC1.jar
org.springframework.jdbc-3.1.0.RC1.jar
org.springframework.transaction-3.1.0.RC1.jar
org.springframework.web-3.1.0.RC1.jar
oro-2.0.8.jar
primefaces-4.0-sources.jar
primefaces-4.0.jar
servlet3-api.jar
spring-2.5.jar
spring-security-config-3.1.0.RC3.jar
spring-security-core-3.1.0.RC3.jar
spring-security-taglibs-3.1.0.RC3.jar
spring-security-web-3.1.0.RC3.jar
spring-webmvc-2.5.jar
standard.jar
xwork-2.0.4.jar

```
package entity;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
```

```
import javax.persistence.Table;

@Entity
@Table(name="produto")
public class Produto implements Serializable{

	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	 private Integer idProduto;
	 @Column
	 private String nomeProduto;
	 @Column
	 private Double preco;

	 public Produto() {
	}

	public Produto(Integer idProduto, String nomeProduto,
Double preco) {
		this.idProduto = idProduto;
		this.nomeProduto = nomeProduto;
		this.preco = preco;
	}

	@Override
	public String toString() {
		return "Produto [idProduto=" + idProduto + ",
nomeProduto=" + nomeProduto + ", preco=" + preco + "]";
	}

	public Integer getIdProduto() {
		return idProduto;
	}
	public void setIdProduto(Integer idProduto) {
		this.idProduto = idProduto;
	}
	public String getNomeProduto() {
		return nomeProduto;
	}
	public void setNomeProduto(String nomeProduto) {
		this.nomeProduto = nomeProduto;
```

```
	}
	public Double getPreco() {
		return preco;
	}
	public void setPreco(Double preco) {
		this.preco = preco;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
	}
}
```

```
package entity;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="usuario")
public class Usuario  implements Serializable{

	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	 private Integer idUsuario;
	@Column(length=50)
	 private String nome;
	@Column(length=50)
	 private String login;
	@Column(length=255)
	 private String senha;
	@Column(length=20)
	 private String perfil;
	@Column
	 private Integer ativo;
```

```
	public Usuario() {

	}

	public Usuario(Integer idUsuario, String nome, String
login, String senha, String perfil, Integer ativo) {
		this.idUsuario = idUsuario;
		this.nome = nome;
		this.login = login;
		this.senha = senha;
		this.perfil = perfil;
		this.ativo = ativo;
	}

	@Override
	public String toString() {
		return "Usuario [idUsuario=" + idUsuario + ", nome="
+ nome + ", login=" + login + ", perfil=" + perfil    + ",
ativo=" + ativo + "]";
	}

	public Integer getIdUsuario() {
		return idUsuario;
	}
	public void setIdUsuario(Integer idUsuario) {
		this.idUsuario = idUsuario;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getLogin() {
		return login;
	}
	public void setLogin(String login) {
		this.login = login;
	}
	public String getSenha() {
		return senha;
	}
```

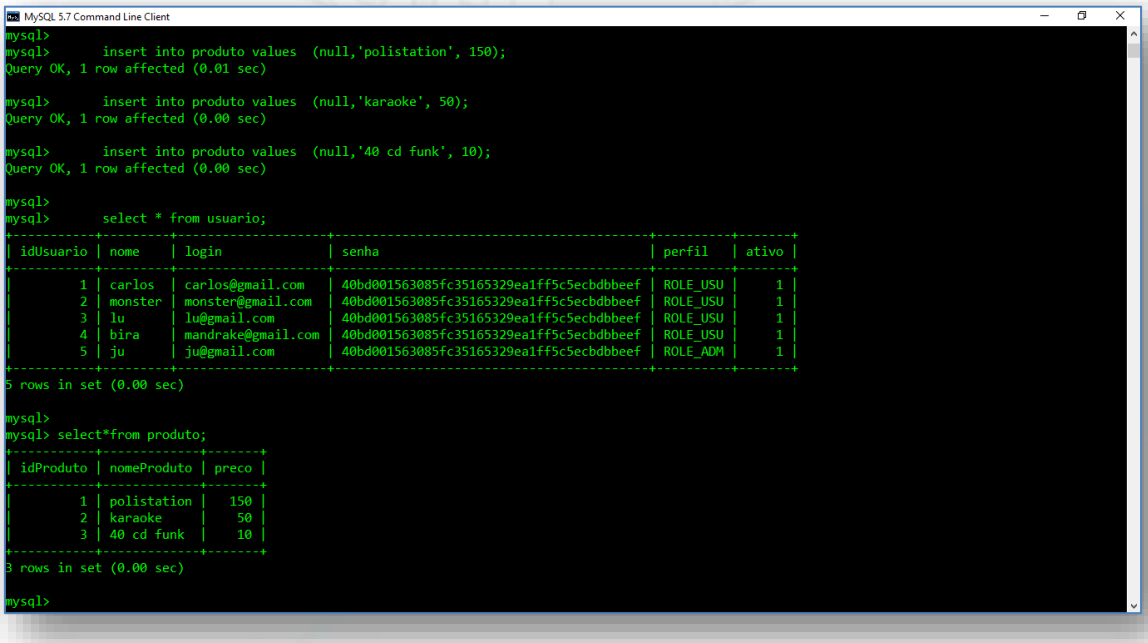
```
	public void setSenha(String senha) {
		this.senha = senha;
	}
	public String getPerfil() {
		return perfil;
	}
	public void setPerfil(String perfil) {
		this.perfil = perfil;
	}
	public Integer getAtivo() {
		return ativo;
	}
	public void setAtivo(Integer ativo) {
		this.ativo = ativo;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
	}
}
```

---

script.sql

```
drop database bd015;

create database bd015;

use bd015;

create table usuario(
	idUsuario int primary key auto_increment,
	  nome varchar (50),
	  login varchar (50) unique,
	  senha varchar (255),
	  perfil enum ('ROLE_ADM','ROLE_USU'),
	 ativo int
);

insert  into usuario values (null,'carlos','carlos@gmail.com',
sha1('123'),'ROLE_USU',1);
insert  into usuario values (null,'monster','monster@gmail.com',
sha1('123'),'ROLE_USU',1);
insert  into usuario values (null,'lu','lu@gmail.com',
sha1('123'),'ROLE_USU',1);
```

```
insert  into usuario values (null,'bira','mandrake@gmail.com',
sha1('123'),'ROLE_USU',1);
insert  into usuario values (null,'ju','ju@gmail.com',
sha1('123'),'ROLE_ADM',1);

      commit;

      create table produto(
     idProduto int primary key auto_increment,
    nomeProduto varchar (50) ,
     preco double
     );

      insert into produto values  (null,'polistation', 150);
      insert into produto values  (null,'karaoke', 50);
      insert into produto values  (null,'40 cd funk', 10);

      select * from usuario;

select*from produto;
```



```
package persistence;

import java.io.Serializable;
import java.util.List;

import org.hibernate.SessionFactory;
import org.springframework.orm.hibernate3.HibernateTemplate;

import entity.Usuario;

public class UsuarioDao implements Serializable {

	private static final long serialVersionUID = 1L;

	private HibernateTemplate hibernate;
                                //gerenciamento de gravacao
	 public void setSessionFactory(SessionFactory
sessionFactory) throws Exception{
		hibernate = new HibernateTemplate(sessionFactory);
	 }
	 //hibernate (espaÃ§o de mem)  set e get

	 public void create(Usuario u) throws Exception{
		hibernate.save(u);
	 }

	 public void update(Usuario u) throws Exception{
		hibernate.update(u);
 }

	 public void delete(Usuario u) throws Exception{
		hibernate.delete(u);
	}

	 public Usuario  findByCode(Integer cod){
		return (Usuario) hibernate.get(Usuario.class, cod);
	}

	 public List<Usuario>  findAll(){
		return  hibernate.loadAll(Usuario.class);
	}
```

```
	public HibernateTemplate getHibernate() {
		return hibernate;
	}

	public void setHibernate(HibernateTemplate hibernate) {
		this.hibernate = hibernate;
	}
}
```

```
package persistence;

import java.io.Serializable;
import java.util.List;

import org.hibernate.SessionFactory;
import org.springframework.orm.hibernate3.HibernateTemplate;

import entity.Produto;

public class ProdutoDao  implements Serializable{

private static final long serialVersionUID = 1L;

	private HibernateTemplate hibernate;

	 public void setSessionFactory(SessionFactory
sessionFactory) throws Exception{
		hibernate = new HibernateTemplate(sessionFactory);
			}
			//hibernate (espaÃ§o de mem)  set e get

			public void create(Produto p) throws Exception{
				hibernate.save(p);
			}

			public void update(Produto p) throws Exception{
			    hibernate.update(p);
		}

			public void delete(Produto p) throws Exception{
```

```
                hibernate.delete(p);
            }

             public Produto  findByCode(Integer cod){
        return (Produto) hibernate.get(Produto.class, cod);
            }

             public List<Produto>  findAll(){
                  return  hibernate.loadAll(Produto.class);
           }

             public HibernateTemplate getHibernate() {
                  return hibernate;
             }

    public void setHibernate(HibernateTemplate hibernate) {
                  this.hibernate = hibernate;
             }
}
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">

  <display-name>pr-springsec</display-name>

  <filter>
    <filter-name>springSecurityFilterChain</filter-name>
    <filter-
class>org.springframework.web.filter.DelegatingFilterProxy</filt
er-class>
  </filter>

  <filter-mapping>
    <filter-name>springSecurityFilterChain</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
```

```
  <listener>
    <listener-
class>org.springframework.web.context.ContextLoaderListener</lis
tener-class>
  </listener>

  <listener>
    <listener-class>
org.springframework.web.context.request.RequestContextListener
    </listener-class>
  </listener>

   <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
    <load-on-startup>1</load-on-startup>
  </servlet>

  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>


</web-app>
```

---

faces-coonfig.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<faces-config
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-facesconfig_2_2.xsd"
    version="2.2">

<application>
     <el-
resolver>org.springframework.web.jsf.el.SpringBeanFacesELResolve
r</el-resolver>
</application>
```

```
</faces-config>
```

---

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
      xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
      xmlns:sec="http://www.springframework.org/schema/security"
      xsi:schemaLocation="http://www.springframework.org/schema/b
eans http://www.springframework.org/schema/beans/spring-beans-
2.5.xsd
       http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop-2.5.xsd
       http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx-2.5.xsd
       http://www.springframework.org/schema/security
http://www.springframework.org/schema/security/spring-security-
3.1.xsd">

    <sec:http    auto-config="true"
                 access-denied-page="/sistema.jsf" >

<sec:intercept-url pattern="/admin/*"
      access="ROLE_ADM"/>
<sec:intercept-url pattern="/sistema.jsf"
      access="ROLE_USU,ROLE_ADM"/>

<sec:form-login login-page="/login.jsf"
                      authentication-failure-
url="/login.jsf?erro=true"
                      default-target-url="/admin/home.jsf" />

<sec:logout logout-success-url="/login.jsf"
                 delete-cookies="true"
                 invalidate-session="true"/>

      </sec:http>

           <sec:authentication-manager>
```

```
            <sec:authentication-provider>
           <sec:password-encoder hash="sha" />

            <sec:jdbc-user-service data-source-ref="ds"
       users-by-username-query=
       "select login, senha, 1 from usuario where  login=?"
       authorities-by-username-query=
"select login, perfil from usuario where login=? and ativo=1"
       />
            </sec:authentication-provider>
      </sec:authentication-manager>

       <!-- SPRING DATA -->
      <bean id="ds"
      class="org.apache.commons.dbcp.BasicDataSource"
            destroy-method="close">

 <property name="driverClassName"
      value="com.mysql.jdbc.Driver" />
 <property name="url" value="jdbc:mysql://localhost:3306/bd015"
/>
 <property name="username" value="root" />
 <property name="password" value="coti" />

      </bean>


      <bean id="sf"

      class="org.springframework.orm.hibernate3.annotation.Annota
tionSessionFactoryBean">
            <property name="dataSource" ref="ds" />

            <property name="annotatedClasses">
                  <list>
                        <value>entity.Usuario</value>
                        <value>entity.Produto</value>
                  </list>
            </property>

            <property name="hibernateProperties">
                  <props>
```

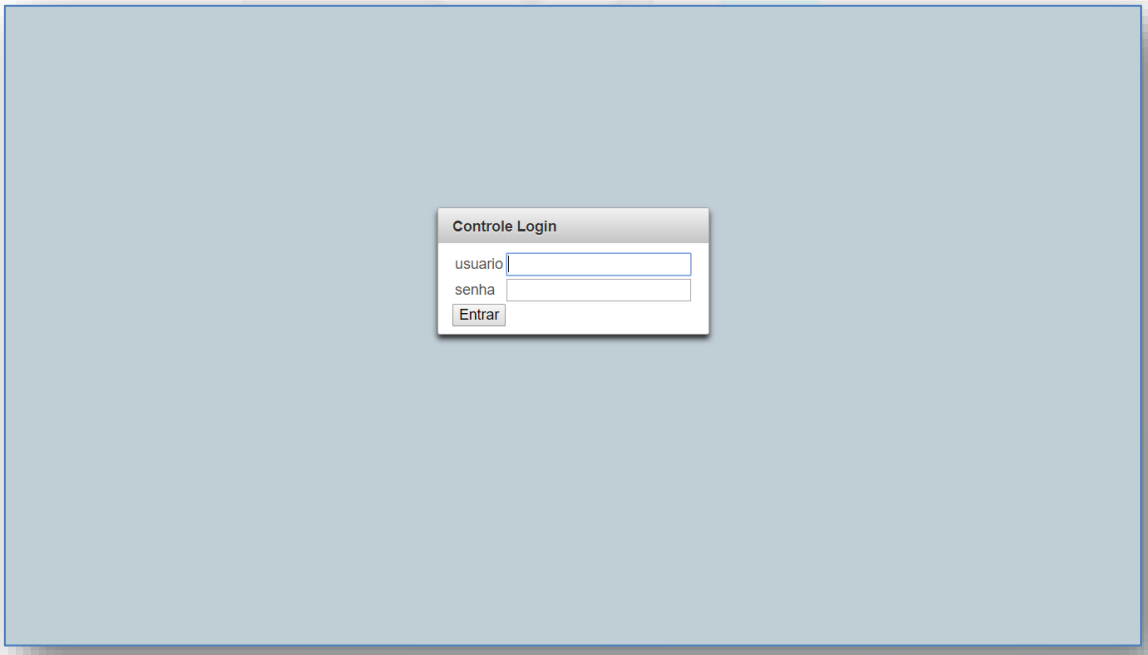
```
                        <prop
key="hibernate.dialect">org.hibernate.dialect.MySQLDialect</prop
>
                    <prop key="hibernate.show_sql">true</prop>
                <prop key="hibernate.format_sql">true</prop>
                <prop key="hibernate.hbm2ddl.auto">update</prop>
                    </props>
                </property>
        </bean>

        <bean id="usuarioDaoSpring" class="persistence.UsuarioDao">
             <property name="sessionFactory" ref="sf"/>
        </bean>

        <bean id="produtoDaoSpring" class="persistence.ProdutoDao">
             <property name="sessionFactory" ref="sf"/>
        </bean>


</beans>
```

---

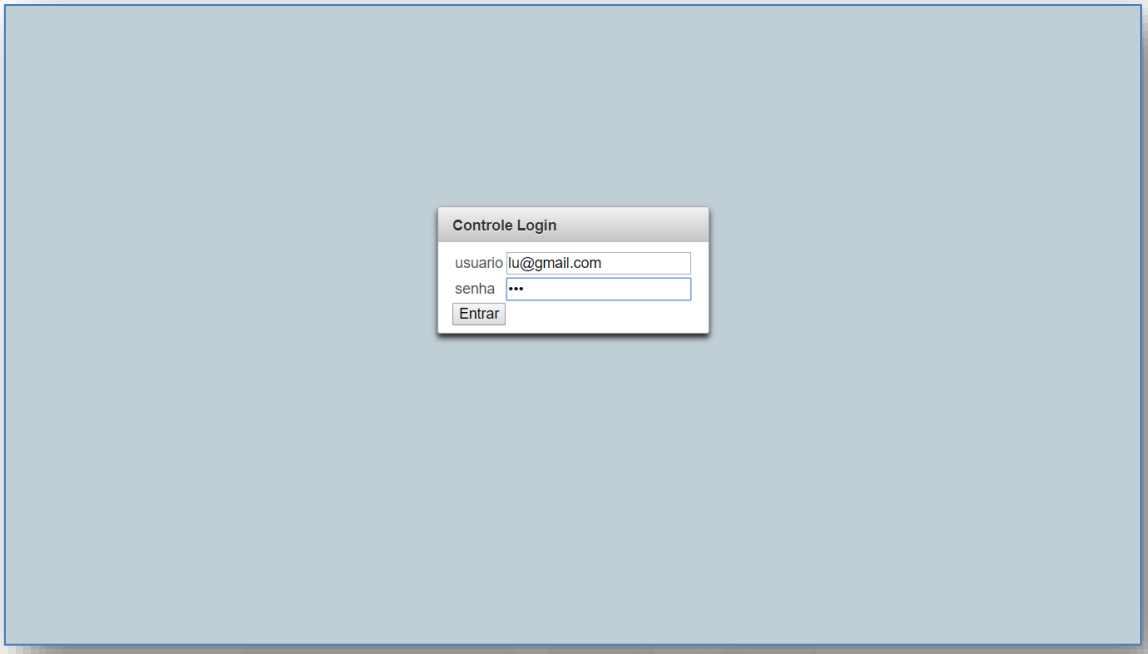
login.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core"
>
  <h:head>
  <style type="text/css">
      .ui-width{font-size:16pt; font-color:#00F};
  </style>
 </h:head>
 <h:body onload="acesso.show()">
    <p:dialog  header="Controle Login"  widgetVar="acesso"
      closable="false" modal="true" resizable="false">
                <p:outputLabel value="Login Invalido"
rendered="#{param.erro}"
                    style="color:red"></p:outputLabel>
```

```
        <form method="post" action="j_spring_security_check">
            <h:panelGrid columns="2">
               <h:outputText value="usuario" />
               <h:inputText id="j_username"
required="true"></h:inputText>
               <h:outputText value="senha" />
               <h:inputSecret id="j_password"
required="true"></h:inputSecret>
             </h:panelGrid>
             <h:commandButton value="Entrar"></h:commandButton>
        </form>
      </p:dialog>
</h:body>
</html>
```



Logando como usuario...

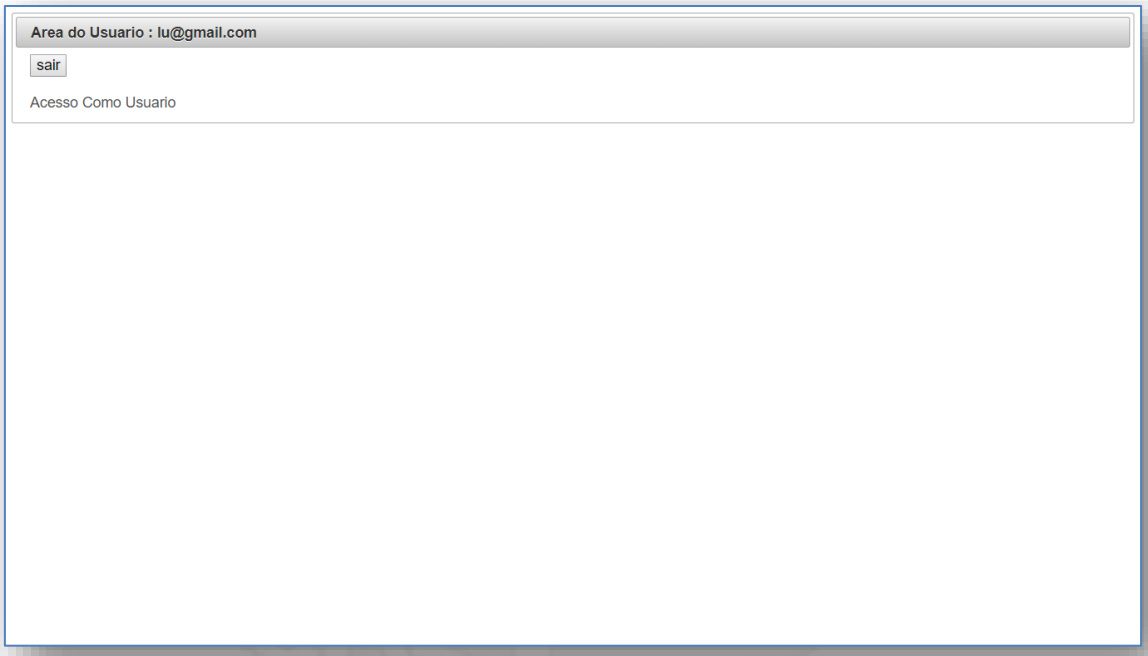


---

sistema.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core"
>
  <h:head>
 </h:head>
 <h:body>
    <p:panel header="Area do Usuario :  #{mb.logado.nome}">
       <form method="post" action="../j_spring_security_logout">
          <h:commandButton value="sair"></h:commandButton>
       </form>
       <br/>
       <h:outputText value="Acesso Como Usuario"   />
    </p:panel>
```

```
</h:body>
</html>
```



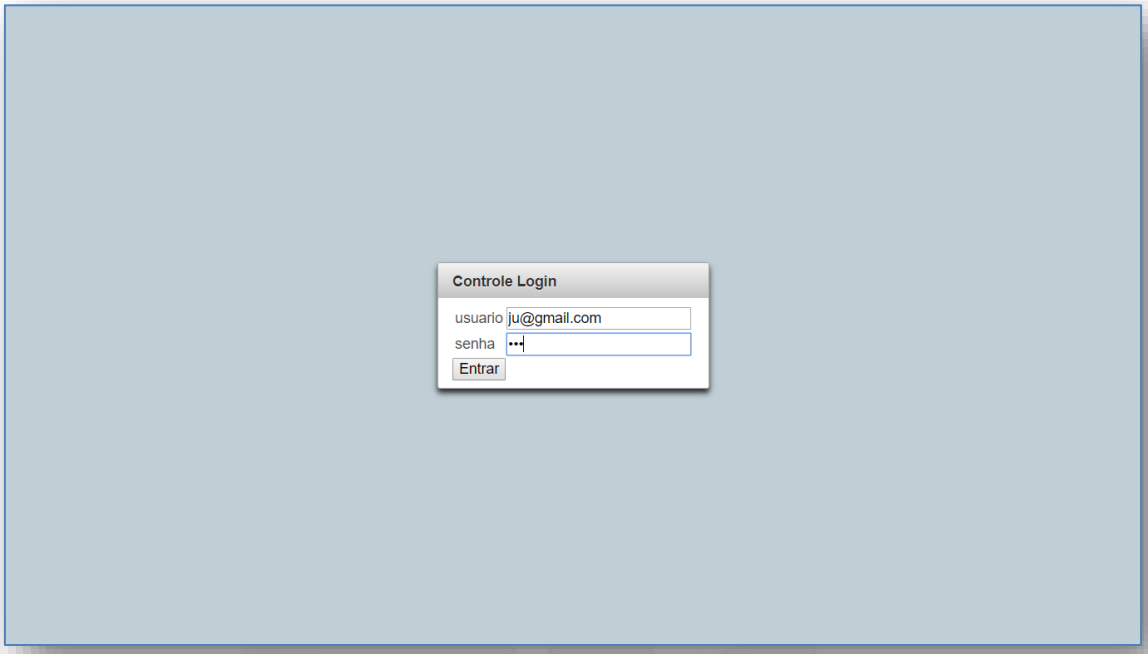
Criar uma pasta (diretório) chamada "admin"...

home.xhtml

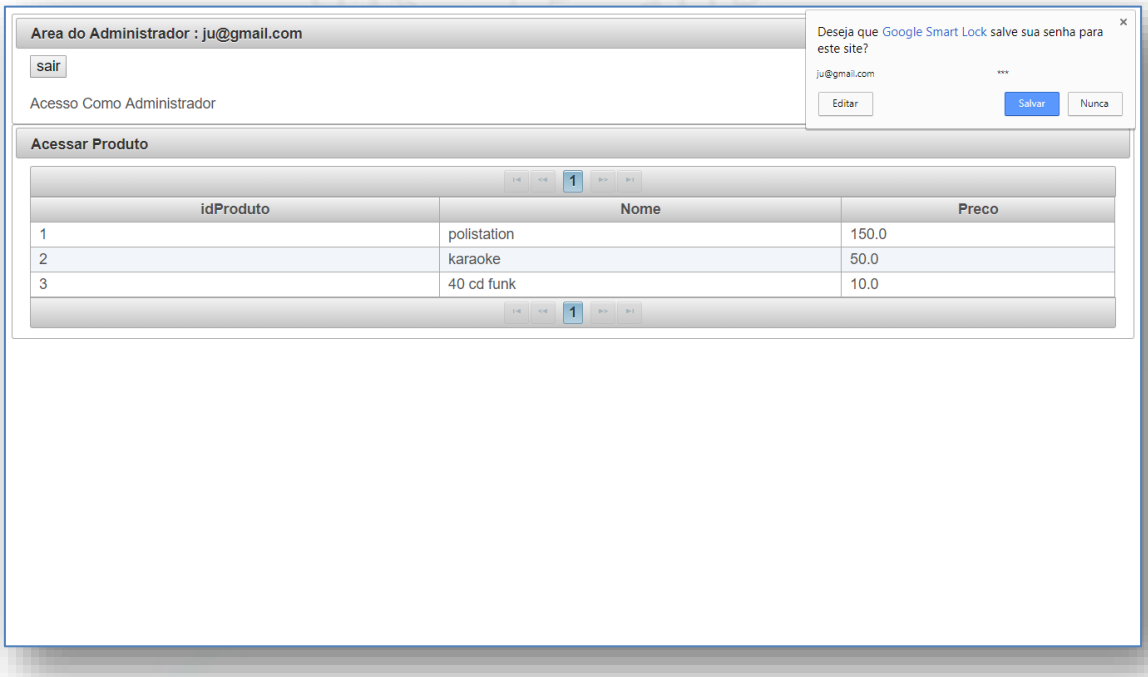
```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core"
>
  <h:head>
 </h:head>
 <h:body>
```

```
    <p:panel header="Area do Administrador :
#{mb.logado.nome}">
       <form method="post" action="../j_spring_security_logout">
          <h:commandButton value="sair"></h:commandButton>
       </form>
       <br/>
       <h:outputText value="Acesso Como Administrador"   />
    </p:panel>

    <p:panel header="Acessar Produto">
      <p:dataTable value="#{mb.produtos}" var="linha"
       rows="3" paginator="true">

        <p:column headerText="idProduto">
          <h:outputText value="#{linha.idProduto}" />
        </p:column>

       <p:column headerText="Nome">
          <h:outputText value="#{linha.nomeProduto}" />
        </p:column>

       <p:column headerText="Preco">
          <h:outputText value="#{linha.preco}" />
        </p:column>
    </p:dataTable>

  </p:panel>


</h:body>
</html>
```

Logando como admin...



Logado como admin...



MongoDB:

MongoDB (do inglês *humongous*, "gigantesco") é uma aplicação de código aberto, de alta performance, sem esquemas, orientado a documentos. Foi escrito na linguagem de programação C++.[1] Além de orientado a documentos, é formado por um conjunto de documentos JSON. Muitas aplicações podem, dessa forma, modelar informações de modo muito mais natural, pois os dados podem ser aninhados em hierarquias complexas e continuar a ser indexáveis e fáceis de buscar.

O desenvolvimento de MongoDB começou em outubro de 2007 pela 10gen. A primeira versão pública foi lançada em fevereiro de 2009.[2]

Mongo (É um em Banco de dados (3ª camada) em Javascript) / Quem Grava o Projeto tb é Javascript (JSON) Comandos do Json para trabalhar com o Banco ... Velocidade, melhor desempenho, BIG DATA ... Quantidade de Dados, Imagem ... Variedades de Informações / e velocidade de Captura de Informações ...

NOSQL (Javascript) _ Banco sem sql ... Comandos de Javascript ...

Como Surgiu o MongoDB

INTRODUÇÃO

É fato de que a maior parte dos bancos de dados existentes no mercado hoje são do tipo modelo relacional. Mas fatores novos estão mudando essa história, pelo menos no que se refere ao crescimento intenso da quantidade de dados de certas organizações, e algumas limitações do modelo relacional têm ajudado para alternativas de banco de dados apareçam buscando sanar essas limitações. A alternativa que tem ganhado bastante espaço nas organizações são os banco de dados chamados de NoSQL. Neste artigo, são apresentadas as principais características desses bancos de dados e, em especial, veremos as características do Banco de Dados Orientado a Documento MongoDB.

SGBDS RELACIONAIS

Surgido nos anos 70, os SGBDs (Sistemas Gerenciadores de Banco de Dados) relacionais tem recursos que são importantes para facilitar o processo de

criação e manutenção de sistemas, alguns deles são: a verificação e garantias de integridade dos dados, o gerenciamento de concorrência, recuperação de falhas, processos de uso de validação, a segurança, a otimização de consultas, etc..

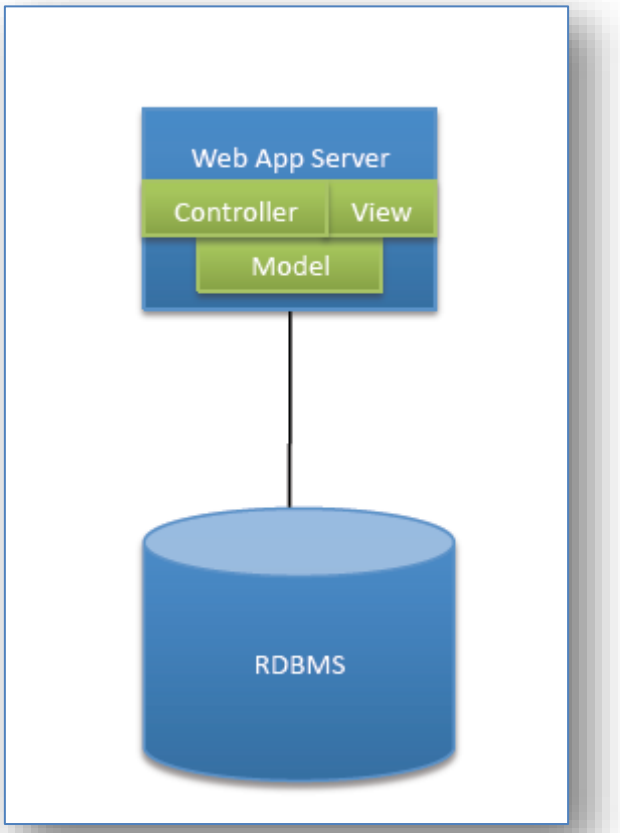
Os Sistemas Gerenciadores de Banco de Dados relacionais mais conhecidos são: SQL Server, Oracle, PostgreSQL, MySQL, etc. Usam uma linguagem de manipulação, consulta e definição chamada SQL (Structured Query Language), que foi criada são as relações (ou tabelas), as quais são compostas de linhas (ou tuplas) e colunas (ou atributos).

Graças ao intenso crescimento do número de aplicações, soluções, recursos e tudo o que se refere a sistemas computacionais nas últimas décadas, o volume de dados associados a tais tipos de sistemas também teve um ritmo de crescimento acelerado. É justamente aí, nesses lugares nos quais o grande volume de dados é o problema, esses bancos baseados no Modelo Relacional, não tem sido eficientes o bastante. E essa é a dificuldade real do uso do Modelo Relacional: a difícil tarefa de conciliar a demanda por escalabilidade cada vez mais intensa.

BANCOS DE DADOS NOSQL

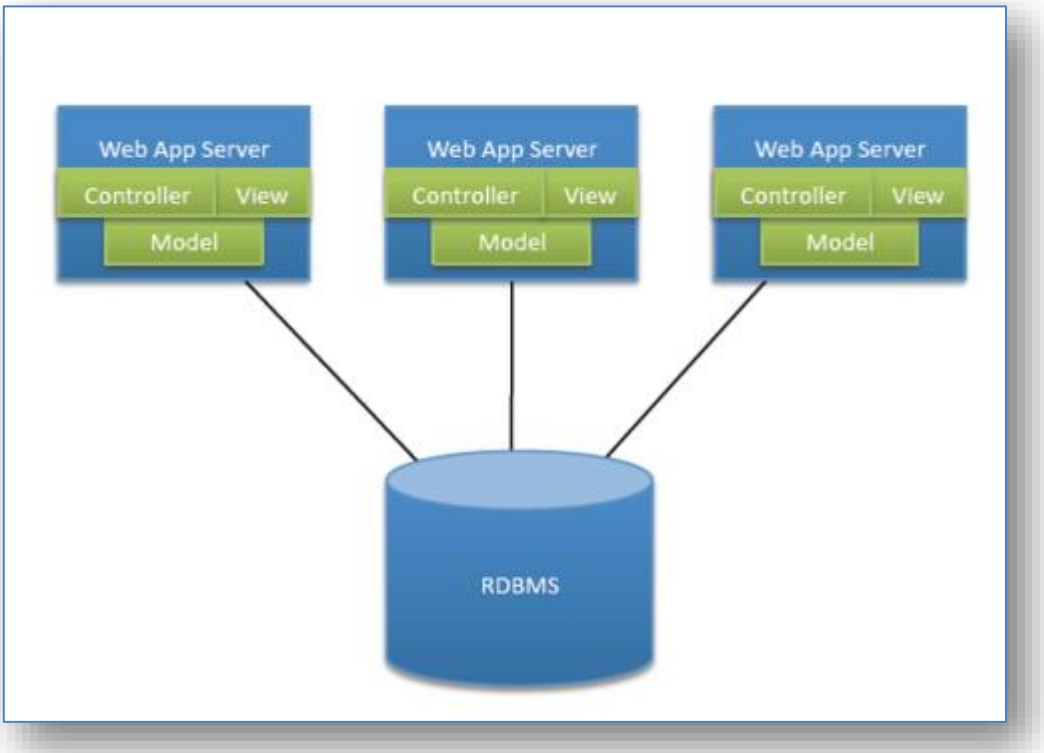
É na Web que encontramos a maior inspiração para o desenvolvimento de novas tecnologias de Banco de Dados. Tudo por causa da necessidade que surge para suprir as aplicações web que cada vez mais tem um grande volume de dados e requisitos diferenciados, como o elevado grau de disponibilidade a escalabilidade sob demanda, surge então uma nova categoria de SGBDs, conhecida como NoSQL, abreviação de Not Only SQL (não apenas SQL), criada inicialmente para foi proposta com o objetivo de atender aos requisitos de gerenciamento de grandes volumes de dados, semi-estruturados ou não estruturados. Ou seja, sendo utilizado principalmente em casos em que o Modelo Relacional não apresentava performance adequada.

É importante deixar claro que o NoSQL não veio para substituir os bancos de dados relacionais. Os bancos de dados NoSQL servem para uma gama de problemas que nem sempre são os mesmos dos bancos de dados relacionais. E quando se fala em NoSQL sempre se fala das grandes vantagens em termos de escala em comparação aos SGBDs relacionais. Não é verdade que os bancos de dados relacionais não escalam. A verdade é que eles não escalam com facilidade. Abaixo um esquema básico de aplicação web:



Se o sistema acima passar a demanda de usuários aumentarem muito surgem na nossa aplicação problemas com a escalabilidade. O que se pode fazer é aumentar a capacidade do servidor, dando-lhe um upgrade no seu processador, no armazenamento e em sua memória.. Ou então, fazer crescer a quantidade das máquinas de servidores web:

www.cotiinformatica.com.br



E se a aplicação web continuar a crescer? Certamente o banco de dados não iria suportar atender a todas as requisições em um tempo hábil. E se colocássemos mais máquinas rodando o nosso banco de dados?

Com isso, agora devemos escalar nosso banco de dados através de múltiplas máquinas, o que é mais complicado do que com os nossos servidores web. Na maioria dos casos não podemos simplesmente ligar mais uma máquina rodando o banco e esperar que tudo funcione. Vamos precisar de uma série de

configurações e alterações nas nossas aplicações para fazer tudo funcionar na nova arquitetura distribuída.

Os bancos de dados NoSQL vem para tornar este processo que muitas vezes é trabalhoso e complicado em um processo mais simples e robusto.

Como exemplo de aplicações que necessitam de um gerenciamento eficaz de grandes quantidades de dados não estruturados estão às redes sociais, exemplos clássicos nós temos o Twitter, Facebook e outras redes, onde a quantidade de informação gerada pelos usuários possui um crescimento que as bases de dados relacionais não conseguem comportar. Tudo isso se deve o fato dos bancos de dados relacionais não serem eficazes nesse requisito.

Esses bancos de dados NoSQL são amplamente adotados em empresas como Facebook, Amazon e Google com o intuito de atender às suas demandas de escalabilidade, alta disponibilidade e dados não estruturados. E vale ressaltar que, atualmente, diversos bancos de dados NoSQL de código livre estão disponíveis, exemplos disso são o Cassandra, Hypertable, MongoDB, Redis, CouchDB e Dynamo.

As primeiras e mais importantes implementações de um sistema realmente não-relacional são:

BigTable – do Google. Surgiu em 2004 como um banco de dados proprietário de alta performance que tinha como objetivo promover maior escalabilidade e disponibilidade.

Dynamo – da Amazon. Surgiu em 2007, o sistema que tinha como característica básica ser um banco de dados não-relacional, de alta disponibilidade, utilizado pelos web services da Amazon.

O Cassandra, de 2008, projetado para ser um sistema de banco de dados distribuído e de alta disponibilidade, desenvolvido pelo site Facebook para lidar com grandes volumes de dados. No início de 2010, o Cassandra desbancou o MySQL como banco de dados do Twitter, demonstrando sua importância cada vez mais crescente.

MongoDB, lançada em 2009, um banco de dados orientado a documentos, escalável, de alta performance e livre de esquemas. E é o grande foco do nosso estudo.
Antes de falarmos mais sobre as soluções NoSQL, mais precisamente do MongoDB, é importante lembrar que o propósito, portanto, não é substituir o Modelo Relacional como um todo, mas apenas em casos nos quais seja necessária uma maior flexibilidade da estruturação do banco.

MONGO DB

Criado pelo ex-Fundador do DoubleClick e CTO Dwight Merriman e DoubleClick ex-engenheiro e fundador ShopWiki e Eliot CTO Horowitz. Eles se basearam as suas experiências em construção de grande escala, alta disponibilidade, sistemas robustos para criar um novo tipo de banco de dados. O próprio Eliot dá a definição:

O MongoDB não foi concebido em um laboratório. Nós construímos o MongoDB com base em nossas experiências na construção de sistemas robustos de grande escala e alta disponibilidade. Não começamos do zero, realmente tentamos descobrir o que estava quebrado, e combater isso. Assim, a maneira que eu penso sobre MongoDB é que se você pegar o MySql, e alterar o modelo de dados do relacional para orientado a documento, você ganha uma grande quantidade de recursos: documentos embarcados para velocidade, facilidade de gerenciamento, desenvolvimento ágil com bancos de dados sem "schema", escalabilidade horizontal mais fácil, pois "joins" não são tão importantes. Há muitas coisas que funcionam muito bem nas bases de dados relacionais: índices, consultas dinâmicas e atualizações para citar alguns, e nós não mudamos muito neste ponto. Por exemplo, a maneira de projetar seus índices no MongoDB deve ser exatamente do jeito que você faz isso no MySQL ou Oracle, você só ganha a opção de indexar um campo embarcado.
– Eliot Horowitz, CTO 10gen e Co-fundador

Fazendo uma busca em literatura disponível, podemos resumir MongoDB nas características abaixo:

- Open Source;
- Alta Performance;
- Schema (Esquema) Aberto – para fácil evolução do esquema;
- Banco de Dados Orientado a Documento;
- Preparador para trabalhar na nuvem;

Atualmente na versão 2.2.

MongoDB é um banco de dados de documento, e isto quer dizer que podemos armazenar dados como documentos.

E o que são documentos? São as unidades básicas de armazenamento e estes não utilizam necessariamente qualquer tipo de estruturação pré-definida, como é o caso das tabelas do Modelo Relacional. Ou seja, o MongoDB armazena coleções de documentos. Um documento, em geral, é um objeto com um identificador único acrescido de um conjunto de campos, que podem ser strings, listas ou documentos aninhados. Neste modelo temos um conjunto de documentos e em cada documento temos um conjunto de campos (chaves) e o valor deste campo. Outra característica importante é que este modelo não depende de um esquema rígido, ou seja, não exige uma estrutura fixa como ocorre nos bancos tradicionais, os relacionais. Assim, é possível que ocorra uma atualização na estrutura do documento, com a adição de novos campos, por exemplo, sem causar problemas na estrutura do banco de dados. Esta flexibilidade é uma das grandes vantagens deste modelo.

O MongoDB é um destes bancos que utiliza o modelo de armazenamento orientado a documento que armazena documentos no estilo parecido JSON chamado BSON, como documentos com esquemas dinâmicos, quer dizer que eu não preciso ter a mesma estrutura em casa registro. O objetivo do MongoDB é preencher a lacuna entre valores/chave lojas (que são rápidos e escalonáveis) e bancos de dados relacionais (que têm a funcionalidade rica). Atualizações in-place rápidas, ou seja, uma transação atômica.
Quem trabalha muito com Banco de Dados Relacionais o MongoDB é uma boa quebra de paradigma já que ele não possui schema – o que quer dizer que não é necessário estipular a estrutura dos dados ao criar o banco, basta adicionar o documento(JSON) contendo os campos desejados, inclusive um documento pode conter outros documentos.

Como o formato BSON pode ser estendido, é possível acrescentar outras informações ao objeto aluno, de maneira que ele incorpore mais informações no objeto criado, como um array de dados por exemplo.

Para um maior entendimento das características do MongoDB, que são expressadas como objetos JSON. O quadro seguinte mostra exemplos da sintaxe SQL e da sintaxe da Linguagem de Consulta do Mongo.

Termo em MySQL	Mongo term/concept
banco de dados	banco de dados
tabela	coleção
índice	índice
linha	BSON documento
coluna	campo BSON
junção	embutir e referenciar
primary key	_id field
group by	aggregation

Exemplos em Javascript e podem ser executado através do Mongo Shell.

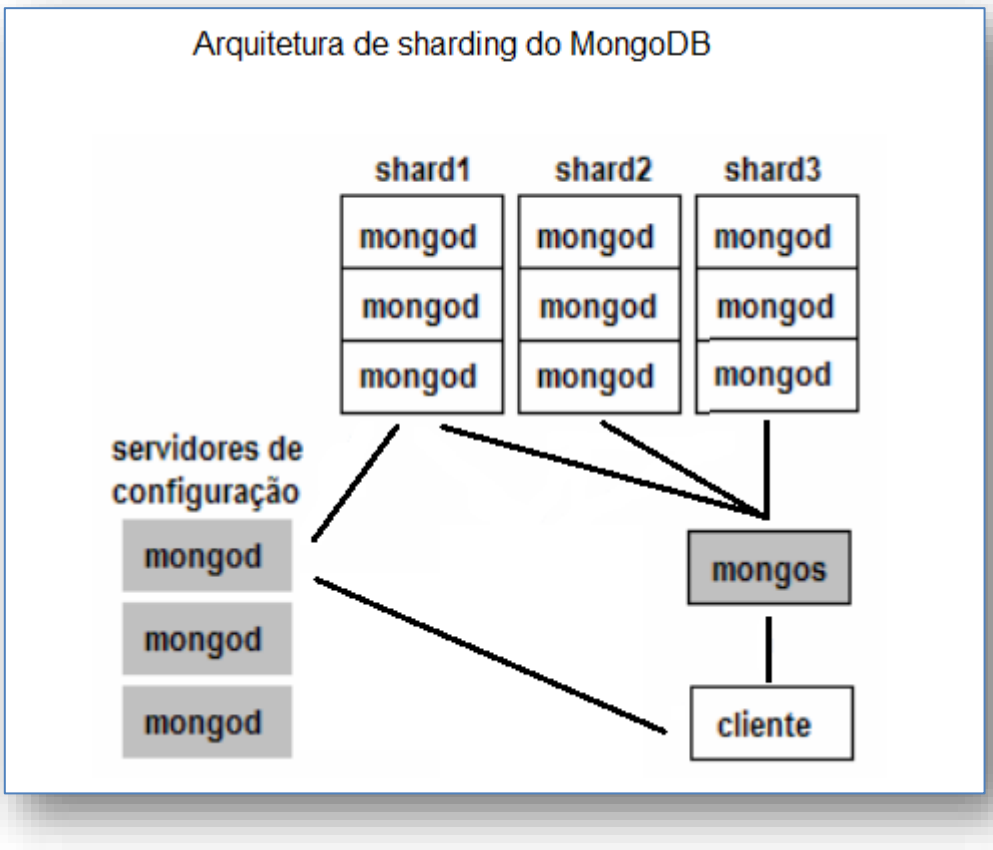
Sentença SQL	Mongo Statement
CREATE TABLE USERS (a Number, b Number)	Implícito; podem também ser feito explicitamente com db.createCollection("mycoll")
ALTER TABLE users ADD ...	implicit
INSERT INTO USERS VALUES(3,5)	db.users.insert({a:3,b:5})
SELECT a,b FROM users	db.users.find({}, {a:1,b:1})
SELECT * FROM users	db.users.find()
SELECT * FROM users WHERE age=33	db.users.find({age:33})
SELECT * FROM users WHERE age=33 ORDER BY name	db.users.find({age:33}).sort({name:1})
SELECT * FROM users WHERE age>33	db.users.find({age:{$gt:33}})
SELECT * FROM users WHERE age!=33	db.users.find({age:{$ne:33}})
SELECT * FROM users WHERE name LIKE "%Joe%"	db.users.find({name:/Joe/})
SELECT * FROM users WHERE age>33 AND age<=40	db.users.find({'age':{$gt:33,$lte:40}})
SELECT * FROM users ORDER BY name DESC	db.users.find().sort({name:-1})

Os bancos de dados NoSQL foram especialmente projetados para atender as características de: maior disponibilidade, menor tempo de resposta para consultas, paralelismo de atualização de dados e maior grau de concorrência. O MongoDB inclui um módulo de sharding automático que permite a construção de um cluster de banco de dados escalável horizontalmente projetado para incorporar novas máquinas de forma dinâmica .

SHARDING

É uma forma de dividir carga no banco que você pode fazer de forma particionada.

Na Figura 1, vemos um cluster consiste de dois ou mais blocos de servidores chamados de shards, um ou mais servidores de configuração e qualquer número de processos roteadores chamados de mongos através dos quais os servidores da aplicação se conectam. Se você tem um monte de dados e está no limite de disco e/ou a falta de espaço, a resposta é ter os seus dados divididos entre várias máquinas. Você fica com mais rendimento e com maior capacidade de armazenamento em disco. Em um mundo perfeito, como seu armazenamento e desempenho precisa crescer, basta acrescentar mais shards.



GRIDFS

Com GridFS, você armazena arquivos no banco de dados. MongoDB foi construído para fazer isso. Quando você guarda os arquivos no MongoDB, recebe toda a capacidade de replicação e sharding gratuitamente. Essa propriedade é indicada pra se trabalhar com um grande volume de arquivos e muitos acessos simultâneos. Ela também é ideal para trabalhar com arquivos maiores que 4 MB como imagens, vídeo e áudio.

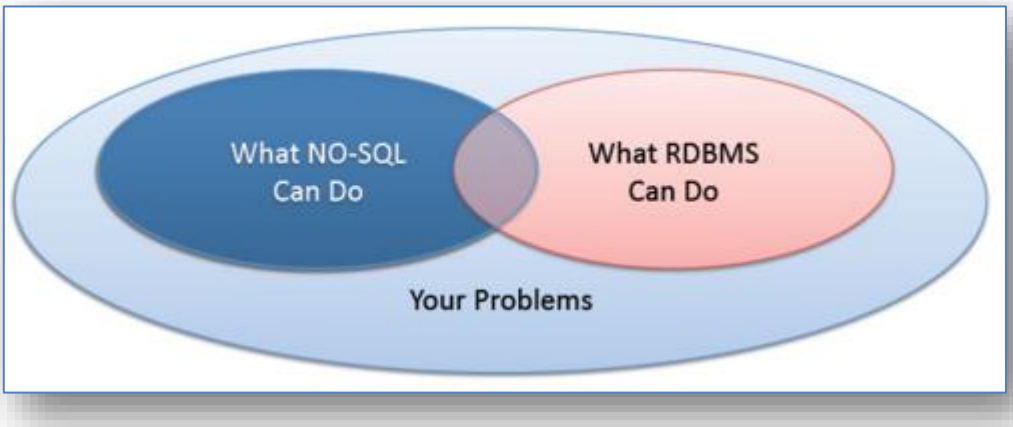
Isso não quer dizer que ela não seja indicada para arquivos menores, muito pelo contrário. A diferença é que, em arquivos maiores que 4MB o GridFS automaticamente divide o arquivo em partes praticamente e gera metadados de também de forma automática.

CONCLUSÕES

Antes de optar por o MongoGB ou um SGSBD tradicional, deve-se levar em consideração as necessidades do problema. Os fatores que devem ser utilizados como critérios de comparação são de tipos diversos, indo desde a escalabilidade do sistema, passando por questões de consistência de dados, até problemas de facilidade de uso ou existência ou não de linguagens de consulta.

Apesar do fato dos bancos NoSQL, e mais especificamente o MongoGB, serem projetados para não precisar ficar usando junções, ainda podem ocorrer consultas que precisam dessas operações, mesmo que não seja do mesmo nível que faz-se necessário em sistemas relacionais comuns.

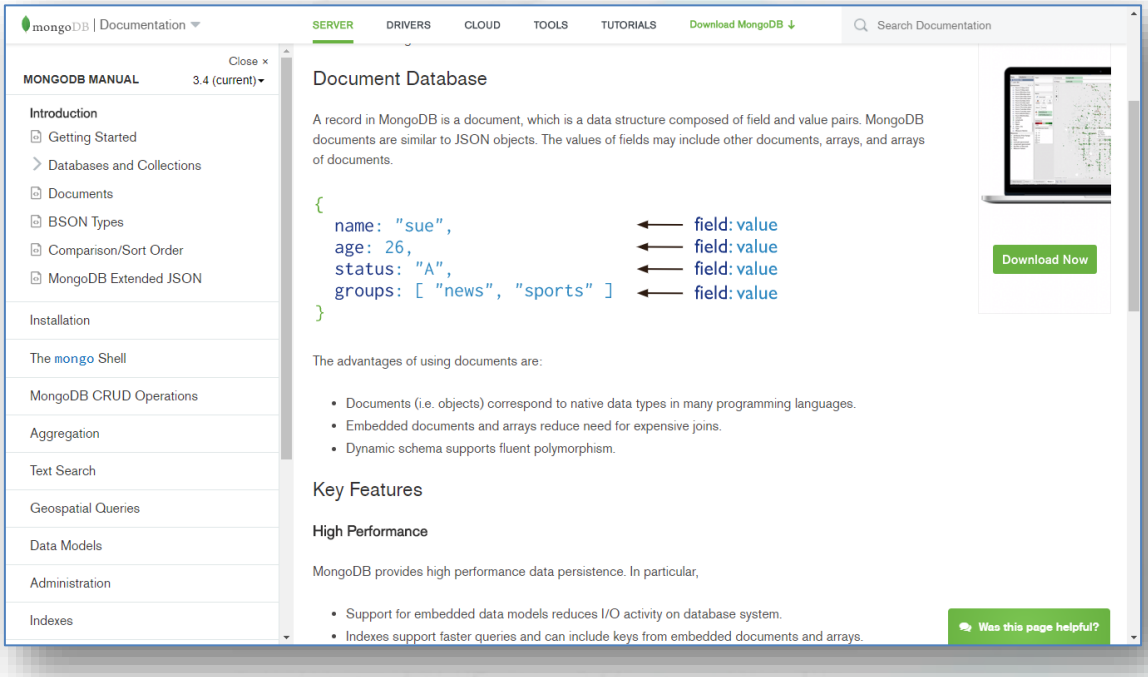
E tanto os bancos de dados relacionais quanto os não relacionais podem coexistir. Eles atendem em sua maioria a casos de usos diferentes como ilustrado na figura abaixo:



Podemos então concluir que a principal vantagem do MongoDB é a alta performance, uma única consulta já te retorna tudo o que você precisa saber a respeito do documento (esta é uma das razões pelas quais vêm sendo adotado em projetos que exigem alta performance). Mas existem outras vantagens, como o Schema Aberto (tipagem dinamica, migrações, flexibilidade, cache), GeoLocalização Nativa e ser Open Source (Diminui seus custos no projeto).

A desvantagem principal é o fato de que se você quiser alterar todos os registros relacionados a uma unidade semântica, precisa tratá-los um a um.

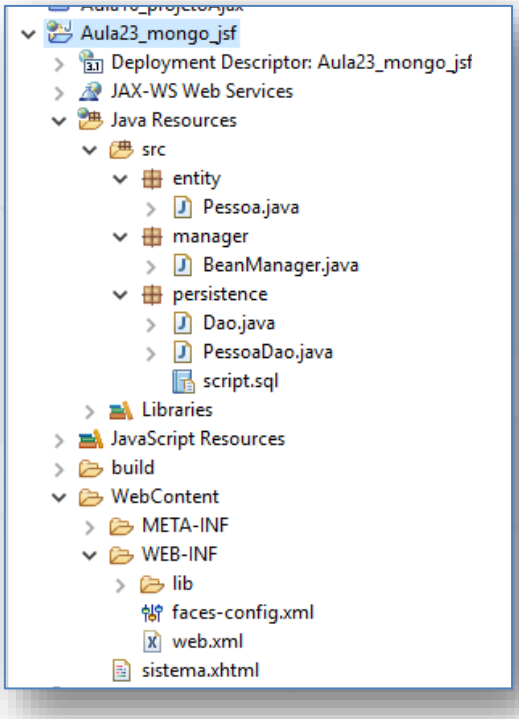
https://docs.mongodb.org/manual/introduction/#document-database



http://imasters.com.br/banco-de-dados/mongodb/mongodb-com-suporte-joins/?trace=1519021197&source=category-archive

https://www.mongodb.org/

Estrutura do projeto:



LIBS:

all-themes-1.0.10.jar
commons-beanutils-1.8.0.jar
commons-collections-3.2.1.jar
commons-digester-2.0.jar
commons-logging-1.1.1.jar
javax.faces-2.0.11.jar
jstl.jar
mongo-2.10.1.jar
primefaces-5.0-sources.jar
primefaces-5.0.jar
standard.jar

```
package entity;

import java.io.Serializable;

public class Pessoa implements Serializable{

	private static final long serialVersionUID = 1L;

	  private  Object _id;
	  private Double id;
	  private String nome;
	  private String email;
	  private Double idade;

	public Pessoa() {
	}

	public Pessoa(Object _id, Double id, String nome, String
email, Double idade) {
		this._id = _id;
		this.id = id;
		this.nome = nome;
		this.email = email;
		this.idade = idade;
	}

	@Override
	public String toString() {
		return "Pessoa [id=" + id + ", nome=" + nome + ",
email=" + email + ", idade=" + idade + "]";
	}

	  public Double getId() {
		return id;
	}
	public void setId(Double id) {
		this.id = id;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
```

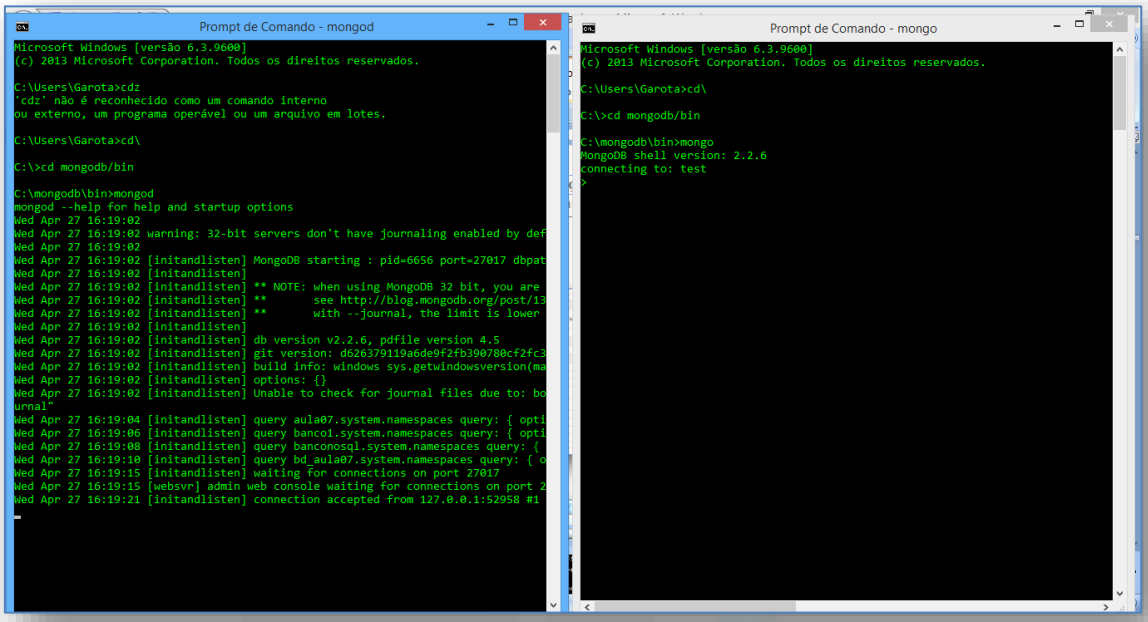
```
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public Double getIdade() {
		return idade;
	}
	public void setIdade(Double idade) {
		this.idade = idade;
	}
	public Object get_id() {
		return _id;
	}
	public void set_id(Object _id) {
		this._id = _id;
	}
}
```

Eu vou entrar 2 vezes no prompt do DOS

Iniciar / executar / cmd
2 x

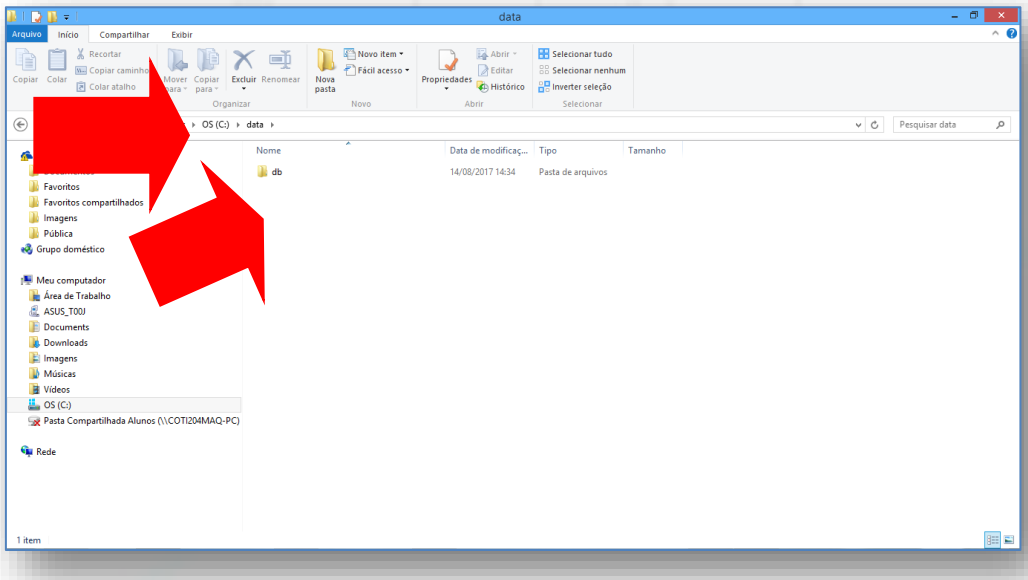
O primeiro prompt irá segurar o Banco de Dados:
Cd\
Cd mongodb/bin
mongod
C:\mongodb\bin\ (start no banco) **mongod**

No Segundo:
Cd\
Cd mongodb/bin
mongo
Cd c:\mongodb\bin
Mongo (start no servidor)



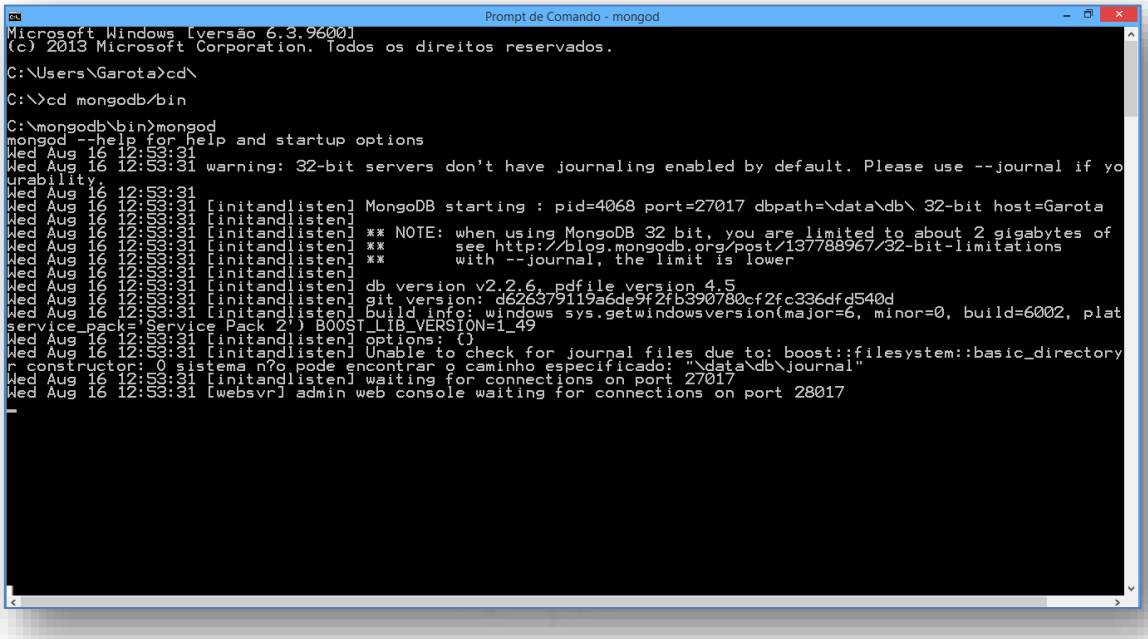
Para instalar o MongoDB no Windows...

- Descompactar o arquivo de instalação no “c:\”
- Renomear a pasta para “mongodb”
- Criar uma pasta chamada “data” e dentro dela criar uma pasta chamada “db”

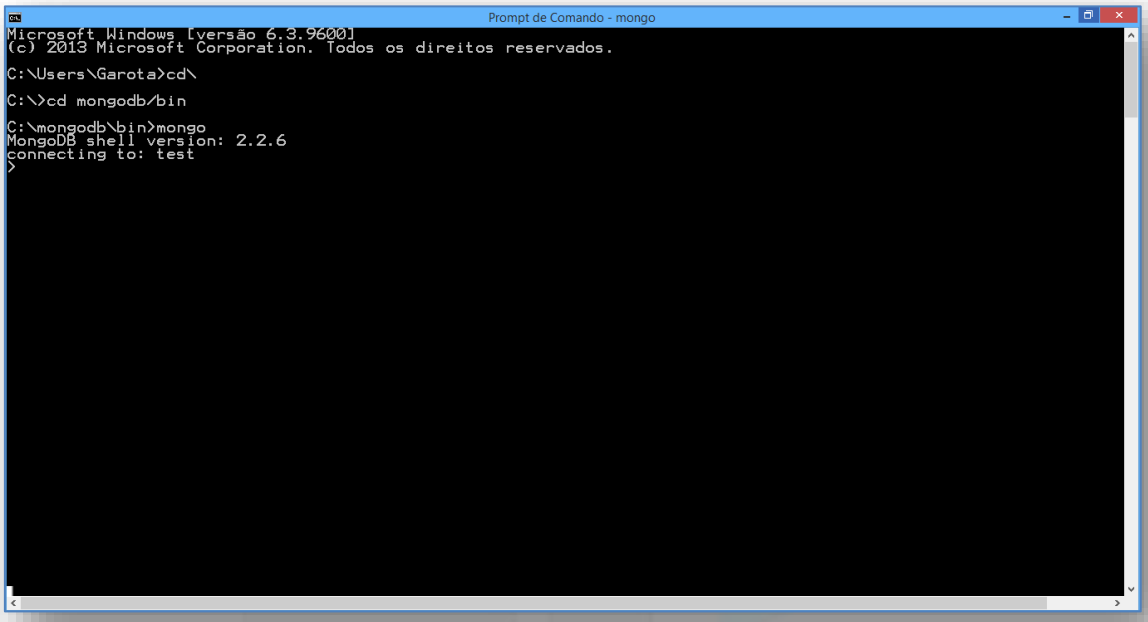


Para abrir o mongo...

- Abrir o promp de comando do DOS.
- Digitar “cd\” para voltar para o diretório raiz.
- Digitar o caminho de onde o mongo está descompactado: “cd mongodb/bin”.
- Digitar “mongod” para abrir o servidor.



- Abrir outro promp de comando do DOS.
- Digitar “cd\” para voltar para o diretório raiz.
- Digitar o caminho de onde o mongo está descompactado: “cd mongodb/bin”.
- Digitar “mongo” para abrir o banco.



script.sql

```
--banco nsql (Documentos) ,  Cassandra (Big Data)
--der, comandos relativos ao mode tradicional de tradicional ...
--Mondogd orientado a Indice ...
--comando json (javascript)
--modelagem nem pensar
--json (linguagem que estÃ¡ por trÃ¡s do mongo) ...
--SeguranÃ§a (nÃ£o tÃ£o confiavel ...)
--Pra que utiliza hibernate...


--nosql (MongoDb) -- Cassandra
-- Sistema Indice
```

Criando e logando no "bdmongo"...

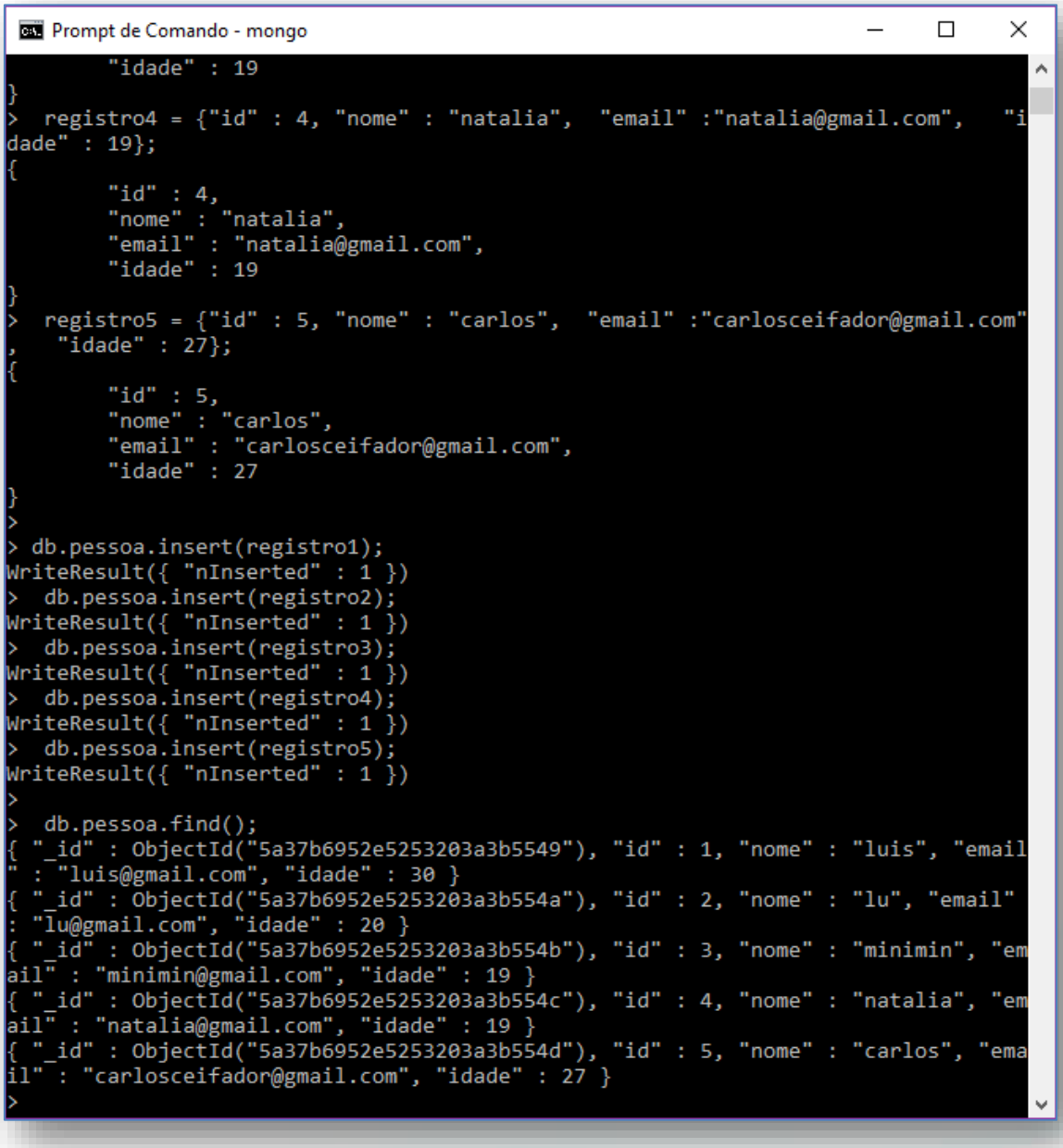
```
use bdmongo
```

Criando as variaveis...

```
  registro1 = {"id" : 1, "nome" : "luis", "email"
:"luis@gmail.com", "idade" : 30};
  registro2 = {"id" : 2, "nome" : "lu", "email" :"lu@gmail.com",
"idade" : 20};
  registro3 = {"id" : 3, "nome" : "minimin", "email"
:"minimin@gmail.com",   "idade" : 19};
 registro4 = {"id" : 4, "nome" : "natalia",  "email"
:"natalia@gmail.com",   "idade" : 19};
 registro5 = {"id" : 5, "nome" : "carlos",  "email"
:"carlosceifador@gmail.com",   "idade" : 27};
```

Criando a tabela "pessoa" e inserindo nela as variaveis...

```
db.pessoa.insert(registro1);
db.pessoa.insert(registro2);
db.pessoa.insert(registro3);
db.pessoa.insert(registro4);
db.pessoa.insert(registro5);

db.pessoa.find();
```


Prompt de Comando - mongo
"idade" : 19
}
> registro4 = {"id" : 4, "nome" : "natalia", "email" :"natalia@gmail.com", "idade" : 19};
{
"id" : 4,
"nome" : "natalia",
"email" : "natalia@gmail.com",
"idade" : 19
}
> registro5 = {"id" : 5, "nome" : "carlos", "email" :"carlosceifador@gmail.com", "idade" : 27};
{
"id" : 5,
"nome" : "carlos",
"email" : "carlosceifador@gmail.com",
"idade" : 27
}
>
> db.pessoa.insert(registro1);
WriteResult({ "nInserted" : 1 })
> db.pessoa.insert(registro2);
WriteResult({ "nInserted" : 1 })
> db.pessoa.insert(registro3);
WriteResult({ "nInserted" : 1 })
> db.pessoa.insert(registro4);
WriteResult({ "nInserted" : 1 })
> db.pessoa.insert(registro5);
WriteResult({ "nInserted" : 1 })
>
> db.pessoa.find();
{ "_id" : ObjectId("5a37b6952e5253203a3b5549"), "id" : 1, "nome" : "luis", "email" : "luis@gmail.com", "idade" : 30 }
{ "_id" : ObjectId("5a37b6952e5253203a3b554a"), "id" : 2, "nome" : "lu", "email" : "lu@gmail.com", "idade" : 20 }
{ "_id" : ObjectId("5a37b6952e5253203a3b554b"), "id" : 3, "nome" : "minimin", "email" : "minimin@gmail.com", "idade" : 19 }
{ "_id" : ObjectId("5a37b6952e5253203a3b554c"), "id" : 4, "nome" : "natalia", "email" : "natalia@gmail.com", "idade" : 19 }
{ "_id" : ObjectId("5a37b6952e5253203a3b554d"), "id" : 5, "nome" : "carlos", "email" : "carlosceifador@gmail.com", "idade" : 27 }
>

Pesquisar todos os registros mostrando de forma organizada..

```
--mostrando
 db.pessoa.find().pretty();
```

```
--exibindo de uma forma Organizada

 show collections;
-- mostra as tabelas
```

pessoa
system.indexes

```
 var consulta = db.pessoa.find();
while(consulta.hasNext()){
     var registro = consulta.next();
      printjson(' Nome --> ' + registro.nome );
}
```

" Nome --> luis"
" Nome --> lu"
" Nome --> minimin"
" Nome --> natalia"
" Nome --> carlos"

```
db.pessoa.find().sort({nome : 1}).pretty();
```

```
db.pessoa.find().sort({id : -1}).pretty();
```

```
db.pessoa.find({nome :/^l/}).pretty();
```

```
{
	"_id" : ObjectId("5a37b6952e5253203a3b5549"),
	"id" : 1,
	"nome" : "luis",
	"email" : "luis@gmail.com",
	"idade" : 30
}
{
	"_id" : ObjectId("5a37b6952e5253203a3b554a"),
	"id" : 2,
	"nome" : "lu",
	"email" : "lu@gmail.com",
	"idade" : 20
}
```

```
db.pessoa.find(
  {idade :  "19"}
).pretty();
```

```
db.pessoa.find(
  {nome :  "lu"}
).pretty();
```

```
{
	"_id" : ObjectId("5a37b6952e5253203a3b554a"),
	"id" : 2,
	"nome" : "lu",
	"email" : "lu@gmail.com",
	"idade" : 20
}
```

Apagando a tabela pessoa...

```
db.pessoa.drop();
```

```
 db.pessoa.find(
   {idade : { $lt : 20 }}
).pretty();
```

```
 db.pessoa.remove(
  { nome : "luis" }
);
```

WriteResult({ "nRemoved" : 1 })

```
 db.pessoa.find().pretty();
```

Prompt de Comando - mongo
> db.pessoa.find().pretty();
{
"_id" : ObjectId("5a37b6952e5253203a3b554a"),
"id" : 2,
"nome" : "lu",
"email" : "lu@gmail.com",
"idade" : 20
}
{
"_id" : ObjectId("5a37b6952e5253203a3b554b"),
"id" : 3,
"nome" : "minimin",
"email" : "minimin@gmail.com",
"idade" : 19
}
{
"_id" : ObjectId("5a37b6952e5253203a3b554c"),
"id" : 4,
"nome" : "natalia",
"email" : "natalia@gmail.com",
"idade" : 19
}
{
"_id" : ObjectId("5a37b6952e5253203a3b554d"),
"id" : 5,
"nome" : "carlos",
"email" : "carlosceifador@gmail.com",
"idade" : 27
}
>

```
db.pessoa.find(
 { $or :[{nome:'lu'},{nome : 'natalia'}]}
).pretty();
```

```
{
	"_id" : ObjectId("5a37b6952e5253203a3b554a"),
	"id" : 2,
	"nome" : "lu",
	"email" : "lu@gmail.com",
	"idade" : 20
}
{
	"_id" : ObjectId("5a37b6952e5253203a3b554c"),
	"id" : 4,
	"nome" : "natalia",
	"email" : "natalia@gmail.com",
	"idade" : 19
}
```

```
package persistence;

import com.mongodb.BasicDBObject;
import com.mongodb.DB;
import com.mongodb.DBCursor;
import com.mongodb.Mongo;

public class Dao {

	DB con;
	BasicDBObject stmt;
	DBCursor rs;

	    public void open() throws Exception{
	        Mongo mongo =new Mongo();
	            con = mongo.getDB("bdmongo");
	        }
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;

import com.mongodb.BasicDBObject;
import com.mongodb.DBCollection;
import com.mongodb.DBObject;

import entity.Pessoa;

public class PessoaDao  extends Dao{

	public void create(Pessoa p) throws Exception{
		open();
		 DBCollection tabela = con.getCollection("pessoa");
		 stmt = new BasicDBObject();
		  stmt.put("id", p.getId());
		  stmt.put("nome", p.getNome());
		  stmt.put("email", p.getEmail());
		  stmt.put("idade",p.getIdade());
		tabela.insert(stmt);
	}

	public List<Pessoa> findAll() throws Exception{
		open();
		 DBCollection tabela = con.getCollection("pessoa");
		 rs = tabela.find(stmt);
		 List<Pessoa> lst = new ArrayList<Pessoa>();
		 while(rs.hasNext()){
			DBObject  registro = rs.next();
			Pessoa p = new Pessoa();
				p.set_id((Object) registro.get("_id"));
				p.setId((Double) registro.get("id"));
			 p.setNome((String) registro.get("nome"));
			 p.setEmail((String) registro.get("email"));
			 p.setIdade((Double) registro.get("idade"));
			lst.add(p);
		 }
		rs.close();
		 return lst;
	}
```

```
        public static void main(String[] args) {
            try{
                new PessoaDao().create(
        new Pessoa(null,200.,"ronaldinho","ron@gmail.com",70.)
                                );
                //new
PessoaDao().findAll().forEach(System.out::println);
                System.out.println("Dados Gravados ....");
            }catch(Exception ex){
                ex.printStackTrace();
            }
        }
}
```

No console...

```
Dados Gravados ....
```

```
package manager;

import java.io.Serializable;
import java.util.List;

import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.RequestScoped;
import javax.faces.context.FacesContext;
import entity.Pessoa;
import persistence.PessoaDao;

@ManagedBean(name="mb")
@RequestScoped
public class BeanManager implements Serializable {

	private static final long serialVersionUID = 1L;

	private Pessoa pessoa;
	private List<Pessoa> pessoas;

	public BeanManager() {
		pessoa = new Pessoa();
	}

	public Pessoa getPessoa() {
		return pessoa;
	}

	public void setPessoa(Pessoa pessoa) {
		this.pessoa = pessoa;
	}

	public List<Pessoa> getPessoas() {
		try{
			this.pessoas = new PessoaDao().findAll();
		   }catch(Exception ex){
			ex.printStackTrace();
		  }
		return pessoas;
	}

	public void setPessoas(List<Pessoa> pessoas) {
```

```
            this.pessoas = pessoas;
      }

      public void  gravar(){
         FacesContext fc = FacesContext.getCurrentInstance();
          try{
             new PessoaDao().create(pessoa);
              pessoa = new Pessoa();
               fc.addMessage(null, new FacesMessage("Dados
Gravados ..."));
          }catch(Exception ex){
             fc.addMessage(null, new FacesMessage("Error :" +
ex.getMessage()));
          }
      }
}
```

---

web.xml

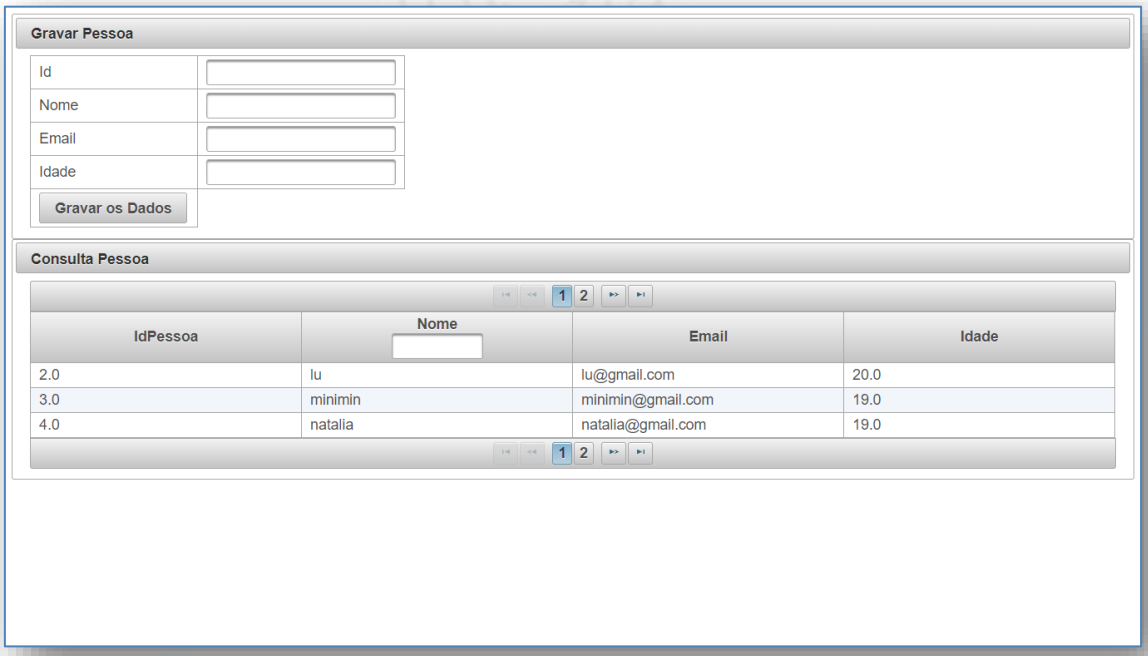
```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
version="3.1">
  <display-name>mongo_jsf</display-name>
  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>
</web-app>
```

---

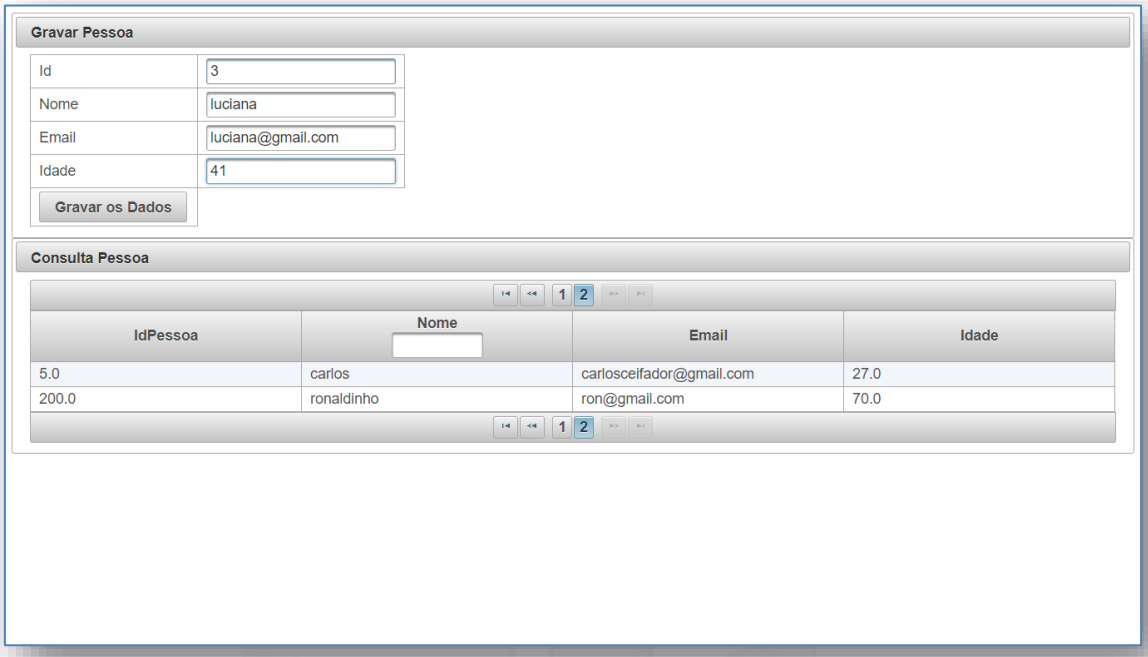
sistema.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head>
</h:head>

<h:body>
   <p:panel header="Gravar Pessoa">
    <h:form id="form1">
      <p:panelGrid columns="2">

          <h:outputText value="Id"></h:outputText>
          <p:inputText value="#{mb.pessoa.id}" required="true"/>

          <h:outputText value="Nome"></h:outputText>
          <p:inputText value="#{mb.pessoa.nome}"
required="true"/>

          <h:outputText value="Email"></h:outputText>
          <p:inputText value="#{mb.pessoa.email}"
required="true"/>

           <h:outputText value="Idade"></h:outputText>
          <p:inputText value="#{mb.pessoa.idade}"
required="true"/>

         <p:commandButton value="Gravar os Dados"
                                    action="#{mb.gravar}"
                                update=":form1, :form2"
                                    ajax="true"

         ></p:commandButton>
    </p:panelGrid>

    <p:messages></p:messages>

    </h:form>
   </p:panel>
```

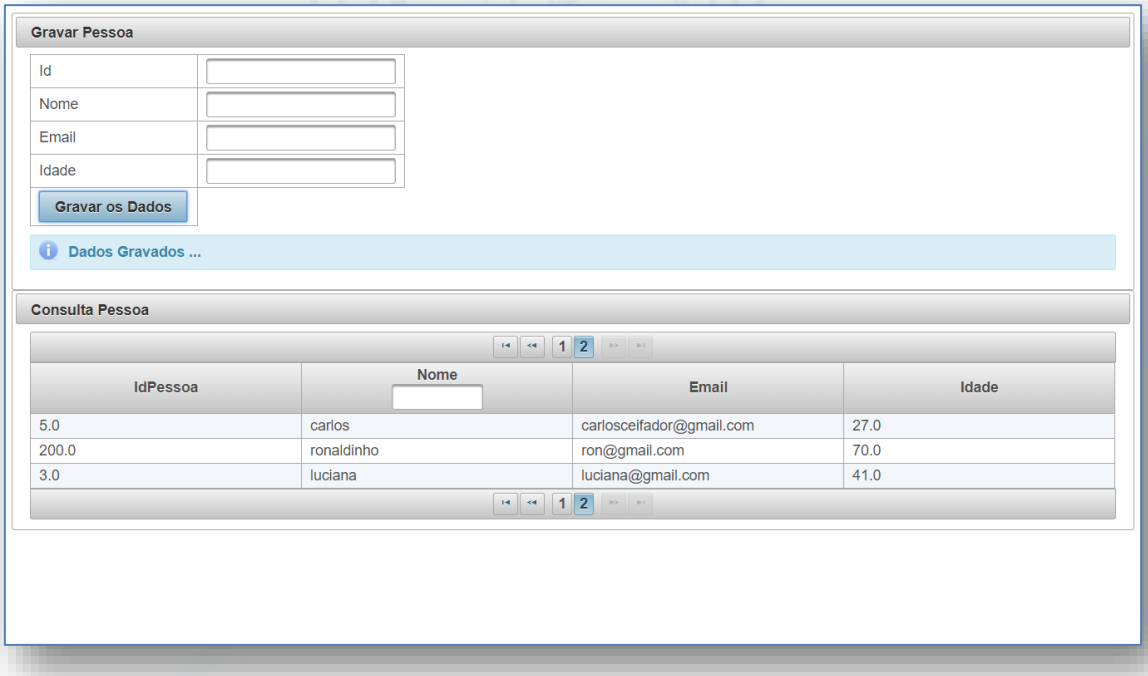
```
    <p:panel header="Consulta Pessoa">
     <h:form id="form2">
       <p:dataTable  value="#{mb.pessoas}" var="linha"
paginator="true" rows="3">
        <p:column headerText="IdPessoa">
            <h:outputText value="#{linha.id}" />
        </p:column>
          <p:column headerText="Nome" filterBy="#{linha.nome}">
            <h:outputText value="#{linha.nome}" />
        </p:column>
        <p:column headerText="Email">
            <h:outputText value="#{linha.email}" />
        </p:column>
        <p:column headerText="Idade">
            <h:outputText value="#{linha.idade}" />
        </p:column>
      </p:dataTable>
      </h:form>
    </p:panel>

</h:body>

</html>
```



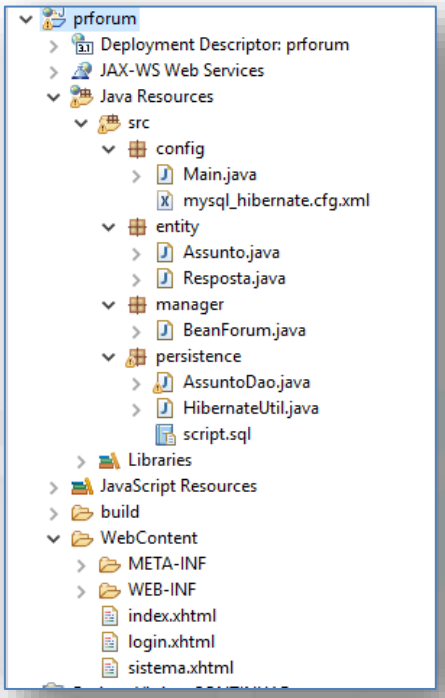
Cadastradno uma pessoa...



Gravado e atualizado na lista...



Estrutura do projeto:



LIBS:

all-themes-1.0.10.jar
antlr-2.7.6.jar
asm-attrs.jar
asm.jar
c3p0-0.9.0.jar
cglib-2.1.3.jar
commons-beanutils-1.8.0.jar
commons-codec-1.3.jar
commons-collections-3.2.1.jar
commons-digester-2.0.jar
commons-el-1.0.jar
commons-fileupload-1.0.jar
commons-lang-2.1.jar
commons-logging-1.1.1.jar
commons-validator.jar
dom4j-1.6.1.jar
ejb3-persistence.jar
hibernate-annotations.jar
hibernate-commons-annotations.jar

hibernate3.jar
jakarta-oro.jar
javax.faces-2.0.11.jar
jcommon_javadoc.jar
jcommon_source.jar
jcommon.jar
jstl-1.1.0.jar
jstl.jar
jta.jar
log4j-1.2.15.jar
mysql-connector-java-5.1.23-bin.jar
oro-2.0.8.jar
primefaces-5.0-sources.jar
primefaces-5.0.jar
servlet3-api.jar
standard.jar

```
package entity;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.FetchType;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToMany;
import javax.persistence.Table;

@Entity
@Table(name="assunto")
public class Assunto implements Serializable {

	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	private  Integer id=0;
	@Column
	private  String  assunto;
```

```
	@Column
	private  String  comentario;
	@Column
	private  String  nome;
	@Column(unique=true)
	private  String  email;
	@Column
	private  String  senha;

    @OneToMany(mappedBy="assunto",fetch=FetchType.EAGER)
	private List<Resposta> respostas;

	public Assunto() {

	}

	public Assunto(Integer id, String assunto, String
comentario, String nome, String email, String senha) {
		super();
		this.id = id;
		this.assunto = assunto;
		this.comentario = comentario;
		this.nome = nome;
		this.email = email;
		this.senha = senha;
	}

	@Override
	public String toString() {
		return "Assunto [id=" + id + ", assunto=" + assunto +
", comentario=" + comentario + ", nome=" + nome + ", email=" +
email + "]";
	}

	public Integer getId() {
		return id;
	}
	public void setId(Integer id) {
		this.id = id;
	}
	public String getAssunto() {
		return assunto;
	}
	public void setAssunto(String assunto) {
		this.assunto = assunto;
```

```
	}
	public String getComentario() {
		return comentario;
	}
	public void setComentario(String comentario) {
		this.comentario = comentario;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getEmail() {
		return email;
	}
	public void setEmail(String email) {
		this.email = email;
	}
	public String getSenha() {
		return senha;
	}
	public void setSenha(String senha) {
		this.senha = senha;
	}
	public List<Resposta> getRespostas() {
		return respostas;
	}
	public void setRespostas(List<Resposta> respostas) {
		this.respostas = respostas;
	}

	public void adicionar(Resposta resposta){
		if (respostas==null){
			respostas = new ArrayList<Resposta>();
		}
		respostas.add(resposta);
	}
}
```

```
package entity;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.FetchType;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.Table;

@Entity
@Table(name="resposta")
public class Resposta implements Serializable{

	private static final long serialVersionUID = 1L;

 @Id
 @GeneratedValue(strategy=GenerationType.AUTO)
	private  Integer idResposta;
 @Column
	private String nome;
 @Column
	private String texto;

  @ManyToOne(fetch=FetchType.EAGER)
  @JoinColumn(name="FKidassunto")
	private Assunto assunto;

	public Resposta() {
	}

	public Resposta(Integer idResposta, String nome, String
texto) {
		this.idResposta = idResposta;
		this.nome = nome;
		this.texto = texto;
	}

	@Override
```

```
	public String toString() {
		return "Resposta [idResposta=" + idResposta + ",
nome=" + nome + ", texto=" + texto + "]";
	}

	public Integer getIdResposta() {
		return idResposta;
	}
	public void setIdResposta(Integer idResposta) {
		this.idResposta = idResposta;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public String getTexto() {
		return texto;
	}
	public void setTexto(String texto) {
		this.texto = texto;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
	}
	public Assunto getAssunto() {
		return assunto;
	}
	public void setAssunto(Assunto assunto) {
		this.assunto = assunto;
	}
}
```

---

mysql_hibernate.cfg.xml

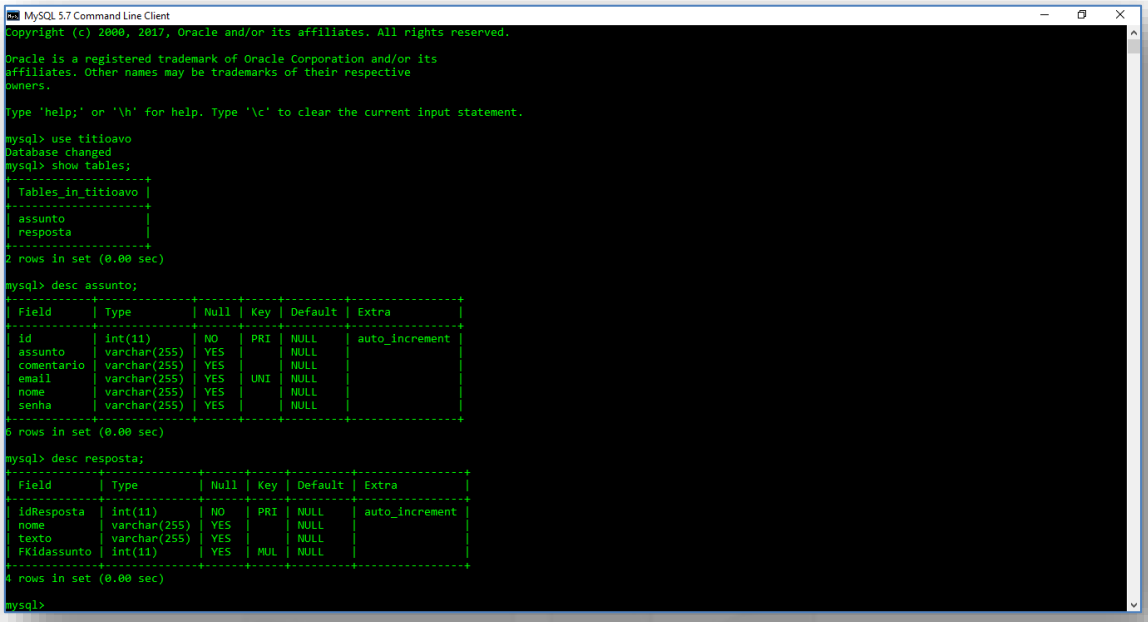
```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
```

```
  <session-factory>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</pro
perty>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</
property>
    <property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/titioavo</property>
    <property
name="hibernate.connection.username">root</property>
    <property
name="hibernate.connection.password">coti</property>

    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

        <mapping class="entity.Assunto"/>
        <mapping class="entity.Resposta"/>

 </session-factory>
</hibernate-configuration>
```

```
package config;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.cfg.Configuration;
import org.hibernate.tool.hbm2ddl.SchemaExport;

public class Main {

     public static void main(String[] args) {

          Configuration cfg = new AnnotationConfiguration().
     configure("config/mysql_hibernate.cfg.xml");
            new SchemaExport(cfg).create(true,true);
          System.out.println("Tabelas  Geradas Com Sucesso");

          /*
           *
           *     alter table resposta
```

```
        drop
        foreign key FKEBB72721784AE1DF

    drop table if exists assunto

    drop table if exists resposta

    create table assunto (
        id integer not null auto_increment,
        assunto varchar(255),
        comentario varchar(255),
        email varchar(255) unique,
        nome varchar(255),
        senha varchar(255),
        primary key (id)
    )

    create table resposta (
        idResposta integer not null auto_increment,
        nome varchar(255),
        texto varchar(255),
        FKidassunto integer,
        primary key (idResposta)
    )

    alter table resposta
        add index FKEBB72721784AE1DF (FKidassunto),
        add constraint FKEBB72721784AE1DF
        foreign key (FKidassunto)
        references assunto (id);
Tabelas  Geradas Com Sucesso


create table teste(id int primary key);
            */
     }
}
```

No banco...



script.sql

```
use titioavo;
#me conectei no banco
show tables;
#mostrei as tabelas
desc assunto;
desc resposta;

select * from assunto;
select * from resposta;

-------------------------------------------------
jdbc:mysql://node154426-cotiforumbelem.jelasticlw.com.br:3306/titioavo
```

```
package persistence;

import org.hibernate.cfg.AnnotationConfiguration;
```

```
import org.hibernate.SessionFactory;

public class HibernateUtil {

    private static final SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new AnnotationConfiguration().
configure("config/mysql_hibernate.cfg.xml").buildSessionFactory(
);
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation
failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

```
package persistence;

import java.util.List;

import org.hibernate.Criteria;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.Transaction;
import org.jcommon.encryption.SimpleMD5;

import entity.Assunto;
import entity.Resposta;

public class AssuntoDao {

     Session session;
     Transaction transaction;
     Criteria criteria;
     Query query;
```

```
    public void criptografar(Assunto a){
         SimpleMD5 md5 = new SimpleMD5(a.getSenha(),
                                 "profedsonbelem@gmail.com");
         a.setSenha(md5.toHexString());
    }

     public void create(Assunto a) throws Exception{
          criptografar(a);
  session = HibernateUtil.getSessionFactory().openSession();
           transaction  = session.beginTransaction();
              session.save(a);
          transaction.commit();
     session.close();
     }

     public void create(Assunto a,Resposta r) throws Exception{
           criptografar(a);
  session = HibernateUtil.getSessionFactory().openSession();
                transaction  = session.beginTransaction();
                    session.save(a);
                       r.setAssunto(a);
                         session.save(r);
              transaction.commit();
          session.close();
          }
                          //Nao estou gravando o Assunto
                              //busca no Assunto
     public void createOnlyResposta(Assunto a,Resposta r)
throws Exception{
  session = HibernateUtil.getSessionFactory().openSession();
                transaction  = session.beginTransaction();
                    r.setAssunto(a);
                  session.save(r);
              transaction.commit();
          session.close();
          }

     public List<Assunto> findAll(){
     session = HibernateUtil.getSessionFactory().openSession();
      List <Assunto> lst = session.createQuery("from
Assunto").list();
     session.close();
     return lst;
     }
```

```
       public Assunto findByCode(Integer cod){
        session =
HibernateUtil.getSessionFactory().openSession();
    Assunto assunto =(Assunto) session.get(Assunto.class, cod);
      session.close();
           return assunto;
      }

      public Assunto findByLogin(Assunto a){
           criptografar(a);
       session = HibernateUtil.getSessionFactory().openSession();
         query = session.createQuery("from Assunto a where
a.email=:param1 and a.senha=:param2");
         query.setParameter("param1", a.getEmail());
         query.setParameter("param2", a.getSenha());
       Assunto resposta = (Assunto) query.uniqueResult();
    session.close();
      return resposta;
      }

      public static void main(String[] args) {
           try{
                //Busquei
        Assunto a = new AssuntoDao().findByCode(1);
                Resposta resp = new Resposta();
                 resp.setNome("lu");
                 resp.setTexto("java com Rest-SpringBoot");
                 resp.setAssunto(a); //relacionei ao assunto
            new AssuntoDao().createOnlyResposta(a, resp);
                System.out.println("Dados Gravados ...");
           }catch(Exception ex){
                ex.printStackTrace();
           }
      }
}
```

```
package manager;

import java.io.Serializable;
import java.util.List;

import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.RequestScoped;
import javax.faces.component.UIParameter;
import javax.faces.context.FacesContext;
import javax.faces.event.ActionEvent;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpSession;

import entity.Assunto;
import entity.Resposta;
import persistence.AssuntoDao;


@ManagedBean(name="mb")
@RequestScoped
public class BeanForum implements Serializable{

	private static final long serialVersionUID = 1L;

	 private Assunto assunto;
	 private Resposta resposta;
	 private Assunto logado;//nao tem espaco de mem

	 private Integer mbCodigo=0;
	 private String  mbResposta="";
	 private Resposta resposta2;
	 private List<Assunto> listaAssuntos;

	 public BeanForum() {
	  assunto = new Assunto();
	  resposta = new Resposta();
	  resposta2 = new Resposta();
	}

	public List<Assunto> getListaAssuntos() {
		  listaAssuntos = new AssuntoDao().findAll();
		return listaAssuntos;
```

```
	}

	public Resposta getResposta2() {
		return resposta2;
	}
	public void setResposta2(Resposta resposta2) {
		this.resposta2 = resposta2;
	}
	public void setListaAssuntos(List<Assunto> listaAssuntos) {
		this.listaAssuntos = listaAssuntos;
	}
	public Assunto getAssunto() {
		return assunto;
	}
	public void setAssunto(Assunto assunto) {
		this.assunto = assunto;
	}
	public Resposta getResposta() {
		return resposta;
	}
	public void setResposta(Resposta resposta) {
		this.resposta = resposta;
	}
	public Assunto getLogado() {
		return logado;
	}
	public void setLogado(Assunto logado) {
		this.logado = logado;
	}
	public static long getSerialversionuid() {
		return serialVersionUID;
	}
	}



	 public void gravarPrimeira(){
	 FacesContext fc =FacesContext.getCurrentInstance();
	 try{
		 AssuntoDao dao = new AssuntoDao();
		 dao.create(assunto, new Resposta());
		     assunto = new Assunto();
fc.addMessage(null, new FacesMessage("Dados Gravados"));
	 }catch(Exception ex){
		 assunto = new Assunto();
```

```
fc.addMessage(null, new FacesMessage("Error:" +
ex.getMessage()));
       }
       }

       public String logar(){
       FacesContext fc = FacesContext.getCurrentInstance();
            try{
      logado = new AssuntoDao().findByLogin(assunto);
       if (logado !=null){
         HttpServletRequest request = (HttpServletRequest)
              fc.getExternalContext().getRequest();
           HttpSession session = request.getSession(true);
           session.setAttribute("usuariolog", logado );
         fc.addMessage(null,new FacesMessage("Usuario Logado") );
         return "sistema.jsf?faces-redirect";
      }
     else{
     fc.addMessage(null,new FacesMessage("Acesso Negado") );
     }
     }catch(Exception ex){
     }
      return null;
       }

       public void mostrarDados(){
            FacesContext fc = FacesContext.getCurrentInstance();
            try{
           HttpServletRequest request = (HttpServletRequest)
                fc.getExternalContext().getRequest();
            HttpSession  session = request.getSession(true);
       Assunto ux =(Assunto) session.getAttribute("usuariolog");
          //  mbCodigo = ux.getId();
            resposta.setAssunto(ux);
            resposta.setNome(ux.getNome());
       fc.addMessage(null, new FacesMessage("Carga nos Dados"));
            }catch(Exception ex){
                 ex.printStackTrace();
         fc.addMessage(null, new FacesMessage("Error:" +
ex.getMessage()));
            }
       }

       public void gravarResposta(ActionEvent event){
            FacesContext fc = FacesContext.getCurrentInstance();
```

```
            try{
            HttpServletRequest request = (HttpServletRequest)
                        fc.getExternalContext().getRequest();
                  HttpSession  session = request.getSession(true);
      Assunto ux =(Assunto) session.getAttribute("usuariolog");
                  assunto = ux;
      UIParameter  p =(UIParameter)
event.getComponent().findComponent("idAssunto");
               mbCodigo = (Integer) p.getValue();
        System.out.println("************ PAssando :" + mbCodigo);
       new AssuntoDao().createOnlyResposta(assunto, resposta2);
        fc.addMessage(null, new FacesMessage("Dados Gravados"));
             }catch(Exception ex){
                  ex.printStackTrace();
          fc.addMessage(null, new FacesMessage("Error:" +
ex.getMessage()));
             }
       }

      public String getMbResposta() {
            return mbResposta;
      }
      public void setMbResposta(String mbResposta) {
            this.mbResposta = mbResposta;
      }
      public Integer getMbCodigo() {
            return mbCodigo;
      }
      public void setMbCodigo(Integer mbCodigo) {
            this.mbCodigo = mbCodigo;
      }
}
```

---

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <display-name>prforum</display-name>
```

```
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>
</web-app>
```

---

index.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head>
</h:head>


<h:body>
<h3 style="margin-top:0">Menu Principal</h3>

<p:menu>
 <p:submenu label="Navigations">
   <p:menuitem value="cadastrar" outcome="index.jsf"
        icon="ui-icon-star"></p:menuitem>
   <p:menuitem value="logar"  outcome="login.jsf"
        icon="ui-icon-star"></p:menuitem>
```

```
 </p:submenu>

</p:menu>


<p/>
<hr/>

<p:panel header="Gravar Forum">
   <h:form id="form1">
     <p:panelGrid columns="2">
          <h:outputText value="Assunto" />
          <p:inputText value="#{mb.assunto.assunto}" />

          <h:outputText value="Comentario" />
          <p:inputText value="#{mb.assunto.comentario}" />

          <h:outputText value="Nome" />
          <p:inputText value="#{mb.assunto.nome}" />

          <h:outputText value="Email" />
          <p:inputText value="#{mb.assunto.email}" />

          <h:outputText value="Senha" />
          <p:password value="#{mb.assunto.senha}" />
          <p:commandButton value="Cadastrar"
                           ajax="true"
                           update=":form1"
                           action="#{mb.gravarPrimeira}">
           </p:commandButton>
           <p:growl></p:growl>
     </p:panelGrid>
   </h:form>
 </p:panel>
</h:body>


</html>
```

login.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">

<h:head>
</h:head>
<h:body>

<p:panel header="Login no Sistema ">
 <h:form id="form2">
 <p:panelGrid columns="2">
  <h:outputText value="Email" />
  <p:inputText value="#{mb.assunto.email}" />

 <h:outputText value="Senha" />
  <p:password value="#{mb.assunto.senha}" />

  <p:commandButton value="Logar no Sistema"
        action="#{mb.logar}"
        ajax="false">
   </p:commandButton>
 </p:panelGrid>

  <p:growl></p:growl>
 </h:form>
 </p:panel>
</h:body>
</html>
```

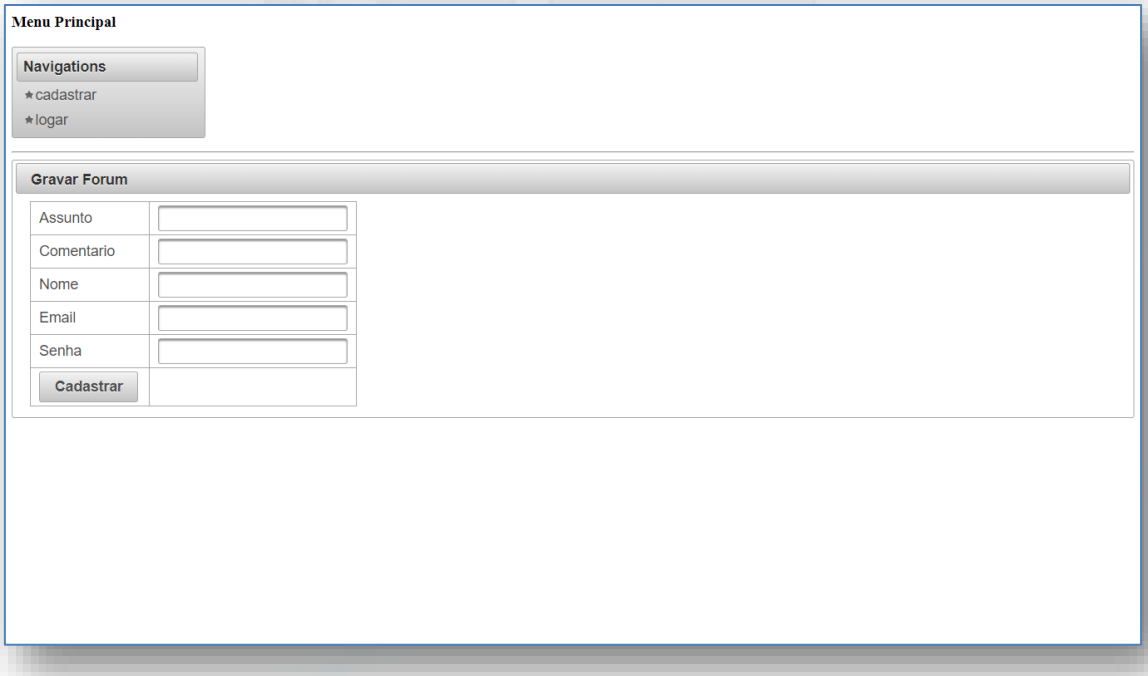
sistema.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">


<h:head>
</h:head>

<h:body>
 <p:panel header="Logado no Sistema: #{usuariolog.nome}">
 <h:form id="form6">
  <p:dataTable value="#{mb.listaAssuntos}" var="assunto"
    border="1" rows="4" paginator="true" width="100%">

    <f:facet name="header">
      Respostas ao Forum
    </f:facet>


      <p:column style="width:16px">
        <p:rowToggler />
      </p:column>
    <p:column headerText="Nome"  style="width:15%" >
        <h:outputText value="#{assunto.nome}" />
    </p:column>
    <p:column headerText="Email"  style="width:15%">
        <h:outputText value="#{assunto.email}" />
    </p:column>
    <p:column headerText="Assunto">
        <h:outputText value="#{assunto.assunto}"
style="width:20%" />
    </p:column>
    <p:column headerText="Comentario">
        <h:outputText value="#{assunto.comentario}"
style="width:20%"/>
    </p:column>

    <p:rowExpansion>
```

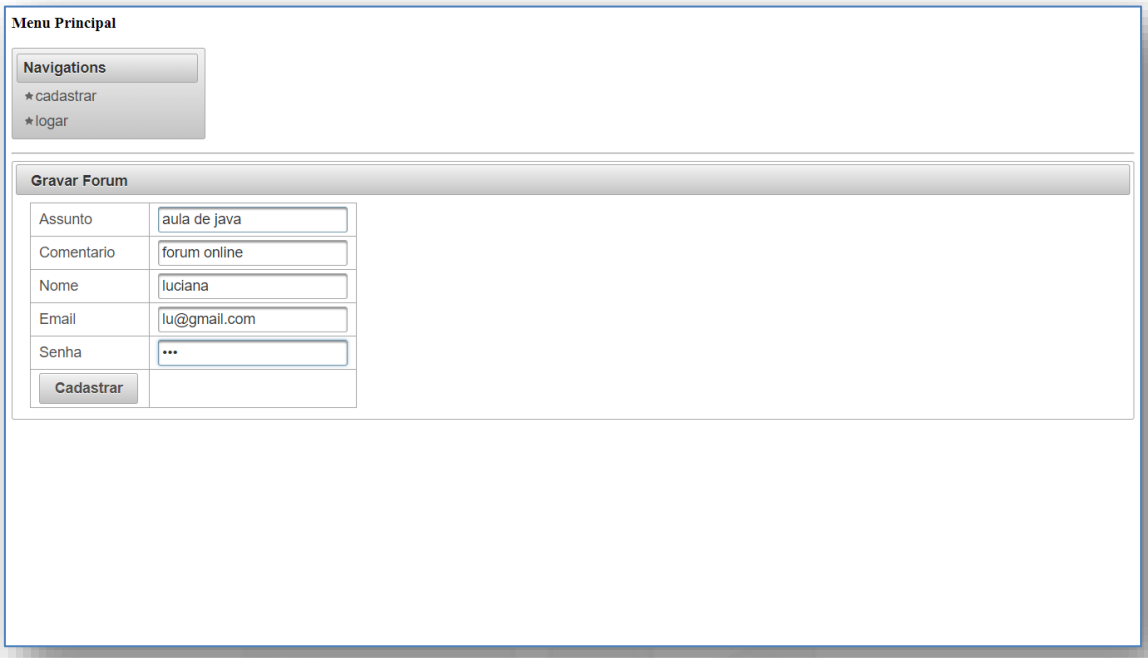
```
      <p:dataGrid value="#{assunto.respostas}" var="linha"
width="100%"
       columns="1">

      <p:panelGrid columns="2">
       <f:facet name="header">
         <h:outputText value="#{linha.nome}" />
       </f:facet>

       <h:outputText value="Nome :" />
       <h:outputText value="#{linha.nome}" />

       <h:outputText value="Texto :" />
       <h:outputText value="#{linha.texto}"  style="color:red"/>
      </p:panelGrid>

    </p:dataGrid>
    <br/>
      <p:commandButton value="Responder"
         update=":form7"
         ajax="true"
         action="#{mb.mostrarDados}"
onclick="PF('resposta1').show();"
         >
    </p:commandButton>
   </p:rowExpansion>

 </p:dataTable>
  </h:form>
 </p:panel>

 <p:dialog header="responder" widgetVar="resposta1"
modal="true">
  <h:form id="form7">
    <p:panelGrid columns="2">
        <h:outputText value="idAssunto" />
        <h:inputText value=" #{mb.resposta.assunto.id}" />

         <h:outputText value="Nome" />
         <h:inputText value=" #{mb.resposta.nome}" />

          <h:outputText value="Texto" />
          <h:inputText value=" #{mb.resposta.texto}" />

        <p:commandButton value="Gravar Resposta"
```

```
                                update=":form7, :form6"
                                action="#{mb.gravarResposta}"
                                ajax="true"
                    >
            <f:setPropertyActionListener value="#{mb.resposta}"
             target="#{mb.resposta2}"></f:setPropertyActionListener>

                    </p:commandButton>
            </p:panelGrid>
              <p:growl></p:growl>
            </h:form>
         </p:dialog>

</h:body>
</html>
```

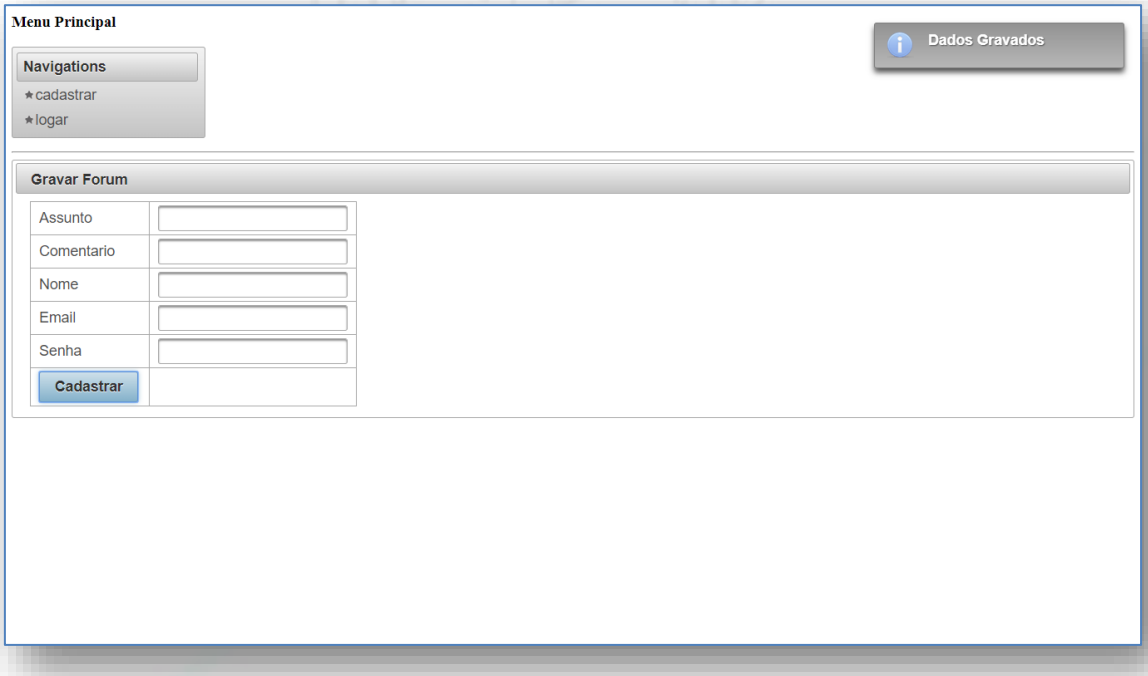
Rodando o projeto LOCALMENTE..



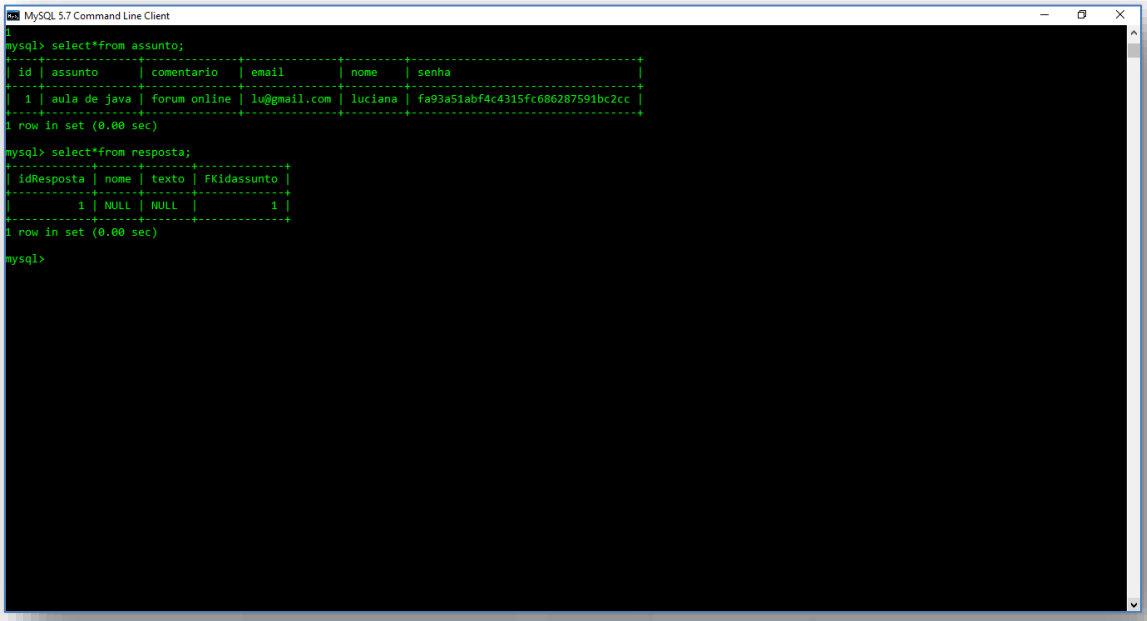
Cadastrando no forum...



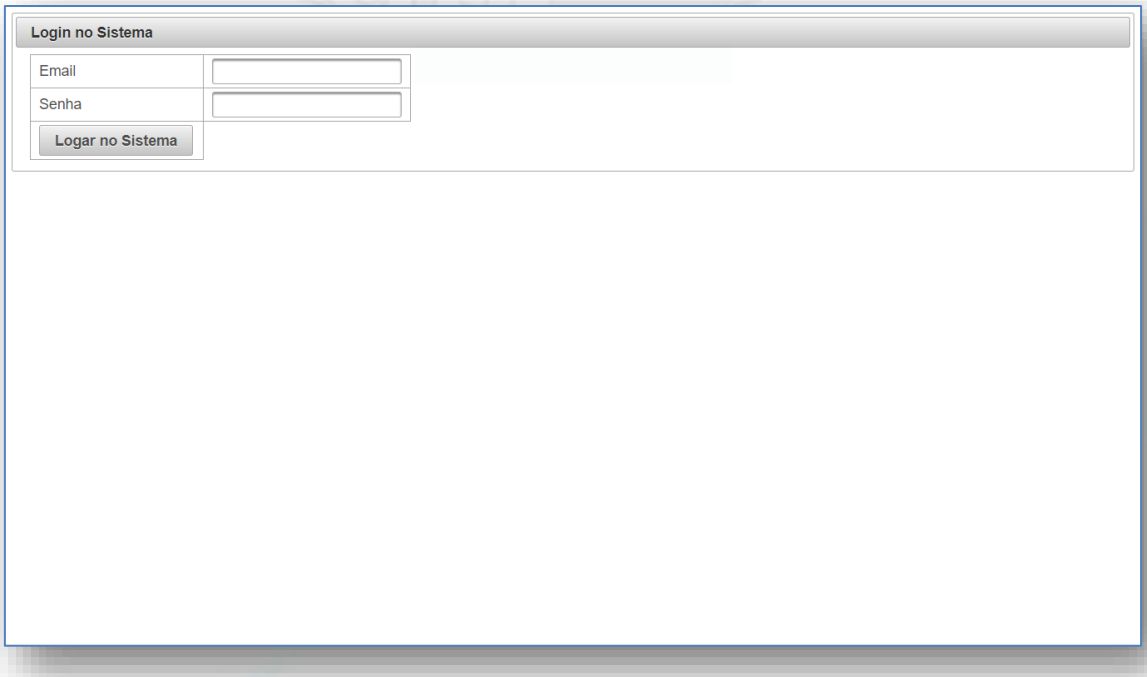
Gravado...



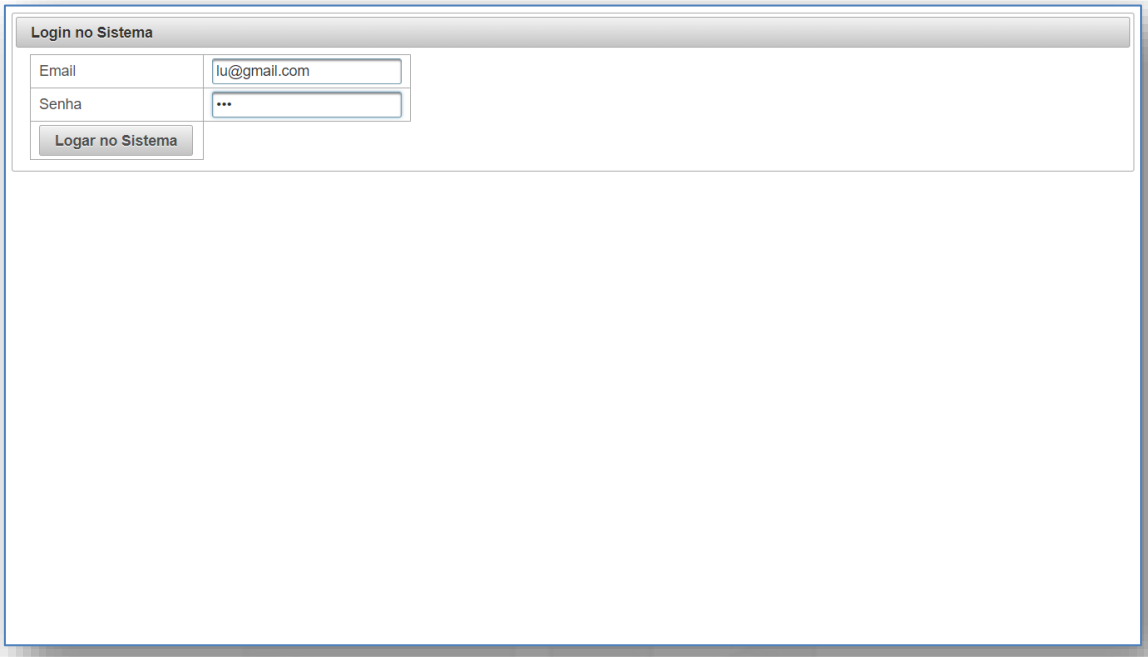
No banco...



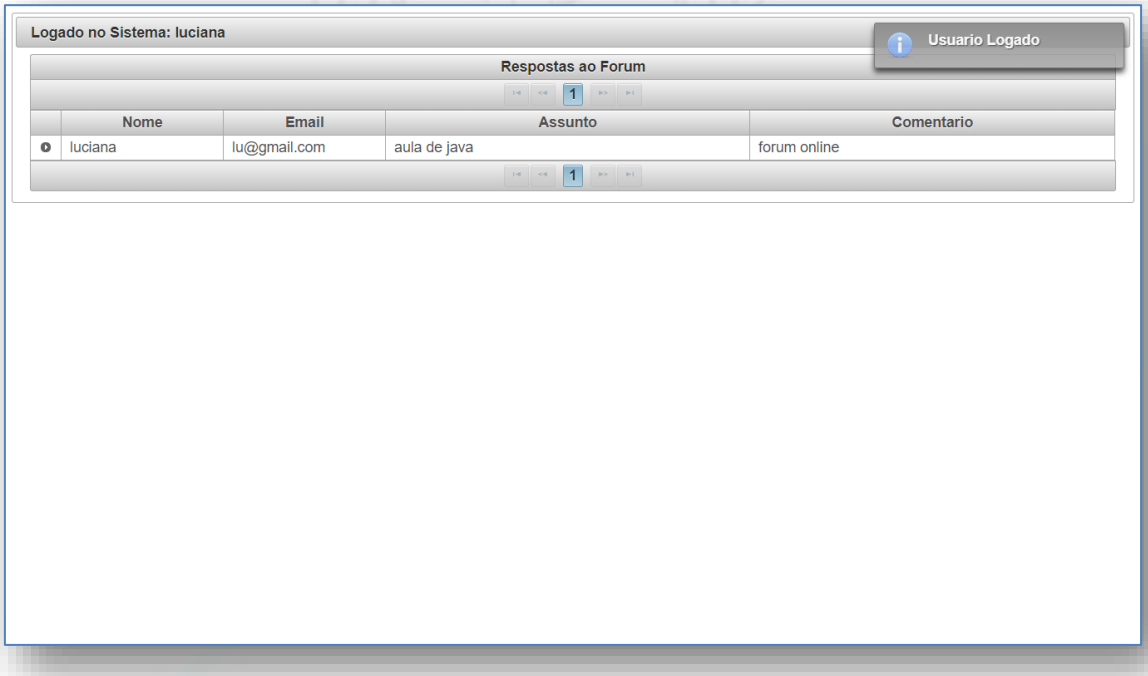
Clicando em logar...



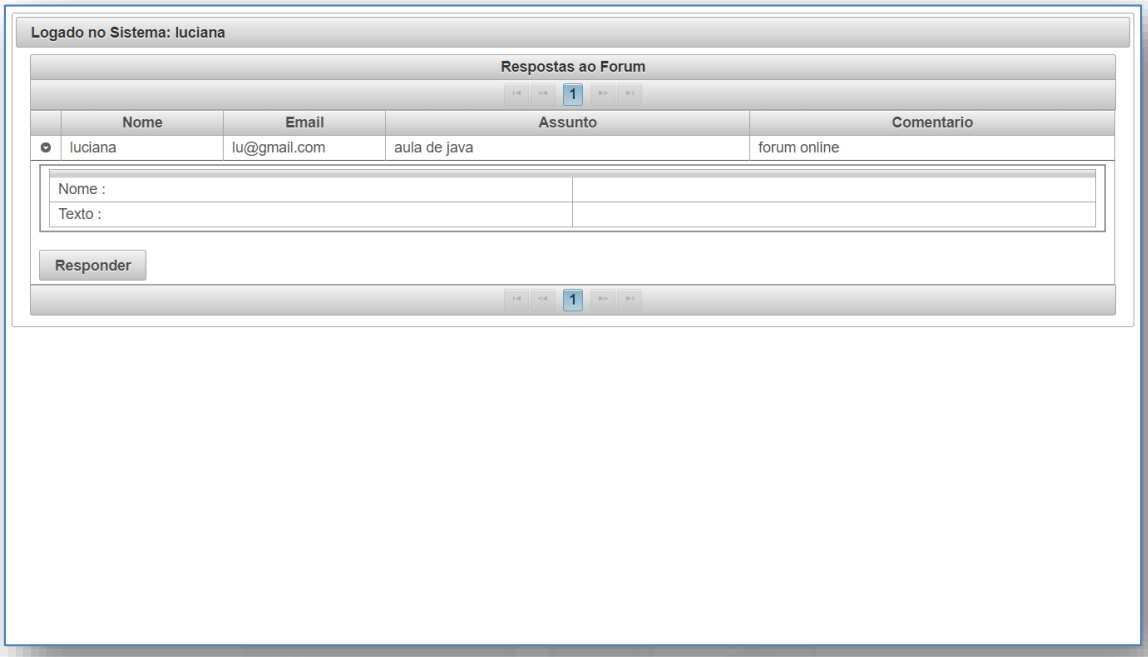
Logando...



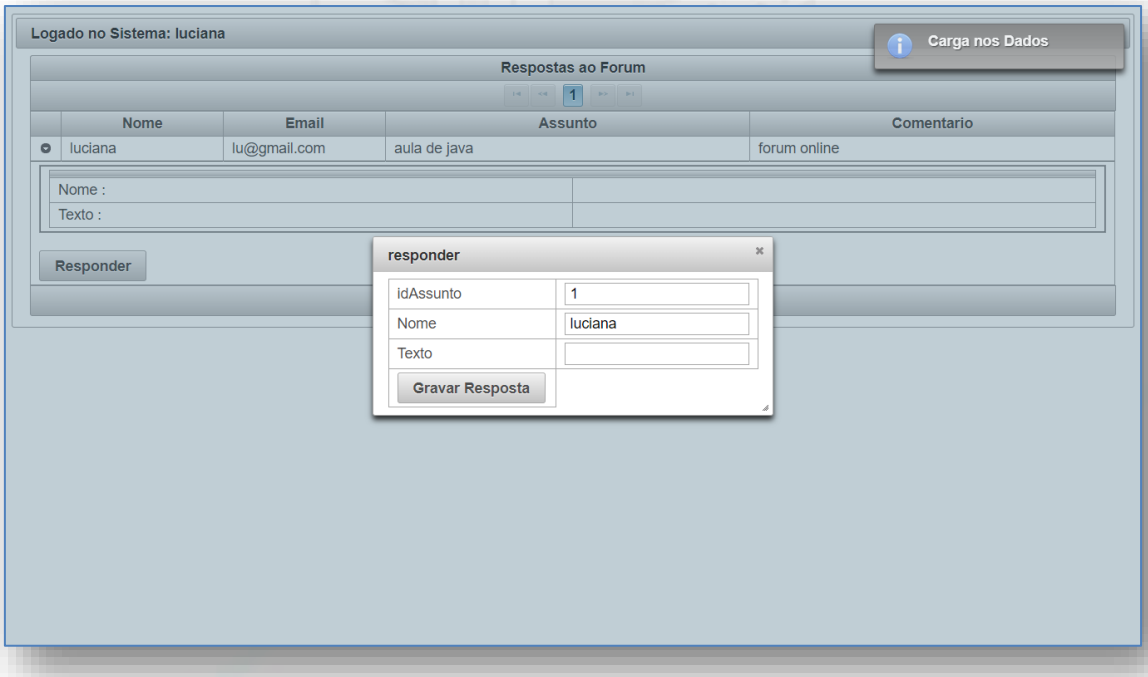
Logado...



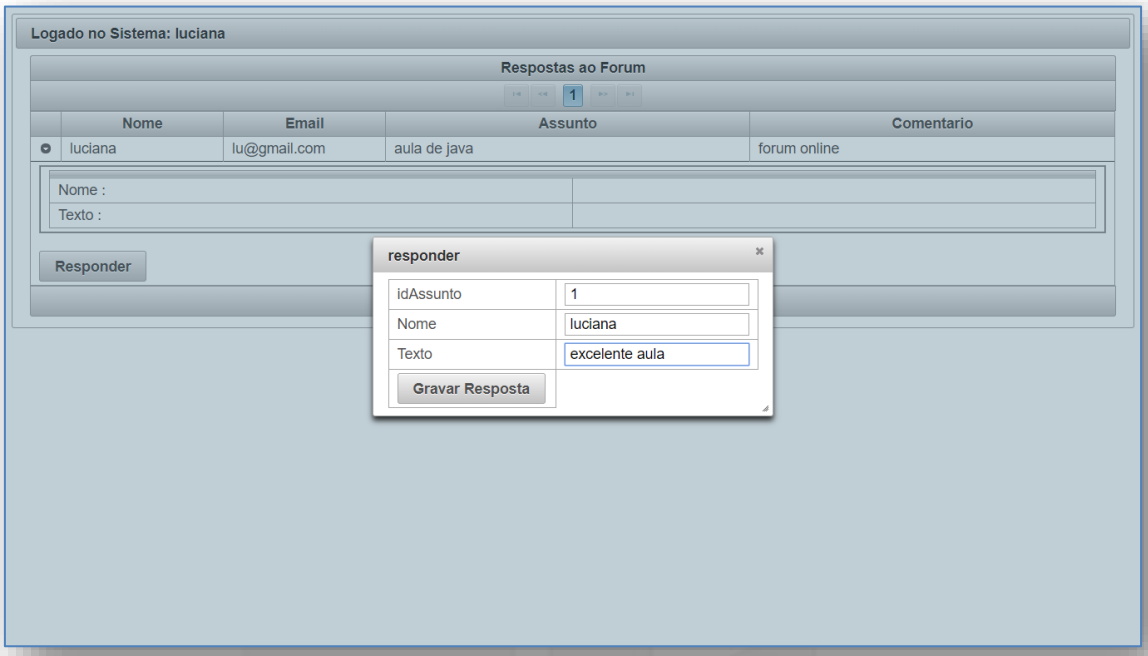
Expandindo o assunto...



Clicando em responder...



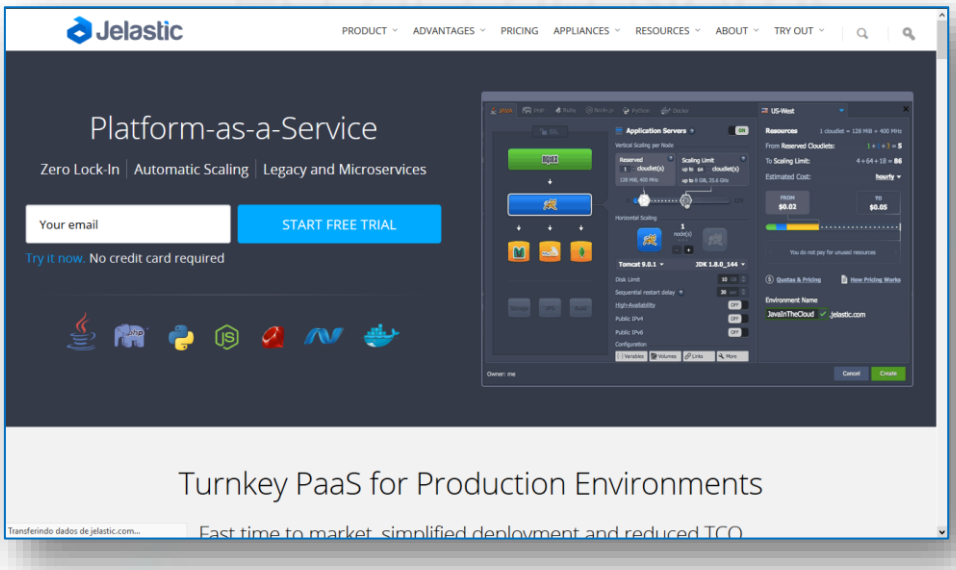
Preenchendo...



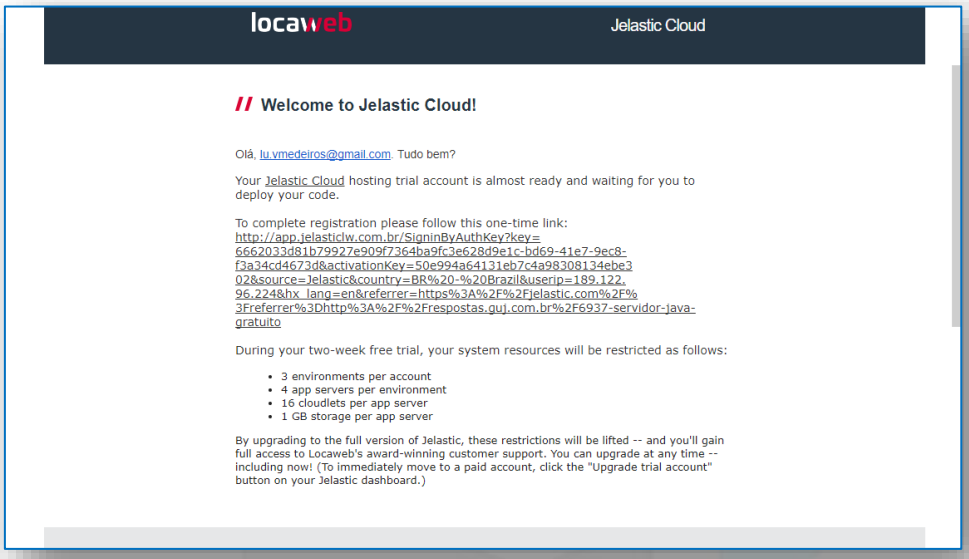
COLOCANDO O PROJETO NO SERVIDOR...

http://jelastic.com/pt/

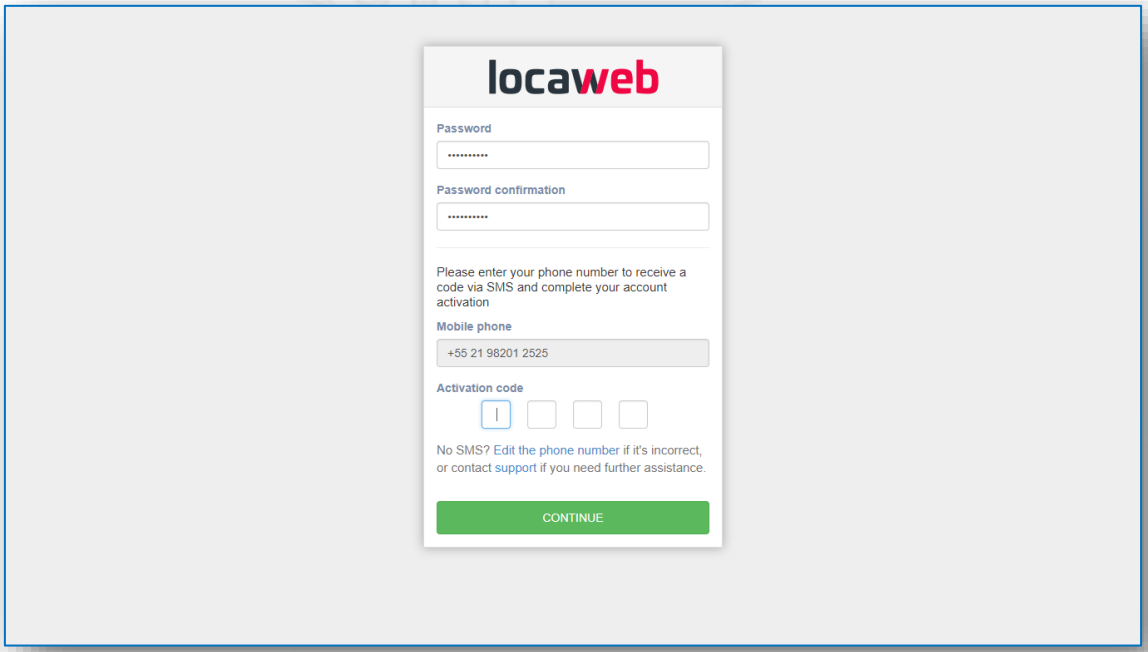
Fazer o cadastro e verificar o email...



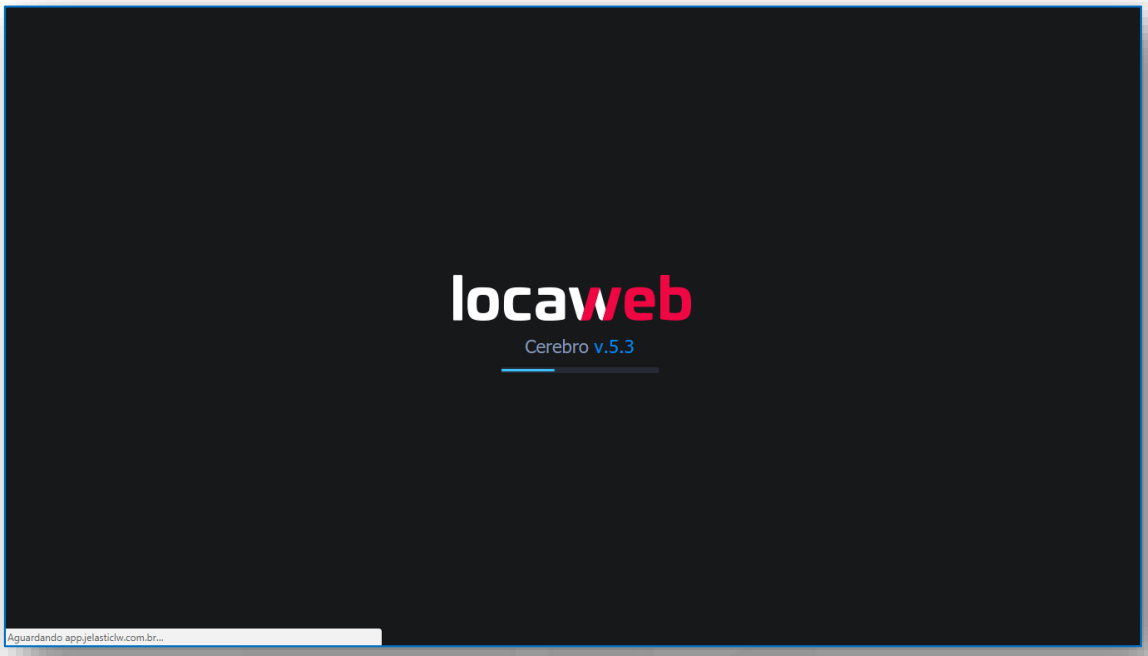
No email, completer o registro...



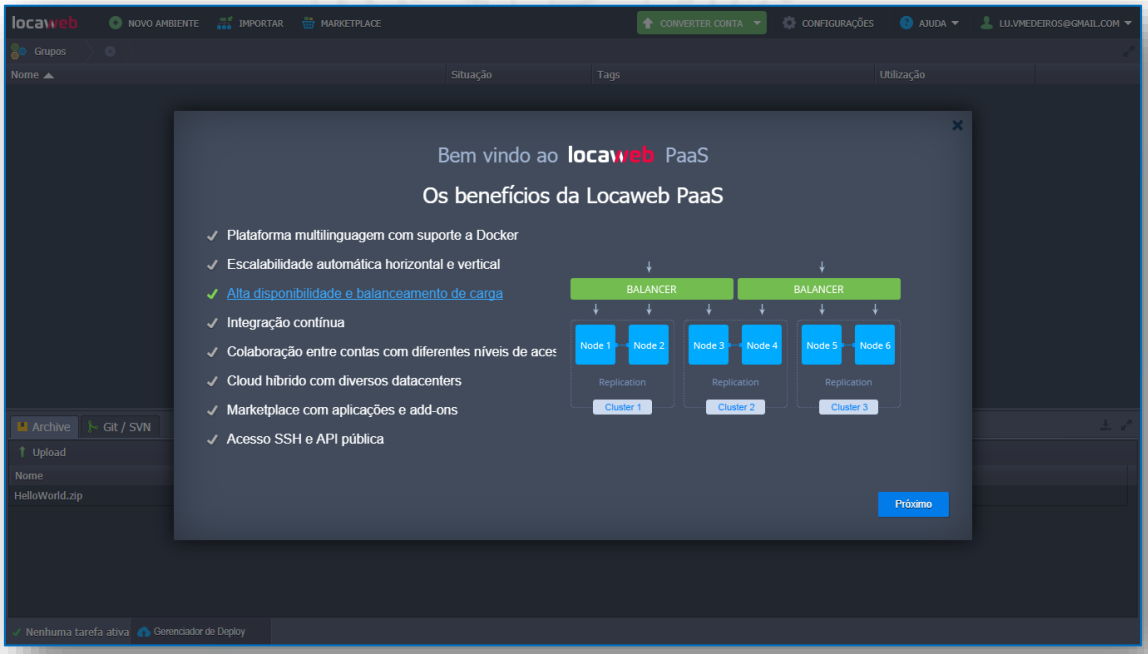
Preencher com os dados... Atenção no celular com os espaços, senão dá erro...



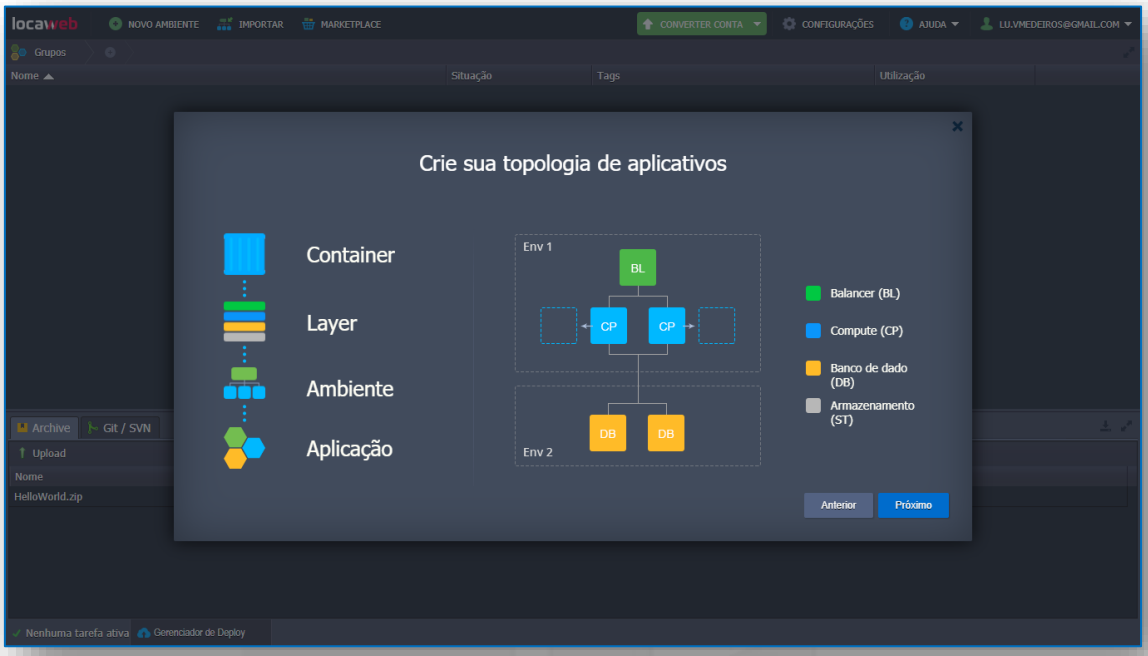
Preencher os números enviados por sms... Aguardar...



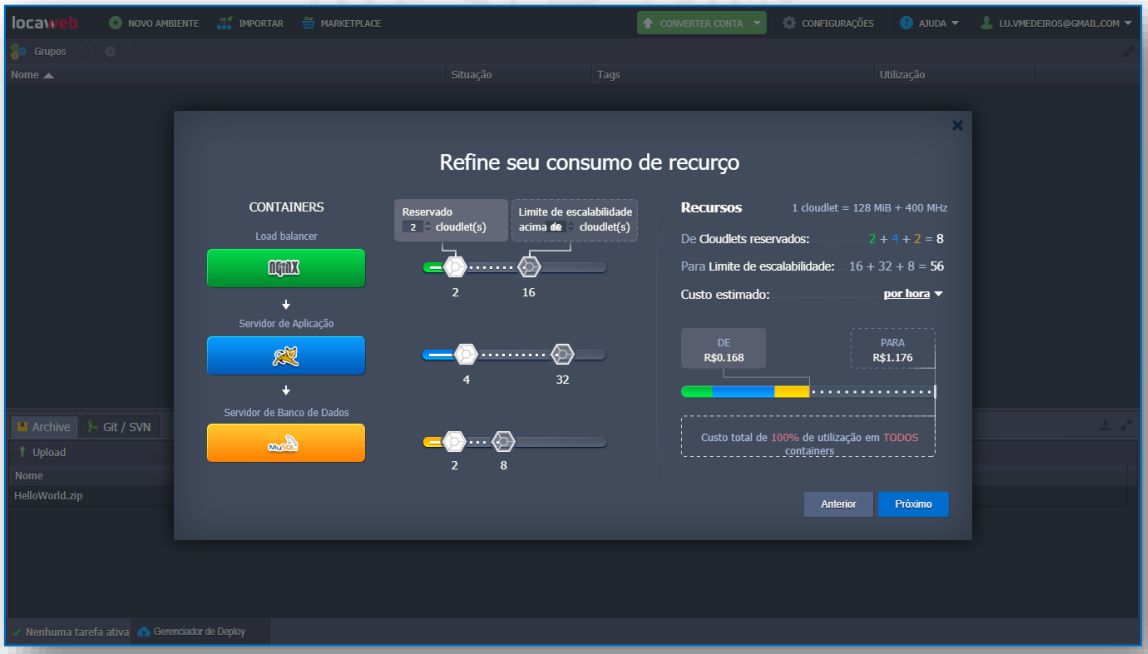
Proximo...



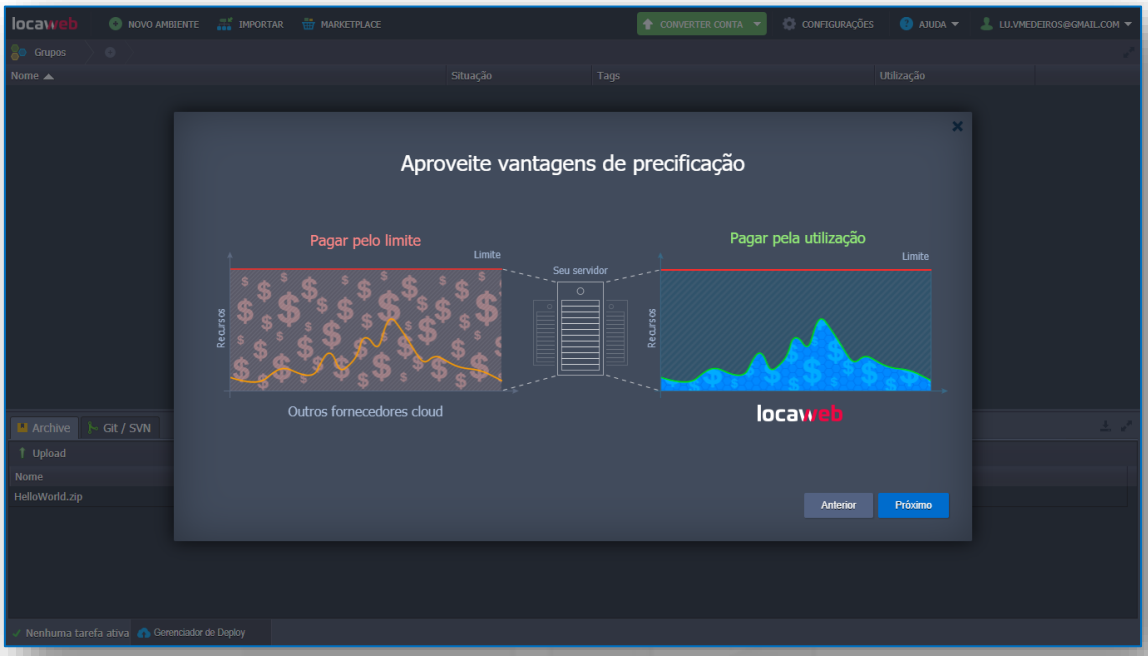
Proximo...



Proximo...

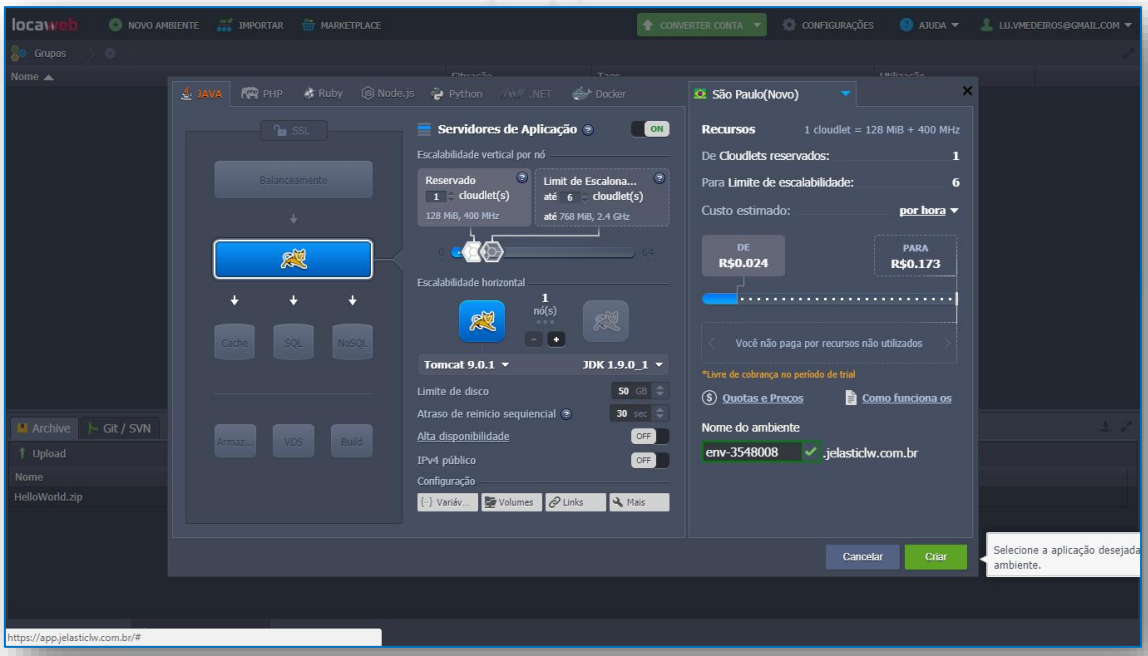


Proximo...

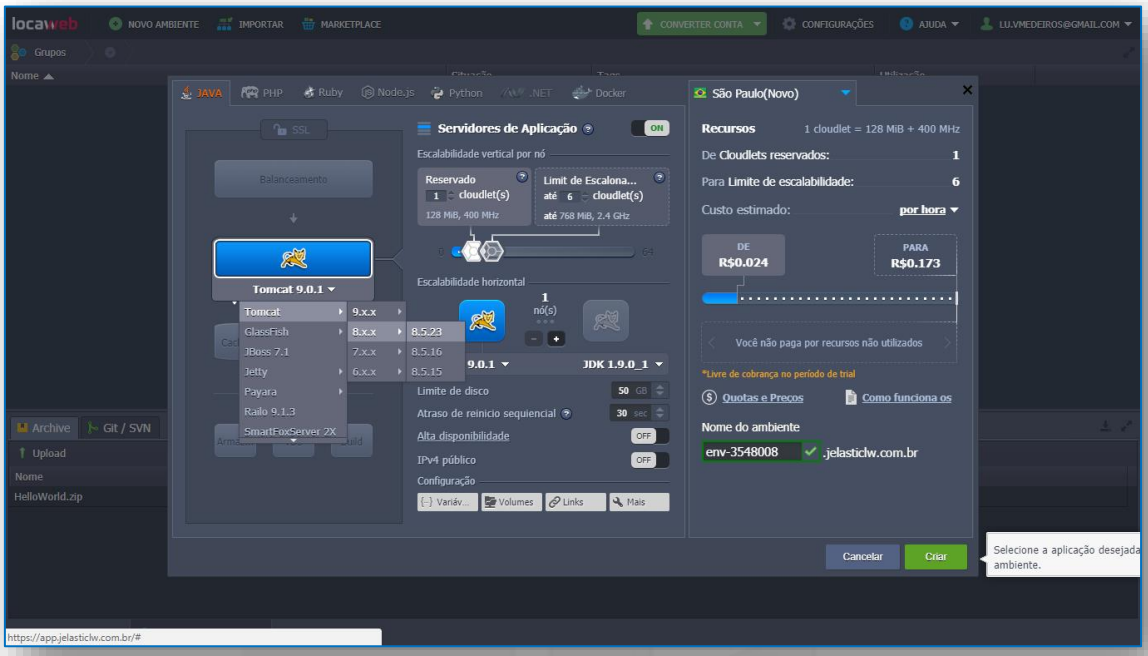


Para criar o ambiente onde hospedará o projeto...

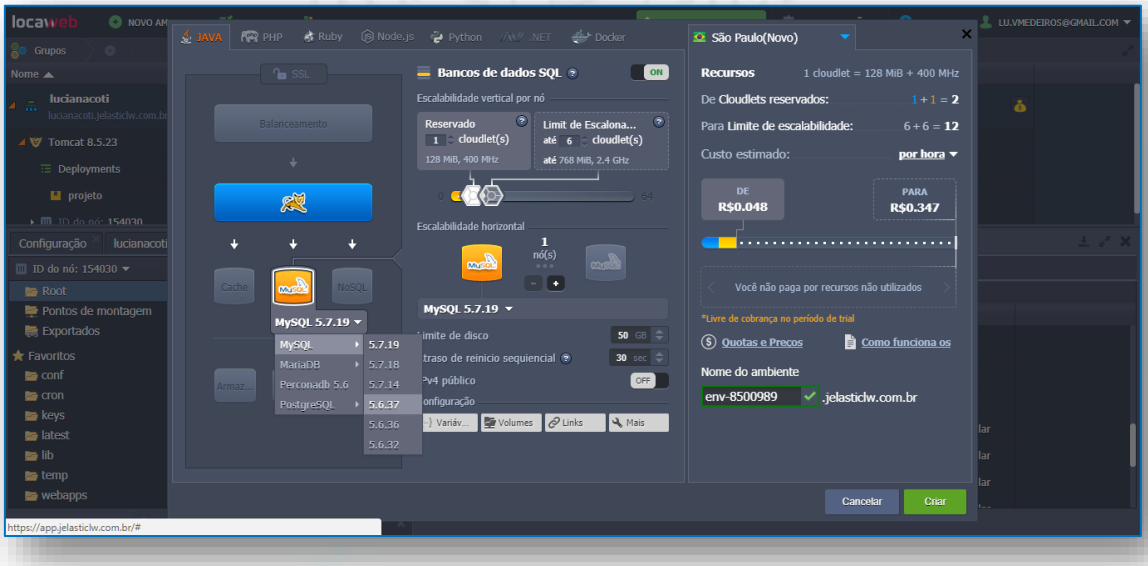
Selecionar a aba de Java...



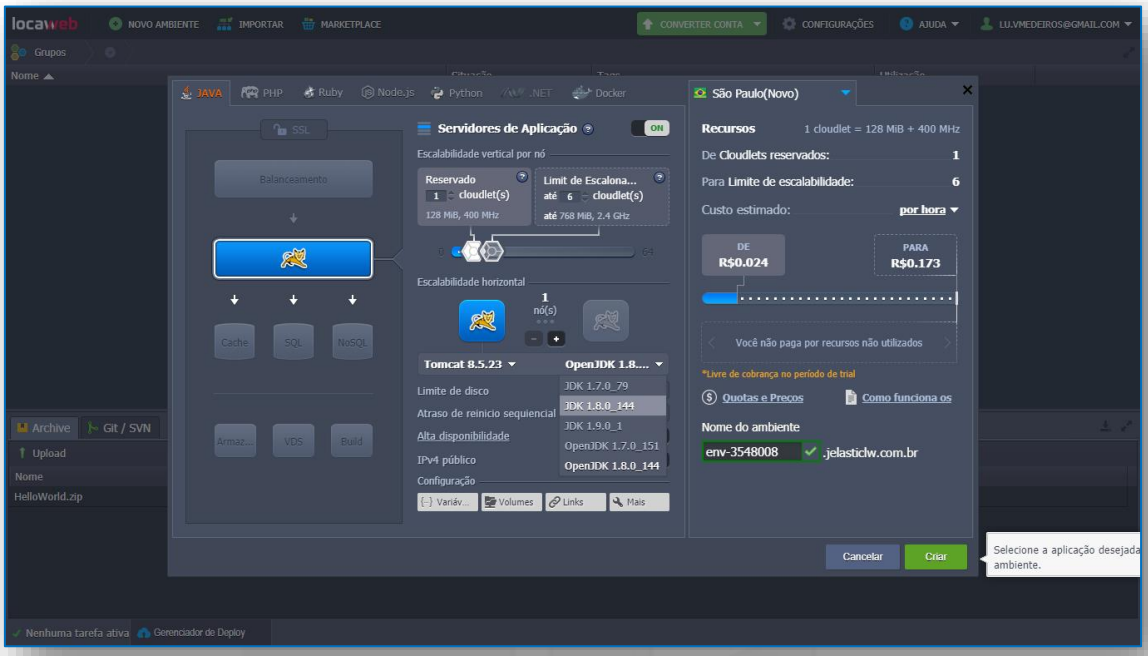
Selecionar a versão do tomcat para 8.xx...



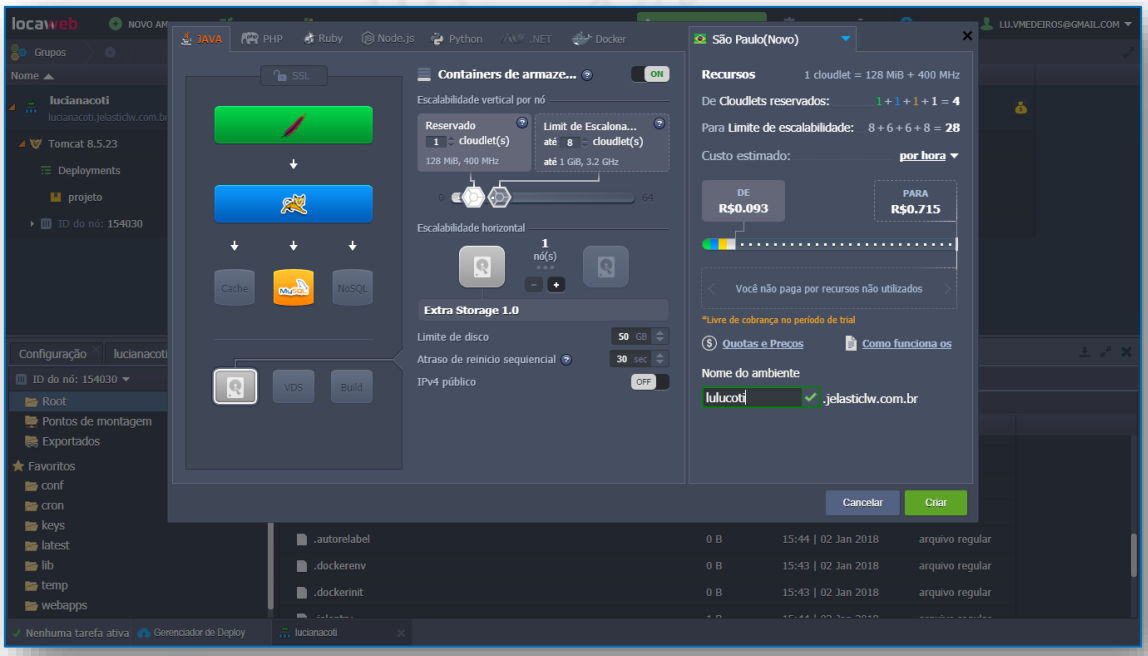
Selecionar o banco mysql e a versão...



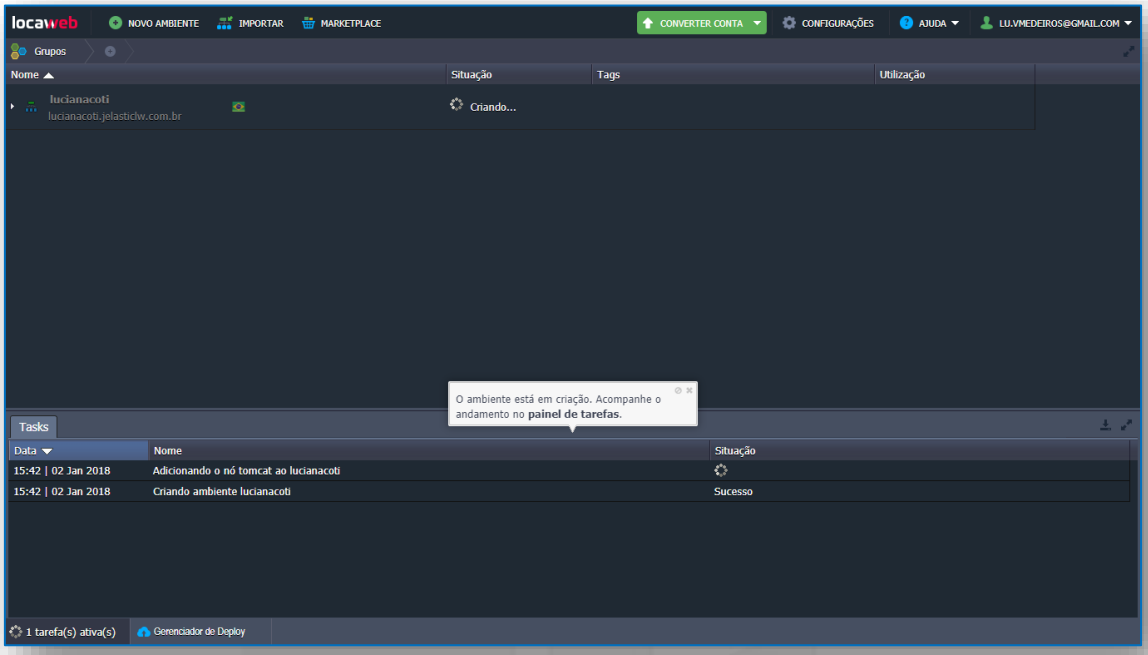
Selecionar a jdk...



Mudar ou não o nome do ambiente para ser o nome do site...
Exemplo: lulucoti.jelasticlw.com.br

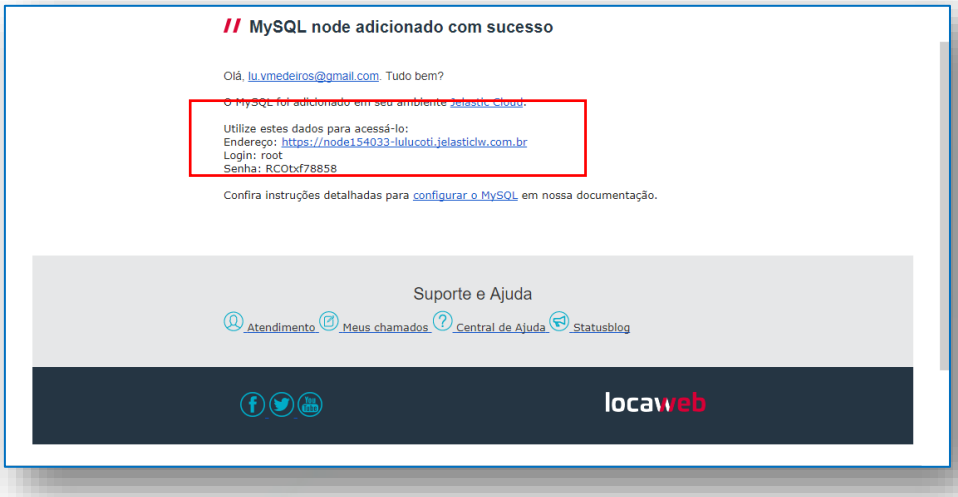


Aguardar a criação do ambiente...

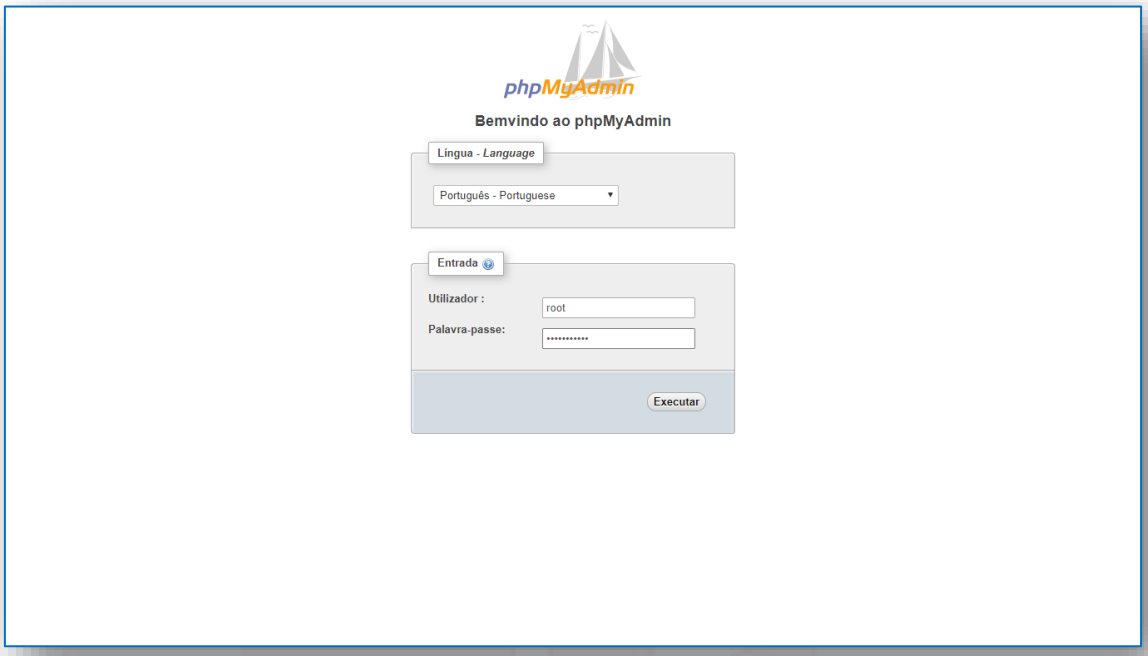


Para configurar o mysql...

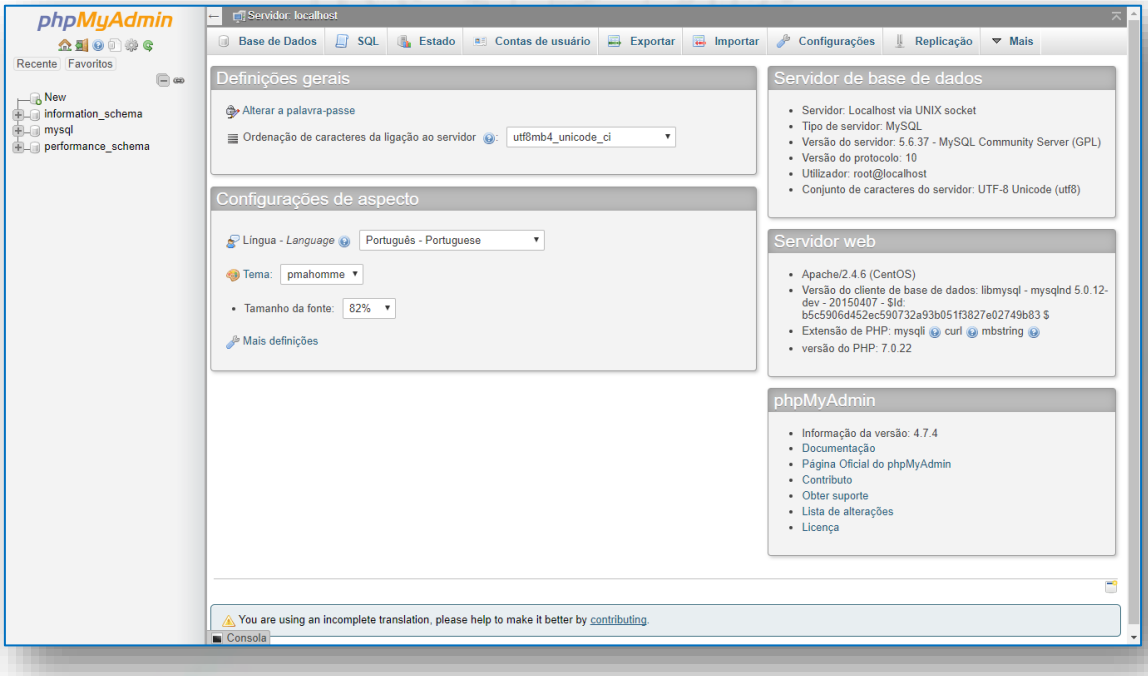
Conferir o email com as configurações do mysql...



Abrindo o link e digitando as informações contidas no email...

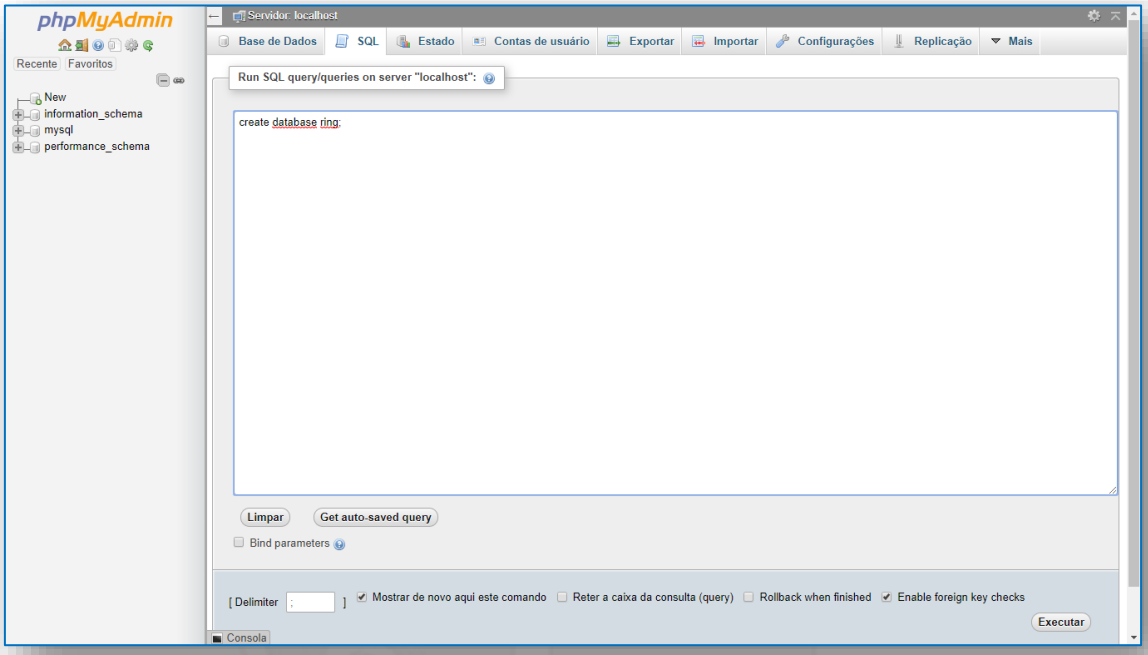


Abriu o admin...

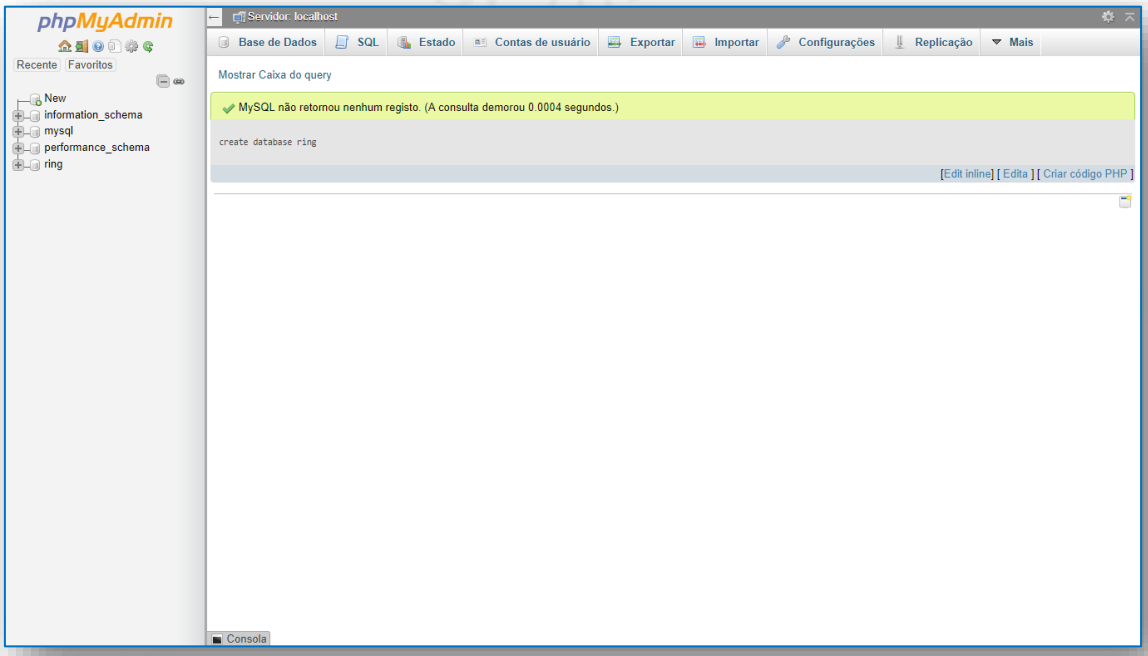


Para Criar o banco...

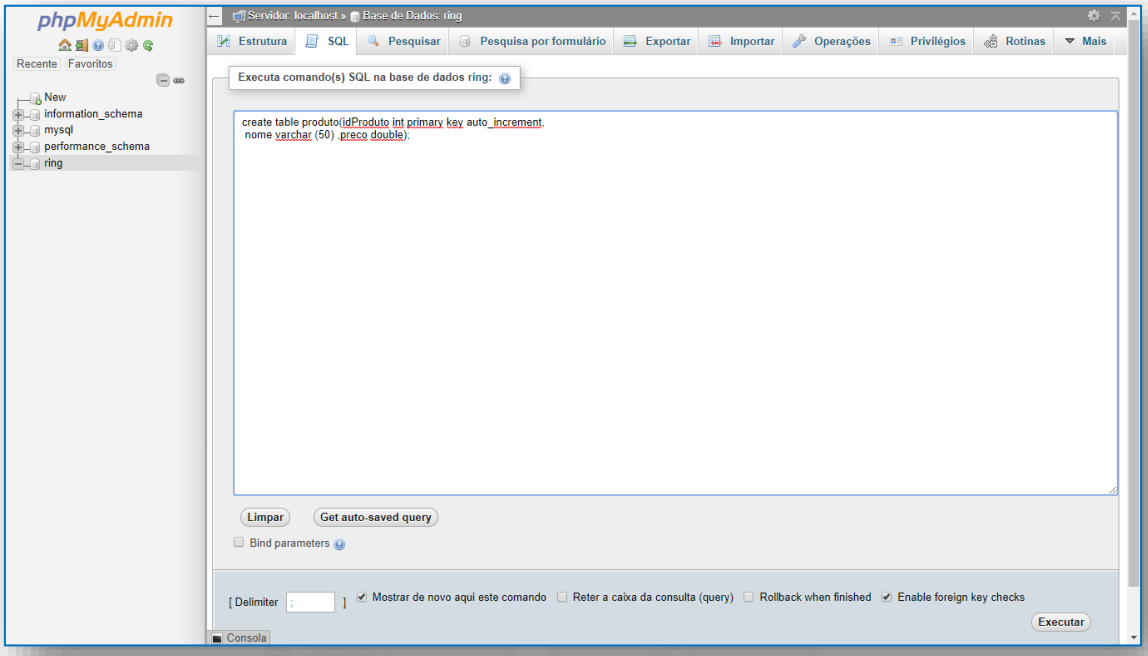
Clicar na aba sql, colar o script e clicar em executar...



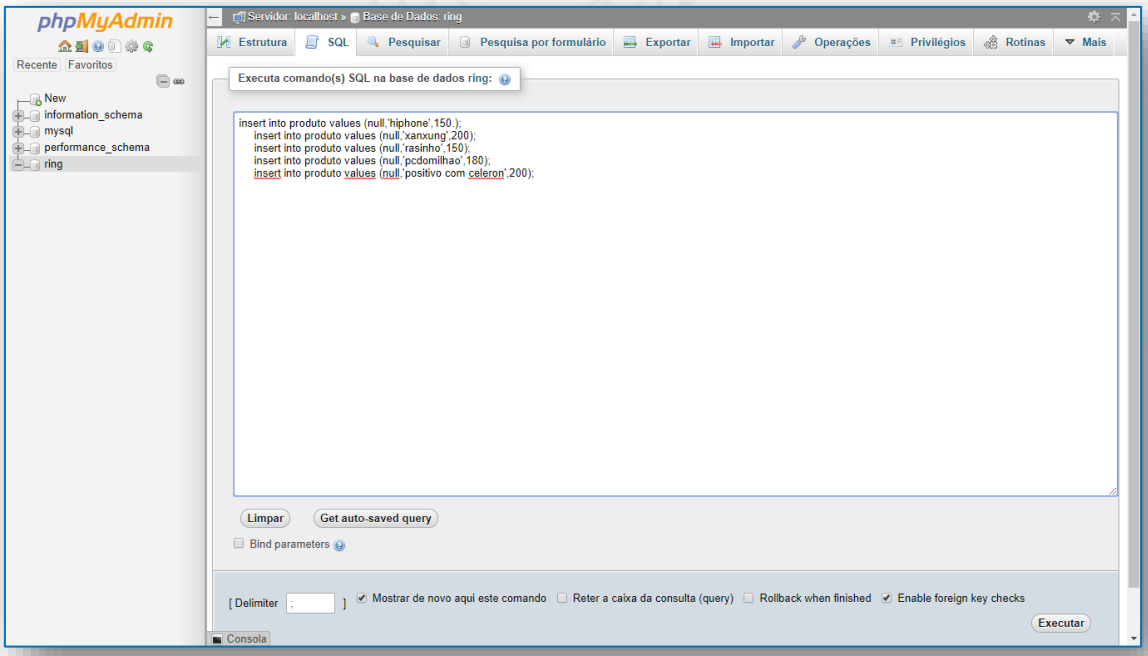
Banco criado...



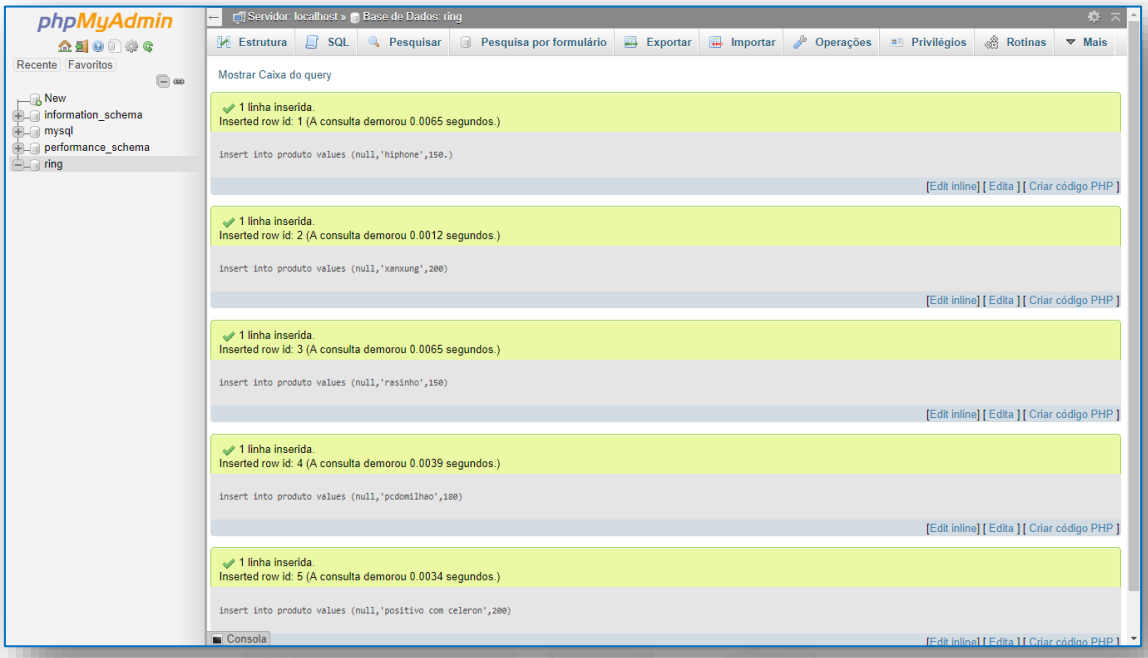
Clicar no banco a direita, clicar na aba “sql” e adicionar o restante do sql -> executar...



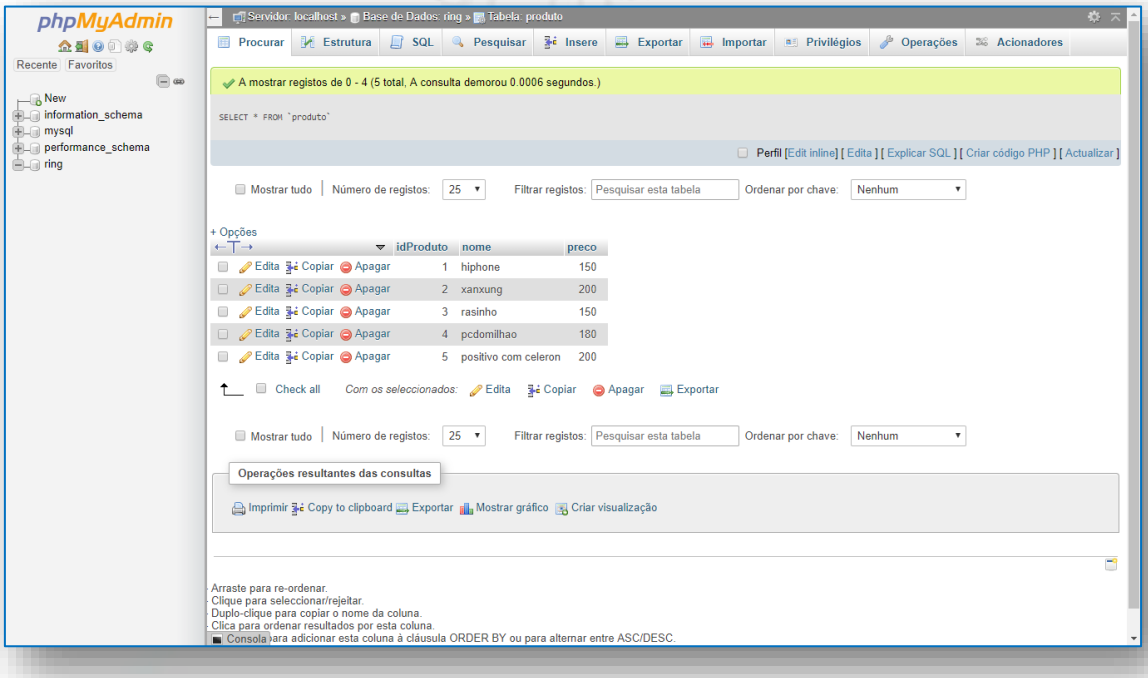
Inserindo o restante do código...



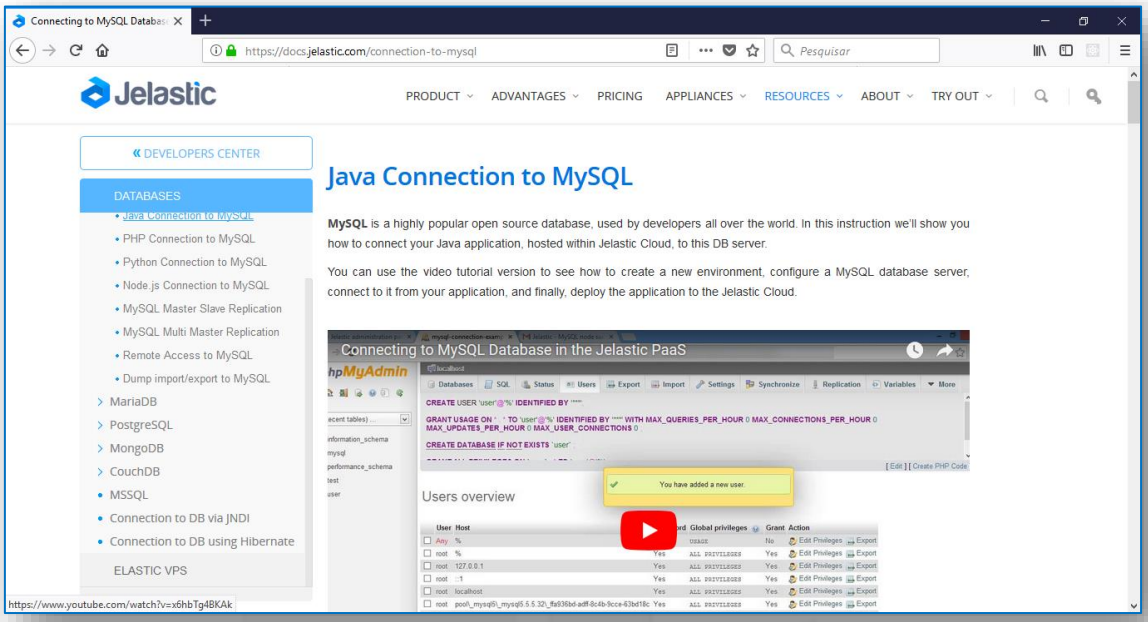
Linhas inseridas...



Produtos inseridos na tabela...



Mais configurações https://docs.jelastic.com/connection-to-mysql



Alterar a classe mysql_hibernate.cfg.xml com as informações...

host=jdbc:mysql://node154033-lulucoti.jelasticlw.com.br:3306/ring
username=root
password=RCOtxf78858
driver=com.mysql.jdbc.Driver

---

mysql_hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</pro
perty>
```

```
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</
property>
```

Utilizar o caminho da conexão passado por email...

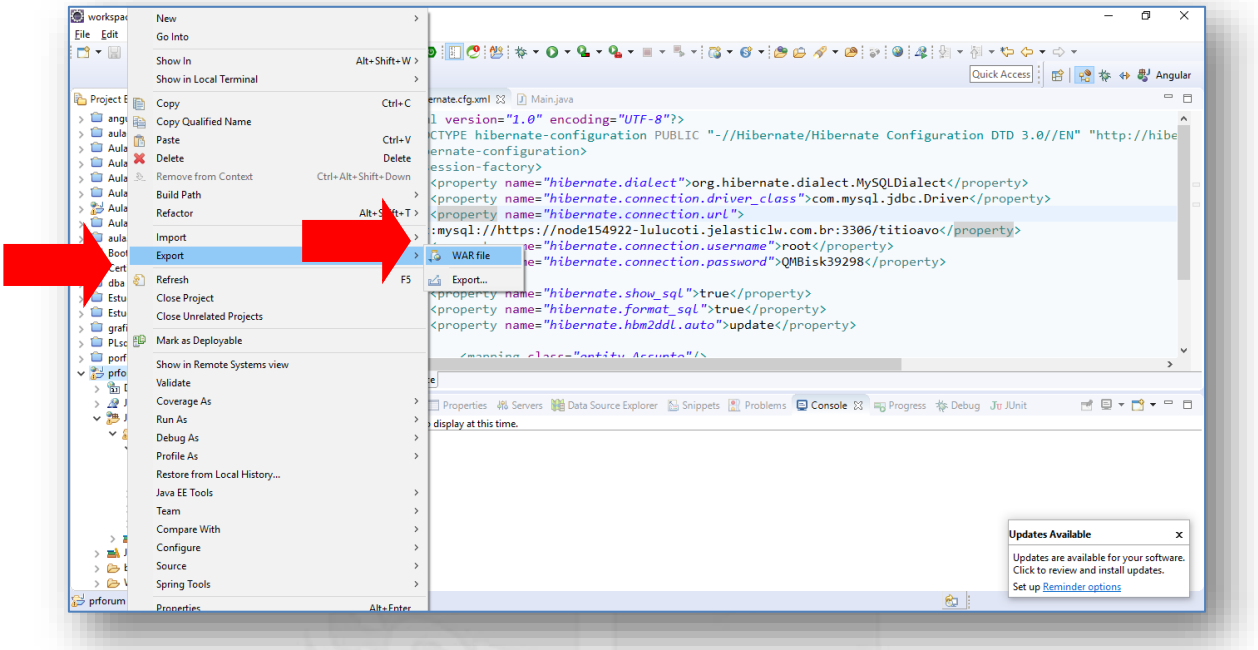
```
    <property name="hibernate.connection.url">
jdbc:mysql://https://node154922-
lulucoti.jelasticlw.com.br:3306/titioavo</property>
    <property
name="hibernate.connection.username">root</property>
```

Utilizar a senha passado por email...

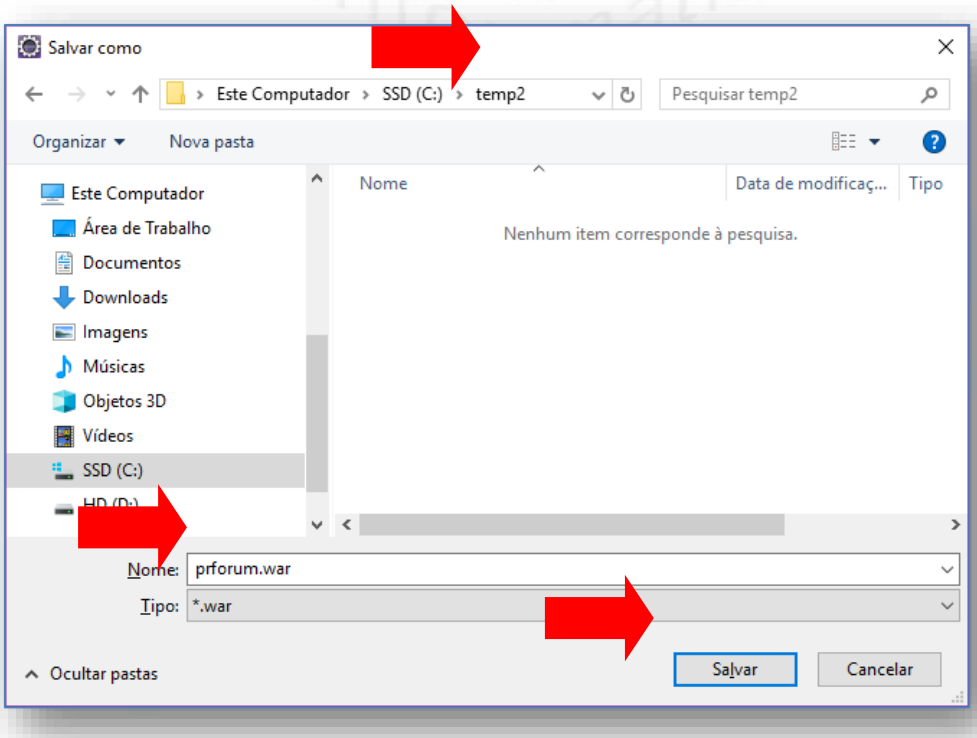
```
    <property
name="hibernate.connection.password">QMBisk39298</property>

    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

        <mapping class="entity.Assunto"/>
        <mapping class="entity.Resposta"/>


 </session-factory>
</hibernate-configuration>
```

Exportar o projeto...

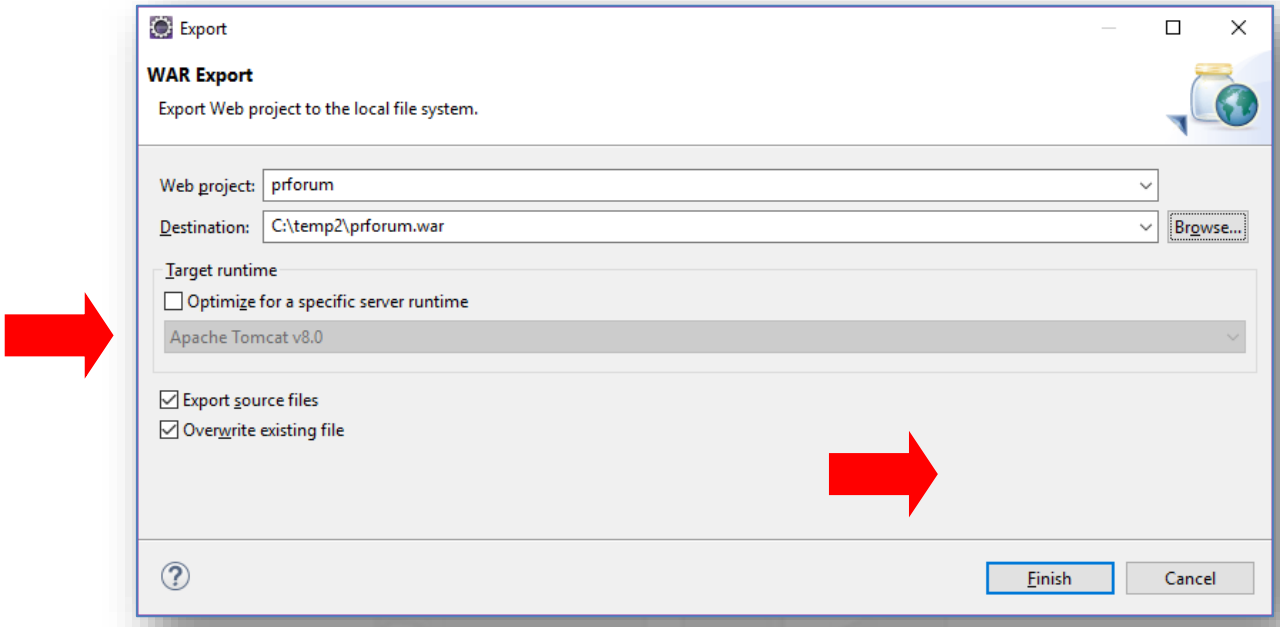
Clicar no projeto com o botão direito -> export -> war file...



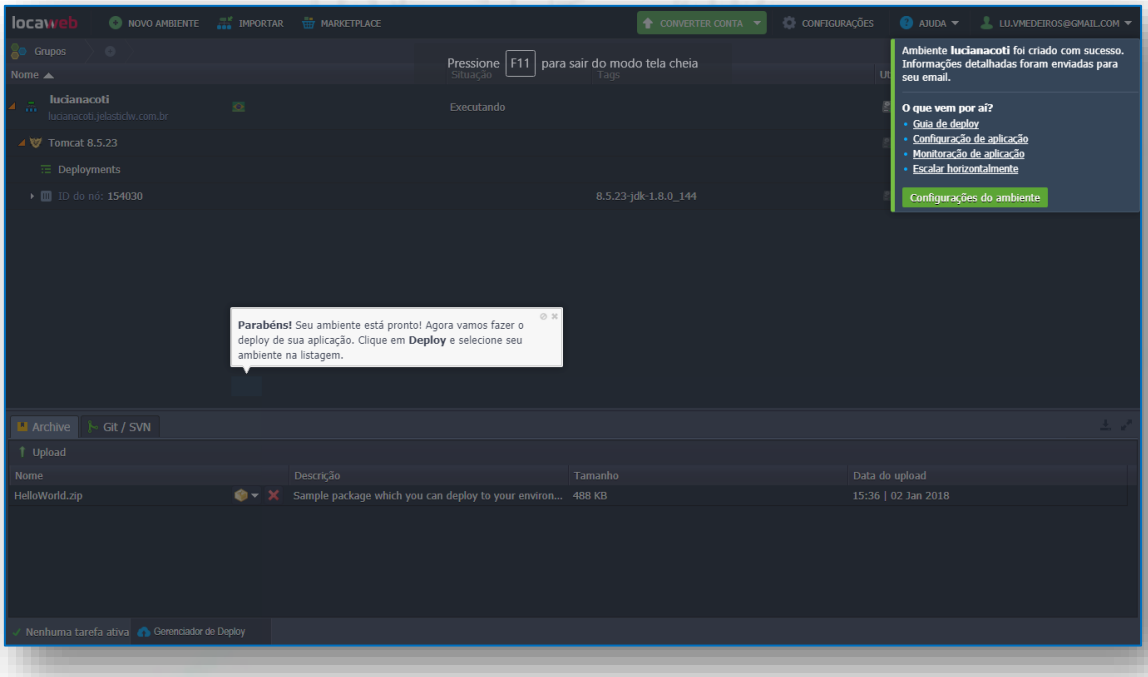
Indicar onde irá salvar o projeto...



Não marcar nenhuma das opções, para não enviar as classes para o servidor > Finish...

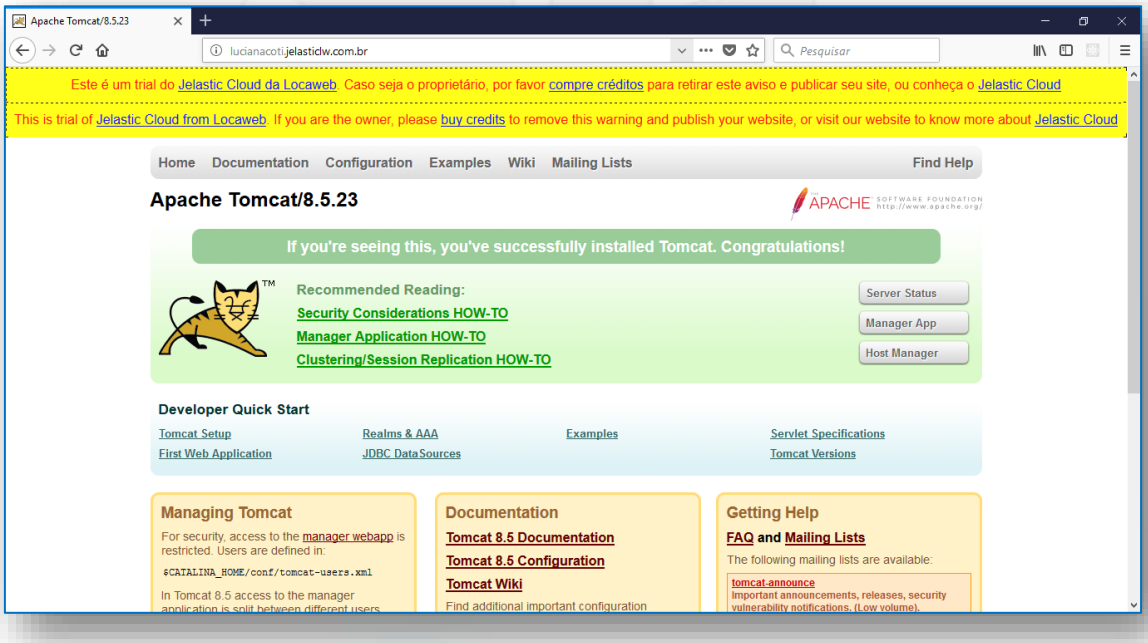


Ambiente pronto...

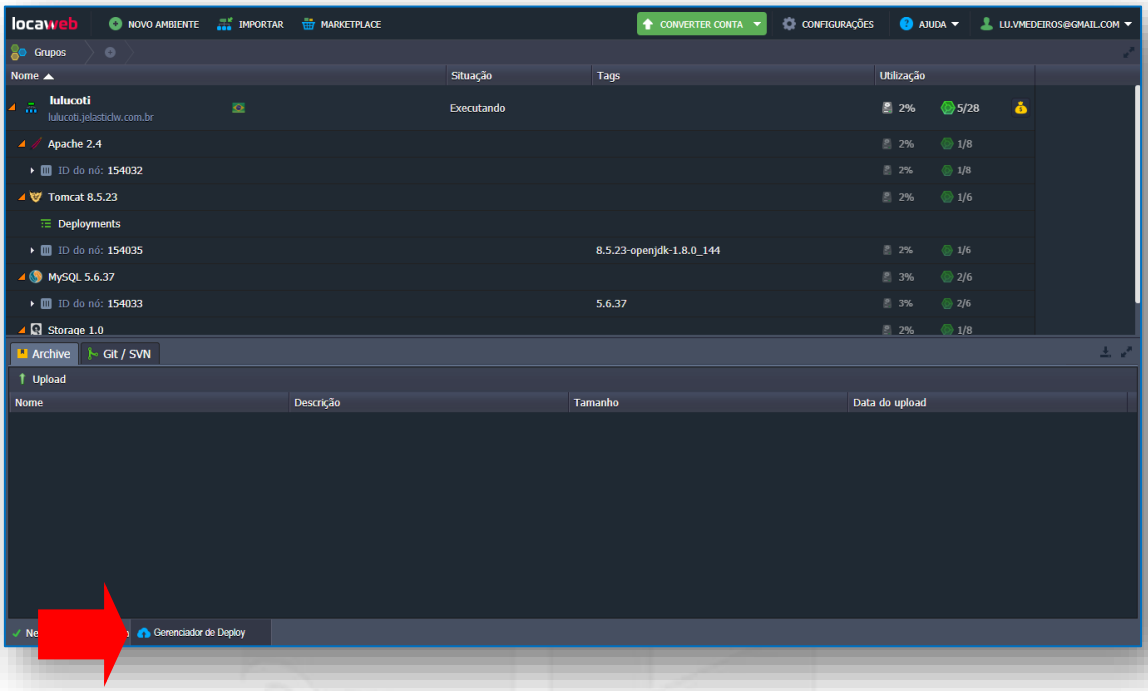


No email...

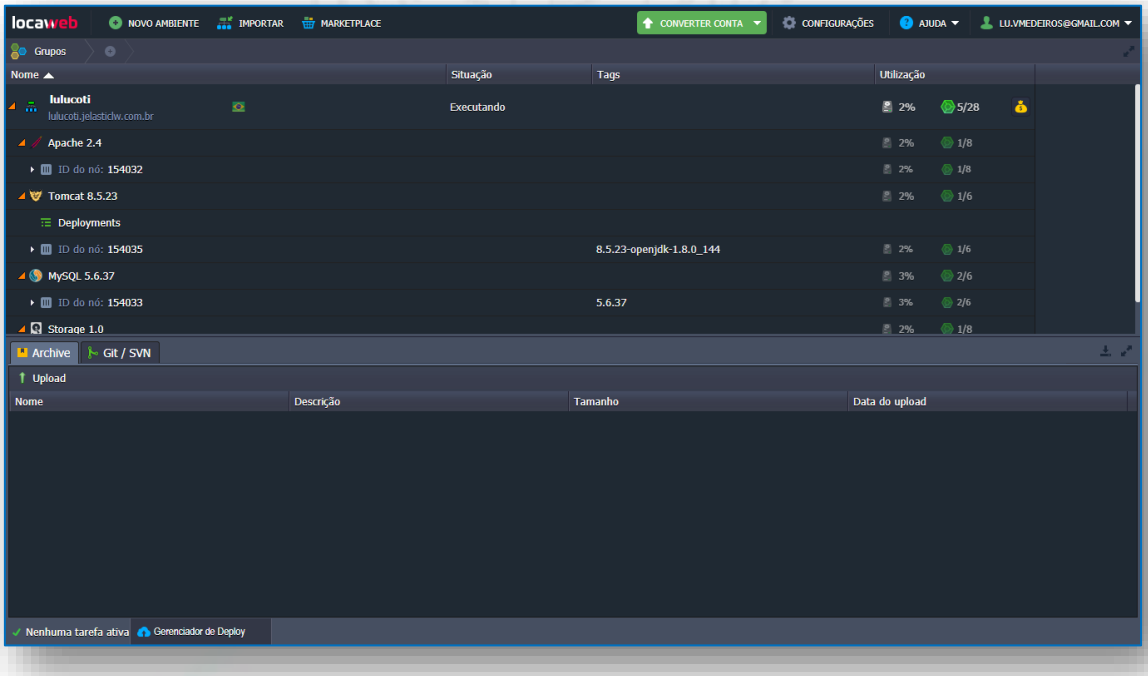
No site criado...



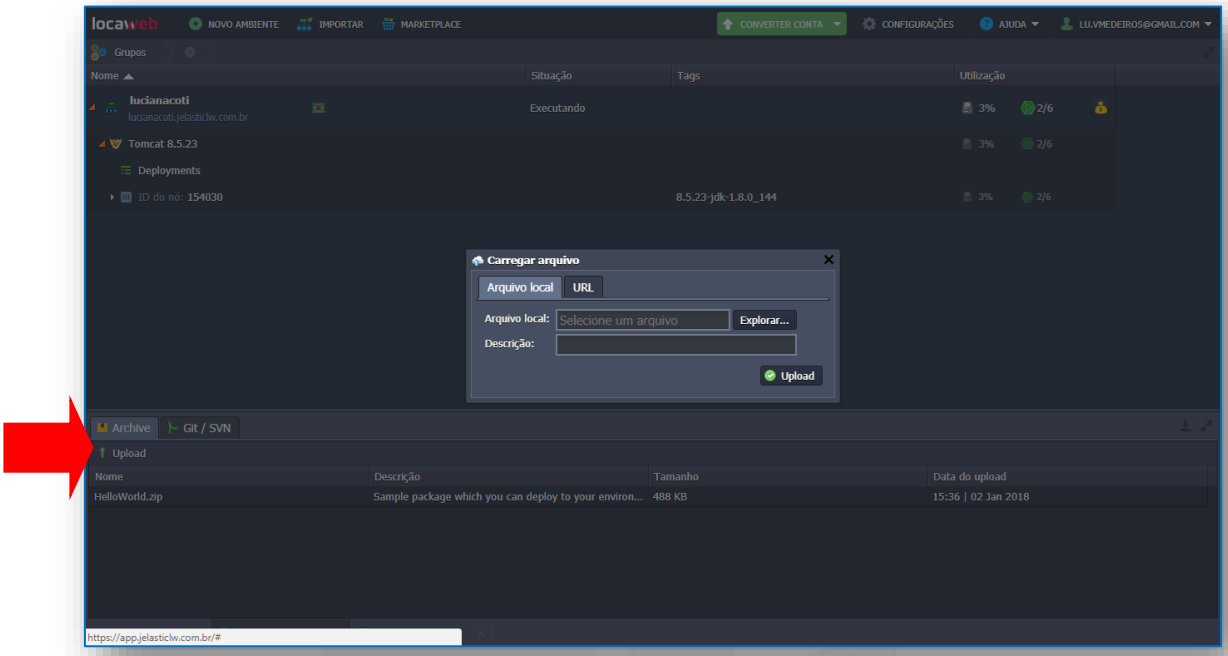
Clicar em gerenciador de deploy..



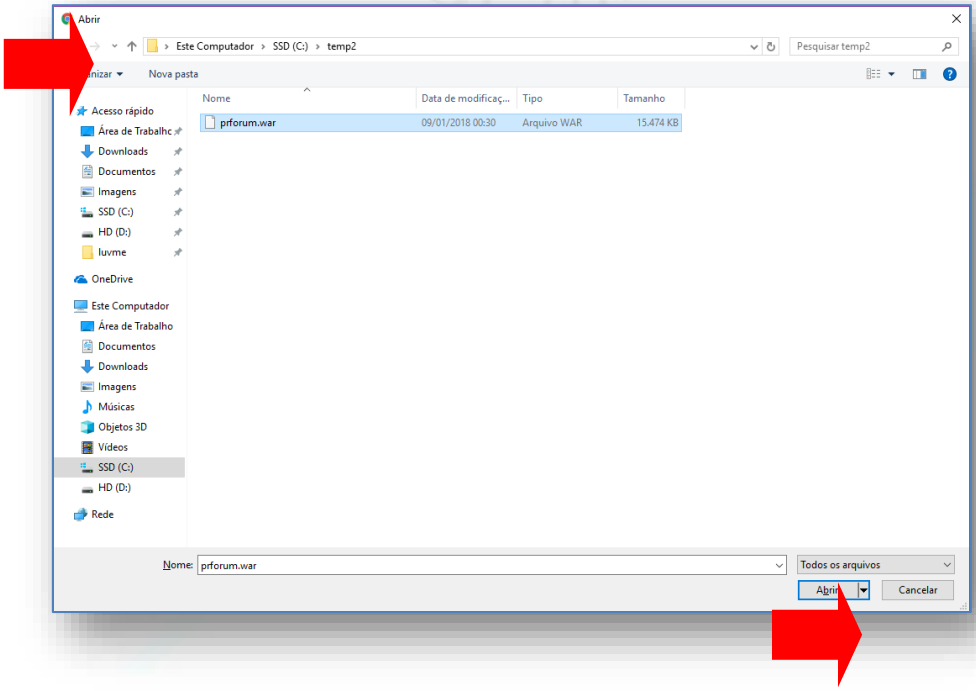
Abrirá uma aba embaixo...



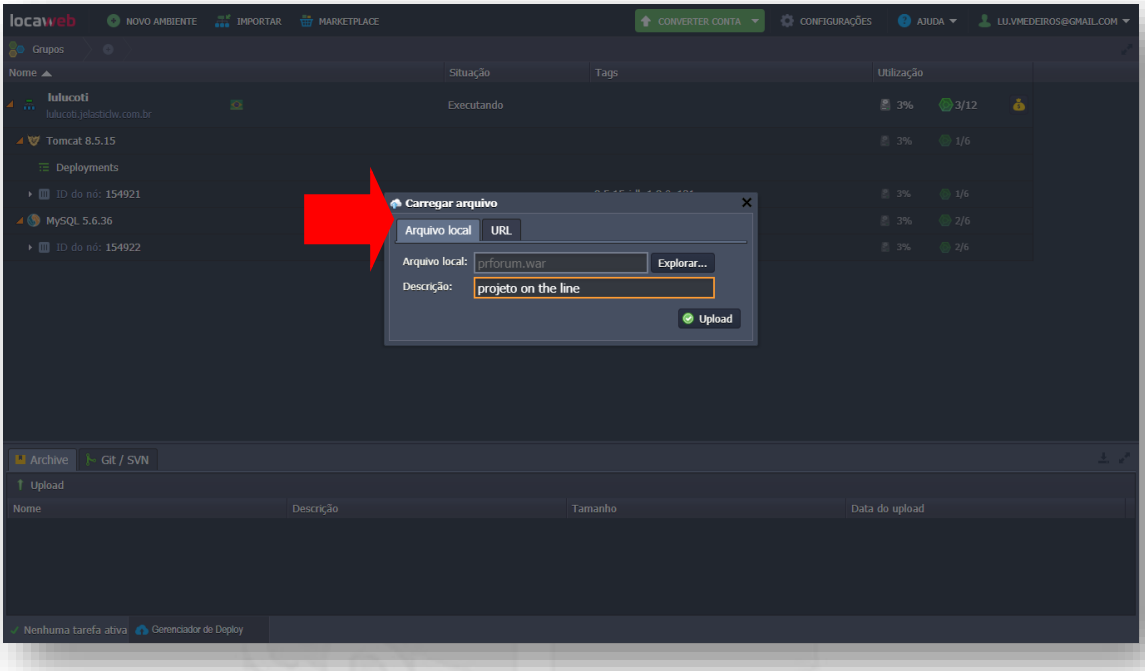
Clicar em upload...



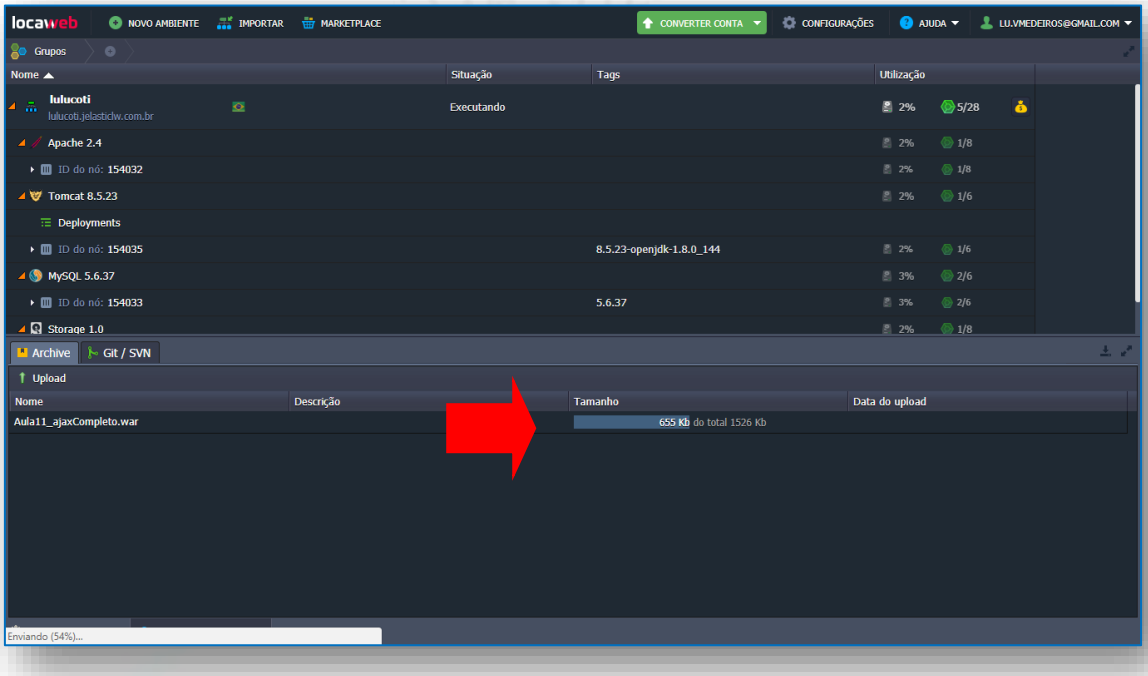
Selecionar um projeto war...



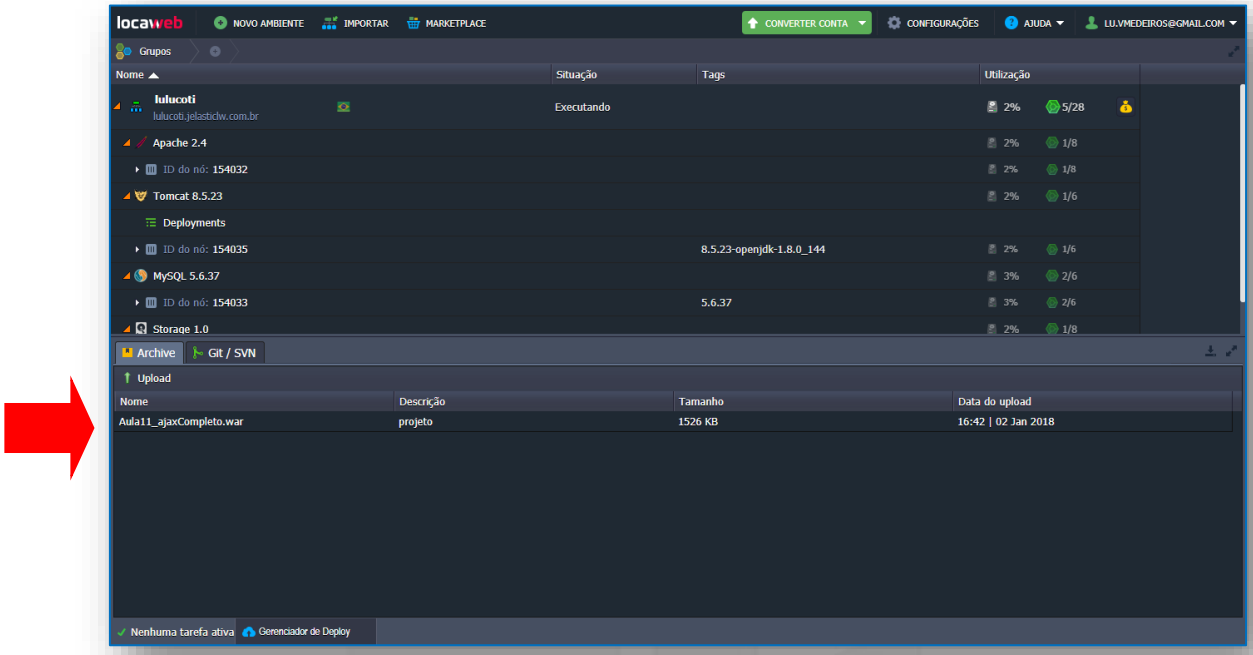
Digitar um nome em descrição que será a continuação do endereço do site e clicar em upload...



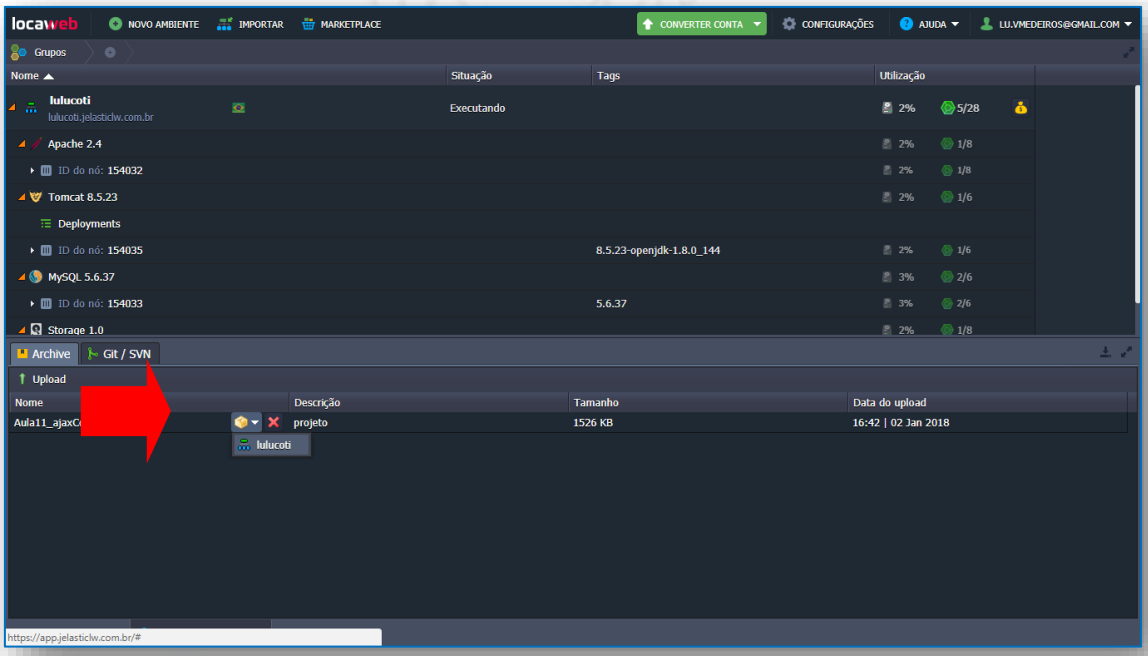
Aguardar carregar o projeto...



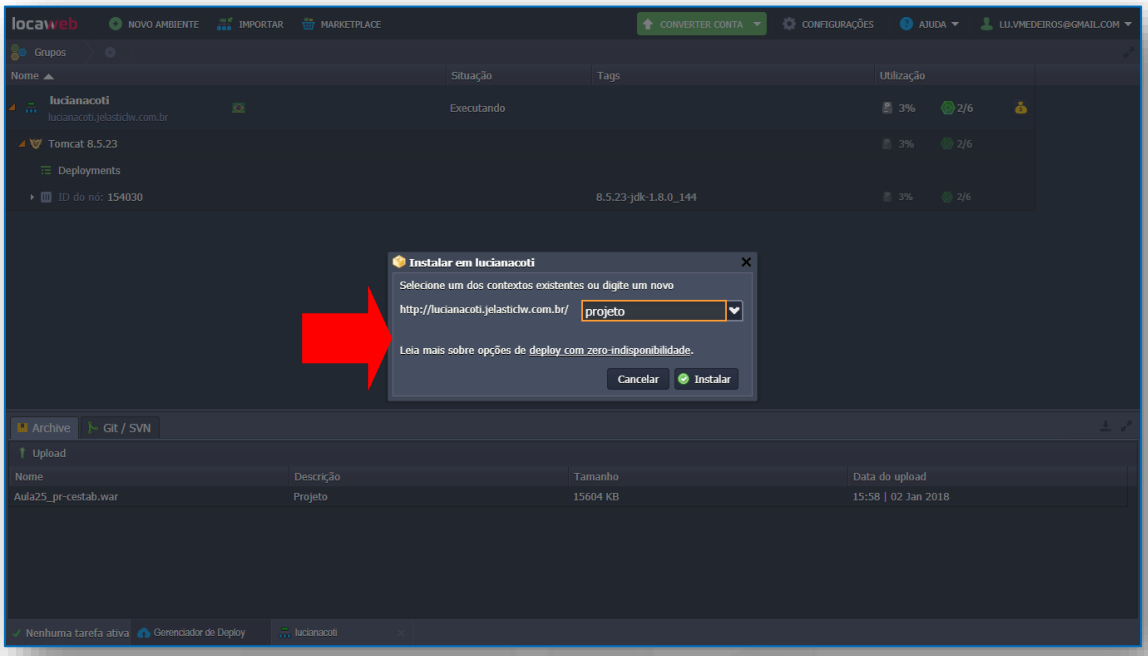
Feito o deploy...



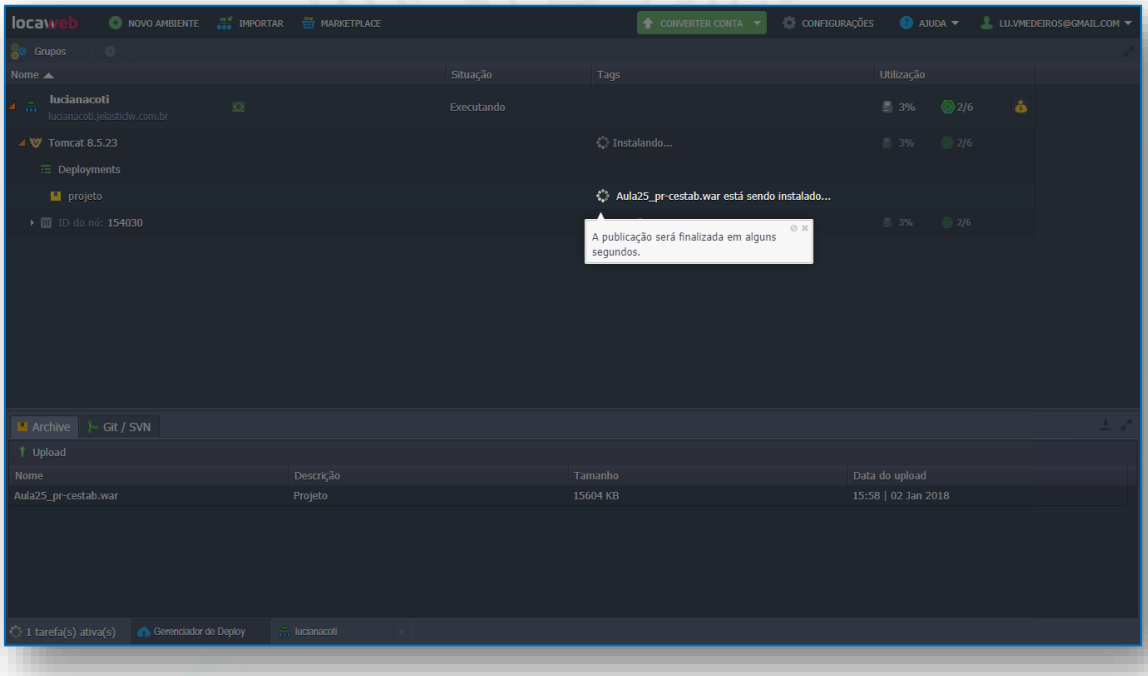
Apaguei o helloword.zip...
Cliquei na caixa do projeto...



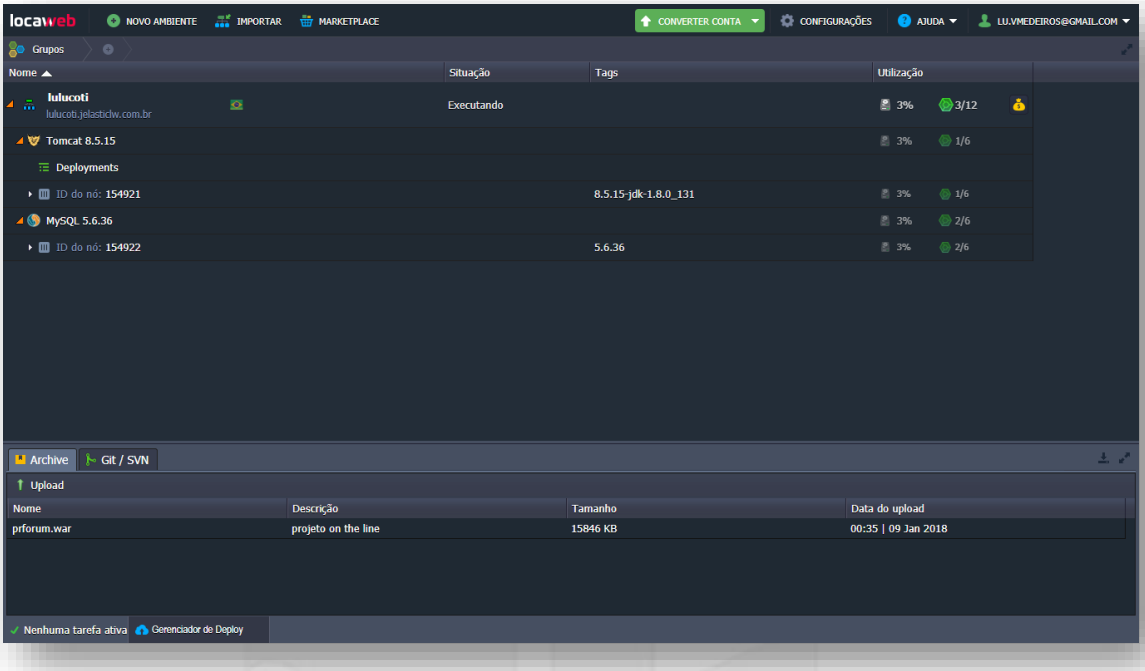
Digite um nome e clicar em instalar...



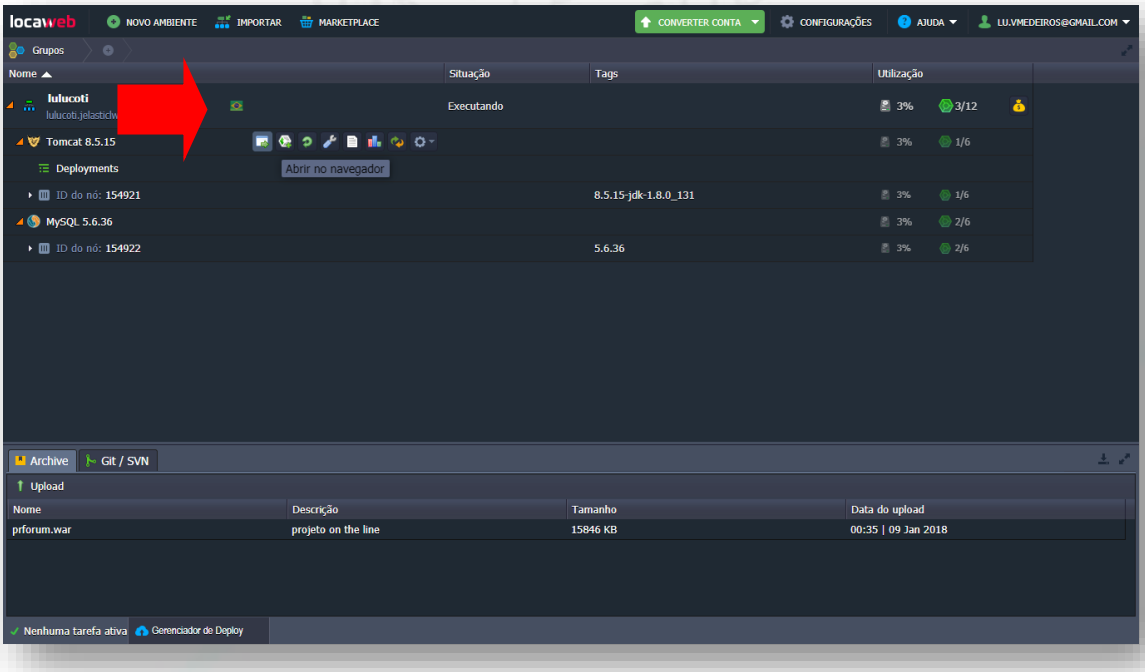
Aguardar a instalação..



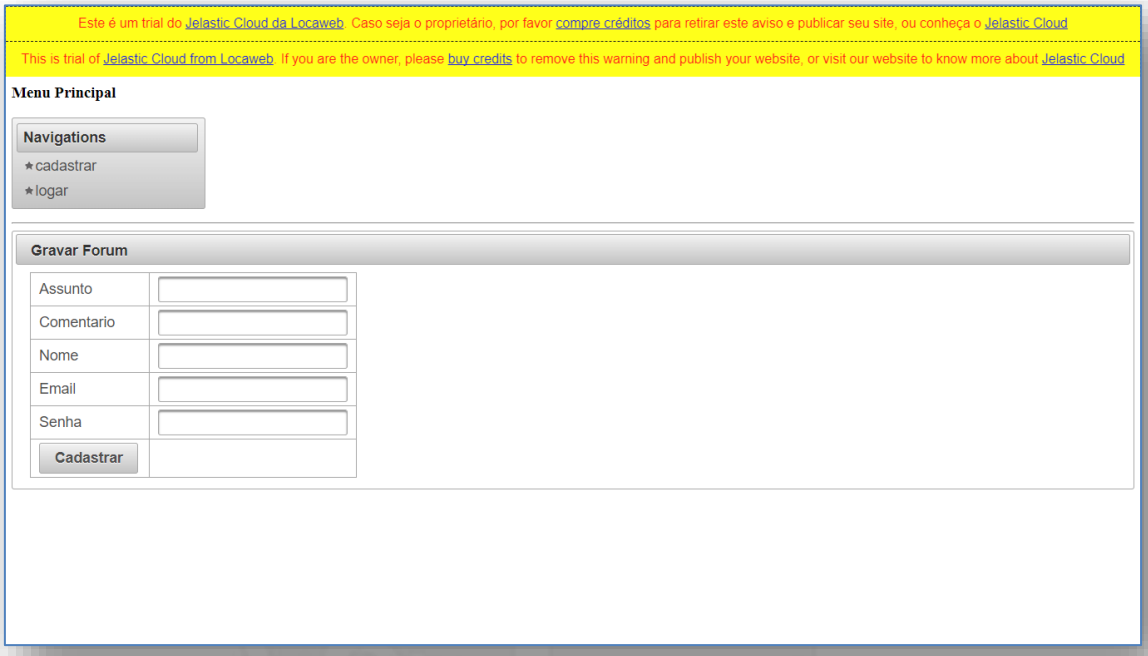
Instalado...



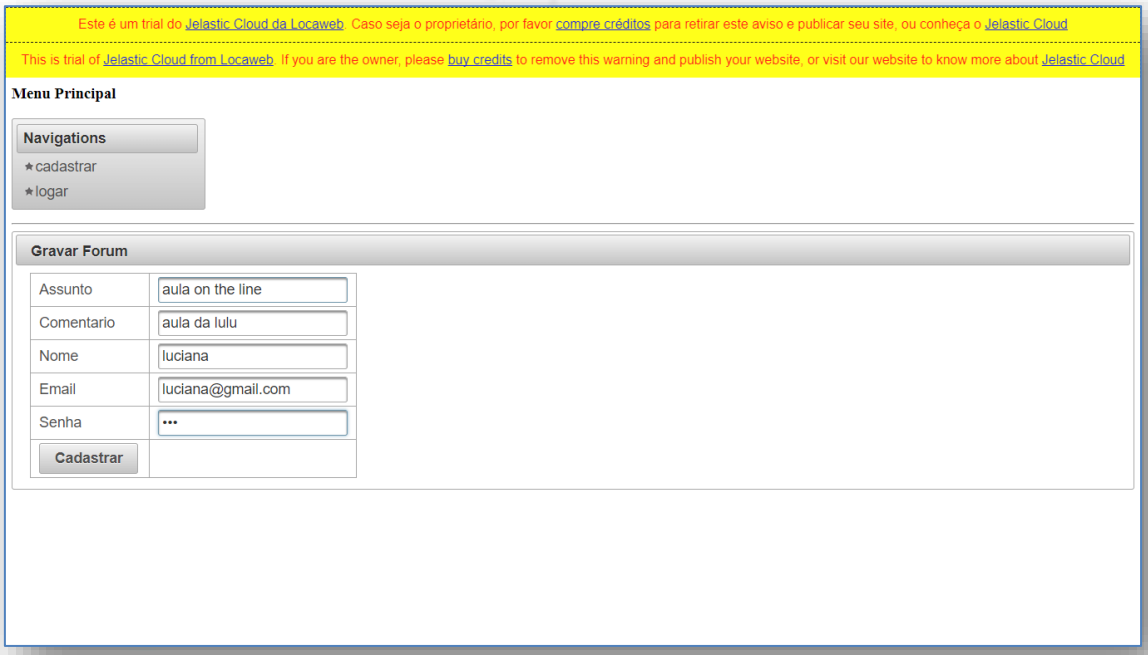
Clicar no icone do navegador..



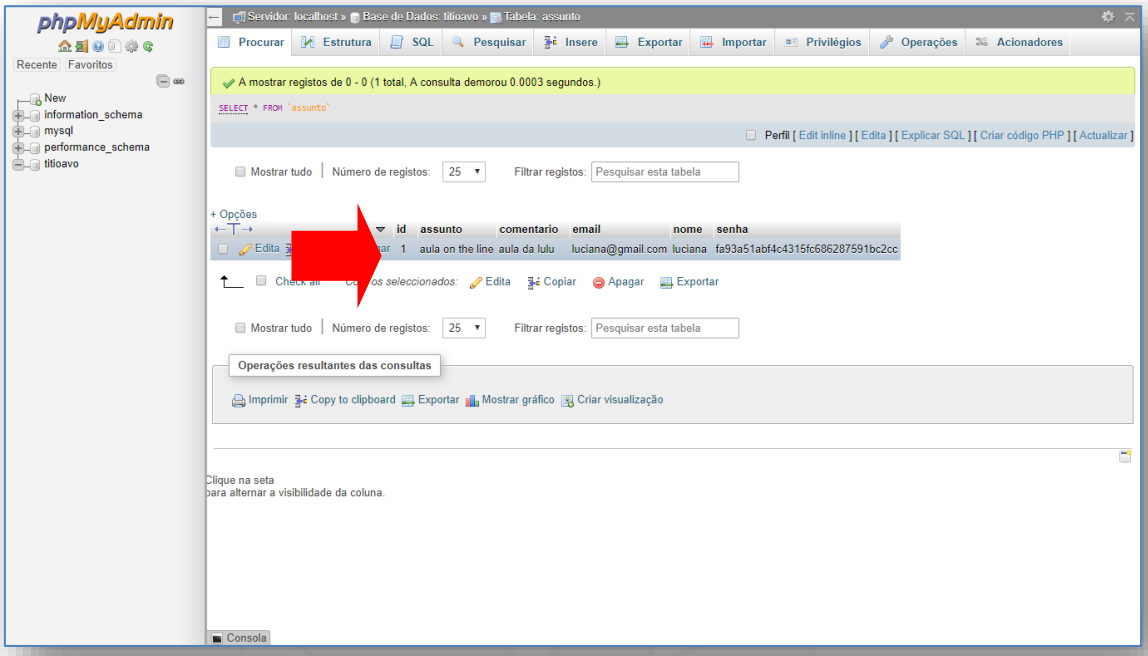
Seguindo o caminho da web...
http://lulucoti.jelasticlw.com.br/projeto/index.jsf



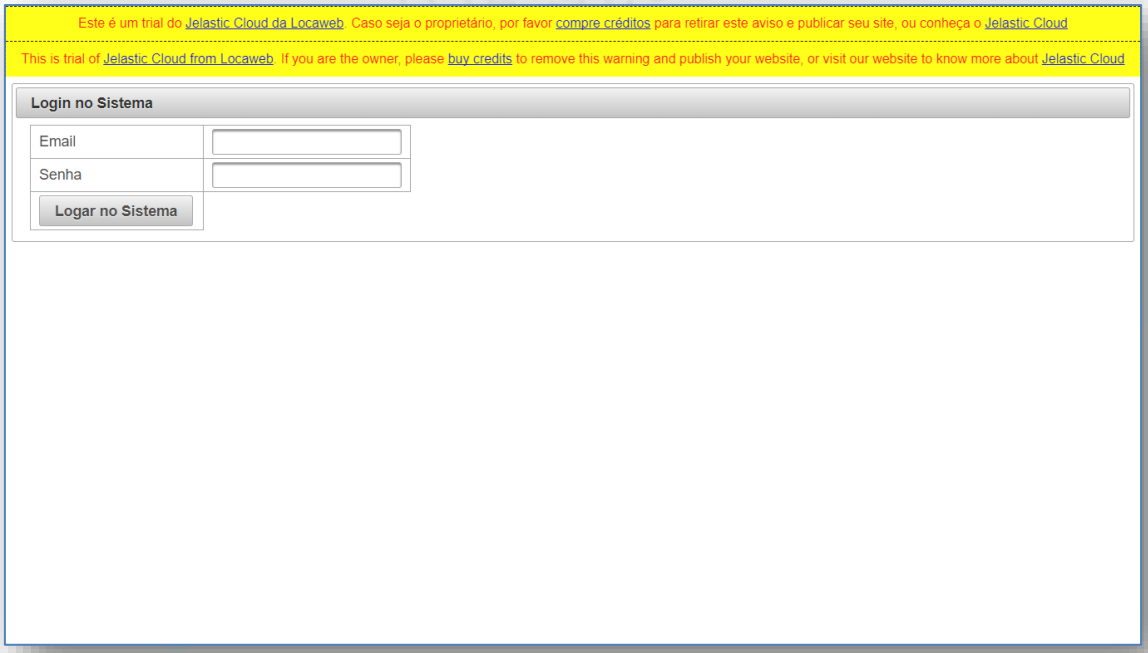
Cadastrando...



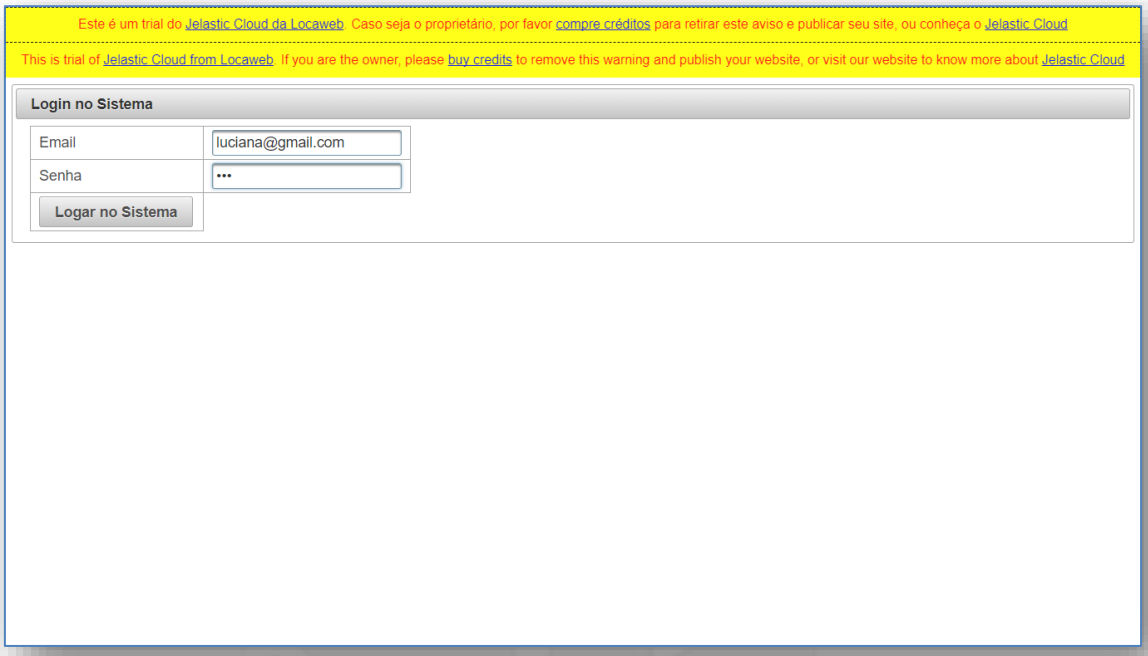
Gravado no banco...



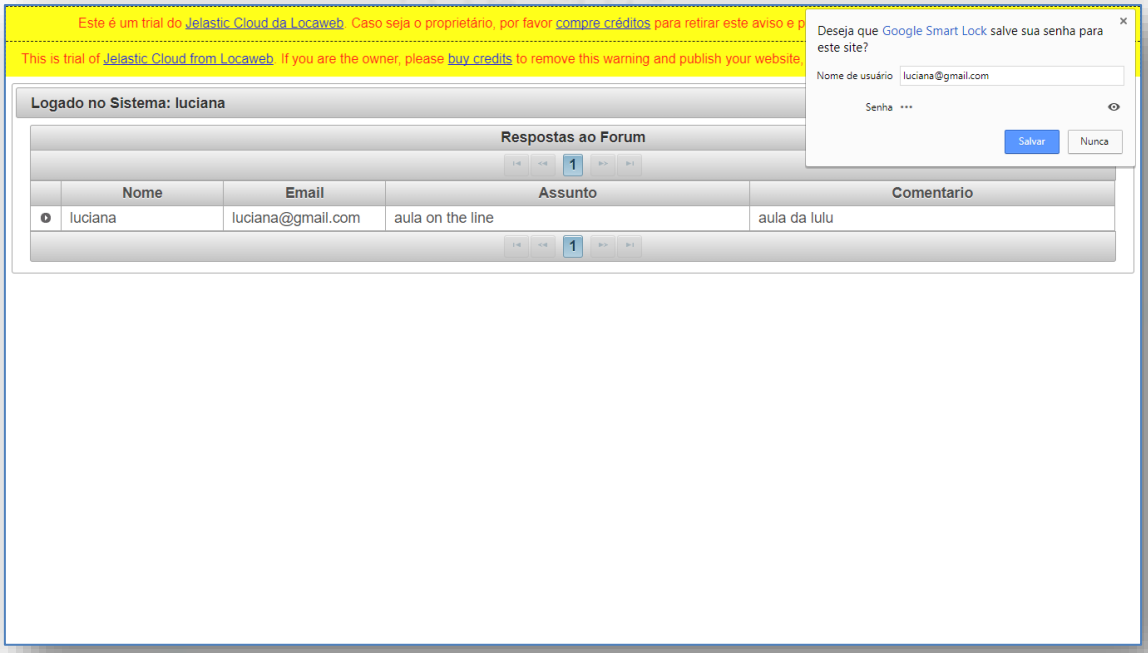
Clicando em logar...



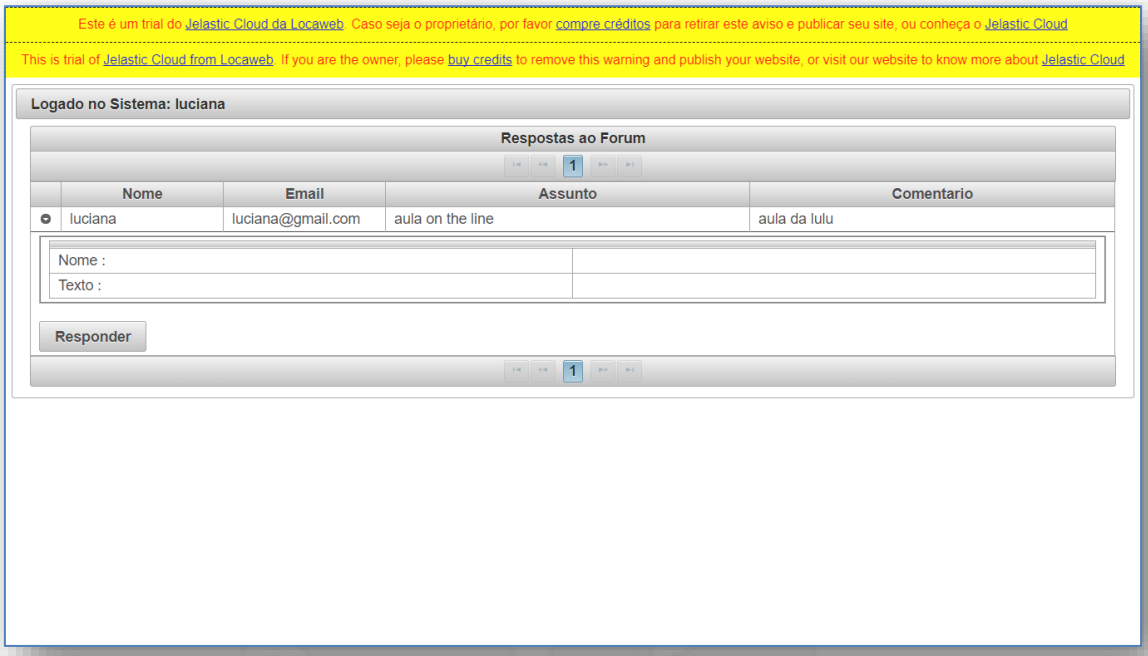
Logando...



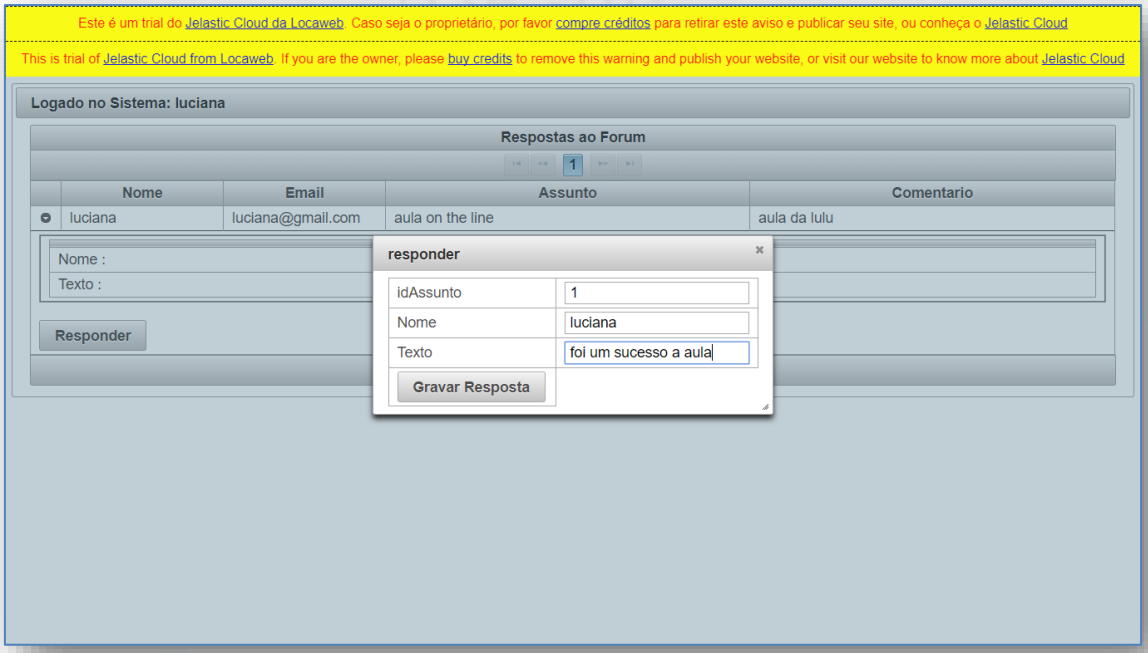
Logado...



Clicando em expandir...

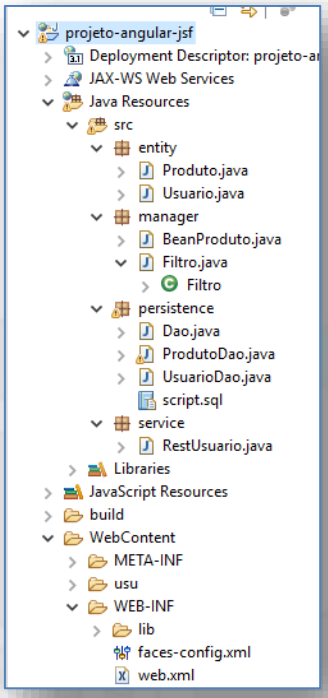


Escrevendo o comentário...



PROJETO SERVER:

Estrutura do projeto:



LIBS:

all-themes-1.0.10.jar
asm-3.1.jar
commons-beanutils-1.8.0.jar
commons-codec-1.6.jar
commons-collections-3.2.1.jar
commons-digester-2.0.jar
commons-logging-1.1.1.jar
fluent-hc-4.2.5.jar
gson-2.2.4.jar
httpclient-4.2.5.jar
httpclient-cache-4.2.5.jar
httpcore-4.2.4.jar
httpmime-4.2.5.jar
jackson-core-asl-1.9.2.jar
jackson-jaxrs-1.9.2.jar
jackson-mapper-asl-1.9.2.jar
jackson-xc-1.9.2.jar

javax.faces-2.0.11.jar
jaxen-1.1.6.jar
jdom-2.0.5.jar
jersey-client-1.12.jar
jersey-core-1.12.jar
jersey-json-1.12.jar
jersey-server-1.12.jar
jersey-servlet-1.12.jar
jettison-1.1.jar
jsr311-api-1.1.1.jar
jstl.jar
mysql-connector-java-5.1.23-bin.jar
primefaces-5.0-sources.jar
primefaces-5.0.jar
servlet3-api.jar
standard.jar
xercesImpl.jar
xml-apis.jar

```
package entity;

public class Produto {

	private Integer codigo;
	private String  nome;
	private Double  preco;

	public Produto() {

	}

	public Produto(Integer codigo, String nome, Double preco) {
		this.codigo = codigo;
		this.nome = nome;
		this.preco = preco;
	}

	@Override
	public String toString() {
		return "Produto [codigo=" + codigo + ", nome=" + nome
+ ", preco=" + preco + "]";
	}
```

```
	public Integer getCodigo() {
		return codigo;
	}
	public void setCodigo(Integer codigo) {
		this.codigo = codigo;
	}
	public String getNome() {
		return nome;
	}
	public void setNome(String nome) {
		this.nome = nome;
	}
	public Double getPreco() {
		return preco;
	}
	public void setPreco(Double preco) {
		this.preco = preco;
	}
}
```

```
package entity;

public class Usuario {

	private Integer id;
	private String login;
	private String senha;

	public Usuario() {

	}

	public Usuario(Integer id, String login, String senha) {
		super();
		this.id = id;
		this.login = login;
		this.senha = senha;
	}

	@Override
	public String toString() {
		return "Usuario [id=" + id + ", login=" + login + ",
senha=" + senha + "]";
```

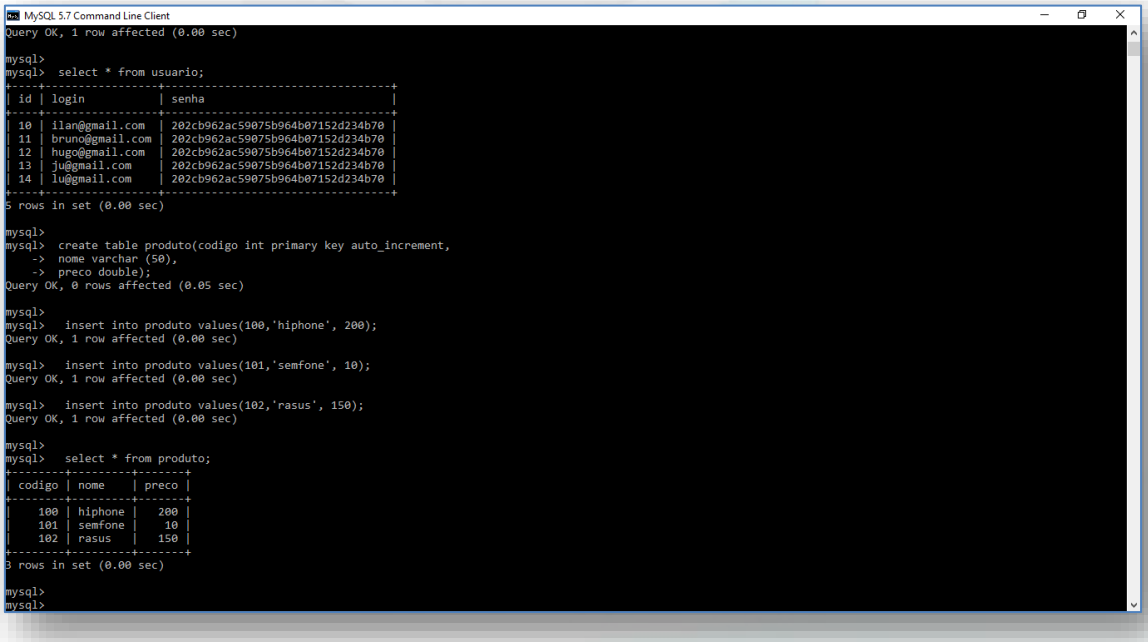
```
	}

	public Integer getId() {
		return id;
	}
	public void setId(Integer id) {
		this.id = id;
	}
	public String getLogin() {
		return login;
	}
	public void setLogin(String login) {
		this.login = login;
	}
	public String getSenha() {
		return senha;
	}
	public void setSenha(String senha) {
		this.senha = senha;
	}
}
```

---

script.sql

```
drop database if  exists git;

create database git;

use git;

 create table usuario(
	id int primary key auto_increment,
	login varchar (100) unique,
	senha varchar (255) not null
);

 insert into usuario values (10,'ilan@gmail.com',md5('123'));
 insert into usuario values (12,'hugo@gmail.com',md5('123'));
 insert into usuario values (11,'bruno@gmail.com',md5('123'));
 insert into usuario values (13,'ju@gmail.com',md5('123'));
 insert into usuario values (14,'lu@gmail.com',md5('123'));
```

```
select * from usuario;

create table produto(
    codigo int primary key auto_increment,
    nome varchar (50),
    preco double
    );

 insert into produto values(100,'hiphone', 200);
 insert into produto values(101,'semfone', 10);
 insert into produto values(102,'rasus', 150);

 select * from produto;
```



```
package persistence;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Dao {
```

```
	Connection con;
	PreparedStatement stmt;
	ResultSet rs;

	public void open() throws Exception{
		Class.forName("com.mysql.jdbc.Driver");
		con = DriverManager.
getConnection("jdbc:mysql://localhost:3306/git","root","coti");
	}

	public void close() throws Exception{
		con.close();
	}
}
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;
import entity.Produto;

public class ProdutoDao  extends Dao{

	public void create(Produto p) throws Exception{
		open();
	  stmt = con.prepareStatement("insert into produto
values(null,?,?)");
		stmt.setString(1,p.getNome());
	    stmt.setDouble(2,p.getPreco());
	    stmt.execute();
	  close();
	}

	public void update(Produto p) throws Exception{
		open();
	  stmt = con.prepareStatement("update produto  set
nome=?,preco=? where codigo=?");
		stmt.setString(1,p.getNome());
	    stmt.setDouble(2,p.getPreco());
	    stmt.setInt(3,p.getCodigo());
	    stmt.execute();
	  close();
	}
```

```
    public void delete(Integer cod)throws Exception{
            open();
              stmt = con.prepareStatement("delete from produto
where codigo=?");
                  stmt.setInt(1,cod);
              stmt.execute();
              stmt.close();
              close();
    }

    public List<Produto> findAll() throws Exception{
          open();
            stmt = con.prepareStatement("select * from produto");
            rs = stmt.executeQuery();
           List <Produto> lst= new ArrayList<Produto>();
           while(rs.next()){
              Produto p =new Produto();
                      p.setCodigo(rs.getInt("codigo"));
                      p.setNome(rs.getString("nome"));
                        p.setPreco(rs.getDouble("preco"));
              lst.add(p);
           }
          close();
          return lst;
    }

    public Produto findByCode(Integer cod) throws Exception{
          open();
     stmt = con.
    prepareStatement("select * from produto where codigo=?");
       stmt.setInt(1, cod);
            rs = stmt.executeQuery();
           Produto p = null;
           if(rs.next()){
               p =new Produto();
                      p.setCodigo(rs.getInt("codigo"));
                      p.setNome(rs.getString("nome"));
                         p.setPreco(rs.getDouble("preco"));
           }
          close();
          return p;
    }
```

```
    public static void main(String[] args) {
       try{
        ProdutoDao dao = new ProdutoDao();

         Produto p1 = new Produto(null,"motobola",400.);
         Produto p2 = new Produto(null,"proximotel",400.);
          //dao.create(p1);
          //dao.create(p2);
           System.out.println("Produtos Gravados");
         dao.findAll().forEach(System.out::println);
             Produto p = dao.findByCode(100);
              System.out.println("Encontrado:" + p);
       }catch(Exception ex){
          ex.printStackTrace();
       }
}
}
```

No console...

```
Produtos Gravados
Produto [codigo=100, nome=hiphone, preco=200.0]
Produto [codigo=101, nome=semfone, preco=10.0]
Produto [codigo=102, nome=rasus, preco=150.0]
Encontrado:Produto [codigo=100, nome=hiphone, preco=200.0]
```

```
package persistence;

import java.util.ArrayList;
import java.util.List;

import entity.Usuario;

public class UsuarioDao extends Dao{

      public void create(Usuario usuario) throws Exception{
            open();
             stmt = con.prepareStatement("insert into usuario
values (null,?,md5(?) )");
             stmt.setString(1, usuario.getLogin());
             stmt.setString(2, usuario.getSenha());
```

```
            stmt.execute();
            stmt.close();
            close();
      }


   public Usuario login(Usuario u) throws Exception{
         open();
         stmt = con.prepareStatement("select * from usuario where
login=? and senha=md5(?)");
         stmt.setString(1, u.getLogin());
         stmt.setString(2, u.getSenha());
         rs = stmt.executeQuery();
         Usuario usu = null;
         if (rs.next()){
               usu = new Usuario(rs.getInt("id"),
                                 rs.getString("login"),
                                 rs.getString("senha")
                                 );

         }
         close();
         return usu;
   }

   public List<Usuario> findAll()throws Exception{
         open();
         stmt = con.prepareStatement("select * from usuario");
         rs = stmt.executeQuery();
         List<Usuario> lst = new ArrayList<Usuario>();
         while (rs.next()){
               Usuario usu = new Usuario(rs.getInt("id"),
                                 rs.getString("login"),
                                 rs.getString("senha")
                                 );
           lst.add(usu);
         }
         close();
         return lst;
   }

   public static void main(String[] args) {
   try{
//  Usuario u = new Usuario(null,"minimim@gmail.com","123");
//  new UsuarioDao().create(u);
```

```
//   System.out.println("Dados Gravados");

        Usuario u = new UsuarioDao().login(
              new Usuario(null,"lu@gmail.com","1234"));

        if (u!=null){
              System.out.println("logado :" + u.getLogin());
        }else{
              System.out.println("nao logado" );
        }
    }catch(Exception ex){
       ex.printStackTrace();
    }
 }
}
```

No console.

```
nao logado
```

---

web.xml

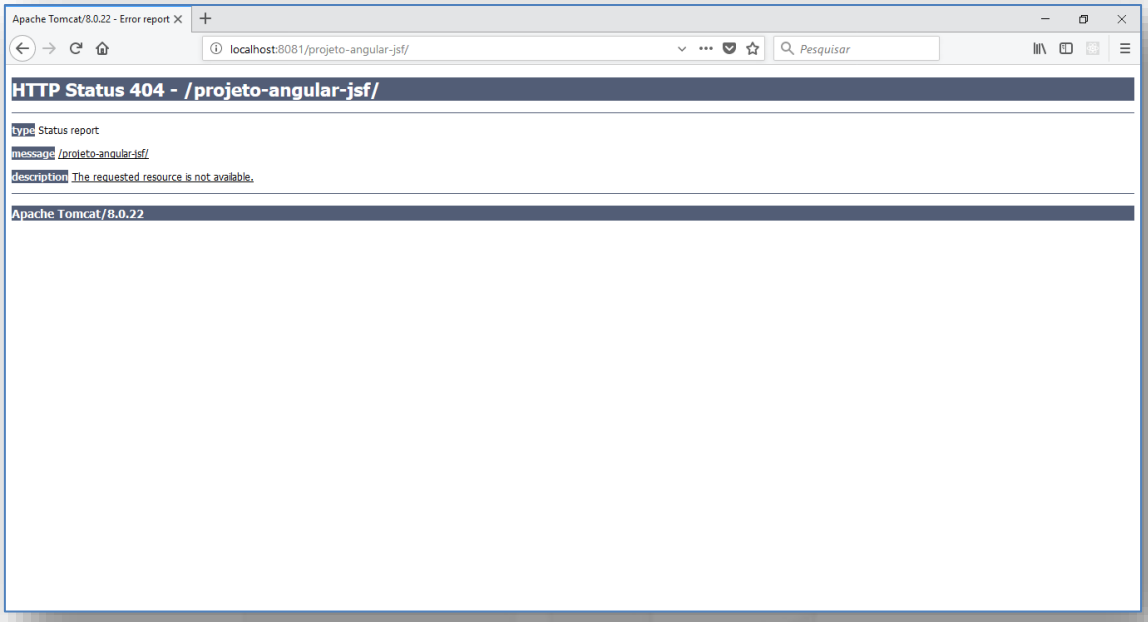
```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <display-name>projeto-angular-jsf</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-
class>
```

```
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
  </servlet-mapping>

  <servlet>
     <servlet-name>Jersey REST Service</servlet-name>
     <servlet-
class>com.sun.jersey.spi.container.servlet.ServletContainer</ser
vlet-class>
     <init-param>
          <param-
name>com.sun.jersey.config.property.packages</param-name>
          <param-value>service</param-value>
     </init-param>
     <load-on-startup>1</load-on-startup>
  </servlet>

  <servlet-mapping>
     <servlet-name>Jersey REST Service</servlet-name>
     <url-pattern>/json/*</url-pattern>
  </servlet-mapping>

</web-app>
```

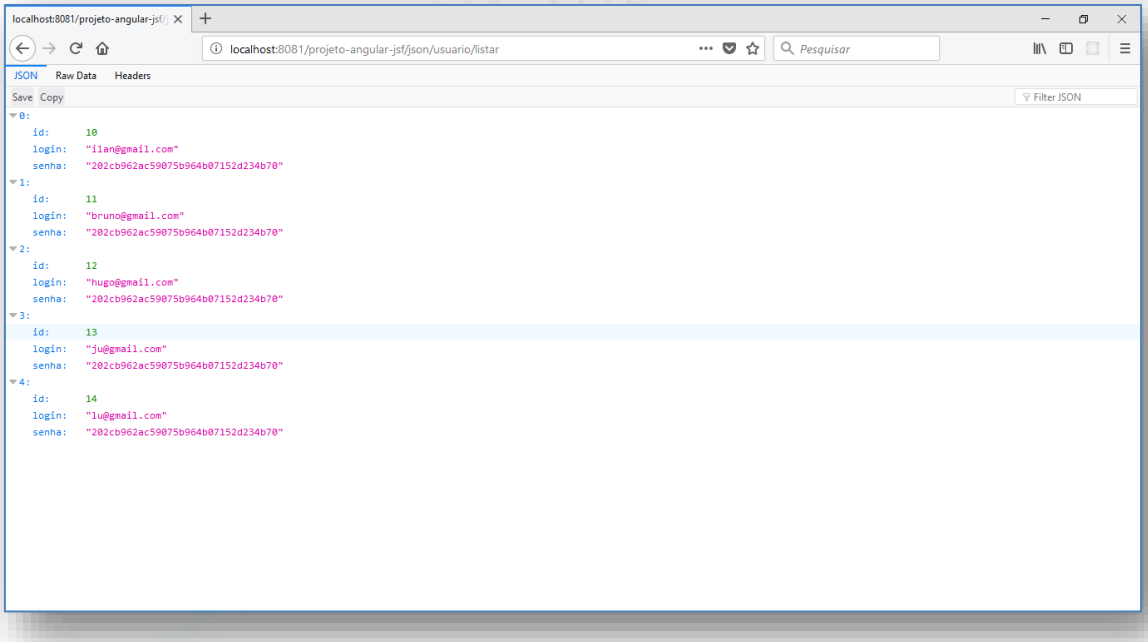
```
package service;

import javax.ws.rs.Consumes;
import javax.ws.rs.GET;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

import com.google.gson.Gson;

import entity.Usuario;
import persistence.UsuarioDao;

@Path("/usuario")
public class RestUsuario {
```

```
        @POST
        @Consumes(MediaType.APPLICATION_JSON)
        @Produces(MediaType.TEXT_PLAIN)
        @Path("/gravar")
        public String gravar(String usuario){
              try{
               System.out.println(usuario);
    Usuario u = new Gson().fromJson(usuario,Usuario.class);
                new UsuarioDao().create(u);
               return "Usuario Gravado";
              }catch(Exception ex){
                   return "-1";
              }
        }


        @GET
        @Produces(MediaType.APPLICATION_JSON)
        @Path("/listar")
        public String listar(){
              try{
          return new Gson().toJson( new UsuarioDao().findAll() );
              }catch(Exception ex){
           return "Error :" + ex.getMessage();
              }
        }
}
```

Rodando pelo projeto...



Passando a URL...
http://localhost:8081/projeto-angular-jsf/json/usuario/listar



```
package manager;

import java.util.List;

import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.RequestScoped;
import javax.faces.context.FacesContext;
import javax.servlet.http.HttpSession;

import entity.Produto;
import entity.Usuario;
import persistence.ProdutoDao;
import persistence.UsuarioDao;

@ManagedBean(name="mb")
@RequestScoped
public class BeanProduto {

	private Usuario usuario;
	private Usuario logado;
	private Produto produto;
	private List<Produto> produtos;

	public BeanProduto() {
	    produto = new Produto();
	    usuario = new Usuario();
	}

	public Produto getProduto() {
		return produto;
	}
	public void setProduto(Produto produto) {
		this.produto = produto;
	}

	public List<Produto> getProdutos() {
		try{
		produtos = new ProdutoDao().findAll();
		}catch(Exception ex){
			ex.printStackTrace();
		}
		return produtos;
	}
```

```
    public void gravarUsuario(){
          FacesContext fc = FacesContext.getCurrentInstance();
          try{
          new UsuarioDao().create(usuario);
          usuario = new Usuario();
          fc.addMessage(null,
                  new FacesMessage("Dados gravados"));
    }catch(Exception ex){
fc.addMessage(null,
           new FacesMessage("Error :" + ex.getMessage()));
          }
    }

    public String logar(){
      FacesContext fc = FacesContext.getCurrentInstance();
      try{
     HttpSession session =(HttpSession)
           fc.getExternalContext().getSession(true);
       logado = (Usuario) new UsuarioDao().login(usuario);
       if (logado !=null){
        //Filtro tenho que tratar Session ...
        session.setAttribute("logado", logado);
        fc.addMessage(null,
                  new FacesMessage("usuario Logado"));
         return "usu/logado.jsf?faces-redirect=true";
       }
       else{
          fc.addMessage(null,
                  new FacesMessage("Login Invalido"));
          session.invalidate();
         return null;
       }

      }catch(Exception ex){
        return null;
      }
    }

    public String logout(){
          FacesContext fc = FacesContext.getCurrentInstance();
          HttpSession session= (HttpSession)
            fc.getExternalContext().getSession(true);
    try{
     session.setAttribute("logado",null);
```

```
        session.invalidate();
    fc.addMessage(null, new FacesMessage("Logout no Sistema "));

    return "../sistema.jsf?faces-redirect=true";
        }catch(Exception ex){
    fc.addMessage(null, new FacesMessage("Erro ..." +
ex.getMessage()) );
    return "../sistema.jsf?faces-redirect=true";
        }
     }

     public void setProdutos(List<Produto> produtos) {
          this.produtos = produtos;
     }
     public Usuario getUsuario() {
          return usuario;
     }
     public void setUsuario(Usuario usuario) {
          this.usuario = usuario;
     }
     public Usuario getLogado() {
          return logado;
     }
     public void setLogado(Usuario logado) {
          this.logado = logado;
     }
}
```

```
package manager;

import java.io.IOException;

import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.FilterConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebFilter(urlPatterns="/usu/logado.jsf")
```

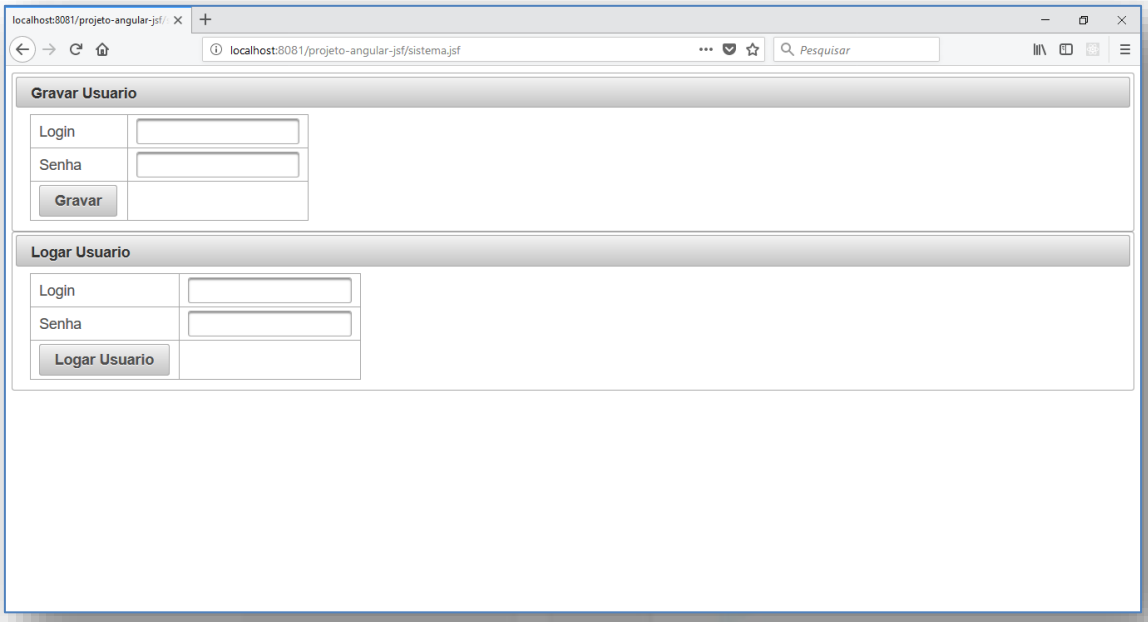
```
public class Filtro implements Filter
{

      public Filtro() {
    }
      public void destroy() {
      }

      public void doFilter(ServletRequest request,
ServletResponse response, FilterChain chain) throws IOException,
ServletException {
  HttpServletRequest  req = (HttpServletRequest) request;
  HttpServletResponse resp=(HttpServletResponse) response;
 HttpSession session = req.getSession();

  if (session.getAttribute("logado")!=null){
      chain.doFilter(request, response);
  }else{
  resp.sendRedirect("../sistema.jsf");
  }
}
        //IONIC (400) _ 4 de 100
       //ANGULAR 1 ao 4 TOTAL - manha (380) depois carnaval

      public void init(FilterConfig fConfig) throws
ServletException {

      }
}
```

---

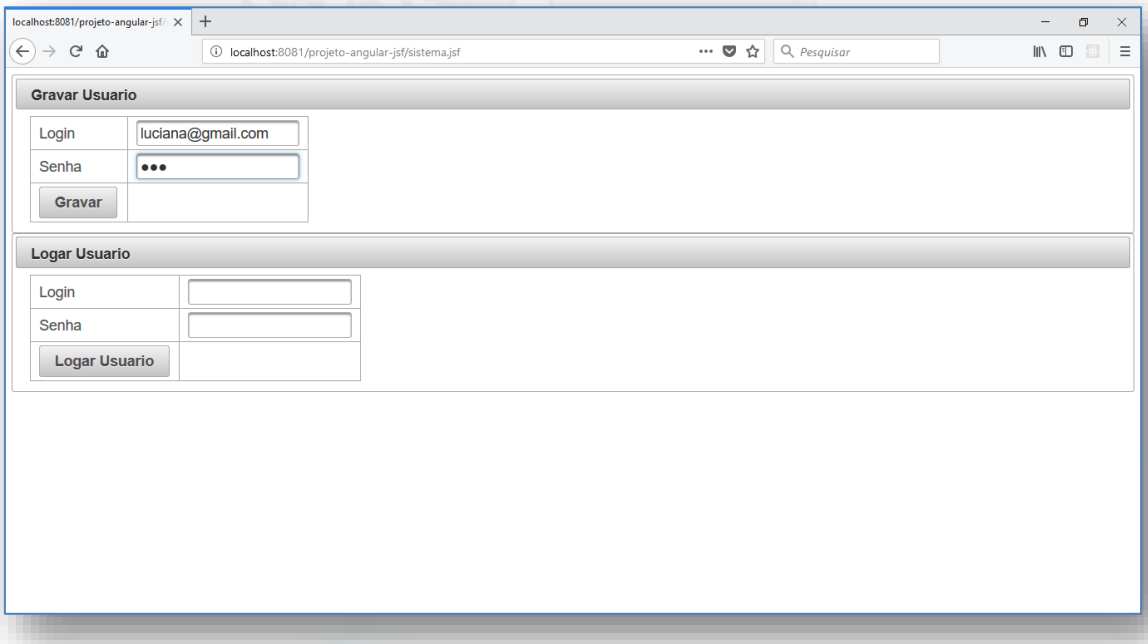
sistema.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:p="http://primefaces.org/ui"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">
 <h:head>
 </h:head>
 <h:body>
```

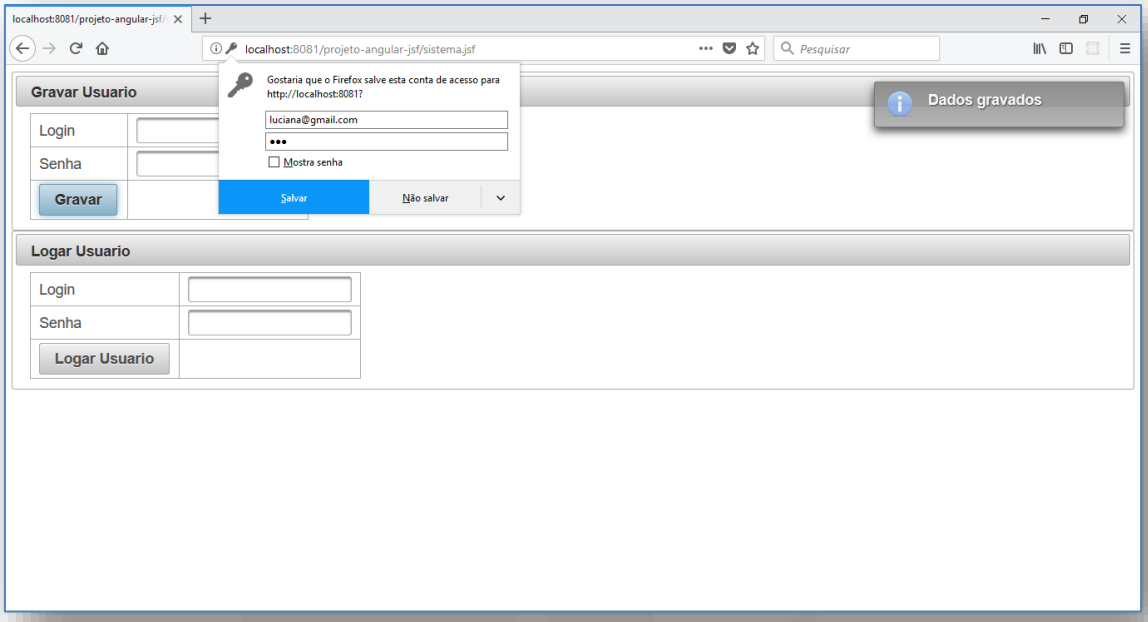
```
<p:panel header="Gravar Usuario">
  <h:form id="form1">
    <p:panelGrid columns="2">
      <h:outputText value="Login" />
      <p:inputText value="#{mb.usuario.login}" />

      <h:outputText value="Senha" />
      <p:password  value="#{mb.usuario.senha}" />

         <p:commandButton value="Gravar"
                          action="#{mb.gravarUsuario}"
                          update=":form1"
                          ajax="true"
          ></p:commandButton>
    </p:panelGrid>
      <p:growl></p:growl>
  </h:form>

</p:panel>
 <p:panel header="Logar Usuario">
  <h:form id="form2">
    <p:panelGrid columns="2">
      <h:outputText value="Login" />
      <p:inputText value="#{mb.usuario.login}" />

      <h:outputText value="Senha" />
      <p:password  value="#{mb.usuario.senha}" />

         <p:commandButton value="Logar Usuario"
                          action="#{mb.logar}"
                          update=":form1"
                          ajax="true"
          ></p:commandButton>
    </p:panelGrid>
      <p:growl></p:growl>
  </h:form>

 </p:panel>


</h:body>

</html>
```



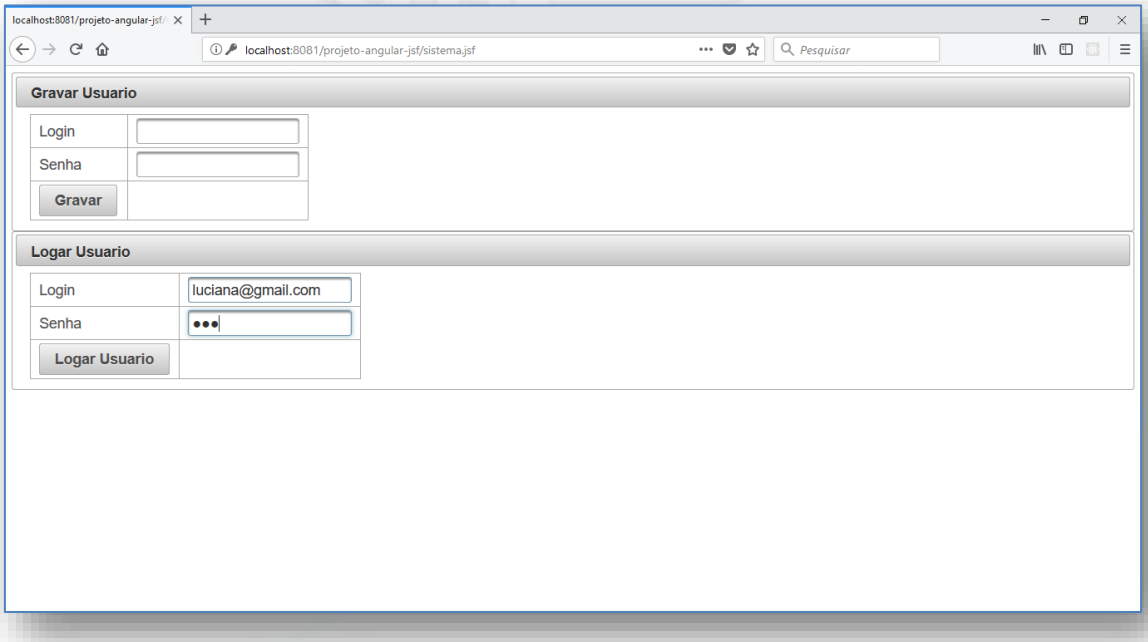
Cadastrando...



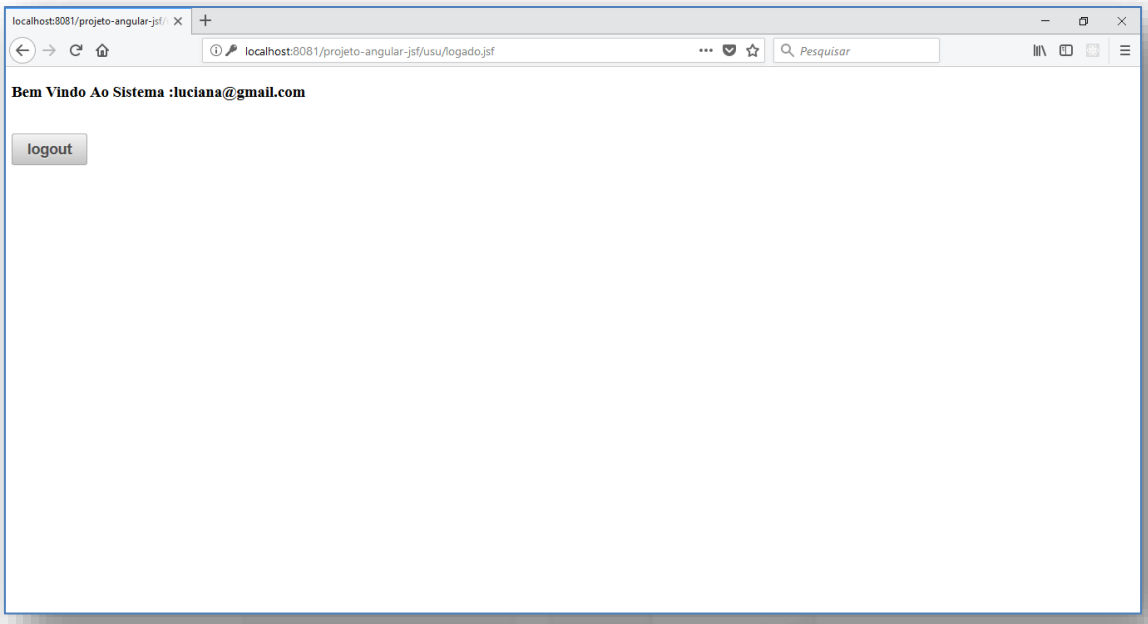
Gravado...



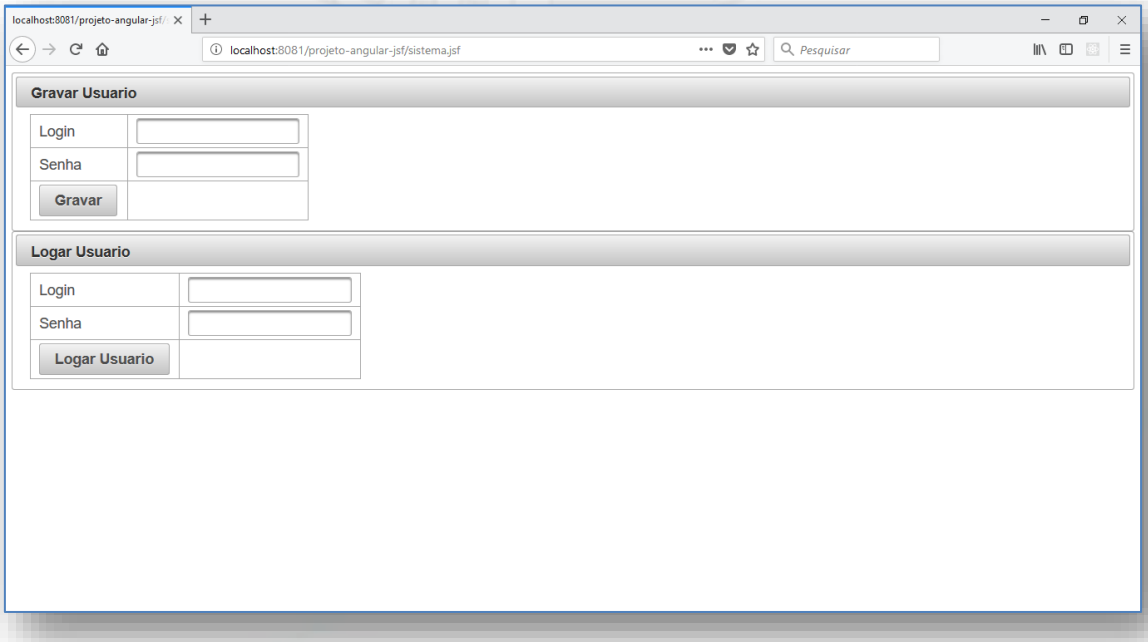
Logando...



Logado...

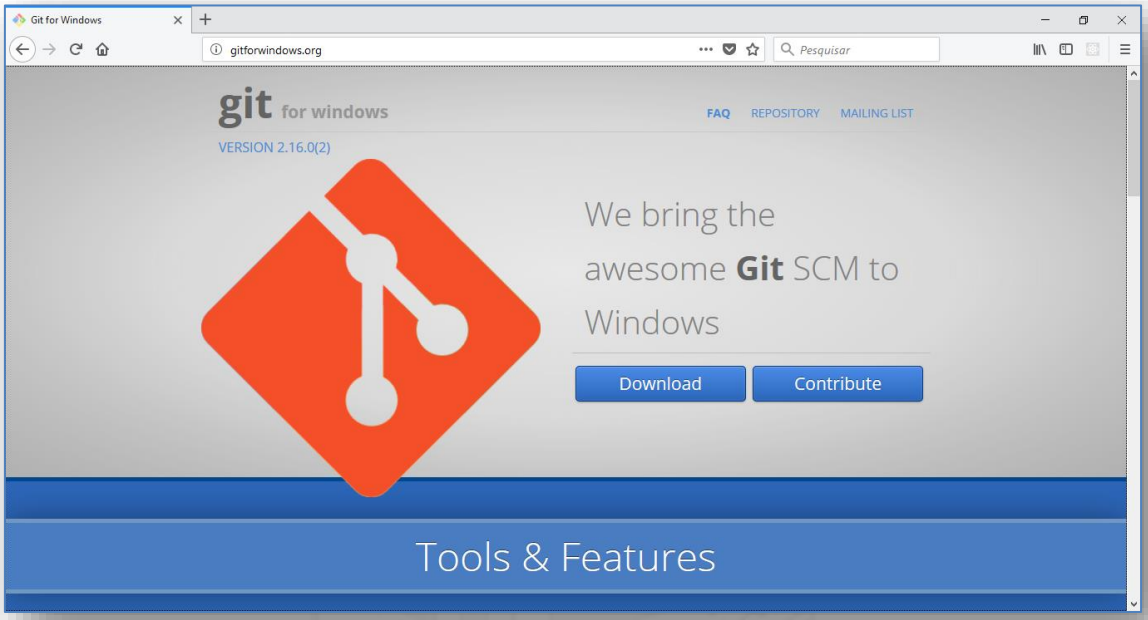


Clicando em logout...

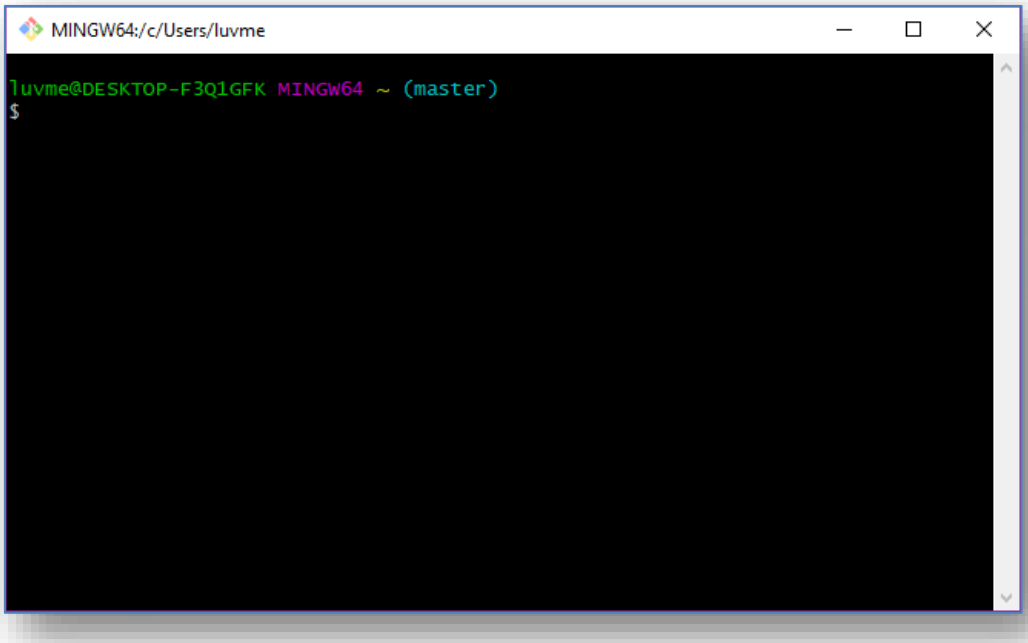


INICIANDO O GIT...

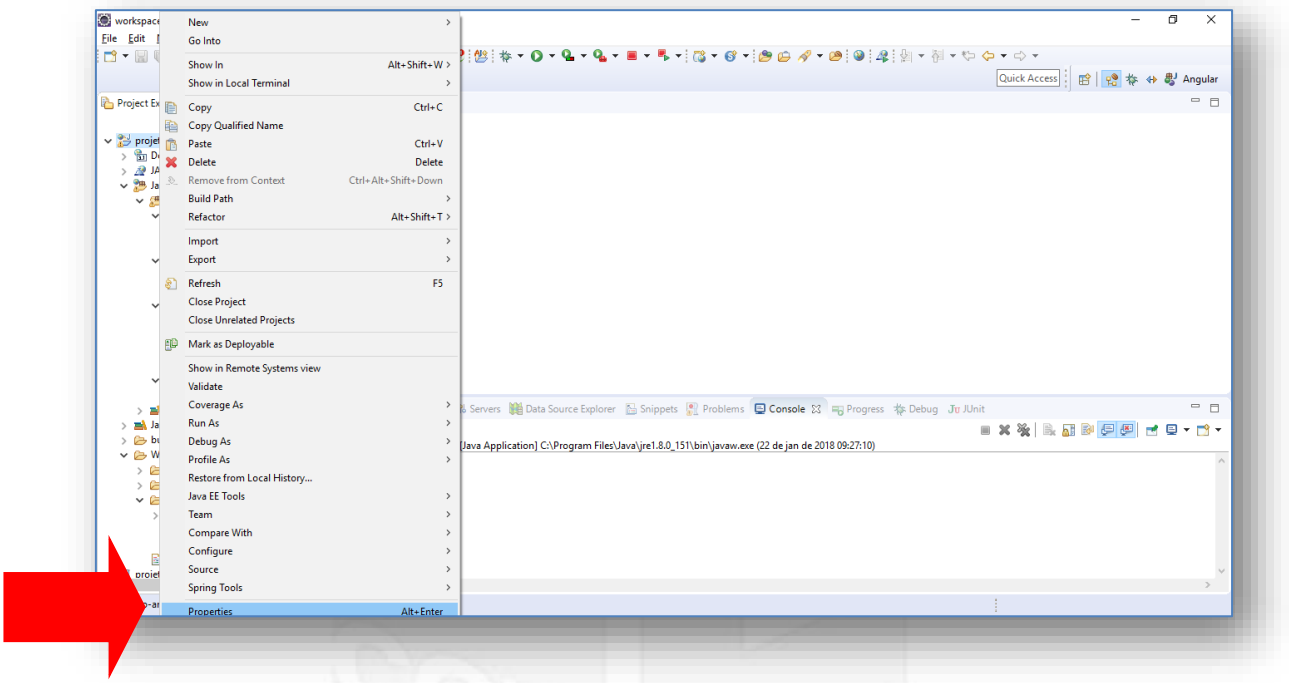
No Windows baixar o GitBash...
http://gitforwindows.org/



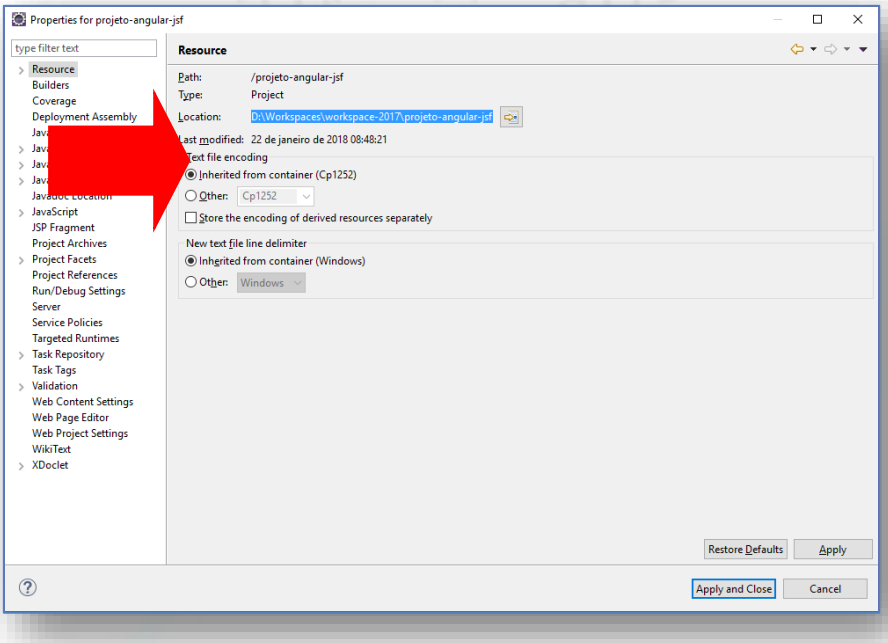
Depois de instalado...



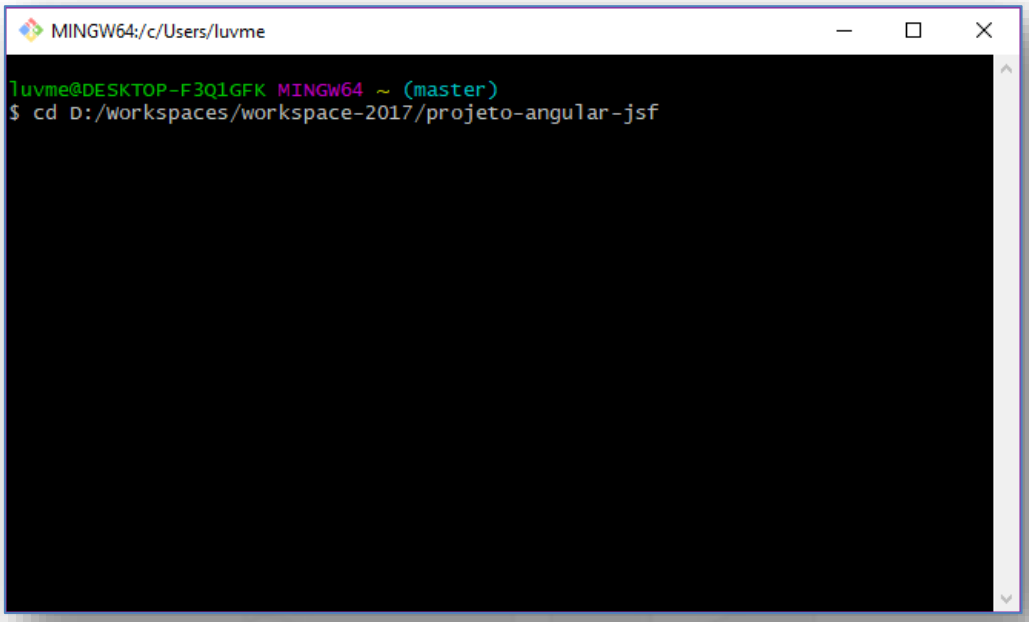
Pegar o caminho do projeto java.
Para pegar o caminho é só clicar com o botão direito no projeto, Properties..



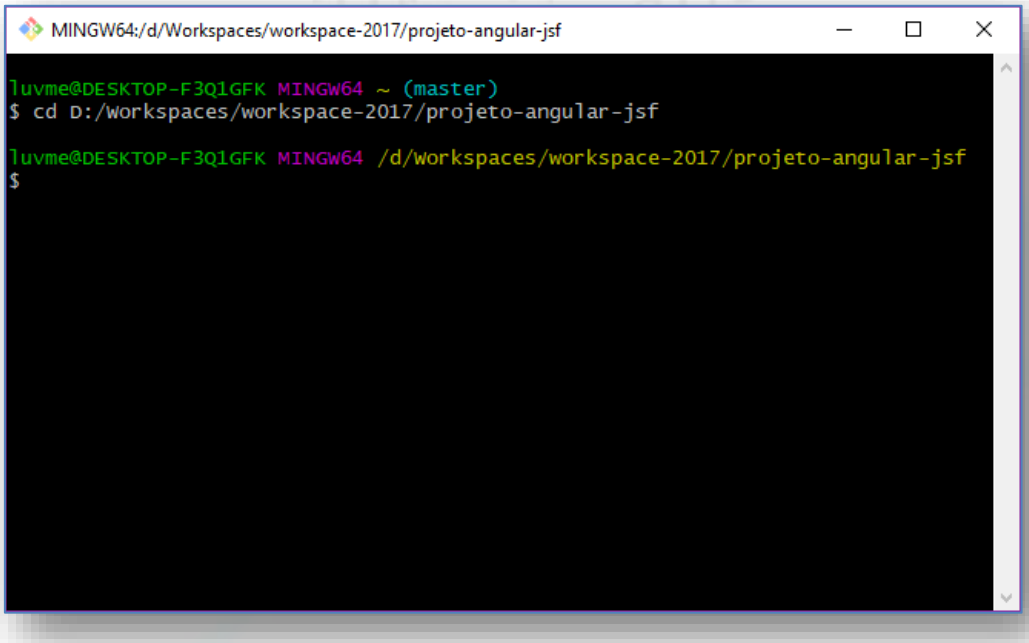
Clicar em Resources, pegar o endereço em Location...



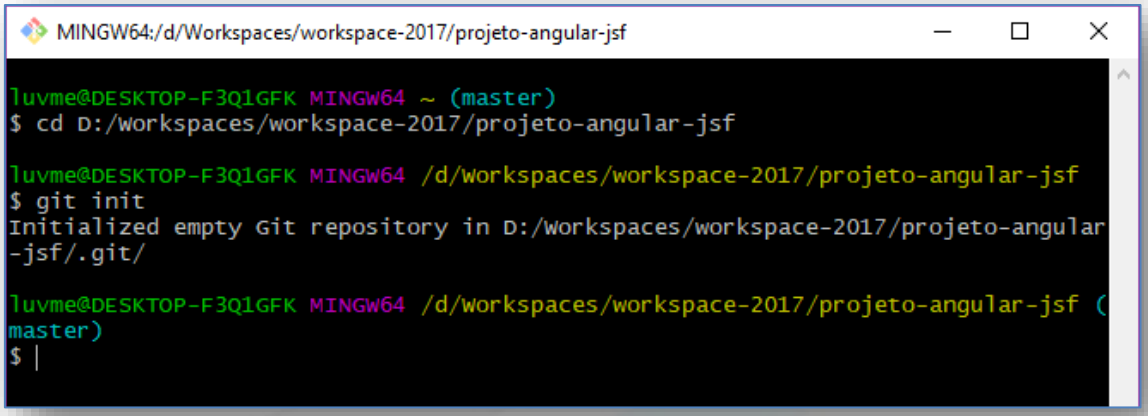
No gitbash, digitar (inverter as barras) -> “cd D:/Workspaces/workspace-2017/projeto-angular-jsf”...



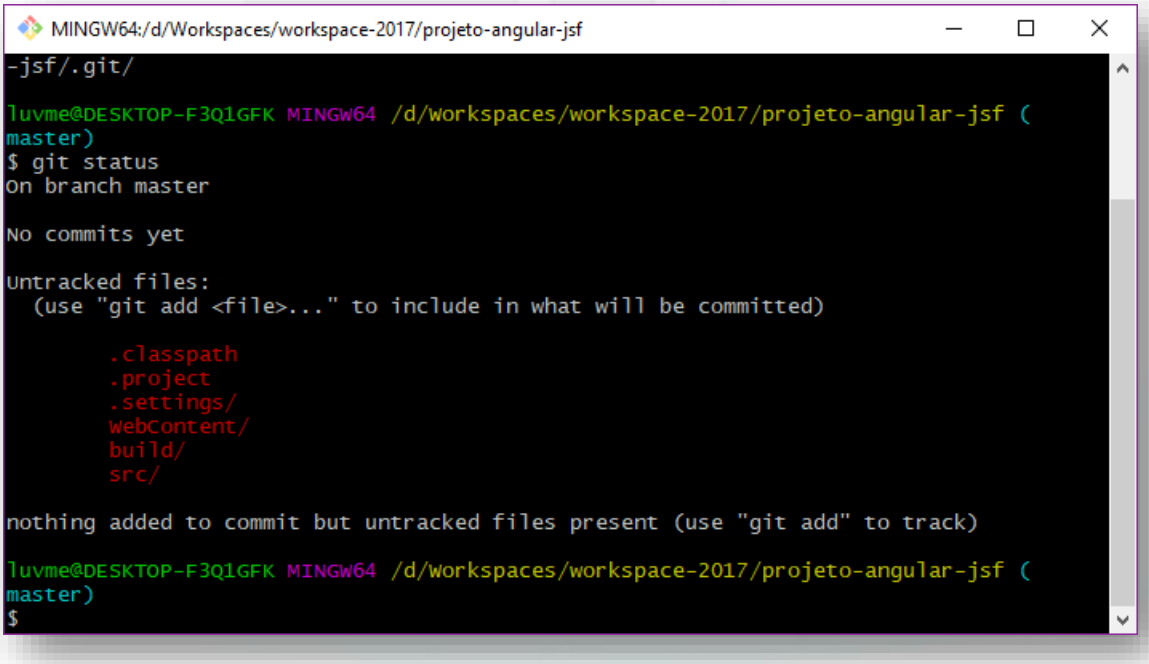
Entrou no diretorio do projeto..



Para iniciar o git, digitar “git init”....

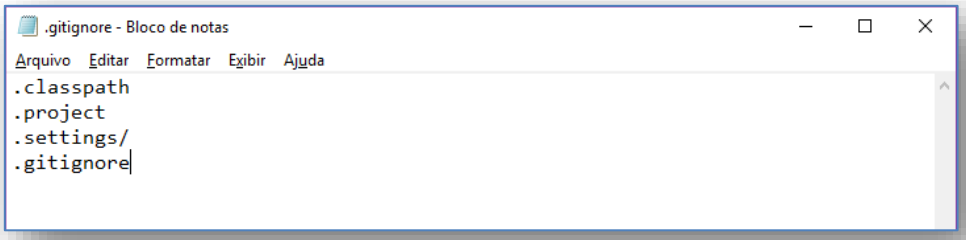


Para mostrar o status do projeto no git, digitar “git status”...

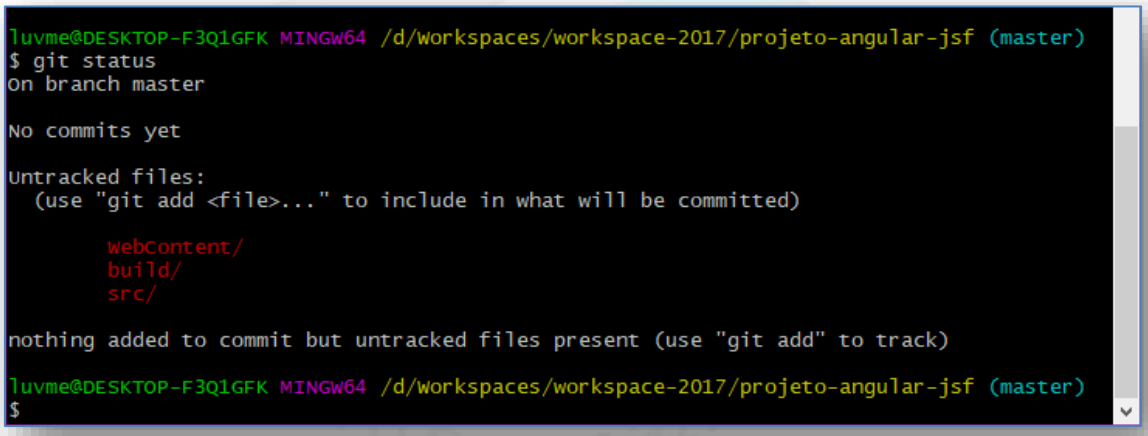


Ir no diretório do projeto e criar um arquivo de texto chamado “.gitignore” contendo o nome das pastas(diretórios) que não serão monitorados pelo git.

.classpath
.project
.settings/
.gitignore

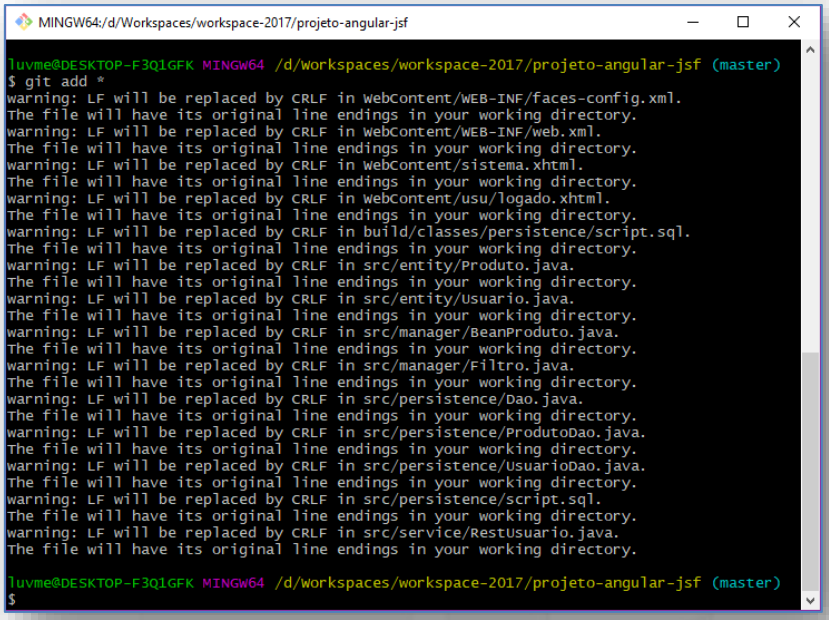


Para verificar o status do git novamente, digitar “git status”...
--mostra os arquivos nao ignorados

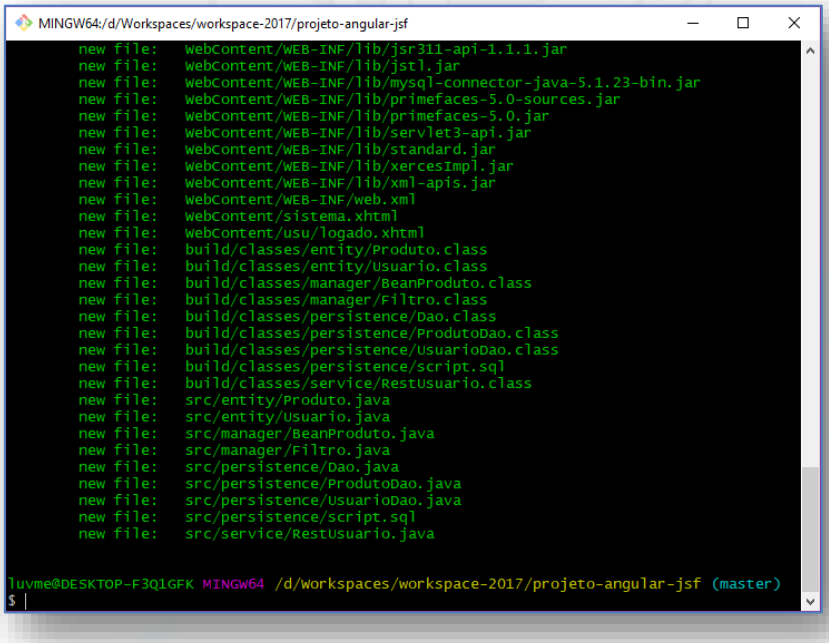


Escola de Nerds
www.cotiinformatica.com.br

Para adicionar as pastas ao git, digitar: “git add *”

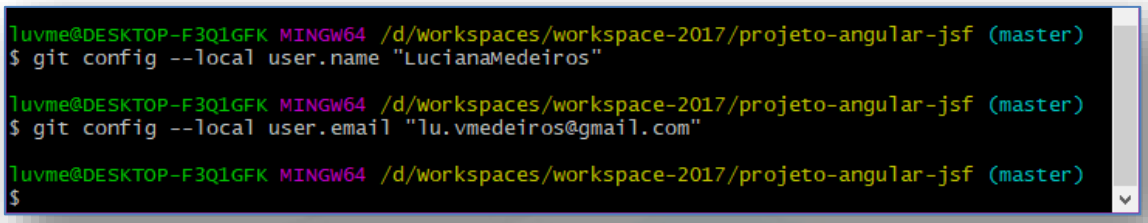


Digitar: “git status” para verificar se todos arquivos do repositorio estarão verdes. Isto é, foram adicionados ao git.

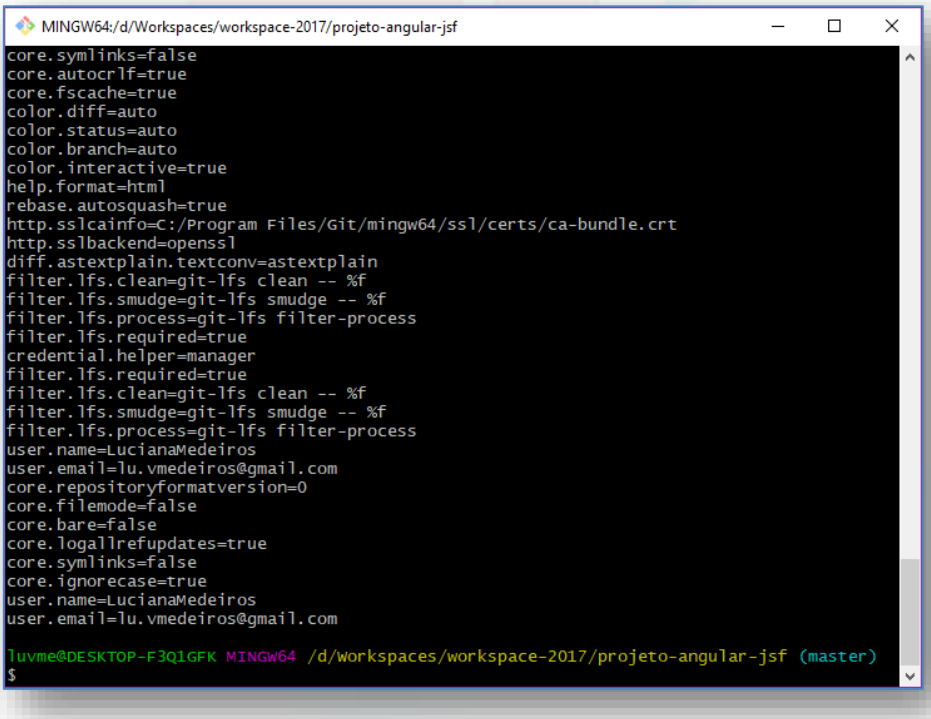


Para cadastrar o usuário e email no git. Digitar o usuario:
git config --local user.name "edsonbelem44"

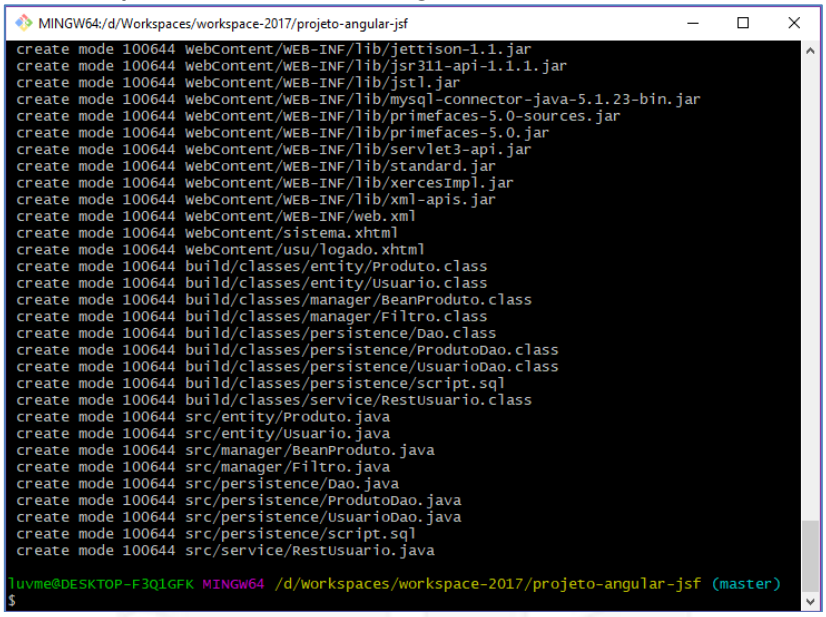
Digitar o email:
git config --local user.email "profedsonbelem@gmail.com"



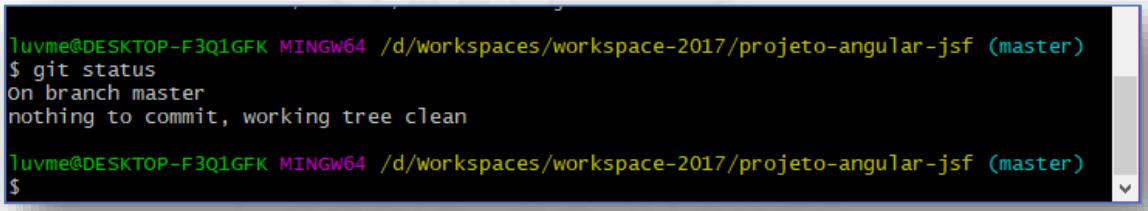
Para mostrar as configurações do git, digitar: “git config --list”



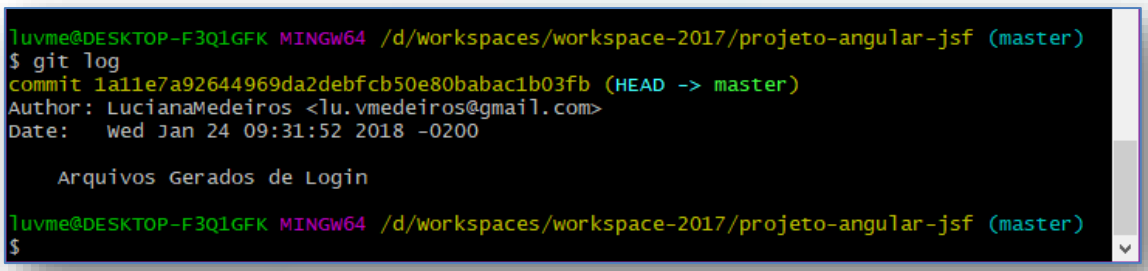
Para atualizar na maquina (no Banco git local), digitar o comando do git e escrever uma mensagem de atualização...
git commit -m "Arquivos Gerados de Login"



Novamente ver o status do projeto, digitar: “git status”

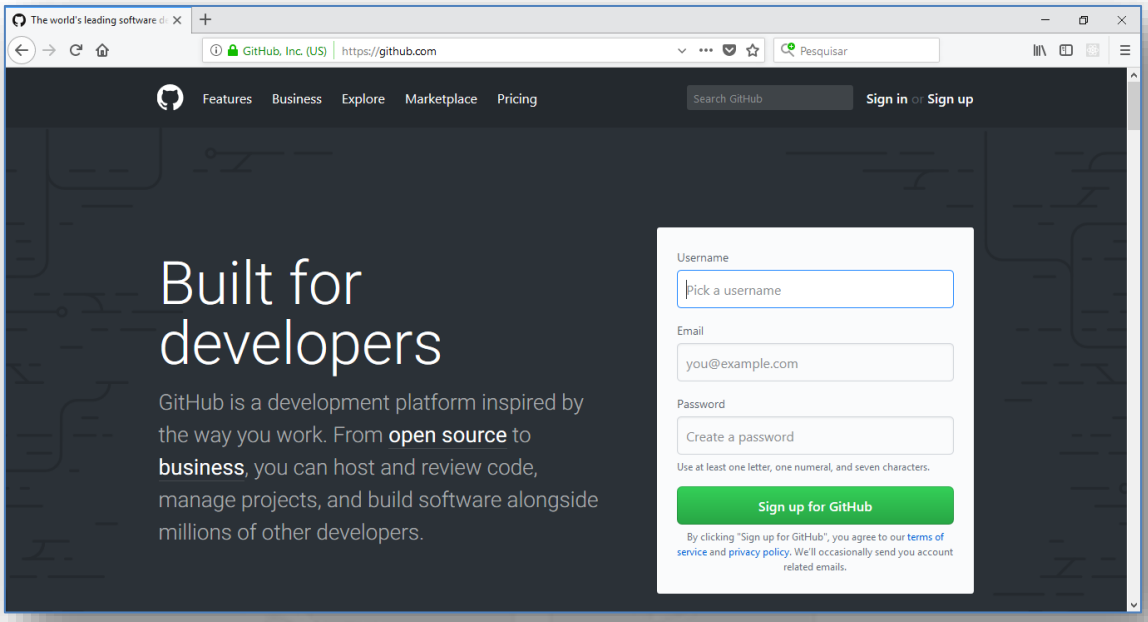


Na mensagem informa que não foram feitas alterações no projeto.
Para ver o histórico do que já foi feito, digitar: “git log”

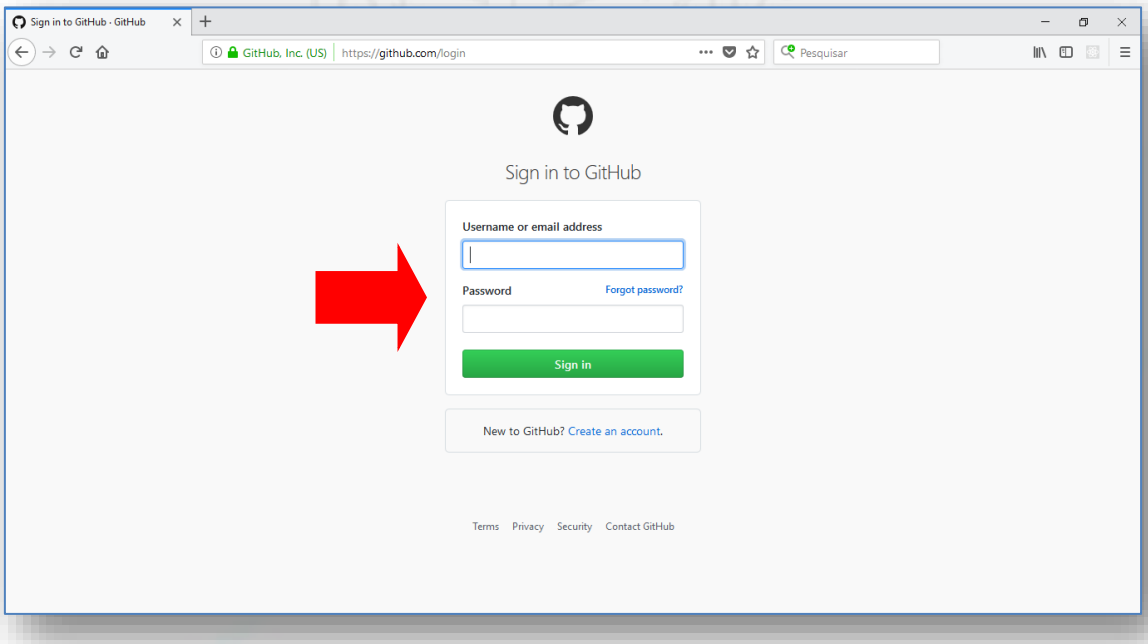


INICIANDO O GITHUB:

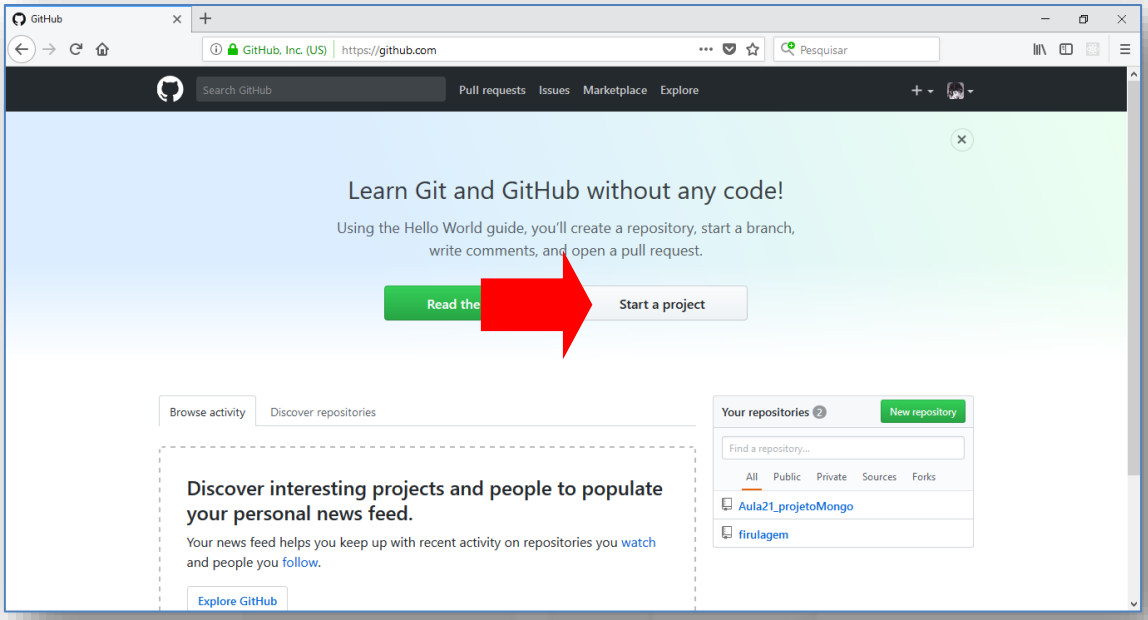
www.github.com



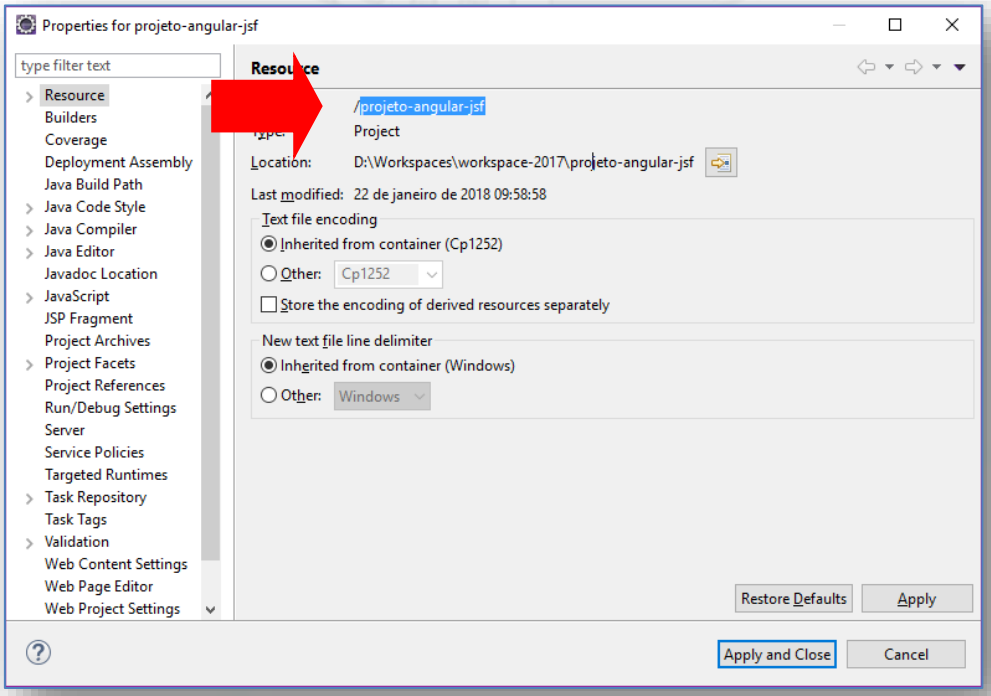
Se não tiver uma conta, criar. Logar...



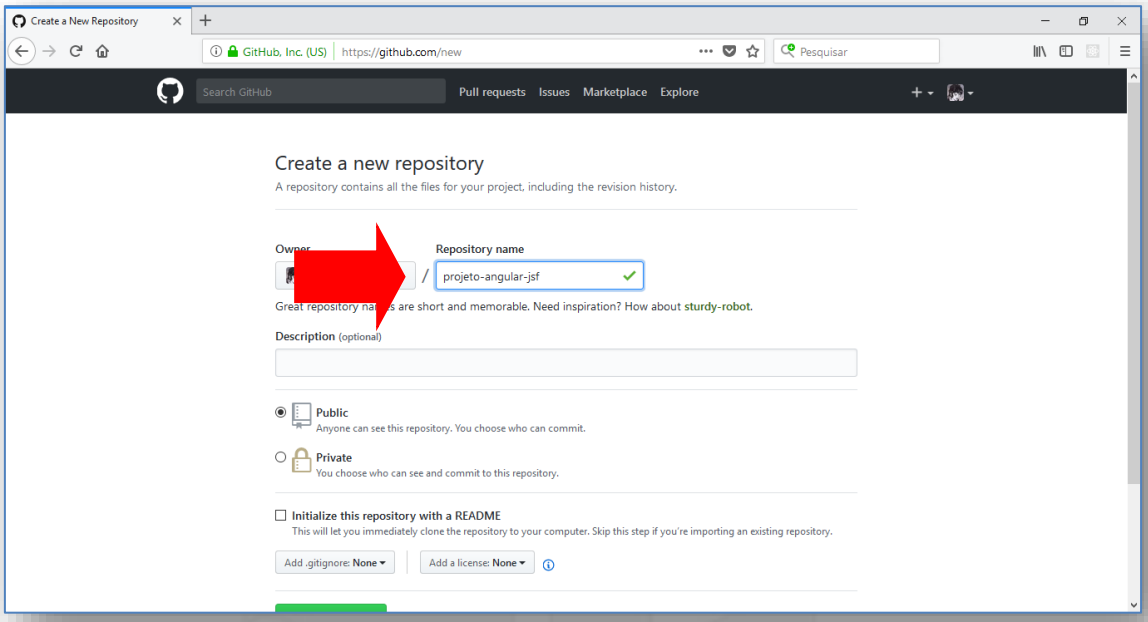
Entrando na sua conta, clicar em Start a Project...



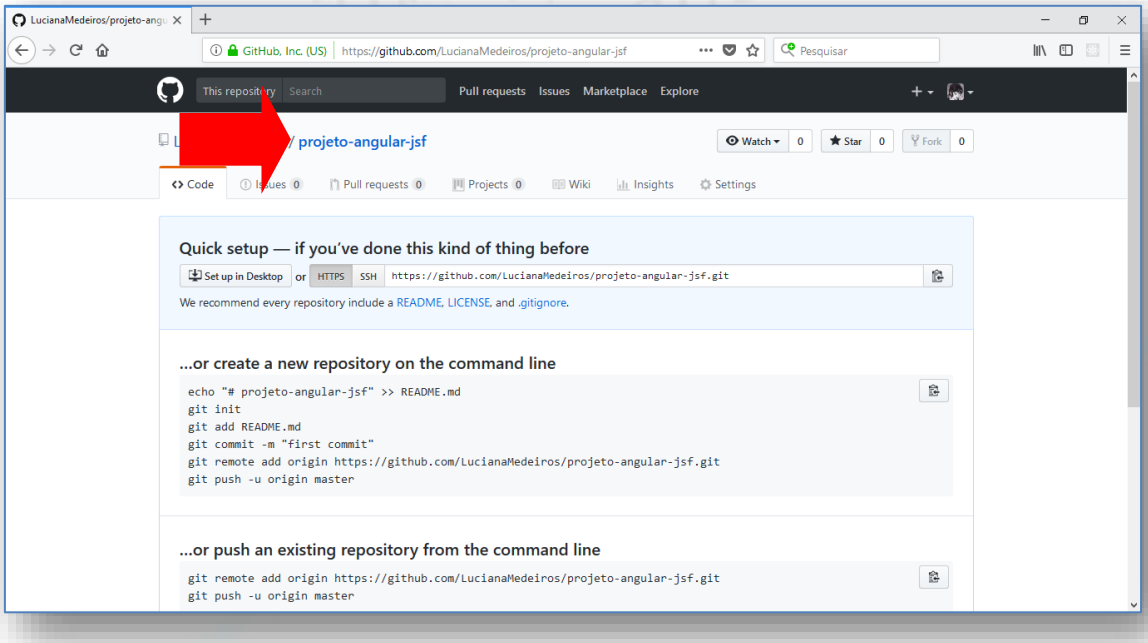
Ir no Eclipse e pegar o nome do projeto:



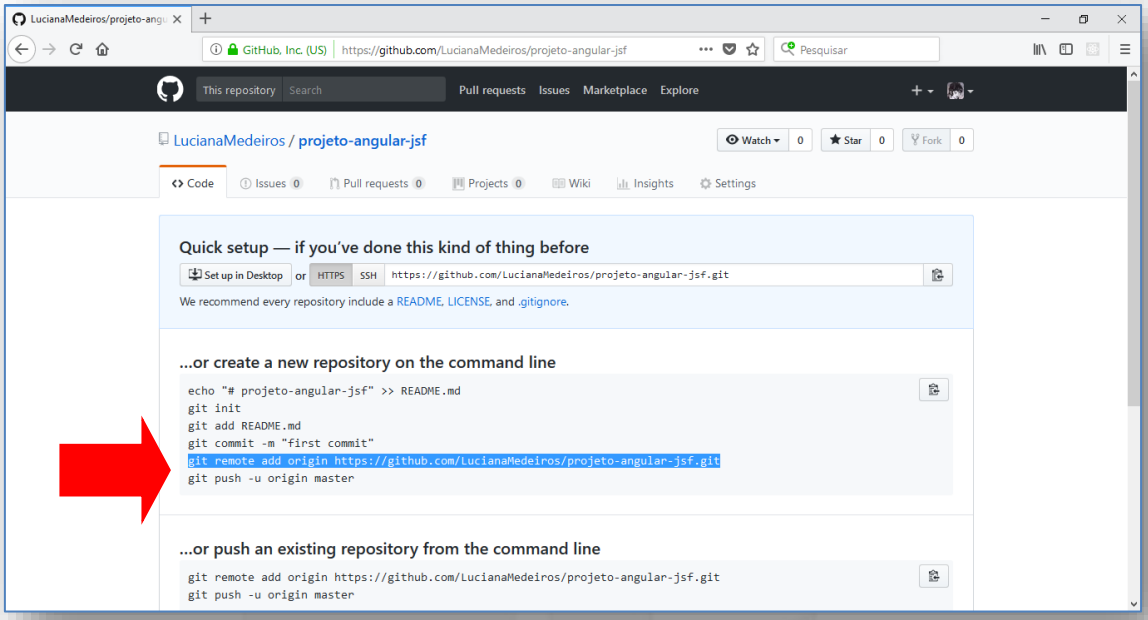
Criar um repositório (preferencialmente) com o mesmo nome do projeto -> Clicar em Create Repository...



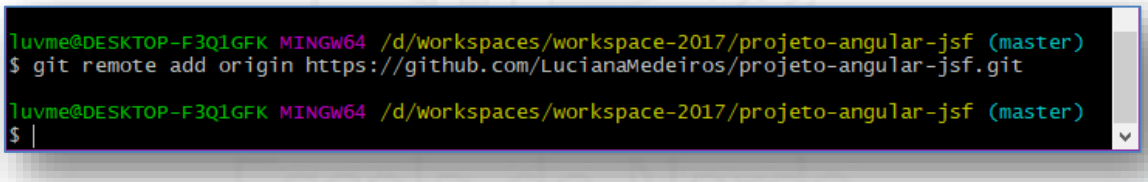
Repositorio criado...



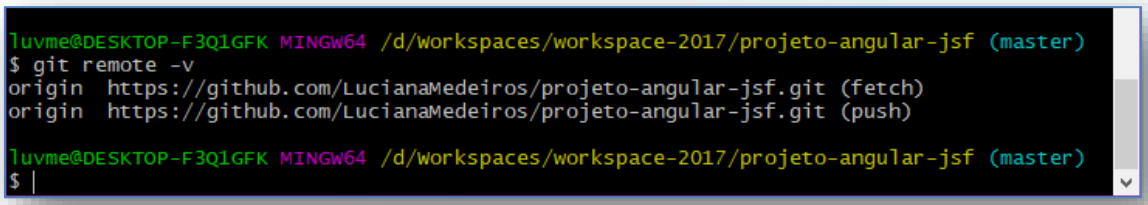
Copiar o link da conexão remota para adicionar ao git.



No gitbash digitar o comando: "git remote add origin https://github.com/LucianaMedeiros/projeto-angular-jsf.git"



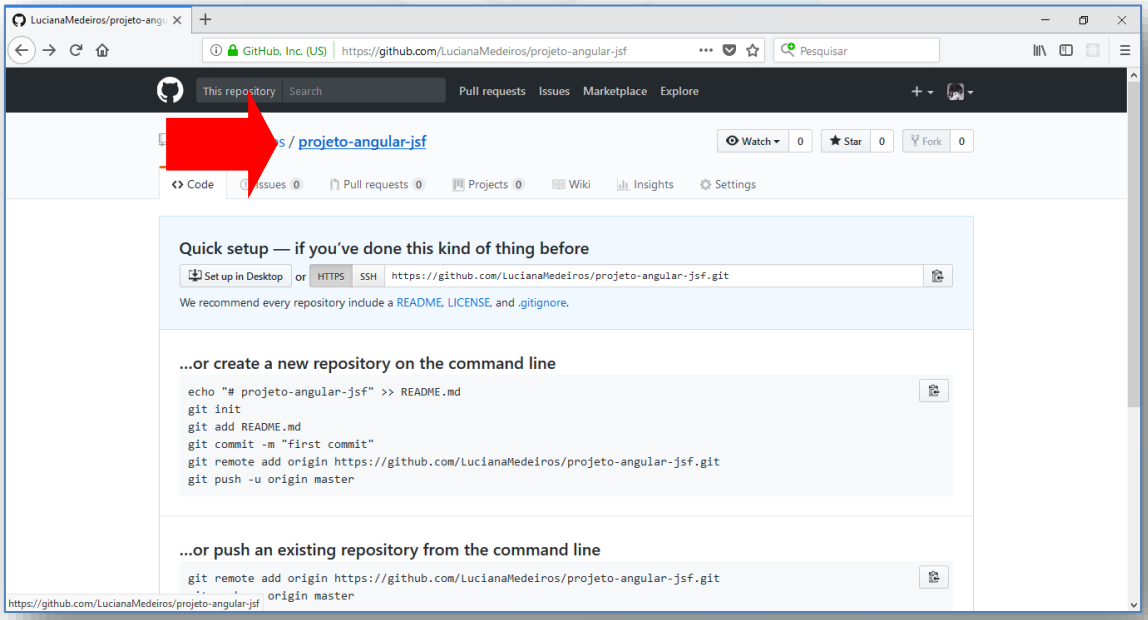
Para habilitar a transferência, digitar: "git remote -v"



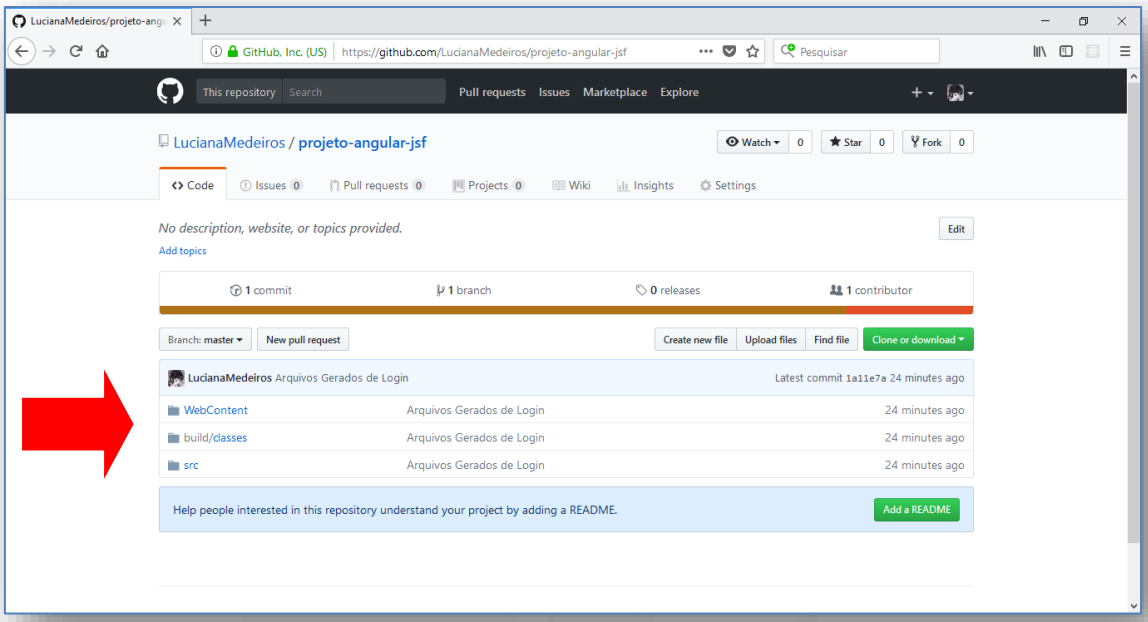
Para transferir os dados do git para o github, digitar: “git push -u origin master” e aguardar completar a transferência.

```
luvme@DESKTOP-F3Q1GFK MINGW64 /d/Workspaces/workspace-2017/projeto-angular-jsf (master)
$ git push -u origin master
Counting objects: 75, done.
Delta compression using up to 4 threads.
Compressing objects: 100% (69/69), done.
Writing objects: 100% (75/75), 14.80 MiB | 101.00 KiB/s, done.
Total 75 (delta 0), reused 0 (delta 0)
To https://github.com/LucianaMedeiros/projeto-angular-jsf.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.

luvme@DESKTOP-F3Q1GFK MINGW64 /d/Workspaces/workspace-2017/projeto-angular-jsf (master)
$
```

No github, clicar no link do projeto...



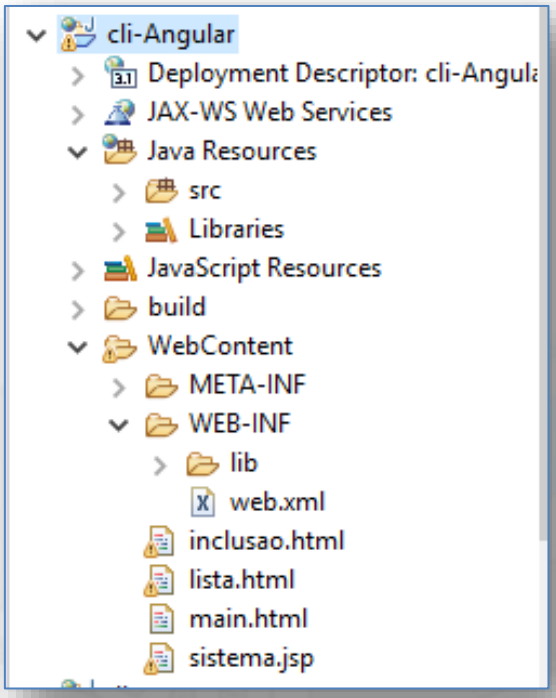
Projeto no github...



Se houver alterações no projeto. Digitar o comando do git e escrever uma mensagem de atualização...
git commit -m "Commit2"

PROJETO CLIENT:

Estrutura do projeto:



LIBS:

servlet3-api.jar

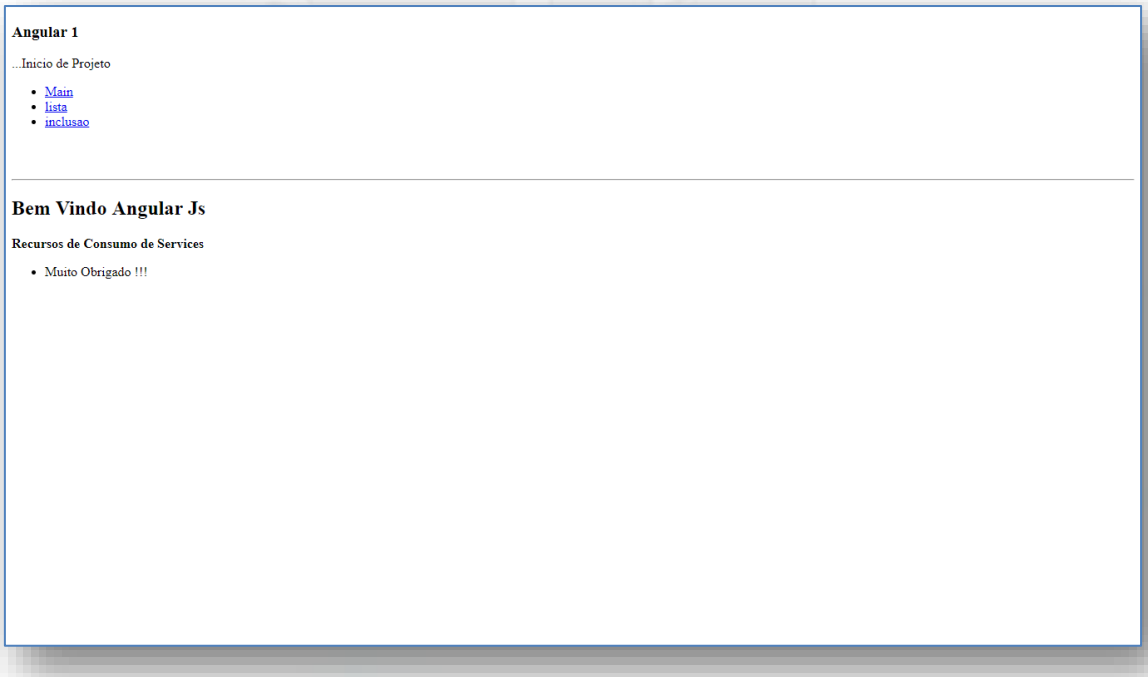
---

sistema.jsp

```
<!DOCTYPE HTML>
<html ng-app="app">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.4/angular.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.6.4/angular-route.js"></script>
</head>
<body>
```

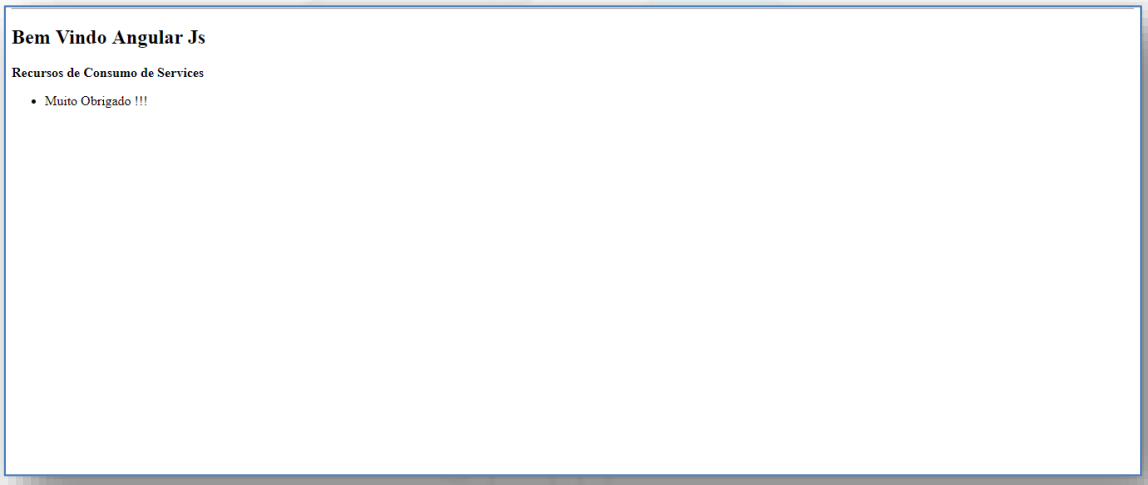
```
 <h3>Angular 1</h3>
 <p>...Inicio de Projeto</p>

 <ul>
   <li><a href="#/!">Main</a></li>
   <li><a href="#!lista">lista</a></li>
   <li><a href="#!inclusao">inclusao</a></li>
 </ul>

 <br/><br/>
 <hr/>
 <p/>
 <div ng-view>
 </div>


</body>
<script>
   var app = angular.module('app',["ngRoute"]);
    app.config(function ($routeProvider){
                        $routeProvider
      .when("/",{
          templateUrl : "main.html",
      })
      .when("/lista",{
          templateUrl : "lista.html",
          controller : 'listactl'
      }).when("/inclusao",{
          templateUrl : "inclusao.html",
          controller : 'inclusaoctl'
      });
    });

app.controller("listactl", function($scope, $http){
   $scope.buscar = function(){
    $http.get(
"http://localhost:8080/projeto-angular-jsf/json/usuario/listar")
    .then(function(msg){
     $scope.listagem = msg.data;
  });
   }
});

app.controller("inclusaoctl", function($scope, $http){
```

```
    $scope.cadastrar = function(){
     $http.post(
"http://localhost:8080/projeto-angular-jsf/json/usuario/gravar"
,$scope.usuario)
     .then(function(msg){
      $scope.message = msg.data;
      $scope.usuario.login="";
      $scope.usuario.senha="";
    })
    .catch(function(msg){
          $scope.message = msg.data;
          $scope.usuario.login="";
            $scope.usuario.senha="";
    })
}
});
</script>
</html>
```



main.html

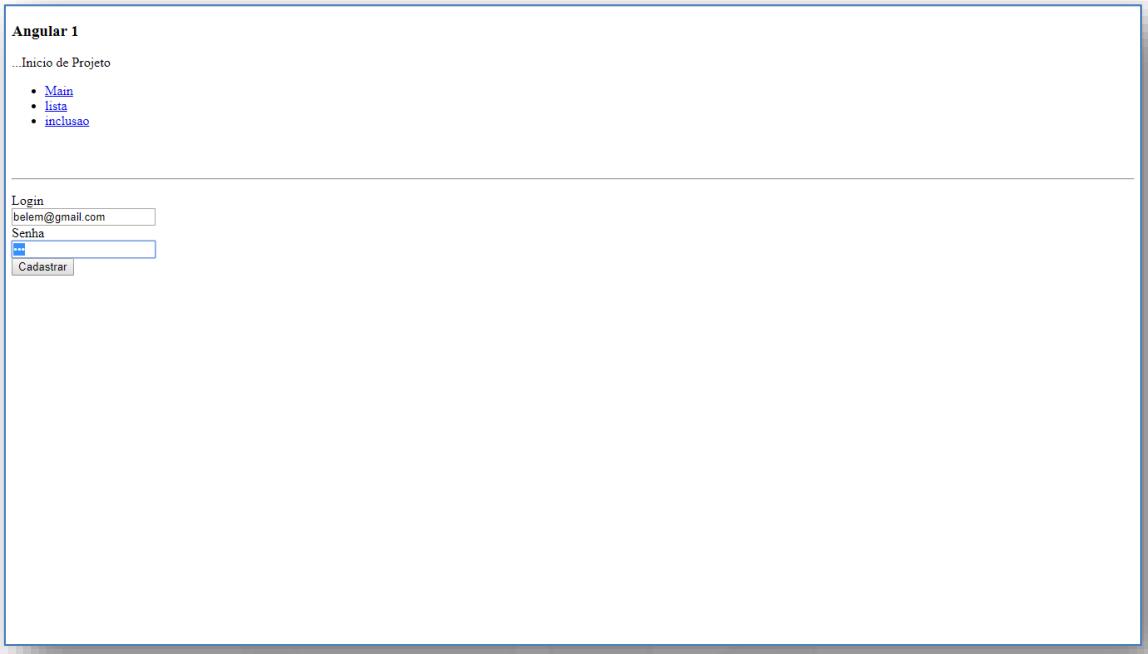
```
<h2>Bem Vindo Angular Js</h2>
<p/>

<b>Recursos de Consumo de Services</b>
<p/>
<ul>
 <li>Muito Obrigado !!!</li>
</ul>
```



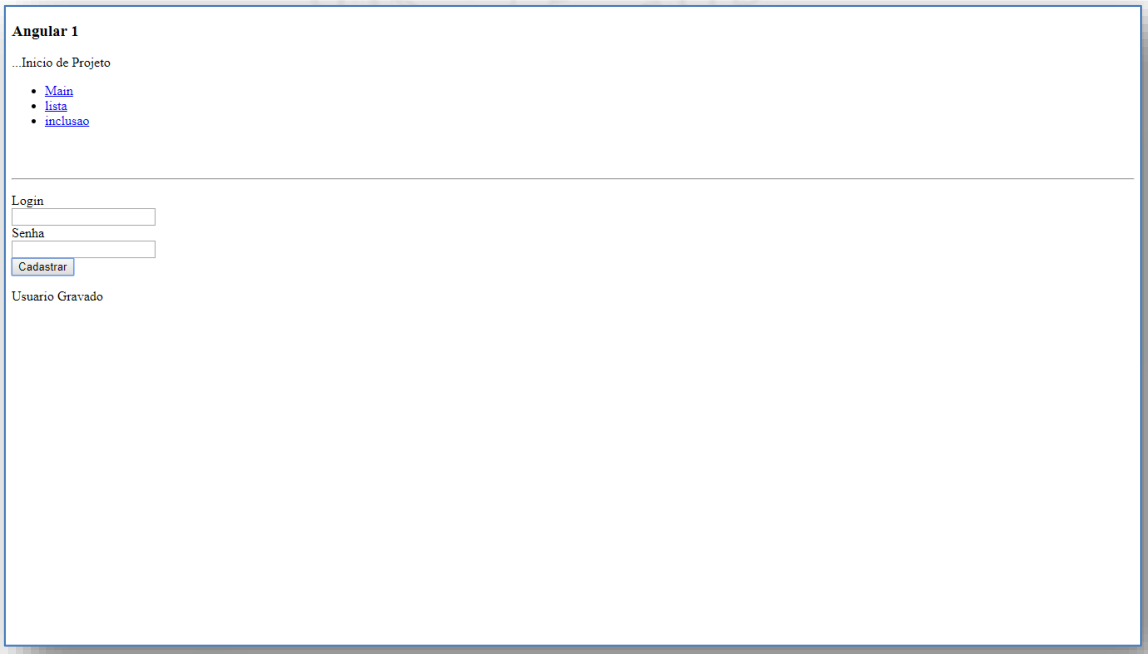
inclusao.html

```
 <form ng-controller="inclusaoctl">

    Login<br/>
   <input type="text" id="login" ng-model="usuario.login" />
   <br/>Senha<br/>
   <input type="password" id="senha" ng-model="usuario.senha" />
  <br/>

  <button type="button" ng-
click="cadastrar()">Cadastrar</button>
 <p/>
  {{message}}
```

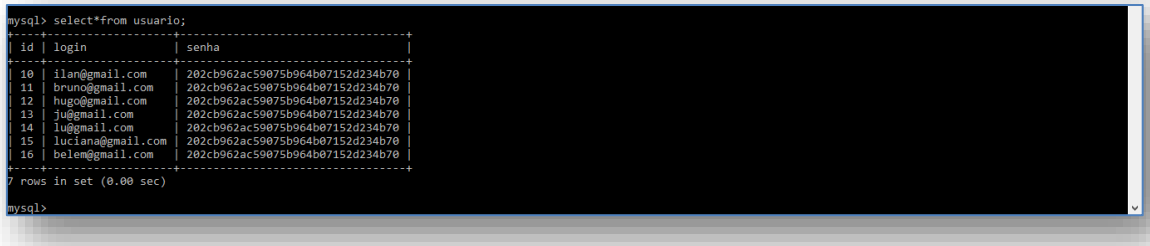
```
</form>
```



Cadastrado...



No banco...

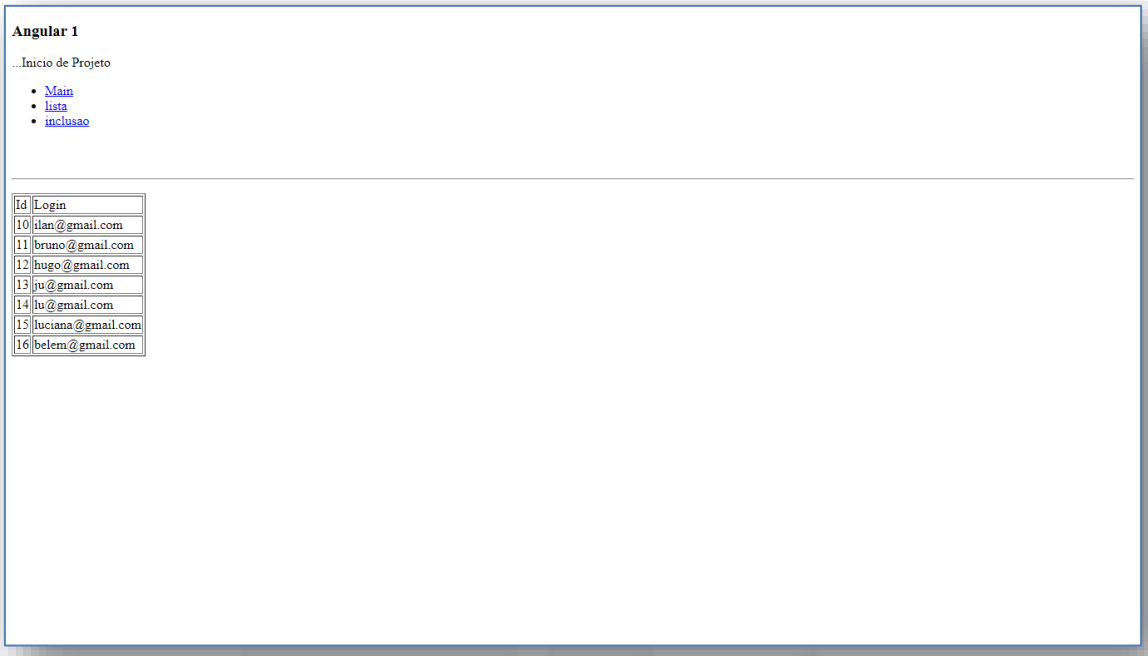


```
mysql> select*from usuario;
+----+------------------+----------------------------------+
| id | login            | senha                            |
+----+------------------+----------------------------------+
| 10 | ilan@gmail.com   | 202cb962ac59075b964b07152d234b70 |
| 11 | bruno@gmail.com  | 202cb962ac59075b964b07152d234b70 |
| 12 | hugo@gmail.com   | 202cb962ac59075b964b07152d234b70 |
| 13 | ju@gmail.com     | 202cb962ac59075b964b07152d234b70 |
| 14 | lu@gmail.com     | 202cb962ac59075b964b07152d234b70 |
| 15 | luciana@gmail.com | 202cb962ac59075b964b07152d234b70 |
| 16 | belem@gmail.com  | 202cb962ac59075b964b07152d234b70 |
+----+------------------+----------------------------------+
7 rows in set (0.00 sec)

mysql>
```

---

lista.html

```
<form ng-controller="listactl">

 <table border="1" ng-init="buscar()">
 <thead>
   <tr>
    <td>Id</td>
    <td>Login</td>
   </tr>
 </thead>
 <tbody>
  <tr ng-repeat="u in listagem">
      <td>{{u.id}}</td>
      <td>{{u.login}}</td>
   </tr>
 </tbody>

 </table>

</form>
```


Angular 1
...Inicio de Projeto
Main
lista
inclusao
Id Login
10 ilan@gmail.com
11 bruno@gmail.com
12 hugo@gmail.com
13 ju@gmail.com
14 lu@gmail.com
15 luciana@gmail.com
16 belem@gmail.com